เอกสารประกอบการสอน
รายวิชาการพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่

ชวลิต โควีระวงศ์

คณะวิทยาศาสตร์และเทคโนโลยี
มหาวิทยาลัยราชภัฏวไลยอลงกรณ์ ในพระบรมราชูปถัมภ์
2568

เอกสารประกอบการสอน
รายวิชาการพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่

ชวลิต โควีระวงศ์
M.Eng. (Information and communication technology for Embedded Systems)

คณะวิทยาศาสตร์และเทคโนโลยี
มหาวิทยาลัยราชภัฏวไลยอลงกรณ์ ในพระบรมราชูปถัมภ์
2568

คำนำ

เอกสารประกอบการสอน รายวิชาการพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ รหัส SCS337 ใช้ในการเรียนการสอนในภาคการศึกษาที่ 2 ปีการศึกษา 2568 ได้แบ่งเนื้อหาการเรียนการสอนออกเป็น 13 บท แต่ละบทใช้เวลาสอนประมาณ 1-2 สัปดาห์

รายวิชานี้มุ่งเน้นให้ผู้เรียนมีความรู้ความสามารถ ในการเข้าใจหลักการและกระบวนการพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่อย่างเป็นระบบ ทั้งรูปแบบออนไลน์และออฟไลน์ การใช้เทคโนโลยี React Native, Expo, Expo Router และ Supabase ในการพัฒนาแอปพลิเคชัน การออกแบบและพัฒนาส่วนติดต่อผู้ใช้ (UI) และประสบการณ์ผู้ใช้ (UX) การเขียนโปรแกรมเชิงฟังก์ชัน การจัดการข้อมูลในเครื่องและฐานข้อมูลคลาวด์ การทำระบบยืนยันตัวตน การอัปโหลดข้อมูล การทำงานร่วมกับพิกัดตำแหน่ง แผนที่ และเซนเซอร์อุปกรณ์ ตลอดจนการทดสอบ แก้ไขข้อผิดพลาด และการเผยแพร่แอปพลิเคชันบนแพลตฟอร์มมาตรฐานอย่างมีจริยธรรม

เอกสารประกอบการสอนนี้คงอำนวยประโยชน์ต่อการเรียนการสอนในรายวิชาการพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ ตามสมควร หากท่านที่นำเอกสารประกอบการสอนนี้ไปใช้ และมีข้อเสนอแนะผู้เขียนยินดีรับฟังและขอขอบคุณในความอนุเคราะห์นั้นมา ณ โอกาสนี้ด้วย

ชวลิต โควีระวงศ์

28 ตุลาคม 2568

สารบัญ

	หน้า
คำนำ.....	(1)
สารบัญ.....	(3)
สารบัญตาราง.....	(8)
สารบัญภาพ	(9)

แผนบริหารการสอนประจำวิชา ..... (11)
แผนบริหารการสอนประจำบทที่ 1 1
บทที่ 1 ความรู้เบื้องต้นเกี่ยวกับ React Native Expo และการเตรียมสภาพแวดล้อม 4
1.1 ทำความรู้จัก React Native และ Expo 4
1.2 การเปรียบเทียบการเลือกพัฒนาแอปพลิเคชัน 4
1.3 การเตรียมความพร้อมและติดตั้งสภาพแวดล้อม (Environment Setup) 5
1.4 การสร้างโปรเจคใหม่ด้วย create-expo-app 6
1.5 โครงสร้างของโปรเจค React Native Expo SDK 54 7
1.6 การรันโปรเจคและการทดสอบแอปพลิเคชัน 8
สรุป 9
แบบฝึกหัด 10
เอกสารอ้างอิง 21
แผนบริหารการสอนประจำบทที่ 2 22
บทที่ 2 การจัดเลย์เอาต์หน้าจอด้วย Flexbox 24
2.1 การเขียนและประกาศสไตล์ใน React Native 24
2.2 โครงสร้างกล่องและคุณสมบัติขนาดขอบเขต (Box Model) 25
2.3 ทิศทางการวางแกนด้วย flexDirection และ flex 25
2.4 การจัดกึ่งกลางและการจัดเรียงด้วย justifyContent และ alignItems 26
2.5 การขึ้นบรรทัดใหม่และการเว้นช่องว่าง (flexWrap & gap) 27
2.6 โค้ดตัวอย่างหน้าเลย์เอาต์โครงสร้าง Grid จำลอง 27
สรุป 29
แบบฝึกหัด 30
เอกสารอ้างอิง 44
แผนบริหารการสอนประจำบทที่ 3 45
บทที่ 3 คอมโพเนนต์พื้นฐานและการจัดการปฏิสัมพันธ์กับผู้ใช้ 48
3.1 คอมโพเนนต์แสดงผลรูปภาพ Image (Local และ Network Assets) 48
3.2 คอมโพเนนต์รับข้อมูลข้อความ TextInput และการควบคุมคีย์บอร์ด 49
3.3 การสร้างพื้นที่เลื่อนดูข้อมูลด้วย ScrollView เปรียบเทียบกับ View 51
3.4 การจัดการเหตุการณ์การสัมผัสและการกดปุ่มด้วย Pressable 52
3.5 การหลีกเลี่ยงพื้นที่ขอบจอด้วย SafeAreaView 53
3.6 แนวคิดการสร้าง Custom Component และการกำหนด Props Interface ด้วย 54

TypeScript .....
สรุป..... 57
แบบฝึกหัด 59
เอกสารอ้างอิง 78

สารบัญ (ต่อ)

หน้า

แผนบริหารการสอนประจำบทที่ 4 79
บทที่ 4 การจัดการสถานะ พร็อพส์ และแบบฟอร์มรับข้อมูล 82
4.1 แนวคิดของสถานะ (State) ใน React Native และการใช้งาน useState Hook ... 82
4.2 ข้อแตกต่างและการทำงานร่วมกันระหว่าง State และ Props 83
4.3 การทำแบบฟอร์มด้วย Controlled Components ผ่าน TextInput 83
4.4 การแบ่งปันข้อมูลย้อนกลับด้วยหลักการ Lifting State Up 84
4.5 การจัดทำฟอร์มคำนวณค่าดัชนีมวลกาย (BMI Calculation Form) 86
4.6 การตรวจสอบความถูกต้องของฟอร์ม (Form Validation) และการแจ้งเตือน ข้อผิดพลาด (Error Feedback) 89
4.7 โค้ดตัวอย่างการประกอบฟอร์มลงทะเบียนพร้อมระบบตรวจสอบแบบเรียลไทม์ 90
สรุป 94
แบบฝึกหัด 95
เอกสารอ้างอิง 115
แผนบริหารการสอนประจำบทที่ 5 116
บทที่ 5 การแสดงผลรายการข้อมูลและประสิทธิภาพการทำงาน 119
5.1 ปัญหาประสิทธิภาพของ ScrollView กับข้อมูลขนาดใหญ่ 119
5.2 การแก้ไขด้วย FlatList: Virtualization และ Recycling 119
5.3 โครงสร้างและพร็อพส์หลักของ FlatList (data, renderItem, keyExtractor) 120
5.4 การดึงข้อมูลจาก REST API มาใช้ร่วมกับ FlatList 121
5.5 การปรับแต่งส่วนประกอบลิสต์ (Header, Footer, Item Separator, และ ListEmptyComponent) 123
5.6 ฟีเจอร์ Pull-to-Refresh และ Infinite Scroll (onEndReached) 123
5.7 การจัดกลุ่มข้อมูลด้วย SectionList 124
สรุป 125
แบบฝึกหัด 127
เอกสารอ้างอิง 143
แผนบริหารการสอนประจำบทที่ 6 144
บทที่ 6 ระบบนำทางและสถาปัตยกรรมการเปลี่ยนหน้าจอด้วย Expo Router 147
6.1 แนวคิดการจัดเส้นทางแบบอิงโครงสร้างไฟล์ (File-based Routing) 147
6.2 การสร้าง Layout และการจัดระเบียบโครงสร้างด้วย _layout.tsx 147

6.3 ระบบนำทางแบบสะสมประวัติ (Stack Navigation)..... 148
6.4 ระบบนำทางแบบแท็บบาร์ล่าง (Tab Navigation)..... 148
6.5 การเชื่อมโยงลิงก์และเปลี่ยนหน้าจอ (Link & Router Object)..... 149
6.6 เส้นทางนำทางแบบไดนามิกและการอ่านค่าพารามิเตอร์ (Dynamic Routes & useLocalSearchParams)..... 151
สรุป..... 152
แบบฝึกหัด 153
เอกสารอ้างอิง 169

สารบัญ (ต่อ)

หน้า

แผนบริหารการสอนประจำบทที่ 7 170
บทที่ 7 การตกแต่งอินเทอร์เฟซ ไอคอน และรูปภาพประสิทธิภาพสูง 173
7.1 ระบบพิกัดตำแหน่งสัมพัทธ์และสัมบูรณ์ (Relative and Absolute Positioning). 173
7.2 การใช้งาน Custom Fonts ด้วย expo-font 175
7.3 การใช้งานระบบเวกเตอร์ไอคอนด้วย @expo/vector-icons 176
7.4 การจัดการรูปภาพประสิทธิภาพสูงด้วย expo-image 178
7.5 การใช้งาน Blurhash Placeholder และ Transition 178
7.6 การรองรับโหมดมืด (Dark Mode / Theme Support) ด้วย useColorScheme 180
สรุป 181
แบบฝึกหัด 182
เอกสารอ้างอิง 196
แผนบริหารการสอนประจำบทที่ 8 197
บทที่ 8 การเก็บข้อมูลในเครื่องและการเชื่อมต่อเครือข่าย API 200
8.1 ความเข้าใจและการติดตั้ง AsyncStorage 200
8.2 การทำงานพื้นฐานกับ AsyncStorage (CRUD Operations) 201
8.3 การบันทึกและดึงข้อมูลออบเจกต์ซับซ้อน (JSON Serialization) 202
8.4 การดึงข้อมูลจาก API ด้วย Fetch (HTTP GET) 202
8.5 การส่งข้อมูลขึ้นเซิร์ฟเวอร์ด้วย Fetch (HTTP POST) 204
8.6 ระบบจัดการสถานะ Loading และการดักจับข้อผิดพลาด (Error Handling) 206
8.7 การประยุกต์ใช้ในการทำระบบจำเซสชันการล็อกอิน (Auth Token Caching) 206
สรุป 208
แบบฝึกหัด 209
เอกสารอ้างอิง 225
แผนบริหารการสอนประจำบทที่ 9 226
บทที่ 9 พิกัดตำแหน่ง แผนที่ และเซนเซอร์อุปกรณ์ 229
9.1 ระบบสิทธิ์การเข้าถึงอุปกรณ์ (Runtime Permissions) 229
9.2 การเข้าถึงพิกัดตำแหน่งอุปกรณ์ด้วย expo-location 230
9.3 การแปลงพิกัดกลับเป็นที่อยู่ (Reverse Geocoding) 231
9.4 การติดตั้งและจัดแสดงแผนที่ด้วย react-native-maps 232
9.5 การควบคุมแผนที่ (Region) และการใช้งาน Marker 232

9.6 การใช้งานเซนเซอร์ตรวจความเร่งด้วย expo-sensors (Accelerometer) 233
9.7 แนวทางการเขียนดักจับเหตุการณ์ปฏิเสธสิทธิ์และคืนค่าทรัพยากร (Permission Denied & Memory Cleanups) 235
สรุป 236
แบบฝึกหัด 237
เอกสารอ้างอิง 253
แผนบริหารการสอนประจำบทที่ 10 254
บทที่ 10 การใช้งานคลาวด์ดาต้าเบสด้วย Supabase 257
10.1 สถาปัตยกรรม BaaS และคลาวด์ดาต้าเบสของ Supabase 257
10.2 การติดตั้งและตั้งค่า Supabase Client ใน Expo (SDK 54) 258
10.3 การอ่านข้อมูลจากตาราง (Read / Select Data) 258
10.4 การเพิ่มข้อมูลแถวใหม่ (Create / Insert Data) 259
10.5 การแก้ไขและปรับปรุงข้อมูล (Update Data) 260
10.6 การลบข้อมูลออกจากตาราง (Delete Data) 260
10.7 การจัดการสถานะการทำงานและประมวลผลข้อผิดพลาด (Loading & Error Handling) 261
สรุป 262
แบบฝึกหัด 263
เอกสารอ้างอิง 279
แผนบริหารการสอนประจำบทที่ 11 281
บทที่ 11 การใช้งานกล้อง อัลบั้มภาพ และระบบจัดเก็บไฟล์เนทีฟร่วมกับคลาวด์ 284
11.1 กลไกการจัดการสิทธิ์กล้องถ่ายรูปและคลังภาพ (Camera & Media Permissions) 284
11.2 การใช้งาน CameraView เพื่อพรีวิวภาพสด (Camera Live Preview) 285
11.3 การลั่นชัตเตอร์ถ่ายภาพนิ่ง (Taking Photos with takePictureAsync) 286
11.4 การเลือกรูปภาพและสื่อผ่าน expo-image-picker (Image Library API) 287
11.5 ระบบจัดการไฟล์แบบที่ 1: Private Sandbox Storage (แอปมองเห็นเฉพาะตัว) 288
11.6 ระบบจัดการไฟล์แบบที่ 2: Shared Media Storage (แอปอื่นมองเห็นและเข้าถึงข้อมูลร่วมกันได้) 289
11.7 ระบบจัดการไฟล์แบบที่ 3: Cloud Storage Network (อัปโหลด/ดาวน์โหลดผ่าน Supabase Storage) 290

สรุป 292
แบบฝึกหัด 293
เอกสารอ้างอิง 314
แผนบริหารการสอนประจำบทที่ 12 316
บทที่ 12 ระบบยืนยันตัวตนและการเข้าถึงข้อมูลด้วย Supabase Authentication 319
12.1 แนวคิดระบบยืนยันตัวตนและการจัดการสิทธิ์ (Authentication vs Authorization) 319
12.2 สถาปัตยกรรมระบบของ Supabase Auth (GoTrue Service) 319
12.3 การสร้างฟังก์ชันสมัครสมาชิกและเข้าสู่ระบบ (Sign Up & Sign In) 320
12.4 การจัดการเซสชันผู้ใช้และตัวดักจับสถานะ (Session State & onAuthStateChange) 321
12.5 การรักษาความปลอดภัยระดับอุปกรณ์ด้วย expo-secure-store 322
12.6 การรีแฟกเตอร์ไปใช้ useContext และ AuthContext 323
สรุป 324
แบบฝึกหัด 325
เอกสารอ้างอิง 344
แผนบริหารการสอนประจำบทที่ 13 346
บทที่ 13 การดีบั๊ก การสร้างแอปพลิเคชัน และการเผยแพร่ด้วย EAS Build 349
13.1 เทคนิคการตรวจสอบและแก้ไขข้อผิดพลาดใน Expo SDK 54 (Debugging) 349
13.2 สถาปัตยกรรมและหลักการของ Expo Application Services (EAS) 350
13.3 การจัดเตรียมความพร้อมแอปพลิเคชันใน app.json 350
13.4 การตั้งค่าโปรไฟล์การคอมไพล์ใน eas.json 351
13.5 ขั้นตอนการสั่งรันคอมไพล์ด้วยคำสั่ง EAS CLI 352
13.6 แนวทางและกระบวนการเผยแพร่แอปขึ้น App Stores 353
13.7 ขั้นตอนการอัปเกรดเวอร์ชันของ Expo SDK (Upgrading Expo SDK) 353
สรุป 355
แบบฝึกหัด 356
เอกสารอ้างอิง 372
บรรณานุกรม 374
ภาคผนวก 379

สารบัญตาราง

ตารางที่		หน้า
1.1	การเปรียบเทียบแนวทางการพัฒนาแอปพลิเคชันด้วย React Native และ Expo	4
2.1	ค่าคงที่และทิศทางการจัดวางของ flexDirection	25
3.1	การเปรียบเทียบคุณสมบัติระหว่าง View และ ScrollView	52
4.1	การเปรียบเทียบคุณลักษณะระหว่าง State และ Props	83
11.1	สรุปเปรียบเทียบระบบจัดเก็บไฟล์ทั้ง 3 รูปแบบ	291

สารบัญภาพ

ภาพที่		หน้า
1.1	โครงสร้างโปรเจค React Native Expo SDK 54 และระบบไฟล์หลัก	7
2.1	แผนผังแกนหลัก (Main Axis) และแกนตัด (Cross Axis) ใน Flexbox	25
3.1	โครงสร้างลำดับชั้นของคอมโพเนนต์พื้นฐานและการส่งผ่าน Props	54
4.1	แผนผังการไหลของข้อมูลแบบทิศทางเดียวและการทำ Lifting State Up	84
5.1	กลไกการทำ Virtualization และ Windowing ของคอมโพเนนต์ FlatList	120
6.1	โครงสร้างไดเรกทอรี app/ และการจับคู่เส้นทางของ Expo Router	147
7.1	การจัดวางเลเยอร์แบบสัมบูรณ์ (Absolute Positioning) และการสลับธีม	173
8.1	วงจรการดึงข้อมูล REST API ร่วมกับการแคชด้วย AsyncStorage	202
9.1	สถาปัตยกรรมการขอสิทธิ์เข้าถึงพิกัดตำแหน่งและเซนเซอร์อุปกรณ์	229
10.1	สถาปัตยกรรม Backend-as-a-Service (BaaS) ของระบบฐานข้อมูล Supabase	257
11.1	แผนผังระบบจัดเก็บไฟล์ 3 รูปแบบ (Sandbox, Shared Media และ Cloud Storage)	288
12.1	กระบวนการยืนยันตัวตนด้วย JWT Token และการจัดเก็บผ่าน SecureStore	319
13.1	กระบวนการบิลด์และเผยแพร่แอปพลิเคชันบนคลาวด์ด้วย EAS Build	350

แผนบริหารการสอนประจำวิชา

รหัสวิชา SCS337
รายวิชา การพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ 3(2-2-5) หน่วยกิต
Mobile Application Development

คำอธิบายรายวิชา

ศึกษาเทคโนโลยีเพื่อใช้พัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ทั้งรูปแบบออนไลน์และออฟไลน์ ภาษาและซอฟต์แวร์ที่นิยมใช้ในปัจจุบัน และประยุกต์ใช้งานในการสร้างและออกแบบแอปพลิเคชัน การเขียนโปรแกรมเชิงฟังก์ชัน การทำระบบยืนยันตัวตน การอัปโหลดข้อมูล การจัดการข้อมูลบนอุปกรณ์เคลื่อนที่และเชื่อมต่อกับเซิร์ฟเวอร์ การทำงานเชื่อมต่อระหว่าง แอปพลิเคชันกับสื่อโซเชียลที่นิยมใช้ในปัจจุบันร่วมกับเซิร์ฟเวอร์ที่พัฒนาขึ้น เผยแพร่แอปพลิเคชันบนแพลตฟอร์มที่ได้รับความนิยมในปัจจุบัน

วัตถุประสงค์ทั่วไป

1. เข้าใจหลักการและกระบวนการพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่อย่างเป็นระบบ
2. ใช้เทคโนโลยี Expo, Expo Router และ Supabase ในการพัฒนาแอปพลิเคชันได้อย่างมีประสิทธิภาพ
3. ออกแบบและพัฒนา ส่วนติดต่อผู้ใช้ (UI) และ ประสบการณ์ผู้ใช้ (UX) ที่เหมาะสมกับอุปกรณ์เคลื่อนที่
4. เขียนโปรแกรมเชิงฟังก์ชันและจัดการข้อมูลระหว่างแอปพลิเคชันกับเซิร์ฟเวอร์หรือฐานข้อมูลได้
5. สร้างระบบยืนยันตัวตน การอัปโหลดข้อมูล และการเชื่อมต่อกับบริการออนไลน์หรือโซเชียลมีเดียได้
6. ทดสอบ แก้ไขข้อผิดพลาด และปรับปรุงประสิทธิภาพของแอปพลิเคชันให้เหมาะสมกับผู้ใช้จริง
7. เผยแพร่และจัดการแอปพลิเคชันบนแพลตฟอร์มที่ได้รับความนิยม เช่น Google Play และ App Store
8. ปฏิบัติงานอย่างมีจริยธรรม เคารพลิขสิทธิ์ และคำนึงถึงความปลอดภัยของข้อมูลผู้ใช้

เนื้อหาวิชา

บทที่ 1 **ความรู้เบื้องต้นเกี่ยวกับ React Native Expo และการเตรียมสภาพแวดล้อม** **4 ชั่วโมง**

1.1 ทำความรู้จัก React Native และ Expo

1.2 การเปรียบเทียบการเลือกพัฒนาแอปพลิเคชัน

1.3 การเตรียมความพร้อมและติดตั้งสภาพแวดล้อม (Environment Setup)

1.4 การสร้างโปรเจคใหม่ด้วย create-expo-app

1.5 โครงสร้างของโปรเจค React Native Expo SDK 54

1.6 การรันโปรเจคและการทดสอบแอปพลิเคชัน

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

บทที่ 2 **การจัดเลย์เอาต์หน้าจอด้วย Flexbox** **4 ชั่วโมง**

2.1 การเขียนและประกาศสไตล์ใน React Native

2.2 โครงสร้างกล่องและคุณสมบัติขนาดขอบเขต (Box Model)

2.3 ทิศทางการวางแกนด้วย flexDirection และ flex

2.4 การจัดกึ่งกลางและการจัดเรียงด้วย justifyContent และ alignItems

2.5 การขึ้นบรรทัดใหม่และการเว้นช่องว่าง (flexWrap & gap)

2.6 โค้ดตัวอย่างหน้าเลย์เอาต์โครงสร้าง Grid จำลอง

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

บทที่ 3 **คอมโพเนนต์พื้นฐานและการจัดการปฏิสัมพันธ์กับผู้ใช้** **4 ชั่วโมง**

3.1 คอมโพเนนต์แสดงผลรูปภาพ Image (Local และ Network Assets)

3.2 คอมโพเนนต์รับข้อมูลข้อความ TextInput และการควบคุมคีย์บอร์ด

3.3 การสร้างพื้นที่เลื่อนดูข้อมูลด้วย ScrollView เปรียบเทียบกับ

View
3.4 การจัดการเหตุการณ์การสัมผัสและการกดปุ่มด้วย Pressable
3.5 การหลีกเลี่ยงพื้นที่ขอบจอด้วย SafeAreaView
3.6 แนวคิดการสร้าง Custom Component และการกำหนด Props Interface ด้วย TypeScript
สรุป
แบบฝึกหัด
เอกสารอ้างอิง

บทที่ 4 **การจัดการสถานะ พร็อพส์ และแบบฟอร์มรับข้อมูล** **4 ชั่วโมง**

4.1 แนวคิดของสถานะ (State) ใน React Native และการใช้งาน useState Hook
4.2 ข้อแตกต่างและการทำงานร่วมกันระหว่าง State และ Props
4.3 การทำแบบฟอร์มด้วย Controlled Components ผ่าน TextInput
4.4 การแบ่งปันข้อมูลย้อนกลับด้วยหลักการ Lifting State Up
4.5 การจัดทำฟอร์มคำนวณค่าดัชนีมวลกาย (BMI Calculation Form)
4.6 การตรวจสอบความถูกต้องของฟอร์ม (Form Validation) และการแจ้งเตือนข้อผิดพลาด (Error Feedback)
4.7 โค้ดตัวอย่างการประกอบฟอร์มลงทะเบียนพร้อมระบบตรวจสอบแบบเรียลไทม์
สรุป
แบบฝึกหัด
เอกสารอ้างอิง

บทที่ 5 **การแสดงผลรายการข้อมูลและประสิทธิภาพการทำงาน** **4 ชั่วโมง**

5.1 ปัญหาประสิทธิภาพของ ScrollView กับข้อมูลขนาดใหญ่
5.2 การแก้ไขด้วย FlatList: Virtualization และ Recycling
5.3 โครงสร้างและพร็อพส์หลักของ FlatList (data, renderItem, keyExtractor)

5.4 การดึงข้อมูลจาก REST API มาใช้ร่วมกับ FlatList
5.5 การปรับแต่งส่วนประกอบลิสต์ (Header, Footer, Item Separator, และ ListEmptyComponent)
5.6 ฟีเจอร์ Pull-to-Refresh และ Infinite Scroll (onEndReached)
5.7 การจัดกลุ่มข้อมูลด้วย SectionList
สรุป
แบบฝึกหัด
เอกสารอ้างอิง

บทที่ 6 **ระบบนำทางและสถาปัตยกรรมการเปลี่ยนหน้าจอด้วย Expo Router** **4 ชั่วโมง**

6.1 แนวคิดการจัดเส้นทางแบบอิงโครงสร้างไฟล์ (File-based Routing)
6.2 การสร้าง Layout และการจัดระเบียบโครงสร้างด้วย _layout.tsx
6.3 ระบบนำทางแบบสะสมประวัติ (Stack Navigation)
6.4 ระบบนำทางแบบแท็บบาร์ล่าง (Tab Navigation)
6.5 การเชื่อมโยงลิงก์และเปลี่ยนหน้าจอ (Link & Router Object)
6.6 เส้นทางนำทางแบบไดนามิกและการอ่านค่าพารามิเตอร์ (Dynamic Routes & useLocalSearchParams)
สรุป
แบบฝึกหัด
เอกสารอ้างอิง

บทที่ 7 **การตกแต่งอินเทอร์เฟซ ไอคอน และรูปภาพประสิทธิภาพสูง** **4 ชั่วโมง**

7.1 ระบบพิกัดตำแหน่งสัมพัทธ์และสัมบูรณ์ (Relative and Absolute Positioning)
7.2 การใช้งาน Custom Fonts ด้วย expo-font
7.3 การใช้งานระบบเวกเตอร์ไอคอนด้วย @expo/vector-icons
7.4 การจัดการรูปภาพประสิทธิภาพสูงด้วย expo-image
7.5 การใช้งาน Blurhash Placeholder และ Transition

7.6 การรองรับโหมดมืด (Dark Mode / Theme Support) ด้วย useColorScheme

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

บทที่ 8	**การเก็บข้อมูลในเครื่องและการเชื่อมต่อเครือข่าย API** 8.1 ความเข้าใจและการติดตั้ง AsyncStorage 8.2 การทำงานพื้นฐานกับ AsyncStorage (CRUD Operations) 8.3 การบันทึกและดึงข้อมูลออบเจกต์ซับซ้อน (JSON Serialization) 8.4 การดึงข้อมูลจาก API ด้วย Fetch (HTTP GET) 8.5 การส่งข้อมูลขึ้นเซิร์ฟเวอร์ด้วย Fetch (HTTP POST) 8.6 ระบบจัดการสถานะ Loading และการดักจับข้อผิดพลาด (Error Handling) 8.7 การประยุกต์ใช้ในการทำระบบจำเซสชันการล็อกอิน (Auth Token Caching) สรุป แบบฝึกหัด เอกสารอ้างอิง	**4 ชั่วโมง**
บทที่ 9	**พิกัดตำแหน่ง แผนที่ และเซนเซอร์อุปกรณ์** 9.1 ระบบสิทธิ์การเข้าถึงอุปกรณ์ (Runtime Permissions) 9.2 การเข้าถึงพิกัดตำแหน่งอุปกรณ์ด้วย expo-location 9.3 การแปลงพิกัดกลับเป็นที่อยู่ (Reverse Geocoding) 9.4 การติดตั้งและจัดแสดงแผนที่ด้วย react-native-maps 9.5 การควบคุมแผนที่ (Region) และการใช้งาน Marker 9.6 การใช้งานเซนเซอร์ตรวจความเร่งด้วย expo-sensors (Accelerometer) 9.7 แนวทางการเขียนดักจับเหตุการณ์ปฏิเสธสิทธิ์และคืนค่าทรัพยากร (Permission Denied & Memory Cleanups) สรุป	**4 ชั่วโมง**

แบบฝึกหัด
เอกสารอ้างอิง

บทที่ 10	**การใช้งานคลาวด์ดาต้าเบสด้วย Supabase** 10.1 สถาปัตยกรรม BaaS และคลาวด์ดาต้าเบสของ Supabase 10.2 การติดตั้งและตั้งค่า Supabase Client ใน Expo (SDK 54) 10.3 การอ่านข้อมูลจากตาราง (Read / Select Data) 10.4 การเพิ่มข้อมูลแถวใหม่ (Create / Insert Data) 10.5 การแก้ไขและปรับปรุงข้อมูล (Update Data) 10.6 การลบข้อมูลออกจากตาราง (Delete Data) 10.7 การจัดการสถานะการทำงานและประมวลผลข้อผิดพลาด (Loading & Error Handling) สรุป แบบฝึกหัด เอกสารอ้างอิง	**4 ชั่วโมง**
บทที่ 11	**การใช้งานกล้อง อัลบั้มภาพ และระบบจัดเก็บไฟล์เนทีฟร่วมกับคลาวด์** 11.1 กลไกการจัดการสิทธิ์กล้องถ่ายรูปและคลังภาพ (Camera & Media Permissions) 11.2 การใช้งาน CameraView เพื่อพรีวิวภาพสด (Camera Live Preview) 11.3 การลั่นชัตเตอร์ถ่ายภาพนิ่ง (Taking Photos with takePictureAsync) 11.4 การเลือกรูปภาพและสื่อผ่าน expo-image-picker (Image Library API) 11.5 ระบบจัดการไฟล์แบบที่ 1: Private Sandbox Storage (แอปมองเห็นเฉพาะตัว) 11.6 ระบบจัดการไฟล์แบบที่ 2: Shared Media Storage (แอปอื่นมองเห็นและเข้าถึงข้อมูลร่วมกันได้) 11.7 ระบบจัดการไฟล์แบบที่ 3: Cloud Storage Network	**4 ชั่วโมง**

(อัปโหลด/ดาวน์โหลดผ่าน Supabase Storage)
สรุป
แบบฝึกหัด
เอกสารอ้างอิง

บทที่ 12	**ระบบยืนยันตัวตนและการเข้าถึงข้อมูลด้วย Supabase Authentication** 12.1 แนวคิดระบบยืนยันตัวตนและการจัดการสิทธิ์ (Authentication vs Authorization) 12.2 สถาปัตยกรรมระบบของ Supabase Auth (GoTrue Service) 12.3 การสร้างฟังก์ชันสมัครสมาชิกและเข้าสู่ระบบ (Sign Up & Sign In) 12.4 การจัดการเซสชันผู้ใช้และตัวดักจับสถานะ (Session State & onAuthStateChange) 12.5 การรักษาความปลอดภัยระดับอุปกรณ์ด้วย expo-secure-store 12.6 การรีแฟกเตอร์ไปใช้ useContext และ AuthContext สรุป แบบฝึกหัด เอกสารอ้างอิง	**4 ชั่วโมง**
บทที่ 13	**การดีบั๊ก การสร้างแอปพลิเคชัน และการเผยแพร่ด้วย EAS Build** 13.1 เทคนิคการตรวจสอบและแก้ไขข้อผิดพลาดใน Expo SDK 54 (Debugging) 13.2 สถาปัตยกรรมและหลักการของ Expo Application Services (EAS) 13.3 การจัดเตรียมความพร้อมแอปพลิเคชันใน app.json 13.4 การตั้งค่าโปรไฟล์การคอมไพล์ใน eas.json 13.5 ขั้นตอนการสั่งรันคอมไพล์ด้วยคำสั่ง EAS CLI 13.6 แนวทางและกระบวนการเผยแพร่แอปขึ้น App Stores 13.7 ขั้นตอนการอัปเกรดเวอร์ชันของ Expo SDK (Upgrading Expo	**4 ชั่วโมง**

SDK)
สรุป
แบบฝึกหัด
เอกสารอ้างอิง

วิธีสอนและกิจกรรม

1. การบรรยายและสาธิต (Lecture & Demonstration) อธิบายหลักการทำงานและสาธิตการเขียนโปรแกรมแบบ Step-by-Step
2. การเรียนรู้แบบลงมือปฏิบัติ (Hands-on Practice / Learning by Doing) ปฏิบัติการเขียนโค้ดจริงผ่านใบงานท้ายบทเรียน
3. การทำงานกลุ่มและโครงงาน (Group Work & Project-based Learning) พัฒนา Mini-Project ประยุกต์ใช้งานจริง
4. การนำเสนอและแลกเปลี่ยนความคิดเห็น (Presentation & Peer Review) นำเสนอแนวคิด ผลงาน และอภิปรายร่วมกัน
5. การเรียนรู้แบบผสมผสาน (Blended Learning) เสริมการเรียนรู้ผ่านระบบคลาวด์ GitHub และ Google Classroom

สื่อการเรียนการสอน

1. เอกสารประกอบการสอนรายวิชา การพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ (รหัสวิชา SCS337)
2. สไลด์นำเสนอเนื้อหาประจำบทเรียน (Presentation Slides) และคู่มือปฏิบัติการ (Lab Sheets)
3. ซอฟต์แวร์และเครื่องมือพัฒนา ได้แก่ Node.js, Visual Studio Code, Expo SDK 54, Git และแอปพลิเคชัน Expo Go
4. ระบบฐานข้อมูลและบริการคลาวด์ Supabase (Database, Authentication และ Storage)
5. ระบบจัดการเรียนการสอนออนไลน์ (Google Classroom) และคลังโค้ดตัวอย่างบน GitHub Repository

การวัดผลและการประเมินผล

1. การวัดผล

1.1 คะแนนระหว่างภาคเรียน รวม 60%

1.1.1 งานปฏิบัติการและแบบฝึกหัดท้ายบทเรียน 20%

1.1.2 การสอบกลางภาคเรียน 20%

1.1.3 รายงานและโครงงานพัฒนาระบบ (Mini-Project) 10%

1.1.4 การมีส่วนร่วม กิจกรรมในชั้นเรียน และจิตพิสัย 10%

1.2 คะแนนสอบปลายภาค 40%

2. การประเมินผล

คะแนนระหว่าง 80 – 100 ได้ระดับคะแนน A

คะแนนระหว่าง 75 – 79 ได้ระดับคะแนน B^+

คะแนนระหว่าง 70 – 74 ได้ระดับคะแนน B

คะแนนระหว่าง 65 – 69 ได้ระดับคะแนน C^+

คะแนนระหว่าง 60 – 64 ได้ระดับคะแนน C

คะแนนระหว่าง 55 – 59 ได้ระดับคะแนน D^+

คะแนนระหว่าง 50 – 54 ได้ระดับคะแนน D

คะแนนระหว่าง 0 – 49 ได้ระดับคะแนน F

แผนบริหารการสอนประจำบทที่ 1

หัวข้อเนื้อหาประจำบท

1.1 ทำความรู้จัก React Native และ Expo

1.2 การเปรียบเทียบการเลือกพัฒนาแอปพลิเคชัน

1.3 การเตรียมความพร้อมและติดตั้งสภาพแวดล้อม (Environment Setup)

1.4 การสร้างโปรเจคใหม่ด้วย create-expo-app

1.5 โครงสร้างของโปรเจค React Native Expo SDK 54

1.6 การรันโปรเจคและการทดสอบแอปพลิเคชัน

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถอธิบายสถาปัตยกรรม ความแตกต่าง และการเลือกใช้งานเครื่องมือระหว่าง Expo Go และ Development Builds ได้อย่างถูกต้อง
2. ผู้เรียนสามารถเตรียมความพร้อม ติดตั้งโปรแกรมพื้นฐาน และกำหนดค่าสภาพแวดล้อมสำหรับการพัฒนาแอปพลิเคชันได้สำเร็จโดยไม่มีข้อผิดพลาด
3. ผู้เรียนสามารถใช้คำสั่งสร้างโปรเจค React Native Expo SDK 54 และทำความเข้าใจเกี่ยวกับบทบาทของไฟล์การตั้งค่าที่สำคัญได้อย่างถูกต้อง
4. ผู้เรียนสามารถรันแอปพลิเคชันแรก (Hello World) และแสดงผลบนเครื่องมือทดสอบ เช่น Expo Go หรือ Simulator ได้อย่างสมบูรณ์

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

1. **การบรรยายประกอบสไลด์ (Lecture):** แนะนำเนื้อหาทฤษฎี ความเป็นมาของ React Native ข้อดีของ Expo SDK 54 ตลอดจนเปรียบเทียบความคุ้มค่าของการพัฒนาในรูปแบบต่าง ๆ
2. **การสาธิตแบบมีส่วนร่วม (Interactive Demonstration):** ผู้สอนสาธิตขั้นตอนการติดตั้งซอฟต์แวร์ที่จำเป็นทั้งหมดทีละขั้นตอน และเปิดโอกาสให้ผู้เรียนติดตั้งตามไปพร้อม ๆ กัน

3. **การลงมือปฏิบัติ (Hands-on Practice):** ผู้เรียนปฏิบัติตามคู่มือการสร้างโปรเจคใหม่ และทดสอบการแก้ไขโค้ดเบื้องต้นเพื่อสังเกตผลกระทบแบบ Fast Refresh
4. **การระดมสมองและถามตอบ (Q&A & Brainstorming):** เปิดโอกาสให้ผู้เรียนสอบถามปัญหาที่พบขณะรันแอปพลิเคชัน และร่วมกันวิเคราะห์วิธีแก้ไขข้อผิดพลาดของการตั้งค่าเบื้องต้น

สื่อการเรียนการสอน

1. เอกสารสไลด์นำเสนอเรื่อง "Introduction to React Native Expo SDK 54"
2. คู่มือขั้นตอนการติดตั้งซอฟต์แวร์และเครื่องมือจำลองระบบปฏิบัติการ (Android Studio & Xcode)
3. วิดีโอสาธิตการทำงานเบื้องต้นและการรันแอปพลิเคชันผ่าน Expo Go บนสมาร์ทโฟน
4. โปรเจคตัวอย่างเริ่มต้นที่จัดเตรียมไว้บนแพลตฟอร์ม Git Repository

การวัดผลและการประเมินผล

1. การวัดผล

1.1 วัดความรู้ความเข้าใจจากความถูกต้องในการตอบคำถามท้ายบทเรียน (Multiple Choice Quiz)

1.2 วัดผลสัมฤทธิ์การปฏิบัติงานจากการสร้างโปรเจคและการนำเสนอหน้าจอแอปพลิเคชันแรกบนอุปกรณ์จริง/ตัวจำลองในชั่วโมงเรียน

1.3 ตรวจสอบความถูกต้องของการแก้ไขและการบันทึกไฟล์โค้ดเริ่มต้นเพื่อแสดงผลข้อความตามโจทย์กำหนด

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถรันโปรเจคเริ่มต้นและแสดงผลหน้าจอ "Hello World" บนอุปกรณ์ทดสอบได้ผ่านเกณฑ์ (ผ่าน/ไม่ผ่าน)

2.3 การสังเกตพฤติกรรมและการมีส่วนร่วมในกิจกรรมตอบคำถามและการลงมือ

แก้ปัญหาในห้องเรียน

บทที่ 1

ความรู้เบื้องต้นเกี่ยวกับ React Native Expo และการเตรียมสภาพแวดล้อม

ยินดีต้อนรับสู่หลักสูตรพัฒนาแอปพลิเคชันมือถือด้วย **React Native และ Expo SDK 54** ในบทนี้เราจะเรียนรู้แนวคิดพื้นฐาน ติดตั้งเครื่องมือ และเริ่มต้นรันแอปพลิเคชันแรกของคุณ

1.1 ทำความรู้จัก React Native และ Expo

React Native คืออะไร?

React Native คือโอเพนซอร์สเฟรมเวิร์กที่พัฒนาโดย Meta (Facebook) ช่วยให้ผู้พัฒนาสามารถสร้างแอปพลิเคชันสำหรับ iOS และ Android ได้พร้อมกันด้วยการเขียนโค้ดภาษา JavaScript และเฟรมเวิร์ก React แทนที่จะต้องแยกกันเขียนด้วย Swift/Objective-C (iOS) และ Kotlin/Java (Android) (React Native, 2024)

• **Native Performance:** แตกต่างจาก Hybrid App ยุคเก่าที่รันผ่าน Web View เพราะ React Native จะแปลงคอมโพเนนต์ JavaScript ไปเป็นคอมโพเนนต์ Native จริง ๆ ของอุปกรณ์ ทำให้ได้ประสิทธิภาพที่ไหลลื่นและรูปลักษณ์ที่เป็นธรรมชาติ

• **Cross-Platform:** เขียนโค้ดชุดเดียว (Single Codebase) สามารถแชร์การทำงานได้มากกว่า 80-90% ระหว่างแพลตฟอร์ม

Expo คืออะไร?

Expo คือชุดเครื่องมือ (Suite of Tools) และบริการเสริมที่ครอบทับอยู่บน React Native อีกชั้นหนึ่ง เพื่อช่วยลดความซับซ้อนในการตั้งค่าและเขียนโค้ด Native โดยในปัจจุบัน Expo เป็นเครื่องมือมาตรฐานที่ทางทีมงาน React Native แนะนำอย่างเป็นทางการในการเริ่มต้นโครงการใหม่ (Expo, 2024)

หมายเหตุ (Note): Expo SDK 54 นำเสนอฟีเจอร์ความปลอดภัย ประสิทธิภาพที่สูงขึ้น การใช้ React 19 และมาพร้อมกับโครงสร้างการจัดเส้นทางโฟลเดอร์แบบไฟล์ (File-based Routing) เวอร์ชันล่าสุดผ่าน **Expo Router v6** (Expo, 2024)

1.2 การเปรียบเทียบการเลือกพัฒนาแอปพลิเคชัน

ก่อนเริ่มต้น เรามาทำความเข้าใจแนวทางในการรันแอปพลิเคชัน 3 แบบหลัก ๆ:

ตารางที่ 1.1 การเปรียบเทียบแนวทางการพัฒนาแอปพลิเคชันด้วย React Native และ Expo

หัวข้อเปรียบเทียบ	Expo Go	Expo	React Native CLI

		Development Builds	
ความง่ายในการเริ่ม	ง่ายที่สุด (ติดตั้งแอปจาก Store)	ปานกลาง (ต้องติดตั้ง SDK ท้องถิ่น)	ยากที่สุด (ต้องตั้งค่า SDK และตัวคอมไพล์)
โค้ดภาษา Native	ใช้ได้เฉพาะคอมโพเนนต์มาตรฐานที่แถมมา	เขียนโค้ด Native (Swift/Kotlin) เสริมได้	เขียนโค้ด Native เสริมได้อย่างอิสระ
การสร้างและติดตั้ง	รันผ่านแอปแวดล้อมแชร์ร่วมกัน	สร้างไฟล์แอปเฉพาะกิจ (.apk / .ipa)	ต้องคอมไพล์ทุกครั้งที่ทดสอบโค้ด
เหมาะสำหรับ	ผู้เริ่มต้น, โปรเจคขนาดเล็ก, ทำ UI	โปรเจคจริงขนาดกลาง-ใหญ่, ใช้ปลั๊กอินภายนอก	โปรเจคขนาดใหญ่ที่ต้องการควบคุมระดับ OS

สำหรับหลักสูตรนี้ เราจะเริ่มต้นด้วย Expo Go เพื่อให้ผู้เรียนได้ทดลองเขียนและรันแอปบนอุปกรณ์จริงอย่างรวดเร็วที่สุด จากนั้นจะขยายไปสู่หัวข้อสร้างแอปพลิเคชันแบบกำหนดเองในบทหลัง

1.3 การเตรียมความพร้อมและติดตั้งสภาพแวดล้อม (Environment Setup)

เพื่อให้เครื่องคอมพิวเตอร์ของคุณพร้อมสำหรับการพัฒนาแอปพลิเคชัน ให้ติดตั้งโปรแกรมต่าง ๆ ตามขั้นตอนดังนี้:

ขั้นที่ 1: ติดตั้ง Node.js

Node.js เป็นโปรแกรมจำเป็นสำหรับรันสคริปต์ JavaScript และตัวติดตั้งแพ็กเกจ (npm)

1. ไปที่เว็บไซต์ Node.js Official Website (Node.js, 2024)
2. ดาวน์โหลดและติดตั้งเวอร์ชัน **LTS** (แนะนำเวอร์ชัน 20 หรือ 22 ขึ้นไป)
3. ตรวจสอบความสำเร็จของการติดตั้งใน Terminal/PowerShell:

```
node -v
npm -v
```

ขั้นที่ 2: ติดตั้ง Visual Studio Code (VS Code)

VS Code เป็นโปรแกรมเขียนโค้ดและแก้ไขข้อความยอดนิยม

1. ดาวน์โหลดที่ VS Code Website (Microsoft, 2024)

2. ติดตั้งส่วนขยาย (Extensions) ที่แนะนำสำหรับการเขียน React Native:

- **ES7+ React/Redux/React-Native snippets** (ช่วยคีย์ลัดสร้างโค้ด)
- **Prettier - Code formatter** (ช่วยจัดรูปแบบโค้ดอัตโนมัติ)
- **Expo Tools** (ช่วยตรวจสอบไฟล์ `app.json`)

ขั้นที่ 3: ติดตั้ง Git

ใช้สำหรับการจัดการเวอร์ชันโค้ด (Version Control) ดาวน์โหลดได้จาก Git SCM (Git, 2024)

ขั้นที่ 4: ติดตั้งแอปพลิเคชัน Expo Go บนสมาร์ทโฟน

1. **iOS:** ดาวน์โหลดแอปพลิเคชัน **Expo Go** จาก App Store
2. **Android:** ดาวน์โหลดแอปพลิเคชัน **Expo Go** จาก Google Play Store
3. ตรวจสอบให้มั่นใจว่าสมาร์ทโฟนและเครื่องคอมพิวเตอร์ที่ใช้เขียนโค้ด เชื่อมต่อวงเครือข่ายอินเทอร์เน็ต (Wi-Fi) วงเดียวกัน

1.4 การสร้างโปรเจคใหม่ด้วย create-expo-app

เปิด Terminal (ใน macOS/Linux) หรือ PowerShell (ใน Windows) และย้ายไดเรกทอรีไปยังตำแหน่งที่ต้องการสร้างโปรเจค จากนั้นรันคำสั่ง:

```
npx create-expo-app@latest my-first-app
```

- **`npx create-expo-app@latest`**: คำสั่งดาวน์โหลดสคริปต์สร้างแอปเวอร์ชันล่าสุด
- **`my-first-app`**: ชื่อโปรเจคและโฟลเดอร์ที่เราต้องการสร้าง

เมื่อการดาวน์โหลดเสร็จสมบูรณ์ ให้ใช้คำสั่งเปลี่ยนโฟลเดอร์เข้าไปในโปรเจค:

```
cd my-first-app
```

การรีเซ็ตโปรเจคเริ่มต้น (Reset Project Layout)

สำหรับเทมเพลตยุคใหม่ของ Expo ที่สร้างผ่านคำสั่งเริ่มต้น หากผู้เรียนไม่ต้องการหน้าจอตัวอย่างหรือ UI ที่ติดมากับระบบ (เช่น แถบ Tabs ด้านล่าง และหน้าจอ Explore) ทาง Expo มีสคริปต์ล้างโครงสร้างเพื่อให้คุณเริ่มต้นทำแอปจากศูนย์ได้อย่างรวดเร็ว (Expo, 2024):

```
npm run reset-project
```

ข้อควรจำ (Important): คำสั่งนี้จะทำการย้ายโค้ดเทมเพลตเริ่มต้นทั้งหมดไปเก็บไว้ที่โฟลเดอร์ `app-example/` เพื่อใช้ในการศึกษาภายหลัง และระบบจะทำการสร้างโฟลเดอร์ `app/`

ขึ้นมาใหม่พร้อมไฟล์ index.tsx แบบเปล่า ๆ เพื่อให้เราพร้อมเขียนหน้าจอแอปพลิเคชันจากศูนย์ทันที

1.5 โครงสร้างของโปรเจค React Native Expo SDK 54

เมื่อเราเปิดโฟลเดอร์โปรเจคด้วย VS Code จะพบโครงสร้างไฟล์หลัก ๆ ดังนี้:

```
my-first-app/
├── app/                    # โฟลเดอร์สำหรับเก็บโค้ดหน้าจอและการนำทาง (Expo Router)
│   ├── _layout.tsx         # เลย์เอาต์หลักครอบทุกหน้าจอ
│   └── index.tsx           # หน้าจอแรกสุดของแอปพลิเคชัน (Home Screen)
├── assets/                 # ที่เก็บรูปภาพ ฟอนต์ และสัญลักษณ์ของแอป
├── app.json                # ไฟล์การตั้งค่าคุณสมบัติแอปพลิเคชัน (App Metadata)
├── package.json            # รายชื่อ Library และชุดคำสั่งสคริปต์ในการรันแอป
├── tsconfig.json           # การตั้งค่าพฤติกรรมสำหรับภาษา TypeScript
└── node_modules/           # โฟลเดอร์ที่เก็บแพ็กเกจ Library ทั้งหมด (ห้ามแก้ไขโดยตรง)
```

รายละเอียดไฟล์ตั้งค่าที่สำคัญ:

1. `app.json`: ใช้กำหนดชื่อแอปพลิเคชัน โลโก้ (icon) ภาพตอนเข้าแอป (splash image) สิทธิ์การเข้าถึงอุปกรณ์ (permissions) และเวอร์ชันของแอป

2. `app/index.tsx`: จุดเริ่มต้นการเขียนหน้าตาแอปพลิเคชัน เช่น:

```tsx
import { StyleSheet, Text, View } from 'react-native';

export default function HomeScreen() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Hello, Expo SDK 54! 👋</Text>
      <Text style={styles.subtitle}>นี่คือแอปพลิเคชันแรกของฉัน</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a', // สีพื้นหลัง Slate เข้มสไตล์โมเดิร์น
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#f8fafc',
    marginBottom: 8,
  },
  subtitle: {
```

```
    fontSize: 16,
    color: '#94a3b8',
  },
});
```

1.6 การรันโปรเจคและการทดสอบแอปพลิเคชัน

ในโฟลเดอร์โปรเจค ให้รันคำสั่งสตาร์ทเซิร์ฟเวอร์นักพัฒนา:

```
npm start
```

หรือรันผ่านคำสั่งเฉพาะ:

```
npx expo start
```

ตัวจัดการการรัน (Expo CLI) จะเริ่มต้นเซิร์ฟเวอร์ **Metro Bundler** และแสดงหน้าอินเทอร์เฟซควบคุมขึ้นมา รวมถึงสร้าง **QR Code** บนหน้าจอคอนโซล (Expo, 2024):

```
› Metro waiting on exp://192.168.1.33:8081
› Scan the QR code above with Expo Go (Android) or the Camera app (iOS)

› Press a │ open on Android
› Press i │ open on iOS simulator
› Press w │ open on web
› Press r │ reload app
```

ขั้นตอนการดูผลลัพธ์บนอุปกรณ์ต่าง ๆ:

A. สำหรับสมาร์ทโฟนส่วนตัว (แนะนำ)

1. **ระบบ Android:** เปิดแอป **Expo Go** -> แตะเมนู "Scan QR Code" -> สแกนโค้ดบนคอมพิวเตอร์ของคุณ

2. **ระบบ iOS:** เปิดแอป **Camera** (กล้องมาตรฐาน) -> ส่อง QR Code -> แตะลิงก์เปิดด้วยแอป Expo Go

3. เมโทรบันด์เลอร์จะทำงานส่งโค้ด JavaScript มาแปลงร่างเป็นคอมโพเนนต์ Native แสดงผลบนหน้าจอมือถือของคุณทันที!

B. สำหรับตัวจำลองคอมพิวเตอร์ (Simulators/Emulators)

• **iOS Simulator:** ต้องติดตั้ง Xcode (เฉพาะ macOS) จากนั้นกดปุ่ม **i** บนคีย์บอร์ดใน Terminal

• **Android Emulator:** ต้องติดตั้ง Android Studio และสร้าง Virtual Device (AVD) จากนั้นกดปุ่ม **a** บนคีย์บอร์ดใน Terminal

เคล็ดลับ (Tip): Fast Refresh: เมื่อคุณทำการแก้ไขข้อความใด ๆ ในไฟล์ **app/index.tsx** แล้วกดบันทึก (Save) หน้าจอแอปพลิเคชันบนมือถือหรือตัวจำลองของคุณจะอัปเดตผลลัพธ์ใหม่ทันทีในเวลาไม่ถึงวินาทีโดยไม่ต้องเริ่มรันแอปใหม่ทั้งหมด

1.7 สรุปท้ายบทเรียน

ทำความรู้จัก React Native และ Expo: React Native เป็นเครื่องมือพัฒนาแอปพลิเคชันแบบครอสแพลตฟอร์มที่ใช้ JavaScript/React ในการควบคุมคอมโพเนนต์ Native จริงของอุปกรณ์เพื่อให้ได้ประสิทธิภาพสูง โดยทำงานร่วมกับ Expo SDK 54 ซึ่งเป็นชุดเครื่องมืออำนวยความสะดวกในการจัดสรรสถาปัตยกรรมและไลบรารีมาตรฐานระดับระบบปฏิบัติการอย่างครบถ้วน

การเปรียบเทียบการเลือกพัฒนาแอปพลิเคชัน: แนวทางการพัฒนาแอปพลิเคชันแบ่งออกเป็น 3 รูปแบบหลัก ได้แก่ Expo Go ซึ่งติดตั้งและเริ่มรันได้ง่ายที่สุดเหมาะแก่การศึกษาและทดลองใช้เครื่องมือมาตรฐาน, Expo Development Builds ที่เหมาะสมสำหรับแอปพลิเคชันขนาดกลางถึงใหญ่เนื่องจากรองรับการปรับแต่งเนทีฟโมดูลเพิ่มเติมได้เอง และ React Native CLI ที่ให้ความอิสระและระดับสิทธิ์การควบคุมระดับลึกที่สุดแก่ผู้พัฒนารวมถึงการเขียนโค้ด Native โดยตรง

การเตรียมความพร้อมและติดตั้งสภาพแวดล้อม: การเริ่มต้นพัฒนาแอปพลิเคชันต้องการสภาพแวดล้อมการเขียนโค้ดที่ประกอบไปด้วย Node.js เวอร์ชัน LTS สำหรับการรัน JavaScript นอกเบราว์เซอร์, VS Code พร้อมส่วนขยายอำนวยความสะดวก เช่น Expo Tools และ Prettier, ระบบควบคุมเวอร์ชันด้วย Git และแอปพลิเคชัน Expo Go บนระบบปฏิบัติการ iOS หรือ Android เพื่อทำหน้าที่แปลงผลและรันแอปบนอุปกรณ์จริงในวงเน็ตเวิร์กเดียวกัน

การสร้างโปรเจคใหม่ด้วย create-expo-app: การสร้างโครงสร้างโฟลเดอร์สำหรับแอปพลิเคชันใหม่ใช้คำสั่ง **npx create-expo-app@latest** ตามด้วยชื่อโปรเจกต์ ซึ่งจะได้โครงสร้างการทำงานแบบสมบูรณ์พร้อมใช้ และผู้พัฒนาสามารถรันคำสั่ง **npm run reset-project** เพื่อล้างหน้าต่างจำลองของเทมเพลตเริ่มต้นและปรับโครงสร้างโปรเจกต์ให้เหลือเพียงหน้ากระดาษเปล่าสำหรับการเขียนแอปพลิเคชันตั้งแต่เริ่มต้น

โครงสร้างของโปรเจค React Native Expo SDK 54: โฟลเดอร์ของโปรเจกต์ที่สร้างขึ้นจะแบ่งสัดส่วนการทำงานออกเป็น โฟลเดอร์ **app/** สำหรับเก็บหน้าจอและระบบการเปลี่ยนหน้าผ่านไฟล์ (Expo Router), โฟลเดอร์ **assets/** เก็บภาพไอคอนและส่วนประกอบด้านมีเดีย, ไฟล์ **app.json** ทำหน้าที่ควบคุมการตั้งค่าคุณสมบัติ (Metadata) ของแอป และไฟล์ **app/index.tsx** ซึ่งเป็นคอมโพเนนต์แรกที่แอปเรียกมาแสดงผลและทำหน้าที่เป็นจุดเริ่มต้นของโค้ดหลัก

การรันโปรเจคและการทดสอบแอปพลิเคชัน: การรันแอปทำได้โดยใช้คำสั่ง `npm start` หรือ `npx expo start` เพื่อเริ่มการทำงานของ Metro Bundler ซึ่งจะทำการรวบรวมไฟล์เขียนโค้ดและส่งผลผ่าน QR Code ให้สแกนดูพฤติกรรมจริงผ่านแอป Expo Go บนอุปกรณ์สมาร์ทโฟน หรือทดสอบผ่านโปรแกรมจำลอง Emulator/Simulator โดยมีการสนับสนุนฟีเจอร์ Fast Refresh ที่ช่วยสะท้อนผลการแก้ไขโค้ดสู่หน้าจอทดสอบโดยทันที

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้และความเข้าใจเกี่ยวกับความรู้พื้นฐานการตั้งค่าระบบ และโครงสร้างเบื้องต้นของ React Native Expo SDK 54

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดอธิบายความแตกต่างที่สำคัญที่สุดระหว่าง React Native และเฟรมเวิร์กประเภท Hybrid App ยุคเก่า (เช่น Cordova หรือ PhoneGap) ได้ถูกต้องที่สุด?

ก. React Native เขียนด้วยภาษา Swift แต่ Hybrid App เขียนด้วยภาษา HTML/CSS

ข. React Native จะแปลงคอมโพเนนต์ไปเป็น Native UI component จริง ๆ ของอุปกรณ์ ส่วน Hybrid App จะแสดงผลผ่าน Web View

ค. React Native รันได้เฉพาะบนระบบปฏิบัติการ iOS เท่านั้น ในขณะที่ Hybrid App รันได้ทุกระบบ

ง. React Native ไม่สามารถเชื่อมต่อกับฮาร์ดแวร์ของอุปกรณ์ได้ แต่ Hybrid App ทำได้

ข้อ 2: เมื่อเราใช้เทมเพลตเริ่มต้นของ Expo SDK 54 และรันคำสั่ง `npm run reset-project` จะเกิดผลลัพธ์ใดขึ้นกับโครงการ?

ก. ลบการเชื่อมต่อเครือข่ายของแอปและล้างข้อมูลแคชทั้งหมด

ข. ลบโปรเจคทิ้งอย่างถาวรและสร้างโฟลเดอร์เปล่าขึ้นมาใหม่

ค. ย้ายไฟล์ตัวอย่างและโครงสร้างเดิมไปไว้ในโฟลเดอร์ `app-example/` และสร้างโฟลเดอร์ `app/` ใหม่พร้อมไฟล์เริ่มต้นที่ว่างเปล่า

ง. ทำการคอมไพล์โปรเจคให้กลายเป็นไฟล์สำหรับติดตั้ง (.apk และ .ipa) ทันที

ข้อ 3: ข้อจำกัดที่สำคัญที่สุดของการใช้เครื่องมือ "Expo Go" ในการทดสอบแอปพลิเคชันคืออะไร?

ก. ไม่สามารถทดสอบบนสมาร์ทโฟนระบบปฏิบัติการ iOS ได้

ข. ไม่สามารถทดสอบโค้ดขณะแก้ไขแบบเรียลไทม์ได้ ต้องคอมไพล์ใหม่ทุกครั้ง

ค. ไม่สามารถทดสอบแอปพลิเคชันที่ติดตั้งไลบรารีหรือเขียนโค้ดภาษา Native เสริมเองภายนอกมาตรฐานของ Expo ได้

ง. ต้องเสียค่าบริการรายเดือนในการสแกน QR Code เพื่อเปิดใช้งาน

ข้อ 4: ไฟล์ใดในโปรเจค React Native Expo ที่ทำหน้าที่เก็บข้อมูลคุณสมบัติแอปพลิเคชัน (App Metadata) เช่น ชื่อแอป โลโก้ (Icon) หน้าต้อนรับ (Splash Screen) และเวอร์ชันของแอป?

ก. `package.json`

ข. `app.json`

ค. `tsconfig.json`

ง. `_layout.tsx`

ข้อ 5: คุณสมบัติ "Fast Refresh" ในสภาพแวดล้อมการพัฒนาของ Expo ทำหน้าที่อะไร?

ก. ช่วยเพิ่มความเร็วในการดาวน์โหลดไลบรารีจากเซิร์ฟเวอร์ npm

ข. อัปเดตอินเทอร์เฟซของแอปบนอุปกรณ์ทดสอบทันทีที่กดบันทึกโค้ด (Save) โดยไม่ต้องเริ่มแอปพลิเคชันใหม่ทั้งหมด

ค. ช่วยเคลียร์แคชและรีสตาร์ตระบบปฏิบัติการของมือถือโดยอัตโนมัติ

ง. การอัปเดตแอปพลิเคชันขึ้นสู่ Google Play Store และ App Store อย่างรวดเร็ว

ข้อ 6: คำสั่งมาตรฐานใดที่แนะนำให้ใช้รันใน Terminal เพื่อเปิดเซิร์ฟเวอร์นักพัฒนา Metro Bundler ของ Expo?

ก. `npm run ios`

ข. `npx expo run:android`

ค. `npx expo start`

ง. `npx react-native start`

ข้อ 7: หากหน้าจอแอปพลิเคชันบนตัวจำลอง (Simulator) ไม่ตอบสนอง หรือ Fast Refresh ไม่ทำงาน คุณสามารถกดปุ่มใดบนคีย์บอร์ดใน Terminal ของเซิร์ฟเวอร์นักพัฒนาเพื่อสั่งโหลดหน้าจอแอปใหม่ด้วยตนเอง (Manual Reload)?

ก. กดปุ่ม `r`

ข. กดปุ่ม `i`

ค. กดปุ่ม `a`

ง. กดปุ่ม `d`

ข้อ 8: การทดสอบแอปพลิเคชันด้วยสมาร์ทโฟนจริงผ่านแอป Expo Go มีเงื่อนไขการเชื่อมต่อเครือข่ายอินเทอร์เน็ตที่จำเป็นอย่างไร?

ก. สมาร์ทโฟนต้องเชื่อมต่อ Cellular 5G ส่วนเครื่องคอมพิวเตอร์เชื่อมต่อ Wi-Fi บ้าน

ข. ทั้งสมาร์ทโฟนและเครื่องคอมพิวเตอร์ต้องเชื่อมต่อผ่านสาย USB ตลอดเวลาเท่านั้น

ค. สมาร์ทโฟนและเครื่องคอมพิวเตอร์ต้องเชื่อมต่อในเครือข่ายวงแลนเดียวกัน (เช่น Wi-Fi วงเดียวกัน)

ง. เครื่องคอมพิวเตอร์และมือถือต้องเปิดสัญญาณ Bluetooth เชื่อมต่อกันก่อน

ข้อ 9: โครงสร้างโฟลเดอร์ของ Expo SDK 54 ที่ใช้ระบบนำทางตามโครงสร้างไฟล์ (File-based Routing) จะมองหาไฟล์เริ่มต้นสำหรับหน้าหลักของแต่ละโฟลเดอร์จากชื่อไฟล์ใด?

ก. `App.tsx`

ข. `index.tsx`

ค. `main.tsx`

ง. `root.tsx`

ข้อ 10: ส่วนขยายใดใน VS Code ที่เป็นตัวช่วยจัดระเบียบและจัดรูปแบบความสวยงามของโค้ดให้ถูกต้องตามมาตรฐานโดยอัตโนมัติทุกครั้งเมื่อกดบันทึกไฟล์?

ก. Prettier - Code formatter

ข. ES7+ React/Redux/React-Native snippets

ค. Expo Tools

ง. GitLens

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: ทั้ง React Native และ Hybrid App ในแง่เขียนโค้ดจะใช้เทคโนโลยีเว็บ (โดยเฉพาะ JavaScript/TypeScript) ไม่ใช่เขียนด้วย Swift เป็นหลัก

• ข. ถูก: นี่คือคุณลักษณะเด่นของ React Native ที่ทำให้ประสิทธิภาพ (Performance) ใกล้เคียงกับ Native App แท้ ๆ เพราะคอมโพเนนต์อย่าง `<View>` หรือ `<Text>` จะถูกจับคู่และสร้างเป็นคอมโพเนนต์ระดับ OS จริง ๆ ของมือถือตัวนั้น ส่วน Hybrid App ยุคเก่าจะรันเว็บแอปจำลองภายในเบราว์เซอร์ครอบ (Web View)

• ค. ผิด: React Native เป็น Cross-platform สามารถรันได้ทั้ง iOS, Android และ Web

• **ง. ผิด:** React Native มีความสามารถสูงในการเข้าถึงฮาร์ดแวร์ผ่าน API ส่วนติดต่อของภาษา Native

เฉลย ข้อ 2: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** คำสั่งนี้ไม่เกี่ยวข้องกับการล้างแคชหรือลบข้อมูลเครื่องจำลอง

• **ข. ผิด:** โปรเจคและไดเรกทอรีหลักยังคงอยู่ไม่ได้ถูกลบทั้งหมด

• **ค. ถูก:** การรัน `npm run reset-project` จะรันสคริปต์เพื่อแยกโค้ดตัวอย่างของระบบ (เช่น explore screen หรือ components ต่าง ๆ) ไปเก็บไว้ที่ `app-example/` และสร้างโฟลเดอร์ `app/` สำหรับเขียนโค้ดที่สะอาดใหม่ขึ้นมา เพื่อเริ่มต้นโครงสร้างของคุณเอง

• **ง. ผิด:** การคอมไพล์โปรเจคเป็นไฟล์ติดตั้งทำโดยใช้บริการของ EAS Build ไม่ใช่สคริปต์ตัวนี้

เฉลย ข้อ 3: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** Expo Go รองรับทั้ง iOS และ Android อย่างสมบูรณ์

• **ข. ผิด:** Expo Go รองรับ Fast Refresh สามารถอัปเดตหน้าจอแบบเรียลไทม์ได้ทันที

• **ค. ถูก:** เนื่องจาก Expo Go เป็นแอปสำเร็จรูปที่มีไลบรารีมาตรฐานติดตั้งมาให้แล้ว หากคุณต้องการเขียนโค้ด Native (เช่น Swift, Kotlin) หรือใช้แพ็กเกจที่จำเพาะเจาะจงที่แก้ไขส่วนประกอบ OS ดั้งเดิม คุณจะต้องเปลี่ยนไปรันผ่าน Expo Development Builds แทน

• **ง. ผิด:** เครื่องมือ Expo Go และการแชร์หน้าจอผ่าน Metro Bundler ใช้งานได้ฟรีไม่มีค่าใช้จ่าย

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `package.json` มีหน้าที่เก็บรายชื่อ Dependencies และสคริปต์คำสั่งรันระบบ

• **ข. ถูก:** `app.json` เป็นไฟล์คอนฟิกูเรชันหลักของ Expo (Expo config) ที่เอาไว้ตั้งค่าข้อมูลทั่วไปของแอปพลิเคชัน โครงสร้างสีเริ่มต้น รูปภาพไอคอน ภาพหน้าต้อนรับ และการกำหนดค่าเฉพาะของแพลตฟอร์มปลายทาง

• **ค. ผิด:** `tsconfig.json` เป็นไฟล์สำหรับตั้งค่าคอมไพเลอร์ของภาษา TypeScript

• **ง. ผิด:** `_layout.tsx` เป็นไฟล์กำหนดโครงสร้างการนำทางและการแสดงผลของหน้าจอ (Layout) ของ Expo Router

เฉลย ข้อ 5: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: การจัดการแพ็กเกจไลบรารีเป็นหน้าที่ของ npm / yarn

• ข. ถูก: Fast Refresh เป็นฟีเจอร์เด่นของการพัฒนาแอปพลิเคชันด้วย React Native ที่จะคอยเฝ้าดูการเปลี่ยนแปลงของไฟล์โค้ด (Watch Mode) เมื่อมีการบันทึกไฟล์ โค้ดที่เปลี่ยนจะถูกส่งไปทำงานบนเครื่องที่เปิดจำลองแอปอยู่ทันที ช่วยลดระยะเวลาที่นักพัฒนาต้องรอรันระบบใหม่ลงอย่างมหาศาล

• ค. ผิด: ฟังก์ชันนี้ไม่มีความสามารถในการยุ่งเกี่ยวกับระบบปฏิบัติการหลักของสมาร์ทโฟน

• ง. ผิด: การอัปโหลดแอปพลิเคชันขึ้นสโตร์ใช้ EAS Submit หรือส่งไฟล์ Build โดยตรง

เฉลย ข้อ 6: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: `npm run ios` ใช้สำหรับสั่งรันแอปบน iOS Simulator ทันที (ซึ่งจะสตาร์ท Metro ให้อยู่แล้ว แต่เป็นคำสั่งจำเพาะเจาะจงแพลตฟอร์ม)

• ข. ผิด: `npx expo run:android` ใช้สำหรับคอมไพล์โปรเจคแบบ Development Builds รันบนเครื่องจำลอง Android ซึ่งช้ากว่าและยุ่งยากกว่าการทดสอบด้วย Expo Go ในขั้นเริ่มต้น

• ค. ถูก: `npx expo start` (หรือ alias `npm start` ใน package.json) คือคำสั่งพื้นฐานและแนะนำที่สุดของ Expo ในการเปิดระบบเซิร์ฟเวอร์ Metro Bundler และสร้าง QR Code เพื่อสแกนรันแอป

• ง. ผิด: `npx react-native start` เป็นคำสั่งรันเซิร์ฟเวอร์สำหรับโปรเจคที่เป็น React Native CLI บริสุทธิ์

เฉลย ข้อ 7: ตอบ ก.

• เหตุผลประกอบ:

• ก. ถูก: ในหน้าต่าง Terminal ของเซิร์ฟเวอร์ Metro การกดปุ่ม `r` ย่อมาจาก **Reload** จะส่งสัญญาณบอกแอปทดสอบบนเครื่องจำลองหรือโทรศัพท์จริงให้รีโหลดหน้าจอใหม่ทั้งหมดแบบแมนนวล

• ข. ผิด: การกดปุ่ม `i` ย่อมาจาก **iOS** ใช้สำหรับสั่งเปิดแอปบน iOS Simulator

• ค. ผิด: การกดปุ่ม `a` ย่อมาจาก **Android** ใช้สำหรับสั่งเปิดแอปบน Android Emulator

• **ง. ผิด:** การกดปุ่ม `d` ย่อมาจาก **Dev Menu** ใช้สำหรับสั่งเปิดเมนูนึกพัฒนา (Developer Menu) บนแอปจำลอง

เฉลย ข้อ 8: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** การเปิดข้ามเครือข่ายผ่าน Cellular อาจทำให้แอปหาเครื่องคอมพิวเตอร์ในวงแลนของเครือข่ายนักพัฒนาไม่เจอ ยกเว้นมีการตั้งค่าอุโมงค์การเชื่อมต่อแบบ Tunnel (ซึ่งทำได้ยากและช้ากว่าปกติมาก)

• **ข. ผิด:** ไม่จำเป็นต้องต่อสาย USB เสมอไปเมื่อใช้งาน Expo Go เพราะระบบสามารถส่งผ่านข้อมูลทาง Wi-Fi วงแลนได้แบบไร้สาย

• **ค. ถูก:** หัวใจสำคัญของการสื่อสารระหว่าง Metro Bundler บน PC และมือถือที่ใช้รันแอปคือต้องอยู่ในเครือข่ายภายในบ้าน/ที่ทำงานเดียวกัน (Subnet วงเดียวกัน) เพื่อให้เครื่องมือถือสามารถค้นหา IP Address และ Port 8081 ของ PC พบ

• **ง. ผิด:** Expo CLI และ Expo Go ไม่สื่อสารโค้ดโปรแกรมหลักผ่านบลูทูธ

เฉลย ข้อ 9: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `App.tsx` เป็นไฟล์เริ่มต้นของโปรเจค React Native แบบดั้งเดิม แต่ไม่ใช่โครงสร้างของ Expo Router ที่เปิดตัวขึ้นมาในรุ่นหลัง

• **ข. ถูก:** โครงสร้าง File-based Routing (Expo Router) จะทำงานคล้ายกับการจัดการหน้าเว็บ (Web Router) โดยไฟล์ที่ทำหน้าที่เป็นจุดเริ่มต้นหรือหน้าแรกของโฟลเดอร์ใด ๆ จะต้องตั้งชื่อว่า `index.tsx` (หรือ `index.js`, `index.ts`) เสมอ

• **ค. ผิด:** `main.tsx` ไม่เกี่ยวข้องกับการหาเส้นทางเริ่มต้นของโฟลเดอร์

• **ง. ผิด:** `root.tsx` ไม่ได้ใช้ระบุตำแหน่งของ Layout เริ่มต้นในระดับโฟลเดอร์

เฉลย ข้อ 10: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก: Prettier** คือตัวจัดการปรับสไตล์โค้ด (Code Formatter) ยอดนิยมที่จะจัดย่อหน้า แทรกเครื่องหมาย semicolon หรือจัดระเบียบวงเล็บปีกกาและช่องว่างโดยอัตโนมัติเมื่อกดจัดรูปแบบหรือเซฟไฟล์

• **ข. ผิด: ES7+ React/Redux/React-Native snippets** เป็นเพียงตัวช่วยสร้างโครงโค้ดแบบรวดเร็ว (เช่น พิมพ์ `rfc` เพื่อสร้าง Component) แต่ไม่ได้มีหน้าที่เป็นตัวจัดรูปแบบโค้ด

• **ค. ผิด: Expo Tools** ช่วยทำไฮไลต์ไวยากรณ์ในไฟล์คอนฟิก `app.json` และช่วยทำ Auto-complete ในบางจุดที่เกี่ยวข้องกับเครื่องมือของ Expo เท่านั้น

• **ง. ผิด: GitLens** เป็นตัวเสริมสำหรับการแสดงประวัติและเจ้าของบรรทัดโค้ดบนระบบ Git

ปฏิบัติการท้ายบทเรียน: การเตรียมสภาพแวดล้อมและการสร้างแอปพลิเคชันแรก

ปฏิบัติการนี้จะช่วยให้ผู้เรียนได้ลงมือทำจริงตั้งแต่ขั้นตอนการติดตั้งโปรแกรมพื้นฐาน การสร้างโปรเจค React Native Expo SDK 54 การรีเซ็ตโครงสร้างตัวอย่างจากเทมเพลตเริ่มต้น การเขียนโค้ดภาษา TypeScript/JSX ตัวแรก และทดลองรันแอปพลิเคชันจริงผ่านเซิร์ฟเวอร์นักพัฒนา

1. วัตถุประสงค์การเรียนรู้

1. สามารถติดตั้งเครื่องมือพัฒนาแอปพลิเคชันมือถือ (Node.js, VS Code) และกำหนดสภาพแวดล้อมการรันได้อย่างถูกต้อง

2. สามารถสร้างโปรเจค React Native Expo ตัวใหม่ผ่านเทมเพลต TypeScript ได้สำเร็จ

3. สามารถใช้สคริปต์ `reset-project` เพื่อล้างโค้ดเทมเพลตตัวอย่างและจัดโครงสร้างโปรเจคเริ่มต้นใหม่ได้

4. สามารถเขียนคอมโพเนนต์พื้นฐานเพื่อสร้างหน้าจออินเทอร์เฟซต้อนรับอย่างง่ายได้ด้วยตนเอง

5. สามารถใช้งาน Metro Bundler และเปิดทดสอบหน้าจอแอปพลิเคชันจริงผ่านสมาร์ทโฟน (Expo Go) หรืออุปกรณ์จำลองได้

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนจะต้องดำเนินการเตรียมเครื่องมือและเขียนหน้าต้อนรับ (Welcome Screen) ลงในโปรเจคตามเงื่อนไขดังนี้:

1. ติดตั้ง Node.js และ VS Code พร้อมขยายระบบตัวช่วยโค้ด (Extensions) ที่กำหนด

2. สร้างโปรเจค Expo ใหม่ด้วยชื่อโฟลเดอร์ `my-first-app`

3. ล้างตัวอย่างโปรเจคดั้งเดิมให้เหลือเฉพาะโครงสร้างไฟล์เริ่มต้นเพื่อรอเขียนโปรแกรม

4. หน้าจอหลักต้องแสดงกล่องข้อมูลต้อนรับผู้เรียนซึ่งกึ่งกลางของหน้าจอ (Center alignment)

5. ปรับเปลี่ยนโทนสีพื้นหลังเป็นสไตล์ Slate เข้ม (`#0f172a`) และข้อความทั้งหมดต้องจัดสไตล์ให้อ่านง่าย มีสีสันสอดคล้องกับธีม (สไตล์ Dark Mode)

6. มีส่วนแสดงผล **ชื่อ-นามสกุล** และ **รหัสนักศึกษา/ชื่อผู้เรียน** รวมถึงแสดงสถานะการเชื่อมต่อว่า "Expo SDK 54 - Active"

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งโปรแกรมที่จำเป็น (Software Installation)

1. **ติดตั้ง Node.js:** ไปที่เว็บไซต์ Node.js ดาวน์โหลดรุ่น **LTS** ล่าสุดมาติดตั้งให้เรียบร้อย จากนั้นเช็กความถูกต้องผ่าน Terminal/PowerShell ด้วยคำสั่ง:

```
node -v
npm -v
```

2. **ติดตั้ง Visual Studio Code (VS Code):** ดาวน์โหลดที่ VS Code และเข้าไปที่เมนู Extensions (ไอคอนสี่เหลี่ยมด้านซ้าย) ค้นหาและติดตั้งตัวช่วยต่อไปนี้:

- *ES7+ React/Redux/React-Native snippets*
- *Prettier - Code formatter*
- *Expo Tools*

ขั้นตอนที่ 2: การสร้างโปรเจคใหม่ด้วย create-expo-app

1. เปิด Command Prompt หรือ PowerShell (ใน Windows) หรือ Terminal (ใน macOS/Linux)

2. เปลี่ยนตำแหน่งไดเรกทอรีไปยังโฟลเดอร์ที่คุณต้องการบันทึกงาน เช่น:

```
cd Documents
```

3. รันคำสั่งสร้างโปรเจคใหม่ด้วยคำสั่ง Expo CLI เวอร์ชันล่าสุด:

```
npx create-expo-app@latest my-first-app
```

4. เมื่อสร้างโปรเจคเรียบร้อยแล้ว ให้ย้ายเข้าสู่โฟลเดอร์โปรเจค:

```
cd my-first-app
```

5. เปิดหน้าต่างโปรเจคขึ้นมาบนโปรแกรม VS Code:

```
code .
```

ขั้นตอนที่ 3: ล้างไฟล์เทมเพลตเริ่มต้น (Reset Project Layout)

ในหน้าต่าง VS Code เปิด Terminal ขึ้นมา (กดปุ่มคีย์บอร์ด `Ctrl + ~`) จากนั้นพิมพ์รันสคริปต์เคลียร์โค้ดเริ่มต้น:

```
npm run reset-project
```

สังเกตผลลัพธ์: โฟลเดอร์ `app/` จะถูกล้างส่วนประกอบตัวอย่างออกทั้งหมดและสร้างโครงสร้างที่สะอาดขึ้นมาใหม่ทันที ส่วนโค้ดตัวอย่างเดิมจะถูกสำรองไว้ที่ `app-example/`

ขั้นตอนที่ 4: เขียนโค้ดปรับปรุงหน้าแรก (app/index.tsx)

ให้เปิดไฟล์ `app/index.tsx` ใน VS Code แล้วเขียนโค้ดทับด้วยคอมโพเนนต์หน้าต้อนรับดังต่อไปนี้:

```
import { StyleSheet, Text, View } from 'react-native';

export default function WelcomeScreen() {
  return (
    <View style={styles.container}>
      {/* ส่วนหัวแสดงไอคอนหรือรูปจำลองด้วยอิโมจิ */}
      <Text style={styles.emoji}>🚀</Text>

      {/* หัวข้อหลัก */}
      <Text style={styles.title}>ยินดีต้อนรับเข้าสู่ชั้นเรียน Mobile App</Text>
      <Text style={styles.subtitle}>React Native Expo (SDK 54)</Text>

      {/* การ์ดข้อมูลผู้เรียน */}
      <View style={styles.card}>
        <Text style={styles.cardLabel}>ผู้พัฒนาแอปพลิเคชัน:</Text>
        <Text style={styles.cardName}>สมชาย ยอดรักดี (นายทะเบียน)</Text>
        <Text style={styles.cardId}>รหัสนักศึกษา: 66-12345-6789</Text>
      </View>

      {/* แถบสถานะระบบ */}
      <View style={styles.statusBadge}>
        <Text style={styles.statusText}>● Status: Running on Expo
Go</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a', // สีพื้นหลังเทาดำ Slate เข้ม
    alignItems: 'center',
    justifyContent: 'center',
    padding: 24,
  },
  emoji: {
```

```
    fontSize: 64,
    marginBottom: 16,
  },
  title: {
    fontSize: 22,
    fontWeight: 'bold',
    color: '#38bdf8', // สีฟ้านีออนโดดเด่นสว่างตา
    textAlign: 'center',
    marginBottom: 4,
  },
  subtitle: {
    fontSize: 16,
    color: '#94a3b8', // สีเทาอ่อน สบายตา
    textAlign: 'center',
    marginBottom: 32,
  },
  card: {
    backgroundColor: '#1e293b', // พื้นหลังการ์ดสีเทา Slate
    borderRadius: 16,
    padding: 20,
    width: '100%',
    maxWidth: 320,
    borderWidth: 1,
    borderColor: '#334155', // เส้นขอบสี Slate จาง ๆ
    alignItems: 'flex-start',
    marginBottom: 24,
  },
  cardLabel: {
    fontSize: 12,
    color: '#64748b',
    textTransform: 'uppercase',
    letterSpacing: 1,
    marginBottom: 6,
  },
  cardName: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#f8fafc',
    marginBottom: 4,
  },
  cardId: {
    fontSize: 14,
    color: '#94a3b8',
  },
  statusBadge: {
    backgroundColor: '#064e3b', // พื้นหลังสีเขียวเข้ม
    paddingVertical: 6,
    paddingHorizontal: 12,
    borderRadius: 20,
  },
  statusText: {
    color: '#34d399', // ข้อความสีเขียวพาสเทล
    fontSize: 12,
    fontWeight: '600',
```

```
  },
});
```

ขั้นตอนที่ 5: เปิดเซิร์ฟเวอร์ Metro Bundler

บันทึกไฟล์โค้ด จากนั้นใน Terminal ของ VS Code ให้รันคำสั่งเริ่มทำงานระบบเซิร์ฟเวอร์นักพัฒนา:

```
npx expo start
```

ขั้นตอนที่ 6: สแกนทดสอบบนอุปกรณ์จริง (Expo Go) หรืออุปกรณ์จำลอง

1. เปิดแอปพลิเคชัน **Expo Go** บนมือถือของคุณ (ดาวน์โหลดติดตั้งผ่าน App Store/Play Store ก่อนหน้านี้)

2. **สำหรับ iOS:** เปิดกล้องมือถือสแกน QR Code บนหน้าจอคอมพิวเตอร์เพื่อสลับเปิดรันด้วย Expo Go

3. **สำหรับ Android:** เปิดแอป Expo Go แล้วกดสแกน QR Code ผ่านแอปโดยตรง

4. หากต้องการทดสอบบนเครื่องจำลอง: กดปุ่ม `i` สำหรับ iOS Simulator (สำหรับเครื่อง macOS) หรือกดปุ่ม `a` สำหรับ Android Emulator (ต้องเปิดโปรแกรม Android Virtual Device ก่อนรันคีย์)

ขั้นตอนที่ 7: ทดสอบฟีเจอร์ Fast Refresh

1. เปิดแสดงผลหน้าจอแอปพลิเคชันควบคู่ไปกับหน้าต่างแก้ไขโค้ด VS Code

2. ให้แก้ไขคำว่า "สมชาย ยอดรักดี" ในโค้ด `app/index.tsx` บรรทัดที่ 16 ให้เปลี่ยนเป็น **"ชื่อและนามสกุลจริงของคุณ"**

3. กดปุ่มบันทึกไฟล์ (`Ctrl + S` หรือ `Cmd + S`)

4. สังเกตหน้าจอทดสอบ จะพบว่าข้อความอัปเดตเป็นชื่อของคุณทันทีภายในไม่กี่วินาทีโดยไม่ต้องปิดและรันแอปใหม่

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

โค้ดเฉลยฉบับสมบูรณ์สำหรับปฏิบัติการนี้ถูกระบุไว้เป็นตัวอย่างโครงสร้างใน **ขั้นตอนที่ 4** ผู้เรียนสามารถคัดลอกและบันทึกไฟล์เพื่อรันตรวจสอบความถูกต้องของระบบได้ทันที

เอกสารอ้างอิง

Expo. (2024). *Create an app*. จาก https://docs.expo.dev/versions/v54.0.0/get-started/create-a-project/

Expo. (2024). *Expo Router Introduction*. จาก https://docs.expo.dev/versions/v54.0.0/router/introduction/

Git. (2024). *Git Reference Manual*. จาก https://git-scm.com/doc

Microsoft. (2024). *Visual Studio Code Documentation*. จาก https://code.visualstudio.com/docs

Node.js. (2024). *Node.js Downloads*. จาก https://nodejs.org/

React Native. (2024). *Environment Setup*. จาก https://reactnative.dev/docs/environment-setup

แผนบริหารการสอนประจำบทที่ 2

หัวข้อเนื้อหาประจำบท

2.1 การเขียนและประกาศสไตล์ใน React Native

2.2 โครงสร้างกล่องและคุณสมบัติขนาดขอบเขต (Box Model)

2.3 ทิศทางการวางแกนด้วย flexDirection และ flex

2.4 การจัดกึ่งกลางและการจัดเรียงด้วย justifyContent และ alignItems

2.5 การขึ้นบรรทัดใหม่และการเว้นช่องว่าง (flexWrap & gap)

2.6 โค้ดตัวอย่างหน้าเลย์เอาต์โครงสร้าง Grid จำลอง

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถอธิบายความแตกต่างของค่าเริ่มต้นของ flexDirection ใน React Native เปรียบเทียบกับ CSS บนเว็บได้ถูกต้อง
2. ผู้เรียนสามารถใช้คำสั่งสไตล์พื้นฐานในการปรับแต่งความกว้าง ความสูง ระยะขอบ และเส้นขอบของคอมโพเนนต์ได้ถูกต้อง
3. ผู้เรียนสามารถเขียนสไตล์การจัดแนวทางคอมโพเนนต์โดยใช้ justifyContent และ alignItems เพื่อจัดโครงสร้างหน้าจอตามที่กำหนดได้
4. ผู้เรียนสามารถประกอบชุดกล่องสไตล์ชีทในรูปแบบรังผึ้ง หรือตาราง (Grid) โดยประยุกต์ใช้คอมโพเนนต์ View และ Text ได้สำเร็จโดยไม่มีข้อผิดพลาด

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

5. **การบรรยายแบบโต้ตอบ (Interactive Lecture):** นำเสนอภาพสไลด์และกล่องคอนเทนเนอร์เพื่อให้เห็นภาพของการทำงานแบบ Flexbox
6. **กิจกรรมจับผิดเลย์เอาต์ (Visual Layout Quiz):** แสดงภาพ UI ที่บิดเบี้ยวหรือจัดตำแหน่งผิด และให้ผู้เรียนวิเคราะห์ร่วมกันว่าขาดสไตล์ใด
7. **การลงมือปฏิบัติสร้างกล่องสี (Layout Coding Practice):** ให้ผู้เรียนเปิดคอมไพเลอร์ลองจัดหน้าจอเป็นธงชาติ หรือกล่องแม่สี 3 สีในทิศทางต่าง ๆ

8. **การสรุปผลและสะท้อนคิด (Reflection):** ร่วมสรุปความท้าทายในการจัดเลย์เอาต์ที่มีขนาดหน้าจอแตกต่างกัน (Cross-Device Layouts)

สื่อการเรียนการสอน

5. เอกสารสไลด์เรื่อง "Flexbox Layout System in React Native"
6. บอร์ดเกมจำลองคุณสมบัติ Flexbox เพื่อช่วยในการตัดสินใจเลือกคำสั่ง
7. โจทย์ตัวอย่างการจัดวาง Layout แบบต่าง ๆ (เช่น Header, Sidebar, Grid)
8. ไฟล์เริ่มต้นโปรเจคที่มีการ์ดกล่องสีจำลองเตรียมไว้

การวัดผลและการประเมินผล

1. การวัดผล

1.1 วัดความรู้จากแบบทดสอบปรนัยท้ายบทเรียน 10 ข้อ

1.2 วัดทักษะจากการตรวจผลงานการต่อบล็อกสี่สีและการเรียงลำดับเนื้อหาในปฏิบัติการท้ายบทเรียน

1.3 ตรวจสอบโครงสร้างความถูกต้องของการเขียนคลาสสไตล์ผ่าน StyleSheet

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถเขียนโค้ดจัดหน้าจอจำลองเมนูสี่เหลี่ยมจัดแนว Flexbox ได้ถูกต้องตามต้นแบบผ่านเกณฑ์ (ผ่าน/ไม่ผ่าน)

2.3 พฤติกรรมการกระตือรือร้นในการทำงานและการให้ความร่วมมือแก้ไขโจทย์ร่วมกับผู้อื่นในชั่วโมงเรียน

บทที่ 2

การจัดเลย์เอาต์หน้าจอด้วย Flexbox

ระบบเลย์เอาต์ (Layout System) ของ React Native ทำงานโดยการอ้างอิงและประยุกต์ใช้ข้อกำหนดของ **Flexbox** เป็นหลัก (React Native, 2024) ซึ่งมีแนวคิดคล้ายกับ CSS บนเว็บ แต่มีคุณสมบัติเฉพาะตัวบางประการที่ออกแบบมาเพื่อให้แสดงผลบนขนาดหน้าจอสมาร์ทโฟนที่มีสัดส่วนหลากหลายได้อย่างสม่ำเสมอ

2.1 การเขียนและประกาศสไตล์ใน React Native

ใน React Native เราไม่มีคลาสสไตล์ชีทแบบ CSS ไฟล์แยกภายนอก (เช่น `.css`) แต่จะใช้งานผ่านออบเจกต์ JavaScript ในรูปแบบ **CSS-in-JS** ซึ่งสามารถเขียนประกาศได้ 2 รูปแบบหลัก:

1. แบบ Inline Styles

เป็นการส่งออบเจกต์สไตล์เข้าไปยังพร็อพส์ `style` ของคอมโพเนนต์โดยตรง เหมาะสำหรับใช้กำหนดคุณสมบัติที่เปลี่ยนแปลงตามสถานะตัวแปร หรือทดสอบเลย์เอาต์แบบเร่งด่วน:

```
<View style={{ width: 100, height: 100, backgroundColor: 'red' }} />
```

2. แบบ StyleSheet.create (แนะนำ)

เป็นการประกาศชุดสไตล์ภายนอกคอมโพเนนต์ (React Native, 2024) ทำให้แยกส่วนการแสดงผลและสไตล์ชัดเจน ตัวคอมไพเลอร์ของ React Native สามารถแปลรหัสและแคชโครงสร้างสไตล์นี้ส่งไปยังฝั่งระบบปฏิบัติการได้มีประสิทธิภาพสูงกว่า:

```
import { StyleSheet, View } from 'react-native';

export default function App() {
  return <View style={styles.box} />;
}

const styles = StyleSheet.create({
  box: {
    width: 100,
    height: 100,
    backgroundColor: '#38bdf8',
  },
});
```

2.2 โครงสร้างกล่องและคุณสมบัติขนาดขอบเขต (Box Model)

การกำหนดกล่องสไตล์ชีทประกอบด้วยคุณสมบัติสำคัญที่เป็นพื้นฐานในการจำกัดพื้นที่ ได้แก่:

• `width` & `height`: กำหนดความกว้างและความสูง สามารถใส่เป็นค่าพิกเซลเดี่ยว (เช่น `150`) หรือค่าเปอร์เซ็นต์ (เช่น `'50%'`)

• `margin` (ระยะขอบภายนอก): ระยะห่างระหว่างขอบกล่องภายนอกไปยังกล่องข้างเคียง

• `padding` (ระยะขอบภายใน): ระยะห่างระหว่างขอบกล่องด้านในไปยังวัตถุหรือเนื้อหาข้างในกล่อง

• `border` (เส้นขอบ): กำหนดขนาดและสีขอบด้วย `borderWidth`, `borderColor`

• `borderRadius` (มุมโค้งมน): ใช้ตัดมุมกล่องให้โค้งงอ (หากต้องการตัดกล่องให้กลมสนิท ให้กำหนด `borderRadius` เป็นครึ่งหนึ่งของขนาดความกว้าง/สูง)

```
// ตัวอย่างกล่องโปรไฟล์ที่ใช้คุณสมบัติสเปซขอบ
const styles = StyleSheet.create({
  card: {
    width: 300,
    padding: 16,          // ระยะเว้นข้อความข้างใน
    margin: 8,            // ระยะเว้นนอกการ์ด
    borderWidth: 2,
    borderColor: '#e2e8f0',
    borderRadius: 12,     // ทำมุมการ์ดโค้งมนสไตล์การ์ดไอที
  }
});
```

2.3 ทิศทางการวางแกนด้วย flexDirection และ flex

หัวใจของการจัดตำแหน่งใน React Native คือทิศทางการวิ่งของข้อมูลในกลุ่มคอนเทนเนอร์ (Container):

1. คุณสมบัติ flexDirection (ทิศทางแกนหลัก)

กำหนดทิศทางเรียงตัวของคอมโพเนนต์ลูก (Children) ทั้งหมดภายในกล่องคอนเทนเนอร์นั้น:

ตารางที่ 2.1 ค่าคงที่และทิศทางการจัดวางของ flexDirection

ค่าคงที่	คำอธิบาย
column (ค่าเริ่มต้น)	เรียงตัวจาก บนลงล่าง (แกนหลักวิ่งแนวตั้ง,

	แกนรองวิ่งแนวนอน)
row	เรียงตัวจาก ซ้ายไปขวา (แกนหลักวิ่งแนวนอน, แกนรองวิ่งแนวตั้ง)
column-reverse	เรียงตัวจาก ล่างขึ้นบน ย้อนกลับตามแนวตั้ง
row-reverse	เรียงตัวจาก ขวาไปซ้าย ย้อนกลับตามแนวนอน

ข้อควรจำ (Important): ความแตกต่างสำคัญ: บนเว็บบราวเซอร์ ค่าเริ่มต้นของ Flexbox จะเป็น `row` (แนวนอน) แต่ใน **React Native** ค่าเริ่มต้นถูกตั้งใจตั้งไว้เป็น `column` (แนวตั้ง) เพื่อสอดคล้องกับพฤติกรรมการเลื่อนหน้าจอสมาร์ทโฟน

2. คุณสมบัติ flex (สัดส่วนการยืดขยาย)

การสเกลพื้นที่ของแต่ละกล่อง หากกำหนด `flex: 1` หมายความว่าให้กล่องนั้นครอบครองพื้นที่ว่างทั้งหมดที่เหลืออยู่ในแถวนั้น

```
// กล่องคอนเทนเนอร์ขนาดใหญ่คลุมหน้าจอ และแบ่งพื้นที่ออกเป็นสองบล็อกเท่ากัน
const styles = StyleSheet.create({
  container: {
    flex: 1, // ขยายครอบพื้นที่หน้าจอทั้งหมด
    flexDirection: 'row', // จัดวางแบบแนวนอน (ซ้ายไปขวา)
  },
  leftColumn: {
    flex: 1, // เอาพื้นที่ครึ่งหนึ่งทางซ้าย
    backgroundColor: '#1e293b',
  },
  rightColumn: {
    flex: 1, // เอาพื้นที่อีกครึ่งหนึ่งทางขวา
    backgroundColor: '#3b82f6',
  }
});
```

2.4 การจัดกึ่งกลางและการจัดเรียงด้วย justifyContent และ alignItems

เมื่อกำหนดทิศทางแกนแล้ว เราจะใช้คำสั่งอีก 2 ชุดในการระบุการกระจายตัวของวัตถุ:

1. justifyContent (จัดวางในทิศทางแกนหลัก)

- `flex-start`: ชิดต้นสายของแกนหลัก
- `center`: จัดวัตถุทั้งหมดให้อยู่ตรงกึ่งกลางของแกนหลัก
- `flex-end`: ชิดท้ายสายของแกนหลัก

• `space-between`: กระจายตัวบล็อกโดยเว้นระยะห่างตรงกลางให้กว้างที่สุดเท่ากัน (หัวท้ายติดขอบ)

• `space-around`: กระจายตัวโดยเว้นระยะรอบข้างบล็อกเท่า ๆ กัน (หัวท้ายมีช่องว่างครึ่งหนึ่งของตรงกลาง)

• `space-evenly`: กระจายช่องว่างระหว่างกล่องทุกบล็อกเท่ากันทั้งหมด รวมถึงหัวและท้าย

2. alignItems (จัดวางในทิศทางแกนรอง)

• `stretch` (ค่าเริ่มต้น): ยืดขยายบล็อกให้ยาวจนเต็มขอบแกนรอง (จะใช้งานได้เมื่อไม่มีการฟิกขนาดด้านนั้น)

• `flex-start`: ชิดจุดเริ่มต้นแกนรอง

• `center`: จัดวัตถุลูกทั้งหมดให้กึ่งกลางแนวแกนรอง

• `flex-end`: ชิดจุดสิ้นสุดแกนรอง

2.5 การขึ้นบรรทัดใหม่และการเว้นช่องว่าง (flexWrap & gap)

1. คุณสมบัติ flexWrap

ในกรณีที่วัตถุลูกมีความกว้างรวมกันเกินความกว้างคอนเทนเนอร์ หากตั้งค่า `flexWrap: 'nowrap'` (ค่าเริ่มต้น) กล่องจะล้นออกไปนอกหน้าจอหรือถูกบีบ แต่หากตั้งค่าเป็น `wrap` วัตถุที่ล้นจะถูกตัดไปขึ้นบรรทัดใหม่โดยอัตโนมัติ เหมาะสำหรับการจัดโครงสร้างแท็กสไตล์ หรือเมนู Grid ภาพไอคอน

2. คุณสมบัติ gap (ความสะดวกใน SDK 54)

ในอดีตการเว้นช่องไฟระหว่างบล็อกสีจะต้องเขียน `marginRight` หรือ `marginBottom` ให้วุ่นวาย แต่ใน Expo SDK 54 เราสามารถใช้คำสั่งในการสร้างระยะห่างได้อย่างเป็นระเบียบ:

• `gap`: ใส่ค่าพิกเซลเว้นระยะทั้งแนวตั้งและแนวนอนเท่า ๆ กัน (เช่น `gap: 12`)

• `rowGap`: เว้นระยะห่างเฉพาะแถวแนวตั้ง

• `columnGap`: เว้นระยะห่างเฉพาะคอลัมน์แนวนอน

2.6 โค้ดตัวอย่างหน้าเลย์เอาต์โครงสร้าง Grid จำลอง

ด้านล่างนี้คือโค้ดตัวอย่างการจัดตำแหน่งแผงปุ่มเมนู 4 บล็อก (Grid Layout 2x2) โดยประยุกต์ใช้ Flexbox และช่องไฟ gap:

```
import { StyleSheet, Text, View } from 'react-native';

export default function GridLayout() {
  return (
```

```
    <View style={styles.container}>
      <Text style={styles.headerTitle}>เลย์เอาต์แบบตาราง 2x2</Text>

      <View style={styles.gridContainer}>
        {/* บล็อกที่ 1 */}
        <View style={[styles.box, { backgroundColor: '#ef4444' }]}>
          <Text style={styles.boxText}>เมนู A</Text>
        </View>
        {/* บล็อกที่ 2 */}
        <View style={[styles.box, { backgroundColor: '#3b82f6' }]}>
          <Text style={styles.boxText}>เมนู B</Text>
        </View>
        {/* บล็อกที่ 3 */}
        <View style={[styles.box, { backgroundColor: '#10b981' }]}>
          <Text style={styles.boxText}>เมนู C</Text>
        </View>
        {/* บล็อกที่ 4 */}
        <View style={[styles.box, { backgroundColor: '#f59e0b' }]}>
          <Text style={styles.boxText}>เมนู D</Text>
        </View>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a',
    paddingTop: 60,
    paddingHorizontal: 20,
  },
  headerTitle: {
    fontSize: 22,
    fontWeight: 'bold',
    color: '#ffffff',
    textAlign: 'center',
    marginBottom: 20,
  },
  gridContainer: {
    flexDirection: 'row',      // วางแนวนอน
    flexWrap: 'wrap',          // ถ้าล้นให้ตัดขึ้นบรรทัดใหม่
    gap: 16,                   // เว้นระยะขอบรอบด้านทุกกล่อง
    justifyContent: 'center', // กระจายชิดใน
  },
  box: {
    width: '45%',              // กินความกว้างเกือบครึ่งหนึ่ง
    height: 120,
    borderRadius: 16,
    justifyContent: 'center', // จัดแกนหลักกึ่งกลาง (แนวตั้ง)
```

```
    alignItems: 'center',      // จัดแกนรองกึ่งกลาง (แนวนอน)
  },
  boxText: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
  },
});
```

สรุปท้ายบทเรียน

การเขียนและประกาศสไตล์ใน React Native: สไตล์ใน React Native ใช้แนวคิด CSS-in-JS ผ่านออบเจกต์ JavaScript โดยสามารถเขียนได้สองรูปแบบหลัก ได้แก่ Inline Styles ซึ่งเป็นการส่งออบเจกต์เข้าไปที่พร็อพส์สไตล์โดยตรง เหมาะสำหรับการทดสอบอย่างรวดเร็ว และ StyleSheet.create ซึ่งเป็นวิธีที่แนะนำเนื่องจากช่วยแยกส่วนสไตล์และทำให้ระบบสามารถประมวลผลพร้อมแคชสไตล์ได้อย่างมีประสิทธิภาพสูงขึ้น

โครงสร้างกล่องและคุณสมบัติขนาดขอบเขต: การควบคุมขนาดและพื้นที่ใน React Native อ้างอิงระบบ Box Model เช่นเดียวกับเว็บ ประกอบด้วยคุณสมบัติหลักอย่าง width และ height เพื่อกำหนดขนาดกล่อง, margin เพื่อระบุระยะห่างจากขอบกล่องภายนอกไปยังส่วนประกอบอื่น, padding สำหรับกำหนดช่องว่างระหว่างขอบด้านในกับเนื้อหาภายใน, border เพื่อระบุขนาดและสีขอบ รวมถึง borderRadius ที่ช่วยในการลบมุมกล่องให้มีความโค้งมน

ทิศทางการวางแกนด้วย flexDirection และ flex: การจัดตำแหน่งคอมโพเนนต์จะยึดหลักแนวแกน โดยคุณสมบัติ flexDirection จะกำหนดทิศทางการเรียงตัวของคอมโพเนนต์ลูก ซึ่งค่าเริ่มต้นใน React Native จะเป็นแบบแนวตั้ง (column) ต่างจากเว็บที่เป็นแนวนอน (row) เพื่อรองรับพฤติกรรมของแอปพลิเคชันมือถือ ในขณะที่คุณสมบัติ flex จะใช้สำหรับกำหนดสัดส่วนการยืดขยายและกระจายพื้นที่ว่างของแต่ละกล่อง

การจัดกึ่งกลางและการจัดเรียงด้วย justifyContent และ alignItems: การกระจายตัวของวัตถุลูกทำได้โดยใช้คุณสมบัติ justifyContent ในการจัดตำแหน่งตามแนวแกนหลัก (เช่น center, space-between หรือ space-around) และใช้ alignItems ในการจัดระเบียบวัตถุลูกตามแนวแกนรอง (เช่น stretch หรือ center) ซึ่งการผสมผสานสองคำสั่งนี้ช่วยให้จัดการเลย์เอาต์ที่ซับซ้อนและจัดข้อมูลให้อยู่กึ่งกลางหน้าจอได้อย่างเป็นระบบ

การขึ้นบรรทัดใหม่และการเว้นช่องว่าง: เมื่อวัตถุมีขนาดรวมเกินขอบเขตกล่องคอนเทนเนอร์ เราสามารถกำหนด flexWrap เป็น wrap เพื่อให้วัตถุขึ้นบรรทัดใหม่โดยอัตโนมัติ นอกจากนี้ใน Expo SDK 54 ยังรองรับการใช้งานคุณสมบัติ gap (Expo, 2024) rowGap และ columnGap ซึ่ง

ช่วยให้สามารถระบุช่องไฟหรือระยะห่างระหว่างบล็อกสีและวัตถุต่าง ๆ ได้สะดวกรวดเร็วโดยไม่ต้องประกาศ margin แยกรายตัว

โค้ดตัวอย่างหน้าเลย์เอาต์โครงสร้าง Grid จำลอง: ตัวอย่างการนำความรู้การจัดหน้าจอด้วย Flexbox มาใช้ร่วมกันจะเห็นได้ชัดจากการสร้างเลย์เอาต์แบบตาราง Grid ขนาด 2x2 โดยใช้ flexDirection: 'row' ร่วมกับ flexWrap: 'wrap' และการกำหนดระยะห่างด้วย gap โดยอาศัย alignItems: 'center' และ justifyContent: 'center' ในการจัดกล่องย่อยแต่ละใบให้เนื้อหาภายในอยู่ตรงกึ่งกลางอย่างสมบูรณ์

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความเข้าใจเกี่ยวกับสไตล์ชีท (StyleSheet), กล่องสเกลขนาด (Box Model) และคุณสมบัติของระบบจัดตำแหน่งแบบ Flexbox ใน React Native Expo

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ใน React Native ข้อใดคือทิศทางการจัดเรียงแกนหลักเริ่มต้น (Default flexDirection) ของคอมโพเนนต์ลูกภายในกล่องคอนเทนเนอร์?

ก. `row`

ข. `column`

ค. `row-reverse`

ง. `column-reverse`

ข้อ 2: การระบุตัวเลขโดด ๆ ในการกำหนดขนาด เช่น `{ width: 150 }` ในสไตล์ของ React Native มีหน่วยวัดเป็นอะไร?

ก. Pixels (px)

ข. Percentage (%)

ค. Points / Density-independent Pixels (dp)

ง. Inches (in)

ข้อ 3: หากต้องการจัดเลย์เอาต์ให้กล่อง View ขนาดกว้าง 100 และสูง 100 มีหน้าตาเป็นวงกลมสมบูรณ์ จะต้องกำหนดคุณสมบัติ `borderRadius` ในสไตล์อย่างไร?

ก. `borderRadius: 100`

ข. `borderRadius: 50`

ค. `borderRadius: 25`

ง. `borderRadius: 10`

ข้อ 4: ข้อใดอธิบายความแตกต่างระหว่างคุณสมบัติ `margin` และ `padding` ได้อย่างถูกต้อง?

ก. `margin` คือขนาดเส้นขอบของวัตถุ ส่วน `padding` คือความโค้งมนของมุมกล่อง

ข. `margin` คือพื้นที่เว้นระยะรอบนอกขอบกล่อง ส่วน `padding` คือพื้นที่เว้นระยะรอบในขอบกล่องไปยังเนื้อหา

ค. `margin` มีผลกับทิศทางแนวตั้งเท่านั้น ส่วน `padding` มีผลเฉพาะทิศทางแนวนอน

ง. `margin` ใช้สำหรับข้อความ ส่วน `padding` ใช้สำหรับรูปภาพ

ข้อ 5: คุณสมบัติ `alignItems` ใน React Native ทำงานและควบคุมการจัดตำแหน่งในแนวแกนใด?

ก. ควบคุมการจัดเรียงในทิศทางแกนหลัก (Main Axis)

ข. ควบคุมการจัดเรียงในทิศทางแกนรอง (Cross Axis)

ค. ควบคุมขนาดของเส้นขอบกล่อง

ง. ควบคุมการหดตัวของขนาดตัวอักษร

ข้อ 6: หากเราเปิดกล่องคอนเทนเนอร์แนวนอนด้วย `flexDirection: 'row'` คุณสมบัติ `justifyContent` จะควบคุมการจัดตำแหน่งและจัดระเบียบข้อมูลตามทิศทางใด?

ก. แนวนอน (ซ้ายไปขวา)

ข. แนวตั้ง (บนลงล่าง)

ค. แนวทะแยงมุม

ง. หมุนตามเข็มนาฬิกา

ข้อ 7: ตัวเลือกข้อใดของ `justifyContent` ที่ช่วยกระจายกล่องตัวเลือกให้อยู่ห่างกันมากที่สุด โดยให้บล็อกแรกและบล็อกสุดท้ายชิดขอบข้างสุดของคอนเทนเนอร์พอดี?

ก. `space-around`

ข. `space-evenly`

ค. `space-between`

ง. `center`

ข้อ 8: เมื่อความกว้างรวมของบล็อกคอมโพเนนต์ลูกมีมากกว่าขนาดพื้นที่ของกล่องแม่ คุณสมบัติใดที่จะต้องตั้งค่าเพิ่มเติมเป็น `'wrap'` เพื่อจัดให้บล็อกที่ล้นหน้าจอตัดขึ้นแถวใหม่โดยอัตโนมัติ?

ก. `flexWrap`

ข. `flexDirection`

ค. `alignContent`

ง. `flexGrow`

ข้อ 9: ใน Expo SDK 54 คุณสมบัติข้อใดที่เพิ่มความสะดวกในการกำหนดระยะห่างระหว่างบล็อกวัตถุในคอนเทนเนอร์แนวตั้งและแนวนอนพร้อมกันโดยไม่ต้องเขียนระยะขอบแยกทีละชิ้น?

ก. `marginHorizontal`

ข. `gap`

ค. `spacing`

ง. `paddings`

ข้อ 10: หากมีบล็อกลูกสองกล่องอยู่ในคอนเทนเนอร์เดียวกัน และกล่องแรกถูกกำหนดให้มีสไตล์ `flex: 1` ในขณะที่กล่องที่สองกำหนดเป็น `flex: 2` ข้อใดอธิบายการจัดสรรสัดส่วนพื้นที่ว่างได้ถูกต้อง?

ก. กล่องแรกจะได้ความยาว 100px และกล่องที่สองได้ 200px

ข. กล่องแรกและกล่องที่สองจะกินพื้นที่เท่ากันโดยสลับกัน

ค. ระบบจะแบ่งพื้นที่เป็น 3 ส่วนเท่า ๆ กัน โดยกล่องแรกจะได้ 1 ส่วน และกล่องที่สองจะได้ 2 ส่วนที่เหลือ

ง. กล่องแรกจะซ้อนอยู่บนกล่องที่สองเนื่องจากค่าดัชนีน้อยกว่า

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: `row` เป็นค่าเริ่มต้นสำหรับ CSS บนบราวเซอร์ทั่วไป แต่ไม่ใช่ของ React Native

• ข. ถูก: React Native ออกแบบมาสำหรับมือถือซึ่งเน้นการอ่านและการเลื่อนลงแนวตั้งเป็นหลัก ดังนั้นทิศทางแกนหลักเริ่มต้น (`flexDirection`) จึงถูกกำหนดเป็น `column` (แนวตั้ง) โดยอัตโนมัติ

• ค. และ ง. ผิด: เป็นรูปแบบการสลับย้อนทางแกน (Reverse) ซึ่งไม่ใช่ค่ามาตรฐานเริ่มต้น

เฉลย ข้อ 2: ตอบ ค.

• เหตุผลประกอบ:

• **ก. ผิด:** React Native ไม่ได้กำหนดสไตล์ด้วยพิกเซลกายภาพตรงตัวเพราะขนาดจอและปริมาณพิกเซลแต่ละเครื่องต่างกันมาก

• **ข. ผิด:** เปอร์เซ็นต์จะทำงานเฉพาะเมื่อใส่ค่าเป็นข้อความสตริง เช่น `'50%'` เท่านั้น

• **ค. ถูก:** ค่าตัวเลขในสไตล์ชีทของ React Native คือหน่วยจุดวัดเชิงพื้นที่ที่ไม่ขึ้นกับความหนาแน่นพิกเซลของจอแต่ละรุ่น (เรียกว่า Points ใน iOS หรือ Density-independent Pixels - dp ใน Android) ซึ่งจะสเกลพิกเซลจริงขึ้นตามหน้าจออุปกรณ์ปลายทางให้อัตโนมัติ

• **ง. ผิด:** นิ้วไม่ใช่หน่วยวัดของเลย์เอาต์หน้าจอแอปพลิเคชัน

เฉลย ข้อ 3: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** การตั้งค่าเท่าความกว้างจะทำให้โค้งมนล้นหรืออาจกลายเป็นรูปแคปซูลหากสัดส่วนอื่นเบี้ยว

• **ข. ถูก:** ตามสูตรเลขคณิตการตัดมุมกล่องสี่เหลี่ยมจัตุรัสให้กลายเป็นวงกลมสมบูรณ์ขอบโค้งมน (`borderRadius`) จะต้องมีขนาดเท่ากับ **ครึ่งหนึ่ง** ของความยาวกว้างและสูง (กว้าง 100, สูง 100 -> ครึ่งหนึ่งคือ 50)

• **ค. และ ง. ผิด:** จะได้มุมกล่องโค้งงอเฉย ๆ แต่ขอบยังเป็นทรงสี่เหลี่ยมด้านเท่า

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ขอบวัตถุและมุมโค้งใช้อีกคลาสคือ `borderWidth` และ `borderRadius`

• **ข. ถูก:** ในสถาปัตยกรรม Box Model ของการวางเลย์เอาต์ `margin` (ระยะขอบภายนอก) จะช่วยสร้างช่องไฟว่าง ๆ ผลักตัวบล็อกให้ออกห่างจากกล่องอื่น ส่วน `padding` (ระยะขอบภายใน) จะใช้เว้นช่องไฟภายในกล่องสีของตนเองเพื่อไม่ให้ตัวหนังสือชิดขอบในเกินไป

• **ค. และ ง. ผิด:** ทั้งสองคำสั่งทำงานได้ทุกแนวทิศทางและควบคุมช่องว่างไม่ได้จำกัดประเภทเนื้อหา

เฉลย ข้อ 5: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** แกนหลักจะถูกควบคุมและเรียงบล็อกผ่านคุณสมบัติ `justifyContent`

• **ข. ถูก:** `alignItems` จะทำหน้าที่ควบคุมและจัดแนวระเบียบให้กับคอมโพเนนต์ย่อยตามทิศทางแนวแกนรอง (Cross Axis) เสมอ (เช่น หากแถวเรียงตัวแนวนอน แกนรองจะเป็นแนวตั้ง)

• **ค. และ ง. ผิด:** ควบคุมขนาดเส้นขอบและตัวอักษรจะใช้ `borderWidth` และ `fontSize`

เฉลย ข้อ 6: ตอบ ก.

• เหตุผลประกอบ:

• ก. ถูก: หากมีการกำหนด `flexDirection: 'row'` (เรียงแนวนอน) ทิศทางแกนหลัก (Main Axis) จะสลับมาอยู่ทางแนวนอนทันที ส่งผลให้ `justifyContent` ทำหน้าที่ผลักหรือจัดบล็อกเรียงชิดตามแนวนอน (ซ้าย-ขวา) ไปด้วย

• ข. ผิด: แกนแนวตั้งจะกลายเป็นแกนรอง (Cross Axis) และถูกควบคุมโดย `alignItems` แทน

• ค. และ ง. ผิด: ระบบ Flexbox ไม่มีการจัดมุมหรือหมุนทิศทางทะแยงในการทำงานหลัก

เฉลย ข้อ 7: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: `space-around` จะแบ่งช่องว่างระหว่างบล็อกเท่า ๆ กันและเหลือช่องว่างข้างหน้า/ท้ายครึ่งหนึ่ง

• ข. ผิด: `space-evenly` จะแบ่งช่องว่างให้กว้างเท่ากันทั้งหมดอย่างเท่าเทียมรวมถึงขอบริมด้วย

• ค. ถูก: `space-between` จะกระจายบล็อกลูกชิดซ้ายและชิดขวาสุดริมขอบหน้าจอพอดี และกระจายสัดส่วนสเปซช่องว่างที่เหลือไว้คั่นตรงกึ่งกลางระว่างกล่องทั้งหมดเท่ากัน

• ง. ผิด: `center` จะรวมกล่องทั้งหมดชิดกึ่งกลางแกน

เฉลย ข้อ 8: ตอบ ก.

• เหตุผลประกอบ:

• ก. ถูก: คุณสมบัติ `flexWrap: 'wrap'` จะคอยสั่งการตัว Bunder เลย์เอาต์ว่า หากความกว้างสะสมของคอมโพเนนต์ลูกเกินพื้นที่คอนเทนเนอร์แม่ ให้ผลักบล็อกถัดไปลงมาขึ้นแถวใหม่ด้านล่างทันที

• ข. ผิด: `flexDirection` ใช้กำหนดเพียงทิศทางการไหลหลักของแกนว่าวางตัวแนวตั้งหรือนอน

• ค. และ ง. ผิด: ใช้สำหรับคำสั่งสัดส่วนการหดและจัดแนวย่อยเมื่อมีหลายแถว

เฉลย ข้อ 9: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: `marginHorizontal` จะเว้นขอบริมภายนอกทั้งซ้ายและขวาของกล่องแต่ละตัว แต่ไม่มีผลกึ่งกลางระหว่างบล็อกย่อย

• **ข. ถูก:** คำสั่ง `gap` (มีพร้อมใช้งานใน SDK ใหม่) ช่วยให้นักพัฒนาสร้างช่องไฟเว้นระหว่างบล็อกย่อยภายในคอนเทนเนอร์ Flexbox ได้ง่ายที่สุด โดยจะคั่นระหว่างบล็อกเท่านั้น โดยไม่ต้องคอยระบุหักลบขอบริมด้านซ้ายขวาของแต่ละกล่อง

• **ค. และ ง. ผิด:** ไม่ใช่ชื่อคุณสมบัติมาตรฐานที่จัดทำในระบบสไตล์ React Native

เฉลย ข้อ 10: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ค่าตัวเลขใน `flex` ไม่ได้หมายความถึงพิกเซล (px) แบบตายตัว แต่เป็นหน่วยอัตราส่วน

• **ข. ผิด:** จะกินสัดส่วนขนาดที่ไม่เท่ากันเพราะตัวเลขต่างกัน

• **ค. ถูก:** คุณสมบัติ `flex` ทำงานโดยหาผลรวมค่าสะสมทั้งหมด (ในโจทย์ 1 + 2 = 3 ส่วน) แล้วแบ่งสเกลสัดส่วนพื้นที่ว่างออกเป็นจำนวนส่วนรวมนั้น จากนั้นจึงนำไปปันสัดส่วนให้กับกล่องตามค่าที่ระบุ (กล่องแรกได้ 1/3 ส่วน และกล่องที่สองได้ 2/3 ส่วน)

• **ง. ผิด:** การซ้อนทับภาพขึ้นอยู่กับลำดับการโค้ดหรือคุณสมบัติ `zIndex` ไม่ใช่ `flex`

ปฏิบัติการท้ายบทเรียน: การจัดเลย์เอาต์หน้าจอแดชบอร์ดจำลองด้วย Flexbox

ปฏิบัติการนี้จะฝึกทักษะการประยุกต์ใช้ระบบ Flexbox ในการออกแบบโครงสร้างส่วนจัดตำแหน่งต่าง ๆ บนหน้าจอจริง โดยจำลองหน้าอินเทอร์เฟซของ **แอปพลิเคชันแดชบอร์ดควบคุมสมาร์ทโฮม (Smart Home Dashboard Mockup)** ผ่านคอมโพเนนต์พื้นฐาน `View` และ `Text`

1. วัตถุประสงค์การเรียนรู้

1. สามารถใช้คุณสมบัติ `flexDirection` ในการวางวัตถุทั้งในแนวแกนตั้งและแนวนอนได้ตามโจทย์กำหนด

2. สามารถใช้คุณสมบัติ `justifyContent` และ `alignItems` ในการกระจายบล็อกสีและจัดวัตถุกึ่งกลางหน้าจอได้อย่างถูกต้อง

3. สามารถใช้งานคุณสมบัติ `flexWrap` และ `gap` เพื่อสร้างรูปแบบ Grid ตารางเมนู 2x2 ได้สำเร็จ

4. มีความเข้าใจหลักการจัดลำดับการเขียนและประกอบสไตล์ชีทภายนอกด้วย `StyleSheet.create`

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนจะต้องเขียนโค้ดเพื่อสร้างหน้าจอแดชบอร์ดจำลองในไฟล์ `app/index.tsx` (หลังจากรัน `npm run reset-project` จากบทที่ 1 แล้ว) โดยมีรายละเอียดเลย์เอาต์ 4 ส่วนย่อยหลัก ดังนี้:

```
+-----------------------------------------+
|  [Header Bar] (Title & Notification Dot) |
+-----------------------------------------+
|  [Status Summary Card] (Full Width)      |
+-----------------------------------------+
|  [2x2 Menu Grid Block]                   |
|  - [Menu 1 (47%)]    - [Menu 2 (47%)]    |
|  - [Menu 3 (47%)]    - [Menu 4 (47%)]    |
+-----------------------------------------+
|  [Footer Navigation Bar]                 |
+-----------------------------------------+
```

รายละเอียดเงื่อนไขการจัดสไตล์:

1. **ธีมพื้นหลังแอปพลิเคชัน:** กำหนดเป็นสีเทาดำหรูหรา (`#0f172a`)
2. **ส่วนหัว (Header Bar):**
   - จัดเรียงแบบแนวนอน (`flexDirection: 'row'`)
   - แบ่งตัวหนังสือชิดซ้ายและขวาสุดริมขอบ (`justifyContent: 'space-between'`)
   - มีชื่อแอป "HOME CONTROL" และฝั่งขวาแสดงจุดไอคอนกลมสีเขียวนีออนแสดงสถานะ Online
3. **การ์ดสรุปสถานะ (Status Summary Card):**
   - กำหนดความกว้างเต็มจอ และมีมุมโค้งมน
   - กำหนดสีพื้นหลังการ์ดเป็นสี Slate กึ่งโปร่งใส (`#1e293b`)
   - ภายในประกอบด้วยข้อความบอกค่าอุณหภูมิ เช่น "24°C - ปกติ"
4. **ตารางบล็อกเมนูย่อย (2x2 Grid):**
   - บล็อกปุ่มเมนูทั้ง 4 ปุ่มมีขนาดความกว้างบล็อกละ **47%** ของพื้นที่ เพื่อให้เรียงตัวคู่กันสองข้างได้พอดี
   - จัดการล้นของวัตถุด้วย `flexWrap: 'wrap'` และมีช่องไฟเว้นระหว่างบล็อกสีเท่ากับ **16** พิกเซล
   - แต่ละบล็อกสีมีพื้นหลังคนละสีกันเพื่อแสดงความเป็นเมนูเฉพาะด้าน
5. **แถบส่วนท้าย (Footer Navigation Bar):**

• ลอยตัวชิดด้านล่างสุดของหน้าจอ

• แสดงแท็บจำลอง 3 เมนู (เช่น "Home", "Devices", "Settings") เรียงแนวนอน กระจายระยะห่างสเปซเท่ากันทั้งหมด (`justifyContent: 'space-evenly'`)

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: เตรียมโปรเจคเริ่มต้น

ตรวจสอบความพร้อมของไฟล์ `app/index.tsx` หากมีโค้ดเก่าอยู่ให้ลบออก หรือเปิดเซิร์ฟเวอร์ด้วยคำสั่ง `npx expo start` เพื่อสแตนด์บายทดสอบผลลัพธ์ผ่านหน้าจออุปกรณ์จริงของคุณ

ขั้นตอนที่ 2: เขียนโค้ดแบ่งโครงสร้างหลัก (Layout Partitioning)

คัดลอกตัวอย่างโครงสร้างโค้ดเริ่มต้นนี้ลงไปใน `app/index.tsx` เพื่อแบ่งโครงสร้างหลักทั้ง 4 ส่วน และทดลองวิเคราะห์ว่าแกนหลักแกนรองทำงานอย่างไร:

```
import { StyleSheet, Text, View } from 'react-native';

export default function SmartHomeDashboard() {
  return (
    <View style={styles.container}>

      {/* 1. Header Bar */}
      <View style={styles.header}>
        <Text style={styles.headerText}>HOME CONTROL</Text>
        <View style={styles.statusContainer}>
          <View style={styles.statusDot} />
          <Text style={styles.statusDotText}>ONLINE</Text>
        </View>
      </View>

      {/* 2. Status Summary Card */}
      <View style={styles.statusCard}>
        <Text style={styles.cardTitle}>อุณหภูมิห้องโดยรวม</Text>
        <Text style={styles.cardValue}>24°C</Text>
        <Text style={styles.cardStatus}>ระบบปรับอากาศกำลังทำงานระดับปกติ</Text>
      </View>

      {/* 3. 2x2 Menu Grid Block */}
      <View style={styles.menuGrid}>

        {/* บล็อกเมนู 1 */}
        <View style={[styles.menuItem, { backgroundColor: '#ea580c' }]}>
          <Text style={styles.menuEmoji}>💡</Text>
          <Text style={styles.menuText}>ห้องนั่งเล่น</Text>
        </View>

        {/* บล็อกเมนู 2 */}
```

```
          <View style={[styles.menuItem, { backgroundColor: '#2563eb' }]}>
            <Text style={styles.menuEmoji}>🔒</Text>
            <Text style={styles.menuText}>ความปลอดภัย</Text>
          </View>

          {/* บล็อกเมนู 3 */}
          <View style={[styles.menuItem, { backgroundColor: '#16a34a' }]}>
            <Text style={styles.menuEmoji}>🔌</Text>
            <Text style={styles.menuText}>เครื่องใช้ไฟฟ้า</Text>
          </View>

          {/* บล็อกเมนู 4 */}
          <View style={[styles.menuItem, { backgroundColor: '#db2777' }]}>
            <Text style={styles.menuEmoji}>🎵</Text>
            <Text style={styles.menuText}>เครื่องเสียง</Text>
          </View>

        </View>

        {/* 4. Footer Navigation Bar */}
        <View style={styles.footer}>
          <Text style={styles.footerItemActive}>หน้าหลัก</Text>
          <Text style={styles.footerItem}>อุปกรณ์</Text>
          <Text style={styles.footerItem}>ตั้งค่า</Text>
        </View>

      </View>
    );
}

const styles = StyleSheet.create({
  // ให้ผู้เรียนระบุสไตล์ชีทสำหรับจัดแนวทาง Flexbox ที่ขั้นตอนถัดไป
});
```

ขั้นตอนที่ 3: ระบุสไตล์และคลาสการจัดตำแหน่งด้วย Flexbox

เขียนข้อกำหนดสไตล์ลงในฟังก์ชัน StyleSheet.create ด้านล่างสุดของไฟล์ app/index.tsx ดังนี้ เพื่อจัดตำแหน่งกล่องตามโจทย์กำหนด:

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a', // ธีมพื้นหลัง Dark Mode Slate
    paddingTop: 60,             // เว้นหลบหน้าจอด้านบน
    paddingHorizontal: 20,
    justifyContent: 'space-between', // ช่วยดึงส่วนหัว ตัวเนื้อหา และส่วนท้ายแยกชิดขอบห่างจากกัน
    paddingBottom: 20,
  },
```

```
  header: {
    flexDirection: 'row',          // เรียงแนวนอน
    justifyContent: 'space-between', // ดึงหัวหนังสือชิดซ้าย จุดเขียวชิดขวา
    alignItems: 'center',          // จัดวัตถุกึ่งกลางแนวแกนรอง (แนวตั้ง)
    borderBottomWidth: 1,
    borderColor: '#1e293b',
    paddingBottom: 12,
  },
  headerText: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
    letterSpacing: 1,
  },
  statusContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
  },
  statusDot: {
    width: 8,
    height: 8,
    borderRadius: 4,               // ทำสัญลักษณ์วงกลมสมบูรณ์
    backgroundColor: '#22c55e',    // สีเขียวนีออน
  },
  statusDotText: {
    color: '#22c55e',
    fontSize: 12,
    fontWeight: 'bold',
  },
  statusCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 20,
    width: '100%',
    borderWidth: 1,
    borderColor: '#334155',
    marginVertical: 10,
  },
  cardTitle: {
    color: '#94a3b8',
    fontSize: 14,
    marginBottom: 4,
  },
  cardValue: {
    color: '#ffffff',
    fontSize: 32,
    fontWeight: 'bold',
    marginBottom: 4,
  },
  cardStatus: {
    color: '#38bdf8',
    fontSize: 12,
  },
```

```
  menuGrid: {
    flexDirection: 'row',          // จัดวัตถุลูกวางแนวนอน
    flexWrap: 'wrap',              // ถ้าวัตถุเกินความกว้าง ให้ตัดขึ้นแถวใหม่
    gap: 16,                       // เว้นระยะขอบรอบทุกด้าน
    justifyContent: 'space-between', // ผลักสัดส่วนเมนูสองฝั่งให้ออกชิดขอบ
    marginVertical: 10,
  },
  menuItem: {
    width: '47%',                  // ความกว้างกล่องต่อชิ้น (แบ่งคู่ได้ 2 บล็อกพอดีและเหลือที่เว้น
gap)
    height: 110,
    borderRadius: 16,
    padding: 16,
    justifyContent: 'space-between', // ช่วยดึง Emoji อยู่บน ชื่อข้อความอยู่ข้างล่าง
  },
  menuEmoji: {
    fontSize: 28,
  },
  menuText: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 16,
  },
  footer: {
    flexDirection: 'row',
    justifyContent: 'space-evenly', // กระจายพื้นที่ว่างแบ่งเท่า ๆ กันระหว่าง 3 ปุ่มเมนู
    backgroundColor: '#1e293b',
    paddingVertical: 14,
    borderRadius: 24,
    borderWidth: 1,
    borderColor: '#334155',
  },
  footerItem: {
    color: '#64748b',
    fontWeight: 'bold',
  },
  footerItemActive: {
    color: '#38bdf8',              // ปรับสีเมนูแรกเป็นสีฟ้านีออนแสดงการแอ็คทีฟหน้าปัจจุบัน
    fontWeight: 'bold',
  },
});
```

ขั้นตอนที่ 4: ทดสอบการทำงานและปรับเปลี่ยนพฤติกรรมเลย์เอาต์ (Experiment)

ให้ผู้เรียนบันทึกไฟล์และสังเกตแดชบอร์ดจำลองบนสมาร์ทโฟน จากนั้นให้ทดลองทดสอบพฤติกรรมสไตล์ดังนี้:

1. ทดลองเปลี่ยนค่า `flexDirection: 'row'` ในสไตล์ `menuGrid` ให้เป็น `'column'` สังเกตว่าเมนู 4 บล็อกถูกจัดเรียงอย่างไร

2. ลองแก้ไขค่า `gap: 16` ในเมนูตารางให้ลดลงเหลือ 8 หรือเปลี่ยนเป็น `rowGap` และ `columnGap` เพื่อเข้าใจความยืดหยุ่นของตัวแปร

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถคัดลอกโค้ดโครงสร้างที่รวมคอมโพเนนต์และสไตล์ชีทเข้าด้วยกันฉบับสมบูรณ์ด้านล่างนี้ ไปใส่แทนที่ในไฟล์ `app/index.tsx` เพื่อตรวจสอบการแสดงผลหน้าจอแดชบอร์ดควบคุมสมาร์ทโฮมที่ถูกต้อง:

```
import { StyleSheet, Text, View } from 'react-native';

export default function SmartHomeDashboard() {
  return (
    <View style={styles.container}>

      {/* 1. Header Bar */}
      <View style={styles.header}>
        <Text style={styles.headerText}>HOME CONTROL</Text>
        <View style={styles.statusContainer}>
          <View style={styles.statusDot} />
          <Text style={styles.statusDotText}>ONLINE</Text>
        </View>
      </View>

      {/* 2. Status Summary Card */}
      <View style={styles.statusCard}>
        <Text style={styles.cardTitle}>อุณหภูมิห้องโดยรวม</Text>
        <Text style={styles.cardValue}>24°C</Text>
        <Text style={styles.cardStatus}>ระบบปรับอากาศกำลังทำงานระดับปกติ</Text>
      </View>

      {/* 3. 2x2 Menu Grid Block */}
      <View style={styles.menuGrid}>

        {/* บล็อกเมนู 1 */}
        <View style={[styles.menuItem, { backgroundColor: '#ea580c' }]}>
          <Text style={styles.menuEmoji}>💡</Text>
          <Text style={styles.menuText}>ห้องนั่งเล่น</Text>
        </View>

        {/* บล็อกเมนู 2 */}
        <View style={[styles.menuItem, { backgroundColor: '#2563eb' }]}>
          <Text style={styles.menuEmoji}>🔒</Text>
          <Text style={styles.menuText}>ความปลอดภัย</Text>
        </View>

        {/* บล็อกเมนู 3 */}
        <View style={[styles.menuItem, { backgroundColor: '#16a34a' }]}>
          <Text style={styles.menuEmoji}>🔌</Text>
```

```
          <Text style={styles.menuText}>เครื่องใช้ไฟฟ้า</Text>
        </View>

        {/* บล็อกเมนู 4 */}
        <View style={[styles.menuItem, { backgroundColor: '#db2777' }]}>
          <Text style={styles.menuEmoji}>🎵</Text>
          <Text style={styles.menuText}>เครื่องเสียง</Text>
        </View>

      </View>

      {/* 4. Footer Navigation Bar */}
      <View style={styles.footer}>
        <Text style={styles.footerItemActive}>หน้าหลัก</Text>
        <Text style={styles.footerItem}>อุปกรณ์</Text>
        <Text style={styles.footerItem}>ตั้งค่า</Text>
      </View>

    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a', // ธีมพื้นหลัง Dark Mode Slate
    paddingTop: 60,             // เว้นหลบหน้าจอด้านบน
    paddingHorizontal: 20,
    justifyContent: 'space-between', // ช่วยดึงส่วนหัว ตัวเนื้อหา และส่วนท้ายแยกชิดขอบห่างจากกัน
    paddingBottom: 20,
  },
  header: {
    flexDirection: 'row',         // เรียงแนวนอน
    justifyContent: 'space-between', // ดึงหัวหนังสือชิดซ้าย จุดเขียวชิดขวา
    alignItems: 'center',         // จัดวัตถุกึ่งกลางแนวแกนรอง (แนวตั้ง)
    borderBottomWidth: 1,
    borderColor: '#1e293b',
    paddingBottom: 12,
  },
  headerText: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
    letterSpacing: 1,
  },
  statusContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
  },
```

```
  statusDot: {
    width: 8,
    height: 8,
    borderRadius: 4,              // ทำสัญลักษณ์วงกลมสมบูรณ์
    backgroundColor: '#22c55e',   // สีเขียวนีออน
  },
  statusDotText: {
    color: '#22c55e',
    fontSize: 12,
    fontWeight: 'bold',
  },
  statusCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 20,
    width: '100%',
    borderWidth: 1,
    borderColor: '#334155',
    marginVertical: 10,
  },
  cardTitle: {
    color: '#94a3b8',
    fontSize: 14,
    marginBottom: 4,
  },
  cardValue: {
    color: '#ffffff',
    fontSize: 32,
    fontWeight: 'bold',
    marginBottom: 4,
  },
  cardStatus: {
    color: '#38bdf8',
    fontSize: 12,
  },
  menuGrid: {
    flexDirection: 'row',         // จัดวัตถุลูกวางแนวนอน
    flexWrap: 'wrap',             // ถ้าวัตถุเกินความกว้าง ให้ตัดขึ้นแถวใหม่
    gap: 16,                      // เว้นระยะขอบรอบทุกด้าน
    justifyContent: 'space-between', // ผลักสัดส่วนเมนูสองฝั่งให้ออกชิดขอบ
    marginVertical: 10,
  },
  menuItem: {
    width: '47%',                 // ความกว้างกล่องต่อชิ้น (แบ่งคู่ได้ 2 บล็อกพอดีและเหลือที่เว้น
gap)
    height: 110,
    borderRadius: 16,
    padding: 16,
    justifyContent: 'space-between', // ช่วยดึง Emoji อยู่บน ชื่อข้อความอยู่ข้างล่าง
  },
  menuEmoji: {
    fontSize: 28,
  },
```

```
  menuText: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 16,
  },
  footer: {
    flexDirection: 'row',
    justifyContent: 'space-evenly', // กระจายพื้นที่ว่างแบ่งเท่า ๆ กันระหว่าง 3 ปุ่มเมนู
    backgroundColor: '#1e293b',
    paddingVertical: 14,
    borderRadius: 24,
    borderWidth: 1,
    borderColor: '#334155',
  },
  footerItem: {
    color: '#64748b',
    fontWeight: 'bold',
  },
  footerItemActive: {
    color: '#38bdf8',              // ปรับสีเมนูแรกเป็นสีฟ้านีออนแสดงการแอ็คทีฟหน้าปัจจุบัน
    fontWeight: 'bold',
  },
});
```


เอกสารอ้างอิง

Expo. (2024). *User Interface and Styling*. จาก https://docs.expo.dev/versions/v54.0.0/develop/user-interface/overview/

React Native. (2024). *Height and Width*. จาก https://reactnative.dev/docs/height-and-width

React Native. (2024). *Layout with Flexbox*. จาก https://reactnative.dev/docs/flexbox

แผนบริหารการสอนประจำบทที่ 3

หัวข้อเนื้อหาประจำบท

3.1 คอมโพเนนต์แสดงผลรูปภาพ Image (Local และ Network Assets)

3.2 คอมโพเนนต์รับข้อมูลข้อความ TextInput และการควบคุมคีย์บอร์ด

3.3 การสร้างพื้นที่เลื่อนดูข้อมูลด้วย ScrollView เปรียบเทียบกับ View

3.4 การจัดการเหตุการณ์การสัมผัสและการกดปุ่มด้วย Pressable

3.5 การหลีกเลี่ยงพื้นที่ขอบจอด้วย SafeAreaView

3.6 แนวคิดการสร้าง Custom Component และการกำหนด Props Interface ด้วย TypeScript

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถอธิบายวิธีแสดงผลภาพจากสองแหล่งข้อมูล (Local และ Network URI) และการปรับสัดส่วนรูปภาพได้ถูกต้อง
2. ผู้เรียนสามารถใช้ TextInput ในการรับข้อมูลประเภทต่าง ๆ และผูกเหตุการณ์ตรวจสอบข้อความผ่านคีย์บอร์ดได้สำเร็จ
3. ผู้เรียนสามารถระบุความแตกต่างและเปรียบเทียบกรณีใช้งานที่เหมาะสมระหว่าง View และ ScrollView ได้อย่างถูกต้อง
4. ผู้เรียนสามารถสร้างปุ่มสัมผัสที่ตอบสนองเมื่อถูกกดโดยประยุกต์ใช้คอมโพเนนต์ Pressable ร่วมกับการจัดระเบียบขอบเขตจอด้วย SafeAreaView ได้อย่างถูกต้อง
5. ผู้เรียนสามารถสร้าง Custom Component และเขียนโค้ดส่งผ่านค่าข้อมูล (Props) ไปทำงานภายในคอมโพเนนต์ย่อยได้อย่างมีหลักการ
6. ผู้เรียนสามารถเขียนโครงสร้าง Interface ใน TypeScript เพื่อกำกับและตรวจสอบความถูกต้องของประเภทข้อมูล Props ที่ส่งเข้ามาได้ถูกต้องสมบูรณ์

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

9. **การบรรยายประกอบการสาธิตเชิงปฏิบัติ (Live Coding Demonstration):**

ผู้สอนสาธิตวิธีการแสดงผลรูปภาพ การพิมพ์รับค่าข้อความ การจัดการคีย์บอร์ด ตลอดจนวิธีการแยกโค้ดสร้าง Custom Component และการส่งค่าข้อมูลผ่าน Props

10. **กิจกรรมต่อยอดโค้ดทีละส่วน (Guided Coding Practice):** ให้ผู้เรียนทดลองสร้างคอมโพเนนต์ปุ่มสัมผัสเป็นของตัวเอง (Custom Button) โดยเปิดให้ส่งข้อความหน้าปุ่มและสีปุ่มต่าง ๆ ผ่านคุณสมบัติ Props เข้าไปปรับเปลี่ยนสไตล์
11. **การลงมือปฏิบัติทดสอบปฏิสัมพันธ์ (Interactive Interaction Lab):** มอบหมายให้ผู้เรียนเขียนแบบฟอร์มรับค่าจำลอง โดยเรียกใช้ Custom Components ที่สร้างขึ้น และจัดส่งผ่านโครงสร้าง Props ของข้อมูลผู้เรียนไปทำงาน
12. **กิจกรรมเรียนรู้เรื่อง Type-Safety (Props Interface Practice):** ฝึกเขียนและจำลองข้อผิดพลาด (Debug) เมื่อส่งประเภทข้อมูลตัวแปร Props ผิดกฎที่ระบุไว้ใน TypeScript Interface

สื่อการเรียนการสอน

9. เอกสารสไลด์นำเสนอหัวข้อ "Core Components and Custom Components with Props in React Native"
10. วิดีโอแนะนำแนวคิดและการแยกส่วนคอมโพเนนต์ (Component Modularization) ด้วย Props
11. ชุดไอคอนรูปภาพประกอบการทดลองรันคอมโพเนนต์ Image
12. ตัวอย่างซอร์สโค้ดการแยกไฟล์ Custom Button และส่งค่า Props บน Git Repository

การวัดผลและการประเมินผล

1. การวัดผล

1.1 ประเมินผลจากการทำคะแนนแบบทดสอบปรนัยท้ายบทเรียน 10 ข้อ

1.2 ประเมินทักษะการแยกสร้าง Custom Component และการส่งผ่านตัวแปร Props ระหว่างคอมโพเนนต์ในห้องเรียน

1.3 ตรวจสอบความถูกต้องของการใช้งานคอมโพเนนต์ Image, TextInput และการตั้งค่า Props Interface ด้วย TypeScript ในแล็บปฏิบัติการท้ายบท

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถสร้าง Custom Component ที่รับส่งค่า Props ได้ถูกต้อง และรันหน้าจอแอปแบบฟอร์มที่มีรูปภาพและกล่องพิมพ์ข้อความได้สำเร็จผ่านเกณฑ์ (ผ่าน/ไม่ผ่าน)

2.3 การสังเกตพฤติกรรมความตั้งใจเรียน ความพยายามในการดีบักข้อผิดพลาดของการส่งค่าตัวแปร และการมีส่วนร่วมในชั้นเรียน

บทที่ 3

คอมโพเนนต์พื้นฐานและการจัดการปฏิสัมพันธ์กับผู้ใช้

ในสภาพแวดล้อมการพัฒนาแอปพลิเคชันด้วย React Native คอมโพเนนต์พื้นฐาน (Core Components) ทำหน้าที่เป็นตัวเชื่อมต่อ (Bridge) ไปยังมุมมองดั้งเดิม (Native UI Views) ของระบบปฏิบัติการ iOS (เช่น **UIView**, **UIImageView**, **UITextField**) และ Android (เช่น **View**, **ImageView**, **EditText**) (React Native, 2024) ส่งผลให้แอปพลิเคชันสามารถประมวลผลอินเทอร์เฟซด้วยประสิทธิภาพระดับเนทีฟอย่างแท้จริง

3.1 คอมโพเนนต์แสดงผลรูปภาพ Image (Local และ Network Assets)

คอมโพเนนต์ **Image** ใช้สำหรับการแสดงผลรูปภาพในแอปพลิเคชัน (React Native, 2024) ทั้งภาพประเภทสเตติกภายในเครื่อง (Static Local Assets) และภาพที่ดาวน์โหลดผ่านอินเทอร์เน็ต (Network Images)

1. การแสดงผลรูปภาพจาก Local Assets

เป็นการดึงภาพที่เก็บรวมอยู่ในโฟลเดอร์ของโปรเจกต์ โดยเรียกใช้ฟังก์ชัน **require()** ในการอ้างอิงตำแหน่งไฟล์ ซึ่งระบบจะประมวลผลขนาดของรูปภาพให้โดยอัตโนมัติตอนทำการบิลด์ (Build Time):

```
import { Image, StyleSheet, View } from 'react-native';

export default function LocalImageExample() {
  return (
    <View style={styles.container}>
      <Image
        source={require('../../assets/images/react-logo.png')}
        style={styles.localImage}
   />
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    justifyContent: 'center',
    alignItems: 'center',
  },
  localImage: {
    width: 100,
    height: 100,
  },
});
```

2. การแสดงผลรูปภาพจากเครือข่ายอินเทอร์เน็ต (Network Images)

เป็นการโหลดรูปภาพผ่านโปรโตคอล HTTP/HTTPS โดยกำหนดค่า `source` เป็นออบเจกต์ที่มีพร็อพส์ `uri`:

ข้อควรจำ (Important): กฎเหล็กของ Network Images: หากเป็นการโหลดภาพผ่าน `uri` ผู้พัฒนา **จำเป็นต้องระบุความกว้าง (width) และความสูง (height) ในสไตล์เสมอ** มิฉะนั้นคอมโพเนนต์จะไม่ทราบขนาดพื้นที่จำลองและไม่แสดงผลภาพขึ้นมา (ขนาดเริ่มต้นจะเป็น 0x0 พิกเซล)

```
<Image
  source={{ uri: 'https://reactnative.dev/img/tiny_logo.png' }}
  style={{ width: 50, height: 50 }}
/>
```

3. การปรับขนาดสัดส่วนด้วย resizeMode

คุณสมบัติ `resizeMode` ทำหน้าที่กำหนดทิศทางว่ารูปภาพจะจัดสัดส่วนอย่างไรเมื่อกรอบของรูปภาพ (สไตล์กว้าง/สูง) มีขนาดต่างจากภาพจริง:

• `cover` (ค่าเริ่มต้น): ยืด/ขยายรูปภาพให้ครอบคลุมเต็มพื้นที่กรอบ โดยรักษาอัตราส่วนภาพไว้ (ทำให้บางส่วนของภาพอาจถูกตัดออก)

• `contain`: ยืด/หดรูปภาพให้อยู่ภายในพื้นที่กรอบได้ครบถ้วนทั้งหมด โดยรักษาอัตราส่วนภาพไว้ (อาจเกิดพื้นที่ว่างซ้าย-ขวา หรือบน-ล่าง)

• `stretch`: ยืดขยายรูปภาพให้เต็มกรอบโดยตรง โดยละทิ้งการรักษาอัตราส่วนเดิม (ภาพจะบิดเบี้ยวได้)

• `center`: จัดกึ่งกลางรูปภาพในเฟรมโดยรักษาขนาดดั้งเดิมไว้ (ไม่มีการยืดขยาย)

3.2 คอมโพเนนต์รับข้อมูลข้อความ TextInput และการควบคุมคีย์บอร์ด

คอมโพเนนต์ `TextInput` ใช้ในการเปิดพื้นที่รับข้อมูลตัวอักษรหรือข้อความจากผู้ใช้งานผ่านแป้นพิมพ์เสมือน (Virtual Keyboard) (React Native, 2024)

1. พร็อพส์พื้นฐานที่จำเป็นสำหรับการสร้างฟอร์ม

• `value`: ตัวแปรสถานะที่เก็บค่าข้อความปัจจุบัน

• `placeholder`: ข้อความแนะนำแบบจางที่จะแสดงเมื่อไม่มีการพิมพ์ข้อความใด ๆ

• `onChangeText`: เหตุการณ์สัมผัสที่จะเรียกฟังก์ชันกลับคืน (Callback Function) ส่งสตริงค่าล่าสุดกลับมาทุกครั้งที่ปุ่มแป้นพิมพ์ถูกกดพิมพ์

• `secureTextEntry`: กำหนดให้เป็น true เพื่อจำลองการกรอกรหัสผ่าน (ตัวอักษรจะเปลี่ยนเป็นสัญลักษณ์จุดวงกลมบดบังสายตา)

```
import { useState } from 'react';
import { TextInput, StyleSheet, View } from 'react-native';

export default function InputExample() {
  const [password, setPassword] = useState('');

  return (
    <View style={styles.container}>
      <TextInput
      placeholder="กรอกรหัสผ่านของคุณ"
        value={password}
        onChangeText={setPassword}
        secureTextEntry={true}
        style={styles.input}
   />
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    padding: 16,
 },
  input: {
    height: 48,
    borderWidth: 1,
    borderColor: '#ccc',
    paddingHorizontal: 12,
    borderRadius: 8,
  },
});
```

2. การควบคุมพฤติกรรมคีย์บอร์ด (Keyboard Control)

ในอุปกรณ์เคลื่อนที่ การจัดเรียงประเภทปุ่มบนแป้นพิมพ์ให้ตรงกับบริบทของอินพุตเป็นสิ่งสำคัญอย่างยิ่งเพื่ออำนวยความสะดวกให้ผู้ใช้งาน:

• `keyboardType`: ระบุรูปแบบคีย์บอร์ดที่ทำงานขึ้นมา

  • 'default': คีย์บอร์ดข้อความปกติ

  • 'numeric': คีย์บอร์ดปุ่มตัวเลข (เหมาะสำหรับการป้อนยอดเงิน รหัส OTP)

  • 'email-address': คีย์บอร์ดที่มีปุ่ม @ และ .com ขึ้นมาอำนวยความสะดวก

  • 'phone-pad': แป้นพิมพ์ปุ่มหมายเลขโทรศัพท์ที่มีเครื่องหมายบวก/ดอกจัน

• `autoCapitalize`: ควบคุมการขึ้นต้นด้วยอักษรตัวใหญ่ภาษาอังกฤษอัตโนมัติ (แนะนำให้ใช้ 'none' สำหรับอีเมลและรหัสผ่าน)

• `autoCorrect`: เปิด/ปิดระบบเดาคำศัพท์และแก้ไขข้อความอัตโนมัติของตัวระบบปฏิบัติการ (true หรือ false)

• `returnKeyType`: เปลี่ยนรูปภาพปุ่มตกลง/ทำรายการที่มุมขวาด้านล่างของคีย์บอร์ด (เช่น 'done', 'next', 'search', 'go')

3.3 การสร้างพื้นที่เลื่อนดูข้อมูลด้วย ScrollView เปรียบเทียบกับ View

โดยปกติแล้ว คอมโพเนนต์ View พื้นฐานจะไม่มีความสามารถในการตอบสนองคำสั่งเลื่อนหน้าจอ (Scrolling) หากเนื้อหาในหน้าจอนั้นมีความยาวเกินกว่าพื้นที่ขอบเขตจอแสดงผลจริง เนื้อหาด้านล่างจะถูกตัดทิ้งและไม่สามารถเลื่อนอ่านได้ (React Native, 2024)

1. พฤติกรรมของ ScrollView

ScrollView คือตู้คอนเทนเนอร์แบบเลื่อนดูเนื้อหาภายในได้ โดยจะทำการแสดงผลแถบแถบเลื่อน (Scrollbar) อัตโนมัติเมื่อเนื้อหาภายในขยายความยาวเกินกว่าขอบเขตหน้าจอ

```
import { ScrollView, Text, StyleSheet } from 'react-native';

export default function LongContent() {
  return (
    <ScrollView style={styles.scroll}>
      <Text style={styles.text}>ย่อหน้าที่ 1...</Text>
      <Text style={styles.text}>ย่อหน้าที่ 2...</Text>
      {/* เพิ่มเนื้อหาจำลองที่มีความยาวลงไปได้เรื่อย ๆ */}
    </ScrollView>
 );
}

const styles = StyleSheet.create({
  scroll: {
    flex: 1,
    backgroundColor: '#fff',
  },
  text: {
    fontSize: 18,
    marginVertical: 40,
  },
});
```

2. การเปรียบเทียบ View กับ ScrollView

ตารางที่ 3.1 การเปรียบเทียบคุณสมบัติระหว่าง View และ ScrollView

คุณสมบัติ	View	ScrollView
การเลื่อนหน้าจอ (Scrolling)	ไม่ทำตามการปัดสัมผัสเลื่อนหน้าจอ	รองรับการเลื่อนแนวตั้งและแนวนอน
พื้นที่ยืดแกนเลย์เอาต์	จัดตำแหน่ง Layout ด้วย Flexbox ได้ทันที	จัดเลย์เอาต์เฉพาะจุดผ่าน contentContainerStyle
การเรนเดอร์หน่วยความจำ	รวดเร็ว แสดงเฉพาะของภายในกล่อง	เรนเดอร์คอมโพเนนต์ลูกทั้งหมดในครั้งเดียวตอนโหลด
กรณีการใช้งานที่เหมาะสม	โครงสร้างบล็อกเลย์เอาต์, ป้าย, พื้นที่ที่ไม่ล้นจอ	หน้าจอกรอกข้อมูลแบบฟอร์ม, หน้าเนื้อหายาว ๆ เช่น ข้อตกลง

คำเตือน (Warning): การใช้สไตล์กับ ScrollView: หากคุณใช้ `style` บน `ScrollView` จะเป็นการจัดลักษณะทางกายภาพภายนอกของตัวเลื่อนเอง แต่หากคุณต้องการกำหนดช่องไฟ ระยะห่าง (Padding) หรือการจัดกึ่งกลางเนื้อหาลูกที่อยู่ข้างในพื้นที่เลื่อน **คุณต้องใช้งานผ่านพร็อพส์ `contentContainerStyle` เท่านั้น**

3.4 การจัดการเหตุการณ์การสัมผัสและการกดปุ่มด้วย Pressable

ในยุคแรกเริ่ม React Native ใช้กลุ่มคอมโพเนนต์ตระกูล Touchable (เช่น `TouchableOpacity`, `TouchableHighlight`) ในการสร้างปุ่ม แต่ในปัจจุบันเวอร์ชันใหม่ ๆ ได้นำเสนอคอมโพเนนต์ที่ยืดหยุ่นและก้าวหน้ากว่าคือ `Pressable` (React Native, 2024) ซึ่งรองรับการตรวจจับระดับการกดสัมผัสในสภาพการใช้งานที่ซับซ้อนขึ้น

1. เหตุการณ์สัมผัสต่าง ๆ บน Pressable

- `onPress`: สัมผัสปุ่มและปล่อยตัวในเวลาปกติ (การคลิกปุ่มทั่วไป)
- `onLongPress`: กดปุ่มสัมผัสแช่ไว้เป็นเวลาอย่างน้อย 500 มิลลิวินาที
- `onPressIn`: สัมผัสเริ่มแรกเมื่อนิ้วเตะโดนปุ่ม
- `onPressOut`: จังหวะที่ยกนิ้วขึ้นจากพื้นที่สัมผัสปุ่ม

2. การจัดการสถานะ Active/Pressed สไตล์ไดนามิก

`Pressable` ยกระดับความสามารถโดยสามารถรับฟังก์ชันในคุณสมบัติ `style` หรือ `children` เพื่อรับทราบข้อมูลสถานะแอปพลิเคชันว่าผู้ใช้งานกำลังเอานิ้วกดคาไว้อยู่หรือไม่ผ่านตัวแปรบูเลียน `{ pressed }`:

```
import { Pressable, Text, StyleSheet } from 'react-native';

export default function CustomButton() {
  return (
    <Pressable
      onPress={() => console.log('Button Pressed!')}
      style={({ pressed }) => [
        styles.button,
        {
     backgroundColor: pressed ? '#1d4ed8' : '#2563eb', // สีเข้มขึ้นเมื่อถูกกด
          transform: [{ scale: pressed ? 0.98 : 1.0 }],    // ย่อตัวลงเล็กน้อยจำลองน้ำหนักกด
        }
   ]}
    >
      <Text style={styles.text}>บันทึกข้อมูล</Text>
    </Pressable>
 );
}

const styles = StyleSheet.create({
  button: {
    paddingVertical: 12,
    paddingHorizontal: 24,
    borderRadius: 8,
    alignItems: 'center',
  },
  text: {
    color: 'white',
    fontWeight: 'bold',
    fontSize: 16,
  },
});
```

3.5 การหลีกเลี่ยงพื้นที่ขอบจอด้วย SafeAreaView

อุปกรณ์สมาร์ทโฟนยุคใหม่มีดีไซน์ที่หลากหลาย เช่น หน้าจอไร้ขอบที่มีรอยบาก (Notch) มีกล้องทรงหยดน้ำ หรือแถบนำทางระบบปฏิบัติการอยู่ด้านล่างหน้าจอ หากเขียนวางคอมโพเนนต์ชิดขอบบนหรือขอบล่างเกินไป เนื้อหาอาจจะถูกกล้อง รอยบาก หรือแถบสถานะ (Status Bar) บดบัง

การใช้งาน SafeAreaView

คอมโพเนนต์ **SafeAreaView** จะคำนวณและเว้นพื้นที่ระยะขอบปลอดภัยเฉพาะอุปกรณ์นั้น ๆ ให้อัตโนมัติ (React Native, 2024) (มีผลเด่นชัดบนระบบ iOS) ทำให้ผู้พัฒนาไม่ต้องเขียนสไตล์ขยับความสูงด้วยค่าคงที่ตายตัว:

```
import { SafeAreaView, View, Text, StyleSheet } from 'react-native';

export default function SafeLayout() {
  return (
    <SafeAreaView style={styles.safeArea}>
      <View style={styles.content}>
        <Text>เนื้อหาของแอปพลิเคชันที่ได้รับการปกป้องจากรอยบากหน้าจอ</Text>
      </View>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#f8fafc', // ให้พื้นที่ปลอดภัยเป็นสีเดียวกับแอป
  },
  content: {
    flex: 1,
    padding: 16,
  },
});
```

3.6 แนวคิดการสร้าง Custom Component และการกำหนด Props Interface ด้วย TypeScript

การพัฒนาแอปพลิเคชันที่มีขนาดใหญ่ขึ้นจำเป็นต้องคำนึงถึงความสะดวกในการบำรุงรักษาโค้ดและการนำชิ้นส่วนอินเตอร์เฟซกลับมาใช้ซ้ำ (Component Reusability) โดยสร้าง **Custom Components** และส่งผ่านค่าข้อมูลความต้องการผ่าน **Props**

1. แนวคิดการแยกสร้าง Custom Component

สมมติว่าเราต้องการสร้างการ์ดแนะนำข้อมูลผู้ใช้งานที่มีรูปโปรไฟล์ ชื่อ และปุ่มติดต่อ หากประกาศไว้ทั้งหมดบนหน้าหลัก โค้ดจะยาวและซ้ำซ้อน เราจึงควรแยกสร้างเป็นไฟล์ย่อยภายนอก

2. การประกาศคุณสมบัติและกำหนดโครงสร้างข้อมูลด้วย TypeScript Interface

ในระดับมืออาชีพการใช้ TypeScript ร่วมกับ React Native จะช่วยควบคุมคุณภาพการพัฒนาได้อย่างดีเยี่ยม โดยจะนำ `interface` มากำหนดเป็นตัวกรองข้อมูลว่าคอมโพเนนต์ย่อยนั้น ๆ ต้องการตัวแปรชนิดใดบ้าง และตัวแปรใดเป็นสิ่งจำเป็นหรือมีตัวเลือกอื่น ๆ:

```
// 1. สร้างไฟล์ย่อยที่ตั้งชื่อว่า components/UserProfileCard.tsx
import { View, Text, Image, StyleSheet, Pressable } from 'react-native';

// กำหนดประเภทข้อมูล Props ด้วย TypeScript Interface
```

```
export interface UserProfileCardProps {
  name: string;
  avatarUrl: string;
  role: string;
  isActive?: boolean; // เครื่องหมาย ? แสดงว่าเป็นตัวเลือก (Optional) ไม่ส่งมาก็ไม่เกิดข้อผิดพลาด
  onPressDetails?: () => void; // ฟังก์ชัน Callback สำหรับส่งสัญญาณเมื่อกดการ์ด
}

export default function UserProfileCard({
 name,
  avatarUrl,
  role,
  isActive = false, // กำหนดค่าเริ่มต้นให้กับพร็อพส์ที่เป็นตัวเลือก
  onPressDetails,
}: UserProfileCardProps) {
  return (
    <View style={styles.card}>
      <Image source={{ uri: avatarUrl }} style={styles.avatar} />
      <View style={styles.info}>
        <Text style={styles.nameText}>{name}</Text>
        <Text style={styles.roleText}>{role}</Text>
        <Text style={[styles.statusText, { color: isActive ? '#10b981' :
'#ef4444' }]}>
          {isActive ? '● กำลังทำงาน' : 'o พักผ่อน'}
        </Text>
      </View>
      {onPressDetails && (
        <Pressable onPress={onPressDetails} style={styles.button}>
          <Text style={styles.buttonText}>ข้อมูลเพิ่มเติม</Text>
        </Pressable>
   )}
    </View>
 );
}

const styles = StyleSheet.create({
  card: {
    flexDirection: 'row',
    alignItems: 'center',
    padding: 16,
    backgroundColor: '#fff',
    borderRadius: 12,
    marginVertical: 8,
    shadowColor: '#000',
    shadowOpacity: 0.1,
    shadowRadius: 4,
    elevation: 3,
  },
  avatar: {
```

```
    width: 60,
    height: 60,
    borderRadius: 30,
  },
  info: {
    flex: 1,
    marginLeft: 16,
  },
  nameText: {
    fontSize: 18,
    fontWeight: 'bold',
  },
  roleText: {
    color: '#64748b',
    fontSize: 14,
  },
  statusText: {
    fontSize: 12,
    marginTop: 4,
    fontWeight: '500',
  },
  button: {
    backgroundColor: '#0f172a',
    paddingVertical: 6,
    paddingHorizontal: 12,
    borderRadius: 6,
  },
  buttonText: {
    color: '#fff',
    fontSize: 12,
  },
});
```

3. การเรียกใช้งานในหน้าจอหลัก (Parent Component)

เราสามารถเรียกนำเข้าและสร้างผู้ใช้หลาย ๆ คนได้โดยส่งข้อมูลที่ต้องการแตกต่างกันผ่านคุณสมบัติ Props:

```
// 2. เรียกใช้ภายในไฟล์หลัก App.tsx หรือ index.tsx
import { View, StyleSheet, Alert } from 'react-native';
import UserProfileCard from './components/UserProfileCard';

export default function HomeScreen() {
  return (
    <View style={styles.container}>
      {/* ส่งข้อมูลครบสมบูรณ์ */}
       <UserProfileCard
```

```
          name="สมชาย ใจดี"
          avatarUrl="https://images.unsplash.com/photo-1535713875002-
d1d0cf377fde"
          role="ผู้พัฒนาโมบายแอป"
          isActive={true}
          onPressDetails={() => Alert.alert('ข้อมูลเพิ่มเติม', 'สมชายเป็นผู้พัฒนามานานกว่า 5
ปี')}
    />

        {/* ละตัวแปร isActive และ onPressDetails ที่เป็นออปชัน เสริมความยืดหยุ่น */}
        <UserProfileCard
          name="สมศรี รักเรียน"
          avatarUrl="https://images.unsplash.com/photo-1494790108377-be9c29b29330"
          role="นักออกแบบ UX/UI"
    />
      </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f1f5f9',
    padding: 16,
  },
});
```

สรุปท้ายบทเรียน

คอมโพเนนต์แสดงผลรูปภาพ Image: คอมโพเนนต์ Image (React Native, 2024) มีหน้าที่แสดงผลรูปภาพในโปรเจกต์ ทั้งภาพภายในเครื่องผ่าน require() และภาพจากอินเทอร์เน็ตผ่านพร็อพส์ uri โดยสำหรับการโหลดภาพออนไลน์นั้นมีข้อบังคับสำคัญคือต้องกำหนดความกว้างและความสูงไว้เสมอมิฉะนั้นภาพจะไม่แสดง และสามารถปรับการจัดวางองค์ประกอบด้วย resizeMode เช่น cover, contain, stretch หรือ center เพื่อรักษาสัดส่วนหรือปรับรูปลักษณ์ตามความเหมาะสม

คอมโพเนนต์รับข้อมูลข้อความ TextInput และการควบคุมคีย์บอร์ด: คอมโพเนนต์ TextInput (React Native, 2024) เป็นเครื่องมือหลักในการสร้างฟอร์มเพื่อรับข้อมูลข้อความจากผู้ใช้งานผ่านคีย์บอร์ด โดยมีพร็อพส์สำคัญ เช่น value, placeholder, secureTextEntry สำหรับการป้อนรหัสผ่าน และ onChangeText สำหรับคอยดึงข้อมูลการพิมพ์ล่าสุด นอกจากนี้เรายัง

สามารถเลือกรูปแบบคีย์บอร์ดให้เหมาะกับประเภทการรับค่าได้ด้วย keyboardType รวมถึงการปรับค่า autoCapitalize และ returnKeyType เพื่อให้การตอบสนองดีที่สุด

การสร้างพื้นที่เลื่อนดูข้อมูลด้วย ScrollView เปรียบเทียบกับ View: แม้ว่า View จะเรนเดอร์ข้อมูลได้รวดเร็วและใช้ในการจัดวางโครงสร้างทั่วไป แต่หากข้อมูลหน้าจอยาวเกินขอบเขตจอภาพก็จะไม่สามารถเลื่อนอ่านได้ ซึ่ง ScrollView จะเข้ามาช่วยแก้ปัญหาและอำนวยความสะดวกในการเลื่อนเนื้อหาแนวตั้งหรือแนวนอน โดยการจัดระยะห่าง (Padding) หรือขอบเขตการจัดตำแหน่งของเนื้อหาภายในจำลองจะต้องใช้งานผ่านพร็อพส์ contentContainerStyle แทนสไตล์ปกติ

การจัดการเหตุการณ์การสัมผัสและการกดปุ่มด้วย Pressable: คอมโพเนนต์ Pressable (React Native, 2024) เป็นเครื่องมือยุคใหม่ที่ถูกพัฒนาต่อยอดขึ้นมาเพื่อตรวจจับเหตุการณ์การกดและสัมผัสแบบซับซ้อน เช่น onPress, onLongPress, onPressIn และ onPressOut โดยผู้พัฒนาสามารถเขียนประกาศฟังก์ชันแบบไดนามิกผ่านพารามิเตอร์ของ `{ pressed }` เพื่อสร้างลวดลายและการตอบสนองของปุ่มเมื่อถูกกดสัมผัสในแบบเรียลไทม์ได้อย่างยืดหยุ่น

การหลีกเลี่ยงพื้นที่ขอบจอด้วย SafeAreaView: SafeAreaView เป็นคอมโพเนนต์สำคัญในการจัดแจงขอบเขตหน้าจอเพื่อช่วยคำนวณและเว้นพื้นที่ว่างในส่วนของรอยบากหน้าจอ แถบกล้อง หรือแถบนำทางระบบปฏิบัติการโดยเฉพาะบนเครื่อง iOS เพื่อให้เนื้อหาหลักของแอปพลิเคชันไม่ถูกรบกวนหรือทับซ้อนและได้รับการจัดระเบียบตามระยะปลอดภัยของมือถือแต่ละรุ่นอย่างเหมาะสม

แนวคิดการสร้าง Custom Component และการกำหนด Props Interface ด้วย TypeScript: การพัฒนาแอปพลิเคชันให้สามารถนำส่วนประกอบ UI มารีไซเคิลใหม่ทำได้โดยการแยกชิ้นส่วนไปเขียนเป็น Custom Component และส่งผ่านตัวแปรต่าง ๆ ผ่านระบบ Props โดยมีการใช้ TypeScript Interface ในการควบคุมความปลอดภัยเพื่อกำหนดความต้องการข้อมูลประเภทจำเพาะ เช่น การใส่เครื่องหมาย ? กำหนดตัวแปรแบบเป็นทางเลือก และฟังก์ชัน Callback สำหรับส่งสัญญาณเมื่อเกิดการสัมผัสอินเทอร์เฟซย่อย

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความเข้าใจเกี่ยวกับคอมโพเนนต์พื้นฐาน (Core Components เช่น `Image`, `TextInput`, `ScrollView`, `Pressable`, `SafeAreaView`) และแนวคิดการจัดการ Props ร่วมกับการใช้ TypeScript ใน React Native Expo SDK 54

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: เมื่อต้องการแสดงผลรูปภาพจากเครือข่ายอินเทอร์เน็ต (Network Image) ด้วยคอมโพเนนต์ Image ข้อใดคือรูปแบบการเขียนและข้อกำหนดที่สำคัญที่สุดเพื่อให้รูปภาพแสดงผลขึ้นมา?

ก. `<Image source="https://example.com/pic.png" />` โดยไม่ต้องระบุขนาด

ข. `<Image source={{ uri: 'https://example.com/pic.png' }} />` โดยไม่ต้องระบุขนาด

ค. `<Image source={{ uri: 'https://example.com/pic.png' }} style={{ width: 100, height: 100 }} />`

ง. `<Image localPath="https://example.com/pic.png" style={{ width: 100, height: 100 }} />`

ข้อ 2: ค่า resizeMode รูปแบบใดของคอมโพเนนต์ Image ที่ทำหน้าที่หดหรือขยายรูปภาพเพื่อให้อยู่ภายใต้กรอบของคอมโพเนนต์ได้ครบถ้วนทั้งภาพ โดยยังคงรักษาอัตราส่วนภาพเดิมไว้ครบถ้วน?

ก. `cover`

ข. `contain`

ค. `stretch`

ง. `center`

ข้อ 3: พร็อพส์ (Props) ใดของ TextInput ที่ใช้ในการกำหนดให้อักขระที่พิมพ์แสดงผลเป็นจุดกลมดำเพื่อป้องกันความปลอดภัยในการกรอกรหัสผ่าน (Password)?

ก. `passwordMask={true}`

ข. `secureTextEntry={true}`

ค. `keyboardType="password"`

ง. `autoCapitalize="none"`

ข้อ 4: เมื่อต้องการรับอินพุตเฉพาะค่าตัวเลข (เช่น ยอดเงิน หรือรหัส OTP) ผู้พัฒนาควรตั้งค่าพร็อพส์ keyboardType บน TextInput อย่างไร?

ก. `keyboardType="email-address"`

ข. `keyboardType="phone-pad"`

ค. `keyboardType="numeric"`

ง. `keyboardType="default"`

ข้อ 5: ข้อใดอธิบายความแตกต่างในพฤติกรรมการเรนเดอร์เนื้อหาระหว่าง `View` และ `ScrollView` ได้อย่างถูกต้อง?

ก. `View` สามารถเลื่อนขึ้นลงได้อัตโนมัติ แต่ `ScrollView` จะไม่สามารถเลื่อนหน้าจอได้

ข. `View` เหมาะสำหรับการจัดแถวในเลย์เอาต์ ส่วน `ScrollView` จะจัดแนวด้วย Flexbox ได้ไวกว่า

ค. `View` จะซ่อนเนื้อหาที่ล้นขอบจอและไม่มีแถบเลื่อน ส่วน `ScrollView` จะแสดงพื้นที่และช่วยให้ใช้นิ้วปัดเลื่อนอ่านเนื้อหาที่เกินขอบจอได้

ง. `View` ใช้สำหรับแสดงรูปภาพเท่านั้น ส่วน `ScrollView` ใช้แสดงตัวอักษรเท่านั้น

ข้อ 6: หากต้องการระบุสไตล์ของระยะห่างภายใน (Padding) หรือจัดกึ่งกลางให้กับคอมโพเนนต์ย่อยที่อยู่ภายใต้ `ScrollView` จะต้องส่งคำสั่งสไตล์ผ่านพร็อพส์ใดจึงจะทำงานได้ถูกต้อง?

ก. `style`

ข. `contentContainerStyle`

ค. `innerStyle`

ง. `scrollStyle`

ข้อ 7: ข้อใดกล่าวถึงประโยชน์และความสามารถเด่นของคอมโพเนนต์ `Pressable` ที่โดดเด่นกว่าคอมโพเนนต์ในกลุ่ม Touchable รุ่นดั้งเดิมได้ถูกต้องที่สุด?

ก. `Pressable` สามารถแสดงผลภาพเคลื่อนไหว GIF ได้ในตัวเอง

ข. `Pressable` รองรับการปรับเปลี่ยนรูปแบบสไตล์ชีท (Style) และคอมโพเนนต์ลูก (Children) แบบไดนามิกตามสถานะการกดสัมผัสจริงของนิ้วมือผู้ใช้งานผ่านค่า `{ pressed }`

ค. `Pressable` ทำการเชื่อมต่ออินเทอร์เน็ตได้เร็วกว่าคอมโพเนนต์อื่น

ง. `Pressable` ไม่จำเป็นต้องมีพร็อพส์ `onPress` ในการดักจับพฤติกรรมผู้ใช้งาน

ข้อ 8: วัตถุประสงค์หลักของการประยุกต์ใช้คอมโพเนนต์ `SafeAreaView` คือข้อใด?

ก. เพื่อสแกนและล้างไวรัสออกจากตัวเครื่องสมาร์ทโฟนที่เปิดแอป

ข. เพื่อบันทึกข้อมูลอินพุตของผู้ใช้ลงในฐานข้อมูลเครื่องอย่างปลอดภัย

ค. เพื่อหลีกเลี่ยงการแสดงผลของเนื้อหาแอปพลิเคชันไปทับซ้อนกับพื้นที่ทางกายภาพ เช่น รอยบาก กล้อง หรือแถบแสดงสถานะระบบของเครื่อง

ง. เพื่อเร่งความเร็วในการประมวลผลโค้ด JavaScript ให้ไวขึ้น

ข้อ 9: ภายใต้กฎพื้นฐานของไลบรารี React และ React Native ข้อใดอธิบายทิศทางการไหลของข้อมูลผ่านคุณสมบัติ Props ได้ถูกต้อง?

ก. ไหลจากคอมโพเนนต์ลูกย้อนกลับไปแก้ไขค่าในคอมโพเนนต์แม่โดยตรง

ข. ไหลแบบทิศทางเดียว (Unidirectional) จากคอมโพเนนต์แม่ส่งต่อไปยังคอมโพเนนต์ลูก และเป็นข้อมูลแบบอ่านอย่างเดียว (Read-only) สำหรับลูก

ค. ข้อมูลจะไหลเวียนและแก้ไขได้อิสระผ่านทุกคอมโพเนนต์ในโปรเจกต์โดยไม่ต้องประกาศรับ

ง. ไหลเฉพาะจากไฟล์ CSS เข้าสู่เนื้อหาข้อความในแอปพลิเคชันเท่านั้น

ข้อ 10: ในการเขียนแบบมีระเบียบ (Type-safe) การระบุเครื่องหมายคำถามท้ายชื่อตัวแปรใน TypeScript Interface เช่น `avatarUrl?: string;` มีความหมายอย่างไร?

ก. ตัวแปร `avatarUrl` จะต้องส่งค่าเป็นประเภทข้อมูล Boolean เท่านั้น

ข. ตัวแปร `avatarUrl` มีสถานะเป็น Optional Props (ผู้ใช้งานคอมโพเนนต์ย่อยจะส่งค่ามาให้หรือไม่ส่งค่ามาให้ก็ได้โดยไม่เกิดข้อผิดพลาดในการรันไทม์)

ค. ตัวแปร `avatarUrl` ห้ามนำไปใช้งานภายในคอมโพเนนต์เด็ดขาด

ง. ตัวแปร `avatarUrl` จะมีระบบสุ่มค่าข้อความให้อัตโนมัติ

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: พร็อพส์ `source` ไม่รับสตริง URL ตรง ๆ สำหรับรูปภาพจากอินเทอร์เน็ต แต่ต้องประกาศผ่านโครงสร้างออบเจกต์ที่มีพร็อพส์ `uri`

• ข. ผิด: แม้จะระบุเป็นรูปเล่มออบเจกต์ `uri` แล้ว แต่เนื่องจากรูปภาพผ่านเครือข่ายไม่มีตัวตนในเครื่องก่อนบิลด์ แอปพลิเคชันจึงไม่ทราบสัดส่วนขนาด หากไม่เขียนระบุกว้าง-สูง (`style={{ width, height }}`) ขนาดภาพจะถูกเซ็ตเป็น 0x0 ทำให้ไม่ปรากฏรูป

• ค. ถูก: ต้องเขียนระบุรูปเล่มออบเจกต์ `{ uri: 'URL' }` ร่วมกับการประกาศสไตล์ควบคุมขนาดกว้างและสูงอย่างเด่นชัดเพื่อให้ระบบแสดงรูป

• ง. ผิด: ไม่มีพร็อพส์ชื่อ `localPath` ในการดึงรูปภาพของคอมโพเนนต์ `Image`

เฉลย ข้อ 2: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: `cover` จะดึงขยายรูปภาพให้เต็มกรอบ ซึ่งอาจทำให้อัตราส่วนของภาพยังคงเดิมแต่ส่วนที่เลยกรอบออกไปจะถูกตัดออก (Cropped)

• ข. ถูก: `contain` จะปรับเปลี่ยนขนาดรูปภาพให้อยู่ในขอบเขตกรอบทั้งหมดโดยที่ไม่มีชิ้นส่วนใดถูกบีบเบี้ยวหรือตัดทิ้ง โดยรักษาอัตราส่วนภาพเดิมไว้ร้อยเปอร์เซ็นต์

• **ค. ผิด:** `stretch` จะยืดสัดส่วนแกนให้พอดีกล่องซึ่งส่งผลให้หน้าตาของรูปภาพเกิดการยืดบิดเบี้ยวได้

• **ง. ผิด:** `center` จะวางรูปภาพขนาดปกติไว้กึ่งกลางกล่องโดยไม่มีการขยายขนาดรูปภาพ

เฉลย ข้อ 3: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ไม่มีพร็อพส์ชื่อ `passwordMask` ในคอมโพเนนต์ `TextInput`

• **ข. ถูก:** การตั้งค่า `secureTextEntry={true}` เป็นมาตรฐานของ React Native ในการสั่งให้ช่องรับข้อมูลซ่อนอักขระจริง และเรนเดอร์เป็นสัญลักษณ์จุดวงกลมเพื่อความปลอดภัย

• **ค. ผิด:** `keyboardType` ใช้เปลี่ยนปุ่มแป้นพิมพ์คีย์บอร์ด ไม่ใช่หน้าตาข้อความข้างในช่องรับค่า

• **ง. ผิด:** `autoCapitalize` เป็นเพียงคำสั่งปิดตัวพิมพ์ใหญ่ภาษาอังกฤษ ซึ่งไม่มีฟังก์ชันบังขอบเขตข้อความ

เฉลย ข้อ 4: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `email-address` คีย์บอร์ดจะมีปุ่มสัญลักษณ์ตัวอักษรและปุ่ม @ โผล่มาหลักๆ

• **ข. ผิด:** `phone-pad` จะแสดงตัวเลขรวมถึงเครื่องหมายที่ใช้ต่อสายโทรศัพท์ เช่น *, #, + ซึ่งเกินกว่าตัวเลขข้อมูลปกติ

• **ค. ถูก:** การส่งค่า `keyboardType="numeric"` จะสั่งให้อุปกรณ์ระบบปฏิบัติการเปลี่ยนมาแสดงแป้นพิมพ์ปุ่มหลักเฉพาะตัวเลข 0-9 เพื่อความรวดเร็วและป้องกันไม่ให้ผู้ใช้พิมพ์ตัวอักษรตัวหนังสือที่ผิดรูปแบบเข้ามาได้

• **ง. ผิด:** `default` จะเรียกคีย์บอร์ดปุ่มกดตัวอักษรข้อความปกติของสมาร์ทโฟนขึ้นมาทำงาน

เฉลย ข้อ 5: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** สลับกัน คอมโพเนนต์ `View` ไม่สามารถเลื่อนหน้าจอได้

• **ข. ผิด:** ทั้งสองตัวทำสไตล์จัดสัดส่วนผ่าน Flexbox ได้ไม่ต่างกันในด้านความยากง่าย

• **ค. ถูก:** `View` มีพฤติกรรมแบบคงที่ หากเนื้อหาภายในมีขนาดใหญ่หรือยาวล้นออกไปนอกพื้นที่ของแม่ เนื้อหาที่ล้นจะถูกตัดการแสดงผลทับไปและเลื่อนดูไม่ได้ ส่วน `ScrollView` จะมีความสามารถดักจับการปัดสัมผัสเลื่อนแกนเพื่อขยับวัตถุที่ล้นเข้ามาในหน้าจอได้ทันที

• **ง. ผิด:** ทั้งสองคอมโพเนนต์ใช้จัดวางหรือใส่ประเภทชิ้นงานรูปภาพข้อความข้างในได้ตามกฎปกติ

เฉลย ข้อ 6: ตอบ ข.

• เหตุผลประกอบ:

• **ก. ผิด:** การใช้ `style` บน `ScrollView` จะทำงานควบคุมเฉพาะขอบเขตสไตล์ภายนอกของตัวกรอบเลื่อน แต่ไม่ส่งสไตล์จัดแนวลงไปหาคอมโพเนนต์ลูกด้านใน

• **ข. ถูก:** พร็อพส์ `contentContainerStyle` ถูกออกแบบมาเพื่อนำสไตล์การจัดช่องไฟ (เช่น `padding`) หรือการจัดตำแหน่ง (เช่น `justifyContent`, `alignItems`) ส่งเข้าไปควบคุมกล่องห่อหุ้มลูกภายในพื้นที่เลื่อนได้สมบูรณ์

• **ค. และ ง. ผิด:** ไม่มีพร็อพส์ชื่อดังกล่าวใน React Native SDK

เฉลย ข้อ 7: ตอบ ข.

• เหตุผลประกอบ:

• **ก. ผิด:** `Pressable` ไม่เกี่ยวข้องกับการจำลองรันไฟล์ภาพเคลื่อนไหว

• **ข. ถูก:** `Pressable` เป็นคอมโพเนนต์รุ่นใหม่ที่ออกแบบมาให้ตรวจเช็คสัญญาณแรงกดนิ้วได้ยืดหยุ่น โดยสามารถประเมินพารามิเตอร์ `{ pressed }` เพื่อนำไปเปลี่ยนสีปุ่ม ขนาด หรือสไตล์ของวัตถุข้างในได้แบบเรียลไทม์ ทำให้แอปพลิเคชันดูมีชีวิตชีวาและโต้ตอบได้สมบูรณ์

• **ค. ผิด:** ไม่มีความเกี่ยวดองกับการทำงานเน็ตเวิร์กของระบบ

• **ง. ผิด:** ยังคงจำเป็นต้องใช้พร็อพส์ `onPress` ในการทำงานดักจับสัมผัสคลิกปุ่มเช่นเดียวกับปุ่มทั่วไป

เฉลย ข้อ 8: ตอบ ค.

• เหตุผลประกอบ:

• **ก. ผิด:** ไม่มีระบบรักษาความปลอดภัยประเภทกำจัดโปรแกรมที่เป็นภัยต่อตัวเครื่อง

• **ข. ผิด:** การบันทึกเก็บข้อมูลดั้งเดิมจะพึ่งพากลุ่ม Local Storage เช่น AsyncStorage ไม่ใช่หน้าที่ของ View

• **ค. ถูก:** `SafeAreaView` ทำหน้าที่ปรับปรุงระเบียบกล่องเลย์เอาต์โดยการตัดพิกเซลเว้นระยะในจุดที่มีความเสี่ยงต่อการโดนฮาร์ดแวร์ขอบหน้าจอบัง (เช่น ขอบบนที่ติดกล้อง รอยบาก หรือขอบล่างของปุ่มนำทาง Home Indicator) ให้เรียบร้อยโดยไม่ต้องปรับเอง

• **ง. ผิด:** ไม่ได้มีคุณลักษณะในการคอมไพล์โค้ดหรือปรับปรุงประสิทธิภาพ JavaScript Engine

เฉลย ข้อ 9: ตอบ ข.

• เหตุผลประกอบ:

• **ก. ผิด:** คอมโพเนนต์ลูกไม่สามารถเข้าทำรายการหรือแก้ไขตัวแปรต้นฉบับในคอมโพเนนต์แม่ผ่าน Props ได้โดยตรง ยกเว้นแต่ว่าคอมโพเนนต์แม่จะส่งฟังก์ชันสำหรับ Callback ลงไปให้ใช้งาน

• **ข. ถูก:** ตามหลักสถาปัตยกรรม React การส่งข้อมูลผ่าน Props จะมีลักษณะจากบนลงล่าง (จาก Parent สู่ Child) เท่านั้น และตัวคอมโพเนนต์ลูกที่ได้รับ Props ไปจะมีสถานะใช้อ่านอย่างเดียว (Immutable/Read-only) เพื่อไม่ให้สเตตของโปรแกรมสับสนกระจัดกระจาย

• **ค. ผิด:** ต้องระบุตัวแปรที่รับมาในพารามิเตอร์ฟังก์ชันเสมอเพื่อให้ TypeScript และ JavaScript อ้างอิงตัวแปรได้ถูกต้อง

• **ง. ผิด:** ข้อมูล Props ไม่เกี่ยวกับการเปลี่ยนทางสไตล์ชีทแบบดังกล่าว

เฉลย ข้อ 10: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ชนิดของตัวแปรในโจทย์ระบุชัดเจนว่าเป็น `string`

• **ข. ถูก:** ใน TypeScript การเขียนต่อท้ายคีย์ด้วยเครื่องหมายคำถาม (`?`) บ่งบอกว่าพร็อพส์นั้น ๆ เป็น "Optional" (ไม่จำเป็นต้องระบุ) ทำให้เวลาเรียกใช้คอมโพเนนต์ย่อยนี้ เราไม่จำเป็นต้องป้อนค่าพร็อพส์ตัวนี้เสมอไป ซึ่งช่วยรันระบบได้เป็นระเบียบและลดคำสั่งแจ้งเตือนความผิดพลาด (Linter warning)

• **ค. ผิด:** ยังคงสามารถอ้างถึงและใช้งานตัวแปรได้ตามปกติ เพียงแต่ค่าอาจเป็น `undefined` หากไม่มีการส่งมาจากแม่

• **ง. ผิด:** ระบบ TypeScript ทำหน้าที่เพียงตรวจสอบประเภทตัวแปร ไม่สามารถประดิษฐ์สุ่มคำให้เกิดขึ้นเองรันไทม์ได้

ปฏิบัติการท้ายบทเรียน: การสร้างหน้าจอโปรไฟล์และฟอร์มตอบรับข้อมูลผู้ใช้งาน

ปฏิบัติการนี้จะฝึกฝนทักษะการประยุกต์ใช้คอมโพเนนต์พื้นฐานหลักของ React Native และแนวคิดการสร้างคอมโพเนนต์ย่อยที่กำหนดขึ้นเอง (Custom Component) ที่สนับสนุนโดยกลไก Type-safety ด้วย TypeScript Interface โดยมีเป้าหมายคือการพัฒนาหน้าจอ **ระบบข้อมูลโปรไฟล์พร้อมฟอร์มกรอกข้อมูลแบบโต้ตอบ (Profile Form with Live Preview Card)**

1. วัตถุประสงค์การเรียนรู้

1. สามารถใช้คอมโพเนนต์ `SafeAreaView` และ `ScrollView` เพื่อรองรับการทำงานของหน้าจอที่เนื้อหายาวโดยไม่ทับซ้อนขอบจอ

2. สามารถใช้คอมโพเนนต์ `Image` ในการแสดงผลภาพจากเน็ตเวิร์กได้อย่างถูกวิธี

3. สามารถประยุกต์ใช้ `TextInput` ในการรับข้อมูลรูปแบบต่าง ๆ พร้อมจัดการการทำงานของแป้นพิมพ์ (Keyboard Control)

4. สามารถควบคุมสไตล์ชีทแบบตอบสนองเมื่อผู้ใช้กดสัมผัสด้วย `Pressable` และพารามิเตอร์ `{ pressed }`

5. เข้าใจและลงมือสร้าง Custom Component พร้อมเขียน TypeScript Interface เพื่อกำกับประเภทตัวแปรของ Props ได้อย่างมีหลักการ

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนจะต้องแก้ไขไฟล์ภายในบทเรียนโดยสร้างโฟลเดอร์สำหรับ Custom Component และเขียนโค้ดประสานงานกันบนหน้าหลัก `app/index.tsx` ดังนี้:

โครงสร้างการจัดแสดงข้อมูลเลย์เอาต์:

1. **พื้นที่ปลอดภัย (`SafeAreaView`):** คลุมรอบนอกสุดเพื่อป้องกันเนื้อหาทับซ้อนช่องบากบน iOS

2. **พื้นที่เลื่อนข้อความ (`ScrollView`):** ครอบเนื้อหาทั้งหมดเพื่อเปิดโอกาสให้เลื่อนสไลด์หน้าจอด้านล่างขึ้นมาได้ยามป้อนข้อมูลผ่านแป้นพิมพ์

3. **การ์ดโปรไฟล์แสดงผล (Live Preview Card):**

- สร้างเป็น Custom Component ชื่อ `ProfileCard` ไว้ในไฟล์แยกต่างหากคือ `components/ProfileCard.tsx`
- มีคุณสมบัติ Props ใน TypeScript กำกับ ได้แก่ `avatarUrl: string`, `name: string`, `role: string`, `statusText?: string`, และ `onPressCard?: () => void`
- แสดงรูปภาพด้วย `Image` (สไตล์ทรงกลมโค้งมน สมมาตร) พร้อมแสดงชื่อ ตำแหน่งงาน และสเตตัสข้อความ

4. **แบบฟอร์มแก้ไขข้อมูลโปรไฟล์ (Profile Form Editor):**

- ช่องพิมพ์รับค่า URL รูปภาพ (`TextInput` กำหนดคีย์บอร์ดประเภทอีเมล/เว็บและปิดระบบตัวพิมพ์ใหญ่)
- ช่องพิมพ์รับชื่อ (`TextInput` ทั่วไป)
- ช่องพิมพ์รับตำแหน่งงาน (`TextInput` ทั่วไป)
- ช่องพิมพ์รับข้อความสเตตัส (`TextInput` ทั่วไป)

5. **ปุ่มอัปเดตข้อมูล (Pressable Button):**

- จัดสไตล์ด้วย `Pressable`
- มีเอฟเฟกต์ตอบสนองอย่างรวดเร็ว (ย่อขนาดลงและสีเข้มขึ้น) ขณะกดปุ่มสัมผัส

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: การสร้าง Custom Component พร้อมระบุโครงสร้าง TypeScript Props Interface

สร้างไฟล์ใหม่ในโฟลเดอร์โครงการตามเส้นทาง `components/ProfileCard.tsx` (หากไม่มีโฟลเดอร์ `components` ให้สร้างโฟลเดอร์นี้ขึ้นมาใหม่ก่อน) และใส่รหัสด้านล่างนี้ลงไป:

```
import { View, Text, Image, StyleSheet, Pressable } from 'react-native';

// 1. ระบุข้อกำหนดประเภทข้อมูล Props ด้วย TypeScript Interface
export interface ProfileCardProps {
  avatarUrl: string;
  name: string;
  role: string;
  statusText?: string;          // เครื่องหมาย ? แสดงว่าเป็นคุณสมบัติทางเลือก (Optional)
  onPressCard?: () => void;  // เหตุการณ์ฟังก์ชันส่งสัญญาณเมื่อกดการ์ด
}

// 2. นำฟังก์ชันคอมโพเนนต์มาแมปตัวแปร Props
export default function ProfileCard({
  avatarUrl,
  name,
  role,
  statusText = 'ไม่มีข้อความอัปเดตสเตตัสในขณะนี้', // ค่าเริ่มต้นหากไม่มีการส่งมา
  onPressCard
}: ProfileCardProps) {

 // กำหนดรูปภาพสำรองหาก URL ถูกส่งมาเป็นข้อความว่างเปล่า
  const validAvatar = avatarUrl.trim() !== ''
    ? avatarUrl
  : 'https://images.unsplash.com/photo-1535713875002-d1d0cf377fde?w=150';

  return (
    <Pressable
      onPress={onPressCard}
      style={({ pressed }) => [
        styles.card,
        { opacity: pressed ? 0.95 : 1.0, transform: [{ scale: pressed ? 0.99 : 1.0
}] }
   ]}
    >
      {/* แสดงภาพโปรไฟล์จาก URL เครือข่าย */}
      <Image source={{ uri: validAvatar }} style={styles.avatar} />

      <View style={styles.infoContainer}>
        <Text style={styles.nameText}>{name}</Text>
        <Text style={styles.roleText}>{role}</Text>
```

```
        <View style={styles.statusBox}>
          <Text style={styles.statusLabelText}>สเตตัสปัจจุบัน:</Text>
          <Text style={styles.statusText}>{statusText}</Text>
        </View>
      </View>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 16,
    flexDirection: 'row',
    alignItems: 'center',
    borderWidth: 1,
    borderColor: '#334155',
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.1,
    shadowRadius: 6,
    elevation: 4,
    marginVertical: 12,
  },
  avatar: {
    width: 80,
    height: 80,
    borderRadius: 40, // ครึ่งหนึ่งของขนาดภาพดึงมุมกลมสมบูรณ์
    backgroundColor: '#334155',
  },
  infoContainer: {
    flex: 1,
    marginLeft: 16,
  },
  nameText: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  roleText: {
    fontSize: 14,
    color: '#38bdf8',
    marginTop: 2,
  },
  statusBox: {
    marginTop: 8,
```

```
  borderTopWidth: 1,
    borderTopColor: '#334155',
    paddingTop: 6,
  },
  statusLabelText: {
    fontSize: 10,
    color: '#94a3b8',
    textTransform: 'uppercase',
  },
  statusText: {
    fontSize: 12,
    color: '#cbd5e1',
    fontStyle: 'italic',
   marginTop: 2,
  },
});
```

ขั้นตอนที่ 2: วางหน้าจอและเก็บสถานะฟอร์มในคอมโพเนนต์หลัก app/index.tsx

เขียนโครงสร้างการรับข้อมูล และการแสดงตัวอย่างการ์ดร่วมกับแบบฟอร์มการพิมพ์ โดยใช้ State ในการสะท้อนข้อมูลแบบ Real-time:

```
import { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  Pressable,
  SafeAreaView,
  ScrollView,
  Alert
} from 'react-native';
import ProfileCard from '../components/ProfileCard'; // เรียกใช้ออบเจกต์ Custom Component

export default function App() {
 // ประกาศ State เพื่อผูกเข้ากับฟอร์ม TextInput
  const [name, setName] = useState('สมชาย ใจดี');
  const [role, setRole] = useState('Full Stack Developer');
  const [avatar, setAvatar] = useState('https://images.unsplash.com/photo-1535713875002-d1d0cf377fde?w=150');
  const [status, setStatus] = useState('กำลังเรียนรู้ React Native Expo SDK 54');

 // สเตตสำหรับการบันทึกจริงตอนกดปุ่ม
  const [savedData, setSavedData] = useState({
    name: 'สมชาย ใจดี',
    role: 'Full Stack Developer',
```

```
    avatar: 'https://images.unsplash.com/photo-1535713875002-
d1d0cf377fde?w=150',
    status: 'กำลังเรียนรู้ React Native Expo SDK 54'
  });

  const handleUpdate = () => {
    setSavedData({ name, role, avatar, status });
    Alert.alert('อัปเดตสำเร็จ', 'ข้อมูลโปรไฟล์ได้รับการบันทึกบนเครื่องสมบูรณ์');
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>

        <Text style={styles.pageTitle}>ระบบจัดการโปรไฟล์ผู้ใช้</Text>
        <Text style={styles.sectionTitle}>การ์ดแสดงผลปัจจุบัน (Preview)</Text>

        {/* เรียกใช้งาน Custom Component และส่งผ่านข้อมูลผ่านโครงสร้าง Props */}
        <ProfileCard
          name={savedData.name}
          role={savedData.role}
          avatarUrl={savedData.avatar}
          statusText={savedData.status}
          onPressCard={() => Alert.alert('ข้อความสเตตัส', savedData.status)}
    />

        <View style={styles.formContainer}>
          <Text style={styles.sectionTitle}>ฟอร์มกรอกข้อมูลการแก้ไข</Text>

          {/* ช่องกรอกรูปภาพ Avatar URL */}
          <Text style={styles.label}>URL รูปภาพโปรไฟล์</Text>
          <TextInput
            style={styles.input}
            value={avatar}
          onChangeText={setAvatar}
            placeholder="ใส่ URL รูปภาพโปรไฟล์..."
            keyboardType="url"
            autoCapitalize="none"
            autoCorrect={false}
     />

          {/* ช่องกรอกชื่อ */}
          <Text style={styles.label}>ชื่อ-นามสกุล</Text>
          <TextInput
            style={styles.input}
            value={name}
            onChangeText={setName}
            placeholder="ป้อนชื่อและนามสกุล..."
```

```
      />

            {/* ช่องกรอกตำแหน่งงาน */}
            <Text style={styles.label}>ตำแหน่งงาน/บทบาท</Text>
            <TextInput
              style={styles.input}
              value={role}
              onChangeText={setRole}
              placeholder="เช่น UI/UX Designer..."
      />

            {/* ช่องกรอกสเตตัส */}
            <Text style={styles.label}>ข้อความสถานะ (Status Message)</Text>
            <TextInput
              style={[styles.input, styles.textArea]}
              value={status}
              onChangeText={setStatus}
              placeholder="ใส่ข้อความทักทายของคุณ..."
              multiline={true}
              numberOfLines={3}
      />

            {/* ปุ่มบันทึกจัดสไตล์ด้วย Pressable */}
            <Pressable
              onPress={handleUpdate}
              style={({ pressed }) => [
                styles.submitButton,
                {
                  backgroundColor: pressed ? '#0369a1' : '#0284c7',
                  transform: [{ scale: pressed ? 0.98 : 1.0 }]
                }
       ]}
            >
              <Text style={styles.submitButtonText}>อัปเดตโปรไฟล์</Text>
            </Pressable>

          </View>
        </ScrollView>
      </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a', // คลุมธีมสีกึ่งน้ำเงินเข้ม
  },
  scrollContainer: {
```

```
    padding: 20,
    paddingBottom: 40,
  },
  pageTitle: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
    textAlign: 'center',
    marginBottom: 8,
  },
  sectionTitle: {
    fontSize: 16,
    fontWeight: '600',
    color: '#94a3b8',
    marginTop: 12,
    marginBottom: 4,
  },
  formContainer: {
    marginTop: 16,
    backgroundColor: '#1e293b',
    padding: 20,
    borderRadius: 16,
    borderWidth: 1,
    borderColor: '#334155',
  },
  label: {
    color: '#cbd5e1',
    fontSize: 14,
  fontWeight: '500',
    marginTop: 12,
    marginBottom: 6,
  },
  input: {
    backgroundColor: '#0f172a',
    borderWidth: 1,
    borderColor: '#334155',
    borderRadius: 8,
    height: 46,
    paddingHorizontal: 12,
    color: '#ffffff',
    fontSize: 14,
  },
  textArea: {
    height: 80,
    paddingTop: 10,
    textAlignVertical: 'top', // ป้องกันข้อความเด้งไปกึ่งกลางแนวแกนตั้งในระบบ Android
```

```
  },
  submitButton: {
    borderRadius: 8,
    height: 48,
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 20,
  },
  submitButtonText: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
});
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ด้านล่างนี้คือไฟล์โค้ดเฉลยฉบับรันได้จริง ประกอบด้วยโค้ดแยกตามโครงสร้างย่อย เพื่อให้ผู้เรียนสามารถประยุกต์ทำปฏิบัติการและนำโครงสร้างนี้ไปรันผลลัพธ์ผ่านหน้าจอจำลองได้ทันที:

4.1 ซอร์สโค้ดไฟล์คอมโพเนนต์: components/ProfileCard.tsx

```
import { View, Text, Image, StyleSheet, Pressable } from 'react-native';

export interface ProfileCardProps {
  avatarUrl: string;
  name: string;
  role: string;
  statusText?: string;
  onPressCard?: () => void;
}

export default function ProfileCard({
  avatarUrl,
  name,
  role,
  statusText = 'ไม่มีข้อความอัปเดตสถานะ',
  onPressCard
}: ProfileCardProps) {

  const validAvatar = avatarUrl.trim() !== ''
    ? avatarUrl
  : 'https://images.unsplash.com/photo-1535713875002-d1d0cf377fde?w=150';

  return (
    <Pressable
      onPress={onPressCard}
      style={({ pressed }) => [
        styles.card,
```

```
        { opacity: pressed ? 0.95 : 1.0, transform: [{ scale: pressed ? 0.99 : 1.0
}] }
    ]}
    >
      <Image source={{ uri: validAvatar }} style={styles.avatar} />
      <View style={styles.infoContainer}>
        <Text style={styles.nameText}>{name}</Text>
        <Text style={styles.roleText}>{role}</Text>
        <View style={styles.statusBox}>
          <Text style={styles.statusLabelText}>สเตตัสปัจจุบัน:</Text>
          <Text style={styles.statusText}>{statusText}</Text>
        </View>
      </View>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 16,
    flexDirection: 'row',
    alignItems: 'center',
    borderWidth: 1,
    borderColor: '#334155',
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.1,
    shadowRadius: 6,
    elevation: 4,
    marginVertical: 12,
  },
  avatar: {
    width: 80,
    height: 80,
    borderRadius: 40,
    backgroundColor: '#334155',
  },
  infoContainer: {
    flex: 1,
    marginLeft: 16,
  },
  nameText: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
  },
```

```
  roleText: {
    fontSize: 14,
    color: '#38bdf8',
    marginTop: 2,
  },
  statusBox: {
    marginTop: 8,
    borderTopWidth: 1,
    borderTopColor: '#334155',
    paddingTop: 6,
  },
  statusLabelText: {
    fontSize: 10,
    color: '#94a3b8',
    textTransform: 'uppercase',
  },
  statusText: {
    fontSize: 12,
    color: '#cbd5e1',
    fontStyle: 'italic',
    marginTop: 2,
  },
});
```

4.2 ซอร์สโค้ดหน้าจอแอปหลัก: **app/index.tsx**

```
import { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  Pressable,
  SafeAreaView,
  ScrollView,
  Alert
} from 'react-native';
import ProfileCard from '../components/ProfileCard';

export default function App() {
  const [name, setName] = useState('สมชาย ใจดี');
  const [role, setRole] = useState('Full Stack Developer');
  const [avatar, setAvatar] = useState('https://images.unsplash.com/photo-
1535713875002-d1d0cf377fde?w=150');
  const [status, setStatus] = useState('กำลังเรียนรู้ React Native Expo SDK 54');

  const [savedData, setSavedData] = useState({
    name: 'สมชาย ใจดี',
```

```
    role: 'Full Stack Developer',
    avatar: 'https://images.unsplash.com/photo-1535713875002-
d1d0cf377fde?w=150',
    status: 'กำลังเรียนรู้ React Native Expo SDK 54'
  });

  const handleUpdate = () => {
    setSavedData({ name, role, avatar, status });
    Alert.alert('อัปเดตสำเร็จ', 'ข้อมูลโปรไฟล์ได้รับการบันทึกบนเครื่องสมบูรณ์');
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>

        <Text style={styles.pageTitle}>ระบบจัดการโปรไฟล์ผู้ใช้</Text>
        <Text style={styles.sectionTitle}>การ์ดแสดงผลปัจจุบัน (Preview)</Text>

        <ProfileCard
          name={savedData.name}
          role={savedData.role}
          avatarUrl={savedData.avatar}
          statusText={savedData.status}
          onPressCard={() => Alert.alert('ข้อความสเตตัส', savedData.status)}
    />

        <View style={styles.formContainer}>
          <Text style={styles.sectionTitle}>ฟอร์มกรอกข้อมูลการแก้ไข</Text>

          <Text style={styles.label}>URL รูปภาพโปรไฟล์</Text>
          <TextInput
            style={styles.input}
            value={avatar}
            onChangeText={setAvatar}
            placeholder="ใส่ URL รูปภาพโปรไฟล์..."
            keyboardType="url"
            autoCapitalize="none"
            autoCorrect={false}
     />

          <Text style={styles.label}>ชื่อ-นามสกุล</Text>
          <TextInput
            style={styles.input}
            value={name}
            onChangeText={setName}
            placeholder="ป้อนชื่อและนามสกุล..."
     />
```

```
          <Text style={styles.label}>ตำแหน่งงาน/บทบาท</Text>
          <TextInput
            style={styles.input}
            value={role}
            onChangeText={setRole}
            placeholder="เช่น UI/UX Designer..."
      />

          <Text style={styles.label}>ข้อความสถานะ (Status Message)</Text>
          <TextInput
            style={[styles.input, styles.textArea]}
            value={status}
            onChangeText={setStatus}
            placeholder="ใส่ข้อความทักทายของคุณ..."
            multiline={true}
            numberOfLines={3}
      />

          <Pressable
            onPress={handleUpdate}
            style={({ pressed }) => [
              styles.submitButton,
              {
                backgroundColor: pressed ? '#0369a1' : '#0284c7',
                transform: [{ scale: pressed ? 0.98 : 1.0 }]
              }
       ]}
          >
            <Text style={styles.submitButtonText}>อัปเดตโปรไฟล์</Text>
          </Pressable>

        </View>
      </ScrollView>
    </SafeAreaView>
 );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  scrollContainer: {
    padding: 20,
    paddingBottom: 40,
  },
  pageTitle: {
    fontSize: 24,
    fontWeight: 'bold',
```

```
    color: '#ffffff',
    textAlign: 'center',
    marginBottom: 8,
  },
  sectionTitle: {
    fontSize: 16,
    fontWeight: '600',
    color: '#94a3b8',
    marginTop: 12,
    marginBottom: 4,
  },
  formContainer: {
    marginTop: 16,
    backgroundColor: '#1e293b',
    padding: 20,
    borderRadius: 16,
    borderWidth: 1,
    borderColor: '#334155',
  },
  label: {
    color: '#cbd5e1',
    fontSize: 14,
    fontWeight: '500',
    marginTop: 12,
    marginBottom: 6,
  },
  input: {
    backgroundColor: '#0f172a',
    borderWidth: 1,
    borderColor: '#334155',
    borderRadius: 8,
    height: 46,
    paddingHorizontal: 12,
    color: '#ffffff',
    fontSize: 14,
  },
  textArea: {
    height: 80,
    paddingTop: 10,
    textAlignVertical: 'top',
  },
  submitButton: {
    borderRadius: 8,
    height: 48,
    justifyContent: 'center',
    alignItems: 'center',
```

```
    marginTop: 20,
  },
  submitButtonText: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
});
```

เอกสารอ้างอิง

React Native. (2024). *Core Components and APIs*. จาก https://reactnative.dev/docs/intro-react-native-components

React Native. (2024). *Image*. จาก https://reactnative.dev/docs/image

React Native. (2024). *Pressable*. จาก https://reactnative.dev/docs/pressable

React Native. (2024). *Text*. จาก https://reactnative.dev/docs/text

React Native. (2024). *TextInput*. จาก https://reactnative.dev/docs/textinput

React Native. (2024). *View*. จาก https://reactnative.dev/docs/view

แผนบริหารการสอนประจำบทที่ 4

หัวข้อเนื้อหาประจำบท

4.1 แนวคิดของสถานะ (State) ใน React Native และการใช้งาน useState Hook

4.2 ข้อแตกต่างและการทำงานร่วมกันระหว่าง State และ Props

4.3 การทำแบบฟอร์มด้วย Controlled Components ผ่าน TextInput

4.4 การแบ่งปันข้อมูลย้อนกลับด้วยหลักการ Lifting State Up

4.5 การจัดทำฟอร์มคำนวณค่าดัชนีมวลกาย (BMI Calculation Form)

4.6 การตรวจสอบความถูกต้องของฟอร์ม (Form Validation) และการแจ้งเตือนข้อผิดพลาด (Error Feedback)

4.7 โค้ดตัวอย่างการประกอบฟอร์มลงทะเบียนพร้อมระบบตรวจสอบแบบเรียลไทม์

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถระบุความแตกต่างและเชื่อมโยงความสัมพันธ์ของ State และ Props ได้อย่างชัดเจน

2. ผู้เรียนสามารถสร้างตัวแปรสถานะและเขียนฟังก์ชันอัปเดตสเตตผ่าน useState เพื่อตอบสนองต่ออินเตอร์เฟซได้ถูกต้อง

3. ผู้เรียนสามารถผูกตัวแปร State เข้ากับช่องป้อนค่าแบบ Controlled Components ได้สมบูรณ์

4. ผู้เรียนสามารถประยุกต์ใช้รูปแบบ Lifting State Up เพื่อจัดโครงสร้างให้คอมโพเนนต์ลูกส่งค่ากลับไปบันทึกยังสเตตของคอมโพเนนต์แม่ได้สำเร็จ

5. ผู้เรียนสามารถเขียนโค้ดตรวจสอบเงื่อนไขความถูกต้องของข้อมูลฟอร์ม (เช่น ความยาวรหัสผ่าน หรือโครงสร้างอีเมล) พร้อมแสดงสถานะแจ้งข้อผิดพลาดใต้ช่องอินพุตได้ถูกต้อง

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

13. **การบรรยายและจำลองแบบทดลอง (Visual State Flow Lecture):** ผู้สอนนำเสนอสไลด์ภาพและยกตัวอย่างแผนภาพกระแสข้อมูลไหลจากบนลงล่าง และการ

ก้าวข้ามขีดจำกัดด้วยการทำ Callback ส่งค่าอัปเดตสถานะกลับขึ้นด้านบน

14. **การทำ Live Coding Demonstration หัวข้อ Interactive Form Setup:** ผู้สอนนำเสนอการสร้างฟอร์มสมัครสมาชิก (Sign Up Form) ที่มีช่องกรอกชื่อ อีเมล รหัสผ่าน และมีการตรวจสอบข้อมูลแบบเรียลไทม์เพื่อปลดล็อกปุ่มกดส่ง
15. **กิจกรรมปฏิบัติกลุ่มย่อยท้าทายสเตต (Form Validation Challenge):** แบ่งกลุ่มผู้เรียนทดลองแก้บั๊กในโค้ดฟอร์มลงทะเบียน โดยต้องเขียนฟังก์ชันตรวจสอบว่าอีเมลมีเครื่องหมาย @ หรือไม่ และแสดงกรอบช่องเป็นสีแดงหากกรอกข้อมูลผิดพลาด
16. **กิจกรรมเขียนระบบรวมการ์ด (Lifting State to Preview Card):** ให้ผู้เรียนฝึกประดิษฐ์ Custom Input Component และส่งสัญญาณเหตุการณ์เปลี่ยนข้อความส่งต่อไปยังการ์ดพรีวิวบนสุดให้เปลี่ยนตามการพิมพ์จริง

สื่อการเรียนการสอน

13. เอกสารสไลด์นำเสนอ "State, Props, and Controlled Forms in React Native"
14. ตัวอย่างโค้ดแอปพลิเคชันลงทะเบียนสมาชิกพร้อมฟังก์ชันตรวจสอบความถูกต้องของข้อมูล (Form Validation Demo) บน Git Repository
15. โปรแกรมจำลองหน้าจอและอุปกรณ์จริงสำหรับการทดสอบระบบตอบสนองแป้นพิมพ์และสเตตอินพุต

การวัดผลและการประเมินผล

1. การวัดผล

1.1 วัดความเข้าใจจากแบบทดสอบปรนัยท้ายบทเรียน 10 ข้อ

1.2 ประเมินความถูกต้องของโครงสร้าง Controlled Components และการจัดสเตตอินพุต

1.3 ตรวจประเมินทักษะการตรวจสอบความถูกต้องของฟอร์ม (Form Validation) และการยกสถานะ (Lifting State Up) ในแล็บปฏิบัติการ

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถเขียนโปรแกรมฟอร์มรับข้อมูลที่มีฟังก์ชันตรวจสอบความถูกต้องและโชว์ปุ่มส่งค่าตอบสนองสถานะได้สมบูรณ์ผ่านเกณฑ์ปฏิบัติการ (ผ่าน/ไม่ผ่าน)

2.3 การประเมินพฤติกรรมการแลกเปลี่ยนความคิดเห็นและการช่วยเหลือกันตอบโจทย์ข้อผิดพลาดในคาบเรียน

บทที่ 4

การจัดการสถานะ พร็อพส์ และแบบฟอร์มรับข้อมูล

อินเทอร์เฟซของโมบายแอปพลิเคชันที่มีการตอบโต้ที่ดีจำเป็นต้องประมวลผลการเปลี่ยนแปลงข้อมูลตามพฤติกรรมของผู้ใช้ เช่น การพิมพ์ข้อความ การสลับหน้าจอปุ่มตัวเลือก หรือการกดส่งฟอร์ม ใน React Native เราใช้แนวคิดของ **State (สถานะ)** ร่วมกับ **Props (คุณสมบัติ)** ในการเป็นตัวควบคุมโครงสร้างข้อมูลและการเรนเดอร์เนื้อหาใหม่ (Re-render) เพื่ออัปเดตหน้าจอให้ทันสมัยโดยอัตโนมัติ

4.1 แนวคิดของสถานะ (State) ใน React Native และการใช้งาน useState Hook

1. State คืออะไร?

State คือ ข้อมูลภายในคอมโพเนนต์ (React, 2024) ที่สามารถเปลี่ยนแปลงค่าได้ตลอดเวลา (Mutable) ในแต่ละรอบอายุการทำงาน (Lifecycle) ของคอมโพเนนต์ เมื่อค่าของ State ถูกแก้ไขด้วยฟังก์ชันเฉพาะ React จะทำการวาดคอมโพเนนต์ย่อยชิ้นนั้น ๆ ใหม่บนหน้าจอ (Re-render) ทันที เพื่อสะท้อนการเปลี่ยนแปลงให้ผู้ใช้งานเห็นหน้าจอใหม่

2. การเรียกใช้งาน useState Hook

ใน Functional Components เราประกาศสถานะเริ่มต้นและฟังก์ชันในการเปลี่ยนแปลงค่าสเตตผ่าน **useState** Hook (React, 2024):

```
import { useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

export default function CounterApp() {
 // ประกาศ State ชื่อ count มีค่าเริ่มต้นเป็น 0 และมีฟังก์ชัน setCount ในการแก้ไขค่า
  const [count, setCount] = useState<number>(0);

  return (
    <View style={styles.container}>
      <Text style={styles.text}>จำนวนคลิกปัจจุบัน: {count}</Text>
      {/* เมื่อคลิกปุ่ม ให้เพิ่มค่า count ขึ้นทีละ 1 */}
      <Button title="กดเพิ่มจำนวน" onPress={() => setCount(count + 1)} />
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
```

```
    alignItems: 'center',
  },
  text: {
    fontSize: 20,
    marginBottom: 20,
  },
});
```

4.2 ข้อแตกต่างและการทำงานร่วมกันระหว่าง State และ Props

การทำความเข้าใจข้อแตกต่างระหว่าง State และ Props ถือเป็นรากฐานสำคัญของการจัดสถาปัตยกรรมข้อมูลใน React Native:

ตารางที่ 4.1 การเปรียบเทียบคุณลักษณะระหว่าง State และ Props

คุณลักษณะ	State (สถานะภายใน)	Props (คุณสมบัติภายนอก)
ที่มา (Source)	กำหนดและสร้างขึ้นภายในคอมโพเนนต์ตัวเอง	ส่งผ่านมาจากคอมโพเนนต์แม่ (Parent)
การแก้ไขค่า (Mutability)	เปลี่ยนแปลงได้อิสระโดยเรียกใช้ฟังก์ชัน Setter (เช่น setCount)	เปลี่ยนแปลงค่าไม่ได้จากภายในลูก (ใช้อ่านอย่างเดียว - Read-Only)
ตัวเหนี่ยวนำให้เกิดการวาดหน้าจอใหม่ (Re-render Trigger)	ส่งผลให้เรนเดอร์ใหม่ทันทีเมื่อเรียกฟังก์ชันอัปเดตค่า	ส่งผลให้คอมโพเนนต์ลูกเรนเดอร์ใหม่ทันทีเมื่อแม่เปลี่ยนค่า Props
บทบาทหน้าที่หลัก	เก็บข้อมูลประวัติย่อย ความชอบ หรืออินพุตที่ผู้ใช้กำลังพิมพ์	ส่งผ่านการตั้งค่า หน้าตา หรือสั่งทำเหตุการณ์กลับไปหาแม่

4.3 การทำแบบฟอร์มด้วย Controlled Components ผ่าน TextInput

ในการพัฒนาเว็บหรือแอปแบบดั้งเดิม ตัวอัปเดตข้อความจะถูกผูกไว้ภายในระบบของแป้นพิมพ์หรือ DOM ดั้งเดิม แต่ใน React Native การพัฒนาแบบฟอร์มที่เป็นระเบียบจะใช้แนวคิด **Controlled Components** (React Native, 2024) ซึ่งหมายถึงการนำ State ของ React มาเป็น "แหล่งข้อมูลที่ถูกต้องเพียงหนึ่งเดียว (Single Source of Truth)" สำหรับคอมโพเนนต์อินพุตนั้น ๆ

1. วิธีการสร้าง Controlled Input

เราจะสร้าง State คอยบันทึกตัวอักษร และผูกพร็อพส์ value เข้ากับสเตตนั้น พร้อมสั่งการผ่านเหตุการณ์ onChangeText ให้นำข้อความสตริงล่าสุดไปอัปเดตสเตตทันที:

```
import { useState } from 'react';
import { View, TextInput, Text, StyleSheet } from 'react-native';

export default function ControlledInput() {
  const [username, setUsername] = useState<string>('');

  return (
    <View style={styles.container}>
   <TextInput
        style={styles.input}
        value={username}
        onChangeText={setUsername} // มีค่าเท่ากับ (text) => setUsername(text)
        placeholder="กรอกชื่อผู้ใช้ของคุณ..."
   />
      <Text style={styles.previewText}>ค่าสเตตปัจจุบัน: {username}</Text>
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    padding: 20,
  },
  input: {
    borderWidth: 1,
    borderColor: '#94a3b8',
    borderRadius: 8,
    padding: 12,
    fontSize: 16,
  },
  previewText: {
    marginTop: 10,
    color: '#64748b',
  },
});
```

4.4 การแบ่งปันข้อมูลย้อนกลับด้วยหลักการ Lifting State Up

ข้อมูลใน React จะไหลเวียนไปในทิศทางเดียวจากบนลงล่าง (Parent to Child) หากเราต้องการให้คอมโพเนนต์ลูกสลับกันเปลี่ยนแปลง หรือส่งข้อมูลอินพุตจากช่องกรอกในคอมโพเนนต์ลูก

ขึ้นไปเปลี่ยนค่าที่คอมโพเนนต์แม่ เราจำเป็นต้องประยุกต์ใช้เทคนิค **Lifting State Up (การยกสถานะขึ้น)**

แนวปฏิบัติของการ Lifting State Up

1. **ยกตำแหน่งเก็บ State:** นำ State ไปเก็บไว้ในคอมโพเนนต์ที่เป็นบรรพบุรุษร่วมที่ใกล้ที่สุด (Common Ancestor)

2. **ส่งผ่านฟังก์ชันอัปเดตค่า:** ตัวคอมโพเนนต์แม่จะสร้างฟังก์ชันการประมวลผล แล้วจัดส่งฟังก์ชันนี้ไปเป็นหนึ่งใน Props ของคอมโพเนนต์ลูก

3. **การโทรกลับ (Callback):** เมื่อลูกสัมผัสหรือพิมพ์รับข้อความ จะทำการกระตุ้นฟังก์ชัน Props นั้น ส่งข้อมูลย้อนกลับขึ้นไปอัปเดตค่าสเตตบนคอมโพเนนต์แม่

```
// 1. ชิ้นส่วนลูก (Child) - ช่องป้อนข้อมูล
interface SearchInputProps {
  query: string;
  onQueryChange: (text: string) => void;
}

function SearchInput({ query, onQueryChange }: SearchInputProps) {
  return (
    <TextInput
      value={query}
      onChangeText={onQueryChange} // กระตุ้น Callback ส่งค่ากลับขึ้นด้านบน
      placeholder="พิมพ์ค้นหาชื่อสินค้า..."
      style={{ borderWidth: 1, padding: 8, borderRadius: 6 }}
  />
 );
}

// 2. ชิ้นส่วนแม่ (Parent) - เก็บสเตตหลัก
import { useState } from 'react';
import { View, Text } from 'react-native';

export default function ProductCatalog() {
  const [searchQuery, setSearchQuery] = useState<string>('');

  return (
    <View style={{ padding: 20 }}>
      <Text style={{ fontSize: 18, marginBottom: 8 }}>แคตตาล็อกสินค้า</Text>
      {/* ส่งผ่านทั้งค่าสเตตและฟังก์ชันอัปเดตสเตตไปให้คอมโพเนนต์ลูก */}
      <SearchInput query={searchQuery} onQueryChange={setSearchQuery} />
      <Text style={{ marginTop: 10 }}>คุณกำลังค้นหาคำว่า: {searchQuery}</Text>
  </View>
 );
}
```

4.5 การจัดทำฟอร์มคำนวณค่าดัชนีมวลกาย (BMI Calculation Form)

นอกจากการพิมพ์เก็บข้อความธรรมดาแล้ว แบบฟอร์มในโมบายแอปพลิเคชันยังนิยมใช้ในการประมวลผลคำนวณทางคณิตศาสตร์จากตัวเลขอินพุตที่ผู้ใช้พิมพ์เข้ามา เช่น ยอดราคาสินค้ารวม หรือดัชนีสุขภาพ โดยข้อมูลอินพุตจาก **TextInput** จะถูกส่งคืนมาเป็นประเภทสตริง (**string**) เสมอ ดังนั้น ผู้พัฒนาจึงต้องแปลงค่าเป็นตัวเลขจริงก่อนเริ่มการคำนวณเพื่อป้องกันข้อผิดพลาดเชิงโปรแกรม (NaN)

1. วิธีการผูกและแปลงค่าตัวเลข (Numeric Binding & Parsing)

• `keyboardType="numeric"`: เปิดแป้นพิมพ์ตัวเลข

• `parseFloat()` / `parseInt()`: ใช้สำหรับแปลงข้อความที่พิมพ์เป็นตัวเลขจำนวนจริงหรือทศนิยม

• **การป้องกันความผิดพลาด (Defensive Programming):** ตรวจสอบว่าผู้ใช้กรอกข้อความว่างเปล่า หรือหารด้วยศูนย์ (Division by Zero) หรือไม่ ก่อนทำรายการคำนวณ

2. โค้ดตัวอย่างคอมโพเนนต์เครื่องคำนวณดัชนีมวลกาย (BMI Calculator)

```
import { useState } from 'react';
import { View, Text, TextInput, Pressable, StyleSheet } from 'react-
native';

export default function BMICalculator() {
  const [weight, setWeight] = useState<string>(''); // เก็บน้ำหนัก (กิโลกรัม) เป็น
ข้อความ
  const [height, setHeight] = useState<string>(''); // เก็บส่วนสูง (เซนติเมตร) เป็น
ข้อความ
  const [bmi, setBmi] = useState<number | null>(null); // เก็บคะแนน BMI คำนวณ
เสร็จ
  const [category, setCategory] = useState<string>(''); // เก็บเกณฑ์สุขภาพ

  const handleCalculate = () => {
    const w = parseFloat(weight);
    const h = parseFloat(height) / 100; // แปลงหน่วยเซนติเมตรเป็นเมตร

    if (w > 0 && h > 0) {
      const calculatedBmi = w / (h * h);
      setBmi(calculatedBmi);

    // แปลผลเกณฑ์ BMI (อ้างอิงตามเกณฑ์เอเชียแปซิฟิก)
      if (calculatedBmi < 18.5) {
```

```
        setCategory('น้ำหนักน้อยกว่าเกณฑ์ (ผอม)');
      } else if (calculatedBmi < 23) {
        setCategory('น้ำหนักปกติ (สุขภาพดี)');
      } else if (calculatedBmi < 25) {
        setCategory('น้ำหนักเกิน (ท้วม)');
      } else {
        setCategory('ภาวะอ้วน (Obese)');
      }
    } else {
      setBmi(null);
      setCategory('กรุณากรอกข้อมูลส่วนสูงและน้ำหนักให้ครบถ้วนถูกต้อง');
    }
  };

  return (
    <View style={styles.card}>
      <Text style={styles.cardTitle}>เครื่องคำนวณหาค่า BMI</Text>

      {/* ช่องกรอกน้ำหนัก */}
      <Text style={styles.label}>น้ำหนักตัว (กิโลกรัม)</Text>
      <TextInput
        style={styles.input}
        placeholder="ตัวอย่าง 65"
        placeholderTextColor="#64748b"
        keyboardType="numeric"
        value={weight}
        onChangeText={setWeight}
      />

      {/* ช่องกรอกส่วนสูง */}
      <Text style={styles.label}>ส่วนสูง (เซนติเมตร)</Text>
      <TextInput
        style={styles.input}
        placeholder="ตัวอย่าง 170"
        placeholderTextColor="#64748b"
        keyboardType="numeric"
        value={height}
        onChangeText={setHeight}
      />

      {/* ปุ่มสั่งคำนวณจัดด้วย Pressable */}
      <Pressable
        onPress={handleCalculate}
        style={({ pressed }) => [
          styles.button,
          pressed ? styles.buttonPressed : null
        ]}
      >
```

```
        <Text style={styles.buttonText}>ประมวลผลดัชนีมวลกาย</Text>
      </Pressable>

      {/* พื้นที่แสดงผลการคำนวณแบบมีเงื่อนไข (Conditional Rendering) */}
      {bmi !== null && (
        <View style={styles.resultBox}>
          <Text style={styles.resultBmiText}>ค่า BMI ของคุณ:
{bmi.toFixed(1)}</Text>
          <Text style={styles.resultCatText}>เกณฑ์ของคุณ: {category}</Text>
        </View>
   )}
    </View>
 );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: '#1e293b',
    padding: 16,
    borderRadius: 12,
    borderWidth: 1,
    borderColor: '#334155',
    marginVertical: 16,
  },
  cardTitle: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
    marginBottom: 12,
  },
  label: {
    color: '#cbd5e1',
    fontSize: 13,
    marginBottom: 6,
  },
  input: {
    backgroundColor: '#0f172a',
    borderWidth: 1,
    borderColor: '#334155',
    borderRadius: 8,
    height: 44,
    paddingHorizontal: 12,
    color: '#ffffff',
    marginBottom: 12,
  },
  button: {
```

```
    backgroundColor: '#10b981', // สีเขียวนีออนสำหรับคำนวณสุขภาพ
    height: 44,
    borderRadius: 8,
    justifyContent: 'center',
    alignItems: 'center',
  },
  buttonPressed: {
    backgroundColor: '#059669',
  },
  buttonText: {
    color: '#ffffff',
    fontWeight: 'bold',
  },
  resultBox: {
    marginTop: 16,
    padding: 12,
    backgroundColor: '#0f172a',
    borderRadius: 8,
    alignItems: 'center',
  },
  resultBmiText: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
  resultCatText: {
    color: '#38bdf8',
    fontSize: 14,
    marginTop: 4,
  },
});
```

4.6 การตรวจสอบความถูกต้องของฟอร์ม (Form Validation) และการแจ้งเตือนข้อผิดพลาด (Error Feedback)

แอปพลิเคชันคุณภาพสูงต้องช่วยป้องกันไม่ให้ผู้ใช้ส่งข้อมูลที่ผิดรูปแบบไปยังเซิร์ฟเวอร์ โดยระบบควรทำการตรวจสอบข้อผิดพลาด (Validation) ทันทีบนตัวเครื่อง (Client-side) และแสดงคำเตือนอย่างเด่นชัด

1. วิธีตรวจสอบข้อมูลแบบเรียลไทม์ (Real-time Validation)

เราสามารถตรวจสอบค่าของตัวแปรสเตตในแต่ละจังหวะการพิมพ์ และเก็บข้อความผิดพลาดลงในสถานะ `errors` หรือคำนวณจากสถานะปัจจุบันได้โดยตรง:

```
// ตรวจสอบว่ารูปแบบอีเมลถูกต้องหรือไม่โดยใช้ Regular Expression (Regex)
const validateEmail = (emailText: string) => {
```

```
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(emailText)) {
    return 'รูปแบบอีเมลไม่ถูกต้อง';
  }
  return '';
};
```

2. การจัดการสถานะปุ่มกดสัมผัสและ Error UI

• **กล่องรับข้อความสีส้ม/แดง:** กำหนดสไตล์ขอบช่องพิมพ์ให้แดงขึ้นเมื่อสเตตข้อผิดพลาดไม่มีค่าว่าง

• **การสั่งปิดความสามารถของปุ่มสัมผัส (Disable Button):** นำค่าความถูกต้องของข้อมูล (เช่น `isFormValid`) มาเป็นตัวกำหนดค่าการตอบสนองของ `Pressable` หากฟอร์มยังกรอกไม่สมบูรณ์ ปุ่มจะจางลงและไม่รับคำสั่งนิ้วสัมผัส

4.7 โค้ดตัวอย่างการประกอบฟอร์มลงทะเบียนพร้อมระบบตรวจสอบแบบเรียลไทม์

โค้ดเต็มด้านล่างนี้แสดงวิธีการรวบรวมสถานะหลายช่องรับอินพุต การทำ Validation ข้อมูล และการปรับแต่งปุ่มสัมผัสอย่างลื่นไหลเมื่อเงื่อนไขถูกต้องสมบูรณ์:

```
import { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  Pressable,
  SafeAreaView,
  Alert
} from 'react-native';

export default function RegisterForm() {
 // 1. ประกาศตัวแปรสำหรับผูกฟอร์มรับอินพุต
  const [email, setEmail] = useState<string>('');
  const [password, setPassword] = useState<string>('');

 // 2. ประกาศตัวแปรเก็บสถานะขอบบกพร่อง
  const [emailError, setEmailError] = useState<string>('');
  const [passwordError, setPasswordError] = useState<string>('');

 // 3. ฟังก์ชันดักจับความถูกต้องของอีเมล
  const handleEmailChange = (text: string) => {
    setEmail(text);
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (text.trim() === '') {
      setEmailError('กรุณากรอกอีเมลของคุณ');
    } else if (!emailRegex.test(text)) {
```

```
      setEmailError('อีเมลต้องเป็นรูปแบบที่ถูกต้อง เช่น name@domain.com');
    } else {
      setEmailError('');
    }
  };

  // 4. ฟังก์ชันดักจับความยาวรหัสผ่าน
  const handlePasswordChange = (text: string) => {
    setPassword(text);
    if (text.trim() === '') {
      setPasswordError('กรุณากรอกรหัสผ่าน');
    } else if (text.length < 6){
      setPasswordError('รหัสผ่านต้องมีความยาวอย่างน้อย 6 ตัวอักษร');
    } else {
      setPasswordError('');
    }
  };

  // 5. ประเมินสัญญานว่าฟอร์มโดยรวมสมบูรณ์พร้อมส่งหรือไม่
  const isFormValid =
    email.trim() !== '' &&
    password.trim() !== '' &&
    emailError === '' &&
    passwordError === '';

  const handleSubmit = () => {
    if (isFormValid) {
      Alert.alert('ลงทะเบียนสำเร็จ', `อีเมล: ${email}\nรหัสผ่านของคุณปลอดภัยเรียบร้อย`);
    }
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <View style={styles.container}>
        <Text style={styles.title}>สร้างบัญชีสมาชิก</Text>

        {/* ฟอร์มรับอีเมล */}
        <View style={styles.inputGroup}>
          <Text style={styles.label}>ที่อยู่อีเมล</Text>
          <TextInput
            style={[
              styles.input,
              emailError ? styles.inputInvalid : null
      ]}
            placeholder="ตัวอย่าง user@email.com"
            placeholderTextColor="#64748b"
            value={email}
            onChangeText={handleEmailChange}
            keyboardType="email-address"
            autoCapitalize="none"
```

```
      />
        {emailError ? <Text style={styles.errorText}>{emailError}</Text> :
null}
        </View>

        {/* ฟอร์มรับรหัสผ่าน */}
        <View style={styles.inputGroup}>
          <Text style={styles.label}>รหัสผ่าน (Password)</Text>
          <TextInput
            style={[
              styles.input,
              passwordError ? styles.inputInvalid : null
      ]}
            placeholder="ตั้งรหัสผ่านอย่างน้อย 6 ตัว"
            placeholderTextColor="#64748b"
            value={password}
          onChangeText={handlePasswordChange}
            secureTextEntry={true}
            autoCapitalize="none"
      />
          {passwordError ? <Text
style={styles.errorText}>{passwordError}</Text> : null}
        </View>

        {/* ปุ่มบันทึกข้อมูลโต้ตอบตามความถูกต้องของข้อมูล */}
        <Pressable
          disabled={!isFormValid} // ล็อกไม่ให้คลิกปุ่มหากฟอร์มไม่สมบูรณ์
          onPress={handleSubmit}
          style={({ pressed }) => [
            styles.submitButton,
      !isFormValid ? styles.submitButtonDisabled : null,
            pressed && isFormValid ? styles.submitButtonPressed : null
      ]}
        >
          <Text style={styles.submitButtonText}>ลงทะเบียนบัญชีใหม่</Text>
        </Pressable>

      </View>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
    justifyContent: 'center',
  },
  container: {
```

```
    padding: 24,
    marginHorizontal: 16,
    backgroundColor: '#1e293b',
    borderRadius: 16,
    borderWidth: 1,
    borderColor: '#334155',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
    textAlign: 'center',
    marginBottom: 24,
  },
  inputGroup: {
    marginBottom: 16,
  },
  label: {
    fontSize: 14,
   color: '#cbd5e1',
    marginBottom: 6,
    fontWeight: '500',
  },
  input: {
    height: 48,
    backgroundColor: '#0f172a',
    borderWidth: 1,
    borderColor: '#334155',
    borderRadius: 8,
    paddingHorizontal: 12,
    color: '#ffffff',
    fontSize: 15,
  },
  inputInvalid: {
    borderColor: '#f43f5e', // เปลี่ยนกรอบเป็นสีส้มแดงเตือนภัย
  },
  errorText: {
    fontSize: 12,
    color: '#f43f5e',
    marginTop: 6,
  },
  submitButton: {
    backgroundColor: '#3b82f6',
    height: 48,
    borderRadius: 8,
    justifyContent: 'center',
    alignItems: 'center',
```

```
    marginTop: 10,
  },
  submitButtonDisabled: {
    backgroundColor: '#1e293b',
    borderColor: '#334155',
    borderWidth: 1,
    opacity: 0.5,
  },
  submitButtonPressed: {
    backgroundColor: '#2563eb',
  },
  submitButtonText: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
});
```

สรุปท้ายบทเรียน

แนวคิดของสถานะ (State) ใน React Native และการใช้งาน useState Hook: สถานะ (State) คือข้อมูลภายในคอมโพเนนต์ที่เปลี่ยนค่าได้ โดยการอัปเดตสเตตผ่านฟังก์ชันแก้ไขของ useState Hook จะกระตุ้นให้ React ทำการวาดอินเตอร์เฟซบนหน้าจอใหม่ (Re-render) โดยอัตโนมัติ เพื่อนำผลลัพธ์ของข้อมูลล่าสุดไปแสดงผลให้กับผู้ใช้ได้อย่างถูกต้องทันที

ข้อแตกต่างและการทำงานร่วมกันระหว่าง State และ Props: State ทำหน้าที่เก็บและจัดการข้อมูลที่มีการเคลื่อนไหวอยู่ภายในคอมโพเนนต์ชิ้นนั้น ๆ ในขณะที่ Props เป็นคุณสมบัติภายนอกที่ส่งผ่านจากคอมโพเนนต์แม่ลงมายังคอมโพเนนต์ลูกแบบอ่านได้อย่างเดียว (Read-Only) ซึ่งการประสานงานระหว่างสองอย่างนี้จะเหนี่ยวนำให้คอมโพเนนต์ลูกเรนเดอร์ใหม่โดยอัตโนมัติเมื่อข้อมูลมีการเปลี่ยนแปลง

การทำแบบฟอร์มด้วย Controlled Components ผ่าน TextInput: การจัดการอินพุตฟอร์มใน React Native ยึดหลัก Controlled Components โดยมีสเตตเป็นแหล่งข้อมูลที่ถูกต้องที่สุดเพียงแหล่งเดียว (Single Source of Truth) ซึ่งเป็นการผูกสเตตเข้ากับคุณสมบัติ value ของกล่องรับข้อมูลข้อความ TextInput และกระตุ้นฟังก์ชันเซ็ตเตอร์ผ่านเหตุการณ์ onChangeText เพื่อเก็บตัวอักษรทุกตัวลงในสเตตแบบเรียลไทม์

การแบ่งปันข้อมูลย้อนกลับด้วยหลักการ Lifting State Up: เพื่อรองรับทิศทางการไหลเวียนข้อมูลทางเดียวจากแม่ไปลูก (One-way Data Flow) การส่งผ่านข้อมูลย้อนกลับจากลูกขึ้นไปหาแม่จะใช้เทคนิค Lifting State Up โดยการย้ายสเตตหลักขึ้นไปไว้บนคอมโพเนนต์บรรพบุรุษ

ร่วม แล้วส่งฟังก์ชันการจัดการ (Callback Function) ผ่าน Props ลงมาให้คอมโพเนนต์ลูกเรียกใช้งานเมื่อต้องการส่งสัญญาณการเปลี่ยนแปลง

การจัดทำฟอร์มคำนวณค่าดัชนีมวลกาย (BMI Calculation Form): การสร้างฟอร์มสำหรับคำนวณ เช่น เครื่องหาดัชนีมวลกาย (BMI) จะใช้ TextInput ควบคู่ไปกับแป้นพิมพ์ตัวเลข (keyboardType="numeric") และเมื่อสั่งทำงานจะต้องทำการแปลงประเภทข้อมูลจากข้อความสตริงเป็นตัวเลขจำนวนจริงผ่าน parseFloat() ก่อนเริ่มกระบวนการคำนวณ พร้อมทั้งจัดระบบป้องกันการคำนวณที่ผิดพลาดจากข้อมูลที่ว่างเปล่าหรือเป็นศูนย์

การตรวจสอบความถูกต้องของฟอร์มและการแจ้งเตือนข้อผิดพลาด: ระบบแบบฟอร์มที่ดีจำเป็นต้องมี Form Validation เพื่อตรวจสอบความถูกต้องของอีเมลหรือรหัสผ่านทันทีในระบบ Client-side ก่อนนำส่งข้อมูลจริง โดยผู้พัฒนาสามารถเก็บข้อมูลข้อผิดพลาดไว้ในสเตตและนำสัญญานตรวจสอบนั้นมาควบคุมการปิดฟังก์ชันปุ่มกด (disabled) รวมถึงเปลี่ยนดีไซน์สีของกรอบอินพุตให้สะดุดตาเพื่อสร้างการตอบสนองที่เข้าใจง่าย

โค้ดตัวอย่างการประกอบฟอร์มลงทะเบียนพร้อมระบบตรวจสอบแบบเรียลไทม์: ตัวอย่างการประกอบฟอร์มลงทะเบียนแสดงผลความสอดคล้องของการนำ TextInput มารับค่าอีเมลและรหัสผ่าน โดยใช้ฟังก์ชันประเมินความถูกต้องของตัวอักษรเพื่ออัปเดตสเตตความผิดพลาดแบบวินาทีต่อวินาที ซึ่งช่วยควบคุมการทำงานของปุ่ม Pressable และเปิดโอกาสให้ผู้ใช้งานกดยืนยันการทำธุรกรรมได้เฉพาะเมื่อข้อมูลในทุกช่องมีรูปแบบที่สมบูรณ์และปลอดภัย

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความเข้าใจเกี่ยวกับแนวคิดเรื่องสถานะ (State) การทำงานร่วมกับ Props โครงสร้าง Controlled Components ของแบบฟอร์ม การส่งค่าข้อมูลกลับขึ้นด้านบนด้วยหลัก Lifting State Up และการดักจับข้อผิดพลาดอินพุต (Form Validation) ใน React Native Expo SDK 54

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดอธิบายวัตถุประสงค์และพฤติกรรมการทำงานหลักของ "State" ใน React Native ได้ถูกต้องและสมบูรณ์ที่สุด?

ก. เป็นสไตล์ชีทภายนอกที่ใช้อ่านข้อมูลจากเครื่องระบบปฏิบัติการปลายทางเท่านั้น

ข. เป็นชุดตัวแปรภายในคอมโพเนนต์ที่สามารถเปลี่ยนค่าข้อมูลได้ โดยระบบจะบังคับทำการวาดโครงสร้างหน้าจอใหม่ (Re-render) อัตโนมัติเมื่อค่านั้นเปลี่ยนไป

ค. เป็นพร็อพส์ภายนอกที่ส่งมาจากคอมโพเนนต์แม่เพื่อสแกนล้างหน่วยความจำ RAM

ง. เป็นฟังก์ชันสำหรับส่งข้อความสั้นหรือแจ้งเตือนแบบพุช (Push Notification) ไปยังมือถือเครื่องอื่น

ข้อ 2: เมื่อต้องการประกาศใช้งานตัวแปรสถานะภายใน Functional Component ผู้พัฒนาต้องเรียกใช้งาน React Hook ตัวใด?

ก. `useEffect`

ข. `useRef`

ค. `useState`

ง. `useMemo`

ข้อ 3: การประกาศรูปแบบฟังก์ชันสถานะตามหลักการทั่วไป เช่น `const [isOnline, setIsOnline] = useState(false);` ข้อใดกล่าวถึงบทบาทหน้าที่ของตัวแปร `setIsOnline` ได้ถูกต้อง?

ก. เป็นตัวแปรเก็บสถานะปัจจุบัน มีค่าประเภทข้อความ (String) เสมอ

ข. เป็นฟังก์ชันที่ทำหน้าที่เดียวในการลบตัวแปร `isOnline` ออกจากสเตตระบบ

ค. เป็นฟังก์ชันประเภท Setter ที่ใช้ในการอัปเดตและเปลี่ยนแปลงค่าของสถานะ `isOnline` เพื่อส่งสัญญาณจัดทำ Re-render หน้าจอ

ง. เป็นคำสั่งที่ส่งไปสร้างจุดกลมสีเขียวนีออนแสดงการเชื่อมต่อเน็ตเวิร์กของเครื่องอัตโนมัติ

ข้อ 4: ข้อใดคือความแตกต่างทางด้านสิทธิ์การแก้ไขค่า (Mutability) ระหว่าง State และ Props ของ React/React Native?

ก. State มีลักษณะใช้อ่านอย่างเดียว (Read-only) ส่วน Props สามารถเขียนโค้ดสลับเปลี่ยนค่าด้านในได้อย่างอิสระ

ข. State สามารถเปลี่ยนแปลงค่าได้ตามจังหวะพิมพ์สัมผัส ส่วน Props มีลักษณะใช้อ่านอย่างเดียวจากมุมมองของคอมโพเนนต์ลูกที่รับไป

ค. ทั้ง State และ Props ต่างไม่ยอมรับการเปลี่ยนแปลงค่าเลยเมื่อแอปพลิเคชันถูกบิลด์เสร็จ

ง. State ใช้แก้ไขเฉพาะประเภทข้อมูลตัวเลข ส่วน Props ใช้แก้ไขประเภทข้อความตัวอักษรเท่านั้น

ข้อ 5: แนวคิดแบบ Controlled Component ของช่องรับอินพุต `TextInput` มีลักษณะการจัดระเบียบข้อมูลอย่างไร?

ก. ปล่อยให้แป้นพิมพ์ของระบบควบคุมและจัดเก็บตัวหนังสือไว้ในฮาร์ดแวร์โดยตรง

ข. นำ State ของ React มาเป็นศูนย์กลางข้อมูลผูกติดกับคอมโพเนนต์ เพื่อคอยควบคุมและสะท้อนค่าตัวหนังสือปัจจุบันเข้าหากันตลอดเวลา

ค. ปิดแป้นคีย์บอร์ดทั้งหมดและรับข้อมูลด้วยวิธีลากผ่านแถบเสียงเท่านั้น

ง. ดึงสไตล์จากไฟล์ภายนอกมาควบคุมและตรวจสอบสีของหน้าแบบฟอร์ม

ข้อ 6: พร็อพส์ (Props) หลักคู่ใดของคอมโพเนนต์ `TextInput` ที่ต้องเขียนผูกทำงานร่วมกันเพื่อให้มีพฤติกรรมเป็น Controlled Component ที่สมบูรณ์?

ก. `placeholder` และ `keyboardType`

ข. `value` และ `onChangeText`

ค. `secureTextEntry` และ `autoCapitalize`

ง. `editable` และ `maxLength`

ข้อ 7: ในกรณีที่คอมโพเนนต์ลูกสองชิ้นที่ไม่มีความสัมพันธ์เป็นแม่-ลูกกันโดยตรง (เช่น ปุ่มสลับธีม และการ์ดแสดงสี) ต้องการใช้สถานะข้อมูลเดียวกัน วิธีการจัดสถาปัตยกรรมสเตตที่ดีที่สุดคืออะไร?

ก. เขียนคัดลอกไฟล์ State แบบเดียวกันไว้ในคอมโพเนนต์ลูกทั้งสองที่แยกอิสระจากกัน

ข. ประยุกต์ใช้เทคนิค "Lifting State Up" โดยย้าย State ขึ้นไปไว้ที่คอมโพเนนต์ที่เป็นบรรพบุรุษร่วมกันที่ใกล้ที่สุด (Common Parent)

ค. ยกเลิกการใช้ State และใช้รูปแบบ CSS Inline ในการเปลี่ยนสี

ง. ส่งข้อมูลผ่านทางเครือข่ายอินเทอร์เน็ตเพื่อเก็บข้อมูลที่ฐานข้อมูลกลางทุกรอบพิมพ์

ข้อ 8: คอมโพเนนต์ลูก (Child Component) สามารถส่งข้อมูลกลับขึ้นไปแจ้งเปลี่ยนแปลงหรืออัปเดตสเตตในคอมโพเนนต์แม่ (Parent Component) ได้อย่างไร?

ก. เรียกใช้ฟังก์ชัน `useState` ของแม่ตรง ๆ ผ่านการอ้างอิงชื่อคลาส

ข. แม่ต้องส่งฟังก์ชันประเภท Callback (ฟังก์ชันอัปเดตค่า) ลงไปให้ลูกในรูปแบบของ Props และให้ลูกเรียกใช้ฟังก์ชันนั้นพร้อมป้อนข้อมูลล่าสุดเป็นอาร์กิวเมนต์

ค. เขียนเปลี่ยนสเตตลงในไฟล์สไตล์ชีทภายนอกเพื่อให้คอมไพเลอร์เปลี่ยนค่า

ง. ไม่มีวิธีการตามข้อกำหนดของ React เนื่องจากข้อมูลไหลได้เฉพาะทิศทางลงล่างเท่านั้น

ข้อ 9: ข้อใดคือเป้าหมายหลักของการเขียนตรวจสอบความถูกต้องแบบฟอร์มในฝั่งไคลเอนต์ (Client-Side Form Validation)?

ก. เพื่อสแกนและเก็บฐานข้อมูลรหัสผ่านส่งต่อไปยังเซิร์ฟเวอร์แบบลับ ๆ

ข. เพื่อให้ผู้ใช้สัมผัสปุ่มกดและเห็นการตอบสนองได้ช้าลงเล็กน้อย

ค. เพื่อให้ข้อมูลที่ป้อนไม่ขัดต่อเงื่อนไขเบื้องต้น ส่งผลให้แสดงข้อเสนอแนะความผิดพลาด (Error Feedback) แก่ผู้ใช้ได้ทันที และช่วยลดการส่งรีเควสขยะไปยังระบบหลังบ้าน

ง. เพื่อเร่งความเร็วการส่งไฟล์รูปภาพขึ้นไคล์เซิร์ฟเวอร์

ข้อ 10: หากฟอร์มรับอินพุตยังกรอกข้อมูลไม่ถูกต้องหรือไม่ครบเงื่อนไข เราจะใช้เทคนิคใดในการปิดการทำงานของปุ่มตอบรับ `Pressable` เพื่อความปลอดภัย?

ก. ตั้งค่าพร็อพส์ `disabled={!isFormValid}` ร่วมกับการปรับสไตล์สีปุ่มให้จางลง (เช่น ใช้ Opacity ต่ำ) เมื่อข้อมูลไม่ถูกต้อง

ข. ตั้งค่าพร็อพส์ `editable={false}` เพื่อลบสุ่มปุ่มออกจากหน้าต่างจอหลัก

ค. เขียนฟังก์ชัน `Alert.alert()` ซ้อนในสไตล์เพื่อให้ผู้ใช้ลบตัวหนังสือทิ้ง

ง. บังคับเปลี่ยนคีย์บอร์ดของมือถือผู้ใช้ให้กลายเป็นหน้าปุ่มตัวเลขทั้งหมด

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: State ไม่ใช่ตัวควบคุมหรืออ่านสไตล์ภายนอกของระบบปฏิบัติการ

• ข. ถูก: State คือสภาพหน่วยความจำชั่วคราวที่เป็นส่วนตัวภายในคอมโพเนนต์ เมื่อเรียกตัวจัดการสเตตมาอัปเดตข้อมูล React จะจัดทำการ Re-render หน้าอินเทอร์เฟซเพื่อให้ผู้เรียนหรือผู้ใช้งานพบภาพสัดส่วนข้อมูลปัจจุบันของคอมโพเนนต์นั้นได้ทันที

• ค. และ ง. ผิด: Props เป็นคนละเรื่องกับสเตตภายใน และ State ไม่เกี่ยวข้องกับการจัดการพุชโนติฟาย

เฉลย ข้อ 2: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: `useEffect` ใช้จัดการผลกระทบข้างเคียง (Side Effects) เช่น การดึงค่า API หรือผูกเซนเซอร์เครื่อง

• ข. ผิด: `useRef` ใช้เก็บตัวแปรที่ใช้อ้างอิงโดยตรงหรืออ็อบเจกต์ที่เมื่อเปลี่ยนค่าแล้วจะไม่กระตุ้นการเรนเดอร์ใหม่

• ค. ถูก: `useState` เป็น Hook ดั้งเดิมที่ React เตรียมไว้ใช้สร้างและเข้าถึงตัวจัดการสเตตใน Functional Components

• ง. ผิด: `useMemo` ใช้แคชค่าการคำนวณที่ซับซ้อนเพื่อเพิ่มประสิทธิภาพ

เฉลย ข้อ 3: ตอบ ค.

• เหตุผลประกอบ:

• **ก. ผิด:** ตัวเก็บค่าคือ `isOnline` ไม่ใช่ตัวแปรฟังก์ชัน `setIsOnline`

• **ข. ผิด:** ฟังก์ชันใช้เปลี่ยนค่าสเตต ไม่ใช่คีย์ลบโครงสร้างตัวแปร

• **ค. ถูก:** ตัวแปรตัวที่สองที่ได้จากการทำ Array Destructuring ของ `useState` จะทำหน้าที่เป็นตัว Setter คอยรับเปลี่ยนสเตต และแจ้งสั่งการให้ตัวเอนจิ้นของ React เข้าสู่วงรอบการแสดงผลหน้าจอใหม่

• **ง. ผิด:** ไม่ได้เป็นคำสั่งออโต้สร้างจุดไอคอน แต่ผู้พัฒนาสามารถนำค่าของสเตต `isOnline` ไปคุมเงื่อนไขสีวงกลมได้เอง

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** สลับบทบาทกันอย่างสิ้นเชิง

• **ข. ถูก:** State กำหนดในตนเองและเปลี่ยนข้อมูลได้เองเรื่อย ๆ ส่วน Props เป็นคุณสมบัติที่คนนอก (Component แม่) ยื่นส่งเข้ามาให้ โดยลูกที่รับไปห้ามแก้ไขสลับค่านั้นเองเพื่อไม่ให้กระแสข้อมูลขัดกัน

• **ค. และ ง. ผิด:** มีการเปลี่ยนแปลงค่าได้ปกติยามรันไทม์ และทำงานได้ทุกประเภทข้อมูลอย่างไม่มีข้อจำกัดด้านชนิด

เฉลย ข้อ 5: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** แป้นพิมพ์ไม่ควรเป็นผู้ตัดสินใจเก็บข้อความหลักเดี่ยว ๆ เพราะแอปจะไม่ทราบค่าปัจจุบันแบบทันที

• **ข. ถูก:** Controlled Component หมายถึงการออกแบบให้อินพุตผูกข้อมูลข้อความปัจจุบันไว้บนสเตตของ React โดยตรง ทุกจังหวะการเปลี่ยนอักขระจะเข้าไปอัปเดตสเตต และตัวสเตตนั้นเองจะสะท้อนความจริงกลับมาป้อนบนหน้าต่างออินพุต

• **ค. และ ง. ผิด:** ไม่ใช่การลบหรือควบคุมแป้น และไม่ใช่เรื่องการดีไซน์เฉดสีอย่างเดียว

เฉลย ข้อ 6: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** พร็อพส์อื่น ๆ ช่วยอำนวยสไตล์และความปลอดภัยเบื้องต้น แต่ไม่เพียงพอต่อการควบคุมสถานะข้อมูล

• **ข. ถูก:** เพื่อให้คอมโพเนนต์รับค่าอยู่ภายใต้การควบคุม (Controlled) พร็อพส์ `value` จะเป็นตัวชี้นำแสดงค่าสตริงของสเตต และพร็อพส์ `onChangeText` จะเป็นตัวส่งสัญญาณเมื่อผู้ใช้ป้อนอักขระล่าสุดเพื่อเปลี่ยนแปลงค่าสเตตตามลำดับ

เฉลย ข้อ 7: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: การทำสำเนาสเตตแยกอิสระจะส่งผลให้หน้าจอทำงานไม่สอดคล้องกัน (เช่น ปุ่มเปลี่ยนธีมแล้วแต่การ์ดยังสีเดิม)

• ข. ถูก: ตามหลัก Lifting State Up การยก State ไปฝากไว้ที่บรรพบุรุษร่วม (Parent Component) จะช่วยให้เราแชร์สเตตชิ้นนั้นส่งผ่านลงมาในรูปแบบ Props ให้ลูกทั้งสองคนได้พร้อม ๆ กันอย่างเป็นระบบและสม่ำเสมอ

• ค. และ ง. ผิด: การใช้ CSS ตรงไม่แก้ไขการเปลี่ยนข้อมูล และการโหลดเน็ตเวิร์กทุกรอบพิมพ์จะทำให้เครื่องช้าและสิ้นเปลือง

เฉลย ข้อ 8: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: React ไม่อนุญาตให้ดึงใช้งานอ้างอิงข้ามฟังก์ชันสเตตของตัวแม่ตรง ๆ ด้วยความสามารถทั่วไป

• ข. ถูก: เราจำเป็นต้องกำหนดพร็อพส์ให้รับฟังก์ชัน Callback จากคอมโพเนนต์แม่ เมื่อเกิดเหตุสัมผัสปุ่มหรืออินพุต คอมโพเนนต์ลูกจะกระตุ้นฟังก์ชันนั้นย้อนกลับไป และสั่งเปลี่ยนค่าสเตตต้นฉบับส่งผลให้แม่และลูกเรนเดอร์อัปเดตกันถ้วนหน้า

• ค. และ ง. ผิด: ไฟล์สไตล์ชีทไม่มีอิมแพคควบคุมข้อมูลสเตต และสามารถส่งข้อมูลขึ้นด้านบนได้ผ่านรูปแบบการใช้ Callback Function

เฉลย ข้อ 9: ตอบ ค.

• เหตุผลประกอบ:

• ก. และ ข. ผิด: ไม่เกี่ยวกับสปายแวร์หลังบ้าน หรือการดึงจังหวะสัมผัสให้ตอบสนองช้าลง

• ค. ถูก: การทำ Form Validation ฝั่งมือถือปลายทางช่วยเตือนคำแนะนำแก่ผู้ใช้งานแอปอย่างฉับพลันเมื่อป้อนข้อความผิดพลาด และป้องกันการส่งขยะ (ข้อมูลไม่ครบหรือผิดโครงสร้าง) ไปเซิร์ฟเวอร์

• ง. ผิด: ระบบส่งรูปขึ้นคลาวด์จะดีขึ้นตามขนาดไฟล์และแบนด์วิดท์ความเร็วเน็ต ไม่เกี่ยวกับกฎข้อบังคับของฟอร์มอินพุต

เฉลย ข้อ 10: ตอบ ก.

• เหตุผลประกอบ:

• ก. ถูก: คอมโพเนนต์ `Pressable` และคอมโพเนนต์ปุ่มอื่น ๆ มีพร็อพส์ `disabled` ซึ่งจะระงับเหตุการณ์ทริกเกอร์ `onPress` เมื่อค่าเป็น `true` ผู้พัฒนาสามารถนำการประเมินสเตตฟอร์มมาผูกร่วมและเขียนสไตล์ปรับเปลี่ยนสปอตไลท์ความสว่างปุ่มได้ชัดเจน

• **ข. ผิด:** `editable` เป็นคุณสมบัติสำหรับกล่อง `TextInput` ไม่ใช่ปุ่มกด

• **ค. และ ง. ผิด:** `Alert.alert` ไม่ระงับการสัมผัสปุ่ม และประเภทคีย์บอร์ดตัวเลขใช้ควบคุมอินพุตเท่านั้น

ปฏิบัติการท้ายบทเรียน: การพัฒนาแบบฟอร์มประเมินความพึงพอใจพร้อมระบบแสดงผลเรียลไทม์

ปฏิบัติการนี้จะฝึกฝนทักษะการประยุกต์ใช้สถานะ (State) การเชื่อมโยงคอมโพเนนต์ลูกด้วยหลักการ Lifting State Up และการเขียนระบบฟอร์มลงทะเบียนประเมินคุณภาพสินค้าพร้อมตรวจเช็คเงื่อนไขความถูกต้อง (Form Validation) โดยจำลองหน้าจอ **ระบบประเมินความพึงพอใจของลูกค้า (Customer Feedback & Review Form)**

1. วัตถุประสงค์การเรียนรู้

1. สามารถใช้งาน `useState` Hook ในการควบคุมการทำงานของแบบฟอร์มที่มีหลายช่องกรอกข้อมูล (Multi-input Form)

2. สามารถเขียนฟังก์ชันตรวจสอบความถูกต้องของข้อมูลแบบเรียลไทม์ (Live Form Validation) และดักจับความยาวข้อความได้

3. สามารถส่งผ่านสเตตอัปเดตย้อนกลับจากชิ้นส่วนคอมโพเนนต์ย่อยแบบปุ่มตัวเลือกผ่านกลไก Lifting State Up

4. สามารถเชื่อมโยงข้อมูลสเตตไปแสดงผลบนการ์ดพรีวิวจำลอง (Live Card Preview) ได้อย่างสม่ำเสมอ

5. สามารถจำกัดการโต้ตอบของปุ่มสัมผัสด้วยพร็อพส์ `disabled` และปรับสไตล์อย่างสอดคล้อง

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องเขียนโค้ดและโครงสร้างแอปประเมินความพึงพอใจลงในโฟลเดอร์ของตนเอง โดยมีองค์ประกอบการทำงานดังนี้:

รายละเอียดเงื่อนไขและเลย์เอาต์แอป:

1. **ส่วนบน: การ์ดตัวอย่างแสดงผลการรีวิว (Live Review Card Preview)**

• แสดงข้อความประเภทชื่อผู้รีวิว คะแนนดาวจำลอง (1-5 ดาว) และข้อเสนอแนะความพึงพอใจ

• ข้อมูลบนการ์ดใบนี้จะเปลี่ยนแปลงสดทันทีเมื่อผู้เรียนพิมพ์ข้อความ หรือเลือกคะแนนดาวในกลุ่มแบบฟอร์ม

2. **ส่วนกลาง: แบบฟอร์มประเมิน (Feedback Form Container)**

• **กล่องกรอกชื่อผู้รีวิว:** ตัวหนังสือความยาวขั้นต่ำ 2 ตัวอักษร หากไม่ถึงเกณฑ์ให้แสดงกรอบสีส้มแดงเตือน และปิดระบบเดาภาษาอังกฤษอัตโนมัติ

• **ส่วนเลือกคะแนนประเมิน (Star Rating Selector):**

• เป็นคอมโพเนนต์ลูกย่อยชื่อ `components/RatingSelector.tsx`

• มีปุ่มคะแนน 1 ถึง 5 เรียงกันแนวนอน

• ปุ่มที่ถูกเลือกจะมีสีพื้นหลังสว่างเข้มกว่าปุ่มอื่น และเมื่อกดเลือกคะแนนใด ๆ จะต้องใช้หลักการ Lifting State Up เพื่อส่งสัญญาณตัวเลขคะแนนกลับไปบันทึกที่ State หลักในคอมโพเนนต์แม่

• **กล่องคำแนะนำ (Comment Textarea):** รับคำบรรยายความพึงพอใจขั้นต่ำ 10 ตัวอักษร หากไม่ถึงเกณฑ์ให้แสดงข้อความคำแนะนำสีแดงด้านล่าง

3. **ส่วนล่าง: ปุ่มกดส่งข้อมูลประเมิน (Submit & Reset Layout)**

• **ปุ่มกดส่ง (Submit):** ล็อกไม่ให้คลิกปุ่มสัมผัสทำงานได้หากกรอกข้อมูลชื่อไม่ครบ 2 อักษร ข้อคิดเห็นไม่ถึง 10 อักษร หรือยังไม่ได้เลือกกดคะแนนประเมิน

• **ปุ่มล้างค่า (Reset):** กดสัมผัสเพื่อคืนสถานะสเตตฟอร์มทั้งหมดกลับเป็นค่าเริ่มต้นเดิม

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: สร้างชิ้นส่วนเลือกระดับคะแนนด้วยเทคนิค Lifting State Up

สร้างไฟล์คอมโพเนนต์ใหม่ชื่อ `components/RatingSelector.tsx` เพื่อเป็นชุดปุ่มเลือกคะแนน 1-5 โดยให้ลูกส่งค่าตัวเลขที่เลือกลับคืนไปให้แม่ผ่าน Props ฟังก์ชัน:

```
import { View, Text, Pressable, StyleSheet } from 'react-native';

// ข้อกำหนด Props: รับระดับคะแนนปัจจุบัน และฟังก์ชัน Callback ส่งคะแนนใหม่กลับขึ้นไป
interface RatingSelectorProps {
  selectedRating: number;
  onSelectRating: (rating: number) => void;
}

export default function RatingSelector({ selectedRating, onSelectRating }:
RatingSelectorProps) {
  const ratings = [1, 2, 3, 4, 5];

  return (
```

```
    <View style={styles.container}>
      <Text style={styles.label}>เลือกคะแนนความพึงพอใจ (1 - 5 ดาว)</Text>
      <View style={styles.starRow}>
        {ratings.map((num) => {
          const isSelected = num === selectedRating;
          return (
            <Pressable
              key={num}
              onPress={() => onSelectRating(num)} // ทริกเกอร์ฟังก์ชันส่งค่ากลับขึ้นด้านบน
              style={[
                styles.starButton,
                isSelected ? styles.starSelected : styles.starUnselected
      ]}
            >
              <Text style={[styles.starText, isSelected ?
styles.starTextActive : null]}>
        ☆ {num}
              </Text>
            </Pressable>
    );
        })}
      </View>
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    marginVertical: 12,
  },
  label: {
    fontSize: 14,
    color: '#cbd5e1',
    fontWeight: '500',
    marginBottom: 8,
  },
  starRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    gap: 8,
  },
  starButton: {
    flex: 1,
    height: 44,
    borderRadius: 8,
    justifyContent: 'center',
    alignItems: 'center',
    borderWidth: 1,
  },
  starSelected: {
```

```
    backgroundColor: '#f59e0b',
    borderColor: '#d97706',
  },
  starUnselected: {
    backgroundColor: '#0f172a',
    borderColor: '#334155',
  },
  starText: {
    color: '#94a3b8',
    fontWeight: 'bold',
    fontSize: 14,
  },
  starTextActive: {
    color: '#ffffff',
  },
});
```

ขั้นตอนที่ 2: ผูกข้อมูล สเตต และการแสดงตัวอย่างการ์ด ในหน้าจอหลัก app/index.tsx

เขียนแบบฟอร์มพร้อมเงื่อนไขการตรวจสอบคำเตือน UI ที่เปลี่ยนไปตามสเตตแบบเรียลไทม์:

```
import { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  Pressable,
  SafeAreaView,
  ScrollView,
  Alert
} from 'react-native';
import RatingSelector from '../components/RatingSelector'; // โหลดคอมโพเนนต์เลือกดาว

export default function App() {
 // สเตตเก็บข้อมูลช่องป้อนแบบฟอร์ม
  const [name, setName] = useState<string>('');
  const [rating, setRating] = useState<number>(0);
  const [comment, setComment] = useState<string>('');

 // สเตตเก็บแจ้งเตือนคำเตือน
  const [nameTouched, setNameTouched] = useState<boolean>(false);
  const [commentTouched, setCommentTouched] = useState<boolean>(false);

 // คำนวณความถูกต้องแบบเรียลไทม์
```

```
  const isNameInvalid = nameTouched && name.trim().length < 2;
  const isCommentInvalid = commentTouched && comment.trim().length < 10;

 // ตรวจสอบว่าปุ่มส่งข้อมูลควรถูกเปิดความต้องการใช้งานหรือไม่
  const isFormValid = name.trim().length >= 2 && comment.trim().length >=
10 && rating > 0;

  const handleSubmit = () => {
    if (isFormValid) {
      Alert.alert(
        'ขอบคุณสำหรับการประเมิน',
        `ชื่อผู้ประเมิน: ${name}\nคะแนนความพอใจ: ${rating}/5 ดาว\nความคิดเห็น: ${comment}`
   );
    }
  };

  const handleReset = () => {
    setName('');
    setRating(0);
    setComment('');
    setNameTouched(false);
    setCommentTouched(false);
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>
        <Text style={styles.headerTitle}>ส่งแบบประเมินบริการ</Text>

        {/* 1. Live Card Preview Area */}
        <View style={styles.previewCard}>
          <Text style={styles.previewTitle}>แสดงผลตัวอย่างคำวิจารณ์ (Live
Preview)</Text>
          <View style={styles.previewBody}>
            <Text style={styles.previewNameText}>
        👤 {name.trim() !== '' ? name : 'ยังไม่ได้ระบุชื่อผู้ประเมิน'}
            </Text>
            <Text style={styles.previewStarsText}>
              {rating > 0 ? '★'.repeat(rating) + '☆'.repeat(5 - rating) :
'☆☆☆☆☆ (กรุณาเลือกคะแนน)'}
            </Text>
            <Text style={styles.previewCommentText}>
        📝 {comment.trim() !== '' ? comment : 'พิมพ์ความคิดเห็นของคุณที่ฟอร์มด้านล่าง...'}
            </Text>
          </View>
        </View>
```

```
        {/* 2. Feedback Form Input Control */}
        <View style={styles.formCard}>

          {/* ช่องป้อนชื่อผู้รีวิว */}
          <View style={styles.inputGroup}>
            <Text style={styles.label}>ชื่อและนามสกุลลูกค้า</Text>
            <TextInput
              style={[styles.input, isNameInvalid ? styles.inputInvalid :
null]}
              placeholder="กรอกชื่อของคุณอย่างน้อย 2 ตัวอักษร"
              placeholderTextColor="#64748b"
              value={name}
              onChangeText={(text) => {
                setName(text);
                setNameTouched(true);
              }}
              autoCapitalize="none"
              autoCorrect={false}
      />
            {isNameInvalid && (
              <Text style={styles.errorText}>ข้อบกพร่อง: ชื่อต้องการอักขระอย่างน้อย 2 ตัวขึ้นไป
</Text>
      )}
          </View>

          {/* เรียกใช้งาน Custom Selector ผ่าน Props ด้วยกลไก Lifting State Up */}
          <RatingSelector selectedRating={rating}
onSelectRating={setRating} />

        {/* ช่องกรอกข้อคิดเห็นประเมินผล */}
          <View style={styles.inputGroup}>
            <Text style={styles.label}>รายละเอียดความพึงพอใจการใช้บริการ</Text>
            <TextInput
              style={[styles.input, styles.textArea, isCommentInvalid ?
styles.inputInvalid : null]}
              placeholder="บรรยายสิ่งที่คุณอยากแนะนำอย่างน้อย 10 ตัวอักษร..."
              placeholderTextColor="#64748b"
              value={comment}
              onChangeText={(text) => {
                setComment(text);
                setCommentTouched(true);
              }}
              multiline={true}
              numberOfLines={4}
      />
            {isCommentInvalid && (
              <Text style={styles.errorText}>ข้อบกพร่อง: กรุณากรอกรายละเอียดอย่างน้อย 10
ตัวอักษร</Text>
      )}
```

```
          </View>

          {/* ปุ่มจัดรายการ Submit และ Reset */}
          <View style={styles.buttonRow}>
            <Pressable
              onPress={handleReset}
              style={({ pressed }) => [
              styles.resetButton,
                pressed ? styles.resetButtonPressed : null
      ]}
            >
              <Text style={styles.resetButtonText}>ล้างฟอร์ม</Text>
            </Pressable>

            <Pressable
              disabled={!isFormValid}
              onPress={handleSubmit}
              style={({ pressed }) => [
                styles.submitButton,
       !isFormValid ? styles.submitButtonDisabled : null,
                pressed && isFormValid ? styles.submitButtonPressed : null
      ]}
            >
              <Text style={styles.submitButtonText}>ส่งข้อมูลรีวิว</Text>
            </Pressable>
          </View>

        </View>
      </ScrollView>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  // รายละเอียด CSS Style จะรวมอยู่ในโครงสร้างเฉลยในส่วนถัดไป
});
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถเปิดดูซอร์สโค้ดที่ถูกต้องรวมการสไตล์ชีทครบถ้วน โดยคัดลอกไฟล์ต่อไปนี้ไปทดสอบการทำงานบนตัวเครื่อง:

4.1 ซอร์สโค้ดไฟล์ปุ่มคะแนน: components/RatingSelector.tsx

```
import { View, Text, Pressable, StyleSheet } from 'react-native';

interface RatingSelectorProps {
  selectedRating: number;
```

```
  onSelectRating: (rating: number) => void;
}

export default function RatingSelector({ selectedRating, onSelectRating }:
RatingSelectorProps) {
  const ratings = [1, 2, 3, 4, 5];

  return (
    <View style={styles.container}>
      <Text style={styles.label}>เลือกคะแนนความพึงพอใจ (1 - 5 ดาว)</Text>
      <View style={styles.starRow}>
        {ratings.map((num) => {
          const isSelected = num === selectedRating;
          return (
            <Pressable
              key={num}
              onPress={() => onSelectRating(num)}
              style={[
                styles.starButton,
                isSelected ? styles.starSelected : styles.starUnselected
      ]}
            >
              <Text style={[styles.starText, isSelected ?
styles.starTextActive : null]}>
        ☆ {num}
              </Text>
            </Pressable>
    );
        })}
      </View>
    </View>
);
}

const styles = StyleSheet.create({
  container: {
    marginVertical: 12,
  },
  label: {
    fontSize: 14,
    color: '#cbd5e1',
    fontWeight: '500',
    marginBottom: 8,
  },
  starRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    gap: 8,
  },
  starButton: {
```

```
    flex: 1,
    height: 44,
    borderRadius: 8,
    justifyContent: 'center',
    alignItems: 'center',
    borderWidth: 1,
  },
  starSelected: {
    backgroundColor: '#f59e0b',
    borderColor: '#d97706',
  },
  starUnselected: {
    backgroundColor: '#0f172a',
    borderColor: '#334155',
  },
  starText: {
    color: '#94a3b8',
    fontWeight: 'bold',
    fontSize: 14,
  },
  starTextActive: {
    color: '#ffffff',
  },
});
```

4.2 ซอร์สโค้ดหน้าหลักฟอร์มโต้ตอบ: app/index.tsx

```
import { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  Pressable,
  SafeAreaView,
  ScrollView,
  Alert
} from 'react-native';
import RatingSelector from '../components/RatingSelector';

export default function App() {
  const [name, setName] = useState<string>('');
  const [rating, setRating] = useState<number>(0);
  const [comment, setComment] = useState<string>('');

  const [nameTouched, setNameTouched] = useState<boolean>(false);
  const [commentTouched, setCommentTouched] = useState<boolean>(false);

  const isNameInvalid = nameTouched && name.trim().length < 2;
```

```
  const isCommentInvalid = commentTouched && comment.trim().length < 10;

  const isFormValid = name.trim().length >= 2 && comment.trim().length >= 10
&& rating > 0;

  const handleSubmit = () => {
    if (isFormValid) {
      Alert.alert(
        'ส่งข้อมูลรีวิวสำเร็จ',
        `ขอบคุณสำหรับข้อแนะนำที่มีค่า!\n\nผู้รีวิว: ${name}\nคะแนน: ${rating}/5 ดาว\nเนื้อหา:
${comment}`
      );
      handleReset();
    }
  };

  const handleReset = () => {
    setName('');
    setRating(0);
    setComment('');
    setNameTouched(false);
    setCommentTouched(false);
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>
        <Text style={styles.headerTitle}>แบบประเมินการให้บริการ</Text>

        {/* 1. Live Preview Card */}
        <View style={styles.previewCard}>
          <Text style={styles.previewTitle}>ตัวอย่างรีวิวสด (Live Preview)</Text>
          <View style={styles.previewBody}>
            <Text style={styles.previewNameText}>
              {name.trim() !== '' ? name : 'ยังไม่ได้ป้อนชื่อของคุณ'}
            </Text>
            <Text style={styles.previewStarsText}>
              {rating > 0
                ? '★'.repeat(rating) + '☆'.repeat(5 - rating)
                : '☆☆☆☆☆ (กรุณาเลือกคะแนนให้คะแนนดาว)'}
            </Text>
            <Text style={styles.previewCommentText}>
              {comment.trim() !== '' ? comment : 'รายละเอียดความคิดเห็นของคุณจะปรากฏที่นี่...'}
            </Text>
          </View>
        </View>

        {/* 2. Interaction Inputs Form */}
```

```
        <View style={styles.formCard}>

          <View style={styles.inputGroup}>
            <Text style={styles.label}>ชื่อ-นามสกุลผู้รีวิว</Text>
            <TextInput
              style={[styles.input, isNameInvalid ? styles.inputInvalid :
null]}
              placeholder="กรอกชื่อ-สกุลจริงของคุณ"
              placeholderTextColor="#64748b"
              value={name}
              onChangeText={(text) => {
                setName(text);
                setNameTouched(true);
              }}
              autoCapitalize="none"
              autoCorrect={false}
      />
            {isNameInvalid && (
              <Text style={styles.errorText}>* กรุณากรอกชื่อจริงอย่างน้อย 2 ตัวอักษร
</Text>
      )}
          </View>

          <RatingSelector selectedRating={rating}
onSelectRating={setRating} />

          <View style={styles.inputGroup}>
            <Text style={styles.label}>ข้อแนะนำและระดับข้อเสนอแนะ</Text>
            <TextInput
              style={[styles.input, styles.textArea, isCommentInvalid ?
styles.inputInvalid : null]}
              placeholder="บอกเหตุผลหรือสิ่งที่ระบบควรปรับปรุง (อย่างน้อย 10 ตัวอักษร)..."
              placeholderTextColor="#64748b"
              value={comment}
              onChangeText={(text) => {
                setComment(text);
                setCommentTouched(true);
          }}
              multiline={true}
              numberOfLines={4}
      />
            {isCommentInvalid && (
              <Text style={styles.errorText}>* ข้อความยาวไม่เพียงพอ (ต้องการขั้นต่ำ 10
อักษร)</Text>
      )}
          </View>

          <View style={styles.buttonRow}>
            <Pressable
              onPress={handleReset}
```

```
              style={({ pressed }) => [
                styles.resetButton,
                pressed ? styles.resetButtonPressed : null
            ]}
              >
                <Text style={styles.resetButtonText}>ล้างแบบฟอร์ม</Text>
              </Pressable>

              <Pressable
                disabled={!isFormValid}
                onPress={handleSubmit}
                style={({ pressed }) => [
               styles.submitButton,
          !isFormValid ? styles.submitButtonDisabled : null,
                pressed && isFormValid ? styles.submitButtonPressed : null
            ]}
              >
                <Text style={styles.submitButtonText}>บันทึกความคิดเห็น</Text>
              </Pressable>
            </View>

          </View>
        </ScrollView>
      </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  scrollContainer: {
    padding: 20,
    paddingBottom: 40,
  },
  headerTitle: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
    textAlign: 'center',
    marginBottom: 16,
  },
  previewCard: {
    backgroundColor: '#1e293b',
    borderColor: '#334155',
    borderWidth: 1,
    borderRadius: 16,
    padding: 16,
```

```
    marginBottom: 20,
  },
  previewTitle: {
    fontSize: 12,
    fontWeight: 'bold',
    color: '#38bdf8',
    textTransform: 'uppercase',
    marginBottom: 10,
    letterSpacing: 0.5,
  },
  previewBody: {
    backgroundColor: '#0f172a',
    padding: 14,
    borderRadius: 8,
    gap: 8,
  },
  previewNameText: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
  previewStarsText: {
    color: '#fbbf24',
    fontSize: 18,
  letterSpacing: 2,
  },
  previewCommentText: {
    color: '#cbd5e1',
    fontSize: 14,
    fontStyle: 'italic',
    lineHeight: 20,
  },
  formCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 20,
    borderColor: '#334155',
    borderWidth: 1,
  },
  inputGroup: {
    marginBottom: 12,
  },
  label: {
    fontSize: 14,
    color: '#cbd5e1',
    fontWeight: '500',
    marginBottom: 6,
```

```
  },
  input: {
    backgroundColor: '#0f172a',
    borderWidth: 1,
    borderColor: '#334155',
    borderRadius: 8,
    height: 46,
    paddingHorizontal: 12,
    color: '#ffffff',
    fontSize: 14,
  },
  inputInvalid: {
    borderColor: '#f43f5e',
  },
  textArea: {
    height: 90,
    paddingTop: 10,
    textAlignVertical: 'top',
  },
  errorText: {
    color: '#f43f5e',
    fontSize: 12,
    marginTop: 4,
  },
  buttonRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginTop: 16,
    gap: 12,
  },
  resetButton: {
    flex: 1,
    height: 48,
    borderRadius: 8,
    borderWidth: 1,
    borderColor: '#334155',
    backgroundColor: '#1e293b',
    justifyContent: 'center',
    alignItems: 'center',
  },
  resetButtonPressed: {
    backgroundColor: '#334155',
  },
  resetButtonText: {
    color: '#cbd5e1',
    fontSize: 15,
    fontWeight: 'bold',
```

```
  },
  submitButton: {
    flex: 2,
    height: 48,
    borderRadius: 8,
    backgroundColor: '#3b82f6',
    justifyContent: 'center',
    alignItems: 'center',
  },
  submitButtonDisabled: {
    backgroundColor: '#1e293b',
    borderColor: '#334155',
    borderWidth: 1,
    opacity: 0.4,
  },
  submitButtonPressed: {
    backgroundColor: '#2563eb',
  },
  submitButtonText: {
    color: '#ffffff',
    fontSize: 15,
    fontWeight: 'bold',
  },
});
```

เอกสารอ้างอิง

React. (2024). *State: A Component's Memory*. จาก https://react.dev/learn/state-a-components-memory

React. (2024). *Synchronizing with Effects*. จาก https://react.dev/learn/synchronizing-with-effects

React Native. (2024). *Handling Text Input*. จาก https://reactnative.dev/docs/handling-text-input

แผนบริหารการสอนประจำบทที่ 5

หัวข้อเนื้อหาประจำบท

5.1 ปัญหาประสิทธิภาพของ ScrollView กับข้อมูลขนาดใหญ่

5.2 การแก้ไขด้วย FlatList: Virtualization และ Recycling

5.3 โครงสร้างและพร็อพส์หลักของ FlatList (data, renderItem, keyExtractor)

5.4 การดึงข้อมูลจาก REST API มาใช้ร่วมกับ FlatList

5.5 การปรับแต่งส่วนประกอบลิสต์ (Header, Footer, Item Separator, และ ListEmptyComponent)

5.6 ฟีเจอร์ Pull-to-Refresh และ Infinite Scroll (onEndReached)

5.7 การจัดกลุ่มข้อมูลด้วย SectionList

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถชี้แจงความแตกต่างเชิงประสิทธิภาพและกรณีใช้งานที่เหมาะสมระหว่าง ScrollView และ FlatList ได้ถูกต้อง
2. ผู้เรียนสามารถระบุและเขียนส่งตัวแปรพร็อพส์หลักของ FlatList เพื่อวาดรายการข้อมูลได้สำเร็จโดยไม่มีปัญหาเรื่องกุญแจจำแนกชิ้นงาน (key)
3. ผู้เรียนสามารถเขียนคำสั่ง Fetch API ร่วมกับ useEffect เพื่อเรียกข้อมูลจากแหล่งข้อมูลออนไลน์ (เช่น JSONPlaceholder) มาแสดงผลบน FlatList ได้สำเร็จ
4. ผู้เรียนสามารถประยุกต์ใช้คอมโพเนนต์เสริมตกแต่งส่วนประกอบขอบลิสต์ (Header, Footer, Divider) และการจัดทำกรณีลิสต์ว่างได้อย่างมีหลักการ
5. ผู้เรียนสามารถเขียนจำลองสเตตและฟังก์ชันดึงข้อมูลใหม่เพื่อทำ Pull-to-Refresh แสดงไอคอนหมุนโหลดข้อมูลสำเร็จ
6. ผู้เรียนสามารถเรียกใช้และอธิบายหลักการทำงานของ onEndReached ในการโหลดข้อมูลหน้าถัดไป (Pagination) ได้ถูกต้อง
7. ผู้เรียนสามารถสร้างแบบจำลองรายการแบบกลุ่มโครงสร้างผ่าน SectionList ได้อย่างเหมาะสม

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

17. **การบรรยายประกอบการเปรียบเทียบเชิงลึก (Theoretical Performance Analysis):** บรรยายการทำงานเบื้องหลังของ React Native VirtualizedList และจัดทำการสาธิตเปรียบเทียบ RAM Usage ระหว่าง ScrollView และ FlatList เมื่อโหลดอาร์เรย์ 2,000 รายการ
18. **การทำ Live Coding Demonstration หัวข้อ REST API FlatList:** ผู้สอนสาธิตการเชื่อมโยงเครือข่ายดึงข้อมูลโพสต์จำลองหรือรูปภาพจำลองจาก JSONPlaceholder REST API ผ่านคำสั่ง fetch นำเข้าแสดงผลใน FlatList
19. **กิจกรรมปฏิบัติเดี่ยวฝึกความจำ (Advanced Layout Implementation):** มอบหมายโจทย์ให้ผู้เรียนปรับปรุงรายการแอปที่มีอยู่เดิม โดยเพิ่ม Header แสดงโลโก้ และสร้าง Empty Component หน้าตาสวยงามที่จะโชว์เมื่อข้อมูลถูกล้างจนหมดแถว
20. **กิจกรรมประยุกต์ทำกลุ่มประเมินผลการจัดกลุ่ม (SectionList Grouping Activity):** แบ่งกลุ่มศึกษาโค้ดโครงสร้างเมนูอาหารที่จำแนกหมวดหมู่ (เช่น ของคาว ของหวาน เครื่องดื่ม) โดยเรียกนำเสนอผลลัพธ์ผ่านคอมโพเนนต์ SectionList

สื่อการเรียนการสอน

16. สไลด์การเรียนการสอนวิชา "Lists Rendering and Core Optimizations in React Native"
17. ชุดข้อมูลอาร์เรย์จำลองของรายชื่อภาพยนต์หรือรายชื่อผู้ติดต่อจำนวน 500 รายการที่มีโครงสร้าง ID และข้อมูลสเตตัส
18. ตัวอย่างการตรวจสอบ FPS (Frame Per Second) ด้วย Performance Tools ของเครื่องจำลองสมาร์ทโฟน
19. ซอร์สโค้ดต้นแบบระบบสินค้าคงคลัง (Inventory Manager Demo) ที่ใช้ FlatList และ SectionList

การวัดผลและการประเมินผล

1. การวัดผล

1.1 วัดความเข้าใจผ่านแบบทดสอบปรนัยท้ายบทเรียน 10 ข้อ

1.2 ตรวจสอบทักษะการเขียนโปรแกรมและการเลือกใช้งาน FlatList แทน ScrollView ให้เหมาะกับลักษณะขนาดงานข้อมูล

1.3 ตรวจประเมินโค้ดแล็บในการใส่ฟังก์ชันรีเฟรช การจัดแบ่งแยกประเภทคีย์ และความสามารถในการโหลดข้อมูลหน้าถัดไป (Infinite Scroll)

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถเขียนส่งชิ้นงานแล็บแสดงผลรายการข่าวสารดึงข้อความรีเฟรชได้ลื่นไหลและรันผ่านโดยไม่มีคำแจ้งเตือนบั๊กเรื่อง Key ได้สำเร็จตามเกณฑ์ (ผ่าน/ไม่ผ่าน)

2.3 พฤติกรรมการรับฟังข้อเสนอแนะ ความพยายามแก้ไขอิมพลิเมนต์ปุ่มดาวน์โหลดข้อมูลเพิ่มเติมด้วยสเตตจนผ่านเกณฑ์

บทที่ 5

การแสดงผลรายการข้อมูลและประสิทธิภาพการทำงาน

ความเร็วและความลื่นไหลในการเลื่อนดูข้อมูลยาว ๆ (Scrolling Performance) ถือเป็นปัจจัยหลักที่ส่งผลต่อคะแนนความพึงพอใจของผู้ใช้งานโมบายแอปพลิเคชัน ในบทเรียนนี้เราจะศึกษาข้อจำกัดของคอมโพเนนต์ `ScrollView` และเจาะลึกวิธีการนำเสนอรายการข้อมูลประสิทธิภาพสูงด้วย `FlatList` และ `SectionList` รวมถึงแนวทางการเชื่อมต่อข้อมูลสดผ่านเน็ตเวิร์ก

5.1 ปัญหาประสิทธิภาพของ ScrollView กับข้อมูลขนาดใหญ่

คอมโพเนนต์ `ScrollView` มีลักษณะการเรนเดอร์ข้อมูลแบบ **กระตือรือร้น (Eager Rendering)** หมายความว่า ทันทีที่คอมโพเนนต์ถูกเปิดขึ้นมารันบนหน้าจอ มันจะประมวลผลวาดโครงสร้างของคอมโพเนนต์ลูกย่อยทุกชิ้นภายในลิสต์ลงสู่หน่วยความจำ (RAM) พร้อมกันในคราวเดียว:

- **จุดเด่น:** ใช้งานง่าย รันคอมโพเนนต์ลูกที่แตกต่างรูปแบบกันได้อย่างอิสระ
- **จุดด้อยที่รุนแรง:** หากข้อมูลมีจำนวนมาก (เช่น เกิน 100 รายการขึ้นไป หรือมีรูปภาพความละเอียดสูงประกอบ) ระบบปฏิบัติการจะต้องสร้าง Native Views คาไว้ใน RAM มหาศาล ส่งผลให้แอปพลิเคชันเกิดการสะดุด เฟรมเรตตก (Frame Drops) และแอปเด้งดับเนื่องจากหน่วยความจำเต็ม (Out of Memory)

ดังนั้น `ScrollView` จึงควรเลือกใช้เฉพาะหน้ารายละเอียดทั่วไป หรือแบบฟอร์มที่มีจำนวนวัตถุคงที่สั้น ๆ เท่านั้น

5.2 การแก้ไขด้วย FlatList: Virtualization และ Recycling

เพื่อจัดการปัญหาข้อมูลล้นจอ React Native ได้จัดเตรียมคอมโพเนนต์ **`FlatList`** (React Native, 2024) (ซึ่งสืบทอดกลไกมาจาก `VirtualizedList`) โดยใช้เทคนิคสถาปัตยกรรมระดับเนทีฟในการจัดระเบียบพื้นที่หน่วยความจำดังนี้:

1. ระบบจำลองเสมือนจริง (Virtualization)

`FlatList` จะวาดเรนเดอร์เฉพาะคอมโพเนนต์รายการที่ **ปรากฏตัวอยู่ในขอบเขตหน้าจอจริงเท่านั้น** (บวกกับพื้นที่ข้างเคียงล่วงหน้าเล็กน้อย เรียกว่า Window Buffer) ข้อมูลที่อยู่นอกจอจะยังไม่ถูกนำมาประมวลผล

2. การหมุนเวียนมุมมองกลับมาใช้ใหม่ (Item View Recycling)

เมื่อผู้ใช้งานใช้นิ้วปัดหน้าจอเพื่อเลื่อนลง กล่อง View รายการที่เคลื่อนล้นหลุดขอบจอด้านบนออกไป จะถูกส่งกลับเข้าไปในคลังสะสมและนำโครงสร้างกล่องนั้นมา **"รีไซเคิลปรับเปลี่ยนข้อมูลข้างในใหม่"** เพื่อนำมารอแสดงผลเป็นรายการใหม่ที่วิ่งเข้ามาจากขอบจอด้านล่าง ส่งผลให้หน่วยความจำถูกจำกัดอยู่คงที่ตามขนาดจอเสมอ ไม่ว่าจะโหลดข้อมูล 100 หรือ 100,000 ชิ้นก็ตาม

5.3 โครงสร้างและพร็อพส์หลักของ FlatList (data, renderItem, keyExtractor)

การทำงานร่วมกับ `FlatList` จำเป็นต้องส่งค่าพร็อพส์ที่ระบุหน้าที่เฉพาะเจาะจง 3 ตัวหลักเสมอ:

• `data`: อาร์เรย์ของข้อมูลดิบที่ต้องการแสดงผล (เช่น `Array<any>`)

• `renderItem`: ฟังก์ชันคืนค่า JSX คอมโพเนนต์ที่อธิบายหน้าตาของแต่ละรายการย่อย โดยรับพารามิเตอร์ออบเจกต์ที่มีพร็อพส์ `{ item, index }`

• `keyExtractor`: ฟังก์ชันส่งสตริงคีย์ที่เป็นเอกลักษณ์ (Unique Key) ของรายการย่อยแต่ละชิ้น เพื่อช่วยให้ระบบ Diffing Engine ของ React ติดตามและระบุการเปลี่ยนแปลงตำแหน่งชิ้นงานได้ถูกต้องโดยไม่ต้องวาดใหม่ทั้งหมด

```
import { FlatList, Text, View, StyleSheet } from 'react-native';

const mockUsers = [
  { id: '1', name: 'สมชาย' },
  { id: '2', name: 'สมศรี' },
  { id: '3', name: 'สมหวัง' },
];

export default function UserList() {
  return (
    <FlatList
      data={mockUsers}
      keyExtractor={(item) => item.id} // ช่วยส่งคืนสตริงระบุคีย์ที่ไม่ซ้ำกัน
      renderItem={({ item }) => (
        <View style={styles.item}>
          <Text style={styles.itemText}>{item.name}</Text>
        </View>
    )}
  />
 );
}

const styles = StyleSheet.create({
  item: {
    padding: 16,
    borderBottomWidth: 1,
    borderBottomColor: '#ccc',
```

```
  },
  itemText: {
    fontSize: 16,
  },
});
```

5.4 การดึงข้อมูลจาก REST API มาใช้ร่วมกับ FlatList

ในระบบงานจริง ข้อมูลรายการมักจะดึงผ่านเน็ตเวิร์กจากเซิร์ฟเวอร์ภายนอก เราสามารถประยุกต์ใช้คำสั่ง `fetch()` ร่วมกับสถานะ `useState` และ `useEffect` ดึงข้อมูลมาอัปโหลดใส่ใน `FlatList` ดังตัวอย่างนี้:

```
import { useState, useEffect } from 'react';
import {
  FlatList,
  Text,
  View,
  ActivityIndicator,
  StyleSheet,
  SafeAreaView
} from 'react-native';

interface Post {
  id: number;
  title: string;
  body: string;
}

export default function ApiListScreen() {
  const [posts, setPosts] = useState<Post[]>([]);
  const [loading, setLoading] = useState<boolean>(true);

 // 1. ฟังก์ชันดึงข้อมูลจาก REST API ของ JSONPlaceholder (JSONPlaceholder, 2024)
  const fetchPosts = async () => {
    try {
     setLoading(true);
       const response = await
fetch('https://jsonplaceholder.typicode.com/posts?_limit=15');
       const data = await response.json();
       setPosts(data);
     } catch (error) {
       console.error('ดึงข้อมูลผิดพลาด:', error);
     } finally {
       setLoading(false);
     }
  };

  useEffect(() => {
    fetchPosts();
```

```
  }, []);

  // 2. แสดงตัวหมุนโหลดข้อมูลชั่วคราวหากเน็ตเวิร์กกำลังดึงข้อมูลอยู่
  if (loading) {
    return (
      <View style={styles.centerContainer}>
        <ActivityIndicator size="large" color="#38bdf8" />
        <Text style={styles.loadingText}>กำลังดึงข้อมูลโพสต์ล่าสุด...</Text>
      </View>
  );
  }

  return (
    <SafeAreaView style={styles.container}>
      <FlatList
        data={posts}
        keyExtractor={(item) => item.id.toString()}
        renderItem={({ item }) => (
          <View style={styles.postCard}>
            <Text style={styles.postTitle}>📌 {item.title}</Text>
            <Text style={styles.postBody}>{item.body}</Text>
          </View>
    )}
    />
    </SafeAreaView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  centerContainer: {
    flex: 1,
    backgroundColor: '#0f172a',
    justifyContent: 'center',
    alignItems: 'center',
  },
  loadingText: {
    color: '#94a3b8',
    marginTop: 10,
  },
  postCard: {
    backgroundColor: '#1e293b',
    padding: 16,
    marginVertical: 8,
    marginHorizontal: 16,
    borderRadius: 12,
```

```
    borderWidth: 1,
    borderColor: '#334155',
  },
  postTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#ffffff',
    textTransform: 'capitalize',
    marginBottom: 6,
  },
  postBody: {
    fontSize: 14,
    color: '#cbd5e1',
    lineHeight: 20,
  },
});
```

5.5 การปรับแต่งส่วนประกอบลิสต์ (Header, Footer, Item Separator, และ ListEmptyComponent)

`FlatList` เตรียมอินเทอร์เฟซอรรถประโยชน์อื่น ๆ ที่ทำให้การออกแบบหน้าตารายการทำได้สมบูรณ์แบบโดยไม่ต้องครอบคอมโพเนนต์ด้วย ScrollView ซ้ำซ้อน:

• **`ListHeaderComponent`**: พื้นที่จัดแสดงสำหรับวางหัวตารางหรือแผงควบคุมส่วนบน (เช่น ช่องแถบค้นหา โฆษณา หรือป้ายหัวข้อใหญ่) ซึ่งจะเลื่อนสไลด์ขึ้นไปตามความเร็วของลิสต์

• **`ListFooterComponent`**: พื้นที่จัดแสดงสำหรับวางท้ายตารางรายการ (เช่น ปุ่มแสดงหน้าถัดไป หรือตัวหมุนบอกว่ากำลังดาวน์โหลดข้อมูลต่อท้าย)

• **`ItemSeparatorComponent`**: คอมโพเนนต์คั่นกลางระหว่างชิ้นรายการเพื่อระบุช่องไฟหรือวาดเส้นแบ่งเส้นคั่น (จะตัดขอบไม่วาดท้ายรายการล่างสุดให้เอง)

• **`ListEmptyComponent`**: คอมโพเนนต์แสดงผลทดแทนเมื่ออาร์เรย์ **`data`** มีขนาดเป็นศูนย์ (ไม่มีข้อมูลใด ๆ ในรายการ) เพื่อแจ้งเตือนผู้ใช้อย่างชัดเจน

5.6 ฟีเจอร์ Pull-to-Refresh และ Infinite Scroll (onEndReached)

1. ระบบ Pull-to-Refresh (ดึงหน้าจอเพื่อรีเฟรชข้อมูล)

เป็นการเปิดโอกาสให้ผู้ใช้งานกวาดนิ้วลงเพื่ออัปโหลดใหม่เพื่อเอาโพสต์ชุดใหม่ โดยควบคุมด้วย 2 พร็อพส์หลัก:

• `refreshing`: ตัวแปรสถานะ Boolean ที่บ่งบอกว่าขณะนี้ระบบกำลังดาวน์โหลดรอบใหม่ค้างอยู่หรือไม่ (เพื่อให้ไอคอนหมุนทำงาน)

• `onRefresh`: ฟังก์ชันที่จะถูกสั่งงานอัตโนมัติเมื่อตรวจพบการลากนิ้วดึงหน้าจอลำดับล่างลงมา

2. ระบบ Infinite Scroll (เลื่อนลงสุดเพื่อดาวน์โหลดต่ออัตโนมัติ)

เทคนิคยอดนิยมบนหน้า Feed ของโซเชียลมีเดียเพื่อรองรับการโหลดข้อมูลแบบแบ่งหน้า (Pagination) ควบคุมผ่าน:

• `onEndReached`: ฟังก์ชันสั่งงานเมื่อผู้ใช้งานเลื่อนหน้าจอลิสต์เข้าใกล้ความยาวล่างสุดของรายการ

• `onEndReachedThreshold`: ค่าตัวเลขสัดส่วนระยะห่างขอบล่างของหน้าต่าง ตัวเลข 0.5 บ่งบอกว่าถ้าเลื่อนมาถึงกึ่งกลางชิ้นงานรายการสุดท้าย ให้เริ่มเรียกการโหลดข้อมูลหน้าถัดไปเตรียมสแตนด์บายได้ทันที

5.7 การจัดกลุ่มข้อมูลด้วย SectionList

เมื่อข้อมูลรายการมีความจำเป็นที่จะต้องจัดแยกจำแนกหมวดหมู่ (เช่น คลังรายชื่ออักษรขึ้นต้น A-Z, การแบ่งหมวดหมู่เมนูอาหาร) คอมโพเนนต์ **SectionList** (React Native, 2024) จะช่วยอำนวยความสะดวกในการจัดสรรความลื่นไหลได้เป็นหมวดหมู่:

โครงสร้างข้อมูลสำหรับ SectionList

ข้อมูลในพร็อพส์ **sections** ต้องเป็นอาร์เรย์ของออบเจกต์ ซึ่งมีคีย์ชื่อ `title` (หรือคีย์จัดกลุ่มใด ๆ) และคีย์อาร์เรย์ชื่อ `data` บรรจุรายละเอียดลูกย่อยในกลุ่มนั้น:

```
import { SectionList, Text, View, StyleSheet, SafeAreaView } from 'react-native';

const MENU_DATA = [
  {
    title: 'ของคาว (Main Dishes)',
    data: ['ข้าวผัดปู', 'ต้มยำกุ้งน้ำข้น', 'ผัดไทยกุ้งสด'],
  },
  {
    title: 'เครื่องดื่ม (Drinks)',
    data: ['ชาไทยนมสด', 'กาแฟอเมริกาโน่เย็น'],
  },
];

export default function RestaurantMenu() {
  return (
    <SafeAreaView style={{ flex: 1 }}>
```

```
      <SectionList
        sections={MENU_DATA}
        keyExtractor={(item, index) => item + index}
    // ฟังก์ชันวาดเมนูอาหารย่อยแต่ละตัว
        renderItem={({ item }) => (
          <View style={styles.item}>
            <Text style={styles.itemText}>{item}</Text>
          </View>
    )}
    // ฟังก์ชันวาดป้ายแถบหัวข้อกลุ่ม (Sticky Header)
        renderSectionHeader={({ section: { title } }) => (
          <View style={styles.header}>
            <Text style={styles.headerText}>{title}</Text>
          </View>
    )}
   />
    </SafeAreaView>
 );
}

const styles = StyleSheet.create({
  header: {
    backgroundColor: '#3b82f6',
    padding: 10,
  },
  headerText: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  item: {
    padding: 16,
    borderBottomWidth: 1,
    borderBottomColor: '#e2e8f0',
  },
  itemText: {
    fontSize: 15,
  },
});
```

สรุปท้ายบทเรียน

ปัญหาประสิทธิภาพของ ScrollView กับข้อมูลขนาดใหญ่: ScrollView ทำงานแบบ Eager Rendering ซึ่งจะโหลดและวาดคอมโพเนนต์ย่อยทั้งหมดลงสู่หน่วยความจำ (RAM) พร้อมกันตั้งแต่เริ่มต้น จึงทำให้เมื่อต้องแสดงผลรายการข้อมูลจำนวนมหาศาลหรือมีรูปภาพประกอบจำนวน

มาก จะเกิดปัญหาหน่วยความจำล้นและทำให้อัตราการแสดงผลเฟรมเรตตกจนแอปพลิเคชันทำงานสะดุดหรือเด้งออก

การแก้ไขด้วย FlatList (Virtualization และ Recycling): เพื่อเพิ่มความลื่นไหลระดับเนทีฟ FlatList ถูกนำเข้ามาช่วยแก้ปัญหาโดยใช้แนวคิด Virtualization ที่เรนเดอร์เฉพาะข้อมูลที่ปรากฏบนขอบจอจริงเท่านั้น และใช้แนวคิด Item View Recycling ในการหมุนเวียนจัดสลับกล่องรายการที่เลื่อนหลุดขอบจอกลับมาสวมใส่ข้อมูลชุดถัดไป ช่วยประหยัดการสร้าง Native Views และประหยัดทรัพยากรระบบได้อย่างมีประสิทธิภาพ

โครงสร้างและพร็อพส์หลักของ FlatList: การประกาศใช้งาน FlatList ต้องการอ็อบชันควบคุมหลัก 3 ประการ ได้แก่ data สำหรับส่งผ่านอาร์เรย์ของข้อมูลดิบทั้งหมด, renderItem ซึ่งคอยทำหน้าที่สร้างรูปแบบ JSX ของแถบข้อมูลรายย่อย และ keyExtractor ที่ใช้กำหนดค่าสตริงคีย์เอกลักษณ์เฉพาะตัวให้แต่ละรายการเพื่อช่วยระบบ Diffing Engine ของ React ประเมินจุดเปลี่ยนได้อย่างรวดเร็ว

การดึงข้อมูลจาก REST API มาใช้ร่วมกับ FlatList: การประยุกต์ใช้งานในระบบจริงจะนำฟังก์ชัน fetch() มาประสานร่วมกับ useState และ useEffect เพื่อขอข้อมูลดิบผ่านระบบเครือข่ายอินเทอร์เน็ต โดยเราสามารถแสดงกล่อง ActivityIndicator บ่งบอกความก้าวหน้าขณะรอผลลัพธ์ ก่อนที่จะจัดเก็บข้อมูลลงในสเตตและนำข้อมูลดังกล่าวไปเป็นแหล่งอ้างอิงหลักในโครงสร้างของ FlatList

การปรับแต่งส่วนประกอบลิสต์: เพื่อป้องกันความซับซ้อนในการเขียนคอมโพเนนต์ซ้อนทับกัน FlatList ได้เตรียมเครื่องมือเฉพาะตัวมาอำนวยความสะดวก เช่น ListHeaderComponent และ ListFooterComponent สำหรับสร้างหัวและท้ายหน้าเลื่อน, ItemSeparatorComponent ในการวาดเส้นคั่นหรือจัดระเบียบช่องไฟระหว่างแถว และ ListEmptyComponent เพื่อแสดงข้อความแจ้งเตือนเมื่อไม่มีข้อมูลแสดงผล

ฟีเจอร์ Pull-to-Refresh และ Infinite Scroll: แอปพลิเคชันที่ทันสมัยสามารถยกระดับความสะดวกได้ด้วยระบบ Pull-to-Refresh ที่เปิดทางให้ผู้ใช้นิ้วปัดหน้าจอลงเพื่อรีเฟรชข้อมูลผ่านพร็อพส์ refreshing และ onRefresh รวมถึงการทำ Infinite Scroll เพื่อดาวน์โหลดหน้าถัดไปแบบต่อเนื่องผ่านพร็อพส์ onEndReached และการกำหนดสัดส่วนระยะเริ่มโหลดผ่าน onEndReachedThreshold

การจัดกลุ่มข้อมูลด้วย SectionList: สำหรับข้อมูลที่ต้องจัดแยกเป็นหมวดหมู่อย่างชัดเจน SectionList คือตัวเลือกในการแสดงผลที่ตอบโจทย์ โดยข้อมูลนำเข้าในพร็อพส์ sections จะต้องจัดเรียงกลุ่มข้อมูลในรูปแบบโครงสร้างออบเจกต์ที่มีคีย์ title สำหรับป้ายหัวข้อกลุ่ม และคีย์ data

สำหรับบรรจุรายการย่อย ซึ่งระบบจะทำการคำนวณและวาดป้ายแถบหัวข้อกลุ่ม (Sticky Header) ให้สอดคล้องกันโดยอัตโนมัติ

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความเข้าใจเกี่ยวกับปัญหาประสิทธิภาพเลื่อนไหล (Scrolling Performance) ข้อแตกต่างระหว่าง `ScrollView` และ `FlatList` โครงสร้างและพร็อพส์หลักในการควบคุมรายการ การเชื่อมต่อเน็ตเวิร์กดึงข้อมูลผ่าน API รวมถึงระบบ Pull-to-Refresh, Infinite Scroll และการจัดรูปเล่มประเภทกลุ่มด้วย `SectionList` ใน React Native Expo SDK 54

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดคือสาเหตุหลักที่ทำให้อินเทอร์เฟซเกิดอาการหน่วง เฟรมเรตตก หรือแอปพลิเคชันค้างเด้งปิดตัว (Crash) เมื่อใช้งาน `ScrollView` แสดงผลข้อมูลจำนวนมาก?

ก. `ScrollView` ล็อกไม่ให้อุปกรณ์ดึงสัญญาณอินเทอร์เน็ต

ข. `ScrollView` ทำการเรนเดอร์และสร้างวัตถุลูกทั้งหมดลงสู่หน่วยความจำ (RAM) พร้อมกันในทันที แม้ว่าชิ้นส่วนข้อมูลจะยังไม่แสดงบนหน้าจอจริงก็ตาม

ค. `ScrollView` ห้ามเขียนจัดตำแหน่งบล็อกสีด้วยระบบ Flexbox

ง. `ScrollView` ไม่รองรับการแสดงผลกล่องอินพุต `TextInput`

ข้อ 2: กลไกแบบ Virtualization ที่ใช้งานภายในคอมโพเนนต์ `FlatList` ช่วยเพิ่มระดับประสิทธิภาพและประหยัดพื้นที่ RAM ได้อย่างไร?

ก. ย้ายเนื้อหางานคำนวณทั้งหมดส่งไปให้คอมพิวเตอร์หลังบ้านประมวลผลแทน

ข. แสดงผลเฉพาะชิ้นข้อมูลรายการที่กำลังอยู่ในขอบข่ายขนาดหน้าจอจริง และทำลาย/รียูสกล่องจัดแสดงของรายการที่หลุดหน้าจอออกไป

ค. ซ่อนข้อมูลทั้งหมดไว้ในเครื่องมือ AsyncStorage ตลอดเวลา

ง. ลดขนาดความละเอียดของภาพที่โหลดให้กลายเป็นภาพขาวดำอัตโนมัติ

ข้อ 3: พร็อพส์ (Props) ใดต่อไปนี้มีหน้าที่สำคัญในการเป็นตัวแปลงข้อมูลวัตถุอาร์เรย์ให้ออกมาเป็น JSX คอมโพเนนต์การ์ดแสดงผลบนหน้าจอลิสต์?

ก. `data`

ข. `renderItem`

ค. `keyExtractor`

ง. `ItemSeparatorComponent`

ข้อ 4: ในฟังก์ชันการดึงค่าของ `renderItem` ข้อใดระบุรูปแบบพารามิเตอร์แบบกระจายออบเจกต์ (Destructuring) ที่ถูกต้องสำหรับการสกัดข้อมูลไอเท็มปัจจุบันมาแสดง?

ก. `renderItem={(item) => <Text>{item.title}</Text>}`

ข. `renderItem={({ item }) => <Text>{item.title}</Text>}`

ค. `renderItem={(title) => <Text>{title}</Text>}`

ง. `renderItem={({ data }) => <Text>{data.title}</Text>}`

ข้อ 5: เหตุใดผู้พัฒนาแอปจึงสมควรใส่ใจกำหนดพร็อพส์ `keyExtractor` ให้ส่งค่าสตริงที่เป็นเอกลักษณ์ (Unique String Key) ให้แก่ FlatList ทุกครั้ง?

ก. เพื่อบังคับให้อุปกรณ์ระบุขนาดของรูปภาพในรายการโดยอัตโนมัติ

ข. เพื่อให้ React สามารถตรวจเช็คสถานะการเปลี่ยนแปลงตำแหน่ง ย้ายสลับ หรือลบรายการได้อย่างถูกต้องแบบเจาะจงเฉพาะชิ้นงาน โดยไม่ต้องเสียเวลาวาดใหม่ทั้งหมด

ค. เพื่อจำกัดไม่ให้ปุ่มสัมผัสทำงานได้เมื่อคีย์บอร์ดปรากฏขึ้น

ง. เพื่อระบุตัวเลขจำนวนหน้าสำหรับดาวน์โหลดข้อมูลเพิ่มเติม

ข้อ 6: เมื่อต้องการดึงรายการโพสต์หรือรูปภาพจาก RESTful API ทันทีที่หน้าจอลิสต์เริ่มต้นทำงาน (Mount) ใน React Native ข้อใดคือแนวทางปฏิบัติมาตรฐาน?

ก. เรียกใช้คำสั่ง `fetch()` ในสไตล์ชีทภายนอก

ข. เขียนฟังก์ชัน `fetch()` ครอบไว้ภายใน `useEffect` Hook ที่มีอาเรย์การพึ่งพาเป็นค่าว่าง (`[]`) เพื่อให้สัญญาณอินพุตทำงานเพียงรอบเดียว

ค. เรียกฟังก์ชันเน็ตเวิร์กภายใน Property ของ `keyExtractor`

ง. ใช้ `useState` ในการดักจับค่า IP ของโทรศัพท์แบบวนซ้ำ

ข้อ 7: พร็อพส์ใดของ `FlatList` ที่ทำหน้าที่แสดงคอมโพเนนต์อื่นทดแทน (เช่น ข้อความแจ้งว่าไม่มีรายการของ) เมื่อพบว่าไม่มีข้อมูลอยู่ในอาเรย์เลย?

ก. `ListFooterComponent`

ข. `ListHeaderComponent`

ค. `ListEmptyComponent`

ง. `ItemSeparatorComponent`

ข้อ 8: การเปิดความสามารถให้ผู้ใช้ลากขอบจอลงด้านล่างสุดเพื่อสั่งดึงข้อมูลชุดใหม่ (Pull-to-Refresh) ใน FlatList จะต้องทำรายการตั้งค่าพร็อพส์คู่ใดร่วมกัน?

ก. `refreshing` และ `onRefresh`

ข. `onEndReached` และ `onEndReachedThreshold`

ค. `active` และ `onActive`

ง. `secureTextEntry` และ `onChangeText`

ข้อ 9: พร็อพส์ `onEndReached` ในคอมโพเนนต์ `FlatList` มีพฤติกรรมการทำงานหลักอย่างไร?

ก. ทำการตรวจสอบสิทธิข้อมูลเมื่อผู้ใช้นิ้วเลื่อนออกจากปุ่มล้างค่า

ข. เรียกฟังก์ชันสั่งการดาวน์โหลดข้อมูลหน้าถัดไป (Infinite Scroll) ทันทีเมื่อผู้ใช้งานใช้นิ้วเลื่อนหน้าจอลิสต์ลงไปจนเกือบถึงส่วนท้ายสุดของตารางรายการ

ค. ล้างสเตตทั้งหมดกลับเป็นค่าเริ่มต้นเมื่อลิสต์แสดงผลเสร็จ

ง. เคลื่อนย้ายโครงสร้างหัวข้อของลิสต์ให้อยู่ชิดปุ่มล่างสุดของขอบอุปกรณ์

ข้อ 10: หากต้องการจัดเลย์เอาต์แสดงผลที่แยกข้อมูลออกเป็นหมวดหมู่ (เช่น คลังเครื่องดื่ม, คลังของหวาน) โดยต้องการให้มีหัวข้อกลุ่มลอยตัวค้างไว้เวลาเลื่อน (Sticky Headers) คอมโพเนนต์ใดตอบโจทย์งานลักษณะนี้ได้ดีที่สุด?

ก. `ScrollView`

ข. `FlatList`

ค. `SectionList`

ง. `SafeAreaView`

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: `ScrollView` ไม่ได้บล็อกการดึงเครือข่ายของระบบ

• ข. ถูก: `ScrollView` ไม่มีระบบหน่วยความจำแบบไดนามิก มันจะประมวลผลแปลง JSX ทุกชิ้นข้างใน (Eager Rendering) ไม่ว่าจะล้นจอไปยาวแค่ไหน ทำให้เปลือง RAM ทวีคูณตามสัดส่วนรายการข้อมูล

• ค. และ ง. ผิด: การเขียนจัดตำแหน่งและใส่อินพุตทำได้ตามรูปแบบทั่วไป

เฉลย ข้อ 2: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: งานเรนเดอร์ UI จะทำบนอุปกรณ์ปลายทางเสมอ ไม่ใช่หลังบ้าน

• ข. ถูก: `FlatList` ยึดหลักเรนเดอร์ขอบเขตเฉพาะส่วนที่มองเห็นจริง และทำการปล่อย RAM คืนระบบเมื่อวัตถุเลื่อนหลุดสายตา พร้อมทั้งดึงเฟรม View ที่ว่างเหล่านั้นกลับมารอรับสวมข้อมูลชิ้นถัดไป (Recycling) ส่งผลให้มีความคงทนสูงและประหยัด RAM

• **ค. และ ง. ผิด:** AsyncStorage มีไว้เก็บบันทึกถาวร และระบบไม่มีการสุ่มเปลี่ยนภาพเป็นขาวดำ

เฉลย ข้อ 3: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `data` เป็นเพียงคลังอาร์เรย์ข้อเท็จจริง (ข้อมูลดิบ) ของโพสต์ แต่ยังไม่มีโครงสร้างสไตล์ภาพข้อความ

• **ข. ถูก:** พร็อพส์ `renderItem` ทำหน้าที่รับฟังก์ชันที่ผู้พัฒนาอธิบายสไตล์ไว้ เพื่อนำข้อมูลดิบในแต่ละดัชนี (Index) มาวาดแปลงให้กลายเป็นโครงสร้าง JSX เพื่อแสดงขึ้นจอจริง

• **ค. ผิด:** `keyExtractor` ทำหน้าที่กำหนดระบุข้อความกุญแจส่วนตัว ไม่ใช่ผู้วาด UI

• **ง. ผิด:** `ItemSeparatorComponent` ใช้สำหรับวาดเส้นแบ่งเส้นคั่นเท่านั้น

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** การเขียนพารามิเตอร์โดยไม่มีปีกกาอาร์กิวเมนต์ `{}` หรือสับสนชื่อคีย์ จะนำเอาอ็อบเจกต์รวมของ MetaData (ซึ่งประกอบด้วยตัวชี้นงาน, ลำดับ Index, ข้อมูลภายนอก) มาใช้ตรง ๆ ส่งผลให้เกิดอาการค่าเป็น `undefined` หรือโปรแกรมล่ม

• **ข. ถูก:** พารามิเตอร์ที่ `renderItem` ส่งมาจะมีรูปเป็นออบเจกต์ `{ item, index, separators }` การใส่ปีกกาสกัดตัวแปรคีย์ชื่อ `item` (`{ item }`) จึงเป็นวิธีที่ถูกต้องดั้งเดิมในการเอาวัตถุเดี่ยวแต่ละดัชนีมาใช้งาน

เฉลย ข้อ 5: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ขนาดของรูปภาพต้องถูกตั้งผ่าน CSS Style เท่านั้น

• **ข. ถูก:** ในสถาปัตยกรรม React การใช้คีย์เฉพาะช่วยประหยัดเวลาอย่างมาก เพราะถ้าเกิดการอัปเดตข้อมูลขึ้นมาจุดใดจุดหนึ่ง React จะดึงสตริงกุญแจตรวจสอบเพื่อประมวลผลขยับเฉพาะกล่อง View นั้นโดยไม่ต้องเรนเดอร์ใหม่หมดทั้งแผง ซึ่งเพิ่มความลื่นไหลเป็นอย่างมาก

• **ค. และ ง. ผิด:** ไม่ได้ควบคุมการทำงานปุ่ม และไม่ได้มีขอบข่ายเกี่ยวกับเลขหน้าข้อมูลดั้งเดิม

เฉลย ข้อ 6: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** สไตล์ชีทมีหน้าที่จัดสเปซเฉดสีเท่านั้น ห้ามเรียกนำข้อมูลเน็ตเวิร์ก

• **ข. ถูก:** การประยุกต์ใช้งาน `useEffect` พร้อมกำหนด Dependency Array ว่างเปล่า [] จะสั่งให้อุปกรณ์เชื่อมต่อทำงานส่งคำขอ Fetch API เพียงแค่ครั้งเดียวตอนโหลดหน้าจอ

ขึ้นมา (Component Did Mount) ซึ่งช่วยบันทึกข้อมูลสเตตและเตรียมนำไปใช้ป้อนต่อ FlatList ได้อย่างมีประสิทธิภาพ

- **ค. และ ง. ผิด:** คีย์เฉพาะและการทำสเตตไอพีไม่ได้เอื้อต่อการดึงค่า API โพสต์

เฉลย ข้อ 7: ตอบ ค.

- **เหตุผลประกอบ:**
  - **ก. และ ข. ผิด:** ใช้ตกแต่งส่วนบนและส่วนล่างสุดของลิสต์ตาราง
  - **ค. ถูก:** `ListEmptyComponent` คอยตรวจเช็คความยาวอาร์เรย์ใน `data` หากข้อมูลมีค่าว่าง จะเรนเดอร์คอมโพเนนต์อื่นที่เราส่งมอบเข้าไปแทน เช่น ภาพการ์ตูนกล่องเปล่าพร้อมคำเตือน "ไม่พบข้อมูลใด ๆ ในขณะนี้"
  - **ง. ผิด:** ใช้สำหรับคั่นระหว่างชิ้นงาน

เฉลย ข้อ 8: ตอบ ก.

- **เหตุผลประกอบ:**
  - **ก. ถูก:** เพื่ออำนวยฟังก์ชันกวาดดึงหน้าจอ (Pull-to-Refresh) เราต้องใช้ `refreshing` (สเตตคอยจำสถานะการดึงประมวลผล) ควบคู่กับเหตุการณ์ `onRefresh` (ฟังก์ชันสั่งยิงเรียกเน็ตเวิร์กอัปเดตใหม่)
  - **ข. ผิด:** เป็นคู่สำหรับระบุงานโหลดเพิ่มตอนท้ายจอ (Infinite Scroll)
  - **ค. และ ง. ผิด:** ไม่ใช่คำสั่งที่อยู่ในขอบข่ายงานดึงหมุนรีเฟรชหน้าต่าง

เฉลย ข้อ 9: ตอบ ข.

- **เหตุผลประกอบ:**
  - **ก. ค. และ ง. ผิด:** พฤติกรรมไม่ใช่การเคลียร์ปุ่ม ล้างสเตต หรือย้ายหัวข้อขยับขอบจอ
  - **ข. ถูก:** เหตุการณ์ `onEndReached` จะถูกส่งสัญญาณเมื่อผู้ใช้นิ้วเลื่อนตารางลงมาจนเข้าเกณฑ์ขอบล่าง เพื่อส่งสัญญาณทำ Callback ไปเรียกหน้าถัดไป (เช่น หน้า 2, 3) มาประกอบลงสเตตเดิมเพื่อสร้างลิสต์แบบต่อเนื่องยาวไม่มีที่สิ้นสุด

เฉลย ข้อ 10: ตอบ ค.

- **เหตุผลประกอบ:**
  - **ก. และ ข. ผิด:** ScrollView เปลือง RAM และไม่มี Sticky Headers ออโต้ ส่น FlatList ออกแบบมาสำหรับแถวเรียงเส้นตรง ไม่ได้สนับสนุนการแยกกลุ่มและป้าย Sticky ประจำเซกชันโดยตรง

• **ค. ถูก:** `SectionList` ดีไซน์ขึ้นมาแยกย่อยข้อมูลกลุ่มประเภทหัวข้อ (Sections) โดยความสามารถหลักคือมีระบบตรึงแถบหัวกลุ่มค้างไว้ (Sticky Section Headers) บนขอบจอยามเลื่อนผ่าน ซึ่งอำนวยความสวยงามสไตล์ระดับเนทีฟยอดนิยม

• **ง. ผิด:** มีหน้าที่ป้องกันเนื้อหาชนขอบบาก ไม่เกี่ยวกับการคำนวณลิสต์กลุ่ม

ปฏิบัติการท้ายบทเรียน: การพัฒนาแอปแสดงรายการรูปภาพจาก API พร้อมระบบรีเฟรชและโหลดข้อมูลเพิ่ม

ปฏิบัติการนี้จะฝึกทักษะการประยุกต์ใช้งานคอมโพเนนต์รายการประสิทธิภาพสูง `FlatList` ร่วมกับการดึงข้อมูลผ่านระบบเครือข่ายออนไลน์จริงด้วย Fetch API (ดึงข้อมูลจำลองจาก JSONPlaceholder) และอัปเดตข้อมูลแบบแบ่งหน้า (Pagination) เพื่อทำระบบโหลดอัตโนมัติเมื่อเลื่อนขอบล่างสุด (Infinite Scroll) พร้อมระบบดึงลงรีเฟรชหน้าจอ (Pull-to-Refresh)

1. วัตถุประสงค์การเรียนรู้

1. สามารถดึงข้อมูลภาพและข้อความจาก REST API ออนไลน์ภายนอกผ่าน Fetch API ร่วมกับ `useEffect` ได้สำเร็จ

2. สามารถจัดสรรหน่วยความจำและเรนเดอร์ภาพจากอินเทอร์เน็ตปริมาณมากในระบบรายการ `FlatList` ได้อย่างลื่นไหล

3. สามารถพัฒนาฟังก์ชัน Pull-to-Refresh เพื่อเคลียร์สเตตกลับคืนหน้า 1 และประมวลผลข้อมูลใหม่จากเซิร์ฟเวอร์

4. สามารถทำระบบแบ่งหน้าโหลดเนื้อหาต่อด้านท้าย (Infinite Scroll) ด้วยพร็อพส์ `onEndReached`

5. สามารถเขียนจัดวางและตกแต่งชิ้นส่วนย่อยประกอบลิสต์ (`ListHeaderComponent`, `ListFooterComponent`, `ListEmptyComponent`, และ `ItemSeparatorComponent`) ได้ถูกต้อง

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนจะต้องเขียนพัฒนาโมเดลจำลอง **แอปพลิเคชันคลังแกลเลอรีภาพออนไลน์ (Live Photo Gallery Stream)** โดยประกอบด้วยเงื่อนไข 2 ส่วนหลักดังนี้:

1. ชิ้นส่วนการ์ดภาพย่อย: `components/PhotoCard.tsx`

• สร้างเป็น Custom Component เพื่อแสดงชิ้นผลงานแต่ละชิ้นประกอบด้วย:

• รูปภาพขนาดย่อ (`thumbnailUrl`) แสดงผลทรงสี่เหลี่ยมโค้งมนสวยงาม

• ข้อความหัวเรื่องรูปภาพ (`title`) บรรยายรายละเอียด

• ตัวเลขดัชนีระบุ ID ของรูปภาพ

• จัดความปลอดภัยด้วย TypeScript Interface `PhotoCardProps`

• ตัวปุ่มการ์ดจัดด้วย `Pressable` ที่มีลูกเล่นการกระพริบสว่างหรือยืดหดน้ำหนักสัมผัสเมื่อถูกคลิก

2. หน้าหลักควบคุมแบบต่อเนื่อง: `app/index.tsx`

• ดึงข้อมูลผ่าน URL: `https://jsonplaceholder.typicode.com/photos`

• **การแบ่งหน้าโหลดข้อมูล (Pagination):** กำหนดให้โหลดข้อมูลเพียงครั้งละ 10 รูป (`_limit=10`) โดยส่งผ่านพารามิเตอร์ `_page` เพื่อระบุหน้าข้อมูล (เช่น หน้า 1 โหลดรูป 1-10, หน้า 2 โหลดรูป 11-20)

• **การดึงเพื่อรีเฟรช (Pull-to-Refresh):** เมื่อดึงหน้าจอลง จะเรียกโหลดข้อมูลหน้า 1 ใหม่ พร้อมสั่งล้างคลังสเตตอาร์เรย์เดิมทิ้ง

• **การเลื่อนลงสุดเพื่อโหลดต่อ (Infinite Scroll):** เมื่อใช้นิ้วเลื่อนลงมาจนถึงระยะกึ่งกลางของไอเท็มสุดท้าย จะทริกเกอร์เรียกโหลดหน้าถัดไป (`page + 1`) เพื่อนำรูปภาพชุดใหม่ไปประกอบต่อท้ายอาร์เรย์เดิม พร้อมเรนเดอร์ไอคอนหมุนโหลดข้อมูล (`ActivityIndicator`) ค้างไว้ที่ด้านล่างสุดของตารางรายการ

• **องค์ประกอบตกแต่ง:**

• **Header:** แสดงข้อความใหญ่ "Photo Stream" และบอกยอดรูปสะสมปัจจุบัน

• **Separator:** เส้นคั่นสีเทาบางกั้นระหว่างภาพ

• **Empty:** โชว์ข้อความ "ไม่พบข้อมูลรูปภาพในคลัง" กรณีสเตตอาร์เรย์เป็นศูนย์

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: การสร้าง Custom Component สำหรับจัดสไตล์ชิ้นการ์ดรูปภาพ

สร้างไฟล์คอมโพเนนต์สำหรับการ์ดแสดงรายการรูปภาพตามเส้นทาง `components/PhotoCard.tsx` และใส่โครงสร้างโค้ดดังนี้:

```
import { View, Text, Image, StyleSheet, Pressable } from 'react-native';

// กำหนดประเภทข้อมูล Props ด้วย TypeScript Interface
export interface PhotoCardProps {
  id: number;
  title: string;
  thumbnailUrl: string;
  onPressCard?: () => void;
}

export default function PhotoCard({ id, title, thumbnailUrl, onPressCard
}: PhotoCardProps) {
```

```
  return (
    <Pressable
      onPress={onPressCard}
      style={({ pressed }) => [
        styles.card,
        { backgroundColor: pressed ? '#1e293b' : '#0f172a' } // เปลี่ยนสีพื้นหลังเล็กน้อยเมื่อแตะ
      ]}
    >
      {/* แสดงรูปภาพจาก URI เครือข่าย */}
      <Image source={{ uri: thumbnailUrl }} style={styles.thumbnail} />

      <View style={styles.textContainer}>
        <Text style={styles.idText}>PHOTO ID: #{id}</Text>
        <Text style={styles.titleText} numberOfLines={2}>
          {title}
        </Text>
      </View>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  card: {
    flexDirection: 'row',
    padding: 12,
    alignItems: 'center',
    borderRadius: 8,
  },
  thumbnail: {
    width: 60,
    height: 60,
    borderRadius: 8,
    backgroundColor: '#334155', // พื้นหลังสีเทาขณะรอดึงรูปจริง
  },
  textContainer: {
    flex: 1,
    marginLeft: 16,
  },
  idText: {
    fontSize: 11,
    color: '#38bdf8',
    fontWeight: 'bold',
    marginBottom: 2,
  },
  titleText: {
    fontSize: 14,
    fontWeight: '500',
```

```
    color: '#cbd5e1',
    textTransform: 'capitalize',
  },
});
```

ขั้นตอนที่ 2: วางหน้าจอเขียนฟังก์ชัน Fetch API แบบแบ่งหน้าในไฟล์หลัก app/index.tsx

เขียนประมวลสเตตจัดการหน้าข้อมูลสัญญานเน็ตเวิร์ก พร้อมประกอบคำสั่งลงในหน้าหลัก:

```
import { useState, useEffect } from 'react';
import {
  StyleSheet,
  Text,
  View,
  FlatList,
  ActivityIndicator,
  SafeAreaView,
  Alert
} from 'react-native';
import PhotoCard from '../components/PhotoCard'; // โหลดการ์ดภาพย่อย

interface Photo {
  id: number;
  title: string;
  thumbnailUrl: string;
  url: string;
}

export default function PhotoStreamScreen() {
 // สเตตเก็บคลังอาร์เรย์รูปภาพ
  const [photos, setPhotos] = useState<Photo[]>([]);

 // สเตตจัดการคีย์เลขหน้าและการจำสถานะหมุนโหลด
  const [page, setPage] = useState<number>(1);
  const [loadingInitial, setLoadingInitial] = useState<boolean>(true); //
โหลดครั้งแรกสุด
  const [loadingMore, setLoadingMore] = useState<boolean>(false);        //
โหลดเพิ่มส่วนท้าย
  const [refreshing, setRefreshing] = useState<boolean>(false);          //
ดึงหน้าจอลงรีเฟรช
  const [hasMoreData, setHasMoreData] = useState<boolean>(true);         //
เช็คว่าหมดคลังหรือยัง

  const LIMIT = 10; // ดึงข้อมูลครั้งละ 10 รายการ

 // 1. ฟังก์ชันดึงข้อมูลผ่านเครือข่าย
  const fetchPhotosData = async (targetPage: number, isRefreshing: boolean
= false) => {
```

```
    try {
      if (isRefreshing) setRefreshing(true);
      else if (targetPage > 1) setLoadingMore(true);

      const url =
`https://jsonplaceholder.typicode.com/photos?_page=${targetPage}&_limit=${L
IMIT}`;
      const response = await fetch(url);
      const data: Photo[] = await response.json();

      if (data.length < LIMIT) {
        setHasMoreData(false); // หากข้อมูลส่งมาไม่ครบตามจำนวนจำกัด แปลว่าสิ้นสุดคลังแล้ว
      }

      if (isRefreshing) {
        setPhotos(data); // ล้างสเตตเดิมและใส่ข้อมูลหน้า 1 ชุดใหม่ลงไปตรงๆ
        setHasMoreData(true);
      } else {
        setPhotos((prev) => [...prev, ...data]); // นำไปต่อยอดพรีเพนท์ข้อมูลเดิม
      }
    } catch (error) {
      console.error('การดึงข้อมูลเครือข่ายผิดพลาด:', error);
      Alert.alert('เกิดข้อผิดพลาด', 'ไม่สามารถเชื่อมต่อสัญญาณอินเทอร์เน็ตได้ในขณะนี้');
    } finally {
      setLoadingInitial(false);
      setLoadingMore(false);
      setRefreshing(false);
    }
  };

  // ดึงข้อมูลหน้า 1 ตอนรันหน้าแอปเริ่มต้น
  useEffect(() => {
    fetchPhotosData(1);
  }, []);

  // 2. ฟังก์ชันสั่งลากนิ้วรีเฟรชด้านบน
  const handleRefresh = () => {
    setPage(1);
    fetchPhotosData(1, true);
  };

  // 3. ฟังก์ชันโหลดเพิ่มต่อท้ายเมื่อเลื่อนลงสุดหน้าจอ
  const handleLoadMore = () => {
    if (!loadingMore && hasMoreData) {
      const nextPage = page + 1;
      setPage(nextPage);
      fetchPhotosData(nextPage);
    }
  };
```

```
  // ... รายละเอียดสปอตไลท์ Render จะแยกอยู่ในส่วนเฉลยท้ายสุด
  return null;
}
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถคัดลอกรายละเอียดโค้ดโปรเจคฉบับสมบูรณ์ที่จัดวางสไตล์ชีทแบบสเปซโมเดิร์นสีเข้ม (Dark Mode) ไปรันผลลัพธ์ผ่านตัวจำลองเพื่อทดสอบระบบดึงภาพได้ทันที:

4.1 ซอร์สโค้ดไฟล์ย่อย: components/PhotoCard.tsx

```
import { View, Text, Image, StyleSheet, Pressable } from 'react-native';

export interface PhotoCardProps {
  id: number;
  title: string;
  thumbnailUrl: string;
  onPressCard?: () => void;
}

export default function PhotoCard({ id, title, thumbnailUrl, onPressCard
}: PhotoCardProps) {
  return (
    <Pressable
      onPress={onPressCard}
      style={({ pressed }) => [
        styles.card,
        { backgroundColor: pressed ? '#1e293b' : '#1e293b' }
   ]}
    >
      <Image source={{ uri: thumbnailUrl }} style={styles.thumbnail} />

      <View style={styles.textContainer}>
        <Text style={styles.idText}>ID: #{id}</Text>
        <Text style={styles.titleText} numberOfLines={2}>
          {title}
        </Text>
      </View>
    </Pressable>
 );
}

const styles = StyleSheet.create({
  card: {
    flexDirection: 'row',
    padding: 14,
    alignItems: 'center',
```

```
    borderRadius: 12,
    borderWidth: 1,
    borderColor: '#334155',
  },
  thumbnail: {
    width: 64,
    height: 64,
    borderRadius: 8,
    backgroundColor: '#334155',
  },
  textContainer: {
    flex: 1,
    marginLeft: 16,
  },
  idText: {
    fontSize: 10,
    color: '#38bdf8',
    fontWeight: 'bold',
    marginBottom: 2,
    letterSpacing: 0.5,
  },
  titleText: {
    fontSize: 14,
    fontWeight: '500',
    color: '#cbd5e1',
    textTransform: 'capitalize',
    lineHeight: 18,
  },
});
```

4.2 ซอร์สโค้ดหน้าจอหลักเชื่อม API: **app/index.tsx**

```
import { useState, useEffect } from 'react';
import {
  StyleSheet,
  Text,
  View,
  FlatList,
  ActivityIndicator,
  SafeAreaView,
  Alert
} from 'react-native';
import PhotoCard from '../components/PhotoCard';

interface Photo {
  id: number;
 title: string;
  thumbnailUrl: string;
```

```
  url: string;
}

export default function App() {
  const [photos, setPhotos] = useState<Photo[]>([]);
  const [page, setPage] = useState<number>(1);
  const [loadingInitial, setLoadingInitial] = useState<boolean>(true);
  const [loadingMore, setLoadingMore] = useState<boolean>(false);
  const [refreshing, setRefreshing] = useState<boolean>(false);
  const [hasMoreData, setHasMoreData] = useState<boolean>(true);

  const LIMIT = 10;

  const fetchPhotosData = async (targetPage: number, isRefreshing: boolean
= false) => {
    try {
      if (isRefreshing) setRefreshing(true);
      else if (targetPage > 1) setLoadingMore(true);

      const url =
`https://jsonplaceholder.typicode.com/photos?_page=${targetPage}&_limit=${
LIMIT}`;
      const response = await fetch(url);
      const data: Photo[] = await response.json();

      if (data.length < LIMIT) {
        setHasMoreData(false);
      }

      if (isRefreshing) {
        setPhotos(data);
        setHasMoreData(true);
      } else {
        setPhotos((prev) => [...prev, ...data]);
      }
    } catch (error) {
      console.error('การดึงข้อมูลเครือข่ายผิดพลาด:', error);
      Alert.alert('เกิดข้อผิดพลาด', 'ไม่สามารถเชื่อมต่อสัญญาณอินเทอร์เน็ตได้ในขณะนี้');
    } finally {
      setLoadingInitial(false);
      setLoadingMore(false);
      setRefreshing(false);
    }
  };

  useEffect(() => {
    fetchPhotosData(1);
  }, []);

  const handleRefresh = () => {
```

```
    setPage(1);
    fetchPhotosData(1, true);
  };

  const handleLoadMore = () => {
    if (!loadingMore && hasMoreData && !loadingInitial) {
      const nextPage = page + 1;
      setPage(nextPage);
      fetchPhotosData(nextPage);
    }
  };

// A. ส่วนตกแต่งแถบหัวของรายการ
  const renderHeader = () => (
   <View style={styles.header}>
      <Text style={styles.headerTitle}>Photo Gallery Feed</Text>
      <Text style={styles.headerSubtitle}>
    จำนวนภาพแสดงในสเตริม: {photos.length} ภาพ
      </Text>
    </View>
);

// B. ตัวแสดงแถบหมุนประมวลผลเพิ่มใต้ตารางลิสต์
  const renderFooter = () => {
    if (!loadingMore) return null;
    return (
      <View style={styles.footerLoader}>
        <ActivityIndicator size="small" color="#38bdf8" />
        <Text style={styles.footerLoaderText}>กำลังโหลดรูปภาพเพิ่มเติม...</Text>
      </View>
  );
  };

// C. ป้ายเตือนในกรณีลิสต์มีค่าว่าง
  const renderEmpty = () => (
    <View style={styles.emptyContainer}>
      <Text style={styles.emptyText}>ไม่พบข้อมูลรูปภาพในคลังจำลอง</Text>
    </View>
);

// D. เส้นประคั่นกลางแบ่งรายการ
  const renderSeparator = () => <View style={styles.separator} />;

  if (loadingInitial) {
    return (
      <View style={styles.centerContainer}>
        <ActivityIndicator size="large" color="#38bdf8" />
        <Text style={styles.loadingText}>กำลังเปิดแกลเลอรีภาพออนไลน์...</Text>
```

```
      </View>
  );
  }

  return (
    <SafeAreaView style={styles.safeArea}>
      <View style={styles.container}>
        <FlatList
          data={photos}
          keyExtractor={(item) => item.id.toString()}
          renderItem={({ item }) => (
            <PhotoCard
              id={item.id}
              title={item.title}
              thumbnailUrl={item.thumbnailUrl}
              onPressCard={() => Alert.alert(`ภาพรหัส #${item.id}`,
item.title)}
      />
     )}
     // ผูกสไตล์องค์ประกอบตกแต่งย่อย
          ListHeaderComponent={renderHeader}
          ListFooterComponent={renderFooter}
          ListEmptyComponent={renderEmpty}
          ItemSeparatorComponent={renderSeparator}

     // ผูกความสามารถรีเฟรชด้วยนิ้วลาก
          refreshing={refreshing}
          onRefresh={handleRefresh}

     // ผูกความสามารถ Infinite Scroll
          onEndReached={handleLoadMore}
          onEndReachedThreshold={0.5} // เลื่อนมาถึงขอบล่าง 50% ของรายการสุดท้ายให้ดึงหน้าใหม่
รอทันที

          contentContainerStyle={styles.listContent}
    />
      </View>
    </SafeAreaView>
 );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
   backgroundColor: '#0f172a',
  },
  container: {
    flex: 1,
  },
```

```
  centerContainer: {
    flex: 1,
    backgroundColor: '#0f172a',
    justifyContent: 'center',
    alignItems: 'center',
  },
  loadingText: {
    color: '#94a3b8',
    marginTop: 12,
   fontSize: 14,
  },
  listContent: {
    paddingHorizontal: 16,
    paddingBottom: 24,
  },
  header: {
    paddingVertical: 20,
    borderBottomWidth: 1,
    borderBottomColor: '#1e293b',
    marginBottom: 16,
  },
  headerTitle: {
    fontSize: 24,
   fontWeight: 'bold',
    color: '#ffffff',
  },
  headerSubtitle: {
    fontSize: 12,
    color: '#94a3b8',
    marginTop: 4,
  },
  separator: {
    height: 12, // ทำหน้าที่เป็นระยะช่องไฟคั่นระหว่างไอเท็มแต่ละตัว
  },
  footerLoader: {
    paddingVertical: 20,
    flexDirection: 'row',
    justifyContent: 'center',
    alignItems: 'center',
    gap: 8,
  },
  footerLoaderText: {
    color: '#94a3b8',
    fontSize: 12,
  },
  emptyContainer: {
    padding: 40,
    alignItems: 'center',
```

```
    justifyContent: 'center',
  },
  emptyText: {
    color: '#64748b',
    fontSize: 16,
  },
});
```

เอกสารอ้างอิง

JSONPlaceholder. (2024). *JSONPlaceholder - Free Fake REST API*. จาก https://jsonplaceholder.typicode.com/

React Native. (2024). *FlatList API Reference*. จาก https://reactnative.dev/docs/flatlist

React Native. (2024). *SectionList API Reference*. จาก https://reactnative.dev/docs/sectionlist

แผนบริหารการสอนประจำบทที่ 6

หัวข้อเนื้อหาประจำบท

6.1 แนวคิดการจัดเส้นทางแบบอิงโครงสร้างไฟล์ (File-based Routing)

6.2 การสร้าง Layout และการจัดระเบียบโครงสร้างด้วย _layout.tsx

6.3 ระบบนำทางแบบสะสมประวัติ (Stack Navigation)

6.4 ระบบนำทางแบบแท็บบาร์ล่าง (Tab Navigation)

6.5 การเชื่อมโยงลิงก์และเปลี่ยนหน้าจอ (Link & Router Object)

6.6 เส้นทางนำทางแบบไดนามิกและการอ่านค่าพารามิเตอร์ (Dynamic Routes & useLocalSearchParams)

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถอธิบายหลักการทำงานและประโยชน์ของ File-based Routing เปรียบเทียบกับระบบจัดเส้นทางแบบดั้งเดิมได้ถูกต้อง
2. ผู้เรียนสามารถสร้างไฟล์โครงสร้าง _layout.tsx เพื่อแบ่งโครงสร้างหน้าจอแบบ Stack และ Tabs ได้สำเร็จ
3. ผู้เรียนสามารถเขียนเปลี่ยนเส้นทางหน้าจอโดยประยุกต์ใช้ <Link> และฟังก์ชันคำสั่งในกลุ่ม router.push() / router.replace() / router.back() ได้ถูกต้องตามสถานการณ์
4. ผู้เรียนสามารถดึงตัวแปรและคีย์ไดนามิกพารามิเตอร์ผ่าน useLocalSearchParams มาแสดงผลข้อมูลเจาะจงรายชิ้นสำเร็จ
5. ผู้เรียนสามารถเขียนสไตล์ส่วนหัวบาร์หน้าจอ (Screen Headers Option) เพื่อแก้ไขชื่อหัวข้อหรือปุ่มนำทางซ่อน/โชว์ได้สำเร็จ

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

21. **การบรรยายอธิบายสถาปัตยกรรม (Routing Architecture Lecture):** บรรยายการจัดวางโครงสร้างของโฟลเดอร์ app/ ในระบบ Expo Router สาธิตการส่งสัญญาณลิงก์และบทบาทของ Layout ของหน้าจอ

22. **การทำ Live Coding Demonstration หัวข้อ Multi-page Business Card App:** ผู้สอนสาธิตสดการออกแบบแอป 2 หน้าหลัก (หน้าแคตตาล็อกสินค้า และหน้ารายละเอียดสินค้าเฉพาะชิ้น) โดยเน้นเขียนส่ง ID สินค้าพ่วงไปกับเส้นทางลิงก์ และดึงแสดงรูปสไตล์ไดนามิก
23. **กิจกรรมปฏิบัติกลุ่มย่อยฝึกแก้เส้นทางเลย์เอาต์ (Tabs-to-Stack Refactoring):** ให้กลุ่มผู้เรียนทดลองเพิ่มแท็บเมนูใหม่ "Settings" ในเลย์เอาต์ Tabs และเขียนปุ่มในหน้าแท็บนั้นเพื่อนำทางผู้ใช้เจาะลึกลงไปยังหน้า Stack ย่อย "Edit Profile"
24. **กิจกรรมเขียนระบบล็อกอินนำทางจำลอง (Auth Redirect Flow Practice):** ฝึกเขียนและดักจับเหตุการณ์หากผู้ใช้ยังไม่ได้กรอกรหัส ให้เปลี่ยนหน้าจอด้วย router.replace() เข้าสู่หน้า Login แทนการย้อนกลับได้

สื่อการเรียนการสอน

20. เอกสารสไลด์เรื่อง "File-based Routing and Advanced Navigation with Expo Router"
21. ตัวอย่างโครงการตั้งต้นที่มีหน้าจอเตรียมไว้แล้วสำหรับใช้ฝึกร้อยเรียงระบบนำทาง (Navigation Sandbox) บน Git
22. ลิงก์เอกสารคู่มืออย่างเป็นทางการของ Expo Router v6 (Expo SDK 54)
23. อุปกรณ์สมาร์ทโฟนตัวทดสอบเพื่อสังเกตความต่างของ Transition หน้าจอระหว่าง iOS และ Android

การวัดผลและการประเมินผล

1. การวัดผล

1.1 ประเมินระดับทฤษฎีจากการทำคะแนนแบบทดสอบปรนัยท้ายบทเรียน 10 ข้อ

1.2 ตรวจสอบโครงสร้างการจัดวางแผนผังไฟล์และตำแหน่งเลย์เอาต์ _layout.tsx ภายใต้โฟลเดอร์ app/

1.3 ตรวจประเมินทักษะการดึงพารามิเตอร์ข้ามหน้าด้วย useLocalSearchParams และการทำงานของแถบเมนูนำทางในแล็บปฏิบัติการ

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถสร้างหน้าจอแอปพลิเคชันที่มีหน้า Tabs ควบคู่หน้า Stack และส่งผ่านข้อมูลข้ามหน้าได้โดยไม่มีการแจ้งเตือนเส้นทางขาด (Broken Link) สำเร็จผ่านเกณฑ์ (ผ่าน/ไม่ผ่าน)

2.3 พฤติกรรมการทำงานร่วมกับผู้อื่น ความมีส่วนร่วมการช่วยนำทางและประคองกลุ่มปฏิบัติการแก้โจทย์

บทที่ 6

ระบบนำทางและสถาปัตยกรรมการเปลี่ยนหน้าจอด้วย Expo Router

ในอดีต การจัดการระบบนำทาง (Navigation) ใน React Native ต้องอาศัยไลบรารี React Navigation ซึ่งมีความซับซ้อนในการตั้งค่าแผนผังหน้าจอ แต่สำหรับ **Expo Router** (เปิดตัวแบบสมบูรณ์ใน Expo SDK 54 / Router v6) ได้นำเสนอแนวคิดใหม่ที่เรียกว่า **File-based Routing** ซึ่งถอดแบบมาจากสถาปัตยกรรมระดับอุตสาหกรรมอย่าง Next.js บนฝั่งเว็บ ช่วยให้นักพัฒนาสร้างและบริหารเส้นทางเดินหน้าจอผ่านโครงสร้างโฟลเดอร์ได้อย่างสะดวก รวดเร็ว และลื่นไหล

6.1 แนวคิดการจัดเส้นทางแบบอิงโครงสร้างไฟล์ (File-based Routing)

File-based Routing คือการแปลงตำแหน่งเส้นทางของไฟล์ (File Paths) ในโฟลเดอร์พิเศษที่ตั้งชื่อว่า **`app/`** ให้กลายมาเป็นเส้นทางการนำทาง (Route Paths) ของแอปพลิเคชันโดยอัตโนมัติ (Expo, 2024):

```
โครงสร้างในโฟลเดอร์ app/                URL / เส้นทางเข้าถึง
app/
├── index.tsx                          =>  "/" (หน้าแรกสุดของแอป)
├── profile.tsx                        =>  "/profile" (หน้าโปรไฟล์)
└── settings/
    └── index.tsx                      =>  "/settings" (หน้าตั้งค่า)
```

เมื่อคุณเพิ่มไฟล์ใหม่เข้าไปในโฟลเดอร์ **app/** ตัวระบบรวบรวมของ Expo (Metro Bundler) จะดักตรวจและเปิดใช้แผนที่นำทางนั้น ๆ ขึ้นมาโดยที่คุณไม่จำเป็นต้องมาเขียนโค้ดเพิ่มหน้าลงในอาเรย์การลงทะเบียนแต่อย่างใด

6.2 การสร้าง Layout และการจัดระเบียบโครงสร้างด้วย _layout.tsx

ไฟล์ที่ชื่อว่า **`_layout.tsx`** ถือเป็นชิ้นส่วนควบคุมพิเศษใน Expo Router (Expo, 2024) มีบทบาทสำคัญในการ:

1. กำหนดรูปแบบเลย์เอาต์หลักของแอป เช่น หน้าจอเรียงซ้อน (Stack) หรือ หน้าจอแบบแท็บบาร์ (Tabs)
2. ห่อหุ้มและแชร์ส่วนหน้าตาร่วมกัน (Shared UI) เช่น แถบสถานะ หรือแถบนำทางหลัก

3. จัดการพฤติกรรมความปลอดภัยและการปกป้องการเข้าถึงหน้าจอ (เช่น ระบบ Auth Guards)

ถ้าเราบันทึกไฟล์ `_layout.tsx` ไว้ที่ระดับบนสุดของโฟลเดอร์ `app/` สไตล์ของเลย์เอาต์นั้นจะคลุมทุกไฟล์ย่อยที่อยู่ภายใต้โฟลเดอร์นั้นทั้งหมด

6.3 ระบบนำทางแบบสะสมประวัติ (Stack Navigation)

เลย์เอาต์แบบ `Stack` คือระบบจัดการนำหน้าจอที่จำลองการวางซ้อนข้อมูลแผ่นการ์ดจากล่างขึ้นบน (Push/Pop Screen):

- เมื่อเปิดหน้าใหม่ หน้าจอใหม่จะถูกดันซ้อนทับอยู่ด้านบนสุดของหน้าเก่า (Push)
- เมื่อกดปุ่มย้อนกลับ (Back) หน้าบนสุดจะถูกดึงทิ้งออกไปเพื่อกลับมาโชว์หน้าจอด้านล่าง (Pop)

1. วิธีประกาศเลย์เอาต์ Stack ใน `_layout.tsx`

เราเรียกใช้คอมโพเนนต์ `<Stack>` เพื่อสร้างโครงสร้างการนำทางแบบซ้อนทับ:

```
// app/_layout.tsx
import { Stack } from 'expo-router';

export default function Layout() {
  return (
    <Stack
      screenOptions={{
        headerStyle: { backgroundColor: '#0f172a' }, // แต่งสีแถบหัวข้อเข้ม
        headerTintColor: '#ffffff', // สีตัวหนังสือปุ่มย้อนกลับ
        headerTitleStyle: { fontWeight: 'bold' },
      }}
    >
      {/* ระบุตั้งค่าสไตล์เฉพาะหน้าจอผ่าน Stack.Screen */}
      <Stack.Screen name="index" options={{ title: 'หน้าหลักเดสก์ท็อป' }} />
      <Stack.Screen name="profile" options={{ title: 'โปรไฟล์ของคุณ' }} />
    </Stack>
  );
}
```

6.4 ระบบนำทางแบบแท็บบาร์ล่าง (Tab Navigation)

เลย์เอาต์แบบ `Tabs` จัดสรรความสามารถยอดนิยมในการควบคุมเปลี่ยนโหมดหลักของแอปพลิเคชันผ่านเมนูปุ่มกดสัมผัสด้านล่างสุดของจอ:

1. วิธีสร้างเลย์เอาต์ Tabs ใน `app/_layout.tsx`

ใช้คำสั่งนำเข้าจาก expo-router และจัดวางไอคอนสไตล์:

```
// app/_layout.tsx
import { Tabs } from 'expo-router';
import { Ionicons } from '@expo/vector-icons'; // โหลดไอคอนมาตรฐาน

export default function TabsLayout() {
  return (
    <Tabs
      screenOptions={{
        tabBarActiveTintColor: '#38bdf8', // สีเมนูขณะถูกคลิกเลือก
        tabBarInactiveTintColor: '#94a3b8',
        tabBarStyle: { backgroundColor: '#0f172a' },
      }}
    >
      <Tabs.Screen
        name="index"
        options={{
          title: 'หน้าหลัก',
          tabBarIcon: ({ color, size }) => (
            <Ionicons name="home" color={color} size={size} />
    ),
          }}
   />
      <Tabs.Screen
        name="settings"
        options={{
          title: 'ตั้งค่า',
          tabBarIcon: ({ color, size }) => (
            <Ionicons name="settings" color={color} size={size} />
    ),
          }}
   />
    </Tabs>
 );
}
```

6.5 การเชื่อมโยงลิงก์และเปลี่ยนหน้าจอ (Link & Router Object)

ใน Expo Router มี 2 วิธีหลักในการสั่งนำทางเปลี่ยนหน้าจอ:

1. การนำทางแบบประกาศผ่านคอมโพเนนต์ <Link> (Declarative Navigation)

เหมาะอย่างยิ่งสำหรับข้อความ ตัวหนังสือ หรือปุ่มสัมผัสสเตติกทั่วไป โดยส่งพาธปลายทางผ่านพร็อพส์ href:

```
import { Link } from 'expo-router';
```

```
export default function HomeScreen() {
  return (
    <View>
      <Link href="/profile" asChild>
        {/* asChild สั่งให้ Link ไปคุมคุณสมบัติสัมผัสของคอมโพเนนต์ลูกย่อย */}
        <Pressable><Text>เข้าหน้าโปรไฟล์</Text></Pressable>
      </Link>
    </View>
  );
}
```

2. การสั่งการนำทางด้วยเหตุการณ์ JavaScript ผ่าน `router` (Imperative Navigation)

กรณีต้องการประมวลผลโลจิกสำเร็จก่อนนำทาง เช่น เช็คว่าผู้ใช้กรอกฟอร์มเสร็จแล้ว หรือดึงบันทึกสำเร็จ จึงค่อยย้ายหน้า:

- `router.push('/path')`: ผลักหน้าจอใหม่ทับซ้อนบนสแต็ค (ผู้ใช้ย้อนกลับได้)
- `router.replace('/path')`: ล้างหน้าจอปัจจุบันออกและสวมทับหน้าใหม่ (ใช้บ่อยในระบบ Auth ล็อกอินสำเร็จแล้วไม่ให้กดย้อนกลับมาหน้ารับค่าอีก)
- `router.back()`: ย้อนกลับหน้าจอก่อนหน้า 1 ระดับ

```
import { router } from 'expo-router';
import { Pressable, Text } from 'react-native';

export default function LoginButton() {
  const handleLogin = () => {
  // สมมติล็อกอินสำเร็จแล้ว
    router.replace('/dashboard'); // เปลี่ยนแบบแทนที่หน้าเดิมทันที
  };

  return (
    <Pressable onPress={handleLogin}>
      <Text>ยืนยันการเข้าสู่ระบบ</Text>
    </Pressable>
  );
}
```

6.6 เส้นทางนำทางแบบไดนามิกและการอ่านค่าพารามิเตอร์ (Dynamic Routes & useLocalSearchParams)

ในกรณีที่หน้าจอต้องการรับตัวแปร ID เพื่อไปยิงต่อค้นหารายละเอียดสินค้าเฉพาะชิ้น (เช่น หน้ารายละเอียดข่าว ID 125) เราใช้เทคนิคการตั้งชื่อไฟล์แบบใส่เครื่องหมายวงเล็บเหลี่ยม `[id].tsx` เพื่อกำหนดเป็น Dynamic Route

1. วิธีจัดเก็บโครงสร้างไฟล์ไดนามิก

```
app/
├── index.tsx                    (หน้ารวมข่าวสาร)
└── news/
    └── [id].tsx                 (หน้าอ่านข่าวเฉพาะชิ้น)
```

2. วิธีสั่งลิงก์นำทางส่งตัวแปร (Navigation with Parameters)

เราส่งพาธไดนามิกแนบข้อมูลพารามิเตอร์ไปได้โดยตรง:

```
// นำทางไปยังแอปหน้าอ่านข่าวรหัส 45
router.push('/news/45');

// หรือส่งผ่านพารามิเตอร์สไตล์ Query String เพิ่มเติม
router.push('/news/45?author=Somchai');
```

3. วิธีการดักจับค่าตัวแปรในหน้าปลายทางด้วย `useLocalSearchParams`

ภายในไฟล์ย่อย `app/news/[id].tsx` เราจะดึงข้อมูลตัวแปรจากพารามิเตอร์ทรานสิทผ่านมาด้วย `useLocalSearchParams` Hook:

```
// app/news/[id].tsx
import { useLocalSearchParams, router } from 'expo-router';
import { View, Text, Button, StyleSheet } from 'react-native';

export default function NewsDetailScreen() {
 // ดักจับค่า id และพารามิเตอร์ author ที่ส่งผ่านเส้นทาง URL
  const { id, author } = useLocalSearchParams<{ id: string; author?: string }>();

  return (
    <View style={styles.container}>
      <Text style={styles.title}>รายละเอียดข่าวสารรหัส: #{id}</Text>
      {author && <Text style={styles.authorText}>ผู้เขียน: {author}</Text>}

      <Button title="กลับหน้าแรก" onPress={() => router.back()} />
    </View>
```

```
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#0f172a',
  },
  title: {
    fontSize: 20,
    color: '#ffffff',
    fontWeight: 'bold',
    marginBottom: 8,
  },
  authorText: {
    fontSize: 14,
    color: '#38bdf8',
    marginBottom: 20,
  },
});
```

สรุปท้ายบทเรียน

แนวคิดการจัดเส้นทางแบบอิงโครงสร้างไฟล์: ระบบนำทางยุคใหม่ใน Expo SDK 54 ใช้สถาปัตยกรรมแบบ File-based Routing ผ่าน Expo Router v6 (Expo, 2024) โดยความสะดวกคือระบบจะแปลงพาธตำแหน่งโครงสร้างไฟล์ภายใต้โฟลเดอร์ app/ ให้กลายเป็นเส้นทางการเปลี่ยนหน้าตาแอปพลิเคชัน (URL Routes) โดยอัตโนมัติ ช่วยลดความยุ่งยากในการลงทะเบียนโค้ดแบบดั้งเดิม

การสร้าง Layout และการจัดระเบียบโครงสร้างด้วย _layout.tsx: ไฟล์ _layout.tsx เป็นกลไกศูนย์กลางเฉพาะของระบบมีประโยชน์ในด้านการควบคุมหน้าตาหลักของแอป เช่น หน้าจอ Stack หรือ หน้าจอแท็บบาร์ (Tabs) โดยทำหน้าที่เป็นโครงสร้างห่อหุ้มอินเทอร์เฟซส่วนกลาง (Shared UI) และช่วยจัดระเบียบความปลอดภัยในการกรองผู้เข้าใช้งานหน้าจอต่าง ๆ ได้อย่างเป็นระบบ

ระบบนำทางแบบสะสมประวัติ (Stack Navigation): โครงสร้าง Stack Navigation จำลองการนำทางในรูปแบบการวางแผ่นการ์ดซ้อนทับกัน (Push/Pop Screen) เมื่อเปิดหน้าจอใหม่จะนำหน้าต่างใหม่ไปทับแผ่นเก่า และเมื่อกดย้อนกลับหน้าจอด้านบนจะถูกดึงออกเพื่อกลับลงไป

แสดงแผ่นถัดลงไป โดยเราเรียกนำเข้าคอมโพเนนต์ `<Stack>` และปรับแต่งคุณสมบัติต่าง ๆ ในหน้า _layout.tsx

ระบบนำทางแบบแท็บบาร์ล่าง (Tab Navigation): เลย์เอาต์แบบ Tabs เป็นการสร้างเมนูนำทางหลักสัมผัสด้านล่างสุดของสมาร์ทโฟนสำหรับสลับหมวดหลักของแอปพลิเคชันอย่างรวดเร็ว โดยสร้าง _layout.tsx แยกตามโฟลเดอร์ของกลุ่มงานและเรียกใช้คอมโพเนนต์ `<Tabs>` ร่วมกับ `<Tabs.Screen>` เพื่อระบุไอคอนและสไตล์สีเมื่อปุ่มทำงานหรือว่างเปล่าได้ตามที่ออกแบบไว้

การเชื่อมโยงลิงก์และเปลี่ยนหน้าจอ: การเปลี่ยนหน้าทำได้ 2 แนวทาง ได้แก่ การใช้คอมโพเนนต์ `<Link>` ร่วมกับพร็อพส์ href สำหรับการกดเชื่อมโยงทั่วไป และการใช้คำสั่ง JavaScript สั่งเปลี่ยนหน้าผ่านออบเจกต์ router (เช่น router.push สำหรับผลักหน้าจอขึ้นสแต็ค, router.replace เพื่อการข้ามหน้าแบบลบหน้าเดิมทิ้ง และ router.back สำหรับย้อนกลับหน้าเดิม)

เส้นทางนำทางแบบไดนามิกและการอ่านค่าพารามิเตอร์: สำหรับหน้าที่ต้องการรับตัวแปรระบุ ID เฉพาะ เราจะใช้การตั้งชื่อไฟล์แบบใส่เครื่องหมายวงเล็บเหลี่ยม เช่น [id].tsx เพื่อกำหนดเป็น Dynamic Route ซึ่งสามารถส่งผ่านพารามิเตอร์และตัวแปรผ่านเส้นทาง URL ได้อย่างอิสระ และฝั่งหน้าปลายทางจะดักจับข้อมูลเหล่านั้นมาใช้งานได้สะดวกผ่าน useLocalSearchParams Hook

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความเข้าใจเกี่ยวกับสถาปัตยกรรม File-based Routing หน้าที่ของไฟล์ `_layout.tsx` เลย์เอาต์แบบ Stack และ Tabs วิธีเปลี่ยนหน้าจอด้วย `<Link>` และ `router` ตลอดจนการทำระบบจัดเส้นทางไดนามิกและการอ่านค่าด้วย `useLocalSearchParams` ใน React Native Expo SDK 54

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดคือคำอธิบายของแนวคิดระบบการจัดเส้นทางแบบ "File-based Routing" ใน Expo Router ได้ถูกต้องที่สุด?

ก. เส้นทางการเชื่อมต่อจะตั้งชื่อตามฟังก์ชันภายในของ Database

ข. โครงสร้างไฟล์และโฟลเดอร์ภายในโฟลเดอร์ `app/` จะถูกนำมาใช้ระบุขอบเขตเส้นทางการนำหน้าจอของแอปพลิเคชันโดยอัตโนมัติ

ค. ต้องทำรายการประกาศเชื่อมพาธในไฟล์ CSS เสมอ

ง. แอปพลิเคชันจะแปลงหน้าทั้งหมดให้ออกมารันบนเว็บบราวเซอร์ผ่านชื่อไฟล์เท่านั้น

ข้อ 2: โฟลเดอร์ใดที่เป็นตำแหน่งรากหลัก (Root Directory) สำหรับการตั้งวางและตรวจจับไฟล์เส้นทางหน้าจอของระบบ Expo Router?

ก. `src/`

ข. `components/`

ค. `app/`

ง. `assets/`

ข้อ 3: ไฟล์พิเศษที่ใช้ชื่อว่า `_layout.tsx` มีหน้าที่และความสำคัญหลักอย่างไรในโฟลเดอร์เส้นทางนำทาง?

ก. คอยจัดเตรียมสไตล์สีและฟอนต์ภายนอกทั้งหมดโดยไม่เกี่ยวกับหน้าจอ

ข. ใช้ระบุกรุปเลย์เอาต์การนำทางร่วมกัน (เช่น Stack หรือ Tabs) ให้แก่กลุ่มไฟล์ในโฟลเดอร์ระดับเดียวกันและโฟลเดอร์ย่อย

ค. เป็นคลังเก็บไฟล์รูปภาพไอคอนของโปรเจกต์

ง. ล้างค่าย้อนหลังข้อมูลสเตตเมื่อผู้ใช้งานปิดเปิดเครื่องใหม่

ข้อ 4: ในระบบนำทางแบบ Stack Navigation พฤติกรรมแบบดั้งเดิมในการจัดการสแต็คหน้าจอเป็นอย่างไร?

ก. หน้าจอจะสลับเรียงแนวนอนแบบข้างชนข้างโดยไม่จัดหน้าทับซ้อนกัน

ข. แสดงผลหน้าจอใหม่ซ้อนทับขึ้นมาอยู่ด้านบนสุด (Push) และดึงหน้าจอนั้นออกไปเพื่อกลับสู่หน้าด้านล่าง (Pop)

ค. นำทางผู้ใช้ไปยังเบราว์เซอร์ภายนอกเพื่อดาวน์โหลดรูปภาพ

ง. ปิดหน้าจออื่นทั้งหมดทันทีและเหลือเพียงหน้าแรกสุดเสมอ

ข้อ 5: หากต้องการแก้ไขชื่อหัวข้อแถบส่วนหัว (Header Title) สำหรับหน้าจอเฉพาะเจาะจงที่อยู่ภายใต้เลย์เอาต์ `<Stack>` ควรเขียนอย่างไร?

ก. กำหนดใน Property ชื่อ `title` ของสไตล์ชีท

ข. เรียกใช้งานคอมโพเนนต์ `<Stack.Screen options={{ title: 'ชื่อหัวข้อที่ต้องการ' }} />` ภายในไฟล์หน้าจอนั้น

ค. ตั้งชื่อไฟล์ด้วยอักขระตัวพิมพ์ใหญ่ทั้งหมด

ง. ส่งค่าตัวหนังสือไปป้อนที่ตัวแปรของ `useLocalSearchParams`

ข้อ 6: การระบุพร็อพส์ `asChild` ในคอมโพเนนต์ `<Link>` (เช่น `<Link href="/profile" asChild>`) ส่งผลอย่างไรต่อคอมโพเนนต์ลูกด้านใน?

ก. สั่งให้ล้างประวัติการนำทางทั้งหมดทิ้งและจำลองเป็นหน้าแรกสุด

ข. ทำหน้าที่คอยจำลองการทำงานส่งต่อเหตุการณ์กดสัมผัส (onPress) และคุณสมบัติการนำทางไปควบคุมชิ้นส่วนของคอมโพเนนต์ลูกแทนการสร้างแท็ก `<a>` ครอบ

ค. บังคับเปลี่ยนรูปภาพลูกให้ยืดหยุ่นตามขนาดของขอบจอ

ง. สั่งเปิดลิงก์ไปยังเว็บเบราว์เซอร์ภายนอก

ข้อ 7: การนำทางโดยเรียกใช้งานคำสั่ง `router.replace('/home')` แตกต่างจากการเรียกใช้งาน `router.push('/home')` อย่างไร?

ก. `replace` จะผลักหน้าซ้อนสแต็คต่อประวัติเดิม แต่ `push` จะล้างข้อมูลออก

ข. `replace` จะทำลายหน้าจอปัจจุบันและสวมทับเส้นทางใหม่ลงไปแทนที่ประวัติเดิม (ย้อนกลับมาไม่ได้) ส่วน `push` จะวางหน้าจอทับซ้อนเพิ่มขึ้นไปในสแต็คสะสม

ค. `replace` ทำงานได้เฉพาะระบบ iOS ส่วน `push` ทำงานเฉพาะระบบ Android

ง. `replace` ไม่รองรับการส่งผ่านค่าข้อมูลไดนามิกพารามิเตอร์

ข้อ 8: หากต้องการออกแบบหน้าต่างแสดงรายละเอียดไอเท็มแบบรับตัวแปรไดนามิก ID เช่น พาธ `/details/12` หรือ `/details/abc` ผู้พัฒนาต้องตั้งชื่อไฟล์อย่างไร?

ก. `details-id.tsx`

ข. `[details].tsx`

ค. `[id].tsx` เก็บไว้ในโฟลเดอร์ชื่อ `details`

ง. `details.id.tsx`

ข้อ 9: ในหน้าจอปลายทางรับค่าเส้นทางไดนามิก เราจะดึงข้อมูลตัวแปรพารามิเตอร์ที่พ่วงส่งผ่านมาดักรับใช้งานด้วยเครื่องมือ (Hook) ตัวใด?

ก. `useState`

ข. `useLocalSearchParams`

ค. `useRouteOptions`

ง. `useNavigationState`

ข้อ 10: เลย์เอาต์การนำทางประเภทใดที่เหมาะสำหรับการสร้างแถบเมนูปุ่มนำทางหลักยึดติดอยู่ท้ายสุดของหน้าจอ เพื่อสลับหน้าการทำงานใหญ่ ๆ ในแอป?

ก. `<Stack>`

ข. `<Tabs>`

ค. `<Drawer>`

ง. `<ScrollView>`

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: การนำทางในแอปมือถือไม่เกี่ยวกับการจัดการดาต้าเบสหลังบ้านโดยตรง

• ข. ถูก: File-based Routing ของ Expo Router ทำงานโดยจับคู่พิกัดไฟล์ใน **app/** มาแปลเป็น Route นำทางอัตโนมัติ ช่วยลดความซับซ้อนในการเขียนแผนผังลงทะเบียน

• ค. และ ง. ผิด: CSS จัดตกแต่งสีสันเท่านั้น และแอปมือถือรันระบบนำทางระดับเนทีฟไม่ใช่รันเว็บพาธตรงตัว

เฉลย ข้อ 2: ตอบ ค.

• เหตุผลประกอบ:

• ก. ข. และ ง. ผิด: เป็นโฟลเดอร์เก็บซอร์สโค้ดอื่น คอมโพเนนต์แชร์ หรือภาพประกอบที่ไม่ใช่จุดตั้งเส้นทางนำทางหลัก

• ค. ถูก: Expo Router บังคับออกแบบให้โฟลเดอร์ **app/** ที่ระดับรากโครงการเป็นตำแหน่งจับคู่เส้นทางนำทางทั้งหมด

เฉลย ข้อ 3: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: ดีไซน์ภายนอกฟอนต์จะใช้การโหลด Asset ปกติ

• ข. ถูก: ไฟล์ **`_layout.tsx`** ใช้คุมเส้นทางร่วมกัน หากเราประกาศ Stack หรือ Tabs ไว้ในนี้ หน้าจอย่อยทั้งหมดในชั้นโฟลเดอร์เดียวกันจะทำงานอยู่ใต้สไตล์เลย์เอาต์นำทางดังกล่าว

• ค. และ ง. ผิด: ไม่ใช่ที่รวมภาพ และสเตตจะล้างตามปกติหากปิดตัวแอป

เฉลย ข้อ 4: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: การสลับเรียงแนวนอนแบบ Slide เป็นเพียง Transition ไม่ใช่หลักพฤติกรรมการจัดการสแต็ค

• ข. ถูก: โครงสร้างนำทางแบบ Stack ทำงานแบบ LIFO (Last In First Out) บล็อกใหม่ซ้อนบนสุด (Push) และเคลียร์ออกด้านหลังเพื่อกลับจุดเดิม (Pop)

• ค. และ ง. ผิด: ไม่ใช่คำสั่งนำออกเบราว์เซอร์ และยังมีกลไกคงหน้าเก่าค้างไว้ใต้สแต็ค

เฉลย ข้อ 5: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: สไตล์ชีทไม่สามารถเปลี่ยนข้อความของแถบหัวเรื่องระบบเนทีฟได้

• **ข. ถูก:** ตัวเสริม `<Stack.Screen>` ของ Expo Router เปิดช่องให้เราสั่ง Override ค่าตัวเลือก (Options) ประจำหน้าจอได้จากไฟล์ของหน้านั้น ๆ โดยตรง

• **ค. และ ง. ผิด:** การตั้งชื่อไฟล์ด้วยตัวพิมพ์ใหญ่ไม่ส่งผลต่อชื่อหัวข้อ และพารามิเตอร์มีไว้รับส่งตัวแปรข้อมูล

เฉลย ข้อ 6: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** ไม่เกี่ยวกับการเคลียร์สแต็ค ปรับขนาดรูป หรือเชื่อมเว็บภายนอก

• **ข. ถูก:** พร็อพส์ `asChild` เป็นตัวช่วยส่งต่อ Event Handler `onPress` ของปุ่มลิงก์ผ่านลงไปยังปุ่ม `Pressable` หรือชิ้นงานลูกย่อยด้านใน ป้องกันการซ้อนทับกันของแท็กระบบนำทาง

เฉลย ข้อ 7: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** สลับพฤติกรรมกัน

• **ข. ถูก:** `router.push()` จะวางหน้าจอทับซ้อนเพิ่มในประวัติทำให้กดย้อนกลับได้ ส่วน `router.replace()` จะเปลี่ยนหน้าแบบทับแทนที่หน้าเก่าทันที เหมาะสำหรับหน้าระบบลงทะเบียนเสร็จแล้วไม่อยากให้ย้อนกลับมาเจอหน้าฟอร์มกรอกซ้ำ

• **ค. และ ง. ผิด:** ทั้งคู่ทำงานได้เหมือนกันทั้งสองระบบ OS และสนับสนุนการส่งพารามิเตอร์ปกติ

เฉลย ข้อ 8: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ข. และ ง. ผิด:** ชื่อรูปแบบอื่นไม่ถูกแปลงเป็นตัวแปรไดนามิกในระบบ Expo Router v6

• **ค. ถูก:** การห่อชื่อไฟล์ด้วยสัญลักษณ์วงเล็บเหลี่ยม `[id].tsx` เป็นการระบุว่า คีย์ชื่อ `id` จะเป็นตัวดักจับสตริงข้อมูลใน URL หรือพาธเพื่อนำไปประมวลผลต่อ

เฉลย ข้อ 9: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `useState` จัดการสถานะภายในของตัวหน้านั้น ไม่เกี่ยวข้องกับการแปลงตัวแปร URL

• **ข. ถูก:** `useLocalSearchParams` ถูกสร้างขึ้นโดยเฉพาะใน Expo Router เพื่ออ่านค่าพารามิเตอร์ที่แนบมากับเส้นทางเดินหน้าจอปลายทาง (เช่น ดึงค่า ID สินค้า)

• **ค. และ ง. ผิด:** ไม่ใช่ชื่อคำสั่งมาตรฐาน

เฉลย ข้อ 10: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: Stack เหมาะสำหรับหน้าจอย่อยซ้อนลึก Drawer เหมาะสำหรับแถบปัดด้านข้าง ส่วน ScrollView ใช้สำหรับเลื่อนข้อความไม่ใช่เลย์เอาต์การเดินหน้าจอหลัก

• ข. ถูก: เลย์เอาต์แบบ **<Tabs>** ออกแบบมาจัดวางแท็บเดินหน้าระดับล่าง (Bottom Tab Bar) เพื่อความลื่นไหลในการสลับใช้งานฟีเจอร์หลัก ๆ ของแอป

ปฏิบัติการท้ายบทเรียน: การสร้างระบบนำทางหน้าข้อมูลจุดท่องเที่ยวแบบไดนามิก

ปฏิบัติการนี้จะฝึกทักษะการประยุกต์ใช้งานระบบจัดเส้นทางเดินหน้าจอด้วยโครงสร้างโฟลเดอร์และไฟล์ของ **Expo Router** โดยผู้เรียนจะได้ออกแบบแอปพลิเคชัน **แนะแนวแหล่งท่องเที่ยวออนไลน์ (Travel Guide & Destinations Directory)** ซึ่งเชื่อมโยงการทำงานแบบหน้าจอหลักเชื่อมสู่หน้ารายละเอียดแบบส่งผ่านค่าตัวแปรไดนามิกพารามิเตอร์ข้ามหน้าจอ

1. วัตถุประสงค์การเรียนรู้

1. สามารถออกแบบและตั้งค่าสถาปัตยกรรมระบบนำทางแบบซ้อนทับ (**Stack**) ด้วยไฟล์ **_layout.tsx**

2. สามารถเขียนฟังก์ชันนำทางไปยังจุดหมายต่าง ๆ โดยเรียกใช้ออบเจกต์คำสั่ง **router**

3. สามารถกำหนดโครงสร้างแฟ้มงานในรูปแบบไดนามิกลิงก์ (**app/destination/[id].tsx**) เพื่อดักจับรับค่าพารามิเตอร์

4. สามารถเรียกใช้ Hook **useLocalSearchParams** เพื่อรับข้อมูลรหัสประจำเมืองและนำมาจัดทำเนื้อหาเฉพาะตัวสำเร็จ

5. สามารถอัปเดตสไตล์และข้อความแถบส่วนหัวบาร์ (Dynamic Header Options) ผ่านคอมโพเนนต์ **<Stack.Screen>**

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนจะต้องเขียนโค้ดจัดโครงสร้างระบบสตรีมนำทางภายใต้โฟลเดอร์หลัก **app/** ดังนี้:

```
โครงสร้างโฟลเดอร์ปฏิบัติการ:
app/
├── _layout.tsx                    => เลย์เอาต์ Stack ควบคุมหลัก
├── index.tsx                      => หน้าหลักรวมการ์ดเมืองท่องเที่ยว (Home List Screen)
└── destination/
    └── [id].tsx                   => หน้ารายละเอียดท่องเที่ยวเจาะลึกเฉพาะเมือง (Dynamic details)
```

รายละเอียดและเงื่อนไขการนำทาง:

1. **เลย์เอาต์หลัก (`app/_layout.tsx`):**
   - กำหนดให้เป็นระบบ `<Stack>`
   - ตกแต่งบาร์ด้านบนให้เป็นเฉดสี Slate โทนน้ำเงินเข้ม (`#1e293b`) ตัวหนังสือสีขาว
   - เปิดระบบแสดงผลปุ่มย้อนกลับสีฟ้าสว่าง (`#38bdf8`) อัตโนมัติบนหน้าย่อย
2. **หน้าหลักรวมจุดท่องเที่ยว (`app/index.tsx`):**
   - มีคลังข้อมูลเมืองท่องเที่ยวเป็นอาร์เรย์ (เช่น Tokyo, Paris, New York) ประกอบด้วย ID ชื่อประเทศ และชื่อภาพ
   - แสดงรูปภาพเมืองและป้ายสั้นโดยจัดด้วย `FlatList`
   - เมื่อผู้เรียนกดคลิกสัมผัสการ์ดเมืองใด ๆ จะต้องทริกเกอร์เรียกคำสั่ง `router.push()` เพื่อนำทางไปยังหน้ารายละเอียดเมืองนั้น พร้อมระบุพ่วงส่ง ID และพารามิเตอร์ชื่อเมือง (Query string) ไปด้วย
3. **หน้ารายละเอียดจุดท่องเที่ยวแบบไดนามิก (`app/destination/[id].tsx`):**
   - ดักจับรหัส ID และตัวแปร Query จากเส้นทางด้วย Hook `useLocalSearchParams`
   - เปลี่ยนข้อความหัวข้อบาร์ด้านบนสุด (Header Title) ของหน้าจอให้กลายเป็น "ข้อมูลของเมือง: [ชื่อเมือง]" ไดนามิกอัตโนมัติด้วย `<Stack.Screen options={{ title: ... }} />`
   - แสดงรายละเอียดคำบรรยายและภาพเมืองนั้น ๆ
   - มีปุ่ม "ย้อนกลับหน้าหลัก" สไตล์โมเดิร์นคุมด้วย `Pressable` ที่จะกระตุ้นคำสั่ง `router.back()` ย้อนหน้ากลับ

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ตั้งค่าเลย์เอาต์นำทาง Stack หลักใน `app/_layout.tsx`

สร้างหรือแก้ไขไฟล์ `app/_layout.tsx` เพื่อนำเสนอรูปแบบแผนผังการจัดหน้าซ้อนสะสม (Stack):

```
import { Stack } from 'expo-router';

export default function RootLayout() {
  return (
    <Stack
      screenOptions={{
        headerStyle: {
          backgroundColor: '#0f172a', // ธีมหัวข้อสีเข้ม
```

```
        },
        headerTintColor: '#38bdf8',    // ปุ่มกด Back สีฟ้านีออน
        headerTitleStyle: {
          fontWeight: 'bold',
          color: '#ffffff',            // ตัวหนังสือสีขาว
        },
        contentStyle: {
          backgroundColor: '#0f172a', // คุมสีพื้นหลังหน้าจอทั้งหมดใน Stack
      }
        }}
      >
        {/* หน้าแรกจะแสดงหัวข้อ "แหล่งท่องเที่ยวทั่วโลก" */}
        <Stack.Screen
          name="index"
          options={{ title: 'แหล่งท่องเที่ยวทั่วโลก' }}
    />
        {/* ส่วนหน้าไดนามิกจะตั้งค่า title แบบยืดหยุ่นในไฟล์ตัวเอง */}
       <Stack.Screen
          name="destination/[id]"
          options={{ title: 'กำลังโหลดข้อมูลเมือง...' }}
    />
      </Stack>
  );
}
```

ขั้นตอนที่ 2: สร้างหน้าจอลิสต์เมืองท่องเที่ยวส่งตัวแปรไดนามิกใน app/index.tsx

ใช้คำสั่งนำเข้าจาก expo-router และเขียนจำลองปุ่มส่งตัวแปรลิงก์:

```
import { View, Text, FlatList, Pressable, Image, StyleSheet } from 'react-
native';
import { router } from 'expo-router';

// ชุดข้อมูลจำลองจุดท่องเที่ยว
const DESTINATIONS = [
  { id: '1', name: 'โตเกียว (Tokyo)', country: 'ประเทศญี่ปุ่น', image:
'https://images.unsplash.com/photo-1540959733332-eab4deceeaf7?w=300' },
  { id: '2', name: 'ปารีส (Paris)', country: 'ประเทศฝรั่งเศส', image:
'https://images.unsplash.com/photo-1502602898657-3e91760cbb34?w=300' },
  { id: '3', name: 'นิวยอร์ก (New York)', country: 'สหรัฐอเมริกา', image:
'https://images.unsplash.com/photo-1496442226666-8d4d0e62e6e9?w=300' },
];

export default function HomeList() {
  return (
    <View style={styles.container}>
      <FlatList
        data={DESTINATIONS}
```

```
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => (
          <Pressable
      // สั่งนำทางแบบ Push ส่งค่า ID ไปที่ Path และพ่วงชื่อเมืองเป็น Query String
            onPress={() =>
router.push(`/destination/${item.id}?cityName=${item.name}`)}
         style={({ pressed }) => [
              styles.card,
              { opacity: pressed ? 0.95 : 1 }
      ]}
            >
              <Image source={{ uri: item.image }} style={styles.image} />
              <View style={styles.info}>
        <Text style={styles.title}>{item.name}</Text>
              <Text style={styles.subtitle}>{item.country}</Text>
              </View>
            </Pressable>
    )}
        contentContainerStyle={styles.list}
   />
    </View>
 );
}

const styles = StyleSheet.create({
 // รายละเอียดการจัดสไตล์คาร์ดจะรวมอยู่ในโค้ดเฉลยเต็ม
});
```

ขั้นตอนที่ 3: รับค่าพารามิเตอร์และแสดงหน้าจอเฉพาะเมืองใน app/destination/[id].tsx

ดึงพารามิเตอร์มาจำลองข้อมูลเนื้อหาและการเซ็ตค่า Header อัตโนมัติ:

```
import { useLocalSearchParams, router, Stack } from 'expo-router';
import { View, Text, Image, Pressable, StyleSheet } from 'react-native';

// คำบรรยายข้อมูลท่องเที่ยวจำลองแยกตาม ID
const DESTINATION_DETAILS: Record<string, { desc: string; rating: string;
cover: string }> = {
  '1': {
    desc: 'โตเกียว เมืองหลวงของญี่ปุ่น ผสมผสานความทันสมัยจากตึกระฟ้าและวัฒนธรรมโบราณจากวัดและศาลเจ้าได้
อย่างลงตัว มีแหล่งช้อปปิ้งและอาหารเลิศรสรอคุณอยู่',
    rating: '☆ 4.9 / 5',
    cover: 'https://images.unsplash.com/photo-1540959733332-eab4deceeaf7?w=500'
  },
  '2': {
    desc: 'ปารีส เมืองหลวงแห่งแฟชั่นและศิลปะของฝรั่งเศส เต็มไปด้วยสถาปัตยกรรมสุดคลาสสิก หอไอเฟล พิพิธภัณฑ์ลูฟร์
และบรรยากาศสุดโรแมนติกริมแม่น้ำแซน',
```

```
    rating: '☆ 4.8 / 5',
    cover: 'https://images.unsplash.com/photo-1502602898657-3e91760cbb34?w=500'
  },
  '3': {
    desc: 'นิวยอร์ก นครที่ไม่เคยหลับใหลของสหรัฐฯ แหล่งรวมความบันเทิงระดับโลก ไทม์สแควร์ เซ็นทรัลพาร์ก โรงละครบรอดเวย์ และความหลากหลายทางวัฒนธรรม',
    rating: '☆ 4.7 / 5',
    cover: 'https://images.unsplash.com/photo-1496442226666-8d4d0e62e6e9?w=500'
  },
};

export default function DestinationDetail() {
 // ดึงรับพารามิเตอร์ ID และตัวแปร Query String
  const { id, cityName } = useLocalSearchParams<{ id: string; cityName?: string }>();

  const detail = DESTINATION_DETAILS[id];

  if (!detail) {
    return (
      <View style={styles.centerContainer}>
        <Text style={styles.errorText}>ไม่พบข้อมูลเมืองท่องเที่ยวนี้</Text>
      </View>
  );
  }

  return (
    <View style={styles.container}>
      {/* ตั้งค่า Title บาร์ด้านบนตามชื่อเมืองที่รับมาแบบเรียลไทม์ */}
      <Stack.Screen options={{ title: cityName || 'รายละเอียดเมือง' }} />

      <Image source={{ uri: detail.cover }} style={styles.coverImage} />

      <View style={styles.content}>
        <Text style={styles.title}>{cityName}</Text>
        <Text style={styles.rating}>{detail.rating} คะแนนยอดนิยม</Text>
        <Text style={styles.desc}>{detail.desc}</Text>

        {/* ปุ่มกดย้อนกลับหน้าแรกจัดด้วย Pressable */}
        <Pressable
          onPress={() => router.back()}
          style={({ pressed }) => [
            styles.backButton,
            pressed ? styles.backButtonPressed : null
    ]}
        >
          <Text style={styles.backButtonText}>ย้อนกลับหน้าหลัก</Text>
        </Pressable>
```

```
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  // การจัดระยะสเปซจะแสดงในส่วนของเฉลยสมบูรณ์
});
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถนำข้อมูลชุดโค้ดเฉลยเต็มรูปแบบด้านล่างนี้แยกบันทึกตามเส้นทางไฟล์ของโปรเจกต์ เพื่อทดสอบการเปลี่ยนหน้าจอส่งผ่านพารามิเตอร์ที่สวยงามแบบเนทีฟ:

4.1 ซอร์สโค้ดไฟล์เลย์เอาต์: app/_layout.tsx

```
import { Stack } from 'expo-router';

export default function RootLayout() {
  return (
    <Stack
      screenOptions={{
        headerStyle: {
          backgroundColor: '#0f172a',
        },
        headerTintColor: '#38bdf8',
        headerTitleStyle: {
          fontWeight: 'bold',
          color: '#ffffff',
        },
        contentStyle: {
          backgroundColor: '#0f172a',
        }
      }}
    >
      <Stack.Screen
        name="index"
        options={{ title: 'จุดหมายปลายทางยอดนิยม' }}
    />
      <Stack.Screen
        name="destination/[id]"
        options={{ title: 'รายละเอียดสถานที่ท่องเที่ยว' }}
    />
    </Stack>
  );
}
```

4.2 ซอร์สโค้ดหน้าหลักสารบัญเมือง: app/index.tsx

```
import { View, Text, FlatList, Pressable, Image, StyleSheet } from 'react-
native';
import { router } from 'expo-router';

const DESTINATIONS = [
  { id: '1', name: 'โตเกียว (Tokyo)', country: 'ประเทศญี่ปุ่น', image:
'https://images.unsplash.com/photo-1540959733332-eab4deceeaf7?w=300' },
  { id: '2', name: 'ปารีส (Paris)', country: 'ประเทศฝรั่งเศส', image:
'https://images.unsplash.com/photo-1502602898657-3e91760cbb34?w=300' },
  { id: '3', name: 'นิวยอร์ก (New York)', country: 'สหรัฐอเมริกา', image:
'https://images.unsplash.com/photo-1496442226666-8d4d0e62e6e9?w=300' },
];

export default function HomeList() {
  return (
    <View style={styles.container}>
      <Text style={styles.listHeader}>สำรวจเมืองในฝันของคุณ</Text>

      <FlatList
        data={DESTINATIONS}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => (
          <Pressable
            onPress={() =>
router.push(`/destination/${item.id}?cityName=${item.name}`)}
            style={({ pressed }) => [
       styles.card,
              { transform: [{ scale: pressed ? 0.98 : 1.0 }] }
      ]}
          >
            <Image source={{ uri: item.image }} style={styles.image} />
            <View style={styles.info}>
              <Text style={styles.title}>{item.name}</Text>
              <Text style={styles.subtitle}>{item.country}</Text>
            </View>
            <Text style={styles.arrowIcon}>chevron-right ➔</Text>
          </Pressable>
    )}
        contentContainerStyle={styles.list}
   />
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a',
```

```
    paddingHorizontal: 16,
  },
  listHeader: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#94a3b8',
    marginVertical: 16,
  },
  list: {
    paddingBottom: 20,
  },
  card: {
    flexDirection: 'row',
    backgroundColor: '#1e293b',
    borderRadius: 16,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 16,
    padding: 12,
    alignItems: 'center',
  },
  image: {
    width: 70,
    height: 70,
    borderRadius: 12,
    backgroundColor: '#334155',
  },
  info: {
    flex: 1,
    marginLeft: 16,
  },
  title: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  subtitle: {
    fontSize: 13,
    color: '#94a3b8',
    marginTop: 4,
  },
  arrowIcon: {
    color: '#38bdf8',
    fontSize: 12,
    fontWeight: 'bold',
    marginRight: 4,
  },
});
```

4.3 ซอร์สโค้ดหน้าแสดงผลละเอียดท่องเที่ยว: app/destination/[id].tsx

```
import { useLocalSearchParams, router, Stack } from 'expo-router';
import { View, Text, Image, Pressable, StyleSheet, ScrollView } from
'react-native';

const DESTINATION_DETAILS: Record<string, { desc: string; rating: string;
cover: string }> = {
  '1': {
    desc: 'โตเกียว เมืองหลวงของญี่ปุ่น ผสมผสานความทันสมัยจากตึกระฟ้าและวัฒนธรรมโบราณจากวัดและศาลเจ้าได้
อย่างลงตัว มีแหล่งช้อปปิ้งและอาหารเลิศรสรอคุณอยู่ ไม่ว่าจะเป็นการเดินชมย่านชิบูย่า พักผ่อนที่สวนอูเอโนะ หรือสัมผัสอาหาร
รสชาติแท้ๆ ในย่านชินจุกุ',
    rating: '☆ 4.9 / 5',
    cover: 'https://images.unsplash.com/photo-1540959733332-eab4deceeaf7?w=600'
  },
  '2': {
    desc: 'ปารีส เมืองหลวงแห่งแฟชั่นและศิลปะของฝรั่งเศส เต็มไปด้วยสถาปัตยกรรมสุดคลาสสิก หอไอเฟล พิพิธภัณฑ์ลูฟร์
และบรรยากาศสุดโรแมนติกริมแม่น้ำแซน การเดินชมสตรีทคาเฟ่ที่มีกลิ่นอายย้อนยุคริมทาง ชิมครัวซองต์อบสดใหม่ และชมแสงสี
วิจิตรในยามค่ำคืน',
    rating: '☆ 4.8 / 5',
    cover: 'https://images.unsplash.com/photo-1502602898657-3e91760cbb34?w=600'
  },
  '3': {
    desc: 'นิวยอร์ก นครที่ไม่เคยหลับใหลของสหรัฐฯ แหล่งรวมความบันเทิงระดับโลก ไทม์สแควร์ เซ็นทรัลพาร์ก โรงละคร
บรอดเวย์ และความหลากหลายทางวัฒนธรรม ตลอดจนตึกเอ็มไพร์สเตตที่ตั้งตระหง่านท้าทายทุกสายตาผู้มาเยือนจากทั่วสารทิศ',
    rating: '☆ 4.7 / 5',
    cover: 'https://images.unsplash.com/photo-1496442226666-8d4d0e62e6e9?w=600'
  },
};

export default function DestinationDetail() {
  const { id, cityName } = useLocalSearchParams<{ id: string; cityName?:
string }>();

  const detail = DESTINATION_DETAILS[id];

  if (!detail) {
    return (
      <View style={styles.centerContainer}>
        <Text style={styles.errorText}>ไม่พบข้อมูลเมืองท่องเที่ยวนี้</Text>
        <Pressable onPress={() => router.back()}
style={styles.backButton}>
          <Text style={styles.backButtonText}>ย้อนกลับหน้าแรก</Text>
        </Pressable>
      </View>
  );
```

```
  }

  return (
    <ScrollView style={styles.container}>
      {/* ตั้งค่าหัวข้อหน้าจอแบบไดนามิกอิงตามชื่อเมือง */}
      <Stack.Screen options={{ title: cityName || 'รายละเอียดสถานที่' }} />

     <Image source={{ uri: detail.cover }} style={styles.coverImage} />

      <View style={styles.content}>
        <Text style={styles.title}>{cityName}</Text>
        <View style={styles.ratingBadge}>
          <Text style={styles.ratingText}>{detail.rating}</Text>
        </View>

        <Text style={styles.sectionTitle}>เกี่ยวกับสถานที่นี้</Text>
        <Text style={styles.desc}>{detail.desc}</Text>

        <Pressable
          onPress={() => router.back()}
          style={({ pressed }) => [
            styles.backButton,
            pressed ? styles.backButtonPressed : null
    ]}
        >
          <Text style={styles.backButtonText}>ย้อนกลับหน้าหลัก</Text>
        </Pressable>
      </View>
    </ScrollView>
 );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  centerContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#0f172a',
    padding: 20,
  },
  coverImage: {
    width: '100%',
    height: 250,
    backgroundColor: '#334155',
  },
```

```
  content: {
    padding: 20,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  ratingBadge: {
    backgroundColor: '#1e293b',
    borderWidth: 1,
    borderColor: '#334155',
    alignSelf: 'flex-start',
    paddingVertical: 4,
    paddingHorizontal: 10,
    borderRadius: 8,
    marginTop: 8,
    marginBottom: 20,
  },
  ratingText: {
    color: '#fbbf24',
    fontSize: 13,
    fontWeight: 'bold',
  },
  sectionTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#38bdf8',
    marginBottom: 8,
  },
  desc: {
    fontSize: 15,
    color: '#cbd5e1',
    lineHeight: 24,
    marginBottom: 28,
  },
  backButton: {
    backgroundColor: '#38bdf8',
    height: 48,
    borderRadius: 12,
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 10,
    marginBottom: 20,
  },
  backButtonPressed: {
    backgroundColor: '#0ea5e9',
```

```
  },
  backButtonText: {
    color: '#0f172a',
    fontSize: 16,
    fontWeight: 'bold',
  },
  errorText: {
    color: '#f43f5e',
    fontSize: 16,
    marginBottom: 20,
    fontWeight: 'bold',
  },
});
```

เอกสารอ้างอิง

Expo. (2024). *Create Routes with Expo Router*. จาก https://docs.expo.dev/versions/v54.0.0/router/create-pages/

Expo. (2024). *Expo Router Introduction*. จาก https://docs.expo.dev/versions/v54.0.0/router/introduction/

Expo. (2024). *Layouts with Expo Router*. จาก https://docs.expo.dev/versions/v54.0.0/router/layouts/

แผนบริหารการสอนประจำบทที่ 7

หัวข้อเนื้อหาประจำบท

7.1 ระบบพิกัดตำแหน่งสัมพัทธ์และสัมบูรณ์ (Relative and Absolute Positioning)

7.2 การใช้งาน Custom Fonts ด้วย expo-font

7.3 การใช้งานระบบเวกเตอร์ไอคอนด้วย @expo/vector-icons

7.4 การจัดการรูปภาพประสิทธิภาพสูงด้วย expo-image

7.5 การใช้งาน Blurhash Placeholder และ Transition

7.6 การรองรับโหมดมืด (Dark Mode / Theme Support) ด้วย useColorScheme

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถระบุความแตกต่างและใช้คำสั่ง position: 'absolute' ในการวางชิ้นงานทับซ้อนไอคอนได้ถูกต้อง
2. ผู้เรียนสามารถเขียนโค้ดดาวน์โหลด Custom Fonts จาก Google Fonts มาใช้ปรับแต่งแบบอักษรในแอปได้สำเร็จ
3. ผู้เรียนสามารถเรียกใช้ชุดไอคอนจากระบบ @expo/vector-icons ในการเพิ่มความสวยงามให้ปุ่มเมนูนำทางได้สำเร็จ
4. ผู้เรียนสามารถเปลี่ยนโค้ดมาใช้ expo-image พร้อมตั้งค่าพร็อพส์ placeholder แบบ Blurhash ได้อย่างถูกต้อง
5. ผู้เรียนสามารถเขียนควบคุมการเปลี่ยนสไตล์สีสันแอปให้เปลี่ยนสลับสีตามโหมดมืด (Dark Mode) ของระบบโทรศัพท์ปลายทางได้สำเร็จ

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

25. **การบรรยายและเปรียบเทียบเชิงวิเคราะห์ประสิทธิภาพ (Image Rendering Performance Lecture):** บรรยายการโหลดภาพแบบเดิมของแอปที่ทำให้กระตุก และนำเสนอเปรียบเทียบคุณประโยชน์ของ expo-image ในการเซฟแคชและใช้ระบบ Blurhash

26. **การทำ Live Coding Demonstration หัวข้อ Modern Profile Dashboard with Theme Switch:** ผู้สอนสาธิตการออกแบบหน้าระบบโปรไฟล์ระดับพรีเมียม (Premium Card Mockup) ที่เรียกใช้ฟอนต์ Outfit จาก Google Fonts, ใช้ไอคอนตกแต่งขอบมุมปุ่มด้วย Absolute position และใช้ระบบ useColorScheme สลับธีมสว่าง/มืด
27. **กิจกรรมแกะสลักสไตล์ดีไซน์ (Blurhash & Caching Challenge):** ให้ผู้เรียนทดลองดึงรูปจากเน็ตเวิร์กที่มีขนาดใหญ่มาก ๆ มาใส่ expo-image พร้อมปรับเปลี่ยนเอฟเฟกต์การ Fade ภาพตอนแสดงผล และสร้าง Blurhash สรุปหน้าตาการหมุน
28. **กิจกรรมร่วมมือรีแฟกเตอร์ฟอนต์ระบบ (Group Fonts & Icons Integration):** ให้กลุ่มผู้เรียนช่วยกันปรับปรุงโค้ดโปรเจกต์เดิมให้ไม่มีฟอนต์ดั้งเดิมของระบบ โดยต้องเปลี่ยนมาใช้ฟอนต์พิเศษทั้งหมดและดักจับไม่ให้แอปแสดงผลตัวหนังสือพังตอนติดตั้งหน้าแรกสุด

สื่อการเรียนการสอน

24. เอกสารสไลด์เรื่อง "Advanced Styling, Custom Fonts, Vector Icons, and Expo Image in React Native"
25. ตัวอย่างสตริงรหัส Blurhash สั้นสำหรับใช้จำลองสกินก่อนโหลดภาพจริง
26. แหล่งข้อมูลสำหรับค้นหาไอคอนเวกเตอร์อย่างเป็นทางการของ Expo (icons.expo.fyi)
27. โครงการซอร์สโค้ดแดชบอร์ดตกแต่งความงามตั้งต้น (Style Sandbox Project) บนคลัง Git

การวัดผลและการประเมินผล

1. การวัดผล

1.1 ประเมินระดับความรู้เชิงทฤษฎีจากการทำคะแนนแบบทดสอบปรนัยท้ายบทเรียน 10 ข้อ

1.2 ตรวจสอบโครงสร้างคำสั่งการเรียกใช้ฟอนต์ การป้องกันตัวหนังสือกระตุก และ

ความสมบูรณ์ในการเรียกใช้เวกเตอร์ไอคอน

1.3 ตรวจประเมินทักษะการประยุกต์ใช้งานคอมโพเนนต์ expo-image และการเขียนระบบ Dark Mode ไดนามิกสไตล์ในปฏิบัติการ

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถสร้างหน้าแดชบอร์ดความงามที่เปลี่ยนโหมดสีมืดสว่างตามระบบเครื่อง และรันภาพผ่าน expo-image ที่มีตัวช่วยพรีวิวภาพเบลอได้สำเร็จผ่านเกณฑ์ (ผ่าน/ไม่ผ่าน)

2.3 สังเกตพฤติกรรมการแลกเปลี่ยนไอเดียศิลปะ ความตั้งใจประณีตในการเขียนระบุ CSS สไตล์ให้ตรงตามต้นแบบจำลอง

บทที่ 7

การตกแต่งอินเทอร์เฟซ ไอคอน และรูปภาพประสิทธิภาพสูง

การออกแบบประสบการณ์ผู้ใช้งาน (User Experience - UX) ที่ดีไม่ได้ขึ้นอยู่กับฟังก์ชันการทำงานที่ครบถ้วนเพียงอย่างเดียว แต่ยังขึ้นอยู่กับความลื่นไหล ความเร็วในการจัดแสดงรูปภาพ และความกลมกลืนของการออกแบบดีไซน์ ในบทเรียนนี้เราจะศึกษาเชิงลึกเกี่ยวกับการควบคุมสไตล์ขั้นสูงด้วยระบบจัดตำแหน่ง **absolute**, การโหลดแบบอักษรพิเศษ (Custom Fonts), การเรียกใช้งานชุดเวกเตอร์ไอคอน และการเพิ่มประสิทธิภาพการแสดงภาพด้วย **expo-image** รวมถึงระบบปรับสีตามธีมของตัวเครื่อง (Dark Mode)

7.1 ระบบพิกัดตำแหน่งสัมพัทธ์และสัมบูรณ์ (Relative and Absolute Positioning)

ในการจัดองค์ประกอบหน้าจอบน React Native ค่าเริ่มต้นของสไตล์ **position** จะเป็น **'relative'** (พิกัดตำแหน่งสัมพัทธ์) ซึ่งสอดคล้องกับการวางตัวตนต่อท้ายกันตามทิศทางของ Flexbox (React Native, 2024) แต่สำหรับการออกแบบ UI ยุคใหม่ เช่น การทำเครื่องหมายแจ้งเตือน (Notification Badge) ทับบนไอคอน หรือการวางปุ่มลอย (Floating Action Button) ค้างไว้ที่มุมขวาล่าง เราจำเป็นต้องเลือกใช้ **'absolute'** (พิกัดตำแหน่งสัมบูรณ์)

ข้อแตกต่างและการประยุกต์ใช้งาน

• **`relative` (Default):** คอมโพเนนต์จะจัดวางพื้นที่และขยับตำแหน่งโดยอ้างอิงจากตำแหน่งที่มันควรจะอยู่ปกติในระบบ Flow โดยที่ยังคงจองพื้นที่ของตัวเองไว้ ไม่ให้ชิ้นงานอื่นมาเบียดทับ

• **`absolute`:** คอมโพเนนต์จะหลุดออกจากระเบียบการจัดแถวของ Flexbox ทันที โดยพิกัดจะถูกระบุด้วยทิศทาง **top**, **bottom**, **left**, **right** สัมพันธ์กับพื้นที่ของ **"คอนเทนเนอร์แม่ตัวแรกสุดที่มีระดับชั้นอยู่เหนือมันและทำตัวเป็นจุดอ้างอิง"** (มักจะตั้งค่าแม่เป็น **position: 'relative'** หรือปล่อยไว้เป็นดีฟอลต์)

```
import { StyleSheet, View, Text } from 'react-native';
import { Ionicons } from '@expo/vector-icons';

export default function BadgeExample() {
  return (
    <View style={styles.container}>
      {/* 1. ปุ่มกระดิ่งแจ้งเตือน */}
      <View style={styles.iconButton}>
        <Ionicons name="notifications" size={32} color="#ffffff" />

        {/* 2. ตัวเลขสีแดงแจ้งเตือนวางทับซ้อนมุมขวาบน */}
```

```
        <View style={styles.badge}>
          <Text style={styles.badgeText}>3</Text>
        </View>
    </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#0f172a',
  },
  iconButton: {
    width: 60,
    height: 60,
    borderRadius: 30,
    backgroundColor: '#1e293b',
    justifyContent: 'center',
    alignItems: 'center',
    position: 'relative', // เป็นหลักแหล่งพิกัดอ้างอิงให้ลูกสะกดตำแหน่ง
    borderWidth: 1,
    borderColor: '#334155',
  },
  badge: {
    position: 'absolute', // หลุดออกจากโฟลว์เพื่อทับซ้อนอิสระ
    top: -4,              // ดันขึ้นไปด้านบน 4px เหนือขอบกระดิ่ง
    right: -4,            // ขยับเยื้องขวา 4px เลยขอบกระดิ่ง
    backgroundColor: '#ef4444', // สีแดงเตือนภัย
    minWidth: 20,
    height: 20,
    borderRadius: 10,
    justifyContent: 'center',
    alignItems: 'center',
    paddingHorizontal: 4,
  },
  badgeText: {
    color: '#ffffff',
    fontSize: 12,
    fontWeight: 'bold',
  },
});
```

7.2 การใช้งาน Custom Fonts ด้วย expo-font

ฟอนต์ดั้งเดิมของระบบปฏิบัติการ (System Fonts) มักจะขาดเอกลักษณ์เฉพาะตัวของแบรนด์ โครงการ Expo SDK 54 สนับสนุนการดาวน์โหลดชุดฟอนต์จากภายนอกมาติดตั้งใช้งานในระดับเนทีฟอย่างสะดวก ผ่านไลบรารี **expo-font** (Expo, 2024) และจัดการความสมบูรณ์ในการโหลดฟอนต์พร้อมกับระบบ Splash Screen เพื่อป้องกันปัญหาสัญลักษณ์เบี้ยวหรือกระตุก (Flash of Unstyled Text)

การดาวน์โหลดและติดตั้งคอมโพเนนต์

ขั้นตอนแรกในการทำงานคือการติดตั้งไลบรารีที่จำเป็น:

```
npx expo install expo-font expo-splash-screen
```

การเขียนโค้ดผูกฟอนต์และหน่วงเวลา Splash Screen

เราต้องสั่งหยุดไม่ให้ Splash Screen ปิดตัวเองก่อนที่ฟอนต์จะถูกโหลดเสร็จลงในหน่วยความจำ RAM ของเครื่อง โดยเรียกใช้ `SplashScreen.preventAutoHideAsync()` และสั่งซ่อนด้วย `SplashScreen.hideAsync()` หลังจากผลลัพธ์การโหลดพร้อมรัน:

```
import { useEffect } from 'react';
import { View, Text, StyleSheet } from 'react-native';
import { useFonts } from 'expo-font';
import * as SplashScreen from 'expo-splash-screen';

// 1. ตรึงภาพ Splash Screen ไว้ไม่ให้หน้าจอดับลงก่อนการโหลด
SplashScreen.preventAutoHideAsync();

export default function FontLoaderApp() {
 // 2. เรียกใช้ useFonts โหลดแบบอักษรพิเศษจากโฟลเดอร์ assets
  const [fontsLoaded, error] = useFonts({
    'Prompt-Regular': require('../../assets/fonts/Prompt-Regular.ttf'),
    'Prompt-Bold': require('../../assets/fonts/Prompt-Bold.ttf'),
  });

  useEffect(() => {
  // 3. เมื่อดาวน์โหลดเสร็จสมบูรณ์หรือมีข้อผิดพลาด ให้ทำการซ่อน Splash Screen
    if (fontsLoaded || error) {
      SplashScreen.hideAsync();
    }
  }, [fontsLoaded, error]);

 // 4. หากฟอนต์ยังไม่พร้อมและไม่พบข้อผิดพลาด ให้คืนค่าว่างเปล่า (null) เพื่อป้องกันการเรนเดอร์พัง
  if (!fontsLoaded && !error) {
    return null;
  }
```

```
  return (
    <View style={styles.container}>
      <Text style={styles.textRegular}>สวัสดีภาษาไทยด้วยแบบอักษร Prompt
(Regular)</Text>
      <Text style={styles.textBold}>หัวข้อสำคัญด้วย Prompt (Bold)</Text>
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 24,
    backgroundColor: '#0f172a',
  },
  textRegular: {
    fontFamily: 'Prompt-Regular', // อ้างอิงตรงตามชื่อคีย์ที่ลงทะเบียนใน useFonts
    fontSize: 18,
    color: '#cbd5e1',
    marginBottom: 12,
  },
  textBold: {
    fontFamily: 'Prompt-Bold',
    fontSize: 24,
    color: '#ffffff',
  },
});
```

7.3 การใช้งานระบบเวกเตอร์ไอคอนด้วย @expo/vector-icons

ไลบรารี **@expo/vector-icons** (Expo, 2024) ติดตั้งมาพร้อมใช้งานในแม่แบบโครงการของ Expo ช่วยลดภาระการโหลดไฟล์ PNG ของไอคอน ซึ่งทำให้ขนาดไฟล์แอปพลิเคชันใหญ่โดยไม่จำเป็น โดยชุดแพ็คเกจนี้จะรวมไอคอนเวกเตอร์ชื่อดังไว้เพียบ เช่น Ionicons, AntDesign, FontAwesome, MaterialIcons, Feather และ Octicons

เครื่องมือสำหรับค้นหาไอคอนอย่างเป็นทางการ

ผู้พัฒนาสามารถเลือกค้นหาชุดภาพและรายชื่อไอคอนที่ถูกต้องได้ผ่านเว็บไซต์ icons.expo.fyi ซึ่งจะบอกคำสั่ง Import และรายชื่อไอคอนทั้งหมด

ตัวอย่างการประยุกต์ใช้งาน

```
import { View, StyleSheet, TouchableOpacity, Text } from 'react-native';
import { Ionicons, Feather } from '@expo/vector-icons';
```

```
export default function MenuGrid() {
  return (
    <View style={styles.grid}>
      {/* การใช้งาน Ionicons สำหรับปุ่มตั้งค่า */}
      <TouchableOpacity style={styles.card}>
        <Ionicons name="settings-outline" size={28} color="#38bdf8" />
        <Text style={styles.label}>ตั้งค่าระบบ</Text>
      </TouchableOpacity>

      {/* การใช้งาน Feather สำหรับปุ่มแผนที่ */}
    <TouchableOpacity style={styles.card}>
        <Feather name="map-pin" size={28} color="#f43f5e" />
        <Text style={styles.label}>สถานที่</Text>
      </TouchableOpacity>
    </View>
 );
}

const styles = StyleSheet.create({
  grid: {
    flexDirection: 'row',
    justifyContent: 'space-around',
    padding: 20,
    backgroundColor: '#0f172a',
  },
  card: {
    backgroundColor: '#1e293b',
    padding: 16,
    borderRadius: 12,
    alignItems: 'center',
    width: '40%',
    borderWidth: 1,
    borderColor: '#334155',
  },
  label: {
    color: '#ffffff',
    marginTop: 8,
    fontSize: 14,
  },
});
```

7.4 การจัดการรูปภาพประสิทธิภาพสูงด้วย expo-image

คอมโพเนนต์ **Image** ดั้งเดิมของ React Native ประสบปัญหาร้ายแรงหลายประการเมื่อต้องทำงานกับภาพจากเน็ตเวิร์กจำนวนมาก เช่น การกระพริบตา (Flickering) เวลาอัปเดตข้อมูล, ไม่มีระบบเซฟเก็บความจำสำรองบนความจุฮาร์ดดิสก์แบบเนทีฟ (Native Caching) ทำให้แอปเปลืองอินเทอร์เน็ต และการเปิดโหลดภาพขนาดใหญ่ก่อให้เกิดอาการค้างชั่วขณะบนโทรศัพท์ Android ระดับล่าง

เพื่อขจัดขีดจำกัดดังกล่าว Expo จึงพัฒนาคอมโพเนนต์ `**expo-image**` (Expo, 2024) ขึ้นมาโดยขับเคลื่อนผ่านไลบรารีประสิทธิภาพสูงของภาษาเนทีฟอย่าง **SDWebImage (iOS)** และ **Glide (Android)**

ความสำคัญของ expo-image

• **ระบบ Caching ชั้นเลิศ:** บันทึกภาพลงบนไดรฟ์เครื่องอัตโนมัติ เปิดซ้ำไม่ต้องโหลดเน็ตอีกครั้ง

• **ความเร็วในการประมวลผลสูง:** ป้องกันไม่ให้แอปพลิเคชันกระตุกขณะผู้ใช้ทำการปัดเลื่อนลิสต์ที่มีภาพประกอบ

• **สอดคล้องกับมาตรฐานเว็บ:** การใช้งานฟังก์ชันขยายสไตล์สอดรับกับคุณสมบัติของ CSS `object-fit` (เช่นการใช้ค่า `contentFit`)

• **รองรับหลากหลายไฟล์สมัยใหม่:** สามารถรันภาพเคลื่อนไหวอย่าง WebP, APNG, GIF รวมถึงไฟล์ความหนาแน่นสูงอย่าง AVIF

```
npx expo install expo-image
```

7.5 การใช้งาน Blurhash Placeholder และ Transition

สองกลยุทธ์สำคัญที่จะช่วยเปลี่ยนแอปที่ดูธรรมดาให้เป็นแอปเกรดพรีเมียม คือการจัดการรอยต่อของเวลาที่รูปภาพกำลังโหลด:

1. ระบบจำลองภาพแบบเลือนลาง (Blurhash Placeholder)

Blurhash คือเทคโนโลยีการแปลงภาพถ่ายขนาดใหญ่ให้กลายเป็นรหัสข้อความสั้น ๆ (เช่น `L6PZUpUR00-y_4V[00aeY5g2o1of`) ซึ่งสามารถวาดแสดงผลขึ้นมาเป็นภาพไล่เฉดสีเบลอ ๆ ได้ในเศษเสี้ยววินาทีโดยไม่ต้องรอเน็ตเวิร์ก ช่วยลดความอึดอัดของผู้ใช้เมื่อเปรียบเทียบกับวงล้อโหลดข้อมูลที่ว่างเปล่า

2. เอฟเฟกต์ Fade-in Transition

เมื่อระบบดาวน์โหลดภาพไฟล์จริงจากเซิร์ฟเวอร์เสร็จสิ้นแล้ว expo-image จะทำการเปลี่ยนผ่านจากภาพเบลอตัวพรีวิวไปสู่รูปจริงอย่างราบรื่นด้วยเอฟเฟกต์อนิเมชันแบบสลัวทีละนิด ช่วยลดความตระหนกสายตาของผู้ใช้ (No Flickering Jump)

```
import { View, StyleSheet, Text } from 'react-native';
import { Image } from 'expo-image';

// ตัวอย่างสตริง Blurhash จำลองเฉดสีเพื่อรอโหลด
const sampleBlurhash = 'L6PZUpUR00-y_4V[00aeY5g2o1of';

export default function HighPerformanceImage() {
  return (
    <View style={styles.container}>
      <Text style={styles.header}>ตัวอย่างการแสดงผลรูปภาพผ่าน expo-image</Text>

      <Image
        style={styles.image}
        source="https://picsum.photos/id/237/800/600"
        placeholder={{ blurhash: sampleBlurhash }} // แสดงภาพพรีวิวเบลอตามรหัสชุดข้อความทันที
        contentFit="cover" // เทียบเท่า object-fit: cover ในระบบ CSS
        transition={600}    // เล่นเอฟเฟกต์เฟดภาพจริง 0.6 วินาทีเพื่อความนุ่มนวล
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#0f172a',
  },
  header: {
    color: '#94a3b8',
    marginBottom: 16,
    fontSize: 15,
  },
  image: {
    width: 300,
    height: 200,
    borderRadius: 16,
  },
});
```

7.6 การรองรับโหมดมืด (Dark Mode / Theme Support) ด้วย useColorScheme

ในโทรศัพท์มือถือยุคปัจจุบัน ระบบปฏิบัติการจะเปิดโอกาสให้ผู้ใช้อัปเกรดหน้าจอไปใช้โทนสีดำ (Dark Mode) เพื่อความปลอดภัยของสายตาและประหยัดพลังงาน แบตเตอรี่ สำหรับแอป React Native เราสามารถเขียนตรวจสอบสถานะสีเครื่องได้โดยตรงผ่าน **useColorScheme** Hook ของ **react-native**

การประกาศตัวแปรสีและผูกใช้ในสไตล์

เมื่อตรวจพบการปรับระบบ โค้ดหน้าจอจะเข้าข่ายและแสดงธีมที่สลับค่าโทนสีอย่างไดนามิก:

```
import { StyleSheet, Text, View, useColorScheme } from 'react-native';

export default function DynamicThemeScreen() {
 // 1. เรียกรับค่าสถานะโหมดระบบปฏิบัติการ ('light' หรือ 'dark')
  const colorScheme = useColorScheme();

 // 2. ตรวจเช็คเพื่อกำหนดธีมสีแบบ Dynamic
  const isDarkMode = colorScheme === 'dark';

  return (
  // 3. กำหนดสไตล์โดยสลับ Object สไตล์ตามสถานะ
    <View style={[styles.container, isDarkMode ? styles.darkContainer :
styles.lightContainer]}>
      <Text style={[styles.title, isDarkMode ? styles.darkText :
styles.lightText]}>
        {isDarkMode ? '🌙 ระบบของคุณเปิดโหมดมืดอยู่' : '🌞 ระบบของคุณเปิดโหมดสว่างอยู่'}
      </Text>
      <Text style={[styles.subtitle, isDarkMode ? styles.darkSub :
styles.lightSub]}>
    สไตล์และโทนสีของแอปตัวนี้จะขยับสับเปลี่ยนตามค่าหน้าจอหลักของสมาร์ทโฟน
      </Text>
    </View>
 );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 24,
  },
 // ธีมสีพื้นหลังสว่าง/มืด
  lightContainer: {
    backgroundColor: '#f8fafc',
```

```
  },
  darkContainer: {
    backgroundColor: '#0f172a',
  },
  title: {
    fontSize: 22,
    fontWeight: 'bold',
    marginBottom: 10,
    textAlign: 'center',
  },
  subtitle: {
    fontSize: 15,
    textAlign: 'center',
    lineHeight: 22,
  },
 // ธีมตัวหนังสือ
  lightText: {
    color: '#0f172a',
  },
  darkText: {
    color: '#ffffff',
  },
  lightSub: {
    color: '#64748b',
  },
  darkSub: {
    color: '#94a3b8',
  },
});
```

สรุปท้ายบทเรียน

ระบบพิกัดตำแหน่งสัมพัทธ์และสัมบูรณ์: การจัดวางอินเทอร์เฟซใน React Native ใช้สไตล์ position ในการกำหนดระบบพิกัด โดยมี relative เป็นค่าเริ่มต้นที่จัดเรียงวัตถุตามระบบ Flexbox โดยยังจองพื้นที่ไว้ และ absolute สำหรับยกวัตถุออกจากโฟลว์เพื่อกำหนดทิศทางผ่าน top, bottom, left และ right สัมพันธ์กับพื้นที่ของคอมโพเนนต์แม่ที่ตั้งค่าเป็นหลักแหล่งอ้างอิง ซึ่งช่วยให้เราสามารถจัดทำส่วนทับซ้อนหรือปุ่มลอยต่าง ๆ ได้ตามต้องการ

การใช้งาน Custom Fonts ด้วย expo-font: การดาวน์โหลดและแสดงผลแบบอักษรแบรนด์เฉพาะตัวกระทำได้ผ่านชุดไลบรารี expo-font ร่วมกับ expo-splash-screen โดยเราจะต้องทำโครงสร้างการหน่วงเวลาเพื่อสั่งให้ระบบคงภาพ Splash Screen ค้างไว้ก่อนในขณะที่ตัวฟังก์ชัน useFonts กำลังโหลดชุดอักษรพิเศษ เพื่อป้องกันการแสดงผลแบบอักษรผิดพลาดหรือเกิดการกระตุกเปลี่ยนแบบอักษรหลังจากเข้าหน้าแอปพลิเคชัน

การใช้งานระบบเวกเตอร์ไอคอนด้วย @expo/vector-icons: เพื่อประหยัดขนาดพื้นที่และความสามารถในการตอบสนองสเกลภาพ เวกเตอร์ไอคอนจากไลบรารี @expo/vector-icons (Expo, 2024) จึงเป็นทางเลือกหลักในการรวบรวมสัญลักษณ์สากลอย่าง Ionicons, Feather หรือ FontAwesome มาใช้งานในโปรเจกต์ ซึ่งสามารถนำรูปภาพที่ถูกต้องจากคู่มือค้นหารายชื่อไอคอน icons.expo.fyi มาใส่สีและกำหนดขนาดตามที่สไตล์ชีทกำหนดไว้ได้อย่างมีประสิทธิภาพ

การจัดการรูปภาพประสิทธิภาพสูงด้วย expo-image: คอมโพเนนต์ดั้งเดิมมีปัญหาด้านแคชข้อมูลภาพจากเน็ตเวิร์กและการกระตุกตอนปัดเลื่อนจอ ทำให้ Expo ได้พัฒนา expo-image ขึ้นมาทำงานทดแทนผ่านเทคโนโลยีฝั่งเนทีฟ (Glide และ SDWebImage) ซึ่งมอบฟีเจอร์การสร้างแคชจัดเก็บบนความจุไดรฟ์เครื่องอัตโนมัติ ทำให้ผู้พัฒนาสามารถจัดสรรการแสดงผลรูปภาพจำนวนมากได้อย่างรวดเร็วและประหยัดอินเทอร์เน็ตของผู้ใช้อย่างเด่นชัด

การใช้งาน Blurhash Placeholder และ Transition: การเสริมสร้างรอยต่อเวลาที่รูปภาพหลักกำลังอยู่ในกระบวนการดาวน์โหลดสามารถทำได้ผ่านสองกลยุทธ์หลัก คือการใช้รหัสข้อความจำลองเฉดสีขนาดเล็กอย่าง Blurhash แสดงผลขึ้นมาเป็นภาพเบลอตัวอย่างโดยไม่ต้องรอเน็ตเวิร์ก และระบบเล่นเอฟเฟกต์ Fade-in Transition ของ expo-image เพื่อทำการเปลี่ยนผ่านภาพจากแบบเบลอไปสู่ภาพจริงได้อย่างนุ่มนวลและลดอาการกระตุกทางสายตา

การรองรับโหมดมืดด้วย useColorScheme: โทรศัพท์มือถือยุคใหม่มักจะให้ผู้ใช้ปรับแต่งธีมสว่างหรือมืดตามใจชอบ ซึ่งเราสามารถดักฟังค่าระบบปฏิบัติการได้ผ่าน useColorScheme Hook ของ react-native เพื่อนำค่า light หรือ dark มาทำการสลับโทนสีของแอปพลิเคชันอย่างไดนามิกผ่านโครงสร้างออบเจกต์สไตล์ชีท ช่วยให้อินเทอร์เฟซมีความกลมกลืนตามความชอบของผู้ใช้งานเครื่องสมาร์ทโฟน

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้และความเข้าใจเกี่ยวกับระบบการควบคุมสไตล์ขั้นสูง (Absolute Positioning), การโหลดและใช้งานแบบอักษรพิเศษ (Custom Fonts), การประยุกต์ใช้เวกเตอร์ไอคอน (@expo/vector-icons), การเพิ่มประสิทธิภาพจัดแสดงรูปภาพ (expo-image) และการรองรับโหมดมืด (Dark Mode / useColorScheme) ใน React Native Expo SDK 54

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ในการกำหนดสไตล์ของ React Native ข้อใดอธิบายพฤติกรรมของ `position: 'absolute'` ได้ถูกต้องที่สุด?

ก. ชิ้นงานจะขยับเยื้องจากจุดเดิมตามแถว Flexbox และยังคงจองพื้นที่ความกว้างและความสูงไว้

ข. ชิ้นงานจะหลุดออกจากโฟลว์การจัดแถว Flexbox ของหน้าจอ โดยสามารถระบุตำแหน่งทับซ้อนได้อย่างอิสระเทียบกับจุดเริ่มต้นของคอนเทนเนอร์แม่ตัวแรกสุดที่มีโครงสร้างล้อมรอบมัน

ค. ชิ้นงานจะถูกลบออกจากสเตตและการแสดงผลบนหน้าจอทันที

ง. ชิ้นงานจะขยายขนาดกว้าง 100% เสมอโดยอ้างอิงจากขนาดพิกเซลความสูงของหน้าจอระบบปฏิบัติการ

ข้อ 2: พร็อพส์บอกทิศทางใดต่อไปนี้ ไม่นิยม นำมาใช้ร่วมกับคอมโพเนนต์ที่กำหนดสไตล์เป็น `position: 'absolute'`?

ก. `top`

ข. `right`

ค. `flexDirection`

ง. `bottom`

ข้อ 3: การประยุกต์ใช้ไลบรารี `expo-font` ร่วมกับ `expo-splash-screen` เพื่อจุดประสงค์ใดเป็นสำคัญ?

ก. เพื่อบีบอัดขนาดไฟล์รูปภาพ PNG ในแอปพลิเคชันให้เล็กลง

ข. เพื่อให้ Splash Screen แสดงผลค้างไว้ในขณะที่แอปกำลังดาวน์โหลดฟอนต์เข้ามาเก็บในหน่วยความจำชั่วคราว ป้องกันปัญหากรอบอักษรพังหรือกระตุก (Flash of Unstyled Text)

ค. เพื่อเปลี่ยนสีหน้าจอหลักของโทรศัพท์ให้กลายเป็นสีธีมเข้มโดยทันที

ง. เพื่อสร้างไฟล์ติดตั้ง APK สำหรับส่งขึ้น Google Play Store

ข้อ 4: จากโค้ดเรียกใช้งาน `const [fontsLoaded, error] = useFonts({...})` หากฟอนต์ดาวน์โหลดพังหรือระบุพาธไฟล์ผิดพลาด ตัวแปรใดที่จะช่วยดักจับและส่งต่อสัญญาณสีแดงแจ้งปัญหาให้เราซ่อน Splash Screen เพื่อป้องกันหน้าจอค้างตลอดกาล?

ก. `fontsLoaded`

ข. `error`

ค. `useFonts`

ง. `SplashScreen`

ข้อ 5: แหล่งสืบค้นคำสั่งนำเข้า (Import) และรายชื่อไอคอนเวกเตอร์อย่างเป็นทางการของ Expo ที่สามารถตรวจสอบรูปทรง ขนาด และสีก่อนเขียนโค้ดได้คือเว็บไซต์ใด?

ก. google.com

ข. npmjs.com

ค. icons.expo.fyi

ง. expo.dev/fonts

ข้อ 6: เหตุใดเราจึงควรเลือกใช้งาน `@expo/vector-icons` ในการออกแบบเมนูหรือปุ่มสัญลักษณ์ แทนการใช้รูปภาพไฟล์ PNG ขนาดเล็ก?

ก. ไอคอนแบบเวกเตอร์สามารถย่อขยายขนาดและเปลี่ยนเฉดสีได้อย่างคมชัดโดยภาพไม่แตก และช่วยประหยัดเนื้อที่ขนาดไฟล์แอปพลิเคชันได้ดีกว่า

ข. ไอคอนเวกเตอร์ช่วยเชื่อมต่อบลูทูธของระบบมือถือเข้าหาอุปกรณ์เสริมได้ไวกว่า

ค. รูปภาพแบบ PNG สามารถปรับเปลี่ยนทิศทางการหมุนได้ดีกว่าเวกเตอร์

ง. เพื่อป้องกันไม่ให้ผู้ใช้งานสามารถก๊อปปี้โลโก้ของแอปได้

ข้อ 7: ข้อใดกล่าวถึงประโยชน์และความจำเป็นของการเปลี่ยนมาใช้คอมโพเนนต์ `Image` จากไลบรารี `expo-image` แทนคอมโพเนนต์ `Image` ดั้งเดิมของ React Native ได้อย่างถูกต้อง?

ก. เพื่อให้แอปพลิเคชันรันความละเอียดรูปภาพแบบ 8K ได้ลื่นไหลบนโทรศัพท์รุ่นเก่า

ข. เพื่อความสามารถในการแคชรูปภาพระดับเนทีฟ (Native Caching) ทั้งความจำสำรองและฮาร์ดดิสก์, ลดการใช้แบนด์วิดท์ และทำให้ปัดเลื่อนรายการที่มีภาพยาว ๆ ได้ลื่นไหลขึ้น

ค. เพื่อให้สามารถแต่งรูปภาพ ฟิลเตอร์สี และตัดภาพได้จากฝั่งโค้ดโปรเจกต์

ง. เพื่อปิดกั้นการใช้รูปจาก URL ภายนอก และอนุญาตเฉพาะภาพในเครื่องเท่านั้น

ข้อ 8: การทำงานของ "Blurhash" ในความสามารถของ `expo-image` มีหน้าที่หลักอย่างไรในการเพิ่มความพึงพอใจให้ผู้ใช้ (User Experience)?

ก. ช่วยสลับรูปภาพหลักให้หมุนเป็นวงกลมเร่งความเร็วการประมวลผล

ข. แปลงภาพจริงให้เป็นรหัสข้อความสั้น ๆ เพื่อนำมารันเป็นภาพตัวอย่างไล่เฉดสีเบลอ ๆ ทันทีในระหว่างที่รอดึงไฟล์ภาพใหญ่จากเน็ตเวิร์ก ช่วยแก้ปัญหาหน้าจอว่างเปล่า

ค. บังคับใส่เอฟเฟกต์เบลอบนหน้าจอเมื่อผู้ใช้พยายามแคปหน้าจอแอป

ง. ทำหน้าที่ย่อขนาดไฟล์ภาพจริงให้เหลือความกว้างเพียง 10 พิกเซลเก็บลงเซิร์ฟเวอร์

ข้อ 9: พร็อพส์ (Props) ตัวใดของ `expo-image` ที่ใช้ในการกำหนดเอฟเฟกต์เฟดภาพจริงขึ้นมาอย่างราบรื่นหลังจากดาวน์โหลดเสร็จสิ้น เพื่อไม่ให้แอปเกิดการกระตุกเปลี่ยนผ่านภาพกะทันหัน?

ก. `transition`

ข. `contentFit`

ค. `placeholder`

ง. `cachePolicy`

ข้อ 10: หากต้องการรับทราบการเปลี่ยนสีธีมระบบเครื่องของระบบปฏิบัติการมือถือปลายทาง (Light/Dark Mode) แบบเรียลไทม์ ผู้พัฒนาต้องเรียกใช้งาน React Native Hook ตัวใด?

ก. `useFonts`

ข. `useColorScheme`

ค. `useRoute`

ง. `useNavigation`

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: เป็นพฤติกรรมของ `position: 'relative'` ที่ยังคงครอบจองพื้นที่เดิมไว้ใน Layout flow

• ข. ถูก: `position: 'absolute'` จะดึงตัวมันเองออกจากโฟลว์ปกติ ทำให้สไตล์ตัวถัดไปขยับขึ้นมาแทนที่ และใช้พิกัด `top/bottom/left/right` ระบุการทับซ้อนเหนือนิ้วและชิ้นงานอื่น ๆ โดยยึดตามพิกัดของ Container แม่ที่มีการระบุขอบเขต

• ค. และ ง. ผิด: ไม่ได้เกี่ยวกับสเตตและไม่ได้บังคับขยาย 100% ตลอดเวลาแต่อย่างใด

เฉลย ข้อ 2: ตอบ ค.

• เหตุผลประกอบ:

• ก. ข. และ ง. ผิด: `top`, `bottom` และ `right` (รวมถึง `left`) เป็นหัวใจสำคัญของ Absolute Positioning เพื่อระบุพิกัดช่องไฟจากขอบแม่

• ค. ถูก: `flexDirection` เป็นพร็อพส์หลักของระบบจัดเรียง Flexbox Layout (เช่นจัดแนวแถวหรือแนวคอลัมน์) ซึ่งปกติคอมโพเนนต์ที่เป็น `absolute` จะหลุดออกจากระบบจัดเรียงนี้ จึงไม่นำมาประกาศคุมทิศทางตนเองโดยตรง (แต่มักใช้คุม Container แม่ หรือ Container ลูกภายใน absolute view อีกที)

เฉลย ข้อ 3: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: ไม่สามารถลดขนาดไฟล์ PNG ในโครงสร้างโฟลเดอร์ได้โดยตรง

• **ข. ถูก:** `expo-splash-screen` ช่วยบล็อกไม่ให้แอปปิดหน้าจอโลโก้ของระบบตั้งต้นก่อนที่สเตตัสการดาวน์โหลด Custom Fonts ของ `expo-font` จะเสร็จสิ้นลง ซึ่งหากโหลดเสร็จสมบูรณ์จึงปล่อยให้เปิดหน้าแอปจริง จะได้ฟอนต์ที่แสดงผลสวยงามทันทีไม่มีหน้ากระตุกอักษรเปลี่ยนรูป

• **ค. และ ง. ผิด:** ธีมเครื่องและฟังก์ชันการบิลด์ APK เป็นบทบาทหน้าที่ของตัวแปรและเอนจิ้นภายนอกตัวอื่น ๆ

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `fontsLoaded` จะเปลี่ยนเป็น `true` เมื่อฟอนต์ถูกโหลดผ่านเกณฑ์สำเร็จเท่านั้น

• **ข. ถูก:** ตัวแปร `error` จากอาร์เรย์ผลลัพธ์ของ `useFonts` จะเก็บสถานะข้อผิดพลาดเมื่อเกิดปัญหาโหลดไฟล์ฟอนต์ไม่ขึ้น ซึ่งเราต้องเขียนดักเช็ค `if (fontsLoaded || error) { SplashScreen.hideAsync(); }` เพื่อให้แอปข้ามไปเรนเดอร์หน้าจอปกติ ไม่ติดค้างอยู่ที่หน้า Splash Screen

• **ค. และ ง. ผิด:** `useFonts` เป็นเพียงฟังก์ชัน Hook และ `SplashScreen` เป็นอ็อบเจกต์คำสั่งปิดหน้ากาก ไม่ใช่ตัวแปรดักความผิดพลาด

เฉลย ข้อ 5: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ข. และ ง. ผิด:** เป็นเว็บไซต์ค้นหาทั่วไป และไม่ใช่หน้าต่างเอกสารแสดงเวกเตอร์ไอคอนอย่างเจาะจง

• **ค. ถูก:** icons.expo.fyi คือสารบัญไอคอนอย่างเป็นทางการของ Expo ที่ช่วยให้เราค้นหาชื่อและคีย์เวิร์ดของชุดไอคอนต่าง ๆ จากคลังเนทีฟได้แม่นยำ

เฉลย ข้อ 6: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** เวกเตอร์ไอคอนถูกเก็บข้อมูลเป็นพิกัดทางคณิตศาสตร์ (Vector SVG/Font) ทำให้ย่อขยายขนาดได้ไม่จำกัดโดยไม่แตกพร่ามัว สามารถปรับแต่งสีสันได้เสรีผ่านโค้ดสไตล์ และมีขนาดข้อมูลที่เล็กกว่าไฟล์ภาพบิตแมป PNG อย่างมหาศาล

• **ข. ค. และ ง. ผิด:** ไม่เกี่ยวข้องกับสัญญาณการจับคู่บลูทูธ และการลากหมุนเวกเตอร์ทำได้ดีกว่า PNG รวมถึงไม่ใช่เครื่องมือป้องกันการก๊อปปี้สัญลักษณ์

เฉลย ข้อ 7: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** ไม่สามารถอัพสเกลคุณภาพหน้าจอเครื่อง หรือใช้ตัดแต่งแก้ไขภาพระดับโฟโต้ชอปได้ และรองรับรีโมท URL ปกติ

• **ข. ถูก:** `expo-image` พัฒนาขึ้นเพื่อแก้ปัญหาด้านประสิทธิภาพโดยใช้ Engine native ระดับสูง มีประสิทธิภาพสูงในการสร้าง Cache เก็บไฟล์ภาพลงฮาร์ดดิสก์เครื่องอัตโนมัติ ช่วยลดปัญหาความล่าช้าและการกินแรมตอนผู้ใช้เลื่อนดูหน้าต่างข้อมูลยาว ๆ

เฉลย ข้อ 8: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** ไม่เกี่ยวข้องกับกลไกความปลอดภัยการก๊อปปี้หน้าจอ หรือการลดความกว้างข้อมูลจริงบนเครื่องแม่ข่าย

• **ข. ถูก:** `Blurhash` คือรูปแบบการคำนวณที่แปลงภาพความละเอียดสูงให้เหลือสตริงอักษรขนาดสั้น เพื่อสร้างแถบสีพรีวิวเบลอ ๆ แสดงขึ้นมาแทนที่กล่องภาพเปล่าระหว่างโหลด ทำให้ผู้ใช้รู้สึกว่าแอปตอบสนองได้อย่างรวดเร็ว

เฉลย ข้อ 9: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** พร็อพส์ `transition` ของ `expo-image` รับค่าเวลาเป็นมิลลิวินาที (เช่น 600 = 0.6 วินาที) สำหรับกำหนดเวลาในการเล่นอนิเมชัน Fade-in เมื่อโหลดรูปจริงมาแสดงผลต่อจาก placeholder

• **ข. ผิด:** `contentFit` ใช้ปรับระดับสเกลภาพให้พอดีกล่อง (เช่น cover, contain)

• **ค. ผิด:** `placeholder` ใช้รับข้อมูลจำลองเพื่อรอโหลดภาพจริง

• **ง. ผิด:** `cachePolicy` ใช้จัดการนโยบายการจัดเก็บความจำสำรอง

เฉลย ข้อ 10: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** เป็นตัวจัดการฟอนต์, จัดระบบส้นทางเราเตอร์ และการนำทางตามลำดับ

• **ข. ถูก:** `useColorScheme` Hook ดึงค่าสภาพแวดล้อมสว่าง/มืด (`'light'` หรือ `'dark'`) ของระบบมือถือผู้ใช้งานกลับมาผูกใน Component เพื่อสลับสไตล์ชีทตามระบบการใช้งานในโหมดมืดได้ทันที

ปฏิบัติการท้ายบทเรียน: การสร้างการ์ดโปรไฟล์พรีเมียมรองรับระบบสลับธีม

ปฏิบัติการนี้จะฝึกทักษะการประยุกต์ใช้งานสไตล์ระดับสูงในระบบ React Native โดยผู้เรียนจะได้ออกแบบแอปพลิเคชันหน้า **แดชบอร์ดการ์ดโปรไฟล์ระดับพรีเมียม (Premium**

Profile Card Dashboard) ที่โหลดฟอนต์พิเศษมาใช้งาน ตกแต่งจุดทับซ้อนไอคอนแบบสไตล์สัมบูรณ์ (`absolute`) โหลดรูปภาพโปรไฟล์ผ่านไลบรารี `expo-image` พร้อมรองรับการแสดงเฉดสีพรีวิวด้วยรหัส Blurhash และการรองรับโหมดมืด (Dark Mode) ตามเครื่องปลายทาง

1. วัตถุประสงค์การเรียนรู้

1. สามารถประยุกต์ใช้พิกัดตำแหน่งสัมบูรณ์ `position: 'absolute'` ในการวางเครื่องหมายออนไลน์ (Online Status Badge) บนรูปภาพโปรไฟล์

2. สามารถติดตั้ง โหลด และใช้งานแบบอักษรพิเศษ (Custom Fonts) ผ่านระบบของ `expo-font` ร่วมกับการใช้ `expo-splash-screen`

3. สามารถดึงและจัดสไตล์สัญลักษณ์เวกเตอร์ไอคอนด้วยไลบรารี `@expo/vector-icons`

4. สามารถเพิ่มประสิทธิภาพและความลื่นไหลการโหลดภาพโปรไฟล์ด้วย `expo-image` พร้อมกำหนดค่า `placeholder` (Blurhash) และการเฟดภาพด้วย `transition`

5. สามารถดักเช็คและประยุกต์ใช้โทนสีกราฟิกตามธีมสว่างหรือธีมมืด (Dark Mode) ด้วย `useColorScheme` Hook

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องสร้างโครงสร้างคอมโพเนนต์การ์ดโปรไฟล์ส่วนตัว (Profile Dashboard Screen) ที่มีเงื่อนไขดังต่อไปนี้:

1. **การโหลดทรัพยากรระบบ (Assets Loading):**

• โหลดฟอนต์จำนวน 2 รูปแบบ ได้แก่ `Prompt-Regular` (สำหรับเนื้อหาทั่วไป) และ `Prompt-Bold` (สำหรับชื่อโปรไฟล์และหัวข้อสำคัญ)

• แสดงหน้า Splash Screen ค้างไว้ และสั่งเปิดเผยหน้าแอปเมื่อโหลดฟอนต์เสร็จเท่านั้น หากเกิดข้อผิดพลาดในการโหลดไฟล์ฟอนต์ ต้องยกเลิกหน้า Splash เพื่อป้องกันหน้าจอค้างตลอดกาล

2. **การจัดอินเทอร์เฟซโปรไฟล์พรีเมียม (Premium Interface Layout):**

• **กรอบภาพโปรไฟล์ (Avatar Container):** รันรูปวงกลมด้วย `expo-image` ดึงภาพจากลิงก์เน็ตเวิร์ก โดยตั้งค่า `placeholder` เป็นรหัส Blurhash และใช้ความเร็ว `transition` เท่ากับ 500 มิลลิวินาที

• **เครื่องหมายบอกสถานะการใช้งาน (Status Badge):** วางวงกลมสีเขียวนีออน (`#22c55e`) ขนาดเล็กระบุ "Online" ทับซ้อนลงบนมุมขวาล่างของรูปภาพวงกลมโปรไฟล์หลักด้วย `position: 'absolute'`

• **ข้อมูลส่วนตัว:** แสดงชื่อจริง ยศตำแหน่งงาน และประวัติสั้น ๆ โดยเรียกใช้ตระกูลอักษร `Prompt-Bold` และ `Prompt-Regular`

3. **ตารางชุดปุ่มคำสั่งด่วน (Action Grid):**

• แสดงชุดปุ่มแบบการ์ดขนาดเล็ก 3 ช่อง ได้แก่ "ส่งอีเมล", "ส่งข้อความ" และ "แชร์ข้อมูล"

• ในการ์ดย่อยแต่ละตัวให้เรียกสัญลักษณ์เวกเตอร์มาจาก `@expo/vector-icons` เช่น `mail`, `message-square`, `share-2`

4. **การสลับสไตล์ตามระบบเครื่อง (Theme Support):**

• ดึงสถานะธีมด้วย `useColorScheme()`

• หากระบบปฏิบัติการเป็นโหมดสว่าง (Light Theme) ให้ใช้สีพื้นหลังเป็น `#f8fafc` ตัวการ์ดสี `#ffffff` ตัวหนังสือสีเข้ม `#0f172a`

• หากระบบปฏิบัติการเป็นโหมดมืด (Dark Theme) ให้ใช้สีพื้นหลังเป็น `#0f172a` ตัวการ์ดสี `#1e293b` ตัวหนังสือสีขาว `#ffffff` และขอบสี `#334155`

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งไลบรารีและจัดสรรทรัพยากรฟอนต์

เรียกใช้คำสั่งเพื่อติดตั้งไลบรารีหลักทั้งหมดในโฟลเดอร์โปรเจกต์:

```
npx expo install expo-font expo-splash-screen expo-image
```

หมายเหตุ: ตรวจสอบให้แน่ใจว่าได้จัดเตรียมไฟล์ฟอนต์ `Prompt-Regular.ttf` และ `Prompt-Bold.ttf` บันทึกเก็บไว้ที่ไดเรกทอรี `assets/fonts/` ของแอปพลิเคชัน

ขั้นตอนที่ 2: โครงสร้างนำเข้าโมดูลและเริ่มหน่วงเวลา Splash Screen

เขียนเริ่มการโหลดฟอนต์ในหน้าแรกสุด:

```
import { useEffect } from 'react';
import { View, Text, StyleSheet, useColorScheme, Pressable } from 'react-native';
import { useFonts } from 'expo-font';
import * as SplashScreen from 'expo-splash-screen';
import { Image } from 'expo-image';
import { Feather } from '@expo/vector-icons';

// ตรึงหน้ารูปภาพโลโก้ไว้ก่อนรอข้อมูลแบบอักษรเสร็จสิ้น
SplashScreen.preventAutoHideAsync();
```

ขั้นตอนที่ 3: ออกแบบสเตตัส Badge ลอยสัมบูรณ์ทับบน Avatar

กำหนดรูปสไตล์รูปภาพให้มีกรอบความกว้างและสูงเป็นวงกลม และใช้ absolute ปักหมุดลูกด้านล่าง:

```
// กล่องครอบรูปวงกลมมีพิกัดเป็น relative
<View style={styles.avatarWrapper}>
  <Image
    source="https://images.unsplash.com/photo-1534528741775-53994a69daeb?w=400"
    placeholder={{ blurhash: 'LKN]~^%hS$of9G-pB%-;U_RPn%s:' }}
    style={styles.avatar}
    transition={500}
  />
  {/* จุดวงกลมออนไลน์วางปักหมุดไว้ที่มุมขวาล่าง */}
  <View style={styles.onlineBadge} />
</View>
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถนำโค้ดเฉลยเต็มระบบชุดนี้ไปบันทึกทับในไฟล์หลักของแอปพลิเคชันเพื่อใช้ทดสอบ โดยสลับการตั้งค่าโหมดหน้าจอมือถือจำลอง (Simulator) ไปมาระหว่าง Light และ Dark Mode เพื่อตรวจผลลัพธ์:

```
import { useEffect } from 'react';
import { StyleSheet, Text, View, useColorScheme, Pressable, ScrollView,
SafeAreaView } from 'react-native';
import { useFonts } from 'expo-font';
import * as SplashScreen from 'expo-splash-screen';
import { Image } from 'expo-image';
import { Feather, Ionicons } from '@expo/vector-icons';

// 1. ตรึงหน้า Splash Screen ไม่ให้หายไปกะทันหันก่อนดาวน์โหลดฟอนต์เสร็จ
SplashScreen.preventAutoHideAsync();

export default function App() {
 // 2. เรียกใช้งานระบบโหลดฟอนต์ไทยยอดนิยม (Prompt)
  const [fontsLoaded, error] = useFonts({
    'Prompt-Regular': require('../../assets/fonts/Prompt-Regular.ttf'),
    'Prompt-Bold': require('../../assets/fonts/Prompt-Bold.ttf'),
  });

  useEffect(() => {
  // 3. ทันทีที่โหลดฟอนต์พร้อม (หรือตรวจพบ Error) ให้ทำการซ่อน Splash Screen
    if (fontsLoaded || error) {
      SplashScreen.hideAsync();
    }
  }, [fontsLoaded, error]);
```

```
  // 4. หากฟอนต์ยังเรนเดอร์ไม่เสร็จสิ้น ให้รอชั่วขณะ
  if (!fontsLoaded && !error) {
    return null;
  }

  // 5. ดักจับเฉดสีธีมเครื่องปลายทางด้วย Hook
  const colorScheme = useColorScheme();
  const isDark = colorScheme === 'dark';

  // 6. กำหนดชุดคู่สีให้ปรับเปลี่ยนตามสเตตัสเครื่อง
  const theme = {
    background: isDark ? '#0f172a' : '#f8fafc',
    cardBackground: isDark ? '#1e293b' : '#ffffff',
    textPrimary: isDark ? '#ffffff' : '#0f172a',
    textSecondary: isDark ? '#94a3b8' : '#64748b',
    borderColor: isDark ? '#334155' : '#e2e8f0',
    primaryColor: '#38bdf8',
  };

  return (
    <SafeAreaView style={[styles.safeArea, { backgroundColor:
theme.background }]}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>
        {/* หัวข้อด้านบนสุด */}
        <View style={styles.header}>
          <Text style={[styles.headerTitle, { color: theme.textPrimary
}]}>แดชบอร์ดส่วนตัว</Text>
          <Text style={[styles.headerSubtitle, { color:
theme.textSecondary }]}>Premium Profile Card</Text>
        </View>

        {/* การ์ดโปรไฟล์หลัก */}
        <View style={[styles.profileCard, { backgroundColor:
theme.cardBackground, borderColor: theme.borderColor }]}>

          {/* ส่วนกรอบตกแต่งรูปวงกลมด้วยสไตล์สัมบูรณ์ */}
          <View style={styles.avatarContainer}>
            <View style={styles.avatarWrapper}>
              <Image
           source="https://images.unsplash.com/photo-1494790108377-
be9c29b29330?w=400"
                placeholder={{ blurhash: 'LKN]~^%hS$of9G-pB%-;U_RPn%s:' }}
                style={styles.avatar}
                contentFit="cover"
                transition={500}
       />
```

```
              {/* สไตล์ Absolute วางทับมุมโปรไฟล์ด้านล่างขวา */}
              <View style={[styles.statusBadge, { borderColor:
theme.cardBackground }]} />
            </View>
          </View>

          {/* รายละเอียดข้อความจำลองโปรไฟล์ */}
          <Text style={[styles.profileName, { color: theme.textPrimary
}]}>ชนิดาภา รัตนพิมล</Text>
          <Text style={[styles.profileRole, { color: theme.primaryColor
}]}>Lead UX/UI Designer</Text>
          <Text style={[styles.profileBio, { color: theme.textSecondary
}]}>
      ชื่นชอบการออกแบบโมบายแอปพลิเคชันระดับพรีเมียม สนุกกับการพัฒนาอินเทอร์เฟซด้วยสไตล์อนิเมชันลื่นไหลและดีไซน์
โหมดมืดแสนดึงดูดสายตา
          </Text>

          {/* แถบเส้นขีดคั่นกลางของการ์ด */}
          <View style={[styles.divider, { backgroundColor:
theme.borderColor }]} />

          {/* สถิติตัวเลขย่อ */}
          <View style={styles.statsRow}>
            <View style={styles.statCol}>
              <Text style={[styles.statNumber, { color: theme.textPrimary
}]}>48</Text>
              <Text style={[styles.statLabel, { color: theme.textSecondary
}]}>โครงการ</Text>
            </View>
            <View style={styles.statCol}>
              <Text style={[styles.statNumber, { color: theme.textPrimary
}]}>12.4K</Text>
              <Text style={[styles.statLabel, { color: theme.textSecondary
}]}>ผู้ติดตาม</Text>
            </View>
            <View style={styles.statCol}>
              <Text style={[styles.statNumber, { color: theme.textPrimary
}]}>150</Text>
              <Text style={[styles.statLabel, { color: theme.textSecondary
}]}>รีวิวดีเยี่ยม</Text>
            </View>
          </View>
        </View>

        {/* เมนูการ์ดคำสั่งปุ่มสัมผัส (Action Grid) */}
        <View style={styles.actionGrid}>
          <Pressable style={[styles.actionCard, { backgroundColor:
theme.cardBackground, borderColor: theme.borderColor }]}>
            <View style={[styles.iconCircle, { backgroundColor: '#e0f2fe'
```

```
}]}>
              <Feather name="mail" size={20} color="#0284c7" />
            </View>
            <Text style={[styles.actionLabel, { color: theme.textPrimary
}]}>ส่งอีเมล</Text>
          </Pressable>

          <Pressable style={[styles.actionCard, { backgroundColor:
theme.cardBackground, borderColor: theme.borderColor }]}>
            <View style={[styles.iconCircle, { backgroundColor: '#f0fdf4'
}]}>
              <Ionicons name="chatbubble-ellipses-outline" size={20}
color="#16a34a" />
            </View>
            <Text style={[styles.actionLabel, { color: theme.textPrimary
}]}>ส่งข้อความ</Text>
          </Pressable>

          <Pressable style={[styles.actionCard, { backgroundColor:
theme.cardBackground, borderColor: theme.borderColor }]}>
            <View style={[styles.iconCircle, { backgroundColor: '#fdf2f8'
}]}>
              <Feather name="share-2" size={20} color="#db2777" />
            </View>
            <Text style={[styles.actionLabel, { color: theme.textPrimary
}]}>แชร์ข้อมูล</Text>
          </Pressable>
        </View>
      </ScrollView>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
  },
  scrollContainer: {
    padding: 24,
  },
  header: {
    marginBottom: 24,
    alignItems: 'center',
  },
  headerTitle: {
    fontSize: 24,
    fontFamily: 'Prompt-Bold',
  },
  headerSubtitle: {
    fontSize: 14,
```

```
    fontFamily: 'Prompt-Regular',
    marginTop: 4,
  },
  profileCard: {
    borderRadius: 24,
    padding: 24,
    alignItems: 'center',
    borderWidth: 1,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.1,
    shadowRadius: 10,
    elevation: 5,
  },
  avatarContainer: {
    marginBottom: 16,
  },
  avatarWrapper: {
    position: 'relative', // วางเพื่อรองรับพิกัด Absolute ของ Badge ลูก
  },
  avatar: {
    width: 100,
    height: 100,
    borderRadius: 50,
    backgroundColor: '#cbd5e1',
  },
  statusBadge: {
    position: 'absolute', // หลุดออกจากโฟลว์เพื่อทับซ้อนมุมโปรไฟล์
    bottom: 2,
    right: 2,
    width: 22,
    height: 22,
    borderRadius: 11,
    backgroundColor: '#22c55e', // สีเขียวนีออนบอกสถานะ Online
    borderWidth: 3,             // เติมขอบตัดเพื่อป้องกันการกลืนสี
  },
  profileName: {
    fontSize: 20,
    fontFamily: 'Prompt-Bold',
    marginBottom: 4,
  },
  profileRole: {
  fontSize: 14,
    fontFamily: 'Prompt-Bold',
    marginBottom: 16,
  },
  profileBio: {
```

```
    fontSize: 14,
    fontFamily: 'Prompt-Regular',
    textAlign: 'center',
    lineHeight: 22,
    marginBottom: 20,
  },
  divider: {
    height: 1,
    width: '100%',
    marginBottom: 20,
  },
  statsRow: {
    flexDirection: 'row',
    justifyContent: 'space-around',
    width: '100%',
  },
  statCol: {
    alignItems: 'center',
  },
  statNumber: {
    fontSize: 18,
    fontFamily: 'Prompt-Bold',
  },
  statLabel: {
    fontSize: 12,
    fontFamily: 'Prompt-Regular',
    marginTop: 2,
  },
  actionGrid: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginTop: 20,
  },
  actionCard: {
    flex: 1,
    marginHorizontal: 4,
    borderRadius: 16,
    paddingVertical: 16,
    alignItems: 'center',
    borderWidth: 1,
  },
  iconCircle: {
    width: 44,
    height: 44,
    borderRadius: 22,
    justifyContent: 'center',
    alignItems: 'center',
    marginBottom: 8,
  },
```

```
  actionLabel: {
    fontSize: 13,
    fontFamily: 'Prompt-Regular',
  },
});
```

5. การวิเคราะห์สรุปผลการประเมินปฏิบัติการ

เมื่อรันโครงการสำเร็จ ให้ผู้เรียนประเมินตรวจสอบเกณฑ์การตรวจวัดดังนี้:

1. ฟอนต์ของตัวอักษรทั้งหมดในแอปพลิเคชันต้องใช้แบบอักษร **Prompt** และ Splash Screen ต้องซ่อนอย่างสมบูรณ์โดยที่ไม่มีการแจ้งพังหรือค้างหน้าจอขาว

2. รูปภาพแสดงผลโปรไฟล์คมชัด ลื่นไหลไม่กะพริบตา และเล่นเอฟเฟกต์เฟดตอนเริ่มหน้าจอ

3. เมื่อเข้าที่ส่วนตั้งค่าเปลี่ยนธีมของเครื่องให้เป็น Dark Mode องค์ประกอบหน้าจอทั้งหมดต้องสับสีดำสลัวโดยอัตโนมัติ

เอกสารอ้างอิง

Expo. (2024). *Expo Image*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/image/

Expo. (2024). *Icons*. จาก https://docs.expo.dev/versions/v54.0.0/guides/icons/

Expo. (2024). *Use custom fonts*. จาก https://docs.expo.dev/versions/v54.0.0/develop/user-interface/fonts/

React Native. (2024). *Style*. จาก https://reactnative.dev/docs/style

แผนบริหารการสอนประจำบทที่ 8

หัวข้อเนื้อหาประจำบท

8.1 ความเข้าใจและการติดตั้ง AsyncStorage

8.2 การทำงานพื้นฐานกับ AsyncStorage (CRUD Operations)

8.3 การบันทึกและดึงข้อมูลออบเจกต์ซับซ้อน (JSON Serialization)

8.4 การดึงข้อมูลจาก API ด้วย Fetch (HTTP GET)

8.5 การส่งข้อมูลขึ้นเซิร์ฟเวอร์ด้วย Fetch (HTTP POST)

8.6 ระบบจัดการสถานะ Loading และการดักจับข้อผิดพลาด (Error Handling)

8.7 การประยุกต์ใช้ในการทำระบบจำเซสชันการล็อกอิน (Auth Token Caching)

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถอธิบายหลักการบันทึกข้อมูลถาวรบนอุปกรณ์ด้วย AsyncStorage และเขียนโค้ดบันทึก-อ่านประเภทออบเจกต์ได้ถูกต้อง
2. ผู้เรียนสามารถใช้คำสั่ง Fetch API ในการยิง HTTP Request ทั้งรูปแบบ GET และ POST ได้สำเร็จ
3. ผู้เรียนสามารถออกแบบและเขียนสเตตคุมการแสดงผลหน้าจอระหว่างรอรับสายข้อมูล (Loading/ActivityIndicator) ได้ถูกต้อง
4. ผู้เรียนสามารถดักจับข้อผิดพลาดกรณีเครือข่ายล้มเหลว (Try-Catch Error handling) และเขียนกล่องป๊อปอัพแจ้งเตือนผู้ใช้งานได้อย่างเหมาะสม
5. ผู้เรียนสามารถเขียนจำลองจำการจำเซสชันบันทึกโทเค็นเมื่อล็อกอิน และลบเซสชันเมื่อล็อกเอาต์ออกจากแอปได้ถูกต้อง

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

29. **การบรรยายเปรียบเทียบกลไกจัดเก็บข้อมูลแบบถาวร (Storage & Client-Server Architecture Lecture):** บรรยายการเก็บข้อมูลแบบ State (หายไปเมื่อปิดแอป) เทียบกับแบบ AsyncStorage (คงอยู่แม้เปิดแอปใหม่) และอธิบาย

สเต็ปการติดต่อหลังบ้านผ่าน REST API

30. **การทำ Live Coding Demonstration หัวข้อ Login Session Cache and Posts Submission:** ผู้สอนสาธิตการทำหน้าจอล็อกอินจำลองที่เก็บข้อมูลคีย์โทเค็นลงในหน่วยความจำเครื่อง และเขียนหน้าเขียนบอร์ดกระทู้ส่งข้อมูลใหม่ขึ้นคลาวด์ผ่าน HTTP POST
31. **กิจกรรมแฮกเกอร์สายกู้ข้อมูล (Storage Recovery and Validation Challenge):** ให้ผู้เรียนทดลองดึงข้อมูลประวัติเดิมมาแสดงผลตอนเริ่มต้นเปิดแอปใหม่ โดยทำการสลักแปลงกลับจาก JSON string พร้อมดักจับเงื่อนไขข้อมูลว่างเปล่า
32. **กิจกรรมประลองระบบทนทานเครือข่าย (Offline & Error Boundaries Simulation):** ให้กลุ่มผู้เรียนช่วยกันแก้ไขโค้ดแอปให้ทำงานได้โดยไม่พังเมื่ออินเทอร์เน็ตหลุดกระทันหัน โดยต้องแสดงข้อความเตือนที่เป็นมิตรพร้อมปุ่ม "ลองใหม่อีกครั้ง (Retry)"

สื่อการเรียนการสอน

28. เอกสารสไลด์เรื่อง "Persistent Local Storage and Remote API Integration in Expo"
29. คลังข้อมูลทดสอบออนไลน์แบบเปิดสาธารณะ (JSONPlaceholder API)
30. รหัสตัวอย่างการดีไซน์ Wrapper Helper สำหรับอ่านและบันทึก AsyncStorage ปลอดภัยแบบเรียลไทม์
31. สภาพแวดล้อมทดสอบเครือข่ายสำหรับจำลองสัญญาณเน็ตขัดข้องบนอุปกรณ์ Simulator

การวัดผลและการประเมินผล

1. การวัดผล

1.1 ตรวจสอบคะแนนสอบแบบทดสอบปรนัยท้ายบทเรียน 10 ข้อ

1.2 ตรวจสอบโครงสร้างความถูกต้องของการเรียกใช้บล็อก異步 (async/await) ในการควบคุมอินเทอร์เฟซ AsyncStorage

1.3 สังเกตการแสดงโครงสร้างสไตล์ตัวหมุนโหลดข้อมูล (Spinner) และการใช้คำสั่งจัดการ JSON ในแอปพลิเคชัน

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถสร้างหน้าแอปบันทึกบันทึกโพสต์และดึง API จากเซิร์ฟเวอร์มาแคชเก็บในโทรศัพท์ได้โดยรันสำเร็จไม่มี Error ผ่านเกณฑ์ (ผ่าน/ไม่ผ่าน)

2.3 สังเกตพฤติกรรมความอดทนในการทดสอบระบบความเสถียรของข้อมูลยามสัญญาณขาดหาย และความใส่ใจในการประมวลผลเซสชันล็อกอิน

บทที่ 8

การเก็บข้อมูลในเครื่องและการเชื่อมต่อเครือข่าย API

ในบทเรียนนี้ เราจะเจาะลึกระบบการจัดการข้อมูลที่มีความมั่นคงถาวรขึ้นไปอีกขั้น โดยเรียนรู้การเขียนบันทึกข้อมูลและเซสชันไว้ในพื้นที่หน่วยความจำของมือถือด้วยคอมโพเนนต์ AsyncStorage รวมถึงสถาปัตยกรรมการติดต่อฐานข้อมูลกับเครื่องแม่ข่ายเซิร์ฟเวอร์ภายนอกผ่านเครือข่ายอินเทอร์เน็ตด้วยคำสั่ง Fetch API ทั้งรูปแบบการดึงข้อมูลและการส่งโพสต์ข้อมูลใหม่ พร้อมแนวทางรับมือกรณีสัญญาณอินเทอร์เน็ตล่ม

8.1 ความเข้าใจและการติดตั้ง AsyncStorage

แอปพลิเคชันโดยทั่วไป หากเก็บตัวแปรข้อมูลไว้บนระดับสถานะ (State) ของ React ข้อมูลเหล่านั้นจะสูญหายทันทีที่ผู้ใช้ปิดแอป หรือเครื่องโทรศัพท์แบตเตอรี่หมด เพื่อรักษาสภาพความคงอยู่ของข้อมูล (Data Persistence) เช่น การจดจำชื่อโปรไฟล์ หรือโทเค็นการล็อกอิน (Auth Token) เราจะใช้หน่วยเก็บข้อมูลถาวรประเภทคีย์-ค่า (Key-Value Storage) ที่เรียกว่า **AsyncStorage** (React Native Async Storage, 2024)

คุณสมบัติหลักของ AsyncStorage:

• **Asynchronous (ทำงานแบบไม่ประสานเวลา):** ทุกคำสั่งการอ่าน เขียน หรือลบข้อมูลจะทำงานเบื้องหลังโดยไม่บล็อกการทริกเกอร์แอนิเมชันหรือการวาดหน้าจอ (คืนค่าเป็น Promise)

• **Persistent (คงอยู่ตลอดไป):** ข้อมูลจะถูกจัดเก็บถาวรในตัวเครื่องจนกว่าแอปพลิเคชันจะถูกถอนการติดตั้ง (Uninstall) หรือมีการรันคำสั่งสั่งล้างข้อมูล

• **String-Only (จัดเก็บเฉพาะสตริง):** ตัวจัดเก็บนี้บันทึกได้เฉพาะข้อมูลที่เป็นข้อความตัวหนังสือ (String) เท่านั้น หากมีข้อมูลประเภทอื่น เช่น ตัวเลข หรือออบเจกต์ จำเป็นต้องมีกระบวนการแปลงค่าก่อนเสมอ

คำสั่งติดตั้งสำหรับ Expo SDK 54:

```
npx expo install @react-native-async-storage/async-storage
```

8.2 การทำงานพื้นฐานกับ AsyncStorage (CRUD Operations)

ในการเรียกเขียนหรืออ่านข้อมูล เราจำเป็นต้องเรียกใช้คำสั่งผ่านบล็อกคำสั่ง `async/await` เสมอ เพื่อรับประกันว่าข้อมูลถูกเขียนหรืออ่านเสร็จก่อนจะนำผลลัพธ์ไปประมวลผลต่อ

1. การเขียนหรืออัปเดตข้อมูล (setItem)

```
import AsyncStorage from '@react-native-async-storage/async-storage';

const saveData = async (value: string) => {
  try {
    await AsyncStorage.setItem('user_name', value);
    console.log('บันทึกข้อมูลเรียบร้อยแล้ว');
  } catch (error) {
    console.error('บันทึกผิดพลาด:', error);
  }
};
```

2. การอ่านข้อมูลเดิมกลับมาใช้ (getItem)

```
const loadData = async () => {
  try {
    const value = await AsyncStorage.getItem('user_name');
    if (value !== null) {
      console.log('ข้อมูลที่อ่านได้:', value);
    }
  } catch (error) {
    console.error('อ่านข้อมูลผิดพลาด:', error);
  }
};
```

3. การลบข้อมูลบางชิ้น (removeItem)

```
const removeData = async () => {
  try {
    await AsyncStorage.removeItem('user_name');
    console.log('ลบข้อมูลสำเร็จ');
  } catch (error) {
    console.error('ลบข้อมูลผิดพลาด:', error);
  }
};
```

8.3 การบันทึกและดึงข้อมูลออบเจกต์ซับซ้อน (JSON Serialization)

เนื่องจาก AsyncStorage สนับสนุนการบันทึกเฉพาะชนิดข้อมูล String หากเราส่งโครงสร้างประเภทออบเจกต์ (Object) หรืออาร์เรย์ (Array) ไปตรง ๆ ระบบจะไม่สามารถทำงานได้ถูกต้อง เราจึงต้องแปลงรูปแบบข้อมูลให้เป็นรูปแบบสตริงด้วยการทำ Serialization

- **ก่อนทำการบันทึก:** แปลงออบเจกต์ให้เป็นสตริงด้วย `JSON.stringify(object)`
- **หลังจากดึงข้อมูลกลับมา:** แปลงสตริงกลับไปเป็นออบเจกต์ด้วย `JSON.parse(string)`

```
import AsyncStorage from '@react-native-async-storage/async-storage';

interface UserProfile {
  name: string;
  age: number;
  skills: string[];
}

// 1. ฟังก์ชันบันทึกข้อมูลประเภทออบเจกต์ลงเครื่อง
export const saveUserProfile = async (profile: UserProfile) => {
  try {
    const jsonString = JSON.stringify(profile); // แปลง Object เป็น JSON String
    await AsyncStorage.setItem('user_profile', jsonString);
  } catch (e) {
    console.error('บันทึกออบเจกต์ผิดพลาด:', e);
  }
};

// 2. ฟังก์ชันดึงและแปลงข้อมูลกลับมาเป็นออบเจกต์เดิม
export const getUserProfile = async (): Promise<UserProfile | null> => {
  try {
    const jsonString = await AsyncStorage.getItem('user_profile');
    return jsonString != null ? JSON.parse(jsonString) : null; // แปลง JSON String กลับเป็น Object
  } catch (e) {
    console.error('ดึงออบเจกต์ผิดพลาด:', e);
    return null;
  }
};
```

8.4 การดึงข้อมูลจาก API ด้วย Fetch (HTTP GET)

นอกเหนือจากการเก็บข้อมูลภายในเครื่องแล้ว โมบายแอปพลิเคชันส่วนใหญ่จำเป็นต้องมีการทำงานร่วมกับระบบเซิร์ฟเวอร์หลังบ้านผ่านสถาปัตยกรรม REST API คำสั่งมาตรฐานที่รองรับการใช้งานใน Expo โดยไม่ต้องติดตั้งไลบรารีภายนอกเพิ่มคือ **`fetch()`** (MDN Web Docs, 2024)

ตัวอย่างคำสั่งดึงข้อมูลล่าสุด (GET) จากเซิร์ฟเวอร์ JSONPlaceholder:

```
import { useState, useEffect } from 'react';
import { View, Text, ActivityIndicator } from 'react-native';

export default function ApiGetExample() {
  const [data, setData] = useState<any>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
```

```
  // ฟังก์ชันดึงข้อมูลแบบ Asynchronous
    const fetchPost = async () => {
      try {
        const response = await
fetch('https://jsonplaceholder.typicode.com/posts/1');
        const json = await response.json(); // แปลงการตอบสนองกลับเป็นออบเจกต์
        setData(json);
      } catch (error) {
        console.error('เกิดข้อผิดพลาดในการดึงข้อมูล:', error);
      } finally {
        setLoading(false);
      }
    };

    fetchPost();
  }, []);

  if (loading) return <ActivityIndicator size="large" />;

  return (
    <View style={{ padding: 20 }}>
      {data && (
        <>
          <Text style={{ fontWeight: 'bold', fontSize: 18
}}>{data.title}</Text>
          <Text style={{ marginTop: 10 }}>{data.body}</Text>
        </>
   )}
    </View>
 );
}
```

8.5 การส่งข้อมูลขึ้นเซิร์ฟเวอร์ด้วย Fetch (HTTP POST)

การสร้างข้อมูลชิ้นใหม่บนเซิร์ฟเวอร์ (เช่น การส่งแบบฟอร์มสมัครสมาชิก, การตั้งกระทู้ใหม่) จะทำงานผ่านการส่ง Request ในรูปแบบ **HTTP POST** ซึ่งเราต้องระบุอ็อพชันหลัก 3 อย่างในพารามิเตอร์ที่สองของคำสั่ง `fetch()`:

1. `method`: กำหนดให้เป็นสตริง `'POST'`
2. `headers`: กำหนดประเภทข้อมูลที่ส่งไป เช่น `'Content-Type': 'application/json'`
3. `body`: ข้อมูล JSON ที่ถูกจัดทำการแปลงเป็นสตริงด้วย `JSON.stringify(data)`

```
import { useState } from 'react';
import { View, TextInput, Button, Alert, ActivityIndicator, StyleSheet }
from 'react-native';
```

```
export default function SendPostScreen() {
  const [title, setTitle] = useState('');
  const [body, setBody] = useState('');
 const [submitting, setSubmitting] = useState(false);

  const handleSubmit = async () => {
    if (!title || !body) {
      Alert.alert('กรุณากรอกข้อมูล', 'จำเป็นต้องกรอกทั้งหัวเรื่องและเนื้อหา');
      return;
    }

    try {
      setSubmitting(true);

   // ส่งข้อมูล POST Request ไปยังเป้าหมายภายนอก
      const response = await
fetch('https://jsonplaceholder.typicode.com/posts', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json; charset=UTF-8',
    },
        body: JSON.stringify({
          title: title,
          body: body,
          userId: 1,
        }),
      });

      if (response.ok) {
        const result = await response.json();
    // แสดงการตอบรับความสำเร็จจากเซิร์ฟเวอร์ (เช่น รหัสไอดี 101)
        Alert.alert('ส่งข้อมูลสำเร็จ!', `สร้างโพสต์ใหม่รหัส ID: ${result.id}`);
        setTitle('');
        setBody('');
      } else {
        throw new Error('เกิดข้อผิดพลาดจากฝั่งเซิร์ฟเวอร์');
      }
    } catch (error: any) {
      Alert.alert('การส่งข้อมูลล้มเหลว', error.message || 'กรุณาตรวจสอบสัญญาณ
อินเทอร์เน็ต');
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <View style={styles.container}>
      <TextInput
        style={styles.input}
```

```
          placeholder="กรอกหัวเรื่อง (Title)"
          value={title}
          onChangeText={setTitle}
    />
        <TextInput
          style={[styles.input, styles.textArea]}
          placeholder="กรอกเนื้อหาโพสต์ (Body)"
          value={body}
          onChangeText={setBody}
          multiline
    />
        {submitting ? (
          <ActivityIndicator size="small" color="#38bdf8" />
    ) : (
          <Button title="ส่งโพสต์ขึ้นเซิร์ฟเวอร์" onPress={handleSubmit} />
    )}
      </View>
  );
}

const styles = StyleSheet.create({
  container: { padding: 20 },
  input: {
    borderWidth: 1,
    borderColor: '#ccc',
    padding: 10,
    borderRadius: 8,
    marginBottom: 12,
  },
  textArea: {
    height: 100,
    textAlignVertical: 'top',
  },
});
```

8.6 ระบบจัดการสถานะ Loading และการดักจับข้อผิดพลาด (Error Handling)

สุนทรียภาพที่ดีของโมบายแอปพลิเคชัน คือการแจ้งให้ผู้ใช้งานรับรู้อย่างชัดเจนว่าแอปพลิเคชันกำลังดำเนินกิจกรรมอะไรอยู่ และสามารถดักจับสถานการณ์เมื่อไม่สามารถติดต่อเซิร์ฟเวอร์ได้

แนวทางการจัดการสถานะข้อมูล:

1. **สถานะ Loading (`isLoading`):** ตั้งค่าเป็น `true` ก่อนที่กระบวนการ `fetch()` จะเริ่มขึ้น และตั้งคืนเป็น `false` เสมอในบล็อก `finally` เพื่อปิดไอคอนตัวหมุน

2. **สถานะ Error (`error`):** ใช้บล็อก `try-catch` ในการสกัดกั้นข้อผิดพลาดของอินเทอร์เน็ต หากพบอาการขัดข้องให้เก็บออบเจกต์ข้อผิดพลาดไว้แสดงหน้าจอแจ้งเตือนปุ่มรีเฟรช (Retry Button)

3. **ตรวจสอบรหัสสถานะความสำเร็จ (`response.ok`):** ในบางครั้งเครือข่ายสัญญาณดีมากแต่ฐานข้อมูลปลายทางมีปัญหา (เช่น ส่งรหัส 404 หรือ 500 กลับมา) ตัวคำสั่ง `fetch` จะไม่ทริกเกอร์เข้าบล็อก `catch` อัตโนมัติ เราจึงต้องเขียนดักเช็คค่าของ `response.ok` (มีค่าเป็น true เมื่อ status อยู่ในช่วง 200-299) ด้วยตัวเอง

8.7 การประยุกต์ใช้ในการทำระบบจำเซสชันการล็อกอิน (Auth Token Caching)

หนึ่งในการประยุกต์ใช้งาน AsyncStorage ที่พบเจอได้บ่อยที่สุดในโปรเจกต์เชิงพาณิชย์คือการจัดการระบบล็อกอินเข้าใช้งานของสมาชิก โดยหลังจากผู้ใช้ทำการล็อกอินสำเร็จ โทเค็น (Token) จะถูกเก็บไว้ใน AsyncStorage เพื่อที่การเปิดแอปใช้งานครั้งถัดไป ผู้ใช้จะไม่ต้องกรอกรหัสผ่านซ้ำอีกรอบ

แผนภาพการไหลของระบบจดจำล็อกอิน:

```
graph TD
    A[เปิดแอปพลิเคชัน] --> B{มี token ใน AsyncStorage หรือไม่?}
    B -- มี token --> C[ข้ามไปที่หน้า Dashboard / หน้าหลัก]
    B -- ไม่มี token --> D[แสดงหน้า Login ให้กรอกข้อมูล]
    D --> E[กรอกรหัสผ่านส่ง API สำเร็จ]
    E --> F[เซฟ token ลง AsyncStorage]
    F --> C
```

ตัวอย่างโครงสร้างโค้ดการจัดการตรวจสอบสิทธิ์ตอนเริ่มต้นเปิดแอป:

```
import { useEffect, useState } from 'react';
import { View, Text, Button, StyleSheet, ActivityIndicator } from 'react-
native';
import AsyncStorage from '@react-native-async-storage/async-storage';

export default function AppSessionManager() {
  const [appReady, setAppReady] = useState(false);
  const [userToken, setUserToken] = useState<string | null>(null);

 // ตรวจเช็คโทเค็นที่ถูกแคชไว้ตอนเปิดแอป
  useEffect(() => {
    const checkToken = async () => {
      try {
        const token = await AsyncStorage.getItem('user_auth_token');
        if (token) {
          setUserToken(token); // อัปเดตสเตตหากพบข้อมูลเก่า
```

```
        }
      } catch (e) {
        console.error(e);
      } finally {
        setAppReady(true);
      }
    };
    checkToken();
  }, []);

 // ฟังก์ชันจำลองล็อกอิน
  const handleLogin = async () => {
    const mockToken = 'jwt-secure-token-123456';
    await AsyncStorage.setItem('user_auth_token', mockToken);
    setUserToken(mockToken);
  };

 // ฟังก์ชันล็อกเอาต์
  const handleLogout = async () => {
    await AsyncStorage.removeItem('user_auth_token');
    setUserToken(null);
  };

  if (!appReady) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" />
      </View>
  );
  }

  return (
    <View style={styles.container}>
      {userToken ? (
        <View>
          <Text style={styles.text}>🔓 ยินดีต้อนรับกลับ! เข้าสู่ระบบสำเร็จ</Text>
          <Text style={styles.subText}>Token: {userToken}</Text>
          <Button title="ออกจากระบบ (Logout)" color="#ef4444"
onPress={handleLogout} />
        </View>
    ) : (
        <View>
          <Text style={styles.text}>🔒 กรุณาลงชื่อเข้าใช้งาน</Text>
          <Button title="ลงชื่อเข้าใช้จำลอง (Login)" onPress={handleLogin} />
        </View>
    )}
    </View>
 );
}
```

```
const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center',
backgroundColor: '#f1f5f9' },
  center: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  text: { fontSize: 18, fontWeight: 'bold', marginBottom: 10, textAlign:
'center' },
  subText: { fontSize: 13, color: '#64748b', marginBottom: 20, textAlign:
'center' },
});
```

สรุปท้ายบทเรียน

ความเข้าใจและการติดตั้ง AsyncStorage: AsyncStorage เป็นเครื่องมือสำคัญในการรักษาความคงอยู่ของข้อมูล in ระดับอุปกรณ์ประเภทคีย์-ค่า (Key-Value) แบบทำงานเบื้องหลังโดยไม่รบกวนหน้าจอ (Asynchronous) ซึ่งข้อมูลจะอยู่ถาวรยกเว้นแต่จะถอนติดตั้งหรือสั่งล้างข้อมูล โดยใน Expo SDK 54 เราสามารถติดตั้งมาใช้งานได้สะดวกผ่านคำสั่ง npx expo install เพื่อนำมาจัดเก็บข้อมูลถาวร เช่น การตั้งค่าหรือเซสชันล็อกอิน

การทำงานพื้นฐานกับ AsyncStorage (CRUD Operations): กระบวนการทำงานพื้นฐานในการเก็บและดึงข้อมูล AsyncStorage จะใช้คำสั่ง async/await ในการประสานงาน โดยมีคำสั่งหลักที่สำคัญคือ setItem สำหรับเขียนหรืออัปเดตข้อมูล, getItem เพื่อดักเรียกอ่านข้อมูลเดิมกลับมาประมวลผลต่อ และ removeItem สำหรับการสั่งลบข้อมูลแต่ละรายการที่ระบุค่าคีย์ไว้

การบันทึกและดึงข้อมูลออบเจกต์ซับซ้อน (JSON Serialization): เนื่องจาก AsyncStorage รองรับข้อมูลชนิดข้อความสตริงเท่านั้น เมื่อต้องการจัดเก็บข้อมูลซับซ้อนประเภทออบเจกต์หรืออาร์เรย์ ผู้พัฒนาจำเป็นต้องแปลงข้อมูลเป็นสตริงผ่านกระบวนการ JSON.stringify(object) ก่อนนำลงไปเก็บบันทึก และทำการแปลงข้อมูลสตริงสกัดกลับเป็นออบเจกต์ดั้งเดิมผ่าน JSON.parse(string) หลังเรียกขึ้นมาใช้งาน

การดึงข้อมูลจาก API ด้วย Fetch (HTTP GET): การประมวลผลดึงข้อมูล API ภายนอกมายังแอปพลิเคชันจะทำงานผ่านฟังก์ชัน fetch() ของระบบเพื่อยิง GET Request ไปยังจุดปลายของเซิร์ฟเวอร์ปลายทาง โดยเรียกใช้คำสั่งควบคู่กับ useEffect ในการดึงข้อมูลสดเมื่อเริ่มรันแอป และทำการแปลงค่าการตอบสนองเป็นโครงสร้างออบเจกต์ด้วยเมธอด .json() ก่อนนำไปแจกจ่าย

การส่งข้อมูลขึ้นเซิร์ฟเวอร์ด้วย Fetch (HTTP POST): การส่งข้อมูลใหม่ขึ้นฝั่งเซิร์ฟเวอร์ภายนอกใช้วิธีส่งแบบ HTTP POST ผ่านการระบุอ็อพชันเพิ่มเติมใน fetch() โดยระบุ method: 'POST', ระบุประเภท content-type เป็น JSON ในส่วนของ headers และส่งข้อมูลหลักที่ถูกแปลง

เป็นสตริงด้วย JSON.stringify ในส่วนของ body ซึ่งช่วยให้เราสามารถสมัครสมาชิกหรือโพสต์ข้อมูลได้สำเร็จ

ระบบจัดการสถานะ Loading และการดักจับข้อผิดพลาด: การเพิ่มสัมผัสความลื่นไหลในการใช้บริการเน็ตเวิร์กจำต้องมีการจัดการสถานะ Loading เพื่อควบคุมการแสดง ActivityIndicator และครอบคลุมระบบดักจับปัญหา Error Handling ผ่านบล็อก try-catch เพื่อดักจับสัญญาณอินเทอร์เน็ตล่ม โดยต้องทำการตรวจสอบค่าผลลัพธ์ผ่าน response.ok ควบคู่กันเพื่อป้องกันกรณีเซิร์ฟเวอร์ส่งข้อผิดพลาดกลับมา

การประยุกต์ใช้ในการทำระบบจำเซสชันการล็อกอิน: การประยุกต์ใช้ AsyncStorage และเน็ตเวิร์ก API ที่มีความสำคัญในแอปพลิเคชันเชิงพาณิชย์คือระบบเซสชันการเข้าสู่ระบบ โดยหลังจากยืนยันตัวตนสำเร็จแล้วจะนำ Auth Token ไปบันทึกลงในเครื่อง และเมื่อเปิดแอปใหม่จะสามารถดักจับโทเค็นดังกล่าวมาล็อกอินอัตโนมัติ ช่วยลดความซับซ้อนให้ผู้ใช้งานไม่ต้องป้อนข้อมูลซ้ำซ้อน

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้ความเข้าใจเกี่ยวกับการใช้งาน AsyncStorage ในการบันทึกข้อมูลถาวรประเภทข้อความและออบเจกต์ซับซ้อน, การดึงและส่งข้อมูลด้วย Fetch API (HTTP GET & POST), การตรวจจับสถานะการตอบสนองความสำเร็จผ่าน `response.ok`, และการทำระบบจดจำล็อกอินด้วยโทเค็น

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดอธิบายคุณลักษณะการจัดเก็บข้อมูลของไลบรารี `AsyncStorage` ใน React Native ได้ถูกต้องที่สุด?

ก. บันทึกข้อมูลประเภทอ็อบเจกต์ของภาษารันไทม์ไว้ในคลาวด์เซิร์ฟเวอร์แบบเรียลไทม์

ข. จัดเก็บข้อมูลแบบถาวรในรูปแบบกุญแจ-ค่า (Key-Value) เฉพาะข้อมูลชนิดข้อความ (String) บนพื้นที่ตัวเครื่องของผู้ใช้

ค. เก็บข้อมูลไว้ในหน่วยความจำ RAM ซึ่งจะหายไปทันทีเมื่อปิดหน้าต่างแอปพลิเคชัน

ง. บันทึกรูปภาพและวิดีโอต้นฉบับลงในแกลเลอรีภาพหลักของสมาร์ทโฟน

ข้อ 2: เมื่อผู้ใช้ปิดแอปพลิเคชันหรือทำการรีสตาร์ทตัวเครื่องโทรศัพท์ใหม่ ข้อมูลที่บันทึกไว้ใน `AsyncStorage` จะได้รับผลกระทบอย่างไร?

ก. ข้อมูลจะถูกเข้ารหัสสุ่มและย้ายไปฝากไว้ที่เซิร์ฟเวอร์ของ Expo

ข. ข้อมูลจะคงอยู่เหมือนเดิมอย่างปลอดภัย เนื่องจากข้อมูลถูกเก็บถาวรลงในไดรฟ์ของระบบปฏิบัติการเครื่อง

ค. ข้อมูลจะสูญหายทันทีเนื่องจาก AsyncStorage ไม่รองรับหน่วยความจำระยะยาว

ง. ข้อมูลจะกลายเป็นสัญลักษณ์เครื่องหมายคำถามทั้งหมดจนกว่าจะเปิดสัญญาณ WiFi

ข้อ 3: หากต้องการเขียนทับบันทึกหรือส่งชุดข้อมูลสตริงใหม่ไปเก็บใน AsyncStorage ผู้พัฒนาแอปพลิเคชันต้องเรียกใช้งานเมธอดใด?

ก. `AsyncStorage.getItem`

ข. `AsyncStorage.removeItem`

ค. `AsyncStorage.setItem`

ง. `AsyncStorage.clear`

ข้อ 4: ข้อใดคือพฤติกรรมที่เกิดขึ้นหากคุณส่งออบเจกต์ผู้ใช้งาน เช่น `const user = { name: "สมชาย", age: 30 };` เข้าไปที่คำสั่ง `AsyncStorage.setItem('user_data', user)` โดยตรงโดยไม่มีการแปลงรูปแบบข้อมูล?

ก. ระบบจะบันทึกข้อมูลออบเจกต์ได้สมบูรณ์และใช้งานได้ปกติ

ข. ระบบจะเกิดข้อผิดพลาดในการทำงาน หรือข้อมูลจะถูกแปลงกลายสตริงข้อความแทนที่ตัวตนจริงเป็นค่า `"[object Object]"` ซึ่งไม่สามารถนำกลับมาแกะข้อมูลด้านในได้

ค. ระบบจะอัปโหลดข้อมูลตัวแปรนี้ขึ้นสู่เซิร์ฟเวอร์ของเฟซบุ๊กโดยอัตโนมัติ

ง. ระบบจะเปลี่ยนสไตล์หน้าจอปุ่มสัมผัสของเครื่องเป็นสีแดงเพื่อเตือนภัย

ข้อ 5: วิธีการและเครื่องมือคู่ใดที่เหมาะสมที่สุดในการบันทึกข้อมูลออบเจกต์ข้ามหน้าด้วย AsyncStorage เพื่อไม่ให้สูญเสียคุณสมบัติข้อมูลลูกด้านใน?

ก. ใช้ `JSON.stringify(object)` แปลงสตริงก่อนจัดเก็บ และใช้ `JSON.parse(string)` แกะกลับเป็นออบเจกต์ตอนดึงข้อมูลกลับมา

ข. ใช้คำสั่ง `toString()` ในการบันทึก และใช้ `Number()` ในการเรียกรับค่ากลับคืนมา

ค. ใช้ระบบ `AsyncStorage.clear()` ทั้งก่อนเขียนและหลังเขียนข้อมูลใหม่

ง. ไม่จำเป็นต้องแปลงค่าใด ๆ เพียงใส่คำสั่ง `await` คุมไว้ระบบเนทีฟจะสืบประเภทให้เอง

ข้อ 6: เมื่อเรียกคำสั่ง `fetch()` เพื่อเชื่อมข้อมูลผ่านอินเทอร์เน็ต หากเซิร์ฟเวอร์หลังบ้านส่งรหัสสถานะข้อผิดพลาด เช่น 404 `Not Found` หรือ 500 `Internal Server Error` กลับมา ตัวกลไก Promise ของ Fetch จะมีพฤติกรรมอย่างไร?

ก. จะสั่ง Reject เข้าสู่บล็อกข้อผิดพลาด `catch` โดยอัตโนมัติ

ข. จะยังทำงานส่งต่อความสำเร็จ (Resolve) ตามปกติโดยไม่วิ่งไปที่บล็อก `catch` ซึ่งผู้พัฒนาต้องตรวจสอบความสมบูรณ์ผ่านพร็อพส์ `response.ok` ด้วยตนเอง

ค. จะปิดระบบเครือข่ายของแอปเพื่อป้องกันความเสียหาย

ง. จะทำลายการเชื่อมต่อสเตตสถานะล็อกอินของผู้ใช้ทั้งหมดทิ้งทันที

ข้อ 7: พร็อพส์ (Properties) ตัวใดของ Response Object ที่ตอบกลับจากคำสั่ง `fetch()` ซึ่งระบุค่าสถานะความสำเร็จของเครือข่ายที่มีรหัส HTTP Status อยู่ในช่วง 200 ถึง 299?

ก. `response.statusText`

ข. `response.headers`

ค. `response.ok`

ง. `response.bodyUsed`

ข้อ 8: การเขียนคำสั่งส่งข้อมูลแบบ HTTP POST ด้วยคำสั่ง Fetch API ข้อใดระบุพารามิเตอร์การจัดส่งข้อมูลในช่อง `fetch(url, options)` ได้ถูกต้องและครบถ้วนตามหลักการพัฒนา?

ก. `options: { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(data) }`

ข. `options: { method: 'GET', headers: { 'Accept': 'text/plain' }, body: data }`

ค. `options: { method: 'DELETE', body: data.toString() }`

ง. `options: { method: 'POST', body: data }`

ข้อ 9: ในขั้นตอนจัดการการตอบสนองผู้ใช้ขณะที่ฟังก์ชัน `fetch()` กำลังวิ่งทำงานอยู่บนเน็ตเวิร์ก ข้อใดคือแนวปฏิบัติที่ดีในการคุมสเตตเพื่อความพึงพอใจของผู้เรียนหรือผู้ใช้แอป?

ก. ตั้งค่าสเตต `isLoading` เป็น `true` ก่อนเรียกเฟทช์ และสลับเป็น `false` ในบล็อก `finally` พร้อมแสดงคอมโพเนนต์ `ActivityIndicator` ระหว่างรอสายข้อมูล

ข. ปิดปุ่มกดไม่ให้สัมผัส และห้ามแสดงความคืบหน้าใด ๆ บนจอจนกว่าแอปจะค้าง

ค. แสดงกล่องข้อความเตือนภัยสีเหลืองกะพริบบนขอบหน้าต่างจอมือถือ

ง. บังคับย้อนกลับหน้าแรกสุดของแอปพลิเคชันทุกครั้งที่ระบบกำลังรอโหลดข้อมูล

ข้อ 10: ในระบบจัดการจดจำสิทธิ์การล็อกอิน (Auth Token Caching) หากเราต้องการให้โทรศัพท์มือถือทำการเช็คประวัติชื่อล็อกอินในเครื่องเพื่อกระโดดข้ามหน้ากรอกแบบฟอร์ม เราต้องเรียกเขียนโค้ดตรวจสอบโทเค็นในจุดใดของแอป?

ก. เขียนตรวจในฟังก์ชัน `onPress` ของปุ่มล็อกเอาต์

ข. เขียนตรวจในบล็อกสเตต `useEffect` ของหน้าต่างแสดงผลแรกสุด (Root Component) เพื่ออ่านค่าคีย์โทเค็นจาก AsyncStorage ตอนเปิดแอปขึ้นมา

ค. เขียนระบุซ้อนอยู่ในสไตล์ชีทภายนอกเพื่อให้คอมไพเลอร์เช็คสี

ง. เขียนส่งในพารามิเตอร์ **body** ของ POST request แรกสุดตอนสลับสี

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: AsyncStorage เก็บข้อมูลบนเครื่องส่วนตัวปลายทาง (Local Storage) ไม่ใช่ฐานข้อมูลระบบ Cloud

• ข. ถูก: เป็นระบบจัดเก็บบนเครื่องเก็บแบบคีย์-ค่า และทำงานรองรับเฉพาะประเภทข้อมูล String เท่านั้น

• ค. ผิด: เป็นระบบจัดเก็บถาวร (Persistent) ข้อมูลจะไม่สูญหายเมื่อปิดแอปพลิเคชัน

• ง. ผิด: ไม่ได้มีสิทธิ์เข้าถึงหรือบันทึกรูปลงคลังภาพส่วนกลางของแอปกล้องโดยตรงโดยไม่มีสิทธิ์พิเศษ

เฉลย ข้อ 2: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: ข้อมูลไม่ได้ถูกส่งไปภายนอก และไม่สูญหายเมื่อไฟดับหรือรีสตาร์ทตัวเครื่อง

• ข. ถูก: หน้าที่ของ AsyncStorage คือการเขียนไฟล์ข้อมูลลงบนฮาร์ดไดรฟ์ภายในเครื่องแบบถาวร ข้อมูลจึงยังคงปลอดภัยอยู่ครบถ้วน

เฉลย ข้อ 3: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: `getItem` ใช้สำหรับอ่านข้อมูล

• ข. ผิด: `removeItem` ใช้สำหรับลบข้อมูลชิ้นนั้น ๆ

• ค. ถูก: `setItem` ทำงานรับค่าพารามิเตอร์ Key และ Value เพื่อบันทึกเขียนข้อมูลทับหรือบันทึกใหม่

• ง. ผิด: `clear` ใช้ล้างลบข้อมูลทุกชิ้นในคลัง

เฉลย ข้อ 4: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: ข้อมูลออบเจกต์จะไม่ถูกเก็บไว้สมบูรณ์หากไม่ผ่านการแปลงค่าก่อน

• **ข. ถูก:** เนื่องจาก AsyncStorage รับค่าสตริงเท่านั้น หากส่ง JavaScript Object เข้าไปโดยไม่ใช้เครื่องมือแปลง ค่าจะถูกแปลงเป็นรูปข้อความ "`[object Object]`" ส่งผลให้ตอนดึงข้อมูลกลับมา ข้อมูลโครงสร้างคุณสมบัติภายในจะหายไปทั้งหมดใช้งานไม่ได้

• **ค. และ ง. ผิด:** ไม่มีส่วนต่อพ่วงความเชื่อมโยงกับเฟซบุ๊ก หรือการเปิดไฟแอปเตือนภัยทางสี

เฉลย ข้อ 5: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** เป็นมาตรฐานสากลในการจัดเก็บออบเจกต์บนระบบ AsyncStorage โดยแปลงเป็นสตริงรูปแบบ JSON ก่อนบันทึก และถอดสตริงข้อความกลับมาเป็น JavaScript Object ใช้งานผ่าน `JSON.parse`

• **ข. ค. และ ง. ผิด:** การทำ `toString` ทั่วไปไม่รองรับโครงสร้างซับซ้อน และ `clear` จะล้างข้อมูลทิ้งทั้งหมด

เฉลย ข้อ 6: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** Fetch จะสั่ง Reject สัญญาณเฉพาะกรณีเกิดปัญหาการเชื่อมต่อทางกายภาพล้มเหลว (เช่น เน็ตตัด สัญญาณหลุด หรือไม่พบ DNS โดเมน) เท่านั้น

• **ข. ถูก:** รหัสข้อผิดพลาดประเภท HTTP Status (เช่น 404, 500) จากเครื่องเซิร์ฟเวอร์ยังถือว่ารับส่งสำเร็จในทางระบบเชื่อมต่อ เอนจิ้น Fetch จึงยัง Resolve คืนผลลัพธ์มาให้ เราจึงจำเป็นต้องตรวจเช็คค่าความสำเร็จผ่าน `response.ok` ด้วยตนเองเพื่อความปลอดภัย

• **ค. และ ง. ผิด:** ไม่ได้ปิดระบบเน็ตเวิร์ก หรือทำลายข้อมูลโทเค็นในเครื่องโดยอัตโนมัติ

เฉลย ข้อ 7: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ข. และ ง. ผิด:** พร็อพส์อื่น ๆ ให้ข้อมูลประเภทข้อมูลหรือส่วนหัว แต่ไม่ได้เก็บสถานะสำเร็จเป็น Boolean

• **ค. ถูก:** `response.ok` เป็นคุณสมบัติประเภท Boolean (true/false) ที่ตัวระบบ Fetch เตรียมให้ตรวจสอบสถานะว่ารหัส HTTP Code ตอบกลับมาจากเซิร์ฟเวอร์เรียบร้อยดีหรือไม่ (มีค่าเป็น true เมื่อรหัสอยู่ในช่วง 200-299)

เฉลย ข้อ 8: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** โครงสร้าง HTTP POST ที่ถูกต้องจำเป็นต้องเปลี่ยน `method` เป็น `'POST'` เติม Headers แจ้งเซิร์ฟเวอร์ว่าเราส่งฟอร์มข้อมูลชนิด JSON และแปลงชุดข้อมูลให้กลายเป็น String ในคีย์ `body`

• **ข. ค. และ ง. ผิด:** ผิดข้อกำหนดด้านวิธี และไม่มีการตั้งค่า Headers ครอบสตรีมสตริงที่ถูกต้อง

เฉลย ข้อ 9: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** การสลับค่าสเตต `isLoading` ตั้งค่าเพื่อควบคุมและเรนเดอร์ไอคอน ActivityIndicator ระหว่างดาวน์โหลดเป็นเทคนิคมาตรฐานในการแจ้งความเคลื่อนไหวเพื่อไม่ให้เครื่องดูค้างนิ่ง

• **ข. ค. และ ง. ผิด:** ไม่ควรปิดความเคลื่อนไหวเงียบโดยไม่มีปฏิกิริยา และไม่ควรกระพริบเตือนตื่นตระหนก หรือบังคับย้ายหน้าจอผู้ใช้ไปมา

เฉลย ข้อ 10: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ปุ่มล็อกเอาต์มีจุดประสงค์เพื่อทำลายโทเค็นทิ้ง ไม่ใช่ตรวจสอบสิทธิ์เข้าใช้งาน

• **ข. ถูก:** วงรอบชีวิตการเรนเดอร์เริ่มต้นของคอมโพเนนต์จะกระตุ้นคำสั่ง `useEffect` เป็นด่านแรกสุด การตรวจค้นหาข้อมูล `user_auth_token` ในจุดนี้ จะทำให้เราสลับเปลี่ยนหน้าจอได้ถูกต้องทันทีตั้งแต่แอปเปิดหน้าแรก

• **ค. และ ง. ผิด:** ไฟล์สไตล์ชีทภายนอกควบคุมเฉพาะความสวยงามดีไซน์ และ POST request ส่งข้อมูลหาเซิร์ฟเวอร์ไม่ใช่การดึงตรวจค่าในโทรศัพท์

ปฏิบัติการท้ายบทเรียน: การสร้างแอปบันทึกข้อคิดเห็นออนไลน์แบบจดจำประวัติ

ปฏิบัติการนี้จะฝึกทักษะการประยุกต์ใช้งานหน่วยความจำถาวรภายในเครื่องร่วมกับการจัดการส่งต่อข้อมูลขึ้นสู่คลาวด์ผ่านเครือข่าย โดยผู้เรียนจะได้สร้างแอปพลิเคชัน **"สมุดโน้ตบันทึกและซิงก์ความคิดเห็นออนไลน์ (Synchronized Note Keeper)"** ซึ่งจะทำหน้าจอดึงข้อมูลบันทึกส่วนกลางมาจาก REST API ด้วย HTTP GET แสดงผลในลิสต์ และเปิดฟอร์มเขียนเพิ่มโพสต์ส่งกลับขึ้นเซิร์ฟเวอร์ด้วย HTTP POST พร้อมเพิ่มความเสถียรโดยการบันทึกสำรอง (Cache) ข้อมูลรอบล่าสุดเก็บลงเครื่องผ่าน `AsyncStorage` เพื่อให้อ่านประวัติโน้ตเก่าขึ้นมารายงานหน้าต่างจอได้ทันทีแม้กรณีสัญญาณอินเทอร์เน็ตขาดหาย

1. วัตถุประสงค์การเรียนรู้

1. สามารถติดตั้งและเขียนโปรแกรมดึงข้อมูลแบบออฟไลน์แคชชิ่ง (Offline-First caching) โดยเรียกอ่านประวัติเก่าจาก AsyncStorage ขึ้นมาจัดแสดงเมื่อแอปเริ่มต้นทำงาน

2. สามารถเขียนระบบดักส่งข้อมูลด้วย Fetch API (HTTP POST Method) เพื่อบันทึกโน้ตใหม่ขึ้นสู่เซิร์ฟเวอร์ภายนอกสำเร็จ

3. สามารถจัดสร้างสถานะ Loading และจัดการป๊อปอัพ Alert แจ้งเตือนข้อผิดพลาดกรณีการติดต่อเครือข่ายล้มเหลวได้อย่างถูกต้อง

4. สามารถสลักแปลงข้อมูลอาร์เรย์ของโพสต์แบบซับซ้อนให้กลายเป็น JSON String และแกะกลับคืนเพื่อจัดสรรร่วมกับคอมโพเนนต์ `FlatList` ได้ถูกต้อง

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องเขียนสร้างแอปพลิเคชันที่มีโครงสร้างองค์ประกอบและการทำงานตามเงื่อนไขดังนี้:

1. การทำงานร่วมกับเซิร์ฟเวอร์ภายนอก (Fetch API Integration):

• **ดึงข้อมูลเริ่มต้น (GET Request):** เมื่อเปิดแอปขึ้นมาเป็นครั้งแรก ให้ทำการยิง GET request ไปที่:

```
GET https://jsonplaceholder.typicode.com/posts?_limit=5
```

เพื่อดึงรายการโน้ตจำนวน 5 โพสต์ล่าสุดมาแสดงผล

• **ออฟไลน์แคชชิ่ง (Local Caching):** เมื่อดึงข้อมูลสำเร็จ ให้บันทึกชุดข้อมูลอาร์เรย์นี้เก็บสำรองถาวรลงใน `AsyncStorage` ภายใต้คีย์ `'cached_notes'` เสมอ

• **การส่งข้อมูลโพสต์ใหม่ (POST Request):** มีแบบฟอร์มให้กรอก "หัวข้อโน้ต (Title)" และ "รายละเอียดเนื้อหา (Content)" เมื่อกดปุ่มส่งข้อมูลให้ยิง POST request ไปที่:

```
POST https://jsonplaceholder.typicode.com/posts
```

โดยเมื่อได้ผลลัพธ์ตอบกลับสำเร็จ (Response Code: 201) ให้นำรายการที่บันทึกสำเร็จสมมุตินั้น ไปพ่วงแทรกต่อเพิ่มไว้ด้านบนสุดของรายการแสดงผลในหน้าจอทันที เพื่อสะท้อนภาพข้อมูลอัปเดตแบบเรียลไทม์

2. ระบบต้านทานสัญญาณอินเทอร์เน็ตล่ม (Offline/Error Resilience):

• หากสมาร์ทโฟนปิดสัญญาณเน็ตหรือเซิร์ฟเวอร์พังจน Fetch ล้มเหลว แอปพลิเคชันต้องไม่ดับพัง แต่ต้องมีกลไกดึงข้อมูลโน้ตที่แคชไว้ล่าสุดใน AsyncStorage ขึ้นมาทำงานแทนที่บนลิสต์ และแจ้งเตือนให้ผู้ใช้งานทราบผ่านคำสั่ง `Alert.alert` ว่า "กำลังทำงานในโหมดออฟไลน์"

3. การแสดงผล UI สไตล์โมเดิร์น:

• จัดสไตล์หน้าจอเป็นโทนสี Slate/Sky หรูหราสะอาดตา มีช่องรับข้อมูล Text inputs ของหัวเรื่องและเนื้อหาแยกกัน มีการแสดงผลตัวหมุน ActivityIndicator ในระหว่างเชื่อมเน็ตเวิร์กและการ์ดแสดงผลโน้ตย่อย

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งแพ็คเกจเสริมสำหรับเก็บข้อมูล

สั่งรันคำสั่งติดตั้งคลังหน่วยเก็บข้อมูลถาวรภายในเครื่อง:

```
npx expo install @react-native-async-storage/async-storage
```

ขั้นตอนที่ 2: สร้างฟังก์ชันสำหรับอ่านและเขียนแคช

เขียนแยกโค้ดจัดการข้อมูล JSON เพื่อให้อ่านเข้าใจง่าย:

```
import AsyncStorage from '@react-native-async-storage/async-storage';

const CACHE_KEY = 'cached_notes';

// บันทึกอาร์เรย์ของโพสต์ลงเครื่อง
const cacheNotes = async (notes: any[]) => {
  try {
    await AsyncStorage.setItem(CACHE_KEY, JSON.stringify(notes));
  } catch (e) {
    console.error('ไม่สามารถแคชข้อมูลได้:', e);
  }
};

// กู้ประวัติอาร์เรย์จากเครื่อง
const getCachedNotes = async () => {
  try {
    const raw = await AsyncStorage.getItem(CACHE_KEY);
    return raw ? JSON.parse(raw) : [];
  } catch (e) {
    console.error('ไม่สามารถดึงข้อมูลแคชได้:', e);
    return [];
  }
};
```

ขั้นตอนที่ 3: จัดการกระแสไหลของสเตต (State Workflow)

ประกาศสเตตควบคุม `notes` (เก็บข้อมูลโพสต์), `isLoading` (เช็คเวลาโหลด), และ `isSubmitting` (เช็คเวลาส่งข้อมูล POST) และจัดทำ Logic คัดกรองเครือข่าย:

```
const loadInitialData = async () => {
  try {
    setIsLoading(true);
    const response = await
fetch('https://jsonplaceholder.typicode.com/posts?_limit=5');

    if (response.ok) {
      const data = await response.json();
      setNotes(data);
    await cacheNotes(data); // บันทึกสำรองลง Storage
    } else {
      throw new Error('การตอบรับจากเซิร์ฟเวอร์ไม่สมบูรณ์');
    }
  } catch (error) {
  // กรณีเน็ตหลุด ให้กู้ข้อมูลเก่าขึ้นมาโชว์ตัว
    const localData = await getCachedNotes();
    setNotes(localData);
    Alert.alert('โหมดออฟไลน์', 'ไม่สามารถเชื่อมต่อเซิร์ฟเวอร์ได้ ดึงข้อมูลล่าสุดที่บันทึกไว้ในเครื่องขึ้นมาแสดงผลแทน');
  } finally {
    setIsLoading(false);
  }
};
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถคัดลอกซอร์สโค้ดของแอปพลิเคชันแบบหน้าต่างเดียวชุดนี้ ไปจัดทำและบันทึกทับในไฟล์หลักของโครงการ (เช่น `app/index.tsx` หรือ `App.tsx`) เพื่อรันและจำลองการดึงส่งข้อมูลเครือข่าย:

```
import React, { useState, useEffect } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  TouchableOpacity,
  FlatList,
  ActivityIndicator,
  Alert,
  SafeAreaView,
  KeyboardAvoidingView,
  Platform
} from 'react-native';
```

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// โมเดลข้อมูลโน้ตข้อความ
interface Note {
  id: number;
  title: string;
  body: string;
}

const STORAGE_KEY = 'cached_notes';

export default function SynchronizedNoteKeeper() {
  const [notes, setNotes] = useState<Note[]>([]);
  const [title, setTitle] = useState('');
  const [content, setContent] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [isSubmitting, setIsSubmitting] = useState(false);

 // 1. ตรวจจับข้อมูลตอนเปิดหน้าจอแรกสุด
  useEffect(() => {
    fetchNotes();
  }, []);

 // 2. ฟังก์ชันหลักในการดึงรายการโน้ต (GET)
  const fetchNotes = async () => {
    try {
      setIsLoading(true);
    // ส่งคำขอดึงข้อมูลจำกัดจำนวน 5 โพสต์ล่าสุด
      const response = await
fetch('https://jsonplaceholder.typicode.com/posts?_limit=5');

      if (response.ok) {
        const remoteData = await response.json();
        setNotes(remoteData);
    // ทำการเซฟสำรองเก็บในเครื่องเป็น Offline Cache
      await AsyncStorage.setItem(STORAGE_KEY, JSON.stringify(remoteData));
      } else {
        throw new Error('เซิร์ฟเวอร์ขัดข้อง');
      }
    } catch (error) {
   // ดักจับกรณีออฟไลน์หรือเน็ตพัง ให้ดึงจาก AsyncStorage
      const localData = await AsyncStorage.getItem(STORAGE_KEY);
      if (localData) {
        setNotes(JSON.parse(localData));
      }
      Alert.alert(
        'สัญญาณขาดหาย',
        'ไม่สามารถดึงข้อมูลจากอินเทอร์เน็ตได้ ระบบทำการดึงข้อมูลล่าสุดที่บันทึกในเครื่องขึ้นแสดงผลทดแทน'
   );
    } finally {
```

```
      setIsLoading(false);
    }
  };

// 3. ฟังก์ชันการส่งข้อมูลโน้ตใหม่ (POST)
  const handleAddNote = async () => {
    if (!title.trim() || !content.trim()) {
      Alert.alert('กรอกข้อมูลไม่ครบ', 'กรุณากรอกหัวข้อโน้ตและรายละเอียดให้ครบถ้วน');
      return;
    }

    try {
      setIsSubmitting(true);

   // สั่งส่งข้อมูลชนิด POST ไปเก็บที่ API ปลายทาง
      const response = await
fetch('https://jsonplaceholder.typicode.com/posts', {
        method: 'POST',
    headers: {
          'Content-Type': 'application/json; charset=UTF-8',
        },
        body: JSON.stringify({
          title: title,
          body: content,
          userId: 1
        }),
      });

      if (response.ok) {
        const newNoteFromServer = await response.json();

    // จำลองโครงสร้างการชิงก์โดยนำโน้ตใหม่ไปต่อไว้ด้านบนสุดของสเตตลิสต์ปัจจุบัน
        const updatedNotes = [
          {
            id: newNoteFromServer.id, // รหัสไอดี 101 ที่ได้คืนมาจาก REST API จำลอง
           title: title,
            body: content
          },
     ...notes
    ];

        setNotes(updatedNotes);

    // อัปเดตไฟล์แคชออฟไลน์ในเครื่องให้มีโน้ตใหม่ติดตัวไปด้วย
        await AsyncStorage.setItem(STORAGE_KEY,
JSON.stringify(updatedNotes));

        Alert.alert('สำเร็จ!', 'บันทึกโน้ตชิงก์ขึ้นระบบเซิร์ฟเวอร์เรียบร้อย');
        setTitle('');
```

```
          setContent('');
        } else {
          throw new Error('เกิดข้อผิดพลาดในการประมวลผลบนคลาวด์');
        }
      } catch (error) {
        Alert.alert('ส่งไม่สำเร็จ', 'ไม่สามารถบันทึกข้อมูลขึ้นเซิร์ฟเวอร์ได้ กรุณาเชื่อมต่ออินเทอร์เน็ต');
      } finally {
        setIsSubmitting(false);
      }
    };

    return (
      <SafeAreaView style={styles.safeArea}>
        <KeyboardAvoidingView
          behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
          style={styles.container}
        >
          {/* หัวข้อแอป */}
          <View style={styles.header}>
            <Text style={styles.headerTitle}>📝 Synchronized Note
Keeper</Text>
            <Text style={styles.headerSubtitle}>ซิงก์บันทึกผ่าน API และแคชข้อมูลออฟไลน์
ถาวร</Text>
          </View>

          {/* แบบฟอร์มเขียนบันทึกส่งข้อมูล */}
          <View style={styles.formCard}>
            <Text style={styles.formLabel}>เขียนโน้ตชิ้นใหม่</Text>
            <TextInput
              style={styles.input}
              placeholder="หัวข้อโน้ต..."
              placeholderTextColor="#64748b"
              value={title}
              onChangeText={setTitle}
        />
            <TextInput
          style={[styles.input, styles.textArea]}
              placeholder="รายละเอียดเนื้อหา..."
              placeholderTextColor="#64748b"
              value={content}
              onChangeText={setContent}
              multiline
        />

       {isSubmitting ? (
              <ActivityIndicator size="small" color="#38bdf8"
style={styles.spinner} />
       ):(
              <TouchableOpacity style={styles.submitButton}
onPress={handleAddNote}>
```

```
              <Text style={styles.submitButtonText}>ซิงก์โพสต์ขึ้นคลาวด์</Text>
            </TouchableOpacity>
        )}
        </View>

        {/* ส่วนแสดงรายการข้อมูลโน้ต */}
        <View style={styles.listContainer}>
          <View style={styles.listHeaderRow}>
            <Text style={styles.listTitle}>บันทึกของคุณในเครื่อง</Text>
            <TouchableOpacity onPress={fetchNotes} disabled={isLoading}>
              <Text style={styles.refreshText}>{isLoading ? 'กำลังโหลด...' :
'🔄 รีเฟรชล่าสุด'}</Text>
            </TouchableOpacity>
      </View>

          {isLoading ? (
            <View style={styles.centerLoading}>
              <ActivityIndicator size="large" color="#38bdf8" />
              <Text style={styles.loadingText}>กำลังติดต่อดึงข้อมูล API...</Text>
            </View>
      ) : (
            <FlatList
              data={notes}
              keyExtractor={(item, index) => item.id.toString() + index}
              renderItem={({ item }) => (
                <View style={styles.noteCard}>
                  <Text style={styles.noteCardTitle}>📌
{item.title}</Text>
                  <Text style={styles.noteCardBody}>{item.body}</Text>
                  <View style={styles.badgeRow}>
                    <View style={styles.syncBadge}>
                      <Text style={styles.badgeText}>ID: {item.id}
(Synced)</Text>
                    </View>
                  </View>
                </View>
        )}
              ListEmptyComponent={
                <View style={styles.emptyContainer}>
                  <Text style={styles.emptyText}>ไม่มีข้อมูลบันทึกในเครื่องขณะนี้</Text>
                </View>
              }
      />
      )}
        </View>
      </KeyboardAvoidingView>
    </SafeAreaView>
  );
}
```

```
const styles = StyleSheet.create({
  safeArea: {
   flex: 1,
    backgroundColor: '#0f172a',
  },
  container: {
    flex: 1,
    padding: 16,
  },
  header: {
    alignItems: 'center',
    marginVertical: 12,
  },
  headerTitle: {
    fontSize: 20,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  headerSubtitle: {
    fontSize: 12,
    color: '#94a3b8',
    marginTop: 4,
  },
  formCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 16,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 16,
  },
  formLabel: {
   color: '#38bdf8',
    fontWeight: 'bold',
    fontSize: 14,
    marginBottom: 10,
  },
  input: {
    backgroundColor: '#0f172a',
    borderRadius: 8,
    borderWidth: 1,
    borderColor: '#334155',
    color: '#ffffff',
    padding: 12,
    fontSize: 14,
    marginBottom: 10,
  },
```

```
  textArea: {
    height: 70,
    textAlignVertical: 'top',
  },
  submitButton: {
    backgroundColor: '#38bdf8',
    borderRadius: 8,
    height: 40,
    justifyContent: 'center',
    alignItems: 'center',
  },
  submitButtonText: {
    color: '#0f172a',
    fontWeight: 'bold',
    fontSize: 14,
  },
  spinner: {
    paddingVertical: 10,
  },
  listContainer: {
    flex: 1,
  },
  listHeaderRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 10,
  },
  listTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  refreshText: {
    color: '#38bdf8',
    fontSize: 13,
  },
  centerLoading: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  loadingText: {
    color: '#94a3b8',
    marginTop: 10,
    fontSize: 13,
  },
  noteCard: {
    backgroundColor: '#1e293b',
```

```
    borderRadius: 12,
    padding: 14,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 12,
  },
  noteCardTitle: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 15,
    textTransform: 'capitalize',
    marginBottom: 6,
  },
  noteCardBody: {
    color: '#cbd5e1',
    fontSize: 13,
    lineHeight: 18,
  },
  badgeRow: {
    flexDirection: 'row',
    marginTop: 10,
  },
  syncBadge: {
    backgroundColor: '#0284c7',
    paddingVertical: 2,
    paddingHorizontal: 8,
    borderRadius: 4,
  },
  badgeText: {
    color: '#ffffff',
    fontSize: 10,
    fontWeight: 'bold',
  },
  emptyContainer: {
    alignItems: 'center',
    paddingVertical: 40,
  },
  emptyText: {
    color: '#64748b',
    fontSize: 14,
  },
});
```

เอกสารอ้างอิง

Expo. (2024). *Using SQLite*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/sqlite/

MDN Web Docs. (2024). *Using the Fetch API*. จาก https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

React Native Async Storage. (2024). *Async Storage Documentation*. จาก https://react-native-async-storage.github.io/async-storage/docs/install/

แผนบริหารการสอนประจำบทที่ 9

หัวข้อเนื้อหาประจำบท

9.1 ระบบสิทธิ์การเข้าถึงอุปกรณ์ (Runtime Permissions)

9.2 การเข้าถึงพิกัดตำแหน่งอุปกรณ์ด้วย expo-location

9.3 การแปลงพิกัดกลับเป็นที่อยู่ (Reverse Geocoding)

9.4 การติดตั้งและจัดแสดงแผนที่ด้วย react-native-maps

9.5 การควบคุมแผนที่ (Region) และการใช้งาน Marker

9.6 การใช้งานเซนเซอร์ตรวจความเร่งด้วย expo-sensors (Accelerometer)

9.7 แนวทางการเขียนดักจับเหตุการณ์ปฏิเสธสิทธิ์และคืนค่าทรัพยากร (Permission Denied & Memory Cleanups)

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถระบุความแตกต่างระหว่าง Foreground และ Background Location Permissions ได้ถูกต้อง
2. ผู้เรียนสามารถใช้คำสั่งดึงค่าพิกัดปัจจุบันและแปลงพิกัดกลับมาเป็นที่อยู่ภาษาไทยผ่าน expo-location ได้สำเร็จ
3. ผู้เรียนสามารถนำเสนอแผนที่พิกัดดาวเทียมผ่านคอมโพเนนต์ MapView ในหน้าจอแอปพลิเคชันได้สำเร็จ
4. ผู้เรียนสามารถเขียนโค้ดปักหมุดแสดงตำแหน่ง (Marker Pin) และระบุขอบข่ายละติจูดลองจิจูดได้ถูกต้อง
5. ผู้เรียนสามารถผูกรับสัญญาณข้อมูลจากเซนเซอร์ตรวจความเร่ง Accelerometer ร่วมกับระบบควบคุมแผนที่ได้ถูกต้อง

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

33. **การบรรยายสถาปัตยกรรมแผนที่และภูมิศาสตร์ (Location Services & Maps Architecture Lecture):** บรรยายการทำงานร่วมกับดาวเทียม (GPS), โครงสร้าง

จัดระเบียบสิทธิ์ตำแหน่ง และการนำคอมโพเนนต์ react-native-maps มาเรนเดอร์ในอุปกรณ์พกพา

34. **การทำ Live Coding Demonstration หัวข้อ Current Location on Interactive Map:** ผู้สอนสาธิตการสร้างหน้าจอแอปแสดงพิกัดแบบเรียลไทม์ โดยดึงค่าตำแหน่งเครื่องปัจจุบันมาเซ็ตค่าศูนย์กลางของ MapView และปักหมุดเป็นสัญลักษณ์รูปบ้าน
35. **กิจกรรมแกะที่อยู่ออกจากพิกัด (Geocoding Coordinates Challenge):** ให้ผู้เรียนทดลองพิมพ์พิกัดละติจูดลองจิจูดจำลอง และเขียนคำสั่งแปลงค่ากลับเป็นรายละเอียดตำบล อำเภอ จังหวัด เพื่อแสดงขึ้นบนจอแผนที่ย่อย
36. **กิจกรรมเขย่าแผนที่ย้ายพิกัด (Shake to Center Device Sensor Activity):** ให้กลุ่มผู้เรียนช่วยกันใช้ความสามารถของ expo-sensors (Accelerometer) ในการจับจังหวะสั่นสะเทือนเครื่อง เพื่อบังคับให้กล้องแผนที่ขยับศูนย์กลางวิ่งกลับมาที่ตำแหน่งพิกัดเครื่องปัจจุบันของผู้ใช้โดยอัตโนมัติ

สื่อการเรียนการสอน

32. เอกสารสไลด์เรื่อง "Geolocation, Map Rendering, and Device Sensors in React Native"
33. อุปกรณ์มือถือสำหรับทดสอบเปิด GPS และเซนเซอร์ความเคลื่อนไหว (หรือตั้งค่า Mock Coordinates ใน Simulator)
34. ตัวอย่างการเซ็ตค่าสิทธิ์ใน app.json สำหรับเปิดบริการ Location
35. ชุดรหัสพิกัดจำลองสถานที่ท่องเที่ยวสำคัญเพื่อใช้ฝึกฝนการปักหมุด

การวัดผลและการประเมินผล

1. การวัดผล

1.1 ประเมินผลคะแนนการทำแบบทดสอบท้ายบทเรียน 10 ข้อ

1.2 ตรวจสอบโครงสร้างความถูกต้องของการเรียกขอสิทธิ์ Location และการกำหนดสเตตัสในโค้ดแผนที่

1.3 สังเกตขั้นตอนการคืนค่า (Unsubscribe) เซนเซอร์ Accelerometer ตอนปิด

หน้าจอเพื่อความปลอดภัยไม่ให้แรมรั่ว

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถสร้างหน้าจอแผนที่ที่ปักหมุดพิกัดปัจจุบันและแจ้งที่อยู่ถนนจริงของผู้ใช้ได้อย่างราบรื่นผ่านเกณฑ์ (ผ่าน/ไม่ผ่าน)

2.3 สังเกตพฤติกรรมความใส่ใจในการเขียนป้องกันเหตุฉุกเฉินเมื่อเครื่องไม่มีบริการ GPS หรือเมื่อผู้ใช้อนุมัติสิทธิ์ไม่สำเร็จ

บทที่ 9

พิกัดตำแหน่ง แผนที่ และเซนเซอร์อุปกรณ์

การพัฒนาแอปพลิเคชันให้สามารถโต้ตอบกับโลกกายภาพภายนอก จำเป็นต้องอาศัยกลไกการเชื่อมโยงระบบพิกัดดาวเทียม (GPS) การจัดแสดงกราฟิกแผนที่เชิงภูมิศาสตร์ และการดักรับสัญญาณเซนเซอร์ของมือถือ ในบทเรียนนี้เราจะศึกษาการจัดการระบบขออนุญาตสิทธิ์ตำแหน่งทางภูมิศาสตร์ (Location Permissions) การเข้าถึงพิกัดด้วย `expo-location`, การเรนเดอร์แผนที่ด้วย `react-native-maps` และการใช้งานเซนเซอร์วัดแรงเร่ง `expo-sensors` (Accelerometer) ในระดับมาตรฐานสากล

9.1 ระบบสิทธิ์การเข้าถึงอุปกรณ์ (Runtime Permissions)

ในการเข้าถึงทรัพยากรที่มีความปลอดภัยและข้อมูลส่วนบุคคลของผู้ใช้งาน (เช่น พิกัดแผนที่ตำแหน่ง GPS) ระบบปฏิบัติการทั้ง iOS และ Android บังคับใช้ระบบ **สิทธิ์การเข้าถึงเวลาทำงาน (Runtime Permissions)** ซึ่งหมายความว่าแอปพลิเคชันไม่สามารถดึงข้อมูลได้ทันทีเมื่อเปิดใช้งาน แต่ต้องแสดงหน้าต่างขออนุมัติสิทธิ์ (Permission Request Pop-up) จากผู้ใช้เสียก่อน

ประเภทของสิทธิ์เข้าถึงพิกัดตำแหน่ง (Location Permission Types):

1. **Foreground Location (ตำแหน่งเบื้องหน้า):** อนุญาตให้เข้าถึงข้อมูล GPS เฉพาะในขณะที่ผู้ใช้กำลังเปิดเปิดหน้าจอแอปพลิเคชันทำงานอยู่เท่านั้น (เป็นสิทธิ์หลักที่เรามักร้องขอ)
2. **Background Location (ตำแหน่งเบื้องหลัง):** อนุญาตให้แอปพลิเคชันคอยติดตามข้อมูล GPS ตลอดเวลาแม้แอปพลิเคชันจะถูกพับปิดหน้าต่างไปแล้ว (ต้องขอสิทธิ์ Foreground สำเร็จก่อนและผ่านเกณฑ์พิจารณาพิเศษจากสโตร์)

สำหรับระบบ Expo SDK 54 การขอสิทธิ์และตรวจสอบสถานะจะถูกควบคุมอย่างเป็นระบบผ่านคำสั่งสัญญา Asynchronous (Promise-based) ซึ่งส่งคืนค่าสถานะเป็นออบเจกต์ที่มีพร็อพส์ `status` บ่งชี้ผลลัพธ์:

- `'granted'`: ได้รับอนุมัติสิทธิ์สำเร็จ ดึงข้อมูลฮาร์ดแวร์ได้
- `'denied'`: ถูกปฏิเสธสิทธิ์ ไม่สามารถเข้าใช้ฮาร์ดแวร์ได้
- `'undetermined'`: ยังไม่เคยร้องขอสิทธิ์ (ผู้ใช้เปิดแอปครั้งแรก)

9.2 การเข้าถึงพิกัดตำแหน่งอุปกรณ์ด้วย expo-location

เพื่อดึงพิกัดตำแหน่งละติจูดและลองจิจูดจากอุปกรณ์เนทีฟ เราจะใช้งานไลบรารีอย่างเป็นทางการของ Expo คือ `expo-location` (Expo, 2024)

การติดตั้งไลบรารี:

```
npx expo install expo-location
```

การตั้งค่าขออนุญาตใน app.json (สำหรับ Android & iOS):

```
{
 "expo": {
  "plugins": [
   [
    "expo-location",
        {
     "locationAlwaysPermission": "อนุญาตให้แอปเข้าถึงตำแหน่งของคุณได้ตลอดเวลาเพื่อซิงก์แผนที่",
     "locationWhenInUsePermission": "อนุญาตให้เข้าถึงตำแหน่งในขณะเปิดใช้งานแอปเพื่อปักหมุดพิกัด"
        }
   ]
  ]
 }
}
```

การเขียนโค้ดดึงตำแหน่ง GPS ล่าสุด:

```
import React, { useState, useEffect } from 'react';
import { StyleSheet, Text, View, ActivityIndicator } from 'react-native';
import * as Location from 'expo-location';

export default function CurrentLocationApp() {
  const [location, setLocation] = useState<Location.LocationObject |
null>(null);
  const [errorMsg, setErrorMsg] = useState<string | null>(null);

  useEffect(() => {
    const requestLocation = async () => {
   // 1. ส่งคำขออนุมัติสิทธิ์เข้าถึงพิกัดตำแหน่งเบื้องหน้า (Foreground)
      let { status } = await Location.requestForegroundPermissionsAsync();
      if (status !== 'granted') {
        setErrorMsg('การเข้าถึงตำแหน่งของคุณถูกปฏิเสธ');
        return;
      }

   // 2. เมื่อได้รับอนุญาต ให้ทำการดึงพิกัดตำแหน่งละติจูดลองจิจูดปัจจุบัน
      let currentLocation = await Location.getCurrentPositionAsync({});
      setLocation(currentLocation);
    };

    requestLocation();
  }, []);
```

```
  return (
    <View style={styles.container}>
      {errorMsg ? (
        <Text style={styles.errorText}>{errorMsg}</Text>
    ) : location ? (
        <View>
          <Text style={styles.text}>Latitude:
{location.coords.latitude}</Text>
          <Text style={styles.text}>Longitude:
{location.coords.longitude}</Text>
          <Text style={styles.subText}>ระดับความแม่นยำ:
{location.coords.accuracy} เมตร</Text>
        </View>
    ) : (
        <ActivityIndicator size="large" color="#38bdf8" />
    )}
    </View>
 );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center',
backgroundColor: '#0f172a' },
  text: { fontSize: 18, color: '#ffffff', fontWeight: 'bold', marginBottom:
8 },
  subText: { fontSize: 13, color: '#94a3b8' },
  errorText: { color: '#ef4444', fontSize: 15 },
});
```

9.3 การแปลงพิกัดกลับเป็นที่อยู่ (Reverse Geocoding)

ข้อมูลละติจูดและลองจิจูด (เช่น 13.7367, 100.5231) เป็นข้อมูลที่ยากต่อการทำความเข้าใจของผู้ใช้ทั่วไป ไลบรารี **expo-location** จึงได้นำเสนอคุณสมบัติ **Reverse Geocoding** ซึ่งทำหน้าที่แปลงพิกัดทางภูมิศาสตร์ตัวเลขให้กลายเป็นออบเจกต์ที่อยู่จริงของที่ตั้งปลายทาง เช่น ชื่ออาคาร, เลขที่ถนน, อำเภอ, จังหวัด และรหัสไปรษณีย์

ตัวอย่างคำสั่งแปลงพิกัด:

```
const getAddressFromCoords = async (latitude: number, longitude: number)
=> {
  try {
  // ส่งละติจูดและลองจิจูดไปถอดข้อมูลภูมิภาค
    const addressResponse = await Location.reverseGeocodeAsync({
      latitude: latitude,
      longitude: longitude,
  });
```

```
    if (addressResponse.length > 0) {
      const address = addressResponse[0];
   // ตัวอย่างการดึงฟิลด์ข้อมูลที่อยู่
      console.log(`ที่อยู่: ถนน ${address.street}, อำเภอ ${address.district}, จังหวัด
${address.region}`);
    }
  } catch (error) {
    console.error('การแปลงที่อยู่ล้มเหลว:', error);
  }
};
```

9.4 การติดตั้งและจัดแสดงแผนที่ด้วย react-native-maps

ในการเรนเดอร์ภาพแผนที่เชิงโต้ตอบ (Interactive Maps) ที่ผู้ใช้งานสามารถย่อขยาย ดึงแผนที่ และมองเห็นความตื้นลึกของระดับสิ่งปลูกสร้างอย่างไหลลื่น เราจะติดตั้งใช้งานไลบรารีภายนอกยอดนิยมคือ **`react-native-maps`** (React Native Maps, 2024)

การติดตั้ง:

```
npx expo install react-native-maps
```

คอมโพเนนต์หลักที่ใช้งาน:

• **`<MapView>`**: คอนเทนเนอร์หลักที่ใช้สำหรับจำลองเรนเดอร์แผนที่ภูมิศาสตร์ลงบนหน้าจออุปกรณ์

• **`<Marker>`**: คอมโพเนนต์ลูกย่อยที่ถูกเรนเดอร์ไว้ข้างใน MapView เพื่อแสดงสัญลักษณ์หมุดปัก (Map Pin) ณ จุดพิกัดใดพิกัดหนึ่งโดยเฉพาะ

9.5 การควบคุมแผนที่ (Region) และการใช้งาน Marker

การกำหนดมุมกล้องและพิกัดซูมบนคอมโพเนนต์ **`<MapView>`** จะถูกควบคุมผ่านพร็อพส์ **`region`** ซึ่งจะรับออบเจกต์ประกอบด้วยข้อมูล 4 ส่วนหลัก:

1. **`latitude`**: ละติจูดของพิกัดที่จะให้เป็นกึ่งกลางหน้าจอแผนที่
2. **`longitude`**: ลองจิจูดของพิกัดที่จะให้เป็นกึ่งกลางหน้าจอแผนที่
3. **`latitudeDelta`**: ระยะการซูมในแนวตั้ง (ค่ายิ่งน้อย ยิ่งซูมเข้าลึก มองเห็นรายละเอียดถนนและซอยชัดเจน)
4. **`longitudeDelta`**: ระยะการซูมในแนวนอน (ค่ายิ่งน้อย ยิ่งซูมเข้าลึก)

```
import React from 'react';
```

```
import { StyleSheet, View } from 'react-native';
import MapView, { Marker } from 'react-native-maps';

export default function SimpleMap() {
  return (
    <View style={styles.container}>
      <MapView
        style={styles.map}
    // กำหนดตำแหน่งศูนย์กลางกล้องแผนที่เริ่มต้น (กรุงเทพมหานคร)
        initialRegion={{
          latitude: 13.7563,
          longitude: 100.5018,
          latitudeDelta: 0.05,
          longitudeDelta: 0.05,
        }}
      >
        {/* ปักหมุดตำแหน่ง ณ เสาชิงช้า */}
        <Marker
          coordinate={{ latitude: 13.7519, longitude: 100.5012 }}
          title="เสาชิงช้า"
          description="จุดเช็คอินใจกลางกรุงเทพมหานคร"
    />
      </MapView>
    </View>
 );
}

const styles = StyleSheet.create({
  container: { flex: 1 },
  map: { width: '100%', height: '100%' },
});
```

9.6 การใช้งานเซนเซอร์ตรวจความเร่งด้วย expo-sensors (Accelerometer)

อุปกรณ์สมาร์ทโฟนสมัยใหม่ทุกเครื่องจะมีชิปเซนเซอร์ภายในเครื่องสำหรับวัดมุม การหมุน และความเร่ง ไลบรารี `expo-sensors` (Expo, 2024) ช่วยให้ผู้พัฒนาสามารถอ่านค่าพฤติกรรมกายภาพของมือถือได้ง่าย

การดาวน์โหลดแพ็คเกจ:

```
npx expo install expo-sensors
```

การใช้งาน Accelerometer (เซนเซอร์วัดแรงเร่ง):

เซนเซอร์นี้จะคอยส่งชุดตัวเลขตามแกนพิกัดสามมิติ (x, y, z) อิงตามแรงโน้มถ่วงและการเคลื่อนที่จริงของตัวเครื่อง:

- **แกน x:** เอียงเครื่องไปซ้าย/ขวา
- **แกน y:** เอียงเครื่องไปหน้า/หลัง
- **แกน z:** แรงดึงแนวตั้งของการถือเครื่อง

```
import React, { useState, useEffect } from 'react';
import { StyleSheet, Text, View } from 'react-native';
import { Accelerometer } from 'expo-sensors';

export default function MotionSensorApp() {
  const [data, setData] = useState({ x: 0, y: 0, z: 0 });
  const [subscription, setSubscription] = useState<any>(null);

 // 1. สั่งเปิดรับฟังการเปลี่ยนแปลงข้อมูลจากชิปเซนเซอร์
  const subscribe = () => {
    setSubscription(
      Accelerometer.addListener((accelerometerData) => {
        setData(accelerometerData);
      })
  );
  // ตั้งรอบเวลาตรวจรับข้อมูลทุก ๆ 200 มิลลิวินาที
    Accelerometer.setUpdateInterval(200);
  };

 // 2. ยกเลิกการดักฟลักซ์ข้อมูลเพื่อเซฟแบตเตอรี่เครื่อง
  const unsubscribe = () => {
    subscription && subscription.remove();
    setSubscription(null);
  };

  useEffect(() => {
    subscribe();
  // คืนค่าฟังก์ชัน Unsubscribe สำหรับ Cleanup ตอนปิดหน้าจอป้องกัน RAM รั่ว
    return () => unsubscribe();
  }, []);

  return (
    <View style={styles.container}>
      <Text style={styles.title}>ค่าความเร่ง Accelerometer</Text>
      <Text style={styles.coord}>X: {data.x.toFixed(3)}</Text>
      <Text style={styles.coord}>Y: {data.y.toFixed(3)}</Text>
      <Text style={styles.coord}>Z: {data.z.toFixed(3)}</Text>
    </View>
 );
}
```

```
const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center',
backgroundColor: '#0f172a' },
  title: { fontSize: 18, color: '#38bdf8', fontWeight: 'bold',
marginBottom: 12 },
  coord: { fontSize: 24, color: '#ffffff', fontFamily: 'monospace',
marginVertical: 4 },
});
```

9.7 แนวทางการเขียนดักจับเหตุการณ์ปฏิเสธสิทธิ์และคืนค่าทรัพยากร (Permission Denied & Memory Cleanups)

กฎเหล็กสำคัญสองประการในการป้องกันไม่ให้โมบายแอปพลิเคชันเกิดอาการค้างหรือกินหน่วยความจำมหาศาลจนแอปเด้งดับ มีดังนี้:

1. การจัดการกรณีสิทธิ์การเข้าถึงถูกปฏิเสธ (Permission Rejection Fallback)

หากผู้ใช้งานไม่ยินยอมเปิด GPS ให้แอปพลิเคชัน ผู้พัฒนาไม่ควรปล่อยให้แอปพลิเคชันเรนเดอร์แผนที่ว่างเปล่าหรือค้างหน้าจอขาว แต่ต้องสลับ UI มาแสดงกล่องข้อความอธิบายความจำเป็นของการใช้พิกัด พร้อมดีไซน์ปุ่มกด "ตั้งค่าอุปกรณ์ (Open Settings)" เพื่อนำทางด่วนไปหน้าตั้งค่าระบบปฏิบัติการของเครื่อง:

```
import * as Linking from 'expo-linking';
import { Button, View, Text } from 'react-native';

const handleOpenSettings = () => {
 // สั่งสั่งระบบทางตรงไปที่แอปพลิเคชันตั้งค่าสิทธิ์บนโทรศัพท์
  Linking.openSettings();
};
```

2. การล้างและปิดสตรีมเซนเซอร์ (Memory Cleanups)

เซนเซอร์ต่าง ๆ (เช่น Accelerometer, Gyroscope) ทำการสตรีมฟีดค่าพารามิเตอร์ขึ้นมาอัปเดตตลอดเวลา หากเปิดทริกเกอร์ฟังข้อมูลไว้แต่มีการเปลี่ยนไปหน้าอื่น สตรีมนี้จะยังทำงานค้างอยู่เบื้องหลัง ดึงแรมและกินแบตเตอรี่จนเครื่องร้อนจัด ผู้พัฒนาจำเป็นต้องคืนค่าและระงับการทำงาน (Unsubscribe) ใน Cleanup Function ของ React **useEffect** เสมอ

สรุปท้ายบทเรียน

ระบบสิทธิ์การเข้าถึงอุปกรณ์ (Runtime Permissions): การรักษาความปลอดภัยด้านความเป็นส่วนตัวบังคับให้แอปพลิเคชันต้องขอสิทธิ์เข้าถึงพิกัด GPS ระหว่างรันแอปผ่านระบบ Runtime Permissions ซึ่งสามารถขอสิทธิ์แบบเบื้องหน้า (Foreground) หรือเบื้องหลัง (Background) โดยระบบ Expo SDK 54 จัดการสถานะสิทธิ์การเข้าถึงส่งคืนค่าเป็นสัญญา Asynchronous ที่ส่งผลลัพธ์หลักเป็น granted (ได้รับสิทธิ์), denied (ถูกปฏิเสธ) หรือ undetermined (ยังไม่ร้องขอ)

การเข้าถึงพิกัดตำแหน่งอุปกรณ์ด้วย expo-location: การค้นหาพิกัดละติจูดและลองจิจูดของโทรศัพท์ใช้ไลบรารี expo-location ซึ่งต้องลงทะเบียนขออนุญาตในไฟล์ตั้งค่าโครงการ app.json ก่อน และใช้คำสั่ง requestForegroundPermissionsAsync ร่วมกับ getCurrentPositionAsync ในการดักจับพิกัดละติจูดและลองจิจูดปัจจุบัน รวมถึงระดับค่าความแม่นยำของจุดปักพิกัดสัญญาณดาวเทียมเพื่อนำไปใช้แสดงผลต่อ

การแปลงพิกัดกลับเป็นที่อยู่ (Reverse Geocoding): เนื่องจากพิกัดตัวเลขทางภูมิศาสตร์เข้าใจยากสำหรับผู้ใช้งานทั่วไป expo-location ได้จัดเตรียมความสามารถ Reverse Geocoding ผ่านฟังก์ชัน reverseGeocodeAsync ในการเปลี่ยนค่าละติจูดและลองจิจูดเป็นที่อยู่จริงของอุปกรณ์ เช่น ชื่อถนน ตำบล อำเภอ จังหวัด และรหัสไปรษณีย์ เพื่อแปลผลพิกัดเป็นข้อมูลที่แสดงผลแบบสื่อสารเข้าใจได้ง่ายบนอินเทอร์เฟซ

การติดตั้งและจัดแสดงแผนที่ด้วย react-native-maps: การแสดงแผนที่เชิงภูมิศาสตร์แบบโต้ตอบได้เต็มรูปแบบจะใช้ไลบรารี react-native-maps (React Native Maps, 2024) โดยประกอบไปด้วยคอมโพเนนต์หลัก ได้แก่ MapView ในการทำหน้าที่วาดหน้าต่างแผนที่ลงบนเฟรมหน้าต่างแอปพลิเคชัน และคอมโพเนนต์ย่อยอย่าง Marker เพื่อระบุพิกัดปักหมุดสีแดง (Map Pin) ณ ละติจูดและลองจิจูดเป้าหมายพร้อมเขียนบรรยายรายละเอียดกำกับ

การควบคุมแผนที่ (Region) และการใช้งาน Marker: การควบคุมการแสดงผลและปรับมุมกล้องแผนที่จะทำงานผ่านคุณสมบัติ Region ใน MapView ประกอบด้วยค่าศูนย์กลางละติจูด-ลองจิจูด และค่าความกว้างของการซูมอย่าง latitudeDelta และ longitudeDelta ซึ่งค่ายิ่งน้อยจะยิ่งซูมเข้าสู่ผิวถนนได้อย่างชัดเจน และเราสามารถปักหมุด Marker ระบุข้อมูล coordinate รวมถึงพิมพ์คำอธิบายหัวข้อจุดเช็คอินต่าง ๆ ได้อย่างอิสระ

การใช้งานเซนเซอร์ตรวจความเร่งด้วย expo-sensors: การเขียนรหัสรับรู้พฤติกรรมการเคลื่อนไหวจริงของโทรศัพท์ใช้ไลบรารี expo-sensors ในการอ่านค่าจากชิป Accelerometer เพื่อฟีดพารามิเตอร์ความเร่งสามแกน (x, y, z) สะท้อนแรงโน้มถ่วงเมื่อพลิก เอียง หรือขยับโทรศัพท์

โดยต้องระบุช่วงความถี่ในการตรวจสอบผ่าน setUpdateInterval เพื่อสตรีมข้อมูลความเคลื่อนไหวมาแสดงผลแบบเรียลไทม์

แนวทางการเขียนดักจับเหตุการณ์ปฏิเสธสิทธิ์และคืนค่าทรัพยากร: โครงสร้างความปลอดภัยของแอปพลิเคชันที่ดีต้องเตรียมระบบรองรับกรณีผู้ใช้ปฏิเสธสิทธิ์โดยใช้ Linking.openSettings() เพื่อพาลัดเข้าหน้าการตั้งค่าเครื่อง และที่สำคัญที่สุดคือการป้องกันปัญหาระบบเมมโมรี่รั่วไหล (Memory Leak) จากฟีดข้อมูลเซนเซอร์ โดยการสั่งยกเลิกติดตามเซนเซอร์ (Unsubscribe) ในส่วนของ Cleanup Function ของ React useEffect ก่อนเปิดหน้าจอแอปพลิเคชันถัดไป

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความเข้าใจเกี่ยวกับกลไกขออนุญาตเข้าใช้ฮาร์ดแวร์ (Location Permissions), การค้นหาพิกัดดาวเทียมปัจจุบันและการแปลงค่าเป็นที่อยู่ภาษาไทย (`expo-location`), การควบคุมมุมซูมและปักหมุดบนแผนที่ (`react-native-maps`), และการรับฟีดข้อมูลเซนเซอร์เคลื่อนไหวรวมถึงกลไกป้องกันหน่วยความจำรั่วไหล (`expo-sensors`) ในระบบปฏิบัติการโทรศัพท์มือถือ

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดอธิบายความแตกต่างระหว่างสิทธิ์การเข้าถึงตำแหน่งชนิด Foreground และ Background Location ได้ถูกต้อง?

ก. Foreground ใช้ได้เฉพาะในระบบ Android ส่วน Background ใช้ได้เฉพาะระบบ iOS

ข. Foreground อนุญาตให้อ่าน GPS เฉพาะขณะเปิดใช้งานแสดงผลหน้าจอแอปพลิเคชัน ส่วน Background อนุญาตให้ติดตามข้อมูลได้ตลอดเวลาแม้แอปจะพับปิดไปเบื้องหลัง

ค. Foreground บังคับใช้เฉพาะสัญญาณอินเทอร์เน็ต WiFi ส่วน Background ใช้กับเครือข่ายดาวเทียมเท่านั้น

ง. Foreground ใช้ดึงข้อมูลพิกัดความเร็วเครื่อง ส่วน Background ใช้บันทึกภาพถ่ายสถานที่

ข้อ 2: เมื่อต้องการขอสิทธิ์เข้าใช้งานพิกัดตำแหน่งเป็นครั้งแรกในขณะที่ผู้เรียนกำลังรันแอป ผู้พัฒนาต้องเรียกใช้งานคำสั่งใดจากไลบรารี `expo-location`?

ก. `Location.getCurrentPositionAsync()`

ค. `Location.requestForegroundPermissionsAsync()`

ค. `Location.reverseGeocodeAsync()`

ง. `Location.openSettings()`

ข้อ 3: เมธอดใดของ `expo-location` ที่ใช้ทำหน้าที่แปลงตัวเลขพิกัดละติจูดและลองจิจูด (Latitude, Longitude) ให้กลายเป็นที่อยู่สิ่งปลูกสร้างหรือถนนจริงภาษาไทย?

ก. `Location.getCurrentPositionAsync`

ข. `Location.requestBackgroundPermissionsAsync`

ค. `Location.reverseGeocodeAsync`

ง. `Location.geocodeAsync`

ข้อ 4: ข้อใดบ่งบอกถึงวัตถุประสงค์หลักของการกำหนดตัวแปร `latitudeDelta` และ `longitudeDelta` ในพร็อพส์ `region` ของคอมโพเนนต์ `<MapView>`?

ก. กำหนดความเร็วในการเลื่อนหมุนแผนที่เมื่อผู้ใช้ใช้นิ้วปัด

ข. กำหนดระดับช่วงการมองเห็นหรือระยะการซูมเข้า/ซูมออกของกล้องแผนที่ (ค่ายิ่งน้อย ยิ่งซูมขยายเข้าลึก)

ค. กำหนดพิกัดทางภูมิศาสตร์ของทิศเหนือและทิศใต้เพื่อล้างค่าขอบเขต

ง. กำหนดมุมเอียงแบบสามมิติของตึกระฟ้าบนแผนที่

ข้อ 5: หากต้องการปักหมุดจุดสัญลักษณ์ เช่น รูปภาพเข็มหมุดสีแดง เพื่อเน้นจุดพิกัดร้านค้าหรือบ้านในแผนที่ `<MapView>` ผู้พัฒนาต้องเรียกใช้งานคอมโพเนนต์ลูกตัวใด?

ก. `<MapView.Circle>`

ข. `<Marker>`

ค. `<View>`

ง. `<Overlay>`

ข้อ 6: พร็อพส์ (Props) ตัวใดที่เป็นตัวคุมละติจูดและลองจิจูดของตำแหน่งหมุดในคอมโพเนนต์ `<Marker>`?

ก. `region`

ข. `initialRegion`

ค. `coordinate`

ง. `center`

ข้อ 7: ชิปเซนเซอร์ "Accelerometer" ในระบบปฏิบัติการโทรศัพท์เคลื่อนที่ คอยส่งมอบชุดตัวเลขชนิดใดเพื่อรายงานพฤติกรรมทางกายภาพของตัวเครื่อง?

ก. ระบุค่าความเร็วการเชื่อมต่อสัญญาณอินเทอร์เน็ตเฉลี่ย 3 ทิศทาง

ข. ระบุค่าความเร่งตามแกนสามมิติ (x, y, z) สัมพันธ์กับแรงโน้มถ่วงและการสั่นสะเทือนของเครื่อง

ค. ระบุทิศทางมุมองศาของเข็มทิศขั้วแม่เหล็กโลกทางภูมิศาสตร์

ง. ระบุปริมาณความกดอากาศและความชื้นสัมพัทธ์ในห้อง

ข้อ 8: เมธอดใดของเซนเซอร์ `Accelerometer` ที่ใช้สำหรับลงทะเบียนและเริ่มต้นดักจับการสั่นสะเทือนเครื่องแบบเรียลไทม์?

ก. `Accelerometer.setUpdateInterval()`

ข. `Accelerometer.addListener()`

ค. `Accelerometer.remove()`

ง. `Accelerometer.removeAllListeners()`

ข้อ 9: เหตุใดการเขียนฟังก์ชัน cleanup คืนค่า `subscription.remove()` ภายใน `useEffect` จึงมีความสำคัญระดับสูงสุดเมื่อเรียกใช้งานเซนเซอร์ในแอป React Native?

ก. เพื่อไม่ให้ตัวแปรของสไตล์ชีทพังเมื่อเข้าสู่โหมดมืด

ข. เพื่อปิดกระบวนการรับฟีดข้อมูลที่ค้างคาอยู่ในแรม (RAM) ช่วยเซฟแบตเตอรี่เครื่องและป้องกันปัญหาเครื่องค้างเนื่องจากหน่วยความจำรั่วไหล (Memory Leak)

ค. เพื่อสั่งดาวน์โหลดฟอนต์ Prompt ใหม่เข้ามาทำงานทดแทน

ง. เพื่อสุ่มเปลี่ยนรูปภาพแผนที่ให้กลายเป็นสกินดาวเทียม

ข้อ 10: หากผู้ใช้เผลอกดปฏิเสธไม่ยอมอนุมัติให้แอปเข้าถึง Location และเราต้องการเขียนปุ่มลัดเพื่ออำนวยความสะดวกให้นำทางผู้ใช้ไปยังหน้าตั้งค่าสิทธิ์บนโทรศัพท์โดยตรง เราควรเลือกใช้เครื่องมือใดของ Expo?

ก. `Linking.openURL()`

ข. `Linking.openSettings()`

ค. `Location.requestForegroundPermissionsAsync()`

ง. `Location.getCurrentPositionAsync()`

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: ทั้ง iOS และ Android ต่างสนับสนุนโมเดล Foreground และ Background ทั้งคู่

• **ข. ถูก:** Foreground จะดึงพิกัดได้เฉพาะขณะผู้ใช้เปิดแอปค้างไว้บนจอ ส่วน Background ดึงได้ตลอดเวลาแม้แอปพลิเคชันจะรันอยู่เงียบ ๆ ด้านหลัง

• **ค. และ ง. ผิด:** ไม่เกี่ยวข้องกับคลื่นสัญญาณ และไม่ใช่เครื่องมือเปิดกล้องบันทึกภาพถ่าย

เฉลย ข้อ 2: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `getCurrentPosition` ใช้ดึงค่าพิกัด แต่จะรันพังทันทีหากยังไม่ได้รับสิทธิ์อนุมัติ

• **ข. ถูก:** คำสั่งที่มี `request` นำหน้า เช่น `Location.requestForegroundPermissionsAsync()` จะส่งสัญญาณเรียกหน้าต่าง Pop-up ดั้งเดิมของระบบปฏิบัติการขึ้นมาสัมภาษณ์สิทธิ์จากผู้ใช้งานแอป

• **ค. และ ง. ผิด:** ใช้สำหรับแปลงค่าที่อยู่ และลิงก์ไปหน้าตั้งค่า ไม่ใช่หน้าต่างกล่องป๊อปอัพตัวเริ่มต้นขอสิทธิ์

เฉลย ข้อ 3: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ดึงพิกัดตัวเลขไม่ใช่ที่อยู่

• **ข. ผิด:** เป็นคำสั่งขอสิทธิ์ตำแหน่งเบื้องหลัง

• **ค. ถูก:** Reverse Geocoding คือเมธอดแปลงจากพิกัด (Coords) กลับคืนเป็น Address (ที่อยู่อักษร)

• **ง. ผิด:** GeocodeAsync จะทำงานกลับทิศกัน (แปลงจากที่อยู่ตัวหนังสือให้กลายเป็นเลขพิกัดละติจูดลองจิจูด)

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** ไม่เกี่ยวกับสปีดการปัดนิ้ว และไม่ใช่ระบบจัดองศาสามมิติตึก

• **ข. ถูก:** Delta เป็นตัวชี้วัดความกว้างของช่วงพิกัดที่กล้องจะเก็บภาพมาเรนเดอร์ ตัวเลขยิ่งน้อยแสดงว่าสัดส่วนกล้องจะขยายโฟกัสซูมเข้าไปใกล้ขอบทางเดินได้ลึกและละเอียดขึ้น

เฉลย ข้อ 5: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** Circle วาดพื้นที่วงกลมครอบรัศมีบนแผนที่

• **ข. ถูก:** คอมโพเนนต์ `<Marker>` เป็นหัวใจของการระบุจุดสำคัญลงบนแผนที่

• **ค. และ ง. ผิด:** View เป็นบล็อกจัดหน้าตาปกติ และ Overlay ใช้ระบุแผ่นพื้นที่ทับซ้อน

เฉลย ข้อ 6: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. และ ข. ผิด:** เป็นตัวคุมแผนที่ MapView ทั้งหมด ไม่ใช่ขอบของหมุดย่อยตัวเดียว

• **ค. ถูก:** พร็อพส์ `coordinate` ของ Marker รับอ็อบเจกต์ที่มีโครงสร้าง `{ latitude: number, longitude: number }` เพื่อนำเข็มหมุดปักตรงจุด

• **ง. ผิด:** ไม่มีคีย์ชื่อ center ในข้อกำหนดมาตรฐาน

เฉลย ข้อ 7: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** ไม่สามารถอ่านค่าเน็ต หรือความกดชื้นอากาศในเครื่องได้ และทิศทางแม่เหล็กโลกเป็นหน้าที่ของชิป Magnetometer (เข็มทิศ)

• **ข. ถูก:** เซนเซอร์วัดแรงเร่ง Accelerometer จับแรงสั่นสะเทือนรอบแกน X (ซ้ายขวา) แกน Y (หน้าหลัง) และแกน Z (แรงโน้มถ่วงแนวตั้ง) เพื่อนำมารายงานพฤติกรรมการขยับเครื่อง

เฉลย ข้อ 8: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `setUpdateInterval` คุมระยะห่างความเร็วการยิงค่า ไม่ใช่การเริ่มสตรีมรับฟัง

• **ข. ถูก:** `addListener` จะเป็นการสมัคร (Subscribe) รับฟังฟลักซ์ข้อมูลจากเซนเซอร์ โดยจะส่ง callback ผลลัพธ์กลับมาเมื่อตรวจพบการเปลี่ยนแปลงตำแหน่ง

• **ค. และ ง. ผิด:** ใช้สำหรับระงับการทำงาน

เฉลย ข้อ 9: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** ไม่ได้เกี่ยวกับโหมดสี ฟอนต์อักษร หรือสกินแผนที่ผ่านดาวเทียม

• **ข. ถูก:** การสตรีมฟีดเซนเซอร์เป็นงานที่ค้างในระบบเบื้องหลังของฮาร์ดแวร์ หากไม่ยอม Unsubscribe ตอนปิดหน้าจอนั้น ๆ หรือปิดแอป สตรีมจะดูดแรมและประมวลผลตลอดไป ส่งผลให้แบตเตอรี่เครื่องเสื่อมไว ร้อนจัด และแอปเด้งดับ

เฉลย ข้อ 10: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `openURL` ใช้ลิงก์ไปหน้าเว็บไซต์ภายนอก

• **ข. ถูก:** `Linking.openSettings()` เป็นคำสั่งลัดพาตัวข้ามระบบ Sandbox ของแอปเพื่อไปเปิดหน้าควบคุมสิทธิ์เข้าถึงของแอปนั้น ๆ ในเมนูตั้งค่าหลักของสมาร์ทโฟน สะดวกต่อการให้ผู้ใช้เปิดสิทธิ์ทดแทน

• **ค. และ ง. ผิด:** เป็นเพียงคำสั่งขอสิทธิ์และอ่านค่าพิกัดปกติ ซึ่งจะรันไม่สำเร็จหากผู้ใช้ปิดสิทธิ์ไปก่อนหน้าแล้ว

ปฏิบัติการท้ายบทเรียน: การพัฒนาแอปพลิเคชันปักหมุดแผนที่และสั่นสะเทือนจดจำตำแหน่ง

ปฏิบัติการนี้จะฝึกทักษะการรวบรวมฟังก์ชันระดับฮาร์ดแวร์ของโทรศัพท์มือถือเข้าหากัน โดยผู้เรียนจะได้ออกแบบแอปพลิเคชัน **"ระบบแผนที่นำทางและสั่นสะเทือนจดจำพิกัด (Interactive Location Map Tracker)"** ซึ่งจะร้องขอสิทธิ์ดาวเทียม ดึงตำแหน่ง GPS ของเครื่องมาประมวลผล ถอดรหัสแปลงกลับเป็นชื่อสถานที่จริง แสดงผลแผนที่เชิงโต้ตอบแบบซูมลึก ปักหมุดตำแหน่งปัจจุบัน และผูกเซนเซอร์ความเร่งตรวจจับความเคลื่อนไหว (Accelerometer) เมื่อผู้ใช้ทำมุมเอียงหรือเขย่าเครื่อง เพื่อบังคับมุมกล้องแผนที่ให้ดีดสไลด์กลับมาโฟกัสพิกัดผู้ใช้ปัจจุบันแบบไดนามิก

1. วัตถุประสงค์การเรียนรู้

1. สามารถเขียนรหัสควบคุมระบบสิทธิ์การเข้าถึงพิกัดตำแหน่ง (Runtime Location Permission) และเขียนระบบป้องกันเมื่อผู้ใช้ปฏิเสธสิทธิ์

2. สามารถใช้คุณสมบัติ Geocoding แปลงละติจูดลองจิจูดให้ออกมาเป็นข้อมูลตำบลและอำเภอจริงเพื่อจัดแสดงผลบนหน้าจอ

3. สามารถกำหนดคอนเทนเนอร์แผนที่ `<MapView>` ร่วมกับสัญลักษณ์ `<Marker>` โดยตั้งค่าระยะซูม Delta ที่เหมาะสมคมชัด

4. สามารถเชื่อมโยงการทำงานร่วมกับเซนเซอร์ความเร่ง `Accelerometer` เพื่อสั่งขยับพิกัดกล้องแผนที่ตามการสั่นเครื่องได้สำเร็จ

5. สามารถเขียนโค้ดระงับสตรีมเซนเซอร์ (Cleanup listeners) ได้ถูกต้องเพื่อประสิทธิภาพของทรัพยากรระบบ

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องพัฒนาแอปพลิเคชันระบบแผนที่และพิกัดตำแหน่งส่วนบุคคลตามเงื่อนไขต่อไปนี้:

1. **สิทธิ์และการเข้าถึงพิกัดตำแหน่ง (Permissions & GPS Tracking):**

• ทันทีที่แอปเปิดทำงาน ต้องทำการขอสิทธิ์ Foreground Location

• หากสิทธิ์ถูกปฏิเสธ (Denied) ให้แสดงหน้าต่างเตือนภัยปุ่มสีแดงระบุข้อความ "กรุณาเปิดรับสิทธิ์ตำแหน่งเพื่อแสดงแผนที่" พร้อมปุ่มกดดึงคำสั่ง `Linking.openSettings()` เพื่ออำนวยความสะดวกให้ผู้ใช้เข้าหน้าระบบควบคุมสิทธิ์เครื่องได้ทันที

• หากสิทธิ์ได้รับอนุมัติ ให้ทำพิกัดละติจูดลองจิจูดปัจจุบัน แปลงที่อยู่อักษรย้อนกลับด้วย `reverseGeocodeAsync` และนำข้อมูลชื่อถนน/จังหวัด แสดงเป็นแผงป้ายชื่อด้านบนสุด

2. **การจัดแสดงภาพแผนที่แบบลึกซูม (Map Rendering):**

• แสดงแผงแผนที่แบบเต็มจอในบริเวณด้านล่าง โดยมีศูนย์กลางอยู่ที่ตำแหน่งละติจูดและลองจิจูดของผู้ใช้ปัจจุบัน

• ตั้งค่าระยะซูมเข้าลึก `latitudeDelta: 0.01` และ `longitudeDelta: 0.01` เพื่อให้เห็นรายละเอียดหมู่บ้านและตรอกซอย

• เรนเดอร์คอมโพเนนต์ `<Marker>` ปักหมุดเข็มสีสว่างไว้ที่จุดพิกัดปัจจุบัน พร้อมเขียนป้ายทูลทิป (Tooltip) ระบุข้อความ "ตำแหน่งของคุณในขณะนี้"

3. **ระบบสั่นสะเทือนจัดศูนย์กลาง (Motion Sensor Integration):**

• สมัครสตรีมสัญญาณข้อมูลเซนเซอร์จากเครื่องมือ `Accelerometer` โดยอัปเดตแกนความเร่ง x, y, z

• กำหนดให้หากผู้ใช้งานเขย่าตัวเครื่องจำลอง หรือเอียงเครื่องทำมุมความเร่ง (เช่น ค่าสัมบูรณ์แกน `x` หรือ `y` มีค่าความแรงเกินกว่า 1.8) ให้แผนที่สไลด์ (Animate) นำศูนย์กลางกล้องแผนที่วิ่งกลับมาโฟกัสพิกัดเริ่มต้นปัจจุบันของผู้ใช้โดยอัตโนมัติ (เสมือนการสั่ง Center Map ด้วยการเขย่า)

4. **ความมั่นคงในการล้างทรัพยากร (Memory Safety Cleanup):**

• ผู้เรียนต้องเขียนส่งมอบฟังก์ชัน Unsubscribe เพื่อยกเลิกลบผู้ฟังค่า Accelerometer ออกจาก RAM ของเครื่องเมื่อคอมโพเนนต์ถูกปิดตัวลง

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งไลบรารีตำแหน่ง แผนที่ และเซนเซอร์

รันคำสั่งติดตั้งคลังฮาร์ดแวร์ของ Expo และ React Native เพื่อใช้งานในโครงการ:

```
npx expo install expo-location react-native-maps expo-sensors
```

ขั้นตอนที่ 2: ตั้งค่าสิทธิ์และขอบเขตใน `app.json`

เปิดไฟล์ app.json และเขียนเพิ่มค่าสิทธิ์ความปลอดภัยลงในคีย์ plugins:

```
{
 "expo": {
  "plugins": [
   [
    "expo-location",
        {
     "locationWhenInUsePermission": "เราจำเป็นต้องเข้าถึงพิกัด GPS เพื่อแสดงจุดสีน้ำเงินบนแผนที่ของคุณ"
        }
   ]
  ]
  }
}
```

ขั้นตอนที่ 3: ออกแบบระบบตรวจสอบสิทธิ์และดักจับการตั้งค่าเครื่อง

เขียนโค้ดตรวจสอบเงื่อนไขและส่งลิงก์ Settings ด้วยคำสั่ง Linking:

```
import * as Linking from 'expo-linking';

// ปุ่มกดนำทางข้ามระบบ Sandbox
<TouchableOpacity
  style={styles.settingsButton}
  onPress={() => Linking.openSettings()}
>
  <Text style={styles.buttonText}>เปิดตั้งค่าสิทธิ์สิทธิ์บนระบบเครื่อง</Text>
</TouchableOpacity>
```

ขั้นตอนที่ 4: เชื่อมโยง Accelerometer เข้าหา MapView อ้างอิงด้วย Ref

เพื่อสั่งสั่งกล้องของแผนที่เลื่อนไปมาอย่างนุ่มนวล เราจะสร้างตัวแปรอ้างอิง mapRef และสั่งการผ่านคำสั่ง animateToRegion:

```
const mapRef = React.useRef<MapView | null>(null);

// หากแรงเหวี่ยงเซนเซอร์เขย่าเกินพิกัด ให้เลื่อนศูนย์กลางกลับมา
if (Math.abs(x) > 1.8 || Math.abs(y) > 1.8) {
  mapRef.current?.animateToRegion({
    latitude: currentCoords.latitude,
    longitude: currentCoords.longitude,
    latitudeDelta: 0.01,
    longitudeDelta: 0.01,
  }, 1000); // ดีดกล้องสไลด์นุ่มนวลในเวลา 1 วินาที
}
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถคัดลอกรหัสต้นฉบับสมบูรณ์ด้านล่างนี้ บันทึกทับในไฟล์หลักของโครงการ (เช่น `app/index.tsx`) เพื่อรันสตรีมนำพิกัดดาวเทียมขึ้นระบบแผนที่พร้อมระบบสั่นตรวจจับการเขย่า:

```
import React, { useState, useEffect, useRef } from 'react';
import {
  StyleSheet,
  Text,
  View,
  ActivityIndicator,
  TouchableOpacity,
  Alert,
  SafeAreaView
} from 'react-native';
import * as Location from 'expo-location';
import MapView, { Marker } from 'react-native-maps';
import { Accelerometer } from 'expo-sensors';
import * as Linking from 'expo-linking';

// โมเดลเก็บข้อมูลพิกัด
interface Coords {
  latitude: number;
  longitude: number;
}

export default function InteractiveLocationTracker() {
  const [hasPermission, setHasPermission] = useState<boolean |
null>(null);
  const [currentCoords, setCurrentCoords] = useState<Coords | null>(null);
  const [address, setAddress] = useState<string>('กำลังค้นหาที่อยู่ของคุณ...');
  const [isLoading, setIsLoading] = useState<boolean>(true);

 // ตัวแปรเก็บค่าพิกัดความเร่งเซนเซอร์
  const [accelerometerData, setAccelerometerData] = useState({ x: 0, y: 0,
z: 0 });
  const accelerometerSubscription = useRef<any>(null);

 // ตัวอ้างอิงออบเจกต์ MapView เพื่อส่งคำสั่งจัดเฟรมกล้อง
  const mapRef = useRef<MapView | null>(null);

  useEffect(() => {
    setupLocationAndSensors();

  // คืนค่าฟังก์ชัน Cleanup เมื่อถอนหน้าจอป้องกัน RAM รั่ว
    return () => {
```

```
      stopAccelerometerListening();
    };
  }, []);

 // 1. ฟังก์ชันจัดระเบียบสิทธิ์และการดึงพิกัดเริ่มต้น
  const setupLocationAndSensors = async () => {
    try {
      setIsLoading(true);

    // ขออนุมัติเข้าถึง Location
      const { status } = await
Location.requestForegroundPermissionsAsync();
      if (status !== 'granted') {
        setHasPermission(false);
        setIsLoading(false);
        return;
      }

      setHasPermission(true);

    // อ่านค่าพิกัดปัจจุบัน
      const location = await Location.getCurrentPositionAsync({
        accuracy: Location.Accuracy.Balanced,
      });

      const coordsObj = {
        latitude: location.coords.latitude,
        longitude: location.coords.longitude
      };

      setCurrentCoords(coordsObj);

    // ดึงที่อยู่ถนนจริงจากพิกัด GPS (Reverse Geocode)
      const reverseResponse = await
Location.reverseGeocodeAsync(coordsObj);
      if (reverseResponse.length > 0) {
        const addr = reverseResponse[0];
        const streetName = addr.street || addr.name || 'ไม่มีระบุถนน';
        const districtName = addr.district || 'ไม่มีระบุอำเภอ';
        const regionName = addr.region || 'ไม่มีระบุจังหวัด';
        setAddress(`📍 ${streetName}, อ.${districtName}, จ.${regionName}`);
      }

    // เริ่มต้นดักฟังข้อมูลเซนเซอร์เขย่า
      startAccelerometerListening(coordsObj);

    } catch (error) {
      console.error(error);
      Alert.alert('ข้อผิดพลาด', 'ไม่สามารถเชื่อมต่อระบบ GPS หรือระบุพิกัดในเครื่องได้');
```

```
    } finally {
      setIsLoading(false);
    }
  };

// 2. ฟังก์ชันจดจำสัญญาณ Accelerometer
  const startAccelerometerListening = (coords: Coords) => {
 // กำหนดรอบการดึงข้อมูลเซนเซอร์ถี่ขึ้น (200 มิลลิวินาที)
    Accelerometer.setUpdateInterval(200);

    accelerometerSubscription.current = Accelerometer.addListener((data)
=> {
      setAccelerometerData(data);

   // ตรวจสอบเงื่อนไขการเขย่าเครื่อง (เช็คความแรงขยับแกน x หรือ y เกิน 1.8)
      if (Math.abs(data.x) > 1.8 || Math.abs(data.y) > 1.8) {
    // บังคับเลื่อนแผนที่วิ่งกลับมาที่จุดพิกัดปัจจุบันของผู้ใช้
        mapRef.current?.animateToRegion({
          latitude: coords.latitude,
          longitude: coords.longitude,
          latitudeDelta: 0.01,
          longitudeDelta: 0.01,
        }, 1000); // สไลด์กล้องแผนที่ใน 1 วินาที
      }
    });
  };

  const stopAccelerometerListening = () => {
    if (accelerometerSubscription.current) {
      accelerometerSubscription.current.remove();
      accelerometerSubscription.current = null;
    }
  };

// 3. ฟังก์ชันปุ่มกดสปอตไลท์ด่วน (Manual Refocus)
  const handleManualCenter = () => {
    if (currentCoords) {
      mapRef.current?.animateToRegion({
        latitude: currentCoords.latitude,
        longitude: currentCoords.longitude,
        latitudeDelta: 0.01,
        longitudeDelta: 0.01,
      }, 800);
    }
  };

// 4. แสดงกรณีสิทธิ์เข้าใช้งานถูกระงับ
  if (hasPermission === false) {
    return (
```

```
      <SafeAreaView style={styles.errorContainer}>
        <View style={styles.errorCard}>
          <Text style={styles.errorEmoji}>🚫</Text>
          <Text style={styles.errorTitle}>ระบบสิทธิ์ตำแหน่งถูกปิดกั้น</Text>
          <Text style={styles.errorDesc}>
        แอปพลิเคชันจำเป็นต้องเข้าใช้งานพิกัด GPS เพื่อทำจัดเรนเดอร์ภาพแผนที่แสดงจุดพิกัดส่วนบุคคล กรุณาเปิดอนุมัติสิทธิ์
ในแอปการตั้งค่า
          </Text>
          <TouchableOpacity
            style={styles.settingsButton}
            onPress={() => Linking.openSettings()}
          >
            <Text style={styles.settingsButtonText}>เปิดหน้าต่างตั้งค่าสิทธิ์แอป</Text>
          </TouchableOpacity>
          <TouchableOpacity
            style={styles.retryButton}
            onPress={setupLocationAndSensors}
          >
            <Text style={styles.retryButtonText}>🔄 ลองขอสิทธิ์ใหม่อีกครั้ง</Text>
          </TouchableOpacity>
        </View>
      </SafeAreaView>
  );
  }

  return (
    <SafeAreaView style={styles.safeArea}>
      <View style={styles.container}>

        {/* แผงควบคุมแสดงรายละเอียดที่อยู่ด้านบนสุด */}
      <View style={styles.addressPanel}>
          <Text style={styles.addressTitle}>พิกัดปัจจุบันของคุณ</Text>
          <Text style={styles.addressText} numberOfLines={2}>
            {isLoading ? '⏳ กำลังค้นหาข้อมูลตำแหน่งพิกัดดาวเทียม...' : address}
        </Text>
          {currentCoords && (
            <Text style={styles.coordsSub}>
              Lat: {currentCoords.latitude.toFixed(5)}, Lng:
{currentCoords.longitude.toFixed(5)}
            </Text>
      )}
        </View>

        {/* ส่วนแสดงแผนที่ระบุพิกัดภูมิศาสตร์ */}
        <View style={styles.mapContainer}>
          {isLoading || !currentCoords ? (
            <View style={styles.loadingWrapper}>
              <ActivityIndicator size="large" color="#38bdf8" />
              <Text style={styles.loadingText}>กำลังจับคู่แผนที่กับสัญญาณดาวเทียม...
```

```
</Text>
            </View>
      ) : (
            <>
              <MapView
                ref={mapRef}
                style={styles.map}
                initialRegion={{
          latitude: currentCoords.latitude,
                  longitude: currentCoords.longitude,
                  latitudeDelta: 0.01,
                  longitudeDelta: 0.01,
                }}
                showsUserLocation={true}
              >
           {/* ปักเข็มหมุดแสดงพิกัดผู้ใช้ */}
                <Marker
                  coordinate={{
                    latitude: currentCoords.latitude,
                    longitude: currentCoords.longitude
                  }}
                  title="ตำแหน่งของคุณ"
                  description="ปักหมุดตรงพิกัดตาม GPS"
        />
              </MapView>

              {/* ปุ่มควบคุมแบบลอยตัว (Floating action buttons over map) */}
              <TouchableOpacity
                style={styles.floatingButton}
                onPress={handleManualCenter}
              >
                <Text style={styles.floatingButtonText}>🎯 ซิงก์กล้องที่จุดคุณ
</Text>
              </TouchableOpacity>

              {/* แถบรายงานค่าเซนเซอร์ความเร่งด้านขวาล่าง */}
              <View style={styles.sensorInfoBox}>
                <Text style={styles.sensorText}>เขย่าเครื่องเบาๆ เพื่อจัดศูนย์กลาง</Text>
                <Text style={styles.sensorTextVal}>
                  X: {accelerometerData.x.toFixed(2)}, Y:
{accelerometerData.y.toFixed(2)}
                </Text>
              </View>
            </>
      )}
        </View>
      </View>
    </SafeAreaView>
  );
}
```

```
const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  container: {
    flex: 1,
  },
  addressPanel: {
    backgroundColor: '#1e293b',
    padding: 16,
    borderBottomWidth: 1,
    borderBottomColor: '#334155',
  },
  addressTitle: {
    color: '#38bdf8',
    fontWeight: 'bold',
    fontSize: 12,
    textTransform: 'uppercase',
    letterSpacing: 1,
  },
  addressText: {
    color: '#ffffff',
    fontSize: 15,
    fontWeight: 'bold',
    marginTop: 4,
  },
  coordsSub: {
    color: '#94a3b8',
    fontFamily: 'monospace',
    fontSize: 11,
    marginTop: 4,
  },
  mapContainer: {
    flex: 1,
    position: 'relative',
  },
  map: {
    width: '100%',
    height: '100%',
  },
  loadingWrapper: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#0f172a',
  },
  loadingText: {
```

```
    color: '#94a3b8',
    marginTop: 12,
    fontSize: 14,
  },
  floatingButton: {
    position: 'absolute',
    bottom: 20,
    left: 20,
    backgroundColor: '#38bdf8',
    paddingVertical: 12,
    paddingHorizontal: 20,
    borderRadius: 24,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.3,
    shadowRadius: 5,
    elevation: 6,
  },
  floatingButtonText: {
    color: '#0f172a',
    fontWeight: 'bold',
    fontSize: 14,
  },
  sensorInfoBox: {
    position: 'absolute',
    bottom: 20,
    right: 20,
    backgroundColor: 'rgba(15, 23, 42, 0.85)',
    padding: 10,
    borderRadius: 12,
    borderWidth: 1,
    borderColor: '#334155',
    alignItems: 'flex-end',
  },
  sensorText: {
    color: '#94a3b8',
    fontSize: 10,
  },
  sensorTextVal: {
    color: '#38bdf8',
    fontFamily: 'monospace',
    fontSize: 11,
    fontWeight: 'bold',
    marginTop: 2,
  },
  errorContainer: {
```

```
    flex: 1,
    backgroundColor: '#0f172a',
    justifyContent: 'center',
    padding: 24,
  },
  errorCard: {
    backgroundColor: '#1e293b',
    borderRadius: 24,
    padding: 24,
    alignItems: 'center',
    borderWidth: 1,
    borderColor: '#334155',
  },
  errorEmoji: {
    fontSize: 48,
    marginBottom: 16,
  },
  errorTitle: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
    marginBottom: 10,
  },
  errorDesc: {
    fontSize: 14,
    color: '#cbd5e1',
    textAlign: 'center',
    lineHeight: 22,
    marginBottom: 24,
  },
  settingsButton: {
   backgroundColor: '#ef4444',
    paddingVertical: 12,
    paddingHorizontal: 24,
    borderRadius: 12,
    width: '100%',
    alignItems: 'center',
    marginBottom: 12,
  },
  settingsButtonText: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 14,
  },
  retryButton: {
    backgroundColor: 'transparent',
    paddingVertical: 12,
```

```
    width: '100%',
    alignItems: 'center',
    borderWidth: 1,
    borderColor: '#334155',
    borderRadius: 12,
  },
  retryButtonText: {
    color: '#94a3b8',
    fontWeight: 'bold',
    fontSize: 14,
  },
});
```

5. การวิเคราะห์สรุปผลการประเมินปฏิบัติการ

เมื่อติดตั้งและทดลองเปิดแอปพลิเคชัน ให้ผู้เรียนประเมินความสมบูรณ์ดังนี้:

1. หากทำการยกเลิกไม่ยอมเปิด GPS แถบแสดงผลของแอปพลิเคชันต้องดีดสลับมาหน้ากล่องข้อความเตือนภัยสีแดง และกดปุ่ม Settings ได้ทันที

2. เมื่อเปิดบริการ GPS รายการแผนที่ต้องดึงศูนย์กลางกล้องมาโฟกัสพิกัดผู้ใช้และปักหมุดได้อย่างแม่นยำ

3. เมื่อใช้นิ้วเลื่อนแผนที่หนีขอบเขตไปไกล และออกแรงเขย่าตัวเครื่อง (หรือปรับจำลอง Accelerometer ในอุปกรณ์ Emulator) แผนที่ต้องดีดตัวอนิเมตกล้องกลับมาที่จุดพิกัดปัจจุบันโดยอัตโนมัติ

เอกสารอ้างอิง

Expo. (2024). *Location*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/location/

Expo. (2024). *Sensors*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/sensors/

React Native Maps. (2024). *React Native Maps Installation*. จาก https://github.com/react-native-maps/react-native-maps

แผนบริหารการสอนประจำบทที่ 10

หัวข้อเนื้อหาประจำบท

10.1 สถาปัตยกรรม BaaS และคลาวด์ดาต้าเบสของ Supabase

10.2 การติดตั้งและตั้งค่า Supabase Client ใน Expo (SDK 54)

10.3 การอ่านข้อมูลจากตาราง (Read / Select Data)

10.4 การเพิ่มข้อมูลแถวใหม่ (Create / Insert Data)

10.5 การแก้ไขและปรับปรุงข้อมูล (Update Data)

10.6 การลบข้อมูลออกจากตาราง (Delete Data)

10.7 การจัดการสถานะการทำงานและประมวลผลข้อผิดพลาด (Loading & Error Handling)

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถอธิบายสถาปัตยกรรมจัดสรรหลังบ้าน BaaS และความแตกต่างเมื่อเทียบกับฐานข้อมูลธรรมดาได้ถูกต้อง
2. ผู้เรียนสามารถติดตั้ง ตั้งค่า และลงทะเบียน Supabase Client ในโปรเจกต์ Expo ได้สำเร็จ
3. ผู้เรียนสามารถเขียนคำสั่งภาษา SQL จำลองผ่าน JS Client ดึงข้อมูลขึ้นมาแสดงผลเป็นรายการตารางงานได้สำเร็จ
4. ผู้เรียนสามารถเขียนคำสั่งเพิ่ม ลบ และอัปเดตสถานะงานของตารางบน Supabase Cloud ได้ถูกต้อง
5. ผู้เรียนสามารถวิเคราะห์สถานะข้อผิดพลาดของข้อมูล (Error objects) และเขียนส่งสัญญาณเตือนอินเตอร์เฟซได้ถูกต้อง

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

37. **การบรรยายสถาปัตยกรรม BaaS และคลาวด์เบสหลังบ้าน (BaaS & Cloud Database Lecture):** อธิบายหลักการของระบบหลังบ้านแบบ Serverless และ

เปรียบเทียบระหว่างสถาปัตยกรรมแบบทำเซิร์ฟเวอร์เอง (Custom Backends) กับการประยุกต์ใช้คลาวด์สำเร็จรูปอย่าง Supabase

38. **การทำ Live Coding Demonstration หัวข้อ Real-time Shopping Task List:** ผู้สอนสาธิตการทำแอปจดลิสต์รายการซื้อของชิงก์ตรงหลังบ้าน Supabase ที่ผู้ใช้สามารถจิ้มขีดฆ่าสินค้าที่ซื้อแล้ว (Update) หรือพิมพ์เพิ่มรายการใหม่ได้ทันที
39. **กิจกรรมแฮกเกอร์เขียนคำสั่งเงื่อนไข (Conditional Queries Challenge):** ให้ผู้เรียนทดลองเขียนตัวกรองดักรับข้อมูลให้แสดงผลเฉพาะสินค้าที่ยังไม่ได้ซื้อ และมีปุ่มกดสับเปลี่ยนลำดับแสดงราคาแพงสุดขึ้นก่อน
40. **กิจกรรมป้องกันความปลอดภัยฟอร์มข้อมูล (Database Constraint Protection):** ให้กลุ่มผู้เรียนช่วยกันพัฒนาเงื่อนไขสกัดกั้นการบันทึกข้อมูล หากช่องรับตัวหนังสือไม่มีค่า เพื่อป้องกันการยิงแถวว่างเปล่าเข้าไปรบกวนฐานข้อมูล

สื่อการเรียนการสอน

36. เอกสารสไลด์เรื่อง "Supabase Cloud Database Integration in React Native Applications"
37. หน้าแผงควบคุมหลักของโครงการ Supabase (Supabase Dashboard) สำหรับตรวจเช็คแถวตารางข้อมูลแบบเรียลไทม์
38. ตัวอย่างการเซ็ตค่าสิทธิ์ RLS (Row Level Security) ขั้นต้น
39. ชุดคำสั่งตัวอย่างการเขียน Wrapper Helper สำหรับดึงข้อมูล Supabase Client ในแอปพลิเคชัน

การวัดผลและการประเมินผล

1. การวัดผล

1.1 ประเมินระดับความรู้เชิงทฤษฎีจากการทำคะแนนแบบทดสอบปรนัยท้ายบทเรียน 10 ข้อ

1.2 ตรวจสอบความสมบูรณ์และถูกต้องในการเรียกใช้ Supabase client และคำสั่ง SQL-like methods (select, insert, etc.)

1.3 ตรวจสอบขั้นตอนการจัดการข้อยกเว้นและตัวบ่งบอกความคืบหน้า (Loading)

ในโค้ดใบงานปฏิบัติการ

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถสร้างโมดูลสมุดรายชื่อสินค้าที่ซิงก์ตรงขึ้นระบบคลาวด์ Supabase โดยแสดงข้อมูลถูกประเภทและรันได้ไม่มีข้อผิดพลาด (ผ่าน/ไม่ผ่าน)

2.3 สังเกตพฤติกรรมการรักษาความปลอดภัยของ Anon Key (ไม่เปิดเผยคีย์สาธารณะ) และความละเอียดรอบคอบในการจัดชนิดข้อมูลอินเทอร์เฟซ

บทที่ 10

การใช้งานคลาวด์ดาต้าเบสด้วย Supabase

การพัฒนาแอปพลิเคชันที่มีระบบฐานข้อมูลหลังบ้านแบบเรียลไทม์ โดยไม่ต้องเสียเวลาเขียนโค้ดและตั้งค่าเครื่องเซิร์ฟเวอร์เอง ถือเป็นความสะดวกที่ทำได้จริงผ่านระบบ **Backend-as-a-Service (BaaS)** ในบทเรียนนี้เราจะศึกษาขั้นตอนการเชื่อมโยงแอปพลิเคชัน Expo กับ **Supabase Cloud Database** ซึ่งขับเคลื่อนด้วย PostgreSQL และเรียนรู้วิธีจัดการฐานข้อมูลครบวงจร (CRUD Operations) ทั้งการเพิ่ม ดึง แก้ไข และลบข้อมูล

10.1 สถาปัตยกรรม BaaS และคลาวด์ดาต้าเบสของ Supabase

Supabase เป็นแพลตฟอร์มหลังบ้านสำเร็จรูปที่เป็นมิตรต่อผู้พัฒนาและเปิดใช้งานแบบ Open-source (Supabase, 2024) โดยมีโครงสร้างฐานข้อมูลหลักเป็น **PostgreSQL** ซึ่งมีความเสถียรและรองรับความสัมพันธ์ของข้อมูล (Relational Database)

ทำไมต้องเลือก Supabase?

• **พร้อมใช้งานทันที:** มีระบบ Database, Auth, และ Storage มาให้พร้อมใช้โดยไม่ต้องติดตั้งและตั้งค่าระบบเครือข่ายเซิร์ฟเวอร์เอง

• **รองรับระบบ SQL แท้:** สามารถเขียน Query จัดความสัมพันธ์ข้อมูล หรือประกาศ Constraints ป้องกันตารางพังได้เหมือนฐานข้อมูลสากลทั่วไป

• **ประสิทธิภาพระดับเนทีฟ:** ไลบรารีพัฒนามาให้รองรับระบบ Asynchronous (Promise-based) ทำงานร่วมกับ React Native ได้อย่างไร้รอยต่อ

ขั้นตอนการเตรียมคลาวด์โปรเจกต์:

1. ลงทะเบียนและล็อกอินเข้าใช้งานที่ supabase.com
2. สร้างโครงการใหม่ (Create New Project) และระบุรหัสผ่านฐานข้อมูล
3. ไปที่เมนู **Project Settings > API** ของแผงควบคุมหลังบ้าน เพื่อคัดลอกค่าความปลอดภัยหลัก 2 คีย์ ได้แก่:

• **Project URL**: ที่อยู่โดเมนชี้เฉพาะสำหรับส่ง API รีเควส

• **Anon Key (Public)**: คีย์สาธารณะสำหรับสแกนส่งสิทธิ์อนุญาตเข้าถึงตารางข้อมูล

10.2 การติดตั้งและตั้งค่า Supabase Client ใน Expo (SDK 54)

เพื่อเชื่อมแอปพลิเคชัน Expo เข้าหาตารางฐานข้อมูล เราจะติดตั้งชุดซอฟต์แวร์พัฒนาอย่างเป็นทางการร่วมกับตัวช่วยจัดระเบียบ URL สำหรับ React Native (Supabase, 2024):

```
npm install @supabase/supabase-js react-native-url-polyfill
```

หมายเหตุ: `react-native-url-polyfill` ทำหน้าที่เป็นสะพานเชื่อมฟังก์ชันการประมวลผลคำสั่ง URL ของตัวเนทีฟเครื่องให้สอดรับกับมาตรฐานเว็บ เนื่องจาก React Native รันไทม์ไม่มีฟังก์ชัน URL ดั้งเดิม

การตั้งค่าตัวแปรสภาพแวดล้อม (Environment Variables) ใน SDK 54:

เพื่อความปลอดภัย ห้ามนำเอาคีย์ความลับอย่าง Anon Key ไปแปะเขียนโค้ดลงไปตรง ๆ แต่ให้ใช้ไฟล์ **`.env`** ในโฟลเดอร์นอกสุดของโปรเจกต์:

```
EXPO_PUBLIC_SUPABASE_URL=https://your-project-id.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

หมายเหตุ: ใน Expo SDK 54 ตัวแปรสภาพแวดล้อมที่ขึ้นต้นด้วย `EXPO_PUBLIC_` จะถูกแปลงและโหลดเข้าสู่ระบบรันไทม์จาวาสคริปต์อัตโนมัติ

การเขียนไฟล์ตั้งค่าเชื่อมโยง client (`lib/supabase.ts`):

```
import 'react-native-url-polyfill/auto'; // นำเข้าเพื่อแก้บั๊กคำสั่ง URL ในระบบเนทีฟ
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL || '';
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY || '';

// สร้างและส่งออก supabase object ไปใช้งานข้ามไฟล์
export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

10.3 การอ่านข้อมูลจากตาราง (Read / Select Data)

การสืบค้นดึงข้อมูลแถวตารางจากฐานข้อมูล Supabase จะทำผ่านเมธอด **`.select()`** โดยสกัดผลลัพธ์ข้อมูลออกมาในคีย์ **data** และตรวจจับปัญหาการอ่านในคีย์ **error**

```
import { supabase } from '../lib/supabase';

interface Task {
  id: number;
  title: string;
  is_completed: boolean;
  created_at: string;
}

const fetchTasks = async (): Promise<Task[]> => {
  const { data, error } = await supabase
```

```
  .from('tasks') // อ้างอิงชื่อตารางใน Supabase
  .select('*')    // ดึงฟิลด์ข้อมูลทุกคอลัมน์
  .order('created_at', { ascending: false }); // เรียงลำดับเอาโพสต์ใหม่ขึ้นก่อน

  if (error) {
    console.error('ดึงข้อมูลล้มเหลว:', error.message);
    return [];
  }

  return data as Task[];
};
```

การใส่ตัวกรอง (Query Filters):

เราสามารถคัดกรองข้อมูลเฉพาะแถวที่ต้องการได้ด้วยการต่อเมธอด เช่น:

- `.eq('is_completed', true)`: แสดงเฉพาะโพสต์ที่เสร็จสมบูรณ์แล้ว (Equal)
- `.ilike('title', '%เรียน%')`: ค้นหาชื่อโพสต์ที่มีตัวอักษร "เรียน" ผสมอยู่โดยไม่สนใจตัวพิมพ์เล็กใหญ่

10.4 การเพิ่มข้อมูลแถวใหม่ (Create / Insert Data)

ในการสร้างแถวข้อมูลชุดใหม่ (เช่น การลงทะเบียนหรือบันทึกสินค้าเพิ่ม) จะส่งผ่านเมธอด `.insert()` โดยยอมรับข้อมูลในรูปแบบของอาร์เรย์ออบเจกต์ (Array of Objects)

```
const createNewTask = async (taskTitle: string) => {
  const { data, error } = await supabase
  .from('tasks')
  .insert([
      {
        title: taskTitle,
        is_completed: false
      }
  ])
  .select(); // บอกให้ Supabase ส่งข้อมูลแถวล่าสุดที่สร้างสำเร็จกลับคืนมาด้วย

  if (error) {
    console.error('บันทึกไม่สำเร็จ:', error.message);
    return null;
  }

  return data[0]; // คืนค่าออบเจกต์ข้อมูลชิ้นแรกที่พึ่งบันทึกสำเร็จ
};
```

10.5 การแก้ไขและปรับปรุงข้อมูล (Update Data)

เมื่อมีการคลิกทำเครื่องหมายติ๊กถูก หรือต้องการเปลี่ยนข้อความแถวข้อมูล เราจะใช้เมธอด `.update()` โดยต้องระบุเงื่อนไขเสมอด้วย `.eq()` เพื่อป้องกันไม่ให้คำสั่งเข้าไปทับข้อมูลของแถวอื่น ๆ ทั้งตาราง

```
const toggleTaskCompletion = async (taskId: number, currentStatus:
boolean) => {
  const { data, error } = await supabase
  .from('tasks')
  .update({
      is_completed: !currentStatus
    }) // ระบุคอลัมน์และค่าใหม่ที่ต้องการแก้ไข
  .eq('id', taskId) // กรองอัปเดตเฉพาะแถวที่มี ID ตรงกัน
  .select();

  if (error) {
    console.error('อัปเดตข้อมูลล้มเหลว:', error.message);
    return null;
  }

  return data[0];
};
```

10.6 การลบข้อมูลออกจากตาราง (Delete Data)

การสั่งถอดถอนแถวข้อมูลออกจากสารระบบอย่างถาวร จะทำผ่านเมธอด `.delete()` โดยต้องผูกตัวกรอง .eq() ชี้เฉพาะเจาะจงเป้าหมาย

```
const deleteTask = async (taskId: number): Promise<boolean> => {
  const { error } = await supabase
  .from('tasks')
  .delete()
  .eq('id', taskId); // ลบเฉพาะแถวที่มี ID ตรงกันเท่านั้น

  if (error) {
    console.error('ลบข้อมูลไม่สำเร็จ:', error.message);
    return false;
  }

  return true; // ส่งค่าความสำเร็จกลับไปอัปเดตสถานะลิสต์บนหน้าจอแอป
};
```

10.7 การจัดการสถานะการทำงานและประมวลผลข้อผิดพลาด (Loading & Error Handling)

จุดแตกต่างของการติดต่อฐานข้อมูลตรงบนคลาวด์เมื่อเทียบกับการเก็บตัวแปรบนเครื่อง คือความเร็วการตอบรับจะแปรผันตามระยะทางและคุณภาพของเครือข่ายอินเทอร์เน็ต ผู้พัฒนาต้องเตรียมกลไกแจ้งสถานะอย่างมีชั้นเชิง:

```
import React, { useState, useEffect } from 'react';
import { View, Text, ActivityIndicator, Alert, Button } from 'react-
native';
import { supabase } from '../lib/supabase';

export default function LoadingExample() {
  const [tasks, setTasks] = useState<any[]>([]);
  const [loading, setLoading] = useState(true);

  const loadData = async () => {
    try {
      setLoading(true);
      const { data, error } = await supabase.from('tasks').select('*');

      if (error) {
        throw error; // ดีดเข้าบล็อก catch เพื่อแยกส่วนจัดเตือน
      }

      setTasks(data || []);
    } catch (error: any) {
      Alert.alert('เชื่อมโยงฐานข้อมูลขัดข้อง', error.message || 'กรุณาลองใหม่อีกครั้ง');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    loadData();
  }, []);

  if (loading) {
    return (
      <View style={{ flex: 1, justifyContent: 'center', alignItems:
'center' }}>
        <ActivityIndicator size="large" color="#38bdf8" />
        <Text style={{ marginTop: 8, color: '#64748b' }}>กำลังเชื่อมต่อฐานข้อมูลระบบ
คลาวด์...</Text>
      </View>
  );
  }
```

```
  return (
    <View style={{ padding: 20 }}>
      <Text style={{ fontSize: 18, fontWeight: 'bold' }}>จำนวนงานคงค้าง:
{tasks.length} รายการ</Text>
      <Button title="🔄 ดึงข้อมูลสดใหม่" onPress={loadData} />
    </View>
  );
}
```

สรุปท้ายบทเรียน

สถาปัตยกรรม BaaS และคลาวด์ดาต้าเบสของ Supabase: Supabase เป็นระบบหลังบ้านสำเร็จรูปประเภท Backend-as-a-Service (BaaS) ที่ขับเคลื่อนด้วยฐานข้อมูล PostgreSQL ทำให้ประยุกต์จัดความสัมพันธ์ของข้อมูลแบบ SQL แท้ได้อย่างเต็มรูปแบบ โดยผู้พัฒนาสามารถเริ่มใช้งานได้ทันทีและเข้าถึงคีย์ความปลอดภัยอย่าง Project URL และ Anon Key เพื่อเข้าสิทธิ์เชื่อมต่อการทำงานต่าง ๆ

การติดตั้งและตั้งค่า Supabase Client ใน Expo: การนำ Supabase เข้าใช้งานในโครงการ Expo SDK 54 ต้องติดตั้งไลบรารี @supabase/supabase-js (Supabase, 2024) ร่วมกับ react-native-url-polyfill เพื่อแก้ปัญหาการจัดการโครงสร้างลิงก์ และควรเก็บคีย์ความลับทั้งหมดไว้ในรูปตัวแปรสภาพแวดล้อมที่ขึ้นต้นด้วย EXPO_PUBLIC_ ภายในไฟล์ .env นอกสุดของโปรเจกต์เพื่อยกระดับความปลอดภัยก่อนเปิดใช้งานจริง

การอ่านข้อมูลจากตาราง (Read / Select Data): เมธอด .select() ใช้สำหรับการเรียกค้นหาและดึงข้อมูลจากตารางฐานข้อมูล Supabase โดยเราสามารถสืบค้นคอลัมน์ทั้งหมดแบบเจาะจง หรือต่อคำสั่งคัดกรองผลลัพธ์ผ่านตัวกรองเฉพาะ เช่น .eq() สำหรับเทียบข้อมูลค่าเท่า และ .ilike() ในการค้นหาบางส่วนของคำเพื่อส่งผลข้อมูลที่จัดเรียงถูกต้องกลับไปที่แอป

การเพิ่มข้อมูลแถวใหม่ (Create / Insert Data): การบันทึกข้อมูลใหม่ลงในตารางใช้เมธอด .insert() โดยยอมรับข้อมูลรูปแาร์เรย์ของออบเจกต์ ซึ่งสามารถอัปเดตฟิลด์ข้อมูลที่ต้องการและเรียกใช้งานคำสั่งพ่วงต่ออย่าง .select() เพื่อร้องขอให้เซิร์ฟเวอร์ส่งข้อมูลบรรทัดที่บันทึกสำเร็จล่าสุดย้อนกลับมาหาตัวแปรบนแอปทันทีเพื่อประมวลผลในขั้นตอนถัดไป

การแก้ไขและปรับปรุงข้อมูล (Update Data): กระบวนการแก้ไขค่าของข้อมูลคอลัมน์ในแถวที่เลือกจะสั่งงานผ่านเมธอด .update() โดยข้อควรระวังหลักคือผู้พัฒนาจำเป็นต้องผูกตัวกรองอย่างเช่น .eq('id', taskId) ต่อท้ายเสมอ เพื่อระบุขอบเขตการแก้ไขให้ตรงตามแถวเป้าหมายและจำกัดไม่ให้คำสั่งเข้าไปเขียนทับแถวข้อมูลอื่น ๆ ทั้งตารางข้อมูล

การลบข้อมูลออกจากตาราง (Delete Data): การถอดถอนแถวข้อมูลออกจากตารางอย่างมั่นคงถาวรทำได้โดยใช้เมธอด .delete() ซึ่งต้องใช้ร่วมกับตัวกรอง .eq() ในการระบุค่าไอดีเอกลักษณ์เพื่อกำจัดการลบแถวข้อมูลที่ไม่พึงประสงค์ โดยคำสั่งจะคืนค่ารายงานความขัดข้องกลับมาในคีย์ error เพื่อให้เราวิเคราะห์และอัปเดตสถานะลิสต์บนหน้าจอได้อย่างถูกต้อง

การจัดการสถานะการทำงานและประมวลผลข้อผิดพลาด: เนื่องจากคุณภาพสัญญาณเน็ตเวิร์กส่งผลให้เวลาในการตอบสนองฐานข้อมูลคลาวด์มีความเร็วแตกต่างกัน การออกแบบระบบจึงจำเป็นต้องครอบคลุมการจัดการสเตต loading เพื่อเปิดปิดไอคอน ActivityIndicator รวมถึงการนำเข้าโครงสร้าง try-catch มาเขียนควบคุมและใช้ response.ok หรือคีย์ error ในการตรวจจับความล้มเหลวของเครือข่ายก่อนสลับการเรนเดอร์อินเตอร์เฟซเตือนผู้ใช้งาน

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้และความเข้าใจเกี่ยวกับแนวคิดระบบหลังบ้านสำเร็จรูป (BaaS), การลงทะเบียนเชื่อมต่อ Supabase Client ในสภาพแวดล้อม Expo SDK 54, การเขียนคำสั่งสืบค้นและคัดกรองข้อมูล (SELECT), การเพิ่มแถวข้อมูล (INSERT), การแก้ไขค่าข้อมูล (UPDATE), การลบข้อมูล (DELETE) และสุนทรียภาพในการจัดการข้อผิดพลาดความปลอดภัย

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดคือบทบาทหน้าที่ที่ถูกต้องที่สุดของสถาปัตยกรรม Backend-as-a-Service (BaaS) เช่น Supabase?

ก. ทำการอัปเกรดความเร็วในการประมวลผล CPU และ RAM บนสมาร์ทโฟนของผู้ใช้งานโดยตรง

ข. จัดสรรระบบบริการหลังบ้านสำเร็จรูป เช่น ฐานข้อมูล ระบบยืนยันตัวตน และคลังเก็บไฟล์บนระบบคลาวด์ เพื่อให้แอปมือถือเรียกใช้งานได้ทันทีโดยไม่ต้องเขียนเซิร์ฟเวอร์เอง

ค. ทำหน้าที่เขียนสไตล์ชีทสำหรับจัดวาง Flexbox ในแบบออโต้

ง. บีบอัดขนาดโค้ดของแอปพลิเคชันให้เหลือเพียงหน้าจอเดียวชั่วคราว

ข้อ 2: ฐานข้อมูลหลักที่รันอยู่เบื้องหลังและคอยจัดสรรแถวตารางข้อมูลในคลาวด์ของ Supabase คือฐานข้อมูลประเภทใด?

ก. MongoDB (NoSQL)

ข. MySQL (Relational)

ค. PostgreSQL (Relational)

ง. Firebase Realtime Database (JSON)

ข้อ 3: การดึงคีย์ Anon Key จากระบบของ Supabase มาใช้เชื่อมต่อนั้น Anon Key มีลักษณะความปลอดภัยอย่างไร?

ก. เป็นคีย์ลับสูงสุดระดับ Root ห้ามใครพบเห็นเด็ดขาดเนื่องจากใช้ลบระบบทั้งหมดได้

ข. เป็นคีย์สาธารณะ (Public API Key) สำหรับส่งไปพร้อม API เพื่อยืนยันสิทธิ์เข้าใช้ขั้นต้นของลูกค้าร่วมกับการตั้งเงื่อนไขสิทธิ์ RLS (Row Level Security)

ค. เป็นรหัสผ่านสำหรับเข้าสู่โหมดนักพัฒนาบนแอนดรอยด์

ง. เป็นรหัสยืนยันตนสำหรับการอัปเดตแอปขึ้นไปบน Google Play Store

ข้อ 4: ใน Expo SDK 54 เมื่อต้องการกำหนดค่ารหัสโครงการและคีย์ Anon Key ไว้ในไฟล์ .env เพื่อไม่ให้โค้ดรั่วไหล ข้อใดคือรูปแบบการตั้งชื่อตัวแปรสภาพแวดล้อม (Environment Variables) ที่ถูกต้อง?

ก. `SUPABASE_URL=...`

ข. `SECRET_SUPABASE_URL=...`

ค. `EXPO_PUBLIC_SUPABASE_URL=...`

ง. `REACT_APP_SUPABASE_URL=...`

ข้อ 5: เหตุใดเราจึงจำเป็นต้องติดตั้งและสั่งนำเข้า `import 'react-native-url-polyfill/auto';` ไว้ที่ส่วนบนสุดของไฟล์กำหนดค่า Supabase Client?

ก. เพื่อบีบอัดขนาดไฟล์ภาพของโลโก้ให้เล็กลง

ข. เพื่อชดเชยและแก้ไขปัญหาที่ React Native รันไทม์ไม่มีคลาสคำสั่งจัดการ URL (URL API Class) ในระดับเนทีฟ ซึ่งทำให้ไลบรารีของ Supabase ส่งรีเควสไม่ได้

ค. เพื่อสั่งเปิดสัญญาณบลูทูธของมือถือเชื่อมเข้าคลาวด์

ง. เพื่อสลับภาษาในแบบฟอร์มให้กลายเป็นภาษาไทยอัตโนมัติ

ข้อ 6: คำสั่งสืบค้นข้อความ SQL-like ใดใน Supabase JS Client ที่ใช้สำหรับระบุชื่อตารางเป้าหมายที่ต้องการเปิดติดต่อข้อมูล?

ก. `.select()`

ข. `.from()`

ค. `.insert()`

ง. `.eq()`

ข้อ 7: หากต้องการดึงข้อมูลทุกคอลัมน์จากตารางชื่อ "tasks" โดยเรียงลำดับแถวเอาโพสต์ที่มีอายุการบันทึกล่าสุดขึ้นก่อน ข้อใดเขียนคำสั่งได้ถูกต้องและมีประสิทธิภาพที่สุด?

ก. `await supabase.from('tasks').select('*').order('created_at', { ascending: false })`

ข. `await supabase.from('tasks').select('all').eq('id', 1)`

ค. `await supabase.from('tasks').insert({ order: 'created_at' })`

ง. `await supabase.from('tasks').delete().order('created_at')`

ข้อ 8: การเพิ่มข้อมูลแถวใหม่เข้าไปในตารางด้วยเมธอด `.insert()` ของ Supabase ข้อมูลดังกล่าวต้องมีโครงสร้างของอาร์กิวเมนต์เป็นประเภทใด?

ก. ข้อความสตริงระบุคีย์ดิบ ๆ

ข. อาร์เรย์ของออบเจกต์ (Array of Objects) เช่น `[{ key: value }]`

ค. ตัวแปรประเภทตัวเลขทศนิยมเดี่ยว

ง. ฟังก์ชันประเภท Callback

ข้อ 9: พฤติกรรมการทำงานของระบบความปลอดภัย RLS (Row Level Security) ของ Supabase ข้อใดอธิบายได้ถูกต้อง?

ก. ทำหน้าที่บังคับล็อกหน้าจอโทรศัพท์มือถือหากกรอกรหัสผ่านผิดเกิน 3 ครั้ง

ข. เป็นกฎความปลอดภัยบนระบบคลาวด์เพื่ออนุญาตหรือปิดกั้นให้เฉพาะผู้ใช้ที่มีสิทธิ์เข้าถึงหรือแก้ไขเฉพาะแถวข้อมูลที่ตนเองเป็นเจ้าของเท่านั้น

ค. ใช้จำกัดความสูงความกว้างของฟอร์ม TextInput

ง. เป็นฟังก์ชันลบข้อมูลขยะออกจากหน่วยความจำ RAM ของมือถือโดยออโต้

ข้อ 10: หากต้องการแก้ไขสถานะการทำงานในคอลัมน์ `is_completed` ให้กลายเป็นค่า `true` เฉพาะแถวงานที่มีรหัสไอดี ID เท่ากับ 5 ข้อใดเขียนคำสั่งระบุเงื่อนไขได้ถูกต้องและเจาะจงเป้าหมายที่สุด?

ก. `await supabase.from('tasks').update({ is_completed: true }).eq('id', 5)`

ข. `await supabase.from('tasks').insert({ is_completed: true }).eq('id', 5)`

ค. `await supabase.from('tasks').delete().eq('id', 5)`

ง. `await supabase.from('tasks').select('is_completed').eq('id', 5)`

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: ไม่สามารถยุ่งเกี่ยวหรือยกระดับฮาร์ดแวร์จริงอย่าง CPU/RAM ของเครื่องปลายทางได้

• ข. ถูก: BaaS (เช่น Supabase) ถูกดีไซน์มาเพื่อประหยัดแรงผู้พัฒนาแอปโดยการจัดการหลังบ้าน เช่น Database และ Auth ให้สำเร็จรูปบนระบบ Cloud

• ค. และ ง. ผิด: ไม่เกี่ยวข้องกับ Flexbox และไม่มีฟังก์ชันช่วยย่อบีบอัดหน้าจอแอป

เฉลย ข้อ 2: ตอบ ค.

• เหตุผลประกอบ:

• ก. ข. และ ง. ผิด: Supabase เลือกใช้งานฐานข้อมูลประเภท Relational SQL และใช้ PostgreSQL เป็นแกนหลักขับเคลื่อนเท่านั้น

• ค. ถูก: โครงสร้างดาต้าเบสเบื้องหลังของ Supabase คือ PostgreSQL 100% ทำให้สนับสนุนฟีเจอร์อย่าง SQL, Constraints และความสัมพันธ์ข้อมูลสมบูรณ์

เฉลย ข้อ 3: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: คีย์สูงสุดคือ Service Role Key ซึ่งห้ามนำมาแปะลงในแอปมือถือเด็ดขาด เพราะข้ามระบบความปลอดภัยได้หมด

• ข. ถูก: Anon Key เป็นคีย์ระดับสาธารณะที่อนุญาตให้ลูกค้ายิง API ค้นหาตารางได้ โดยคีย์นี้จะทำงานร่วมกับระบบ RLS (Row Level Security) เพื่อจำกัดความปลอดภัยของตารางข้อมูล

• ค. และ ง. ผิด: ไม่ใช่รหัสสำหรับ Android หรือสิทธิ์ในการส่งแอปขึ้น Play Store

เฉลย ข้อ 4: ตอบ ค.

• เหตุผลประกอบ:

• ก. ข. และ ง. ผิด: ตัวแปรอื่นจะไม่ได้รับการดึงเข้าระบบจาวาสคริปต์อัตโนมัติในฝั่ง Expo Client

• ค. ถูก: ใน Expo SDK 54 ตัวแปรในไฟล์ **.env** ที่แอปจะอ่านเข้าไปรันใช้งานได้ในโค้ดดิ้งฝั่งไคลเอนต์ จำเป็นต้องมีคำนำหน้า (Prefix) เป็น `**EXPO_PUBLIC_**` เสมอ

เฉลย ข้อ 5: ตอบ ข.

• เหตุผลประกอบ:

- **ก. ผิด:** ไม่เกี่ยวข้องกับการย่อขนาดไฟล์รูปภาพใด ๆ
- **ข. ถูก:** รันไทม์ของ React Native ต่างจากเว็บบราวเซอร์ตรงที่ไม่มีคลาส `URL` หรือ `URLSearchParams` ดั้งเดิม การนำเข้าตัว polyfill จะชดเชยช่องว่างนี้ เพื่อให้ Supabase JS Client ส่ง URL รีเควสเชื่อมต่อได้สมบูรณ์โดยแอปไม่ระเบิด
- **ค. และ ง. ผิด:** ไม่สามารถควบคุมการเปิดบลูทูธเครื่อง หรือแก้ไขฟอร์มภาษาไทยได้โดยออโต้

เฉลย ข้อ 6: ตอบ ข.

- **เหตุผลประกอบ:**
  - **ก. ผิด:** `.select()` คุมการค้นหาคอลัมน์
  - **ข. ถูก:** `.from('ชื่อตาราง')` บ่งชี้เฉพาะถึงแหล่งข้อมูลตารางเป้าหมายที่ลงทะเบียนไว้ใน Supabase
  - **ค. ผิด:** `.insert()` คุมการสร้างแถวข้อมูล
  - **ง. ผิด:** `.eq()` ทำหน้าที่กรองค่าเงื่อนไข

เฉลย ข้อ 7: ตอบ ก.

- **เหตุผลประกอบ:**
  - **ก. ถูก:** โครงสร้างที่สมบูรณ์คือระบุตารางด้วย `.from()`, เลือกคอลัมน์ทั้งหมดด้วย `.select('*')` และสั่งเรียงข้อมูลตามเวลาด้วย `.order('created_at', { ascending: false })`
  - **ข. ค. และ ง. ผิด:** รูปแบบอื่นระบุคำสั่งไม่ถูกประเภท หรือมีพารามิเตอร์ผิดจุดประสงค์

เฉลย ข้อ 8: ตอบ ข.

- **เหตุผลประกอบ:**
  - **ก. ค. และ ง. ผิด:** เมธอดของ insert ปฏิเสธค่าข้อความเพลน ๆ หรือค่าทศนิยมเดี่ยว และไม่ใช่ฟังก์ชัน
  - **ข. ถูก:** พารามิเตอร์ของ `.insert()` จะต้องรับค่าเป็น JSON Array เสมอ แม้ว่าเราจะยิงเพื่อสร้างเพียงแถวเดียว ก็ยังต้องส่งเป็น Array ที่มีออบเจกต์ชิ้นเดียวนอนอยู่ด้านใน

เฉลย ข้อ 9: ตอบ ข.

- **เหตุผลประกอบ:**
  - **ก. ผิด:** ไม่ใช่ความปลอดภัยในการจำกัดรหัสผ่านบนฮาร์ดแวร์หน้าจอมือถือ

• **ข. ถูก:** Row Level Security (RLS) ของ PostgreSQL/Supabase ทำหน้าที่เช็คว่าแถวนี้เขียนค่านำเข้าโดยผู้ใช้คนไหน และใช้บล็อกไม่ให้คนนอกเขียนทับหรืออ่านข้อมูลข้ามบัญชีของคนอื่น

• **ค. และ ง. ผิด:** สไตล์ TextInput คุมด้วย CSS และ RAM คุมด้วยรันไทม์ระบบไม่ใช่ RLS

เฉลย ข้อ 10: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** เมธอด `.update()` จะยอมรับฟิลด์ใหม่ และต้องสั่งกรองระบุเฉพาะตัวด้วยเมธอด `.eq('id', 5)` เพื่อให้แก้เฉพาะไอดี 5

• **ข. ผิด:** insert ใช้สร้างแถวใหม่ ไม่ใช่แก้ไขค่า

• **ค. ผิด:** delete เป็นคำสั่งถอนลบข้อมูลไม่ใช่การเปลี่ยนสถานะ

• **ง. ผิด:** select เป็นเพียงการเรียกรับค่าเดิมมาแสดง

ปฏิบัติการท้ายบทเรียน: การสร้างแอปพลิเคชันซิงก์ข้อมูลสินค้าโปรดด้วย Supabase CRUD

ปฏิบัติการนี้จะฝึกทักษะการรวบรวมฟังก์ชันการติดต่อกับคลาวด์ดาต้าเบสระดับโลกอย่าง Supabase โดยผู้เรียนจะได้ออกแบบแอปพลิเคชัน **"สมุดจดสินค้าที่ต้องการและรายการโปรดออนไลน์ (Supabase Wishlist Manager)"** ซึ่งผู้ใช้สามารถพิมพ์บันทึกชื่อสินค้าและราคา (Create / Insert) โหลดลิสต์สินค้ามาเรนเดอร์ในแอป (Read / Select) กดจิ้มหัวใจสลับสถานะสินค้าโปรด (Update) และกดถอนรายการทิ้งออกจากตาราง (Delete) โดยทุกจังหวะการทำงานจะซิงก์ประสานกับตาราง SQL บนเซิร์ฟเวอร์ Supabase แบบเรียลไทม์

1. วัตถุประสงค์การเรียนรู้

1. สามารถตั้งค่าและลงทะเบียนใช้งาน `@supabase/supabase-js` ร่วมกับระบบคีย์ลับสภาพแวดล้อม `.env` ได้สำเร็จ

2. สามารถพัฒนาฟังก์ชันสำหรับค้นหาดึงข้อมูล (GET/Select) แถวจากคลาวด์มาเรนเดอร์ลงในคอมโพเนนต์ `FlatList`

3. สามารถเขียนระบบฟอร์มนำเข้าและยิงส่งข้อมูลแถวใหม่ (POST/Insert) ขึ้นสู่ตารางคลาวด์ได้ถูกต้อง

4. สามารถระบุคำสั่งแก้ไขค่าฟิลด์ตาราง (PATCH/Update) และคำสั่งลบข้อมูล (DELETE/Delete) ร่วมกับการคัดกรอง `.eq()`

5. สามารถจัดการสถานะกล่องความคืบหน้า (Loading) และรับมือข้อผิดพลาดเครือข่ายฐานข้อมูลได้ถูกต้อง

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องสร้างโครงสร้างโปรเจกต์ประกอบด้วยไฟล์ดังต่อไปนี้:

```
โครงสร้างไฟล์ปฏิบัติการ:
├── .env                                  => บรรจุคีย์โครงการ EXPO_PUBLIC_SUPABASE_URL และ EXPO_PUBLIC_SUPABASE_ANON_KEY
├── lib/
│   └── supabase.ts                       => สร้าง Supabase Client
└── app/
    └── index.tsx                         => หน้าจอหลักควบคุมการทำงานของแอป Wishlist (CRUD Interface)
```

ข้อมูลโครงสร้างตารางหลังบ้านบน Supabase (Table Schema):

ผู้เรียนต้องไปที่ Supabase SQL Editor หรือ Table Editor เพื่อเปิดสร้างตารางชื่อ `wishlist` ประกอบด้วยคอลัมน์ดังนี้:

- `id` (int8 / identity): คีย์หลัก (Primary Key) สั่งสร้างรหัสอัตโนมัติ
- `title` (text): ชื่อของสินค้าที่อยากได้
- `price` (numeric): ราคาสินค้า
- `is_favorite` (boolean): สถานะความชอบพิเศษ (ค่าเริ่มต้น: `false`)
- `created_at` (timestamptz): วันเวลาบันทึก (ค่าเริ่มต้น: `now()`)

เงื่อนไขการจัดการข้อมูลในแอปพลิเคชัน:

1. **การโหลดข้อมูลเริ่มต้น (Read):** ดึงข้อมูลจากตาราง `wishlist` ทั้งหมดมาแสดง เรียงตามเวลาล่าสุด

2. **การบันทึกชิ้นใหม่ (Create):** มีฟอร์มรับอินพุตกรอกชื่อสินค้าและราคา เมื่อส่งสำเร็จให้นำผลลัพธ์ข้อมูลแถวล่าสุดที่ได้ย้อนกลับมาจาก Supabase เสียบแทรกด้านบนสุดของสเตตลิสต์ทันที

3. **การสลับหัวใจรายการโปรด (Update):** เมื่อกดแตะรูปสัญลักษณ์หัวใจ ให้ยิงอัปเดตสลับค่าบูลีนของ `is_favorite` เฉพาะไอดีนั้น ๆ บนคลาวด์

4. **การถอนรายการ (Delete):** เมื่อกดไอคอนถังขยะ ให้ลบแถวนั้นทิ้ง และรีแฟกเตอร์ฟิลเตอร์กรองแถว ID นั้นออกจากจอแอป

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งคลังไลบรารีและสร้างระบบคีย์ลับ

สั่งรันติดตั้งคำสั่งในแอปพลิเคชัน:

```
npm install @supabase/supabase-js react-native-url-polyfill
```

จากนั้นสร้างไฟล์ .env ที่โฟลเดอร์นอกสุดของแอป ป้อน URL และ Anon Key ที่ได้มาจาก Supabase:

```
EXPO_PUBLIC_SUPABASE_URL=https://your-supabase-url.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=your-supabase-anon-key-here
```

ขั้นตอนที่ 2: ตั้งค่าโมดูล Client ใน `lib/supabase.ts`

สร้างไฟล์ `lib/supabase.ts` เพื่ออำนวยความสะดวกในการจัดแชร์ออบเจกต์คำสั่ง:

```
import 'react-native-url-polyfill/auto';
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL || '';
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY || '';

export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

ขั้นตอนที่ 3: เขียนกระบวนการ CRUD เชื่อม Supabase

สร้างฟังก์ชันสำหรับทริกเกอร์เหตุการณ์ต่าง ๆ:

```
// ดึงข้อมูล Select
const fetchItems = async () => {
  const { data, error } = await supabase
  .from('wishlist')
  .select('*')
  .order('created_at', { ascending: false });
  if (!error) setWishlist(data || []);
};

// เขียนบันทึก Insert
const handleAddItem = async () => {
  const { data, error } = await supabase
  .from('wishlist')
  .insert([{ title, price: parseFloat(price), is_favorite: false }])
  .select();
  if (!error) setWishlist([data[0], ...wishlist]);
};
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถนำโค้ดอินเทอร์เฟซ Wishlist ฉบับสมบูรณ์ชุดนี้บันทึกใส่ลงในไฟล์ app/index.tsx เพื่อสั่งทำงานและซิงก์ฐานข้อมูลออนไลน์แบบเรียลไทม์กับโครงการ Supabase:

```
import React, { useState, useEffect } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  TouchableOpacity,
  FlatList,
  ActivityIndicator,
  Alert,
  SafeAreaView,
  Keyboard
} from 'react-native';
import { supabase } from '../lib/supabase';

// โครงสร้างประเภทข้อมูล Wishlist
interface WishItem {
  id: number;
  title: string;
  price: number;
  is_favorite: boolean;
  created_at: string;
}

export default function SupabaseWishlistManager() {
  const [wishlist, setWishlist] = useState<WishItem[]>([]);
  const [title, setTitle] = useState('');
  const [price, setPrice] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [isSubmitting, setIsSubmitting] = useState(false);

 // 1. เรียกดึงข้อมูลตารางตอนเริ่มต้นเปิดแอป
  useEffect(() => {
    fetchWishlist();
  }, []);

 // 2. ฟังก์ชัน READ: ค้นหาข้อมูลสินค้าทั้งหมด
  const fetchWishlist = async () => {
    try {
      setIsLoading(true);
      const { data, error } = await supabase
    .from('wishlist')
    .select('*')
```

```
    .order('created_at', { ascending: false });

      if (error) {
        throw error;
      }
      setWishlist(data || []);
    } catch (error: any) {
      Alert.alert('เกิดข้อผิดพลาด', error.message || 'ไม่สามารถโหลดข้อมูลคลาวด์ได้');
    } finally {
      setIsLoading(false);
    }
  };

// 3. ฟังก์ชัน CREATE: บันทึกข้อมูลสินค้าแถวใหม่
  const handleAddItem = async () => {
    const parsedPrice = parseFloat(price);

    if (!title.trim() || isNaN(parsedPrice)) {
      Alert.alert('ข้อมูลไม่ครบ', 'กรุณาระบุชื่อสินค้าและราคาให้ถูกต้อง');
      return;
    }

    try {
      setIsSubmitting(true);
      Keyboard.dismiss();

    // ยิงข้อมูลขึ้นคลาวด์ตาราง wishlist
      const { data, error } = await supabase
    .from('wishlist')
    .insert([
          {
            title: title.trim(),
            price: parsedPrice,
            is_favorite: false
          }
    ])
    .select();

      if (error) {
        throw error;
      }

    // พ่วงข้อมูลใหม่ต่อหน้าสุดของลิสต์บนจอ
      if (data && data.length > 0) {
        setWishlist([data[0], ...wishlist]);
        setTitle('');
        setPrice('');
        Alert.alert('สำเร็จ!', 'บันทึกรายการโปรดของคุณเรียบร้อย');
      }
```

```
    } catch (error: any) {
      Alert.alert('บันทึกไม่สำเร็จ', error.message || 'เกิดข้อขัดข้องในการซิงก์ข้อมูล');
    } finally {
      setIsSubmitting(false);
    }
  };

 // 4. ฟังก์ชัน UPDATE: อัปเดตสลับสถานะความชอบสินค้าโปรด (Favorite status)
  const handleToggleFavorite = async (itemId: number, currentStatus:
boolean) => {
    try {
      const { error } = await supabase
    .from('wishlist')
    .update({ is_favorite: !currentStatus })
    .eq('id', itemId);

      if (error) {
        throw error;
      }

    // อัปเดตสเตตบนหน้าจอทันทีเพื่อแสดงสีหัวใจที่สลับตามจริง
      setWishlist(
        wishlist.map((item) =>
          item.id === itemId ? { ...item, is_favorite: !currentStatus } :
item
    )
   );
    } catch (error: any) {
      Alert.alert('แก้ไขข้อมูลล้มเหลว', error.message);
    }
  };

 // 5. ฟังก์ชัน DELETE: ถอดรายการสินค้าทิ้ง
  const handleDeleteItem = async (itemId: number) => {
    Alert.alert(
      'ยืนยันการลบ',
      'คุณแน่ใจหรือไม่ว่าต้องการนำรายการโน้ตนี้ออกถาวร?',
   [
        { text: 'ยกเลิก', style: 'cancel' },
        {
          text: 'ลบทิ้ง',
          style: 'destructive',
          onPress: async () => {
            try {
              const { error } = await supabase
        .from('wishlist')
        .delete()
        .eq('id', itemId);
```

```
              if (error) {
                throw error;
              }

        // กรองลบไอดีออกจากหน้าจอ
              setWishlist(wishlist.filter((item) => item.id !== itemId));
      } catch (error: any) {
              Alert.alert('ไม่สามารถลบข้อมูลได้', error.message);
            }
          }
        }
  ]
 );
 };

 return (
   <SafeAreaView style={styles.safeArea}>
     <View style={styles.container}>

    {/* หัวข้อแดชบอร์ด */}
        <View style={styles.header}>
          <Text style={styles.headerTitle}>🛒 Supabase Wishlist</Text>
          <Text style={styles.headerSubtitle}>จัดการความต้องการชิงก์คลาวด์
เรียลไทม์</Text>
        </View>

        {/* แบบฟอร์มเพิ่มรายการสินค้าใหม่ */}
        <View style={styles.formCard}>
          <TextInput
            style={styles.input}
            placeholder="ชื่อสินค้าที่อยากได้ (เช่น iPhone 16)..."
            placeholderTextColor="#64748b"
            value={title}
            onChangeText={setTitle}
    />
          <TextInput
            style={styles.input}
            placeholder="ระบุราคาสินค้า (บาท)..."
            placeholderTextColor="#64748b"
            value={price}
            onChangeText={setPrice}
            keyboardType="numeric"
    />

          {isSubmitting ? (
            <ActivityIndicator size="small" color="#38bdf8"
style={styles.spinner} />
```

```
      ):(
            <TouchableOpacity style={styles.submitButton}
onPress={handleAddItem}>
              <Text style={styles.submitButtonText}>➕ บันทึกข้อมูลขึ้น
คลาวด์</Text>
            </TouchableOpacity>
      )}
        </View>

        {/* รายการแสดงผล Wishlist */}
        <View style={styles.listSection}>
        <View style={styles.listHeader}>
            <Text style={styles.listTitle}>ของที่คุณอยากได้</Text>
            <TouchableOpacity onPress={fetchWishlist}
disabled={isLoading}>
              <Text style={styles.refreshText}>{isLoading ? 'กำลังซิงก์...' :
'🔄 รีเฟรชฐานข้อมูล'}</Text>
            </TouchableOpacity>
          </View>

          {isLoading ? (
            <View style={styles.centerLoading}>
              <ActivityIndicator size="large" color="#38bdf8" />
              <Text style={styles.loadingText}>กำลังดึงข้อมูลตารางสินค้าโปรด...</Text>
            </View>
      ):(
            <FlatList
              data={wishlist}
              keyExtractor={(item) => item.id.toString()}
              renderItem={({ item }) => (
          <View style={styles.itemCard}>
                  <View style={styles.itemInfo}>
                    <Text style={styles.itemTitle}>{item.title}</Text>
                    <Text style={styles.itemPrice}>฿
{item.price.toLocaleString()}</Text>
            </View>

                  {/* ปุ่มเปลี่ยนสถานะ Favorite และปุ่มลบ */}
                  <View style={styles.actionRow}>
                    <TouchableOpacity
                      onPress={() => handleToggleFavorite(item.id,
item.is_favorite)}
                      style={styles.actionIcon}
                    >
                      <Text style={styles.heartText}>{item.is_favorite ?
'❤️' : '🤍'}</Text>
                    </TouchableOpacity>

                    <TouchableOpacity
                      onPress={() => handleDeleteItem(item.id)}
```

```
                        style={styles.actionIcon}
                      >
                        <Text style={styles.trashText}>🗑️</Text>
                      </TouchableOpacity>
                </View>
                    </View>
          )}
                ListEmptyComponent={
                  <View style={styles.emptyContainer}>
                    <Text style={styles.emptyText}>ไม่มีข้อมูลสินค้าในตารางของคุณ
ขณะนี้</Text>
                  </View>
                }
        />
      )}
          </View>
        </View>
      </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  container: {
    flex: 1,
    padding: 16,
  },
  header: {
    alignItems: 'center',
    marginVertical: 10,
  },
  headerTitle: {
    fontSize: 22,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  headerSubtitle: {
    fontSize: 12,
    color: '#94a3b8',
    marginTop: 4,
  },
  formCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 16,
    borderWidth: 1,
```

```
    borderColor: '#334155',
    marginBottom: 20,
  },
  input: {
    backgroundColor: '#0f172a',
    borderRadius: 8,
    borderWidth: 1,
    borderColor: '#334155',
   color: '#ffffff',
    padding: 12,
    fontSize: 14,
    marginBottom: 10,
  },
  submitButton: {
    backgroundColor: '#38bdf8',
    borderRadius: 8,
    height: 44,
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 4,
  },
  submitButtonText: {
    color: '#0f172a',
    fontWeight: 'bold',
    fontSize: 14,
  },
  spinner: {
    paddingVertical: 10,
  },
  listSection: {
    flex: 1,
  },
  listHeader: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 12,
  },
  listTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  refreshText: {
    color: '#38bdf8',
    fontSize: 13,
  },
  centerLoading: {
```

```
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  loadingText: {
    color: '#94a3b8',
    marginTop: 8,
    fontSize: 13,
  },
  itemCard: {
    flexDirection: 'row',
    backgroundColor: '#1e293b',
    borderRadius: 12,
    padding: 14,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 10,
    alignItems: 'center',
    justifyContent: 'space-between',
  },
  itemInfo: {
    flex: 1,
  },
  itemTitle: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 15,
    textTransform: 'capitalize',
  },
  itemPrice: {
    color: '#10b981',
    fontWeight: 'bold',
    fontSize: 13,
    marginTop: 4,
  },
  actionRow: {
    flexDirection: 'row',
    alignItems: 'center',
  },
  actionIcon: {
    marginLeft: 16,
    padding: 6,
  },
  heartText: {
    fontSize: 18,
  },
  trashText: {
    fontSize: 16,
  },
```

```
  emptyContainer: {
    alignItems: 'center',
    paddingVertical: 40,
  },
  emptyText: {
    color: '#64748b',
    fontSize: 14,
  },
});
```

เอกสารอ้างอิง

Supabase. (2024). *React Native Supabase Quickstart*. จาก https://supabase.com/docs/guides/getting-started/tutorials/with-react-native

Supabase. (2024). *Supabase Database Quickstart*. จาก https://supabase.com/docs/guides/database/overview

Supabase. (2024). *Supabase JavaScript Client Database API*. จาก https://supabase.com/docs/reference/javascript/select

แผนบริหารการสอนประจำบทที่ 11

หัวข้อเนื้อหาประจำบท

11.1 กลไกการจัดการสิทธิ์กล้องถ่ายรูปและคลังภาพ (Camera & Media Permissions)

11.2 การใช้งาน CameraView เพื่อพรีวิวภาพสด (Camera Live Preview)

11.3 การลั่นชัตเตอร์ถ่ายภาพนิ่ง (Taking Photos with takePictureAsync)

11.4 การเลือกรูปภาพและสื่อผ่าน expo-image-picker (Image Library API)

11.5 ระบบจัดการไฟล์แบบที่ 1: Private Sandbox Storage (แอปมองเห็นเฉพาะตัว)

11.6 ระบบจัดการไฟล์แบบที่ 2: Shared Media Storage (แอปอื่นมองเห็นและเข้าถึงข้อมูลร่วมกันได้)

11.7 ระบบจัดการไฟล์แบบที่ 3: Cloud Storage Network (อัปโหลด/ดาวน์โหลดผ่าน Supabase Storage)

สรุปเปรียบเทียบระบบจัดเก็บไฟล์ทั้ง 3 รูปแบบ

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถเขียนโครงสร้างขออนุญาตสิทธิ์เข้าถึงกล้อง คลังภาพ และการบันทึกสื่อลงเครื่องตามมาตรฐานความปลอดภัยได้ถูกต้อง
2. ผู้เรียนสามารถประยุกต์ใช้คอมโพเนนต์ CameraView ในการเรนเดอร์หน้าจอเลนส์กล้องพรีวิวสดได้สำเร็จ
3. ผู้เรียนสามารถเขียนปุ่มคำสั่งควบคุมการถ่ายรูปและรับค่าพาธรูปภาพชั่วคราว (Image URI) มาใช้งานได้ถูกต้อง
4. ผู้เรียนสามารถดึงภาพถ่ายจากอัลบั้มเครื่องผู้ใช้ขึ้นมาแสดงผลเป็นอวาตาร์กลมด้วย expo-image-picker ได้สำเร็จ
5. ผู้เรียนสามารถอธิบายความแตกต่างและเปรียบเทียบการใช้งาน Private Sandbox (expo-file-system) กับ Shared Gallery (expo-media-library) ได้อย่างถูกต้อง
6. ผู้เรียนสามารถเรียกใช้ expo-file-system คัดลอกรูปภาพมาเก็บไว้ในพื้นที่โฟลเดอร์

ถาวรของแอปเพื่อป้องกันไฟล์หายได้สำเร็จ

7. ผู้เรียนสามารถบันทึกภาพถ่ายเนทีฟลงคลังรูปภาพส่วนกลางของระบบเครื่องด้วย expo-media-library เพื่อให้แอปพลิเคชันอื่นเข้าถึงรูปได้สำเร็จ

8. ผู้เรียนสามารถส่งอัปโหลดไฟล์ภาพโปรไฟล์ขึ้นสู่ระบบคลาวด์ผ่าน Supabase Storage และดึง URL สาธารณะมาใช้แสดงผลในแอปพลิเคชันได้สำเร็จ

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

41. **การบรรยายการเชื่อมโยงระบบสื่อและแฟ้มข้อมูล 3 ระดับ (Three-Level File Storage Lecture):** บรรยายการสร้าง View ของกล้องถ่ายรูป, โครงสร้างของ Image Picker และเจาะลึกความแตกต่างระหว่างไฟล์ในแคชชั่วคราว, สารระบบ Sandbox ของแอป, คลังภาพสาธารณะของระบบปฏิบัติการ และฐานข้อมูลจัดเก็บไฟล์รูปภาพบนคลาวด์ (Supabase Storage)
42. **การทำ Live Coding Demonstration หัวข้อ Multi-Destination Profile Image Saver:** ผู้สอนสาธิตการออกแบบระบบอวาตาร์ โดยถ่ายรูปหรือเลือกภาพจากอัลบั้ม แล้วประมวลผลเซฟพร้อมกัน 3 ช่องทาง: คัดลอกลงโฟลเดอร์ Document ส่วนตัวของแอป, บันทึกส่งออกคลังรูปภาพสาธารณะของเครื่อง และอัปโหลดไฟล์ตรงเข้าสู่ Supabase Storage Bucket
43. **กิจกรรมสลับหน้าเลนส์และสิทธิ์ความปลอดภัย (Facing & Permissions Challenge):** ให้ผู้เรียนทดลองเพิ่มระบบขอยื่นสิทธิ์ของกล้องและคลังภาพอย่างสมบูรณ์แบบ และทดลองสลับกล้องหน้า/หลังในการพรีวิว
44. **กิจกรรมฝึกฝนอัปโหลดและดาวน์โหลดผ่าน Cloud Network (Supabase Storage Challenge):** ให้กลุ่มผู้เรียนสร้างการแปลงภาพถ่ายจากพาธ URI ดิบเป็น ArrayBuffer หรือ Base64 เพื่อยิงส่งอัปโหลดเข้า Bucket และดึง URL ลิงก์ตรงมาป้อนเข้าคอมโพเนนต์แสดงผลภาพ

สื่อการเรียนการสอน

40. เอกสารสไลด์เรื่อง "Native Camera View, Image Picker and Three-Level Storage Systems in React Native"
41. เครื่องสมาร์ทโฟนจริงสำหรับเสียบทดสอบการทำงานของกล้อง แกลเลอรี และการบันทึกรูปภาพจริงลงดิสก์
42. ข้อมูลบัญชีผู้ใช้บริการ Supabase สำหรับการสร้าง Bucket และกำหนดนโยบาย

RLS ของ Storage

43. ลิงก์เอกสารอ้างอิงของ API expo-camera, expo-image-picker, expo-file-system, expo-media-library และ Supabase JS Client

การวัดผลและการประเมินผล

1. การวัดผล

1.1 ประเมินผลคะแนนการทำแบบทดสอบท้ายบทเรียน 10 ข้อ

1.2 ตรวจสอบโครงสร้างความถูกต้องของการอนุญาตสิทธิ์กล้อง อัลบั้มภาพ และการจัดการเขียนไฟล์ลงในระบบเอกสาร

1.3 สังเกตขั้นตอนการส่งอัปโหลดไฟล์ขึ้นระบบคลาวด์และเรียกคืนภาพผ่านโครงข่ายเครือข่าย

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถสร้างโมดูลเปลี่ยนรูปถ่ายโปรไฟล์ที่บันทึกข้อมูลภาพลงแซนด์บ็อกซ์ของแอป, อัลบั้มแกลเลอรีส่วนกลาง และ Supabase Storage ได้สำเร็จผ่านเกณฑ์ (ผ่าน/ไม่ผ่าน)

2.3 สังเกตทักษะความประณีตในการออกแบบหน้าจอกล้อง และความถูกต้องปลอดภัยในการเชื่อมต่อกับบริการคลาวด์

บทที่ 11

การใช้งานกล้อง อัลบั้มภาพ และระบบจัดเก็บไฟล์เนทีฟร่วมกับคลาวด์

การสร้างสรรค์โมบายแอปพลิเคชันที่ต้องทำงานกับรูปภาพ ไม่เพียงแค่การถ่ายรูปหรือดึงภาพมาแสดงบนหน้าจอเท่านั้น แต่นักพัฒนาจำเป็นต้องมีความเข้าใจอย่างลึกซึ้งเกี่ยวกับการจัดการพื้นที่จัดเก็บไฟล์ (File Storage) เพื่อให้ภาพที่ได้ไม่สูญหายและถูกเผยแพร่ในระดับสิทธิ์ที่เหมาะสม ในบทเรียนนี้เราจะเรียนรู้ตั้งแต่การเข้าถึงกล้องและอัลบั้มภาพเนทีฟ ไปจนถึงระบบการจัดการไฟล์ 3 รูปแบบหลัก ได้แก่ ไฟล์เฉพาะแอป (Sandbox), ไฟล์แชร์ร่วมกับแอปอื่น (Shared Gallery) และไฟล์บนคลาวด์ (Supabase Cloud Storage)

11.1 กลไกการจัดการสิทธิ์กล้องถ่ายรูปและคลังภาพ (Camera & Media Permissions)

การเข้าถึงกล้องถ่ายรูปและการสืบค้นรูปภาพเก่าในเครื่องถือเป็นเรื่องละเอียดอ่อนต่อความเป็นส่วนตัว (Privacy) ระบบปฏิบัติการ iOS และ Android จึงมีระบบความปลอดภัยและบังคับร้องขอสิทธิ์แยกประเภทกันอย่างชัดเจน:

1. สิทธิ์การเปิดเลนส์กล้อง (Camera Permission)

สำหรับการใช้ถ่ายรูปสด ๆ ในแอป เราสามารถใช้ Hook `useCameraPermissions()` ของแพ็คเกจ `expo-camera` เพื่อตรวจสอบสถานะสิทธิ์และยื่นขอสิทธิ์ได้โดยตรง:

```
import { useCameraPermissions } from 'expo-camera';

const [permission, requestPermission] = useCameraPermissions();

// ตรวจสอบสถานะการได้รับสิทธิ์
if (!permission) {
  // สถานะยังคงกำลังโหลดเช็ค
}
if (!permission.granted) {
  // สิทธิ์ยังไม่ได้เปิดอนุมัติ ต้องกดเรียก requestPermission()
}
```

2. สิทธิ์การเข้าถึงคลังสื่อเพื่อเลือกรูปภาพ (Media Library / Photos Permission)

สำหรับการดึงภาพถ่ายเดิมจากคลังภาพมาใช้งานผ่าน `expo-image-picker` จะต้องมีการร้องขอสิทธิ์แยกออกต่างหาก:

```
import * as ImagePicker from 'expo-image-picker';
```

```
const requestGalleryPermission = async () => {
  const { status } = await
ImagePicker.requestMediaLibraryPermissionsAsync();
  if (status !== 'granted') {
    alert('จำเป็นต้องขอสิทธิ์เข้าถึงคลังภาพเพื่อเลือกรูปอวาตาร์');
  }
};
```

11.2 การใช้งาน CameraView เพื่อพรีวิวภาพสด (Camera Live Preview)

ใน Expo SDK 54 คอมโพเนนต์หลักที่ใช้เปิดช่องเลนส์รับภาพสดคือ `CameraView` (ถูกพัฒนามาทดแทนคอมโพเนนต์ Camera ตัวเดิม) (Expo, 2024)

พร็อพส์หลักที่ใช้ควบคุม <CameraView>:

• `facing`: กำหนดประเภทเลนส์ที่จะใช้เปิด รับค่า 'front' (กล้องหน้า) หรือ 'back' (กล้องหลัง)

• `autofocus`: เปิดหรือปิดระบบดักจับโฟกัสภาพอัตโนมัติ ('on' หรือ 'off')

• `ref`: ตัวเชื่อมโยง (Reference) เพื่อเรียกคำสั่งลั่นชัตเตอร์ถ่ายภาพนิ่ง

```
import React, { useState } from 'react';
import { StyleSheet, Text, View, Button, TouchableOpacity } from 'react-
native';
import { CameraView, CameraType, useCameraPermissions } from 'expo-
camera';

export default function CameraPreviewApp() {
  const [facing, setFacing] = useState<CameraType>('front');
  const [permission, requestPermission] = useCameraPermissions();

  if (!permission) return <View />; // กำลังโหลดเช็คสิทธิ์

  if (!permission.granted) {
    return (
      <View style={styles.container}>
        <Text style={styles.text}>แอปต้องการสิทธิ์เข้าใช้กล้องถ่ายรูป</Text>
        <Button onPress={requestPermission} title="อนุมัติสิทธิ์การเข้าใช้งานกล้อง" />
      </View>
  );
  }

  return (
    <View style={styles.container}>
      <CameraView style={styles.camera} facing={facing}>
        <View style={styles.buttonContainer}>
          <TouchableOpacity
            style={styles.button}
```

```
            onPress={() => setFacing(curr => curr === 'back' ? 'front' :
'back')}
          >
        <Text style={styles.buttonText}>🔄 สลับมุมกล้อง</Text>
          </TouchableOpacity>
        </View>
      </CameraView>
    </View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center' },
  camera: { flex: 1 },
 buttonContainer: { flex: 1, justifyContent: 'flex-end', margin: 40 },
  button: { backgroundColor: 'rgba(0,0,0,0.5)', padding: 12, borderRadius: 8,
alignItems: 'center' },
  buttonText: { color: 'white', fontWeight: 'bold' },
  text: { textAlign: 'center', marginBottom: 10 }
});
```

11.3 การลั่นชัตเตอร์ถ่ายภาพนิ่ง (Taking Photos with takePictureAsync)

เพื่อบันทึกรูปภาพพรีวิวเป็นรูปนิ่ง (Snapshot) เราจะอ้างอิง Ref ไปยังคอมโพเนนต์ `<CameraView>` แล้วเรียกใช้เมธอด Asynchronous ชื่อ **`takePictureAsync()`**:

```
import React, { useRef } from 'react';
import { CameraView } from 'expo-camera';
import { TouchableOpacity, Text } from 'react-native';

export default function CaptureComponent() {
  const cameraRef = useRef<CameraView | null>(null);

  const takePhoto = async () => {
    if (cameraRef.current) {
      const result = await cameraRef.current.takePictureAsync({
        quality: 0.8,          // คุณภาพการบีบอัดภาพ (0.0 ถึง 1.0)
        skipProcessing: false // จัดการหมุนทิศทางภาพถ่ายให้อัตโนมัติ
      });
   // ผลลัพธ์จะคืนค่าเป็นอ็อบเจกต์ที่มี URI ไฟล์ภาพชั่วคราวชี้ที่หน่วยความจำแคช
      console.log('ที่อยู่รูปถ่ายชั่วคราว:', result?.uri);
    }
  };

  return (
    <CameraView ref={cameraRef} style={{ flex: 1 }}>
      <TouchableOpacity onPress={takePhoto} style={{ alignSelf: 'center',
```

```
margin: 40 }}>
        <Text style={{ color: 'white', fontSize: 18, fontWeight: 'bold'
}}>📷 ลั่นชัตเตอร์</Text>
      </TouchableOpacity>
    </CameraView>
 );
}
```

11.4 การเลือกรูปภาพและสื่อผ่าน expo-image-picker (Image Library API)

ในการพัฒนาฟีเจอร์ดึงไฟล์ภาพที่มีอยู่แล้วในคลังอัลบั้มของเครื่องขึ้นมาจัดแสดง เราจะใช้ไลบรารีมาตรฐานยอดนิยมคือ `expo-image-picker` (Expo, 2024) ผ่านคำสั่ง Asynchronous ชื่อ `launchImageLibraryAsync()`:

```
import React, { useState } from 'react';
import { View, Button, Image, StyleSheet } from 'react-native';
import * as ImagePicker from 'expo-image-picker';

export default function PickImageApp() {
  const [imageUri, setImageUri] = useState<string | null>(null);

  const pickImage = async () => {
    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ImagePicker.MediaTypeOptions.Images, // คัดเฉพาะประเภทไฟล์ภาพนิ่ง
      allowsEditing: true, // เปิดกรอบแก้ไข/ตัดรูปภาพหน้าเนทีฟของอุปกรณ์
      aspect: [1, 1],       // บังคับครอปเป็นสัดส่วนจัตุรัส กว้าง:สูง
      quality: 0.8,         // กำหนดความละเอียดผลลัพธ์
    });

    if (!result.canceled && result.assets && result.assets.length > 0) {
      setImageUri(result.assets[0].uri);
    }
  };

  return (
    <View style={styles.container}>
      <Button title="เลือกรูปภาพจากอัลบั้ม" onPress={pickImage} />
      {imageUri && <Image source={{ uri: imageUri }} style={styles.image}
/>}
    </View>
 );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center' },
```

```
  image: { width: 150, height: 150, borderRadius: 75, marginTop: 15 }
});
```

11.5 ระบบจัดการไฟล์แบบที่ 1: Private Sandbox Storage (แอปมองเห็นเฉพาะตัว)

รูปภาพที่ได้จากการสั่งถ่ายด้วยกล้องถ่ายรูป หรือได้จากการดึงภาพของ Image Picker จะได้รับการสร้างเป็นที่อยู่พาธชั่วคราวอยู่ใน **Cache Directory** เสมอ (Expo, 2024) (เช่น `file:///.../Caches/xxx.jpg`)

ในระบบปฏิบัติการ iOS และ Android พื้นที่ความจำของแอปพลิเคชันจะถูกจำกัดอยู่ในเขตปลอดภัยที่เรียกว่า **Sandbox** โดยแบ่งออกเป็นสองระดับหลัก ๆ:

1. **Cache Folder (`cacheDirectory`)**: ใช้เก็บผลลัพธ์ชั่วคราว มีข้อเสียหลักคือ **OS สามารถกวาดล้างลบไฟล์ออกได้อัตโนมัติ** เมื่ออุปกรณ์ต้องการทวงคืนพื้นที่ดิสก์ว่างคืน หากเปิดแอปขึ้นมาอีกครั้ง ภาพโปรไฟล์ก็อาจชำรุดเสียหายได้

2. **Documents Folder (`documentDirectory`)**: เป็นโฟลเดอร์สำหรับเอกสารสำคัญของแอป **ไม่มีทางโดนระบบปฏิบัติการลบทิ้งอย่างเด็ดขาด** ยกเว้นตัวผู้ใช้งานจะเป็นคนกดถอนการติดตั้งแอปพลิเคชัน (Uninstall) ออกไปด้วยตัวเอง

ดังนั้น หากเราจำเป็นต้องทำระบบบันทึกรูปโปรไฟล์ออฟไลน์ให้อยู่ได้คงทนถาวร เราต้องคัดลอกไฟล์ย้ายออกจากห้องแคชด้วยไลบรารี **`expo-file-system`**:

```
import * as FileSystem from 'expo-file-system';

const copyToPrivateStorage = async (temporaryUri: string): Promise<string
| null> => {
  try {
    const filename = `profile_avatar_${Date.now()}.jpg`; // ตั้งชื่อใหม่ไม่ให้ชนกัน
    const destinationUri = `${FileSystem.documentDirectory}${filename}`;

  // ย้ายไฟล์จาก Cache ไปยังโฟลเดอร์ Document ส่วนตัวของแอป
    await FileSystem.copyAsync({
      from: temporaryUri,
      to: destinationUri,
    });

    console.log('ไฟล์ถูกเก็บไว้แบบถาวรเฉพาะตัวแอปที่:', destinationUri);
    return destinationUri; // คืนค่าที่อยู่ใหม่เพื่อนำไปเซฟลง AsyncStorage ต่อไป
  } catch (error) {
    console.error('คัดลอกไฟล์ Sandbox ล้มเหลว:', error);
    return null;
  }
};
```

11.6 ระบบจัดการไฟล์แบบที่ 2: Shared Media Storage (แอปอื่นมองเห็นและเข้าถึงข้อมูลร่วมกันได้)

หากแอปพลิเคชันของเราต้องการถ่ายรูปภาพ แล้วบันทึกส่งออกไปจัดแสดงอยู่ในแกลเลอรีหลักของโทรศัพท์มือถือ (Camera Roll / Photos App) เพื่อให้แอปพลิเคชันภายนอก (เช่น Facebook, Instagram, LINE) ค้นหาและหยิบภาพนั้นไปใช้งานต่อได้ เราจะไม่บันทึกใน Sandbox แต่จะต้องบันทึกเข้าสู่คลังภาพสาธารณะของระบบ โดยใช้ไลบรารี `expo-media-library`

การติดตั้ง:

```
npx expo install expo-media-library
```

การตั้งค่าการขอสิทธิ์เข้าใช้งานใน app.json (สำหรับสร้าง Asset):

```
{
 "expo": {
  "plugins": [
   [
    "expo-media-library",
        {
     "photosPermission": "แอปต้องการขอสิทธิ์เข้าถึงเพื่อบันทึกไฟล์ภาพถ่ายลงสู่แกลเลอรีของคุณ"
        }
   ]
  ]
   }
}
```

ตัวอย่างการส่งออกไฟล์ไปที่คลังแกลเลอรีสาธารณะ:

เมื่อเรียกใช้งานคำสั่ง createAssetAsync รูปจะถูกนำไปบรรจุลงในคลังภาพส่วนกลางของตัวเครื่องทันที และเปิดโอกาสให้ระบบแอปอื่น ๆ สแกนตรวจพบภาพนี้ได้:

```
import * as MediaLibrary from 'expo-media-library';
import { Alert } from 'react-native';

const saveToSharedGallery = async (localUri: string) => {
  try {
  // 1. ตรวจสอบสิทธิ์เข้าถึงคลังสื่อในเครื่องก่อนเขียนข้อมูล
    const { status } = await MediaLibrary.requestPermissionsAsync();
    if (status !== 'granted') {
      Alert.alert('สิทธิ์ถูกปฏิเสธ', 'กรุณาอนุมัติสิทธิ์เข้าถึงแกลเลอรีสาธารณะ');
      return;
    }
```

```
    // 2. สร้างสินทรัพย์ภาพ (Asset) บันทึกลงเครื่องภายนอกแซนด์บ็อกซ์
      const asset = await MediaLibrary.createAssetAsync(localUri);
      Alert.alert('บันทึกสำเร็จ!', 'รูปภาพของท่านถูกจัดเก็บลงแกลเลอรีส่วนกลางแล้ว');
      console.log('Asset ID:', asset.id);
    } catch (error) {
      console.error('เกิดข้อผิดพลาดในการบันทึกลงแกลเลอรีหลัก:', error);
    }
};
```

11.7 ระบบจัดการไฟล์แบบที่ 3: Cloud Storage Network (อัปโหลด/ดาวน์โหลดผ่าน Supabase Storage)

หากรูปโปรไฟล์นั้นต้องซิงก์แลกเปลี่ยนนำไปเปิดดูในสมาร์ทโฟนของเพื่อน หรือนำไปแสดงผลบนเว็บไซต์กลาง ข้อมูลภาพถ่ายจะไม่สามารถเก็บอยู่เพียงในเครื่องเครื่องเดียวได้ เราจึงจำเป็นต้องอัปโหลดส่งไฟล์ผ่านทางเครือข่ายเครือข่าย (Network System) ขึ้นไปเก็บไว้บนคลาวด์ เช่น บริการ **Supabase Storage Buckets**

กลไกการอัปโหลดไฟล์รูปภาพใน React Native ไปยัง Supabase:

เนื่องจากสภาพแวดล้อมระบบ React Native ไม่มีฟังก์ชันจัดการ Blob หรือ File แบบมาตรฐานเดียวกันกับบราวเซอร์คอมพิวเตอร์ วิธีการส่งไฟล์ที่เสถียรที่สุดคือการอ่านไฟล์เป็นรูปแบบสตริงรหัส **Base64** จากดิสก์ด้วย `expo-file-system` แล้วทำการแกะถอดรหัสออกมาเป็นรหัสฐานข้อมูลแบบไบนารี **`ArrayBuffer`** ก่อนส่งยิงขึ้น API:

1. การติดตั้งไลบรารีแปลงข้อมูล Base64:

```
npm install base64-js
```

2. โค้ด Asynchronous สำหรับอัปโหลดขึ้น Supabase Storage:

ผู้พัฒนาต้องเข้าไปที่แดชบอร์ดของ Supabase แล้วเปิดสร้างโฟลเดอร์ฝากไฟล์ที่เรียกว่า **Bucket** (เช่นสร้างชื่อตารางฝากไฟล์ชื่อ `avatars`) และตั้งค่านโยบายสิทธิ์ (RLS Rules) ให้เป็น Public หรืออนุญาตสิทธิ์การอัปโหลดให้พร้อมใช้งาน:

```
import { supabase } from '../lib/supabase';
import * as FileSystem from 'expo-file-system';
import { decode } from 'base64-js';

const uploadToSupabaseCloud = async (localFileUri: string): Promise<string
| null> => {
  try {
```

```
  // 1. อ่านข้อมูลไฟล์ภาพจากเครื่องปลายทางออกมาเป็น Base64 String
    const base64Data = await FileSystem.readAsStringAsync(localFileUri, {
      encoding: FileSystem.EncodingType.Base64,
    });

  // 2. แปลงสตริงรหัส Base64 ให้กลายเป็น ArrayBuffer
    const arrayBuffer = decode(base64Data);

  // 3. กำหนดชื่อและพาธปลายทางที่จะบันทึกไว้ในระบบคลาวด์
    const fileName = `profile_${Date.now()}.jpg`;
    const filePath = `user_avatar/${fileName}`;

  // 4. สั่งส่งอัปโหลด (Upload) ข้อมูลขึ้นไปฝากที่คลาวด์บัคเก็ต 'avatars'
    const { data, error } = await supabase.storage
   .from('avatars') // ระบุชื่อ Bucket เป้าหมาย
   .upload(filePath, arrayBuffer, {
        contentType: 'image/jpeg',
        upsert: true // หากเจอพาธเดียวกันให้สั่งเขียนทับทันที
      });

    if (error) {
      throw error;
    }

  // 5. ดึงข้อมูลที่อยู่พาธ URL สาธารณะ (Public URL) สำหรับนำไปใช้เรนเดอร์แสดงผล
    const { data: publicUrlData } = supabase.storage
   .from('avatars')
   .getPublicUrl(filePath);

    console.log('อัปโหลดขึ้นคลาวด์สำเร็จ! ลิงก์รูปภาพสาธารณะคือ:', publicUrlData.publicUrl);
    return publicUrlData.publicUrl; // ลิงก์ตรงที่นำไปส่งต่อให้ API อื่นใช้งานได้ทั่วโลก

  } catch (error: any) {
    console.error('ไม่สามารถส่งข้อมูลขึ้นระบบคลาวด์ได้:', error.message || error);
    return null;
  }
};
```

สรุปเปรียบเทียบระบบจัดเก็บไฟล์ทั้ง 3 รูปแบบ

ตารางที่ 11.1 สรุปเปรียบเทียบระบบจัดเก็บไฟล์ทั้ง 3 รูปแบบ

รูปแบบการจัดเก็บ	พื้นที่จัดเก็บ (Directory/Server)	การแบ่งปัน (Sharing)	ความคงทนถาวร (Persistence)

1. Private Sandbox	โฟลเดอร์เอกสารภายในแอป (documentDirectory)	**เฉพาะแอปตนเองเท่านั้น แอปอื่นไม่มีสิทธิ์มาแตะต้อง**	**ถาวรในเครื่อง จนกว่าแอปพลิเคชันจะถูกถอนออก**
2. Shared Storage	แกลเลอรีสื่อหลักของโทรศัพท์ (expo-media-library)	**แบ่งปันสาธารณะในเครื่อง แอปอื่นเปิดแกลเลอรีก็สแกนค้นเจอ**	**ถาวรตามสิทธิ์ผู้ใช้ดำรงอยู่ตลอดไป จนกว่าจะกดยืนยันลบในคลังภาพ**
3. Cloud Storage	บนเซิร์ฟเวอร์ฝากไฟล์คลาวด์ (Supabase Storage)	**เข้าถึงข้ามอุปกรณ์ทั่วโลก ผ่านการกระจายตัวของ URL เครือข่าย**	**ถาวรตามระบบเซิร์ฟเวอร์ ลบออกเมื่อมีคำสั่ง API Trigger ลบทิ้งเท่านั้น**

สรุปท้ายบทเรียน

กลไกการจัดการสิทธิ์กล้องถ่ายรูปและคลังภาพ: การเข้าถึงกล้องและภาพถ่ายในเครื่องผู้ใช้จำเป็นต้องทำตามมาตรการความปลอดภัยและได้รับอนุญาตจากระบบปฏิบัติการ โดยเราสามารถตรวจสอบและขอสิทธิ์ใช้งานเลนส์กล้องแยกด้วย useCameraPermissions Hook ของ expo-camera และขอสิทธิ์อ่านคลังรูปภาพแกลเลอรีด้วยฟังก์ชันขออนุญาตของ ImagePicker (Media Library Permissions) ก่อนที่จะเรียกเข้าใช้งาน

การใช้งาน CameraView เพื่อพรีวิวภาพสด: คอมโพเนนต์ CameraView เข้ามาทำหน้าที่เปิดช่องรับสัญญาณภาพและแสดงพรีวิวภาพเคลื่อนไหวสดจากกล้องบนหน้าจอมือถือใน Expo SDK 54 โดยเราสามารถส่งผ่านพร็อพส์ facing บ่งชี้เลนส์หน้าหรือหลัง และใช้ autofocus ในการสั่งจัดระเบียบความคมชัดอัตโนมัติ ซึ่งต้องเตรียมการดีไซน์ปุ่มสัมผัสในการเปิดปิดมุมกล้องให้ใช้งานได้อย่างสะดวกสบาย

การลั่นชัตเตอร์ถ่ายภาพนิ่ง: การถ่ายภาพนิ่งแบบ Snapshot จากตัวรับพรีวิวสดจะกระทำได้โดยการประกาศผูกตัวแปรอ้างอิง Ref เข้าหาคอมโพเนนต์ CameraView และเรียกฟังก์ชัน Asynchronous ยอดนิยมอย่าง takePictureAsync ซึ่งจะบันทึกรูปภาพนิ่งชั่วคราวลงในหน่วยความจำแคชของอุปกรณ์ และส่งคืนค่าพารามิเตอร์ของรูปภาพและที่อยู่ไฟล์ (URI) กลับมาประมวลผลต่อ

การเลือกรูปภาพและสื่อผ่าน expo-image-picker: เพื่ออำนวยความสะดวกให้ผู้ใช้งานสามารถดึงข้อมูลรูปภาพหรือสื่อที่มีการเก็บบันทึกไว้เดิมในเครื่องมาประยุกต์ใช้งาน (เช่น การเลือกภาพอวาตาร์) เราจะประยุกต์ใช้คำสั่ง launchImageLibraryAsync ของ expo-image-picker ในการเปิดหน้าต่างแกลเลอรีเนทีฟของโทรศัพท์เพื่อให้ผู้ใช้จัดแจงครอปภาพและส่งค่า URI รูปที่ถูกเลือกกลับมา

ระบบจัดการไฟล์แบบที่ 1 (Private Sandbox Storage): ไฟล์ที่ได้จากการถ่ายภาพหรือการดึงภาพจากแกลเลอรีจะได้รับการสร้างเป็นที่อยู่พาธชั่วคราวอยู่ใน cacheDirectory เสมอ ซึ่งเสี่ยงต่อการโดนระบบปฏิบัติการล้างข้อมูลทิ้งเมื่อพื้นที่เต็ม ผู้พัฒนาจึงต้องใช้ expo-file-system ในการคัดลอกไฟล์ย้ายเข้าไปจัดเก็บถาวรใน Sandbox ส่วนตัวของแอป หรือเรียกว่า documentDirectory ซึ่งมีความปลอดภัยและระบบจะไม่เข้ามากวนข้อมูล

ระบบจัดการไฟล์แบบที่ 2 (Shared Media Storage): กรณีที่รูปถ่ายจากแอปพลิเคชันของเราต้องการส่งต่อไปใช้งานร่วมกับแกลเลอรีหลักของโทรศัพท์ เพื่อเปิดโอกาสให้แอปภายนอก เช่น LINE หรือ Facebook สแกนดึงรูปนี้ไปใช้ได้ต่อ เราจะเปลี่ยนจากการเก็บข้อมูลในแซนด์บ็อกซ์ส่วนตัวมาส่งออกข้อมูลผ่านคำสั่ง createAssetAsync ของ expo-media-library เพื่อไปจดทะเบียนลงทะเบียนไฟล์ในคลังภาพสาธารณะของระบบ

ระบบจัดการไฟล์แบบที่ 3 (Cloud Storage Network): เพื่อให้รูปภาพถ่ายสามารถเผยแพร่ข้ามอุปกรณ์และส่งต่อขึ้นฝั่งเว็บไซต์ได้ ข้อมูลรูปภาพจำเป็นต้องอัปโหลดขึ้นฝั่งคลาวด์ เช่น Supabase Storage Buckets โดย React Native จะต้องใช้ expo-file-system ในการอ่านข้อมูลจากพิกัดไฟล์ออกมาเป็นสตริงรหัส Base64 จากนั้นใช้ไลบรารี base64-js ถอดรหัสเป็น ArrayBuffer และส่งยิงอัปโหลดขึ้นเซิร์ฟเวอร์เพื่อให้ได้ลิงก์ตรง (Public URL) สาธารณะ

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้และความเข้าใจเกี่ยวกับแนวคิดระบบความปลอดภัยของภาพถ่ายเนทีฟ, พฤติกรรมการทำงานของคอมโพเนนต์ `CameraView` และ `expo-image-picker` ใน Expo SDK 54, ตลอดจนกลไกสถาปัตยกรรมการจัดเก็บรูปภาพ 3 รูปแบบหลัก ได้แก่ ระบบจัดเก็บไฟล์เฉพาะแอป (Sandbox), การส่งออกไฟล์สาธารณะข้ามแอป (Shared Gallery) และระบบจัดเก็บไฟล์ข้ามเครือข่ายบนคลาวด์ (Supabase Storage)

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดอธิบายพฤติกรรมการเข้าถึงและขออนุมัติใช้งานสื่อรูปภาพแบบเนทีฟได้ถูกต้องที่สุด?

ก. เมื่อแอปพลิเคชันได้รับสิทธิ์เปิดใช้กล้องถ่ายรูปแล้ว จะสามารถดึงประวัติรูปภาพเก่าในคลังอัลบั้มมาประมวลผลได้ทันทีโดยไม่ต้องร้องขอสิทธิ์แกลเลอรี

ข. เนื่องจากเกี่ยวข้องกับสิทธิ์ความเป็นส่วนตัวต่างประเภทกัน ระบบจึงบังคับแยกขอสิทธิ์ระหว่างกล้องถ่ายรูป (Camera Permission) และคลังรูปภาพในอุปกรณ์ (Media Library / Photos Permission) แยกจากกันเป็นเอกเทศ

ค. สิทธิ์ในการเข้าถึงคลังสื่อภาพนิ่งจะถูกสั่งอนุมัติให้อัตโนมัติในฝั่งเซิร์ฟเวอร์โดยที่ผู้ใช้งานมือถือไม่มีสิทธิ์ปฏิเสธ

ง. สิทธิ์การใช้กล้องของระบบเนทีฟทำงานโดยไม่ต้องอาศัยการกำหนดค่าใด ๆ ในไฟล์ `app.json`

ข้อ 2: คอมโพเนนต์ใดที่ได้มาตรฐานของ Expo SDK 54 ที่แนะนำให้นักพัฒนาใช้สร้างหน้าต่างเลนส์พรีวิวภาพสด (Camera Live Preview) จากเลนส์มือถือ?

ก. `<Camera>`

ข. `<CameraPreview>`

ค. `<CameraView>`

ง. `<NativeLens>`

ข้อ 3: การตั้งค่าสลับการใช้หน้าเลนส์พรีวิวสดของกล้องถ่ายรูปสลับไปมาระหว่าง "กล้องหน้า" และ "กล้องหลัง" บนคอมโพเนนต์กล้องรุ่นใหม่ ต้องควบคุมค่าที่พร็อพส์ (Prop) และส่งค่าใด?

ก. พร็อพส์ `type` รับค่า `front` หรือ `back`

ข. พร็อพส์ `facing` รับค่า `front` หรือ `back`

ค. พร็อพส์ `lens` รับค่า `selfie` หรือ `rear`

ง. พร็อพส์ `facing` รับค่า `user` หรือ `environment`

ข้อ 4: พฤติกรรมการกักเก็บรูปภาพเริ่มต้นของผลลัพธ์ที่ได้จากการยิงคำสั่งถ่ายภาพ `takePictureAsync()` หรือเลือกภาพจาก `launchImageLibraryAsync()` ข้อใดกล่าวได้ถูกต้อง?

ก. ข้อมูลไบนารีของภาพถ่ายทั้งหมดจะถูกเขียนพ่วงบันทึกลงฐานข้อมูลหลักของเครื่องโดยทันที

ข. ผลลัพธ์จะถูกจัดเก็บเป็นที่อยู่พาธภาพถ่ายชั่วคราวชี้ไปที่โฟลเดอร์แคช (`cacheDirectory`) ซึ่งระบบปฏิบัติการ (OS) มีสิทธิ์สุ่มล้างข้อมูลทิ้งได้ตลอดเวลาเมื่อดิสก์เครื่องเต็ม

ค. ตัวไฟล์ภาพจะถูกกักเก็บไว้อยู่ในสารระบบ `documentDirectory` แบบคงทนถาวรทันทีหลังเสร็จสิ้นกระบวนการ

ง. ไฟล์ภาพจะถูกส่งอัปโหลดผ่านระบบเครือข่ายอินเทอร์เน็ตขึ้นคลาวด์โดยทันทีโดยไม่ต้องรอเขียนโปรแกรมเชื่อมต่อ

ข้อ 5: แฟ้มไดเรกทอรีระบบใดภายใน Sandbox ของแอปพลิเคชันที่มีคุณสมบัติเก็บรักษาทรัพยากรได้อย่างถาวรปลอดภัย ไม่มีวันโดนระบบปฏิบัติการลบทิ้ง ยกเว้นผู้ใช้จะสั่งถอนการติดตั้งแอป?

ก. `FileSystem.cacheDirectory`

ข. `FileSystem.temporaryDirectory`

ค. `FileSystem.documentDirectory`

ง. `FileSystem.sharedDirectory`

ข้อ 6: หากต้องการพัฒนาแอปพลิเคชันถ่ายรูปแล้วสั่งบันทึกส่งออกไฟล์ภาพไปเก็บไว้ในอัลบั้มหลักของมือถือ เพื่อให้แอปพลิเคชันอื่นนอกตัวแอป (เช่น Facebook, IG) สามารถดึงรูปนี้ไปใช้ได้ด้วย จะต้องบูรณาการผ่านคำสั่งของไลบรารีใด?

ก. `expo-file-system`

ข. `expo-media-library`

ค. `expo-image-picker`

ง. `expo-sharing`

ข้อ 7: คำสั่งใดของไลบรารี `expo-media-library` ที่ใช้ระบุตัวเลือกสั่งบันทึกไฟล์ภาพเนทีฟตรงเข้าสู่แกลเลอรีแชร์ข้ามแอปพลิเคชัน?

ก. `MediaLibrary.saveToLibraryAsync()` หรือ `MediaLibrary.createAssetAsync()`

ข. `MediaLibrary.uploadImageAsync()`

ค. `MediaLibrary.copyToSharedSpace()`

ง. `MediaLibrary.exportPhoto()`

ข้อ 8: การอัปโหลดไฟล์รูปภาพจากฝั่ง React Native ขึ้นสู่ระบบฝากไฟล์ Supabase Storage Buckets วิธีการส่งชุดข้อมูลวิธีใดที่ทำงานได้เสถียรที่สุดเพื่อหลีกเลี่ยงข้อจำกัดการอ่านไฟล์แบบ Blob เนทีฟ?

ก. การดึงที่อยู่พาธ URI ไปยิงคำสั่งอัปโหลดผ่าน REST API ตรง ๆ โดยไม่ผ่าน SDK

ข. การอ่านไฟล์ข้อมูลออกมาเป็นรหัสสตริง Base64 แล้วทำการแปลงถอดรหัสออกมาเป็นแบบ `ArrayBuffer` ไบนารี

ค. การดึงพิกเซลสีทั้งหมดมาเรนเดอร์ในคอมโพเนนต์ Canvas แล้วเก็บรหัส JSON

ง. การอัปโหลดเฉพาะชื่อไฟล์ขึ้นคลาวด์ แล้วให้ฝั่งคลาวด์ดาวน์โหลดไฟล์จากมือถือกลับไปเอง

ข้อ 9: ในกระบวนการอ่านไฟล์และแปลงรหัส Base64 จากพื้นที่จัดเก็บเครื่องฝั่ง Client ของโทรศัพท์มือถือเป็น Asynchronous จะต้องสั่งการเรียกใช้โมดูลใดเป็นแกนหลัก?

ก. `FileSystem.readAsStringAsync()` ร่วมกับออปชัน `{ encoding: FileSystem.EncodingType.Base64 }`

ข. `ImagePicker.readAsBase64Async()`

ค. `AsyncStorage.getItem()` ร่วมกับคีย์ไฟล์

ง. `MediaLibrary.readBytesAsync()`

ข้อ 10: หลังจากที่ทำการส่งอัปโหลดไฟล์เข้าไปใน Supabase Storage Bucket เป็นที่เรียบร้อย หากต้องการลิงก์ตรงภายนอก (Direct URL) สำหรับนำมาเรนเดอร์แสดงผลในแอปทันที จะต้องเรียกใช้เมธอดสืบค้นใดของ Supabase Client?

ก. `supabase.storage.from('bucket').getPublicUrl(filePath)`

ข. `supabase.storage.from('bucket').download(filePath)`

ค. `supabase.storage.from('bucket').select('url')`

ง. `supabase.storage.from('bucket').retrieveLink(filePath)`

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: การอนุญาตสิทธิ์กล้อง ไม่ได้ครอบคลุมการสืบค้นภาพเก่าในคลังอัลบั้ม ผู้ใช้ต้องเลือกยินยอมสิทธิ์แยกส่วนกัน

• ข. ถูก: ระบบความปลอดภัยของ iOS/Android แยกสิทธิ์การใช้เลนส์สัมผัสกล้อง (Camera) และการอ่านคลังไฟล์ส่วนตัวในอัลบั้มเครื่อง (Photos / Media Library) ออกจากกันอย่างเด็ดขาดเพื่อความเป็นส่วนตัวสูงสุดของผู้ใช้

• ค. และ ง. ผิด: ระบบสิทธิ์ถูกควบคุมโดย OS ฝั่งมือถือ ไม่ใช่ฝั่งเซิร์ฟเวอร์ และจำเป็นต้องมีการลงทะเบียนไว้ในไฟล์ `app.json` เสมอ

เฉลย ข้อ 2: ตอบ ค.

• เหตุผลประกอบ:

• **ก. ข. และ ง. ผิด:** ไม่มีส่วนคอมโพเนนต์ชื่อเหล่านี้จัดเตรียมไว้ใน API ของ Expo SDK 54

• **ค. ถูก:** Expo ได้ยุบและปรับปรุงคอมโพเนนต์การพรีวิวเลนส์กล้องตัวเก่า (`Camera`) และเปิดตัวคอมโพเนนต์ `CameraView` ขึ้นมาขับเคลื่อนแทนในแพ็คเกจ `expo-camera`

เฉลย ข้อ 3: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** พร็อพส์ `type` ถูกเปลี่ยนชื่อออกไปแล้วใน API รุ่นใหม่

• **ข. ถูก:** พร็อพส์ที่ระบุหน้าเลนส์คือ `facing` โดยยอมรับค่าสตริงสองชนิดคือ `'front'` (เซลฟี่) หรือ `'back'` (กล้องหลัง) เท่านั้น

• **ค. และ ง. ผิด:** ค่าโหมดหรือเลนส์เหล่านี้ไม่เข้าเกณฑ์พารามิเตอร์ของ Expo Camera API

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** จะไม่มีการเขียนหรือสร้างข้อมูลลงในฐานข้อมูลหลักให้อัตโนมัติ

• **ข. ถูก:** ทั้งภาพที่ถ่ายสดหรือดึงจาก Picker จะได้รับการสร้างไว้ในห้องแคชชั่วคราว (`cacheDirectory`) ซึ่งมีความเสี่ยงสูงที่จะโดนกวาดล้างลบไฟล์ทิ้งโดยระบบ OS เมื่อดิสก์มีความจุน้อยลงหรือแอปพลิเคชันปิดการทำงาน

• **ค. และ ง. ผิด:** ไม่ได้เข้าสารระบบ Document อัตโนมัติ และไม่สามารถวิ่งส่งขึ้นระบบคลาวด์ได้เองโดยไม่มีอินเทอร์เฟซคำสั่งอัปโหลด

เฉลย ข้อ 5: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. และ ข. ผิด:** โฟลเดอร์ห้อง Cache หรือ Temporary มีสิทธิ์โดนลบขยะได้ตลอดเวลาโดย OS

• **ค. ถูก:** แฟ้มเอกสาร `FileSystem.documentDirectory` คือพื้นที่กักเก็บไฟล์ส่วนตัวแบบปลอดภัยภายใน Sandbox ของตัวแอปพลิเคชัน ซึ่งปลอดภัยจากการลบกวาดล้างของ OS เสมอ

• **ง. ผิด:** ไม่มีไดเรกทอรีชื่อนี้

เฉลย ข้อ 6: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `expo-file-system` ใช้จัดเก็บไฟล์ภายในแซนด์บ็อกซ์ส่วนตัวของตัวแอปเองเป็นหลัก

• **ข. ถูก:** `expo-media-library` คือเครื่องมือและไลบรารีที่เปิดทางให้แอปพลิเคชันเข้าถึง จัดการ และเซฟสื่อบันทึกลงสู่โฟลเดอร์ภาพสาธารณะ (Shared Storage) เพื่อแชร์แชร์ข้ามแอปพลิเคชันเครื่องได้สำเร็จ

• **ค. และ ง. ผิด:** `expo-image-picker` ใช้เปิดเลือกรูป ไม่ใช่การเซฟบันทึกคลังภาพส่วน `expo-sharing` ใช้ยิงหน้าต่างระบบแชร์ไฟล์ระบบ (Share Sheet) ไม่ได้จัดเก็บถาวรในแกลเลอรี

เฉลย ข้อ 7: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** ฟังก์ชันที่รับหน้าที่จัดสร้าง asset รูปภาพและนำเข้าคลังรูปสาธารณะคือ `MediaLibrary.createAssetAsync()` หรือสั่งบันทึกไฟล์ภาพเนทีฟตรงด้วย `MediaLibrary.saveToLibraryAsync()`

• **ข. ค. และ ง. ผิด:** ชื่อเมธอดไม่มีระบุไว้ในสารระบบของคลังคำสั่ง `expo-media-library`

เฉลย ข้อ 8: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** การทำ Blob ใน React Native เจอปัญหาความเข้ากันได้ และการส่งพิกเซลสีหรือส่งเพียงชื่อไฟล์ไม่สามารถย้ายไฟล์จริงขึ้นคลาวด์ได้

• **ข. ถูก:** การสกัดดึงข้อมูลออกมาเป็นสตริงความปลอดภัยรหัส Base64 ด้วย `expo-file-system` แล้วใช้ฟังก์ชันถอดค่าออกมาเป็นไบนารีแบบ `ArrayBuffer` ถือเป็นรูปแบบมาตรฐานที่เสถียรที่สุดในการยิงรีเควสผ่าน `@supabase/supabase-js`

เฉลย ข้อ 9: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** โมดูล Asynchronous ของระบบไฟล์ของ Expo คือ `FileSystem.readAsStringAsync` ซึ่งเราต้องกำหนดคีย์ออปชันรหัสผ่านเป็น `{ encoding: FileSystem.EncodingType.Base64 }`

• **ข. ค. และ ง. ผิด:** คำสั่งอื่นไม่มีอยู่จริง หรือใช้เก็บบันทึกความสัมพันธ์ตัวเลขธรรมดาอย่าง AsyncStorage ไม่สามารถแปลงไฟล์ไบนารีได้

เฉลย ข้อ 10: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** เมธอดสืบค้นลิงก์ตรงคือ .**getPublicUrl(filePath)** ซึ่งจะส่งอ็อบเจกต์ที่มีแอตทริบิวต์ระบุพาธ .**data.publicUrl** กลับออกมาให้ใช้ส่งเข้า Image Component เพื่อเรนเดอร์ภาพจากอินเทอร์เน็ตได้ทันที

• **ข. ผิด:** .**download()** คุมการรับข้อมูลไบนารีดิบลงหน่วยความจำ ไม่ใช่ขอลิงก์ URL

• **ค. และ ง. ผิด:** โครงสร้างคำสั่งเหล่านี้ระบุพารามิเตอร์และ API ชื่อไม่ถูกต้อง

ปฏิบัติการท้ายบทเรียน: การพัฒนาแอปเลือกรูปโปรไฟล์อวาตาร์บันทึก 3 ระบบ

ปฏิบัติการนี้จะฝึกทักษะระดับสูงในการทำโมดูลอวาตาร์ โดยเชื่อมประสานกล้องพรีวิวสดของระบบเนทีฟและการกักเก็บไฟล์ 3 ระบบ ได้แก่ **Private Sandbox** (ในดิสก์จำกัดเฉพาะแอป), **Shared Media Library** (ในดิสก์สาธารณะแชร์ข้ามแอป) และ **Cloud Network Storage** (อัปโหลดเซิร์ฟเวอร์ Supabase Storage)

1. วัตถุประสงค์การเรียนรู้

1. สามารถเขียนขอยื่นสิทธิ์ความปลอดภัยเข้าใช้กล้องถ่ายรูป อัลบั้มส่วนตัว และการเขียนสื่อลงแกลเลอรีสาธารณะได้สำเร็จ

2. สามารถควบคุมกล้องเซลฟี่สดผ่าน **CameraView** และสลับกล้องหน้าหลังรวมถึงสั่งชัตเตอร์ Asynchronous

3. สามารถเรียกใช้ **expo-file-system** ย้ายไฟล์ภาพจาก Cache Folder ไปฝากถาวรใน **documentDirectory** ภายใน Sandbox

4. สามารถประยุกต์ใช้งาน **expo-media-library** ส่งออกไฟล์ภาพไปเก็บบันทึกในคลังภาพสาธารณะของระบบเครื่องเพื่อแชร์ข้ามแอปพลิเคชัน

5. สามารถเขียนตรรกะการแปลงสตริง Base64 และ ArrayBuffer เพื่อยิงภาพอัปโหลดตรงขึ้นระบบฝากไฟล์ Supabase Storage Bucket และดึง URL ลิงก์ตรงมาจัดแสดงผล

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องจัดสร้างโปรเจกต์ React Native Expo โดยมีโครงสร้างดังนี้:

```
โครงสร้างไฟล์ปฏิบัติการ:
├── .env                                  => บรรจุคีย์โครงการ
EXPO_PUBLIC_SUPABASE_URL และ EXPO_PUBLIC_SUPABASE_ANON_KEY
├── app.json                              => ลงทะเบียนคำอนุญาตสิทธิ์ใช้งานกล้อง อัลบั้มภาพ และ
คลังภาพส่วนกลาง
├── lib/
│   └── supabase.ts                       => สร้างอินสแตนซ์ Supabase Client
```

```
└── app/
    └── index.tsx                    => หน้าจอจัดแสดงโปรไฟล์ แผงกล้อง Modal และฟังก์ชันการเขียนไฟล์ 3 ระบบ
```

การตั้งค่าตารางจัดเก็บไฟล์ฝากคลาวด์บน Supabase (Supabase Storage Setup):

1. ให้ผู้เรียนไปที่คอนโซลของ Supabase เข้าหัวข้อ **Storage**

2. กดสร้าง Bucket ใหม่ขึ้นมาชื่อว่า `**avatars**`

3. ปรับเปลี่ยนนโยบายความปลอดภัย (RLS Policies) ของบัคเก็ต **avatars** นี้โดยกดเพิ่มกฎใหม่ (New Policy) -> เลือกเทมเพลต **"Enable read access to everyone"** (เพื่อให้บุคคลภายนอกสามารถดึงภาพจาก URL สาธารณะได้) และกฎ **"Enable insert/upload access for authenticated and anonymous users"** (สำหรับการเปิดให้อัปโหลดไฟล์ส่งขึ้นไป)

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งคลังไลบรารีทั้งหมด

เปิดใช้งานเทอร์มินัลในโฟลเดอร์โปรเจกต์ของคุณแล้วสั่งรันบรรทัดต่อไปนี้:

```
npx expo install expo-camera expo-image-picker expo-file-system expo-media-library @react-native-async-storage/async-storage base64-js
```

ขั้นตอนที่ 2: ตั้งค่าระบบขอสิทธิ์ในไฟล์ app.json

เปิดไฟล์ **app.json** แล้วระบุ Plugins สิทธิ์เข้าใช้งานของอุปกรณ์ให้เรียบร้อย:

```
{
 "expo": {
  "name": "MultiAvatarApp",
  "slug": "multi-avatar-app",
  "version": "1.0.0",
  "plugins": [
   [
    "expo-camera",
        {
     "cameraPermission": "แอปต้องการขอสิทธิ์เข้าใช้งานกล้องเพื่อใช้ในการถ่ายภาพโปรไฟล์"
      }
   ],
   [
    "expo-image-picker",
        {
     "photosPermission": "แอปต้องการขอสิทธิ์เข้าถึงคลังสื่อภาพเพื่อดึงภาพมาอัปเกรดโปรไฟล์"
        }
```

```
    ],
    [
     "expo-media-library",
          {
      "photosPermission": "แอปต้องการขอสิทธิ์เขียนบันทึกไฟล์ภาพถ่ายของแอปเพื่อนำไปบันทึกลงคลังแกลเลอรีเครื่อง"
          }
    ]
   ]
  }
}
```

ขั้นตอนที่ 3: จัดการระบุคีย์ลับเซิร์ฟเวอร์ใน `lib/supabase.ts`

สร้างไฟล์ .env ไว้ที่โฟลเดอร์ชั้นนอกสุดของโปรเจกต์เพื่อนำส่งคีย์ลับ:

```
EXPO_PUBLIC_SUPABASE_URL=https://your-project-id.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=your-anon-public-key-here
```

และสร้างไฟล์เชื่อมโยง `lib/supabase.ts`:

```
import 'react-native-url-polyfill/auto';
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL || '';
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY || '';

export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถนำโค้ดสำหรับควบคุม จัดการรูปโปรไฟล์และประมวลผลเซฟและอัปโหลดส่งไฟล์ 3 ระบบนี้ บันทึกลงในไฟล์ `app/index.tsx` เพื่อรันระบบทดสอบระดับเนทีฟ:

```
import React, { useState, useEffect, useRef } from 'react';
import {
  StyleSheet,
  Text,
  View,
  Image,
  TouchableOpacity,
  Modal,
  ActivityIndicator,
  Alert,
  SafeAreaView,
  ScrollView
} from 'react-native';
```

```
import { CameraView, CameraType, useCameraPermissions } from 'expo-
camera';
import * as ImagePicker from 'expo-image-picker';
import * as FileSystem from 'expo-file-system';
import * as MediaLibrary from 'expo-media-library';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { decode } from 'base64-js';
import { supabase } from '../lib/supabase';

const LOCAL_STORAGE_KEY = '@local_avatar_path';
const CLOUD_STORAGE_KEY = '@cloud_avatar_url';

export default function App() {
  const [tempImageUri, setTempImageUri] = useState<string | null>(null);

 // สถานะพาธและผลลัพธ์จัดเก็บรูปภาพในระบบต่าง ๆ
  const [sandboxPath, setSandboxPath] = useState<string | null>(null);
  const [sharedGalleryStatus, setSharedGalleryStatus] = useState<string |
null>(null);
 const [cloudUrl, setCloudUrl] = useState<string | null>(null);

  const [isCameraVisible, setIsCameraVisible] = useState(false);
  const [facing, setFacing] = useState<CameraType>('front');
  const [isProcessing, setIsProcessing] = useState(false);

  const [cameraPermission, requestCameraPermission] =
useCameraPermissions();
  const [mediaLibraryPermission, requestMediaLibraryPermission] =
MediaLibrary.usePermissions();
  const cameraRef = useRef<CameraView | null>(null);

 // 1. โหลดข้อมูลที่เคยเก็บจำไว้ตอนเปิดเข้าแอป
  useEffect(() => {
    loadSavedSettings();
  }, []);

  const loadSavedSettings = async () => {
    try {
      const savedLocal = await AsyncStorage.getItem(LOCAL_STORAGE_KEY);
      const savedCloud = await AsyncStorage.getItem(CLOUD_STORAGE_KEY);

      if (savedLocal) {
        const info = await FileSystem.getInfoAsync(savedLocal);
        if (info.exists) {
          setSandboxPath(savedLocal);
        }
      }
      if (savedCloud) {
        setCloudUrl(savedCloud);
   }
    } catch (e) {
      console.error(e);
```

```
    }
  };

  // 2. ดึงภาพถ่ายจากอัลบั้มเครื่อง
  const handlePickImage = async () => {
    const { status } = await
ImagePicker.requestMediaLibraryPermissionsAsync();
    if (status !== 'granted') {
      Alert.alert('คำเตือน', 'กรุณาเปิดสิทธิ์เข้าใช้คลังภาพในการตั้งค่าเครื่อง');
      return;
    }

    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ImagePicker.MediaTypeOptions.Images,
      allowsEditing: true,
      aspect: [1, 1],
      quality: 0.8
    });

    if (!result.canceled && result.assets && result.assets.length > 0) {
      setTempImageUri(result.assets[0].uri);
    // เคลียร์ผลลัพธ์การเซฟรอบเก่า
      setSandboxPath(null);
      setSharedGalleryStatus(null);
    setCloudUrl(null);
    }
  };

  // 3. เริ่มเปิดใช้งานเลนส์กล้องพรีวิวสด
  const handleStartCamera = async () => {
    if (!cameraPermission || !cameraPermission.granted) {
      const response = await requestCameraPermission();
      if (!response.granted) {
        Alert.alert('สิทธิ์ถูกปฏิเสธ', 'กรุณายอมรับสิทธิ์ใช้งานเลนส์ถ่ายรูป');
        return;
      }
    }
    setIsCameraVisible(true);
  };

  const handleCapturePhoto = async () => {
    if (cameraRef.current) {
      try {
        const photo = await cameraRef.current.takePictureAsync({
          quality: 0.8,
          skipProcessing: false
        });
        if (photo && photo.uri) {
          setTempImageUri(photo.uri);
          setIsCameraVisible(false);
```

```
            setSandboxPath(null);
            setSharedGalleryStatus(null);
            setCloudUrl(null);
          }
        } catch (e) {
          Alert.alert('ผิดพลาด', 'ไม่สามารถบันทึกรูปภาพจากตัวเลนส์ได้');
        }
      }
    };

  // 4. บูรณาการระบบบันทึกรูปโปรไฟล์ 3 ระดับพร้อมกัน (Multi-Destination Process)
    const handleSaveAndSyncAll = async () => {
      if (!tempImageUri) {
        Alert.alert('แจ้งเตือน', 'กรุณาถ่ายรูปหรือเลือกภาพจากแกลเลอรีก่อนกดบันทึก');
        return;
      }

      try {
        setIsProcessing(true);

        // --- ระบบที่ 1: บันทึกเฉพาะตัวใน Private Sandbox (Document Directory) ---
        const filename = `avatar_${Date.now()}.jpg`;
        const destinationSandboxUri =
`${FileSystem.documentDirectory}${filename}`;
        await FileSystem.copyAsync({
          from: tempImageUri,
          to: destinationSandboxUri
        });
        setSandboxPath(destinationSandboxUri);
        await AsyncStorage.setItem(LOCAL_STORAGE_KEY,
destinationSandboxUri);

        // --- ระบบที่ 2: บันทึกลง Shared Media Gallery (แชร์ข้ามแอปในเครื่อง) ---
        let sharedAssetId = 'ปฏิเสธสิทธิ์เข้าใช้คลังสื่อ';
        if (!mediaLibraryPermission || !mediaLibraryPermission.granted) {
          const mediaPerm = await requestMediaLibraryPermission();
          if (mediaPerm.granted) {
            const asset = await MediaLibrary.createAssetAsync(tempImageUri);
            sharedAssetId = `สำเร็จ (Asset ID: ${asset.id})`;
          }
        } else {
          const asset = await MediaLibrary.createAssetAsync(tempImageUri);
          sharedAssetId = `สำเร็จ (Asset ID: ${asset.id})`;
        }
        setSharedGalleryStatus(sharedAssetId);

        // --- ระบบที่ 3: อัปโหลดขึ้น Cloud Storage ของ Supabase (ซิงก์เน็ตเวิร์ก) ---
        // อ่านข้อมูลไฟล์เป็น Base64
        const base64Data = await FileSystem.readAsStringAsync(tempImageUri, {
      encoding: FileSystem.EncodingType.Base64
```

```
      });
    // แปลง Base64 เป็น ArrayBuffer
      const arrayBuffer = decode(base64Data);

      const cloudFilePath = `profile_pics/${filename}`;
      const { data, error } = await supabase.storage
    .from('avatars')
    .upload(cloudFilePath, arrayBuffer, {
          contentType: 'image/jpeg',
          upsert: true
        });

      if (error) {
        throw error;
      }

    // ดึงลิงก์รูปภาพสาธารณะจาก Supabase
      const { data: publicUrlData } = supabase.storage
    .from('avatars')
    .getPublicUrl(cloudFilePath);

      const publicLink = publicUrlData.publicUrl;
      setCloudUrl(publicLink);
      await AsyncStorage.setItem(CLOUD_STORAGE_KEY, publicLink);

      Alert.alert('บันทึก 3 ระบบเสร็จสิ้น!', 'ภาพถูกจัดเก็บลงดิสก์ส่วนตัว แกลเลอรีเครื่อง และอัปโหลดขึ้นคลาวด์เรียบร้อย');
    } catch (err: any) {
      Alert.alert('เกิดข้อผิดพลาดในการรันระบบบันทึก', err.message || err);
      console.error(err);
    } finally {
      setIsProcessing(false);
    }
  };

// 5. ล้างประวัติลบรูปภาพทั้งหมดออก
  const handleClearEverything = async () => {
    Alert.alert(
      'ยืนยันการเคลียร์ข้อมูล',
      'ล้างประวัติที่อยู่และลบไฟล์ภาพในแซนด์บ็อกซ์ดิสก์ทั้งหมดหรือไม่?',
    [
        { text: 'ยกเลิก', style: 'cancel' },
        {
          text: 'ล้างข้อมูล',
          style: 'destructive',
          onPress: async () => {
            try {
      // ลบรูปภาพใน Sandbox เครื่อง
```

```
              if (sandboxPath) {
            await FileSystem.deleteAsync(sandboxPath, { idempotent: true });
              }
              await AsyncStorage.removeItem(LOCAL_STORAGE_KEY);
              await AsyncStorage.removeItem(CLOUD_STORAGE_KEY);

              setTempImageUri(null);
             setSandboxPath(null);
              setSharedGalleryStatus(null);
              setCloudUrl(null);
              Alert.alert('ล้างค่าเรียบร้อยแล้ว');
            } catch (e) {
              console.error(e);
            }
          }
        }
    ]
  );
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>

        {/* หัวเรื่องแอป */}
        <View style={styles.header}>
          <Text style={styles.headerTitle}>📸 Profile Avatar Pro</Text>
          <Text style={styles.headerSubtitle}>ทบทวนการจัดการไฟล์ภาพถ่ายเนทีฟและคลาวด์ 3
ระบบ</Text>
        </View>

        {/* แผงเปรียบเทียบรูปภาพพรีวิว */}
        <View style={styles.avatarCard}>
          <Text style={styles.cardLabel}>อวาตาร์โปรไฟล์ปัจจุบัน</Text>
          <View style={styles.avatarWrapper}>
            {/* ดึงภาพแสดงตามลำดับความคงทน: 1. ลิงก์ตรงคลาวด์ 2. Sandbox 3. ภาพชั่วคราวดึงสด 4. ภาพ
ว่างเปล่า */}
            {cloudUrl ? (
              <Image source={{ uri: cloudUrl }} style={styles.avatarImage}
/>
      ) : sandboxPath ? (
              <Image source={{ uri: sandboxPath }}
style={styles.avatarImage} />
      ) : tempImageUri ? (
              <Image source={{ uri: tempImageUri }}
style={styles.avatarImage} />
      ) : (
              <View style={styles.avatarPlaceholder}>
                <Text style={styles.placeholderText}>No Pic</Text>
              </View>
```

```
      )}
          </View>
          <Text style={styles.imageSourceIndicator}>
            {cloudUrl ? '☁️ แสดงจาก Cloud Storage' : sandboxPath ? '💾 แสดงจาก Local Sandbox' : tempImageUri ? '⏳ ภาพรอการบันทึก' : '⚙️ พร้อมรับข้อมูลภาพ'}
          </Text>
        </View>

        {/* แผงคำสั่งทริกเกอร์เลือกภาพ */}
        <View style={styles.buttonGroupCard}>
          <Text style={styles.cardHeaderTitle}>1. นำเข้าสื่อถ่ายภาพโปรไฟล์</Text>
          <View style={styles.rowButtons}>
            <TouchableOpacity style={styles.primaryButton} onPress={handleStartCamera}>
              <Text style={styles.buttonText}>📷 เปิดกล้องถ่ายภาพ</Text>
            </TouchableOpacity>
            <TouchableOpacity style={[styles.primaryButton, styles.darkBlueButton]} onPress={handlePickImage}>
              <Text style={styles.buttonText}>🖼️ ดึงจากอัลบั้มเครื่อง</Text>
            </TouchableOpacity>
          </View>
        </View>

        {/* แผงประมวลผลบันทึก 3 รูปแบบ */}
        <View style={styles.actionCard}>
          <Text style={styles.cardHeaderTitle}>2. ประมวลผลและทดสอบจัดเก็บไฟล์ 3 ทิศทาง</Text>

          {isProcessing ? (
            <View style={styles.spinnerBlock}>
              <ActivityIndicator size="small" color="#38bdf8" />
              <Text style={styles.processingText}>กำลังย้าย Sandbox / คัดลงอัลบั้ม / อัปโหลดส่งคลาวด์...</Text>
            </View>
      ) : (
            <TouchableOpacity
              style={[styles.actionTriggerButton, !tempImageUri && styles.disabledButton]}
              onPress={handleSaveAndSyncAll}
            disabled={!tempImageUri}
            >
              <Text style={styles.actionTriggerText">💾 กดรันบันทึก 3 ระบบพร้อมกัน</Text>
            </TouchableOpacity>
      )}

          {/* รายงานสถานะ Log การบันทึก */}
          <View style={styles.logBox}>
```

```
            <Text style={styles.logHeader}>📌 บันทึกผลสำเร็จ (Storage
Logs):</Text>
            <View style={styles.logLine}>
              <Text style={styles.logTitle}>1. Local Sandbox
(Private):</Text>
              <Text style={styles.logValue} numberOfLines={2}>
                {sandboxPath ? sandboxPath : 'ยังไม่ได้จัดเก็บลงดิสก์เอกสารแอป'}
              </Text>
            </View>
            <View style={styles.logLine}>
              <Text style={styles.logTitle}>2. Shared Library
(Public):</Text>
              <Text style={styles.logValue}>
                {sharedGalleryStatus ? sharedGalleryStatus : 'ยังไม่ได้ส่งเขียนแกล
เลอรีกล้องหลัก'}
              </Text>
            </View>
            <View style={styles.logLine}>
              <Text style={styles.logTitle}>3. Cloud Storage
(Supabase):</Text>
              <Text style={styles.logValueCloud} numberOfLines={2}>
                {cloudUrl ? cloudUrl : 'ยังไม่ได้ส่งอัปโหลดขึ้นเครือข่าย'}
              </Text>
            </View>
          </View>

          {/* ปุ่มลบประวัติ */}
          {(sandboxPath || cloudUrl || tempImageUri) && (
            <TouchableOpacity style={styles.clearButton}
onPress={handleClearEverything}>
              <Text style={styles.clearButtonText}>🗑️ ล้างภาพและไฟล์ Sandbox
ทั้งหมด</Text>
            </TouchableOpacity>
      )}
        </View>

      </ScrollView>

      {/* โมดูลเต็มจอสำหรับเปิดกล้องถ่ายภาพ */}
      <Modal visible={isCameraVisible} animationType="slide"
transparent={false}>
        <View style={styles.cameraContainer}>
          <CameraView
            ref={cameraRef}
            style={styles.cameraViewport}
            facing={facing}
          >
            {/* ส่วนควบคุมกล้องด้านบน */}
            <View style={styles.cameraHeader}>
```

```
              <TouchableOpacity
                style={styles.cameraCircleBtn}
                onPress={() => setIsCameraVisible(false)}
              >
                <Text style={styles.cameraCloseIcon}>✕</Text>
              </TouchableOpacity>
              <TouchableOpacity
                style={styles.cameraCircleBtn}
                onPress={() => setFacing(curr => curr === 'back' ? 'front'
: 'back')}
              >
                <Text style={styles.cameraCloseIcon}>🔄</Text>
            </TouchableOpacity>
              </View>

              {/* ส่วนล่างชัตเตอร์ด้านล่าง */}
              <View style={styles.cameraFooter}>
                <TouchableOpacity
                  style={styles.shutterRing}
                  onPress={handleCapturePhoto}
                >
                  <View style={styles.shutterDot} />
                </TouchableOpacity>
                <Text style={styles.cameraTip}>เลนส์พรีวิวสด: {facing === 'front' ?
'กล้องหน้า' : 'กล้องหลัง'}</Text>
              </View>
            </CameraView>
          </View>
        </Modal>
      </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  scrollContainer: {
    padding: 20,
  },
  header: {
    alignItems: 'center',
    marginVertical: 10,
  },
  headerTitle: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  headerSubtitle: {
    fontSize: 12,
```

```
    color: '#64748b',
    textAlign: 'center',
    marginTop: 6,
    lineHeight: 18,
  },
  avatarCard: {
   backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 20,
    alignItems: 'center',
    borderWidth: 1,
    borderColor: '#334155',
    marginVertical: 15,
  },
  cardLabel: {
    color: '#94a3b8',
    fontSize: 13,
    fontWeight: '600',
    marginBottom: 12,
  },
  avatarWrapper: {
    width: 150,
    height: 150,
    borderRadius: 75,
    borderWidth: 4,
    borderColor: '#38bdf8',
    justifyContent: 'center',
    alignItems: 'center',
    overflow: 'hidden',
    backgroundColor: '#0f172a',
  },
  avatarImage: {
    width: '100%',
    height: '100%',
    resizeMode: 'cover',
  },
  avatarPlaceholder: {
    width: '100%',
    height: '100%',
    justifyContent: 'center',
    alignItems: 'center',
  },
  placeholderText: {
    color: '#475569',
    fontWeight: 'bold',
    fontSize: 16,
  },
```

```
  imageSourceIndicator: {
    color: '#10b981',
    fontSize: 12,
    fontWeight: '600',
    marginTop: 12,
  },
  buttonGroupCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 16,
   borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 15,
  },
  cardHeaderTitle: {
    color: '#ffffff',
    fontSize: 14,
    fontWeight: 'bold',
    marginBottom: 12,
  },
  rowButtons: {
    flexDirection: 'row',
    justifyContent: 'space-between',
  },
  primaryButton: {
    flex: 0.48,
    backgroundColor: '#0284c7',
    borderRadius: 10,
    height: 44,
    justifyContent: 'center',
    alignItems: 'center',
  },
  darkBlueButton: {
    backgroundColor: '#334155',
  },
  buttonText: {
   color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 13,
  },
  actionCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 16,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 20,
  },
```

```
  actionTriggerButton: {
    backgroundColor: '#10b981',
    borderRadius: 10,
    height: 48,
    justifyContent: 'center',
    alignItems: 'center',
  },
  disabledButton: {
    backgroundColor: '#475569',
    opacity: 0.5,
  },
  actionTriggerText: {
    color: '#ffffff',
  fontWeight: 'bold',
    fontSize: 14,
  },
  spinnerBlock: {
    alignItems: 'center',
    marginVertical: 10,
  },
  processingText: {
    color: '#94a3b8',
    fontSize: 11,
    marginTop: 6,
  },
  logBox: {
    backgroundColor: '#0f172a',
    borderRadius: 10,
    padding: 12,
    borderWidth: 1,
    borderColor: '#334155',
    marginTop: 15,
  },
  logHeader: {
    color: '#f8fafc',
    fontSize: 12,
    fontWeight: 'bold',
    marginBottom: 8,
  },
  logLine: {
    marginBottom: 10,
  },
  logTitle: {
    color: '#38bdf8',
    fontSize: 11,
    fontWeight: '600',
  },
  logValue: {
```

```
    color: '#e2e8f0',
    fontSize: 10,
    fontFamily: 'monospace',
    marginTop: 2,
  },
  logValueCloud: {
    color: '#a7f3d0',
    fontSize: 10,
    fontFamily: 'monospace',
    marginTop: 2,
  },
  clearButton: {
    borderWidth: 1,
    borderColor: '#ef4444',
    borderRadius: 10,
    height: 40,
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 15,
  },
  clearButtonText: {
    color: '#ef4444',
    fontSize: 12,
    fontWeight: 'bold',
  },
// ส่วนประกอบกล้อง Modal
  cameraContainer: {
    flex: 1,
    backgroundColor: '#000000',
  },
  cameraViewport: {
    flex: 1,
    justifyContent: 'space-between',
  },
  cameraHeader: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    paddingTop: 50,
    paddingHorizontal: 20,
  },
  cameraCircleBtn: {
    width: 42,
    height: 42,
    borderRadius: 21,
    backgroundColor: 'rgba(0,0,0,0.5)',
    justifyContent: 'center',
    alignItems: 'center',
  },
```

```
  cameraCloseIcon: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
  cameraFooter: {
    alignItems: 'center',
    paddingBottom: 40,
    backgroundColor: 'rgba(0,0,0,0.3)',
    paddingTop: 15,
  },
  shutterRing: {
    width: 74,
    height: 74,
    borderRadius: 37,
    borderWidth: 4,
    borderColor: '#ffffff',
    justifyContent: 'center',
    alignItems: 'center',
    marginBottom: 12,
  },
  shutterDot: {
    width: 58,
    height: 58,
    borderRadius: 29,
    backgroundColor: '#ffffff',
  },
  cameraTip: {
    color: '#cbd5e1',
    fontSize: 11,
    fontWeight: '500',
  },
});
```

เอกสารอ้างอิง

Expo. (2024). *Camera*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/camera/

Expo. (2024). *FileSystem*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/filesystem/

Expo. (2024). *ImagePicker*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/imagepicker/

แผนบริหารการสอนประจำบทที่ 12

หัวข้อเนื้อหาประจำบท

12.1 แนวคิดระบบยืนยันตัวตนและการจัดการสิทธิ์ (Authentication vs Authorization)

12.2 สถาปัตยกรรมระบบของ Supabase Auth (GoTrue Service)

12.3 การสร้างฟังก์ชันสมัครสมาชิกและเข้าสู่ระบบ (Sign Up & Sign In)

12.4 การจัดการเซสชันผู้ใช้และตัวดักจับสถานะ (Session State & onAuthStateChange)

12.5 การรักษาความปลอดภัยระดับอุปกรณ์ด้วย expo-secure-store

12.6 [เนื้อหาพิเศษตอนท้าย (Bonus Section)]: การรีแฟกเตอร์ไปใช้ useContext และ AuthContext

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถอธิบายแนวคิดเรื่อง JWT (JSON Web Tokens) และสถาปัตยกรรมยืนยันตัวตนของ Supabase Auth ได้ถูกต้อง
2. ผู้เรียนสามารถเขียนคำสั่งและสร้างแบบฟอร์มรับสมัครสมาชิก (signUp) และลงชื่อเข้าใช้ (signInWithPassword) ได้สำเร็จ
3. ผู้เรียนสามารถดึงค่าเซสชันและใช้งาน onAuthStateChange ร่วมกับ useState เพื่อทำระบบสลับการแสดงผลหน้าจอแบบ Conditional Rendering ได้สำเร็จ
4. ผู้เรียนสามารถเขียนโปรแกรมนำ expo-secure-store ไปใช้จัดเก็บเซสชันล็อกอินของ Supabase Client ในแบบเนทีฟได้อย่างปลอดภัย
5. ผู้เรียนสามารถระบุและรับมือข้อผิดพลาดทั่วไปจาก Supabase API เช่น อีเมลซ้ำ หรือรหัสผ่านไม่ถูกต้อง และแสดงผลให้ผู้ใช้ทราบอย่างปลอดภัย
6. ผู้เรียนสามารถอธิบายหลักการและแนวทางของการประยุกต์ใช้ useContext ในการขยายสเกลระบบยืนยันตัวตนของแอปพลิเคชันได้ถูกต้อง

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

45. **การบรรยายเรื่อง JWT & Direct Login Flow:** บรรยายหลักการจัดเก็บสถานะผู้ใช้ด้วยโทเค็นความปลอดภัย, การทำ Conditional Rendering สลับหน้า Login/Dashboard บนหน้าจอเดียว และความสำคัญของการเลือกใช้ Secure Store บนโมบายแอปพลิเคชัน
46. **การทำ Live Coding Demonstration หัวข้อ Single-Page Supabase Login & Secure Storage:** ผู้สอนสาธิตการสร้างแอปที่มีกลไกรันตรวจสอบสถานะเซสชันดั้งเดิม, แสดงผลฟอร์มล็อกอินเมื่อไม่มีผู้ใช้งาน และโชว์หน้าตารางแดชบอร์ดเมื่อล็อกอินเข้าสู่ระบบได้สำเร็จ โดยเชื่อมต่อกับ expo-secure-store
47. **กิจกรรมตรวจสอบความหนาแน่นรหัสผ่าน (Auth Validation Challenge):** ให้ผู้เรียนฝึกฝนเขียนรหัสกรองอีเมลและรหัสผ่านก่อนยิง API ของ Supabase เพื่อประหยัดการยิงรีเควสแบบไม่มีคุณภาพ
48. **กิจกรรมสัมมนารีแฟกเตอร์สู่ระบบสากล (Context Refactoring Demonstration):** ผู้สอนสาธิตหัวข้อของแถมพิเศษด้วยการนำโค้ดในหน้าจอหลักมารีแฟกเตอร์ขยายระบบ โดยแยกเป็น AuthContext.Provider เพื่ออธิบายว่าหากผู้เรียนต้องการทำแอปที่มีหน้าจอหลากหลายของ Expo Router จะสามารถนำ Context ไปดักปิดป้อนสิทธิ์การเปิดอ่านไฟล์และหน้าจอได้อย่างไร

สื่อการเรียนการสอน

44. เอกสารสไลด์เรื่อง "Supabase Authentication, Secure Store and Refactoring to useContext"
45. บัญชีผู้ใช้งานระบบคลาวด์ Supabase (สำหรับเปิดดูตาราง Authentication -> Users)
46. เครื่องมือสมาร์ทโฟนหรืออุปกรณ์จำลองสำหรับการทดสอบปิดแอปพลิเคชันแล้วเปิดใหม่เพื่อเช็คการคงอยู่ของล็อกอิน
47. คู่มืออ้างอิง API ของ Supabase Auth, expo-secure-store และ React Context API

การวัดผลและการประเมินผล

1. การวัดผล

1.1 ประเมินผลคะแนนการทำแบบทดสอบท้ายบทเรียน 10 ข้อ

1.2 ตรวจสอบความถูกต้องของการนำ expo-secure-store มาควบคุมเซสชันของ Supabase Client ในโค้ดใบงานปฏิบัติการ

1.3 สังเกตประสิทธิภาพความลื่นไหลการเข้าสู่ระบบและจดจำเซสชันเมื่อผู้ใช้เปิดแอปขึ้นมาใหม่

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถสร้างระบบล็อกอิน สมัครสมาชิก และจดจำสเตตัสผู้ใช้งานในหน้าจอหลักผ่าน expo-secure-store ได้สำเร็จผ่านเกณฑ์ (ผ่าน/ไม่ผ่าน)

2.3 สังเกตพฤติกรรมการจัดการข้อผิดพลาดระบบหลังบ้าน และความเข้าใจเบื้องต้นในการรีแฟกเตอร์สู่โครงสร้าง Context

บทที่ 12

ระบบยืนยันตัวตนและการเข้าถึงข้อมูลด้วย Supabase Authentication

การสร้างแอปพลิเคชันที่มีการระบุตัวตน เช่น การรักษาความปลอดภัยสมุดจดบันทึก หรือการจำกัดสิทธิ์ให้เฉพาะเจ้าของบัญชีเข้าถึงข้อมูล จำเป็นต้องใช้ระบบยืนยันตัวตน (Authentication) ในบทเรียนนี้เราจะศึกษาการเชื่อมโยงระบบหลังบ้านของ **Supabase Auth** เข้ากับหน้าจออินเทอร์เฟซในแอปพลิเคชัน React Native Expo SDK 54 ตั้งแต่การจัดการเซสชันผู้ใช้แบบตรงไปตรงมาในจุดเดียว ไปจนถึงการเก็บรักษาความปลอดภัยของโทเค็นด้วยฮาร์ดแวร์เก็บข้อมูลลับ (`expo-secure-store`) และการต่อยอดไปสู่ระบบแชร์สเตตัสระดับสากลด้วย `useContext`

12.1 แนวคิดระบบยืนยันตัวตนและการจัดการสิทธิ์ (Authentication vs Authorization)

ในการเขียนโปรแกรมระบบความปลอดภัย มีสองแนวคิดที่เราต้องทำความเข้าใจแยกจากกัน:

1. **การยืนยันตัวตน (Authentication - AuthN):** คือกระบวนการพิสูจน์ทราบว่า *"ผู้ใช้งานคือใครจริงหรือไม่"* เช่น การกรอกอีเมลและรหัสผ่าน จากนั้นหลังบ้านจะส่งโทเค็นระบุตัวตนยืนยันกลับมา (เช่น JWT - JSON Web Tokens)
2. **การอนุญาตสิทธิ์ (Authorization - AuthZ):** คือกระบวนการดักกรองว่า *"ผู้ใช้งานที่ยืนยันตนเข้ามาแล้วนั้น มีสิทธิ์ทำอะไรได้บ้าง"* เช่น ผู้ใช้งานทั่วไปเข้าถึงได้เฉพาะตารางของตัวเอง ส่วนผู้ใช้ที่เป็นแอดมินสามารถเข้าดูและลบแถวข้อมูลของทุกคนได้ (ทำงานร่วมกับระบบ RLS ของ PostgreSQL ใน Supabase)

ในการทำระบบล็อกอินบนโทรศัพท์มือถือ อุปกรณ์ไคลเอนต์จะได้รับ Access Token และ Refresh Token จากระบบคลาวด์หลังบ้านมาเก็บรักษาไว้ และต้องส่งแนบโทเค็นนี้ไปกับ API Request ทุกครั้งเพื่อยืนยันตัวตน

12.2 สถาปัตยกรรมระบบของ Supabase Auth (GoTrue Service)

Supabase มีระบบที่พัฒนาขึ้นจากบริการหลังบ้านแบบ Open Source ชื่อ **GoTrue API** (Supabase, 2024) ซึ่งคอยจัดสรรและประมวลผลข้อมูลผู้ใช้อยู่เบื้องหลัง โดยมีฟังก์ชันการยืนยันตัวตนที่หลากหลาย เช่น Email/Password, Magic Link, และ OAuth (เช่น Sign in with Google หรือ Apple)

ในบทเรียนนี้เราจะมุ่งเน้นไปที่การระบุตัวตนที่คลาสสิกและสำคัญที่สุดคือ **Email and Password Authentication** ซึ่งแอปจะคุยตรงกับเซิร์ฟเวอร์ GoTrue ผ่าน Supabase Client SDK

12.3 การสร้างฟังก์ชันสมัครสมาชิกและเข้าสู่ระบบ (Sign Up & Sign In)

ฟังก์ชันการทำงานพื้นฐานประกอบด้วยการลงทะเบียนบัญชีใหม่ และการล็อกอินเข้าใช้งานผ่านชุดคำสั่ง Asynchronous ต่อไปนี้:

1. การสมัครสมาชิกใหม่ (Sign Up)

เมธอด `signUp` จะรับค่าเป็นออบเจกต์ที่มีคีย์ `email` และ `password` เพื่อนำไปจัดสร้างแถวบัญชีใหม่บน Supabase:

```
const handleSignUp = async () => {
  const { data, error } = await supabase.auth.signUp({
    email: 'user@example.com',
    password: 'securepassword123',
  });

  if (error) {
    console.error('สมัครสมาชิกขัดข้อง:', error.message);
  } else {
    console.log('สร้างบัญชีสำเร็จแล้ว:', data.user);
  }
};
```

2. การลงชื่อเข้าใช้งาน (Sign In)

เมธอด `signInWithPassword` จะระบุค่าอินพุตเพื่อกู้คืนสเตตัสการเชื่อมต่อและดึงสิทธิ์โทเค็นกลับมา:

```
const handleSignIn = async () => {
  const { data, error } = await supabase.auth.signInWithPassword({
    email: 'user@example.com',
    password: 'securepassword123',
  });

  if (error) {
    console.error('ล็อกอินไม่ผ่าน:', error.message);
  } else {
    console.log('ยินดีต้อนรับ เข้าสู่ระบบสำเร็จ:', data.session);
  }
};
```

12.4 การจัดการเซสชันผู้ใช้และตัวดักจับสถานะ (Session State & onAuthStateChange)

เพื่อให้ตัวแอปพลิเคชันทราบความเคลื่อนไหวตลอดเวลาว่าปัจจุบันผู้ใช้กำลังเชื่อมต่ออยู่หรือล็อกเอาต์หลุดระบบไปแล้ว เราจะใช้เครื่องมือดักฟังความเคลื่อนไหวเซสชันของ Supabase ที่ชื่อว่า **`onAuthStateChange()`**

คำสั่งนี้เสมือนเป็นสตรีมข้อมูล (Subscription Stream) ที่จะคอยส่งสัญญาณทริกเกอร์เรียกฟังก์ชัน Callback ทำงานทุกครั้งเมื่อสเตตมีการขยับเปลี่ยน (เช่น `INITIAL_SESSION`, `SIGNED_IN`, `SIGNED_OUT`, `TOKEN_REFRESHED`)

ตัวอย่างการคุมสเตตแบบง่ายในระดับ Component (Direct State Monitoring):

เราสามารถใช้ `useState` บันทึกค่าเซสชันไว้ในหน้าจอหลัก และใช้ `useEffect` ดักฟังเหตุการณ์เพื่อสลับ UI ในหน้าเดียวได้อย่างง่ายดาย:

```
import React, { useState, useEffect } from 'react';
import { View, Text, ActivityIndicator } from 'react-native';
import { supabase } from '../lib/supabase';

export default function RootScreen() {
  const [session, setSession] = useState<any>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
  // 1. ดึงเซสชันเริ่มต้นที่อาจค้างอยู่ออกมาดูก่อน
    supabase.auth.getSession().then(({ data: { session } }) => {
      setSession(session);
      setIsLoading(false);
    });

  // 2. สมัครรับสัญญาณคอยฟังเมื่อสเตตล็อกอินขยับเปลี่ยน
    const { data: { subscription } } =
supabase.auth.onAuthStateChange((_event, session) => {
      setSession(session);
    });

  // 3. คืนค่าแรมยกเลิกสมัครสตรีมเมื่อคอมโพเนนต์ถูกถอดถอน
    return () => {
      subscription.unsubscribe();
    };
  }, []);

  if (isLoading) {
    return <ActivityIndicator size="large" />;
  }
```

```
  // ทำ Conditional Rendering สลับหน้าแบบตรงไปตรงมา
    return session ? <Dashboard session={session} /> : <LoginForm />;
}
```

12.5 การรักษาความปลอดภัยระดับอุปกรณ์ด้วย expo-secure-store

ตามค่าเริ่มต้นของ Supabase JS Client ในโปรเจกต์ทั่วไปมักจะเก็บโทเค็นของระบบลงใน **AsyncStorage** ของเครื่องมือถือ แต่มีข้อเสียร้ายแรงด้านความปลอดภัยคือ **AsyncStorage เก็บข้อมูลไว้เป็นเพียงไฟล์ข้อความ Plaintext JSON ธรรมดาบนเครื่อง** ซึ่งไม่มีการเข้ารหัสข้อมูล (Encryption) ทำให้โทรศัพท์ที่ผ่านการ Root หรือ Jailbreak สามารถแอบแกะโทเค็นไปสวมสิทธิ์แทนเจ้าของได้

สำหรับโมบายแอปพลิเคชันยุคใหม่ จึงบังคับให้นำโทเค็นข้อมูลลับเหล่านี้ไปเก็บไว้ในช่องเก็บข้อมูลลับที่มีระบบเข้ารหัสความปลอดภัยระดับฮาร์ดแวร์ (Keychain สำหรับ iOS และ Keystore สำหรับ Android) โดยเราจะใช้ไลบรารี **`expo-secure-store`** (Expo, 2024)

1. การติดตั้ง Secure Store:

```
npx expo install expo-secure-store
```

2. การสร้างคลาสเก็บข้อมูลความปลอดภัยเชื่อมต่อ Supabase Client:

เราสามารถดีไซน์อ็อบเจกต์ทำหน้าที่เป็น Storage Adapter เพื่อบังคับให้โมดูล Auth ของ Supabase เก็บโทเค็นลงใน Secure Store แทน AsyncStorage:

```
import 'react-native-url-polyfill/auto';
import { createClient } from '@supabase/supabase-js';
import * as SecureStore from 'expo-secure-store';

// 1. สร้างคลาสจัดการ Secure Store Adapter
const ExpoSecureStoreAdapter = {
  getItem: (key: string) => {
   return SecureStore.getItemAsync(key);
  },
  setItem: (key: string, value: string) => {
    return SecureStore.setItemAsync(key, value);
  },
  removeItem: (key: string) => {
    return SecureStore.deleteItemAsync(key);
  },
};

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL || '';
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY || '';
```

```
// 2. ประกาศเชื่อมโดยส่ง adapter เข้าไปใน config
export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  auth: {
    storage: ExpoSecureStoreAdapter,
    autoRefreshToken: true,    // เปิดระบายโทเค็นใหม่ให้อัตโนมัติเบื้องหลัง
    persistSession: true,        // บังคับจำค่าเซสชันข้ามการเปิดปิดแอป
    detectSessionInUrl: false,
  },
});
```

12.6 การรีแฟกเตอร์ไปใช้ useContext และ AuthContext

แม้ว่าการจัดเก็บสเตตัสล็อกอินใน Component เดียวจะเข้าใจง่าย แต่หากเราขยายแอปพลิเคชันจนมีหลายหน้าจอ (เช่น หน้าโปรไฟล์, หน้าตั้งค่า, หน้าแก้ไขข้อมูล) ทุกหน้าจอจำเป็นต้องทราบค่า Session ล็อกอินปัจจุบัน ซึ่งการส่ง Prop ต่อกันไปเป็นทอด ๆ (Prop Drilling) จะสร้างความยุ่งยากอย่างมหาศาล

แนวทางปฏิบัติที่ดีที่สุดคือการยกระดับตรรกะมาเก็บไว้ใน `AuthContext` เพื่อแชร์ข้อมูล Session ให้กับทุกหน้าจอทั่วแอปพลิเคชัน

ตัวอย่างการสร้าง context/AuthContext.tsx

```
import React, { createContext, useState, useEffect, useContext } from
'react';
import { supabase } from '../lib/supabase';
import { Session, User } from '@supabase/supabase-js';

interface AuthContextType {
  session: Session | null;
  user: User | null;
  isLoading: boolean;
  signOut: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType>({
  session: null,
  user: null,
  isLoading: true,
  signOut: async () => {}
});

export const AuthProvider = ({ children }: { children: React.ReactNode })
=> {
  const [session, setSession] = useState<Session | null>(null);
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
```

```
  // เช็คเซสชันเริ่มต้น
    supabase.auth.getSession().then(({ data: { session } }) => {
      setSession(session);
      setUser(session ? session.user : null);
      setIsLoading(false);
    });

  // ดักฟังการอัปเดตสเตตัสเซสชัน
    const { data: { subscription } } =
supabase.auth.onAuthStateChange((_event, session) => {
      setSession(session);
      setUser(session ? session.user : null);
      setIsLoading(false);
    });

    return () => subscription.unsubscribe();
  }, []);

  const signOut = async () => {
    await supabase.auth.signOut();
  };

  return (
    <AuthContext.Provider value={{ session, user, isLoading, signOut }}>
      {children}
    </AuthContext.Provider>
 );
};

// สร้าง Custom Hook เพื่อความสะดวกในการดึงข้อมูล
export const useAuth = () => useContext(AuthContext);
```

เมื่อใช้แนวทางนี้ ในไฟล์หน้าจอ Layout หลัก (app/_layout.tsx) หรือคอมโพเนนต์อื่น ๆ เราจะเรียกใช้งานเพื่ออ่านและทริกเกอร์ล้างสเตตัสเพียงบรรทัดเดียวได้ทันที:

```
const { user, signOut } = useAuth();
```

สรุปท้ายบทเรียน

แนวคิดระบบยืนยันตัวตนและการจัดการสิทธิ์: ระบบความปลอดภัยของแอปพลิเคชันแบ่งเป็น 2 ส่วนหลักคือ การยืนยันตัวตน (Authentication) เพื่อระบุตัวตนและตรวจสอบว่าผู้ใช้คือใครจริงหรือไม่ และการอนุญาตสิทธิ์ (Authorization) เพื่อตรวจสอบสิทธิ์การเข้าถึงข้อมูลตามขอบเขตที่รับรองไว้ ซึ่งในระบบสมาร์ทโฟนไคลเอนต์จะได้รับ Access Token และ Refresh Token จากระบบคลาวด์เพื่อใช้แนบไปกับ API Request ในการยืนยันตัวตนเสมอ

สถาปัตยกรรมระบบของ Supabase Auth: Supabase Auth ทำงานโดยยึดรากฐานหลักผ่านระบบหลังบ้าน GoTrue API ซึ่งเป็นซอฟต์แวร์บริการที่เปิดโอกาสให้ผู้พัฒนาจัดการข้อมูลสมาชิกและสิทธิ์การเข้าถึงได้อย่างเป็นระบบ โดยรองรับทั้งรูปแบบ Magic Link, การล็อกอินผ่านโซเชียลมีเดียชื่อดัง (OAuth) หรือระบบดั้งเดิมอย่างการล็อกอินแบบใช้อีเมลและรหัสผ่าน (Email/Password Auth) ที่เป็นมาตรฐานสากล

การสร้างฟังก์ชันสมัครสมาชิกและเข้าสู่ระบบ: การเริ่มเปิดระบบบัญชีสมาชิกใช้เมธอด supabase.auth.signUp() โดยการส่งตัวแปรข้อมูลของอีเมลและรหัสผ่านเพื่อบันทึกข้อมูลสมาชิกใหม่ขึ้นตารางเก็บข้อมูลหลังบ้าน และใช้เมธอด supabase.auth.signInWithPassword() เพื่อสแกนตรวจสอบความถูกต้องและกู้คืนสิทธิ์เซสชันของข้อมูลสมาชิกนั้นกลับมาแสดงผลที่หน้าจอแอปพลิเคชัน

การจัดการเซสชันผู้ใช้และตัวดักจับสถานะ: เพื่อให้ตัวแอปพลิเคชันตื่นตัวและเปลี่ยนหน้าต่างอินเทอร์เฟซได้อย่างไหลลื่นตามสถานะผู้ใช้ปัจจุบัน เราจะใช้ตัวดักฟังความเคลื่อนไหวเซสชันของ Supabase ที่ชื่อว่า onAuthStateChange() ในการคอยฟลักซ์สัญญาณส่งข้อมูลกลับมา (Callback) เมื่อตรวจพบการสลับหน้า ข้ามเซสชัน หรือล็อกเอาต์ระบบ เพื่อควบคุมสถานะตัวแปรและวาดหน้าจอใหม่ได้อย่างถูกต้อง

การรักษาความปลอดภัยระดับอุปกรณ์ด้วย expo-secure-store: การเก็บบันทึกข้อมูลโทเค็นความปลอดภัยลงใน AsyncStorage มีความเสี่ยงเนื่องจากไม่มีการเข้ารหัสข้อมูล (Plaintext) ผู้พัฒนาโปรเจกต์เชิงพาณิชย์จึงบังคับใช้ expo-secure-store ในการเข้ารหัสข้อมูลระดับลึกผ่านระบบฮาร์ดแวร์ Key Store ของระบบปฏิบัติการ โดยเราสามารถจัดทำ ExpoSecureStoreAdapter มาต่อเชื่อมกับตัวแอป Supabase client ได้เพื่อความปลอดภัยขั้นสูงสุด

การรีแฟกเตอร์ไปใช้ useContext และ AuthContext: เมื่อโครงสร้างแอปพลิเคชันใหญ่ขึ้น การส่งผ่านค่าเซสชันข้อมูลลงไปทีละหน้าจอ (Prop Drilling) จะยุ่งยากและไร้ประสิทธิภาพ การรีแฟกเตอร์โครงสร้างสเตตล็อกอินหลักด้วยเครื่องมือ React Context API อย่างเช่น useContext และ AuthProvider จะช่วยแก้ปัญหาโดยทำหน้าที่กระจายสัญญาณส่งมอบข้อมูลสิทธิ์ไปหาคอมโพเนนต์ย่อยทั่วทั้งแอปพลิเคชันได้โดยตรง

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้และความเข้าใจเกี่ยวกับหลักสถาปัตยกรรมความปลอดภัยของแอปพลิเคชันมือถือ, การใช้คำสั่ง API ของ Supabase Auth สำหรับการสมัครสมาชิกและเข้าสู่ระบบ, การควบคุมสถานะการใช้งาน (Session Lifecycle), การ

รักษาความปลอดภัยระดับอุปกรณ์ด้วย `expo-secure-store` และแนวคิดของการจัดสร้างโครงสร้างตัวเก็บสเตตัสส่วนกลางด้วย React Context API

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดนิยามความแตกต่างระหว่างแนวคิด "การยืนยันตัวตน (Authentication)" และ "การอนุญาตสิทธิ์ (Authorization)" ได้อย่างถูกต้องที่สุด?

ก. Authentication คือการตั้งข้อจำกัดขนาดกว้างยาวของหน้าจอแอป ส่วน Authorization คือการควบคุมความสว่างของหน้าเลนส์กล้อง

ข. Authentication คือการตรวจสอบพิสูจน์ตนว่าผู้ใช้คือใครจริงหรือไม่ (เช่น ล็อกอินผ่านอีเมล) ส่วน Authorization คือการดักกรองเงื่อนไขว่าผู้ใช้นั้นมีสิทธิ์เข้าถึงหรือแก้ไขข้อมูลชิ้นใดบ้าง (เช่น กฎ RLS)

ค. Authentication คือการเขียนสไตล์จัดกลุ่มกล่อง Flexbox ส่วน Authorization คือการส่งคีย์ผ่านทางสายเครือข่ายอินเทอร์เน็ต

ง. ทั้งสองแนวคิดคือเรื่องเดียวกันและไม่มีส่วนใดแตกต่างกันในทางสถาปัตยกรรมระบบคอมพิวเตอร์

ข้อ 2: บริการหลังบ้านตัวใดที่รันทำหน้าที่ควบคุมกลไกความปลอดภัยและระบบสมาชิก (Authentication Engine) อยู่เบื้องหลังของ Supabase?

ก. Firebase Auth

ข. PostgreSQL Engine

ค. GoTrue API

ง. Auth0 Core

ข้อ 3: หากต้องการเขียนโค้ดบันทึกสร้างบัญชีผู้ใช้งานคนใหม่ด้วยอีเมลและรหัสผ่านเข้าสู่ดาต้าเบส จะต้องเรียกใช้เมธอด Asynchronous ใดของ Supabase JS Client?

ก. `supabase.auth.createUser()`

ข. `supabase.auth.signUp()`

ค. `supabase.auth.register()`

ง. `supabase.auth.insertUser()`

ข้อ 4: เมธอดล็อกอินใดของ Supabase Auth ที่มีบทบาทใช้ส่งอีเมลและรหัสผ่านเพื่อเรียกคืนข้อมูลสิทธิ์และรับเซสชันโทเค็นปัจจุบัน?

ก. `supabase.auth.signIn()`

ข. `supabase.auth.loginWithPassword()`

ค. `supabase.auth.signInWithPassword()`

ง. `supabase.auth.authenticate()`

ข้อ 5: เมื่อต้องการเฝ้าติดตามรับฟังการเปลี่ยนผ่านระดับความเคลื่อนไหวเซสชัน (เช่น เมื่อล็อกอินสำเร็จ หรือล็อกเอาต์หลุดระบบ) ควรเลือกรับสัญญาณจากคำสั่งใดของ Supabase?

ก. `supabase.auth.getSession()`

ข. `supabase.auth.onAuthStateChange()`

ค. `supabase.auth.watchSession()`

ง. `supabase.auth.getUser()`

ข้อ 6: เพราะเหตุใด AsyncStorage จึงถูกมองว่ามีความปลอดภัยต่ำและไม่ควรนำมาใช้เป็นที่เก็บโทเค็นยืนยันตัวตนหลักในโมบายแอปพลิเคชันระดับโปรดักชัน?

ก. เนื่องจาก AsyncStorage เก็บข้อมูลเป็นไฟล์ข้อความธรรมดา (Plaintext JSON) ที่ไม่มีการเข้ารหัสความปลอดภัยระดับเนทีฟ ทำให้อุปกรณ์ที่ถูก Root/Jailbreak สามารถดึงโทเค็นออกไปสวมสิทธิ์ได้ง่าย

ข. เนื่องจาก AsyncStorage ถูกจำกัดความจุเพียง 10 ตัวอักษรเท่านั้น

ค. เพราะคำสั่ง AsyncStorage จะลบข้อมูลทิ้งโดยอัตโนมัติทันทีที่ปิดหน้าจอแอปพลิเคชัน

ง. เนื่องจาก AsyncStorage รองรับเฉพาะการทำงานในระบบ iOS เท่านั้น

ข้อ 7: ไลบรารีใดของ Expo ที่มีหน้าที่ช่วยเข้ารหัสและจัดเก็บรหัสกุญแจสำคัญลงสู่ Keychain/Keystore ของอุปกรณ์มือถือในระดับฮาร์ดแวร์ได้อย่างเสถียรปลอดภัย?

ก. `expo-secure-storage`

ข. `expo-safe-store`

ค. `expo-secure-store`

ง. `expo-keychain`

ข้อ 8: การต่อเชื่อมให้ Supabase Client บังคับหันมาบันทึกและจัดการโทเค็นล็อกอินผ่าน Secure Store แทน AsyncStorage จะต้องส่งอ็อบเจกต์ Adapter เข้าผ่านพร็อพเพอร์ตี้ใดในตอนประกาศตัวสร้าง `createClient`?

ก. `auth.storage`

ข. `auth.secureStore`

ค. `auth.adapter`

ง. `auth.persistSession`

ข้อ 9: ในกรณีที่เราสร้างแอปพลิเคชันขนาดเล็กแบบ Single-page โดยใช้สเตตล็อกอินในหน้านั้นหน้าเดียว วิธีการสลับหน้าจอระหว่าง "หน้าฟอร์มเข้าสู่ระบบ" และ "หน้าแผงควบคุม Dashboard" ใช้วิธีใดที่ง่ายและกระชับที่สุด?

ก. การเปิดกล่อง Modal ทับไว้ตลอดเวลา

ข. การใช้งาน Conditional Rendering ผ่านการดักตัวแปรเซสชัน เช่น `session ? <Dashboard /> : <LoginForm />`

ค. การบังคับยิงคำสั่งเปลี่ยน Router ดีดข้ามโฟลเดอร์หน้าจอโดยออโต้

ง. การสลับหน้าจอด้วยการเปลี่ยนพิกัดความกว้างในสไตล์ชีท

ข้อ 10: หากต้องการขยายแอปพลิเคชันให้ใหญ่ขึ้น (Multi-page) และประสงค์จะแชร์เซสชันผู้ใช้อวาตาร์รวมถึงสิทธิ์การเข้าถึงให้ทุกหน้าจอรับรู้ค่าตรงกันทันทีโดยไม่ต้องส่งพร็อพส์ต่อกันเป็นทอด ๆ (Prop Drilling) ข้อใดคือสถาปัตยกรรม React ที่เหมาะสมที่สุดที่จะนำมาประยุกต์ใช้เป็นของแถมพิเศษ?

ก. การทำลูปสตรีมดักในปุ่มกดของทุกหน้าจอ

ข. การใช้งาน React Context API ผ่านการทำ `useContext` และ `AuthContext.Provider`

ค. การนำข้อมูลเซสชันไปจัดบันทึกฝากไว้ในตัวแปร Global Windows Node

ง. การอัดบันทึกพิกัดตำแหน่งลงใน API พิกัดตำแหน่ง GPS

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: ข้อจำกัดขนาดกล่องเป็นเรื่องของ CSS/Flexbox และระบบทั้งสองมีบทบาทและการจัดเก็บคีย์ที่แตกต่างกันอย่างสิ้นเชิง

• ข. ถูก: Authentication คือด่านระบุตัวตนบุคคล (เช่น การกรอกฟอร์มเพื่อแสดงตัวว่าเป็นเจ้าของบัญชีจริง) ส่วน Authorization คือการเช็คว่าบัญชีที่ระบุตนเข้ามาแล้วนั้นมีสิทธิ์ระดับไหนในการอ่านเขียนแก้ไขแถวข้อมูล (เช่น กฎ RLS ควบคุมตาราง wishlist หลังบ้าน)

เฉลย ข้อ 2: ตอบ ค.

• เหตุผลประกอบ:

• ก. ข. และ ง. ผิด: Supabase พัฒนาขึ้นมาโดยใช้ PostgreSQL เป็นเบสฐานข้อมูล แต่ตัวที่ทำหน้าที่รันจัดการ Session/Auth โดยตรงคือ GoTrue Service

• ค. ถูก: GoTrue คือระบบ API เบื้องหลังที่ทำหน้าที่ควบคุม จัดสรร และระบายโทเค็นความปลอดภัยของตารางข้อมูลผู้ใช้งานบน Supabase Auth

เฉลย ข้อ 3: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: คำสั่งอื่นไม่มีระบุไว้ในไวยากรณ์การประกาศฟังก์ชันของจาวาสคริปต์ Supabase Client

• ข. ถูก: ไวยากรณ์สมัครสมาชิกใช้งานของ Supabase Auth คือคำสั่ง Asynchronous ชื่อ **`supabase.auth.signUp()`**

เฉลย ข้อ 4: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: ในอดีตเคยมี **`signIn()`** แต่ปัจจุบันใน Supabase JS client v2 ได้ปรับเปลี่ยนมาใช้คำสั่งที่ระบุวิธีการล็อกอินชัดเจนขึ้นเพื่อแยกแยะการใช้เมลกับรหัสผ่าน

• ข. และ ง. ผิด: คำสั่งไม่มีระบุใน API

• ค. ถูก: คำสั่งที่ถูกต้องในการลงชื่อเข้าใช้ด้วยชุดเมลรหัสผ่านคือ **`supabase.auth.signInWithPassword()`**

เฉลย ข้อ 5: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: **`getSession()`** คืนค่าเซสชันของจังหวะนั้น ๆ ครั้งเดียว ไม่ได้รับฟังสัญญาณเรียลไทม์

• ข. ถูก: **`supabase.auth.onAuthStateChange()`** ทำหน้าที่เฝ้าดักฟังและพ่นสัญญาณ Callback ออกมาทุกครั้งที่มีการเปลี่ยนผ่านสเตตล็อกอิน ล็อกเอาต์ หรือการเปลี่ยนเปลี่ยนโทเค็น

• ค. และ ง. ผิด: คำสั่งระบุผิดวัตถุประสงค์ หรือไม่มีในสารระบบ Auth API ของ Supabase

เฉลย ข้อ 6: ตอบ ก.

• เหตุผลประกอบ:

• ก. ถูก: AsyncStorage มีความปลอดภัยต่ำเพราะเก็บข้อมูลเป็นตัวอักษรดิบที่แกะดูได้ง่ายบนอุปกรณ์ที่ผ่านการแฮกหรือเข้าถึง root หากมีข้อมูลสำคัญอย่างโทเค็นล็อกอิน ก็เสี่ยงที่จะถูกขโมยสิทธิ์

• ข. ค. และ ง. ผิด: AsyncStorage เก็บข้อมูลได้ไม่จำกัดสิทธิ์ (ขีดจำกัดปกติคือ 6MB) และข้อมูลไม่ถูกลบอัตโนมัติเมื่อปิดแอป รวมถึงสามารถรันได้ทั้งบน Android และ iOS

เฉลย ข้อ 7: ตอบ ค.

• เหตุผลประกอบ:

• **ก. ข. และ ง. ผิด:** ชื่อสะกดไม่ถูกต้องตามหลักไลบรารีของระบบ Expo

• **ค. ถูก:** ไลบรารีอย่างเป็นทางการในการเข้ารหัสและจัดเก็บคีย์ลับลง Keychain/Keystore บนระดับเนทีฟของ Expo คือ `**expo-secure-store**`

เฉลย ข้อ 8: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** ในการประกาศสร้าง client ตัวเลือกสำหรับจัดส่ง Storage ตัวใหม่คือระบุเข้าที่พร็อพเพอร์ตี้ `**auth.storage**` ภายใต้ออบเจกต์ config ของ **createClient**

• **ข. ค. และ ง. ผิด:** ตัวเลือกชื่ออื่นสะกดผิดไวยากรณ์การรับพารามิเตอร์ของ Supabase

เฉลย ข้อ 9: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** วิธีอื่นระบุคำสั่งควบคุมซับซ้อน หรือส่งผลเสียต่อการรันสเตตของ React

• **ข. ถูก:** การทำ **Conditional Rendering** ด้วยการประเมินค่า **session ? <Dashboard /> : <LoginForm />** เป็นแนวทางที่ดีและสั้นที่สุดในการสลับภาพหน้าจอในหน้าเดียว โดยไม่ต้องรันจัดการเรื่อง Navigation หลายระดับ

เฉลย ข้อ 10: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** วิธีการเหล่านั้นไม่ใช่การแชร์สเตตที่เป็นระเบียบตามมาตรฐานหรือทำให้เกิดผลเสียต่อ Memory Leak

• **ข. ถูก: React Context API** (**useContext** และ **Provider**) คือสถาปัตยกรรมของ React ที่ดีที่สุดในการจัดแชร์สเตตตรงส่วนกลาง เพื่อแชร์ข้อมูลลงสู่หน้าต่างและคอมโพเนนต์ลูกหลานทั้งหมดโดยตรง

ปฏิบัติการท้ายบทเรียน: การสร้างระบบยืนยันตัวตน Email/Password ด้วย Supabase Auth และ Secure Store

ปฏิบัติการนี้จะฝึกทักษะการสร้างระบบสมัครสมาชิก (Sign Up) และระบบเข้าสู่ระบบ (Sign In) ผ่านคลาวด์ **Supabase Auth** บนโทรศัพท์มือถือ โดยผู้เรียนจะได้ออกแบบแอปพลิเคชัน **"สมุดบันทึกรับล็อกสิทธิ์ความปลอดภัยส่วนตัว (Secure Personal Diary)"** ผู้ใช้จะกรอกอีเมลและพาสเวิร์ดที่ผ่านการ Validation ปลายทางเครื่องเพื่อล็อกอินรับโทเค็นความปลอดภัยที่จะถูกเข้ารหัสปกป้องถาวรในฮาร์ดแวร์เก็บข้อมูลลับด้วย **expo-secure-store** และปิดท้ายปฏิบัติการ

ด้วย **แนวเนื้อหาพิเศษ (Bonus Section)** เพื่อทดลองเรียนรู้วิธีการรีแฟกเตอร์แยกส่วนตรรกะไปเป็นระบบจัดเก็บข้อมูลส่วนกลางแบบแชร์ข้ามจอด้วย `useContext`

1. วัตถุประสงค์การเรียนรู้

1. สามารถติดตั้งและตั้งค่าโปรเจกต์ Expo SDK 54 ให้จัดเก็บโทเค็นของ Supabase Auth ลงในระดับฮาร์ดแวร์ด้วย `expo-secure-store` ได้สำเร็จ

2. สามารถเขียนฟอร์มรับอินพุต พร้อมการกรองตรวจสอบอีเมลและรหัสผ่าน (Form Validation) ได้ถูกต้องตามมาตรฐาน

3. สามารถเรียกใช้ API `signUp` และ `signInWithPassword` ในการจัดการสมัครและยืนยันตนผู้ใช้กับระบบคลาวด์ได้สำเร็จ

4. สามารถใช้งาน `onAuthStateChange` ร่วมกับสเตตภายใน Component เพื่อสลับสลับสิทธิ์หน้าจอระหว่าง Login และ Dashboard (Conditional Rendering) ได้สำเร็จ

5. สามารถทำความเข้าใจและอธิบายสถาปัตยกรรมของการยกระดับระบบความปลอดภัยล็อกอินไปใช้โครงสร้างแชร์สเตตกลางแบบ `useContext`

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องจัดวางโครงสร้างระบบไฟล์ดังต่อไปนี้:

```
โครงสร้างไฟล์หลักปฏิบัติการ:
├── .env                                => คีย์ความปลอดภัย EXPO_PUBLIC_SUPABASE_URL และ EXPO_PUBLIC_SUPABASE_ANON_KEY
├── lib/
│   └── supabase.ts                     => กำหนดค่า Supabase Client เชื่อมต่อผ่าน Secure Store Adapter
└── app/
    └── index.tsx                       => หน้าควบคุมฟังก์ชัน สมัคร/ล็อกอิน และ Dashboard จัดการข้อมูลสมุดบันทึกส่วนตัว
```

รายละเอียดเงื่อนไขการทำงาน (Logics):

1. **การยืนยันความถูกต้องฟอร์ม (Form Validation):**

- ฟิลด์ Password ต้องกำหนดเงื่อนไขบังคับป้อนไม่น้อยกว่า 6 ตัวอักษร
- ฟิลด์ Email ต้องกรอกอิงตามรูปแบบ Regular Expression ของอีเมลมาตรฐาน

หากไม่ผ่านเกณฑ์ ให้ป้ายปุ่มบันทึกเป็นสีเทาและแสดงข้อความแจ้งเตือนสีแดงด้านล่าง

2. **การสมัครสมาชิก (Sign Up Process):**

• กดทริกเกอร์สร้างผู้ใช้และแสดงผล Alert ยืนยันความสำเร็จ

3. **การเข้าสู่ระบบและคงอยู่ (Sign In & Session Retention):**

• ทันทีที่ล็อกอินผ่าน ให้นำผู้ใช้สลับหน้าไปที่หน้าจอบันทึกส่วนตัว (Secure Diary Dashboard)

• เมื่อผู้ใช้ทำการปิดแอปพลิเคชันไป แล้วกดรันเปิดแอปกลับมาใหม่ ระบบต้องทำการดึงสถานะโทเค็นความปลอดภัยที่เก็บสะสมไว้ใน `expo-secure-store` และนำทางผู้ใช้งานตรงเข้ามาที่หน้า Dashboard ทันทีโดยไม่ต้องให้พิมพ์ล็อกอินซ้ำซ้อน

4. **ความประณีตในการรายงาน Error:**

• ดักจับ Error จาก Supabase API (เช่น กรอกรหัสผิด หรืออีเมลเคยลงทะเบียนไปแล้ว) และแสดงผลเป็นป้ายกล่องแจ้งเตือนความปลอดภัยสีแดงสดที่เข้าใจง่าย

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งไลบรารีที่จำเป็น

เปิดใช้งานเทอร์มินัลพิมพ์รันการติดตั้งคำสั่งต่อไปนี้:

```
npx expo install expo-secure-store
```

ขั้นตอนที่ 2: ติดตั้งและเชื่อม Supabase Client ผ่าน Secure Store

สร้างหรือปรับปรุงไฟล์ `lib/supabase.ts` เพื่อเปลี่ยนให้ Supabase Client บันทึกค่าเซสชันลงในหน่วยความจำเข้ารหัสระดับอุปกรณ์เนทีฟ:

```
import 'react-native-url-polyfill/auto';
import { createClient } from '@supabase/supabase-js';
import * as SecureStore from 'expo-secure-store';

// คลาสตัวช่วยจัดการ Secure Store
const ExpoSecureStoreAdapter = {
  getItem: (key: string) => {
    return SecureStore.getItemAsync(key);
  },
  setItem: (key: string, value: string) => {
    return SecureStore.setItemAsync(key, value);
  },
  removeItem: (key: string) => {
    return SecureStore.deleteItemAsync(key);
  },
};

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL || '';
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY || '';
```

```
export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  auth: {
    storage: ExpoSecureStoreAdapter,
    autoRefreshToken: true,
    persistSession: true,
    detectSessionInUrl: false,
  },
});
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

รหัสซอร์สโค้ดฉบับสมบูรณ์นี้ทำงานในไฟล์ `app/index.tsx` โดยจะจัดการระบบสมัครสมาชิก เข้าระบบ สลับหน้าจอ และบันทึกโทเค็นความปลอดภัยอย่างครบถ้วนจบในไฟล์เดียว:

```
import React, { useState, useEffect } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  TouchableOpacity,
  ActivityIndicator,
  Alert,
  SafeAreaView,
  KeyboardAvoidingView,
  Platform,
  Keyboard,
  TouchableWithoutFeedback
} from 'react-native';
import { supabase } from '../lib/supabase';

export default function SecureDiaryAuthApp() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const [session, setSession] = useState<any>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

 // 1. ตรวจสอบสแกนหาประวัติล็อกอินเดิมที่เก็บไว้ใน Secure Store
  useEffect(() => {
    supabase.auth.getSession().then(({ data: { session } }) => {
      setSession(session);
      setIsLoading(false);
    });

  // ดักฟังการอัปเดตสเตตัสล็อกอิน/ล็อกเอาต์
    const { data: { subscription } } =
supabase.auth.onAuthStateChange((_event, session) => {
```

```
      setSession(session);
      setErrorMessage(null); // เคลียร์ข้อความแจ้งเตือนข้อผิดพลาดเก่า
    });

    return () => subscription.unsubscribe();
  }, []);

// 2. ตรวจสอบความถูกต้องฟอร์ม (Validation Rules)
  const isEmailValid = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
  const isPasswordValid = password.length >= 6;
  const isFormReady = isEmailValid && isPasswordValid;

// 3. ฟังก์ชันลงทะเบียนสมัครสมาชิก (Sign Up)
  const handleSignUp = async () => {
    if (!isFormReady) return;
    try {
      setIsSubmitting(true);
      setErrorMessage(null);
      Keyboard.dismiss();

      const { data, error } = await supabase.auth.signUp({
        email: email.trim(),
        password: password,
      });

      if (error) throw error;

      Alert.alert(
        'สมัครสมาชิกสำเร็จ!',
        'ระบบส่งอีเมลยืนยันตัวตนไปยังที่อยู่ของท่านแล้ว (กรุณาเช็คอินบ็อกซ์หรือโฟลเดอร์สแปม)',
    [{ text: 'ตกลง' }]
   );
   // เคลียร์ฟิลด์รหัสผ่าน
      setPassword('');
    } catch (error: any) {
      setErrorMessage(error.message || 'เกิดข้อผิดพลาดในการลงทะเบียน');
    } finally {
      setIsSubmitting(false);
    }
  };

// 4. ฟังก์ชันเข้าสู่ระบบ (Sign In)
  const handleSignIn = async () => {
    if (!isFormReady) return;
    try {
      setIsSubmitting(true);
      setErrorMessage(null);
      Keyboard.dismiss();
```

```
      const { error } = await supabase.auth.signInWithPassword({
        email: email.trim(),
        password: password,
      });

      if (error) throw error;

    // เมื่อล็อกอินผ่าน onAuthStateChange จะดักสลับหน้าจอให้อัตโนมัติ
    } catch (error: any) {
      setErrorMessage(error.message || 'รหัสผ่านหรืออีเมลไม่ถูกต้อง');
    } finally {
      setIsSubmitting(false);
    }
  };

// 5. ออกจากระบบ (Sign Out)
  const handleSignOut = async () => {
    try {
      setIsLoading(true);
      await supabase.auth.signOut();
    } catch (error) {
      Alert.alert('ข้อผิดพลาด', 'ไม่สามารถลงชื่อออกจากระบบได้');
    } finally {
      setIsLoading(false);
    }
  };

  if (isLoading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#38bdf8" />
        <Text style={styles.loadingText}>กำลังตรวจสอบสิทธิ์เข้าระบบ...</Text>
      </View>
  );
  }

// --- แผงการเรนเดอร์สลับ UI (Conditional Rendering) ---
  if (session) {
  // 1. หน้าจอหลักเมื่อเข้าสู่ระบบได้สำเร็จ (Secure Diary Dashboard)
    return (
      <SafeAreaView style={styles.safeArea}>
        <View style={styles.dashboardContainer}>
     <View style={styles.headerBlock}>
            <Text style={styles.dashboardTitle}>🔒 Secure Diary</Text>
            <Text style={styles.userEmailLabel}>ผู้ใช้:
{session.user.email}</Text>
          </View>

          {/* กล่องบันทึกข้อมูลส่วนตัวจำลอง */}
          <View style={styles.diaryContentCard}>
```

```
            <Text style={styles.diaryCardHeader}>📝 บันทึกลับสุดยอดประจำวัน</Text>
            <Text style={styles.diaryCardBody}>
        "วันนี้เราได้ศึกษาความแตกต่างระหว่าง Authentication และ Authorization สำเร็จ รวมถึงการ
เปลี่ยนไปใช้ expo-secure-store แทนคีย์ plain text ข้อมูลทั้งหมดถูกป้องกันอย่างแน่นหนาแล้วบนระบบคลาวด์
..."
            </Text>
            <Text style={styles.securityBadge}>🛡️ เข้ารหัสด้วย
Keystore/Keychain</Text>
          </View>

          <TouchableOpacity style={styles.signOutBtn}
onPress={handleSignOut}>
            <Text style={styles.signOutBtnText}>🚪 ลงชื่อออกจากระบบ</Text>
          </TouchableOpacity>
        </View>
      </SafeAreaView>
  );
  }

 // 2. หน้าจอแบบฟอร์มยืนยันตนเมื่อยังไม่มีเซสชัน (Login & Register Interface)
  return (
    <SafeAreaView style={styles.safeArea}>
      <TouchableWithoutFeedback onPress={Keyboard.dismiss}>
        <KeyboardAvoidingView
          behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
          style={styles.keyboardAvoid}
        >
          <View style={styles.loginContainer}>

            {/* หัวข้อล็อกอิน */}
            <View style={styles.loginHeader}>
              <Text style={styles.loginTitle}>🔧 Private Vault</Text>
              <Text style={styles.loginSubtitle}>กรุณาเข้าสู่ระบบด้วยอีเมลส่วนตัวเพื่อเปิด
ไดอารี่ลับ</Text>
            </View>

            {/* แผงแสดงข้อผิดพลาด */}
            {errorMessage && (
              <View style={styles.errorAlertBox}>
                <Text style={styles.errorAlertText}>⚠️
{errorMessage}</Text>
              </View>
        )}

            {/* ช่องกรอกข้อมูล */}
            <View style={styles.formCard}>
              <Text style={styles.inputTitle}>อีเมล (Email Address)</Text>
              <TextInput
                style={styles.input}
```

```
                placeholder="กรอกอีเมล (เช่น user@email.com)..."
                placeholderTextColor="#64748b"
                value={email}
                onChangeText={setEmail}
                keyboardType="email-address"
                autoCapitalize="none"
            />

            <Text style={styles.inputTitle}>รหัสผ่าน (Password - ขั้นต่ำ 6 ตัวอักษร)
</Text>
            <TextInput
          style={styles.input}
                placeholder="กรอกรหัสผ่านปลอดภัย..."
                placeholderTextColor="#64748b"
                value={password}
                onChangeText={setPassword}
                secureTextEntry
                autoCapitalize="none"
            />

            {/* ข้อแนะนำการพิมพ์ฟอร์ม */}
            {!isFormReady && (email.length > 0 || password.length > 0) &&
(
              <Text style={styles.formWarningText}>
          * รูปแบบอีเมลต้องถูกต้อง และรหัสผ่านต้องมีความยาวอย่างน้อย 6 ตัวอักษร
              </Text>
            )}

            {isSubmitting ? (
              <ActivityIndicator size="small" color="#38bdf8"
style={styles.spinner} />
            ) : (
              <View style={styles.actionButtonGroup}>
                <TouchableOpacity
                  style={[styles.btn, styles.signInBtn, !isFormReady &&
styles.disabledBtn]}
                  onPress={handleSignIn}
                  disabled={!isFormReady}
                >
                  <Text style={styles.btnText}>🔑 เข้าสู่ระบบ</Text>
                </TouchableOpacity>

                <TouchableOpacity
                  style={[styles.btn, styles.signUpBtn, !isFormReady &&
styles.disabledBtn]}
                  onPress={handleSignUp}
                  disabled={!isFormReady}
                >
                  <Text style={styles.signUpBtnText}>📝 สมัครสมาชิก</Text>
```

```
                    </TouchableOpacity>
                  </View>
        )}
            </View>
          </View>
        </KeyboardAvoidingView>
      </TouchableWithoutFeedback>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  center: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#0f172a',
  },
  loadingText: {
    color: '#94a3b8',
    marginTop: 12,
    fontSize: 14,
  },
  keyboardAvoid: {
    flex: 1,
  },
  loginContainer: {
    flex: 1,
   padding: 24,
    justifyContent: 'center',
  },
  loginHeader: {
    alignItems: 'center',
    marginBottom: 24,
  },
  loginTitle: {
    fontSize: 26,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  loginSubtitle: {
    fontSize: 13,
    color: '#94a3b8',
    textAlign: 'center',
    marginTop: 8,
```

```
    lineHeight: 18,
  },
  errorAlertBox: {
    backgroundColor: '#ef444422',
    borderWidth: 1,
    borderColor: '#ef4444',
    padding: 12,
    borderRadius: 8,
    marginBottom: 16,
  },
  errorAlertText: {
    color: '#f87171',
    fontSize: 12,
    fontWeight: '600',
  },
  formCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 20,
    borderWidth: 1,
    borderColor: '#334155',
  },
  inputTitle: {
    color: '#cbd5e1',
  fontSize: 12,
    fontWeight: '600',
    marginBottom: 6,
    marginTop: 10,
  },
  input: {
    backgroundColor: '#0f172a',
    borderRadius: 8,
    borderWidth: 1,
    borderColor: '#334155',
    color: '#ffffff',
    padding: 12,
    fontSize: 14,
  marginBottom: 10,
  },
  formWarningText: {
    color: '#fbbf24',
    fontSize: 11,
    marginTop: 4,
    marginBottom: 10,
    lineHeight: 16,
```

```
  },
  actionButtonGroup: {
    marginTop: 14,
  },
  btn: {
    borderRadius: 8,
    height: 44,
    justifyContent: 'center',
    alignItems: 'center',
    marginBottom: 10,
  },
  signInBtn: {
    backgroundColor: '#38bdf8',
  },
  signUpBtn: {
    backgroundColor: 'transparent',
    borderWidth: 1,
    borderColor: '#38bdf8',
  },
  disabledBtn: {
    backgroundColor: '#475569',
    borderColor: '#475569',
    opacity: 0.4,
  },
  btnText: {
    color: '#0f172a',
    fontWeight: 'bold',
    fontSize: 14,
  },
  signUpBtnText: {
    color: '#38bdf8',
    fontWeight: 'bold',
    fontSize: 14,
  },
  spinner: {
    paddingVertical: 12,
  },
// ส่วนสไตล์หน้าจอ Dashboard
  dashboardContainer: {
    flex: 1,
    padding: 24,
    justifyContent: 'space-between',
  },
  headerBlock: {
    alignItems: 'center',
    marginTop: 20,
  },
  dashboardTitle: {
    fontSize: 24,
```

```
    fontWeight: 'bold',
    color: '#ffffff',
  },
  userEmailLabel: {
    fontSize: 13,
    color: '#10b981',
    marginTop: 6,
    fontWeight: '600',
    backgroundColor: '#065f4633',
    paddingVertical: 4,
    paddingHorizontal: 12,
    borderRadius: 12,
  },
  diaryContentCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 20,
    borderWidth: 1,
    borderColor: '#334155',
    marginVertical: 40,
  },
  diaryCardHeader: {
    color: '#ffffff',
    fontWeight: 'bold',
   fontSize: 15,
    marginBottom: 10,
  },
  diaryCardBody: {
    color: '#94a3b8',
    fontSize: 13,
    lineHeight: 22,
  },
  securityBadge: {
    color: '#10b981',
    fontSize: 11,
    fontWeight: '600',
    marginTop: 15,
  },
  signOutBtn: {
    backgroundColor: '#ef4444',
    borderRadius: 8,
    height: 46,
    justifyContent: 'center',
    alignItems: 'center',
    marginBottom: 20,
  },
  signOutBtnText: {
```

```
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 14,
  },
});
```

5. [เนื้อหาของแถมพิเศษ (Bonus Advanced Section)]

หลังจากผู้เรียนพัฒนาโค้ดแบบ Single-page สำเร็จเรียบร้อยแล้ว ในการสเกลหรือขยายแอปพลิเคชันสู่ระบบที่มีหลายหน้าจอในระบบ Expo Router เราจะประยุกต์ใช้แนวทาง `useContext` ในการแชร์เซสชันล็อกอินข้ามคอมโพเนนต์

ให้นักพัฒนาจัดสรรสร้างโครงสร้างไฟล์เพิ่มเติมดังนี้:

ไฟล์ที่ 1: จัดทำ Provider ที่ context/AuthContext.tsx

```
import React, { createContext, useState, useEffect, useContext } from
'react';
import { supabase } from '../lib/supabase';
import { Session, User } from '@supabase/supabase-js';

interface AuthContextType {
  session: Session | null;
  user: User | null;
  isLoading: boolean;
  signOut: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType>({
  session: null,
  user: null,
  isLoading: true,
  signOut: async () => {}
});

export const AuthProvider = ({ children }: { children: React.ReactNode })
=> {
  const [session, setSession] = useState<Session | null>(null);
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    supabase.auth.getSession().then(({ data: { session } }) => {
      setSession(session);
      setUser(session ? session.user : null);
      setIsLoading(false);
    });

    const { data: { subscription } } =
supabase.auth.onAuthStateChange((_event, session) => {
```

```
      setSession(session);
      setUser(session ? session.user : null);
      setIsLoading(false);
    });

    return () => subscription.unsubscribe();
  }, []);

  const signOut = async () => {
    await supabase.auth.signOut();
  };

  return (
    <AuthContext.Provider value={{ session, user, isLoading, signOut }}>
      {children}
    </AuthContext.Provider>
 );
};

export const useAuth = () => useContext(AuthContext);
```

ไฟล์ที่ 2: ดักจับความปลอดภัยหน้าจอที่ Root Layout app/_layout.tsx

นำเอา Provider เข้ามาครอบโครงหน้าของแอปพลิเคชัน เพื่อแบ่งกลุ่มเส้นทางและสกรีน ด้วยตัวดัก Route Guard:

```
import React, { useEffect } from 'react';
import { Slot, useRouter, useSegments } from 'expo-router';
import { AuthProvider, useAuth } from '../context/AuthContext';
import { ActivityIndicator, View } from 'react-native';

function InitialLayout() {
  const { session, isLoading } = useAuth();
  const segments = useSegments();
  const router = useRouter();

  useEffect(() => {
    if (isLoading) return;

  // เช็คว่าหน้าจอที่จะเข้าอยู่ในกรอบกลุ่มล็อกอินหรือไม่
    const inAuthGroup = segments[0] === '(auth)';

    if (!session && !inAuthGroup) {
   // หากไม่มีเซสชันและไม่ได้อยู่หน้าล็อกอิน ให้ดีบผู้ใช้ไปหน้าล็อกอิน
      router.replace('/(auth)/login');
    } else if (session && inAuthGroup) {
   // หากมีเซสชันและคาอยู่หน้าล็อกอิน ให้กระโดดข้ามไปหน้าแดชบอร์ดหลัก
      router.replace('/(app)/dashboard');
    }
```

```
  }, [session, isLoading, segments]);

  if (isLoading) {
    return (
      <View style={{ flex: 1, justifyContent: 'center', alignItems:
'center' }}>
        <ActivityIndicator size="large" />
      </View>
  );
  }

  return <Slot />;
}

export default function RootLayout() {
  return (
    <AuthProvider>
      <InitialLayout />
    </AuthProvider>
 );
}
```


เอกสารอ้างอิง

Expo. (2024). *SecureStore*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/securestore/

Supabase. (2024). *Supabase Auth with React Native*. จาก https://supabase.com/docs/guides/auth/auth-helpers/react-native

Supabase. (2024). *Supabase JavaScript Auth API Reference*. จาก https://supabase.com/docs/reference/javascript/auth-signup

Supabase. (2024). *User Management with Supabase*. จาก https://supabase.com/docs/guides/auth

แผนบริหารการสอนประจำบทที่ 13

หัวข้อเนื้อหาประจำบท

13.1 เทคนิคการตรวจสอบและแก้ไขข้อผิดพลาดใน Expo SDK 54 (Debugging)

13.2 สถาปัตยกรรมและหลักการของ Expo Application Services (EAS)

13.3 การจัดเตรียมความพร้อมแอปพลิเคชันใน app.json

13.4 การตั้งค่าโปรไฟล์การคอมไพล์ใน eas.json

13.5 ขั้นตอนการสั่งรันคอมไพล์ด้วยคำสั่ง EAS CLI

13.6 แนวทางและกระบวนการเผยแพร่แอปขึ้น App Stores

13.7 ขั้นตอนการอัปเกรดเวอร์ชันของ Expo SDK (Upgrading Expo SDK)

สรุป

แบบฝึกหัด

เอกสารอ้างอิง

วัตถุประสงค์เชิงพฤติกรรม

1. ผู้เรียนสามารถใช้เครื่องมือดีบั๊กเกอร์ (Chrome DevTools) ในการดักจับ Breakpoints และตรวจสอบค่าตัวแปรในแอปพลิเคชันได้ถูกต้อง
2. ผู้เรียนสามารถตั้งค่าชื่อแพ็คเกจสากล (Package Name / Bundle Identifier) และเลขรุ่นเวอร์ชันในไฟล์ app.json ได้ถูกต้องตามเกณฑ์ของ Store
3. ผู้เรียนสามารถจัดทำไฟล์ตั้งค่า eas.json เพื่อแบ่งแยกโปรไฟล์การคอมไพล์แอปประเภทต่าง ๆ ได้สำเร็จ
4. ผู้เรียนสามารถรันคำสั่งสั่งคอมไพล์แอปพลิเคชันบนคลาวด์ของ Expo (EAS Build) เพื่อสร้างไฟล์ติดตั้ง APK สำหรับ Android ได้สำเร็จ
5. ผู้เรียนสามารถทำและอธิบายขั้นตอนการอัปเกรดเวอร์ชัน Expo SDK และการปรับรุ่นไลบรารีอื่น ๆ ด้วยคำสั่ง npx expo install --fix ได้อย่างถูกต้อง
6. ผู้เรียนสามารถอธิบายขั้นตอนเตรียมเอกสารและบัญชีผู้พัฒนาสำหรับการส่งมอบแอปขึ้น Google Play Store และ Apple App Store ได้ถูกต้อง

วิธีสอนและกิจกรรมการเรียนการสอนประจำบท

49. **การบรรยายเรื่อง Debugging, Upgrades & EAS Compilations:** บรรยาย

หลักการแกะหาสาเหตุบั๊กด้วยเครื่องมือดีบั๊กเกอร์ผ่าน Chrome, วิธีการวางแผนขั้นตอนการอัปเกรด SDK ของ Expo ข้ามรุ่น, และเปรียบเทียบสถาปัตยกรรม EAS Cloud Build กับการคอมไพล์แอปแบบดั้งเดิม

50. **การทำ Live Coding Demonstration หัวข้อ Config, Upgrade & EAS Android Preview Build:** ผู้สอนสาธิตการเรียกคำสั่งตรวจสอบและเคลียร์แคชระบบเมื่อมีการขยับรุ่น SDK, การปรับแต่งเลขรุ่นใน app.json, การสร้างไฟล์โปรไฟล์ eas.json และเรียกใช้คำสั่งดีทริกเกอร์ eas build เพื่อสร้างไฟล์ APK สำเร็จบนเว็บคลาวด์
51. **กิจกรรมตั้งชื่อรหัสสากลป้องกันซ้ำ (Bundle Identifier Challenge):** ให้ผู้เรียนฝึกฝนตรวจเช็คและระบุรหัสชื่อแพ็คเกจ เช่น com.yourcompany.appname ใน app.json และทดลองคำนวณเวอร์ชันอัปเกรดแอปแบบทศนิยมเพื่อส่งอัปเดตระบบขึ้นคลาวด์
52. **กิจกรรมล่าบั๊กและซ่อมไลบรารีข้ามรุ่น (Bug Hunting & SDK Upgrade Challenge):** ผู้สอนจัดเตรียมโค้ดเก่าที่มีพารามิเตอร์ตกค้างของ SDK รุ่นเก่า ให้ผู้เรียนทดลองใช้คำสั่ง npx expo install --fix เพื่ออัปเกรดและแก้ความขัดแย้งของไลบรารี (Dependency conflicts) จากนั้นเปิดแก้บั๊กด้วย Chrome Debugger ให้สำเร็จ

สื่อการเรียนการสอน

48. เอกสารสไลด์เรื่อง "Debugging, SDK Upgrades, EAS Build Cloud Engine and Publishing to App Stores"
49. บัญชีเข้าใช้งานระบบ Expo Dev (Expo Accounts)
50. เครื่องมือระบบ CLI ของ Expo และชุดคำสั่งคอมไพล์ EAS CLI
51. ลิงก์คู่มือเอกสารอย่างเป็นทางการของ EAS Build, Google Play Console, Apple Developer Portal และคู่มือ Expo Upgrade Guide

การวัดผลและการประเมินผล

1. การวัดผล

1.1 ประเมินผลคะแนนการทำแบบทดสอบท้ายบทเรียน 10 ข้อ

1.2 ตรวจสอบโครงสร้างความถูกต้องของการจัดป้อนข้อมูลในไฟล์ app.json และ eas.json รวมถึงคำสั่งที่ใช้ในการแก้ไขความขัดแย้งไลบรารีในใบงานปฏิบัติการ

1.3 สังเกตความสำเร็จในการรันคำสั่ง Build และได้รับไฟล์ติดตั้ง APK ปลายทางที่มีขนาดพร้อมใช้งาน

2. การประเมินผล

2.1 ผู้เรียนทำคะแนนแบบทดสอบปรนัยท้ายบทเรียนได้ไม่น้อยกว่าร้อยละ 80 (อย่างน้อย 8 จาก 10 ข้อ)

2.2 ผู้เรียนสามารถสร้างโครงสร้างตั้งค่า แก้ไขไลบรารีตกหล่นในการขยับรุ่น และส่งแอปขึ้นไปคอมไพล์บนระบบคลาวด์ผ่านคำสั่ง EAS Build จนได้รับไฟล์ผลลัพธ์ติดตั้งได้สำเร็จผ่านเกณฑ์ (ผ่าน/ไม่ผ่าน)

2.3 สังเกตความระมัดระวังในการเก็บรักษา Credentials คีย์การลงทะเบียน และความถูกต้องในการตรวจสอบระบบ Log

บทที่ 13

การดีบั๊ก การสร้างแอปพลิเคชัน และการเผยแพร่ด้วย EAS Build

การพัฒนาโมบายแอปพลิเคชันจนสามารถพิมพ์โค้ดรันได้ในเครื่องจำลอง (Simulator) ถือเป็นเพียงครึ่งแรกของกระบวนการพัฒนา ด่านสำคัญถัดไปคือการแกะรอยแก้ไขข้อผิดพลาดในระดับอุปกรณ์จริง (Native Debugging) และการแปลงซอร์สโค้ด React Native ให้กลายเป็นไฟล์ติดตั้งไบนารีเนทีฟ (เช่น `.apk` สำหรับ Android หรือ `.ipa` สำหรับ iOS) เพื่อส่งขึ้น Store ในบทเรียนสุดท้ายนี้ เราจะเรียนรู้เทคนิคการดีบั๊กใน Expo SDK 54, ทำความเข้าใจระบบคลาวด์คอมไพล์แอป **EAS Build** และขั้นตอนเตรียมความพร้อมเพื่อเผยแพร่แอปพลิเคชันสู่ผู้ใช้งานจริงทั่วโลก

13.1 เทคนิคการตรวจสอบและแก้ไขข้อผิดพลาดใน Expo SDK 54 (Debugging)

ในการเขียนแอปพลิเคชัน ย่อมเกิดข้อผิดพลาดหรือข้อขัดข้องขึ้นได้ ทั้งจากโค้ดจาวาสคริปต์ของเราเอง หรือเออเรอร์ระดับเครื่องยนต์เนทีฟ (Native Crash) Expo SDK 54 ได้จัดสรรเครื่องมือในการค้นหาความจริง (Expo, 2024) ดังนี้:

1. การเรียกใช้งาน Expo Developer Menu

ขณะรันแอปบนจำลองหรือสมาร์ทโฟนจริงผ่าน Expo Go เราสามารถกดเปิดเมนูผู้พัฒนาได้โดย:

- **ในเครื่อง Simulator:** กดปุ่ม `m` บนคีย์บอร์ดในเทอร์มินัลที่รัน Expo CLI
- **ในสมาร์ทโฟนจริง:** ทำการ **เขย่าโทรศัพท์มือถือ (Shake Device)**

ในเมนูนี้เราสามารถสั่งรีโหลดแอปใหม่ทั้งหมด (Reload) หรือเปิดใช้งานเครื่องมือการสืบค้นองค์ประกอบ UI (Toggle Inspector)

2. การเชื่อมต่อ Chrome Developer Tools

เราสามารถดีบั๊กโค้ด JavaScript ที่กำลังรันอยู่บนโทรศัพท์มือถือผ่านเครื่องมืออันคุ้นเคยของ Google Chrome โดยเปิดบราวเซอร์ Chrome ขึ้นมาแล้วทำตามขั้นตอนดังนี้:

1. เปิด Developer Tools ใน Chrome (กด `F12`)
2. กดปุ่ม `Ctrl + Shift + P` (หรือ `Cmd + Shift + P` บน Mac) พิมพ์คำสั่ง **`Open Debugger`** หรือคลิกที่ลิงก์ดีบั๊กเกอร์ที่ Expo CLI แสดงขึ้นในหน้าต่างเทอร์มินัล (มักเป็นคีย์ลัด `j` ในคอมพิวเตอร์)
3. ระบบจะสลับหน้าจอไปที่แถบดีบั๊กเกอร์ ซึ่งเปิดโอกาสให้เราสามารถดักจับตัวแปรในหน้า Console, ใส่จุดหยุดทำงาน (Breakpoints) และตรวจสอบตาราง Call Stack ได้ทันทีเสมือนการพัฒนาเว็บแอปพลิเคชัน

13.2 สถาปัตยกรรมและหลักการของ Expo Application Services (EAS)

ในอดีต การสร้างไฟล์ติดตั้ง (Build App) ของ React Native เป็นเรื่องยุ่งยากอย่างมหาศาล เพราะนักพัฒนาจำเป็นต้องตั้งค่าแอนดรอยด์ SDK อย่างหนาแน่นในเครื่อง หรือหากต้องการสร้างไฟล์สำหรับ iOS ก็ถูกบังคับให้ต้องมีเครื่องคอมพิวเตอร์ Mac ที่ลงระบบ Xcode เท่านั้น

เพื่อขจัดความยุ่งยากนี้ Expo จึงได้พัฒนาบริการคลาวด์สำเร็จรูปที่ชื่อว่า **EAS (Expo Application Services)** (Expo, 2024) ซึ่งเป็นระบบเซิร์ฟเวอร์คลาวด์ประสิทธิภาพสูงที่จะคอยรับโค้ดของเราขึ้นไปคอมไพล์เป็นไฟล์ติดตั้งเนทีฟให้โดยอัตโนมัติ

ข้อดีของ EAS Build:

• **ไม่ต้องมีเครื่อง Mac ก็สร้างแอป iOS ได้:** เพราะการ Build เกิดขึ้นบนเซิร์ฟเวอร์คลาวด์ของ Expo เอง

• **ประหยัดทรัพยากรเครื่องคอมพิวเตอร์:** ไม่ต้องรันคอมไพล์หนักหน่วงจน RAM เต็มในเครื่องโลคัลของเรา

• **จัดการคีย์ความปลอดภัยให้อัตโนมัติ:** คอยดูแลและสร้างใบรับรองดิจิทัล (Signing Credentials) สำหรับแอนดรอยด์และ iOS ให้ปลอดภัย

13.3 การจัดเตรียมความพร้อมแอปพลิเคชันใน `app.json`

ก่อนที่เราจะส่งโค้ดขึ้นไปประกอบเป็นไฟล์แอปพลิเคชันบนคลาวด์ เราต้องระบุคุณลักษณะที่จำเป็นในไฟล์ `app.json` ซึ่งทำหน้าที่เสมือนบัตรประชาชนระบุตัวตนของแอปพลิเคชัน:

```
{
 "expo": {
  "name": "My Personal Diary",        // ชื่อแอปที่จะโชว์ใต้ไอคอนบนหน้าจอโทรศัพท์
  "slug": "my-personal-diary",
  "version": "1.0.0",                 // เลขเวอร์ชันของแอปที่โชว์ให้ผู้ใช้เห็น
  "orientation": "portrait",
  "icon": "./assets/icon.png",        // ที่อยู่รูปภาพไอคอนแอปพลิเคชัน
  "userInterfaceStyle": "dark",
  "splash": {
   "image": "./assets/splash-image.png", // หน้าจอต้อนรับตอนเปิดแอป (Splash Screen)
   "resizeMode": "contain",
   "backgroundColor": "#0f172a"
    },
  "ios": {
   "supportsTablet": true,
   "bundleIdentifier": "com.yourdomain.mydiary", // รหัสระบุตัวตนห้ามซ้ำของ iOS
   "buildNumber": "1"
```

```
    },
  "android": {
   "adaptiveIcon": {
    "foregroundImage": "./assets/adaptive-icon.png",
    "backgroundColor": "#0f172a"
      },
   "package": "com.yourdomain.mydiary",          // รหัสระบุตัวตนห้ามซ้ำของ Android (Package Name)
   "versionCode": 1                  // เลขรันของรุ่นแอปในสารระบบดาต้าเบสสโตร์ (ต้องเพิ่มค่าขึ้นทีละ 1 ทุกครั้งที่อัปเดตสโตร์)
    }
  }
}
```

คำเตือน (Warning): ฟิลด์ `android.package` และ `ios.bundleIdentifier` ต้องใช้รูปแบบจดโดเมนย้อนกลับ (Reverse Domain Name) เช่น `com.`บริษัท.ชื่อแอป และชื่อนี้ห้ามไปซ้ำกับแอปพลิเคชันอื่นใดในโลกเด็ดขาด มิฉะนั้นสโตร์จะปฏิเสธการอัปโหลดเข้าสู่คลังระบบ

13.4 การตั้งค่าโปรไฟล์การคอมไพล์ใน `eas.json`

เราสามารถกำหนดค่าพฤติกรรมการคอมไพล์แอปแยกประเภทได้ผ่านไฟล์ `eas.json` (ซึ่งตั้งอยู่ที่โฟลเดอร์นอกสุดของโปรเจกต์) โดยเรามักสร้างไฟล์นี้อัตโนมัติด้วยคำสั่ง:

```
eas build:configure
```

ตัวอย่างการกำหนดค่าในไฟล์ `eas.json`:

สำหรับการนำมาสอนหรือทดสอบติดตั้งเองในเครื่อง โดยไม่ต้องส่งขึ้นสโตร์ทันที เราจะตั้งค่าโปรไฟล์สำหรับดาวน์โหลดไฟล์ APK (สำหรับ Android) เอาไว้ทดลองรันภายในองค์กร (Internal Distribution) ดังนี้:

```
{
 "cli": {
  "version": ">= 10.0.0"
  },
 "build": {
  "development": {
   "developmentClient": true,
   "distribution": "internal"
    },
  "preview": {
   "distribution": "internal",
   "android": {
```

```
      "buildType": "apk"            // บังคับให้คอมไพล์ออกมาเป็นไฟล์ติดตั้งด่วนประเภท .apk
        }
      },
    "production": {
     "distribution": "store"        // บังคับเตรียมไฟล์สำหรับส่งขึ้น App Store / Play Store
      }
    }
}
```

13.5 ขั้นตอนการสั่งรันคอมไพล์ด้วยคำสั่ง EAS CLI

เมื่อเตรียมความพร้อมไฟล์คอนฟิกสำเร็จแล้ว ให้ทำตามขั้นตอนการยิงคำสั่งคอมไพล์จริงบนคลาวด์ด้วย EAS CLI (Expo, 2024) ดังนี้:

ขั้นตอนที่ 1: ติดตั้ง EAS CLI ส่วนกลางในเครื่องคอมพิวเตอร์ของคุณ

```
npm install -g eas-cli
```

ขั้นตอนที่ 2: ลงชื่อเข้าใช้บัญชี Expo

ทำการล็อกอินระบบด้วยบัญชีที่สมัครไว้กับเว็บไซต์ Expo:

```
eas login
```

ขั้นตอนที่ 3: สั่งรันเริ่มคอมไพล์ภาพแอป

ส่งคำสั่งสั่งรันโปรไฟล์ `preview` เพื่อสร้างไฟล์ติดตั้ง APK สำหรับ Android:

```
eas build --platform android --profile preview
```

ขั้นตอนที่เกิดขึ้นเบื้องหลัง (Build Pipeline):

1. **EAS CLI** จะตรวจสอบความถูกต้องของไฟล์ `app.json` และ `eas.json`

2. ระบบจะบีบอัดแพ็คเกจซอร์สโค้ดฝั่ง Client (เฉพาะโค้ดดิ้ง ไม่รวมโฟลเดอร์ node_modules) ส่งผ่านสายขึ้นคลาวด์ของ Expo

3. คลาวด์ของ Expo จะเปิดระบบจำลองเซิร์ฟเวอร์ขึ้นมาทำการติดตั้งแพ็คเกจและทำลายคอมไพล์โค้ดเป็นไบนารีระดับเนทีฟ

4. เมื่อกระบวนการเสร็จสิ้น เทอร์มินัลของคุณจะแสดง **ลิงก์ดาวน์โหลดเว็บ** พร้อมกับรหัส QR Code ขึ้นมาบนหน้าจอคอมพิวเตอร์

5. ผู้เรียนสามารถนำกล้องถ่ายรูปของสมาร์ทโฟนแอนดรอยด์สแกน QR Code นี้เพื่อเปิดดาวน์โหลดไฟล์ติดตั้ง .apk มาติดตั้งรันทำงานบนโทรศัพท์มือถือจริงได้ทันที!

13.6 แนวทางและกระบวนการเผยแพร่แอปขึ้น App Stores

เมื่อผู้พัฒนาทดสอบแอปจนมั่นใจและพร้อมที่จะเผยแพร่ให้คนดาวน์โหลดใช้งานทั่วไป ขั้นตอนการดำเนินงานขึ้นร้านค้าทางการมีดังนี้:

```
graph TD
    A[คอมไพล์แอปพลิเคชันแบบ Production] --> B{ระบบปฏิบัติการ?}
    B -->|Android| C[สร้างไฟล์ .aab - Android App Bundle]
    B -->|iOS| D[สร้างไฟล์ .ipa พร้อมใบรับรอง Apple]
    C --> E[สมัครบัญชี Google Play Console - มีค่าบริการครั้งเดียว $25]
    D --> F[สมัครบัญชี Apple Developer Program - ค่าบริการรายปี $99]
    E --> G[อัปโหลดไฟล์ผ่านเว็บ Google Play Console / eas submit]
    F --> H[อัปโหลดไฟล์ผ่าน Transporter หรือ eas submit]
    G --> I[ตั้งค่าฟอร์มความปลอดภัย / ข้อมูลจำกัดอายุ]
    H --> J[ตั้งค่า App Store Connect / นโยบายความเป็นส่วนตัว]
    I --> K[ส่งตรวจสอบและได้รับอนุมัติขึ้นดาวน์โหลดบน Play Store]
    J --> L[ส่งตรวจสอบและได้รับอนุมัติขึ้นดาวน์โหลดบน App Store]
```

การส่งด้วยความสามารถคำสั่งอัตโนมัติ (EAS Submit):

หลังจากที่สร้างแอปแบบโปรไฟล์การผลิตเสร็จสิ้น เราสามารถสั่งส่งอัปโหลดตรงเข้าร้านค้าสโตร์ต่างๆ ผ่านคำสั่ง:

```
eas submit --platform android
```

คำสั่งนี้จะดึง Credentials และเชื่อม API ไดเรกทอรีของสโตร์เพื่อส่งมอบแอปขึ้นไปเตรียมรีวิวในฝั่งหลังบ้านได้อย่างรวดเร็วและปลอดภัยลดความซับซ้อนของผู้พัฒนาแอปพลิเคชันอย่างมหาศาล

13.7 ขั้นตอนการอัปเกรดเวอร์ชันของ Expo SDK (Upgrading Expo SDK)

ทีมงาน Expo จะทำการเปิดตัวรุ่น SDK ใหม่ ๆ (Major Releases) ทุก ๆ 3 เดือนเพื่อเพิ่มฟีเจอร์ ปรับแต่งประสิทธิภาพ และอัปเกรดรุ่นซอฟต์แวร์เบื้องหลังของ React Native และ React ให้ทันสมัยและปลอดภัย การมีทักษะในการอัปเกรด SDK ข้ามเวอร์ชันจึงเป็นเรื่องจำเป็นอย่างยิ่งสำหรับนักพัฒนาระดับอาชีพ โดยมีลำดับขั้นตอนที่เป็นมาตรฐานดังนี้:

ขั้นตอนที่ 1: ตรวจสอบและอัปเดตเครื่องมือ EAS CLI

ก่อนเริ่มทำการอัปเกรดโปรเจกต์ ควรทำการปรับปรุงเครื่องมือบิลด์ให้เป็นเวอร์ชันล่าสุด เพื่อรองรับพารามิเตอร์ของ SDK ตัวใหม่:

```
npm install -g eas-cli
```

ขั้นตอนที่ 2: ติดตั้งรุ่นของตัวหลัก Expo SDK ปลายทาง

เราสามารถระบุให้โปรเจกต์เปลี่ยนไปใช้ Expo เวอร์ชันล่าสุดได้ด้วยคำสั่ง:

```
npm install expo@latest
```

หรือหากต้องการเจาะจงระบุเป็นรุ่นเวอร์ชันเฉพาะ เช่น ขยับขึ้นไปที่ Expo SDK 54 ให้ป้อนระบุรุ่นดังนี้:

```
npx expo install expo@^54.0.0
```

ขั้นตอนที่ 3: จัดการปรับปรุงรุ่นไลบรารีอื่น ๆ (Dependency Reconciliation)

ข้อควรจำ (Important): นี่คือขั้นตอนที่สำคัญที่สุดในการอัปเกรดระบบ Expo

เมื่อตัวแกนหลักอย่าง `expo` ได้ขยับรุ่นขึ้นไปแล้ว แพ็คเกจอื่น ๆ เช่น `react-native`, `react`, `expo-router` หรือ `expo-image` อาจจะยังทำงานค้างอยู่ที่เวอร์ชันเก่าซึ่งจะทำให้รันแอปพังหรือบิลด์ไม่ผ่าน

เราห้ามลบและไล่ติดตั้งทีละตัวด้วยตัวเอง แต่ให้พิมพ์รันคำสั่งพิเศษของ Expo ดังนี้:

``bash npx expo install --fix ` คำสั่งนี้จะทำการสแกนไฟล์ package.json` ค้นหาความขัดแย้งของรุ่นทั้งหมด และทำการสั่งดาวน์โหลดแพ็คเกจอื่น ๆ ทุกตัวที่เกี่ยวข้องให้เข้าสู่รุ่นเวอร์ชันที่เหมาะสมและผ่านการรับรองความเสถียร (Compatible versions) จากทางทีมพัฒนาของ Expo โดยออโต้

ขั้นตอนที่ 4: ล้างหน่วยความจำแคช (Clear Metro Cache)

ทุกครั้งที่ขยับรุ่นซอฟต์แวร์ เราต้องบังคับล้างประวัติแคชของการรันคอมไพเลอร์ Metro เพื่อไม่ให้มีเศษไฟล์เก่ามาทำให้แอปทำงานพังเงียบ:

```
npx expo start -c
```

(คีย์ออปชัน `-c` ย่อมาจากคำสั่ง `--clear` เพื่อล้างแคชระบบก่อนสตาร์ตโปรแกรมรันใหม่)

ขั้นตอนที่ 5: บิลด์ Dev Client ใหม่ (กรณีใช้งานคัสตอมไบนารี)

หากแอปพลิเคชันเดิมทำระบบใช้งานร่วมกับ Custom Development Clients (แทนการใช้แอป Expo Go มาตรฐาน) การขยับ SDK จะมีผลทำให้โค้ดฝั่งเนทีฟ (Android/iOS) ขยับรุ่นตาม

ไปด้วย นักพัฒนาจำเป็นต้องสั่งรัน `eas build --profile development` เพื่อรับไฟล์ติดตั้งตัวทดสอบใหม่มาลงทับในสมาร์ทโฟนเสมอ

สรุปท้ายบทเรียน

เทคนิคการตรวจสอบและแก้ไขข้อผิดพลาดใน Expo SDK 54: กระบวนการตรวจหาและดีบั๊กข้อผิดพลาดใน Expo SDK 54 สามารถใช้ Expo Developer Menu ในการสั่งรีโหลดหน้าจอหรือเปิดระบบ Inspector โดยสามารถเรียกใช้งานผ่านคีย์บอร์ด m ในเครื่องจำลองหรือการเขย่าอุปกรณ์จริง และสามารถเชื่อมโยงคำสั่ง JavaScript เข้ากับ Chrome Developer Tools ผ่านการทริกเกอร์ j ในคอมพิวเตอร์เพื่อเปิดหน้าต่างตรวจสอบ Console บันทึกสเตตัสและใส่จุดหยุดทำงานได้อย่างละเอียด

สถาปัตยกรรมและหลักการของ Expo Application Services (EAS): บริการ EAS Build คลาวด์อำนวยความสะดวกในการจัดทำไฟล์ติดตั้งเนทีฟสำเร็จรูปให้ผู้พัฒนาโดยแก้ปัญหาความยุ่งยากของการจัดตั้งเครื่อง Android SDK หรือความจำเป็นต้องมีเครื่อง Mac ในการคอมไพล์ระบบ iOS โดยระบบจะนำซอร์สโค้ดไปประกอบและจัดการคีย์ความปลอดภัยของแอปพลิเคชัน (Signing Credentials) บนคลาวด์เซิร์ฟเวอร์ความเร็วสูงให้โดยอัตโนมัติ

การจัดเตรียมความพร้อมแอปพลิเคชันใน app.json: ก่อนส่งมอบโค้ดไปประกอบไฟล์ติดตั้งระดับเนทีฟ ผู้พัฒนาจำเป็นต้องระบุคุณสมบัติตัวตนของแอปพลิเคชันในไฟล์ app.json เช่น ชื่อแอป เวอร์ชันของรุ่น ที่อยู่ของรูปไอคอนและภาพต้อนรับตอนเริ่มแอป รวมถึงการตั้งรหัสผ่านชุดจดทะเบียนที่ห้ามซ้ำกันเด็ดขาดในระดับสากลอย่าง android.package และ ios.bundleIdentifier

การตั้งค่าโปรไฟล์การคอมไพล์ใน eas.json: ไฟล์ eas.json ใช้สำหรับระบุเป้าหมายและขั้นตอนพฤติกรรมการบิลด์แอปพลิเคชัน โดยเราเขียนปรับแต่งให้แยกประเภทโปรไฟล์ได้ เช่น โปรไฟล์ development เพื่อสร้างไคลเอนต์ทดสอบในทีม, โปรไฟล์ preview ที่บังคับคอมไพล์ออกมาเป็นไฟล์ติดตั้ง .apk โดยตรงสำหรับเครื่อง Android และโปรไฟล์ production เพื่อสร้างไฟล์ติดตั้งมาตรฐานในการส่งตัวแอปเตรียมยื่นขึ้นร้านค้าอย่างเป็นทางการ

ขั้นตอนการสั่งรันคอมไพล์ด้วยคำสั่ง EAS CLI: ลำดับการสั่งประกอบแอปเริ่มจากติดตั้ง eas-cli และทำรายการล็อกอินด้วยคำสั่ง eas login จากนั้นจึงยิงคำสั่งคอมไพล์หลักตามโปรไฟล์ที่ตั้งค่าไว้ ซึ่งระบบจะนำรหัสโค้ดฝั่งไคลเอนต์ขึ้นประมวลผลบนเซิร์ฟเวอร์คลาวด์ของ Expo และแสดงที่อยู่ลิงก์และ QR Code บนหน้าต่างเทอร์มินัลเพื่อให้ผู้พัฒนาดาวน์โหลดไฟล์ไบนารี .apk มาลงตรวจสอบความเรียบร้อยบนโทรศัพท์จริงได้

แนวทางและกระบวนการเผยแพร่แอปขึ้น App Stores: กระบวนการยื่นแอปขึ้นร้านค้าทำได้โดยบิลด์โปรไฟล์การผลิตเพื่อให้ได้ไฟล์ .aab สำหรับ Android หรือไฟล์ .ipa สำหรับ iOS เพื่อ

เตรียมยื่นขอขึ้นร้านค้าอย่างเป็นทางการผ่านบัญชี Google Play Console หรือ Apple Developer Program โดยใช้เครื่องมือส่งเอกสารผ่านเว็บบอร์ดหรือสั่งงานอย่างสะดวกผ่านระบบจัดส่งอัตโนมัติด้วยคำสั่ง eas submit

ขั้นตอนการอัปเกรดเวอร์ชันของ Expo SDK: การปรับแต่งและขยับรุ่นโปรเจกต์ให้มีประสิทธิภาพและมีความปลอดภัยสูงสุดตามรุ่น SDK ล่าสุด ทำได้โดยติดตั้งตัวแกนหลัก expo รุ่นใหม่ จากนั้นสั่งทำงาน npx expo install --fix เพื่อปรับปรุงรุ่นไลบรารีอื่น ๆ ทั้งระบบให้มีความเข้ากันและผ่านการรับรองความเสถียร พร้อมทั้งบังคับล้างประวัติแคชรัน Metro ด้วย npx expo start -c เพื่อเคลียร์ไฟล์เก่าไม่ให้ระบบทำงานขัดข้อง

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้และความเข้าใจเกี่ยวกับกระบวนการตรวจสอบและดีบั๊กโค้ดแอปพลิเคชัน, สถาปัตยกรรมคลาวด์คอมไพล์ของบริการ Expo Application Services (EAS), การตั้งค่าคุณสมบัติแอปพลิเคชันในไฟล์ `app.json` และการจัดการตั้งค่าโปรไฟล์ใน `eas.json` ตลอดจนกลไกการอัปเดตอัปเกรดเวอร์ชันของ Expo SDK และการนำส่งแอปขึ้นเผยแพร่บนร้านค้าสากล

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: หากกำลังเชื่อมต่อแอปพลิเคชันเพื่อทดสอบอยู่บนสมาร์ทโฟนเครื่องจริง และผู้สอนสั่งให้ทำการตรวจสอบหน้าจอ UI ผ่านคำสั่งใน "Expo Developer Menu" จะต้องทริกเกอร์เรียกแผงคำสั่งนี้ขึ้นมาอย่างไรบนหน้าจอโทรศัพท์?

ก. ทำการแตะหน้าจอรัว ๆ 10 ครั้งติดต่อกัน

ข. ทำการเขย่าตัวเครื่องโทรศัพท์มือถือ (Shake Device)

ค. กดปุ่มลดระดับเสียงของเครื่อง 3 ครั้ง

ง. สั่งตัดสัญญาณอินเทอร์เน็ตบนตัวเครื่องชั่วคราว

ข้อ 2: การเข้าถึงหน้าจอดีบั๊กเกอร์สำหรับตรวจเช็คการทำงานของรหัสจาวาสคริปต์ ดักตรวจสอบตัวแปรในแท็บ Console และการปักหมุดจุดหยุดทำงาน (Breakpoints) นักพัฒนาสามารถเปิดหน้าต่างนี้ผ่านเครื่องมือใด?

ก. หน้าจอ Android Studio Logcat

ข. Google Chrome Developer Tools (โดยกดเรียกดีบั๊กเกอร์ผ่าน CLI)

ค. ระบบสไลด์เมนูของตัวจำลอง Simulator

ง. แอปพลิเคชันตั้งค่าพื้นฐานของตัวเครื่องมือถือ

ข้อ 3: คำว่า "EAS" ในระบบบริการคลาวด์ของ Expo ย่อมาจากคำเต็มในข้อใด?

ก. Expo Application Services

ข. Easy App Solutions

ค. Expo Android Systems

ง. Enterprise App Server

ข้อ 4: ข้อใดคือประโยชน์สูงสุดและเห็นภาพชัดเจนที่สุดในการเลือกใช้งานบริการระบบคลาวด์ EAS Build เข้ามาจัดสร้างประกอบไฟล์ติดตั้งแอปพลิเคชัน?

ก. ช่วยเร่งความเร็วในการดาวน์โหลดไฟล์รูปภาพในแอปพลิเคชันของฝั่งลูกค้า

ข. เปิดโอกาสให้นักพัฒนาสามารถสร้างคอมไพล์ไฟล์แอปเนทีฟ (เช่น แอป iOS) ได้สำเร็จจากระยะไกลโดยไม่จำเป็นต้องติดตั้งสภาพแวดล้อม Xcode หรือมีคอมพิวเตอร์ Mac เป็นของตนเอง

ค. ทำหน้าที่เขียนคำสั่งสืบค้น SQL-like ส่งไปยัง Supabase อัตโนมัติ

ง. สั่งปิดสัญญาณสัญญานเครือข่ายมือถือเพื่อประหยัดหน่วยความจำ RAM

ข้อ 5: แฟ้มข้อมูลใดในโฟลเดอร์โครงการ Expo ที่ทำหน้าที่บรรจุรายละเอียดระบุตัวตน ชื่อแอป ไอคอนแอป หน้า Splash Screen และรหัสแพ็คเกจปลายทางของตัวแอปพลิเคชัน?

ก. `eas.json`

ข. `package.json`

ค. `app.json`

ง. `babel.config.js`

ข้อ 6: การกรอกฟิลด์ระบุตัวตนสากลของแอปใน `app.json` เช่น `android.package` หรือ `ios.bundleIdentifier` ข้อใดคือรูปแบบการเขียนชื่อที่ถูกต้องและเป็นมาตรฐานยอมรับในทุกสโตร์?

ก. เขียนข้อความสตริงเพลน ๆ ตัวพิมพ์ใหญ่ เช่น `MY_DIARY_APP`

ข. เขียนเป็นรหัสพิกัดพิกสเปซ เช่น `/app/mydiary`

ค. เขียนโดยใช้รูปแบบโดเมนกลับหัว (Reverse Domain Name Notation) เช่น `com.yourcompany.appname`

ง. เขียนเป็นตัวเลขแสดงลำดับไอดีโครงการ เช่น 123.456.789

ข้อ 7: ในกรณีที่ต้องการเปลี่ยนประเภทของการคอมไพล์สำหรับฝั่ง Android ให้เปลี่ยนจากไฟล์ App Bundle มาเป็นไฟล์ดาวน์โหลดติดตั้งได้โดยตรง (APK File) เพื่อทดสอบภายในองค์กร จะต้องระบุค่าออปชันใดในโปรไฟล์ Preview ของไฟล์ `eas.json`?

ก. `"buildType": "apk"`

ข. `"android": { "format": "apk" }`

ค. `"android": { "buildType": "apk" }`

ง. `"distribution": "apk"`

ข้อ 8: ชุดไฟล์กำหนดค่าการทำงานใน `eas.json` ทำหน้าที่จัดแจงแบ่งแยกกลุ่มระดับประเภทการ Build เพื่ออะไร?

ก. เพื่อจัดกลุ่มหน้าจอ Route ใน Expo Router

ข. เพื่อแยกคุณลักษณะโปรไฟล์การสร้างแอปพลิเคชัน เช่น สำหรับพัฒนา (Development), สำหรับทดสอบภายใน (Preview) หรือสำหรับจัดส่งสโตร์จริง (Production)

ค. เพื่อจำกัดสิทธิ์ความปลอดภัยการขออนุญาตกล้องและพิกัดแผนที่

ง. เพื่อเก็บรักษาคีย์ anon key และ URL ของฐานข้อมูล

ข้อ 9: ภายหลังจากที่เราทำการสั่งอัปเกรดรหัสแกนหลักของ Expo ในไคลเอนต์สำเร็จ (เช่น ขยับจาก SDK 53 เป็น SDK 54) ข้อใดคือชุดคำสั่งคำสั่งวิเศษของ Expo CLI ที่ต้องรันสั่งการเพื่อแก้ปัญหาความเข้ากันได้ของรุ่นไลบรารีอื่น ๆ (Dependency Reconciliation) ป้องกันแอปพัง?

ก. `npm update --force`

ข. `npx expo install --fix`

ค. `npx expo doctor`

ง. `npx expo upgrade`

ข้อ 10: เมื่อนักพัฒนาเตรียมส่งมอบและอัปโหลดแอปพลิเคชันขึ้นร้านค้า Google Play Store เพื่อเผยแพร่เชิงพาณิชย์เป็นไฟล์รุ่นผลิต (Production) รูปแบบนามสกุลไฟล์แอปประเภทใดที่ Google Play Console บังคับให้จัดส่งอัปโหลดขึ้นสโตร์ในปัจจุบัน?

ก. **.apk** (Android Package)

ข. **.exe** (Executable)

ค. **.aab** (Android App Bundle)

ง. **.ipa** (iOS App Store Package)

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: วิธีเหล่านี้ไม่ใช่สัญญาณกระตุ้นการเปิดหน้าต่าง Dev Menu ของตัวเครื่องเนทีฟ

• **ข. ถูก:** การเขย่าเครื่องจริง (Shake Device) เป็นกลไกระดับฮาร์ดแวร์เพื่อสั่งเด้งตัวแผงควบคุม Expo Developer Menu ขึ้นมาให้กดเลือกเปิดฟังก์ชันตรวจสอบ

เฉลย ข้อ 2: ตอบ ข.

• เหตุผลประกอบ:

• **ก. ค. และ ง. ผิด:** Logcat เก็บเพียงรายงาน Log ของระบบ ไม่สามารถปัก Breakpoint หรือใช้งานเครื่องมือ JavaScript Console ได้ และแอปพลิเคชันบราวเซอร์ไม่ควบคุมการทำงานของแอปเนทีฟตรง ๆ

• **ข. ถูก:** Google Chrome DevTools (สามารถสั่งเปิดรันได้ผ่านคำสั่งดีบั๊กเกอร์ของ Expo CLI คีย์ลัด `j`) ทำหน้าที่เชื่อมช่องทางเข้าเว็บ JS Engine ในแอป และดึงมาสืบค้นและวาง Breakpoints ดับเบิ้ลตรวจสอบได้อย่างราบรื่น

เฉลย ข้อ 3: ตอบ ก.

• เหตุผลประกอบ:

• **ข. ค. และ ง. ผิด:** คำสะกดอื่นระบุความหมายผิดพลาด

• **ก. ถูก: EAS** ย่อมาจากคำว่า **`Expo Application Services`** ซึ่งเป็นกลุ่มบริการระบบคลาวด์แบบเช่าคลาวด์ของฝั่ง Expo (ประกอบด้วย EAS Build, EAS Submit และ EAS Update)

เฉลย ข้อ 4: ตอบ ข.

• เหตุผลประกอบ:

• **ก. ค. และ ง. ผิด:** ไม่เกี่ยวข้องกับขนาดการดึงรูปภาพ ไม่เขียนคำสั่ง CRUD ให้ และไม่ปิดเครือข่ายเพื่อประหยัดแรม

• **ข. ถูก:** ระบบ EAS Build จะยกภาระงานทั้งหมดไปประมวลผลบนคลาวด์เซิร์ฟเวอร์ของ Expo ทำให้นักพัฒนาที่ใช้คอมพิวเตอร์ Windows หรือ Linux ทั่วไปสามารถกดสั่งเพื่อสร้างแอปพลิเคชันระบบ iOS ออกมาเป็นไฟล์ `.ipa` เพื่อติดตั้งทดสอบได้โดยไม่ต้องซื้อเครื่อง Mac ราคาแพง

เฉลย ข้อ 5: ตอบ ค.

• เหตุผลประกอบ:

• **ก. ผิด:** `eas.json` คุมการตั้งค่าโปรไฟล์และคอมไพล์ของ EAS CLI

• **ข. ผิด:** `package.json` เก็บข้อมูลชื่อแพ็คเกจ Node และสคริปต์รันคำสั่ง

• **ค. ถูก:** **`app.json`** คือไฟล์ที่เก็บข้อมูลและ Metadata ของโปรเจกต์ Expo ทั้งหมดที่จะติดตัวไปกับโครงสร้างไบนารีหลังคอมไพล์เสร็จ

• **ง. ผิด:** babel คุมทรานสไพล์ภาษาจาวาสคริปต์

เฉลย ข้อ 6: ตอบ ค.

• เหตุผลประกอบ:

• ก. ข. และ ง. ผิด: สตริงพิมพ์ใหญ่หรือพาธและไอดีตัวเลขไม่ใช่เกณฑ์ยอมรับในมาตรฐานการระบุแอปของ Google และ Apple

• ค. ถูก: มาตรฐานสากลสำหรับชื่อแอปพลิเคชันเพื่อป้องกันการชนกันของชื่อ (Unique Identifier) คือเขียนในรูป Reverse Domain Name เช่น **com.**ชื่อบริษัท.ชื่อแอป เสมอ

เฉลย ข้อ 7: ตอบ ค.

• เหตุผลประกอบ:

• ก. ข. และ ง. ผิด: โครงสร้าง JSON ของ **eas.json** ต้องมีคีย์ย่อยควบคุมตามแพลตฟอร์มชัดเจน คำสั่งอื่นกำหนดผิดจุดประสงค์

• ค. ถูก: รูปแบบโครงสร้างใน **eas.json** ต้องกำหนดคีย์ภายในโปรไฟล์เป็น **"android": { "buildType": "apk" }** เพื่อบังคับให้เซิร์ฟเวอร์คลาวด์ของ Expo เปลี่ยนประเภทจาก App Bundle กลายมาเป็นไฟล์ติดตั้งด่วนประเภท **.apk**

เฉลย ข้อ 8: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: ไม่เกี่ยวข้องกับ Route, ไม่ใช้ตั้งค่า Permissions บล็อกกล้อง และไม่ใช้เก็บคีย์ลับ Supabase

• ข. ถูก: ไฟล์ **eas.json** ถูกดีไซน์มาเพื่อจัดสร้างและแยกแยะสภาพแวดล้อมในการคอมไพล์แอปพลิเคชันปลายทาง ว่าจะใช้เพื่อทดสอบเขียนในเครื่อง, ส่งออกไฟล์เทสให้เพื่อนร่วมงาน หรือสร้างเพื่อเตรียมโพสต์ขึ้น App Stores

เฉลย ข้อ 9: ตอบ ข.

• เหตุผลประกอบ:

• ก. ค. และ ง. ผิด: ชุดคำสั่งอื่นไม่ช่วยปรับปรุงหรือแก้ไขรุ่นความปลอดภัยของ library อื่นให้ตรงกับ SDK โดยออโต้ และบางตัวใช้สำหรับตรวจสอบความขัดข้องระบบเฉย ๆ

• ข. ถูก: **`npx expo install --fix`** คือคำสั่งพิเศษเฉพาะกิจของ Expo ในการตรวจเช็ค library ทุกตัวในไฟล์ **package.json** แล้วสั่งแก้ไขอัปเดตรุ่นให้สัมพันธ์และเสถียรปลอดภัยเข้าเกณฑ์รองรับของ SDK รุ่นใหม่ที่พึ่งอัปเกรดมาทันที

เฉลย ข้อ 10: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: ไฟล์ **.apk** ใช้สำหรับแจกจ่ายทดสอบภายนอกสโตร์เท่านั้น ปัจจุบันไม่รับอัปโหลดในระดับโปรดักชัน

• **ข. ผิด:** `.exe` เป็นไฟล์โปรแกรม Windows

• **ค. ถูก:** ตั้งแต่ปี 2021 เป็นต้นมา Google Play Console บังคับว่าแอปพลิเคชันใหม่ทั้งหมดที่ส่งขึ้นเป็นโปรดักชันบนสโตร์จะต้องนำส่งข้อมูลในรูปแบบไฟล์ `**Android App Bundle (.aab)**` เท่านั้นเพื่อให้ขนาดแอปพลิเคชันมีขนาดเหมาะสมที่สุดเมื่อถูกโหลดลงโทรศัพท์มือถือปลายทาง

• **ง. ผิด:** `.ipa` เป็นรูปแบบติดตั้งของฝั่ง iOS

ปฏิบัติการท้ายบทเรียน: การดีบั๊กแอปพลิเคชันและการจัดทำไฟล์ติดตั้ง APK ผ่านระบบ EAS Build

ปฏิบัติการในบทเรียนสุดท้ายนี้จะมุ่งเน้นกระบวนการปลายน้ำในการจัดเตรียมความพร้อมของแอปพลิเคชันสู่ระดับโปรดักชัน โดยผู้เรียนจะได้ร่วมกันปรับแต่งไฟล์โครงสร้างคุณลักษณะระบบของแอปพลิเคชัน (`app.json`), กำหนดเส้นทางคอมไพล์คลาวด์เพื่อรับไฟล์ติดตั้งด่วน Android Package (`eas.json`), รันการตรวจสอบและเตรียมปรับปรุงความขัดแย้งของแพ็คเกจสำหรับเตรียมตัวอัปเกรด SDK, ออกแบบหน้าจอแดชบอร์ดวินิจฉัยสำหรับกดจำลองบั๊กหาจุดเกิดเหตุด้วยดีบั๊กเกอร์ (Diagnostics & Debugging Client) และรันชุดคำสั่งถอดรหัสบิลด์ตัวแอปขึ้นไปบนระบบคลาวด์ของ Expo EAS Build

1. วัตถุประสงค์การเรียนรู้

1. สามารถแก้ไขไฟล์ `app.json` เพื่อลงทะเบียนชื่อแพ็คเกจที่เป็นเอกลักษณ์ (Android Package Name) และเวอร์ชันอิงตามหลักเกณฑ์สากลได้ถูกต้อง

2. สามารถจัดสรรและกำหนดค่าโปรไฟล์สำหรับประกอบไฟล์ APK ลงใน `eas.json` ได้ถูกต้อง

3. สามารถเรียกใช้ชุดคำสั่ง `npx expo install --fix` และเคลียร์ Metro cache เพื่อรับมือกับการขยับอัปเกรดรุ่น Expo SDK ได้สำเร็จ

4. สามารถใช้งาน Chrome Developer Tools ในการระบุจุดเกิดเหตุของบั๊กประเภท JavaScript Crash ในคอมโพเนนต์ได้สำเร็จ

5. สามารถใช้งาน EAS CLI ในการรันประมวลผลเซฟและส่งโค้ดขึ้นบิลด์บนคลาวด์จนได้รับลิงก์ดาวน์โหลดหรือรหัส QR Code ไฟล์ติดตั้งจริง

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องจัดสร้างโปรเจกต์ React Native Expo โดยมีโครงสร้างดังนี้:

โครงสร้างไฟล์ปฏิบัติการ:

```
├── app.json                          => ลงทะเบียนชื่อแอป คลังภาพไอคอน และรหัส com.yourdomain.diaryapp
├── eas.json                          => ปรับแต่ง Preview Profile ให้สั่ง Build สรุปประเภทไฟล์เป็น APK
└── app/
    └── index.tsx                     => หน้าแอปวินิจฉัย (Diagnostics Dashboard) พร้อมปุ่มทดสอบสกัดบั๊ก
```

การตั้งค่าการจำลองบั๊กและวิเคราะห์ (Diagnostics Engine):

1. **หน้าจอวินิจฉัย (Diagnostics UI):** แสดงรายงานสเตตัสคุณสมบัติของแอป เช่น เวอร์ชัน และระบบปฏิบัติการ

2. **ปุ่มดีบั๊กจำลองข้อผิดพลาด (Trigger Simulation Crash):** เมื่อผู้ใช้กดคลิก ปุ่มนี้จะต้องรันฟังก์ชันที่จงใจให้เกิด Runtime Crash (เช่น พยายามเข้าถึงคุณสมบัติของอ็อบเจกต์ที่ไม่มีอยู่จริงหรือเป็น `undefined`) เพื่อทดลองเปิด Chrome DevTools ตรวจหาสาเหตุการเกิด Stack Trace

3. **ปุ่มทดสอบความเสถียร (Run Normal API Mock):** ปุ่มสำหรับทดสอบการทำงานปกติเพื่อเปรียบเทียบ

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: การลงทะเบียนติดตั้งคลังคำสั่ง EAS CLI

สั่งรันคำสั่งติดตั้งตัวเชื่อมต่อในระบบส่วนกลาง (Global Node) ของคอมพิวเตอร์ของคุณ:

```
npm install -g eas-cli
```

จากนั้นสั่งล็อกอินเข้าระบบบัญชีผู้พัฒนาของ Expo:

```
eas login
```

ขั้นตอนที่ 2: ตั้งรหัสโดเมนกลับใน `app.json`

ไปที่ไฟล์ `app.json` และระบุรหัส package ของฝั่ง Android (และ bundleIdentifier สำหรับ iOS) ให้ถูกต้อง:

```
"android": {
 "package": "com.student.diaryapp",
 "versionCode": 1
}
```

ขั้นตอนที่ 3: ตั้งค่าโปรไฟล์บิลด์ APK ใน eas.json

รันคำสั่ง `eas build:configure` เพื่อให้ระบบสร้างไฟล์ `eas.json` ขึ้นมา จากนั้นให้เข้าแก้ไขในส่วนโปรไฟล์ `"preview"` ให้ระบุการบิลด์เป็นประเภทไฟล์ APK (เนื่องจากสเปกเริ่มต้นของ EAS จะบิลด์เป็นไฟล์ AAB สำหรับขึ้นสโตร์อย่างเดียว ซึ่งไม่สามารถติดตั้งลงมือถือตรง ๆ ได้):

```
"preview": {
 "distribution": "internal",
 "android": {
  "buildType": "apk"
  }
}
```

ขั้นตอนที่ 4: ฝึกทักษะการตรวจสอบความเข้ากันได้และการเคลียร์แคชกรณีอัปเกรด SDK

ก่อนทำการส่งมอบโค้ดขึ้นระบบคลาวด์ หากมีการขยับรุ่นของ SDK หลัก เราต้องดึงไลบรารีอื่น ๆ ที่มีรุ่นขัดแย้งกันใน `package.json` ให้มาเชื่อมโยงกันอย่างมั่นคงด้วยการใช้คำสั่ง:

```
npx expo install --fix
```

และสั่งเคลียร์แคชของตัว Metro Bundler ที่ค้างสะสมไว้จากการรันแอปข้ามรุ่นด้วยการเรียกใช้:

```
npx expo start -c
```

ขั้นตอนที่ 5: การเปิดเครื่องมือส่องความลับดีบั๊กเกอร์ (Chrome DevTools)

สั่งรันแอปพลิเคชันในโหมดพัฒนาปกติ:

```
npx expo start
```

ขณะที่รันโปรแกรม ให้กดปุ่ม `j` ในหน้าจอคอมพิวเตอร์ หรือเปิดเบราว์เซอร์ Chrome ขึ้นมาแล้วพิมพ์ค้นหา Debugger เพื่อเชื่อมประสาน Console เข้ากับโทรศัพท์มือถือในการปัก Breakpoints

ขั้นตอนที่ 6: สั่งบิลด์แอปพลิเคชันบน EAS Cloud

เปิดเทอร์มินัลใหม่ยิงคำสั่งบิลด์ส่งขึ้นเซิร์ฟเวอร์ Expo เพื่อรับลิงก์ดาวน์โหลด APK:

```
eas build --platform android --profile preview
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถกำหนดค่าไฟล์สำคัญทั้ง 3 ไฟล์ตามซอร์สโค้ดเฉลยฉบับสมบูรณ์ต่อไปนี้:

ไฟล์ที่ 1: รหัสคอนฟิก app.json

```
{
 "expo": {
  "name": "SecureDiary",
  "slug": "secure-diary",
  "version": "1.0.0",
  "orientation": "portrait",
  "icon": "./assets/icon.png",
  "userInterfaceStyle": "dark",
  "splash": {
   "image": "./assets/splash.png",
   "resizeMode": "contain",
   "backgroundColor": "#0f172a"
    },
  "ios": {
   "supportsTablet": true,
   "bundleIdentifier": "com.student.securediary"
    },
  "android": {
   "adaptiveIcon": {
    "foregroundImage": "./assets/adaptive-icon.png",
    "backgroundColor": "#0f172a"
      },
   "package": "com.student.securediary",
   "versionCode": 1
    },
  "web": {
   "favicon": "./assets/favicon.png"
    }
  }
}
```

ไฟล์ที่ 2: รหัสคอนฟิก eas.json

```
{
 "cli": {
  "version": ">= 10.0.0"
  },
```

```
 "build": {
  "development": {
   "developmentClient": true,
   "distribution": "internal"
    },
  "preview": {
   "distribution": "internal",
   "android": {
    "buildType": "apk"
   }
    },
  "production": {
   "distribution": "store"
    }
  },
 "submit": {
  "production": {}
  }
}
```

ไฟล์ที่ 3: โค้ดอินเทอร์เฟซวินิจฉัย app/index.tsx

```
import React, { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TouchableOpacity,
  ScrollView,
  SafeAreaView,
  Platform,
  Alert
} from 'react-native';

interface BugLog {
  id: number;
  time: string;
  type: 'info' | 'error';
  message: string;
}

export default function DiagnosticsDashboard() {
  const [logs, setLogs] = useState<BugLog[]>([]);
  const [triggerCrash, setTriggerCrash] = useState(false);

  const addLog = (type: 'info' | 'error', message: string) => {
    const newLog: BugLog = {
      id: Date.now(),
      time: new Date().toLocaleTimeString(),
```

```
      type,
      message
    };
    setLogs((prev) => [newLog, ...prev]);
  };

  // 1. จำลองการยิงเรียกใช้งาน API ปกติ
  const handleNormalOperation = () => {
    addLog('info', 'เรียกใช้งาน API ดึงข้อมูลตามปกติสำเร็จ');
  };

  // 2. จงใจทริกเกอร์ให้เกิด JavaScript Error เพื่อใช้ทดสอบกับ Chrome DevTools
  const handleTriggerMockError = () => {
    try {
      addLog('info', 'กำลังเตรียมทริกเกอร์ JavaScript Error...');

    // การทำงานที่ผิดพลาด: พยายามเข้าถึงคุณสมบัติจากตัวแปรที่เป็น undefined
      const mockUserData: any = undefined;
      const userProfileName = mockUserData.profile.name; // จุดนี้จะระเบิดและเด้ง
Error ทันที!

      console.log(userProfileName);
    } catch (error: any) {
    // ดักข้อความส่งเข้าดีบั๊กเกอร์แอป
      addLog('error', `ดักจับข้อผิดพลาดสำเร็จ: ${error.message}`);
      console.error('พบข้อบกพร่องในระบบ (ดูรายละเอียดใน Console):', error);
      Alert.alert('ตรวจสอบพบคอขวดข้อผิดพลาด', error.message);
    }
  };

  // 3. จำลองพฤติกรรมการแอปค้างพังเฉียบพลัน (สำหรับจำลอง Native Crash ในดีบั๊กเกอร์)
  const handleSimulateNativeCrash = () => {
    addLog('info', 'สั่งเปิดใช้งานจำลอง Runtime Crash ใน 1 วินาที...');
    setTimeout(() => {
      setTriggerCrash(true); // จะสั่งพ่นตัวแปรคอมโพเนนต์พัง
    }, 1000);
  };

  // เมื่อตัวแปรเป็น true ระบบจะพังการเรนเดอร์ทิ้งจำลอง
  if (triggerCrash) {
    const brokenData: any = null;
    return (
      <View style={styles.center}>
        <Text>{brokenData.nonExistentField.crashApp}</Text>
      </View>
  );
  }
```

```
  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.container}>

        {/* หัวข้อเรื่อง */}
        <View style={styles.header}>
          <Text style={styles.headerTitle}>⚙️ App Diagnostics</Text>
          <Text style={styles.headerSubtitle}>โมดูลวินิจฉัยและจำลองข้อผิดพลาดเพื่อทดสอบดีบั๊กเกอร์</Text>
        </View>

        {/* ข้อมูล Metadata แอป */}
    <View style={styles.metadataCard}>
          <Text style={styles.cardTitle}>📌 ข้อมูลจำเพาะของแอป (App Metadata)</Text>
          <View style={styles.metaRow}>
            <Text style={styles.metaLabel}>ชื่อแอป (App Name):</Text>
            <Text style={styles.metaValue}>SecureDiary</Text>
          </View>
          <View style={styles.metaRow}>
            <Text style={styles.metaLabel}>รุ่นเวอร์ชัน (Version):</Text>
            <Text style={styles.metaValue}>1.0.0 (android/versionCode: 1)</Text>
        </View>
          <View style={styles.metaRow}>
            <Text style={styles.metaLabel}>รหัสแพ็คเกจ (Package):</Text>
            <Text style={styles.metaValue}>com.student.securediary</Text>
          </View>
          <View style={styles.metaRow}>
            <Text style={styles.metaLabel}>แพลตฟอร์มที่รัน (OS):</Text>
            <Text style={styles.metaValue}>{Platform.OS === 'ios' ? 'iOS' : 'Android'}</Text>
          </View>
        </View>

        {/* ปุ่มทริกเกอร์ระบบ */}
        <View style={styles.card}>
          <Text style={styles.cardTitle}>🛠️ แผงควบคุมดีบั๊กเกอร์ (Debug Controls)</Text>
          <Text style={styles.cardDesc}>
      ทดลองเชื่อมต่อ Chrome Developer Tools (คีย์ลัด 'j' ใน CLI) จากนั้นกดปุ่มจำลองด้านล่างเพื่อหัดตรวจวิเคราะห์สาเหตุของบั๊ก
          </Text>

          <TouchableOpacity style={styles.btnNormal} onPress={handleNormalOperation}>
            <Text style={styles.btnText">🟢 ทริกเกอร์ทำงานปกติ (API Success)</Text>
          </TouchableOpacity>
```

```
      <TouchableOpacity style={styles.btnWarn}
onPress={handleTriggerMockError}>
            <Text style={styles.btnText}>🟡 ทริกเกอร์ข้อผิดพลาด (Catchable
Error)</Text>
          </TouchableOpacity>

          <TouchableOpacity style={styles.btnDanger}
onPress={handleSimulateNativeCrash}>
            <Text style={styles.btnText}>🔴 ทริกเกอร์แอปพลิเคชันระเบิด (Fatal Render
Crash)</Text>
          </TouchableOpacity>
        </View>

        {/* รายงาน Log Realtime บนจอภาพ */}
        <View style={styles.logCard}>
          <View style={styles.logHeaderRow}>
            <Text style={styles.cardTitle}>📜 บันทึกสถานะหน้าจอ (On-Screen
Logs)</Text>
            <TouchableOpacity onPress={() => setLogs([])}>
              <Text style={styles.clearLogsText}>ล้างสเตต</Text>
            </TouchableOpacity>
          </View>

          <View style={styles.logScrollWrapper}>
            {logs.length === 0 ? (
              <Text style={styles.emptyLogsText}>ยังไม่มีความเคลื่อนไหวเกิดขึ้นในดีบั๊กเกอร์
ขณะนี้</Text>
      ) : (
              logs.map((log) => (
                <View key={log.id} style={styles.logLine}>
                  <Text style={styles.logTime}>[{log.time}]</Text>
                  <Text style={log.type === 'error' ? styles.logErrorText
: styles.logInfoText}>
                    {log.message}
                  </Text>
                </View>
      ))
      )}
          </View>
        </View>

      </ScrollView>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
```

```
    flex: 1,
    backgroundColor: '#0f172a',
  },
  center: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#7f1d1d',
  },
  container: {
    padding: 20,
  },
  header: {
    alignItems: 'center',
    marginVertical: 12,
  },
  headerTitle: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  headerSubtitle: {
    fontSize: 12,
    color: '#64748b',
    textAlign: 'center',
    marginTop: 6,
    lineHeight: 18,
  },
  metadataCard: {
    backgroundColor: '#1e293b',
    borderRadius: 14,
    padding: 16,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 15,
  },
  cardTitle: {
    fontSize: 14,
    fontWeight: 'bold',
    color: '#ffffff',
    marginBottom: 10,
  },
  cardDesc: {
    color: '#94a3b8',
    fontSize: 11,
    lineHeight: 16,
    marginBottom: 12,
  },
```

```
  metaRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    paddingVertical: 6,
    borderBottomWidth: 1,
    borderBottomColor: '#0f172a',
  },
  metaLabel: {
    color: '#94a3b8',
    fontSize: 12,
  },
  metaValue: {
    color: '#38bdf8',
    fontSize: 12,
    fontWeight: '600',
    fontFamily: 'monospace',
  },
  card: {
    backgroundColor: '#1e293b',
    borderRadius: 14,
    padding: 16,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 15,
  },
  btnNormal: {
    backgroundColor: '#10b981',
    borderRadius: 8,
    height: 40,
    justifyContent: 'center',
    alignItems: 'center',
    marginBottom: 10,
  },
  btnWarn: {
    backgroundColor: '#f59e0b',
    borderRadius: 8,
    height: 40,
    justifyContent: 'center',
    alignItems: 'center',
    marginBottom: 10,
  },
  btnDanger: {
    backgroundColor: '#ef4444',
    borderRadius: 8,
    height: 40,
    justifyContent: 'center',
    alignItems: 'center',
```

```
  },
  btnText: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 13,
  },
  logCard: {
    backgroundColor: '#0f172a',
    borderRadius: 14,
    padding: 16,
    borderWidth: 1,
    borderColor: '#334155',
    minHeight: 180,
  },
  logHeaderRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 10,
  },
  clearLogsText: {
    color: '#38bdf8',
    fontSize: 12,
    fontWeight: '600',
  },
 logScrollWrapper: {
    maxHeight: 160,
  },
  emptyLogsText: {
    color: '#475569',
    fontSize: 12,
    textAlign: 'center',
    marginTop: 40,
  },
  logLine: {
    flexDirection: 'row',
    marginBottom: 6,
    alignItems: 'flex-start',
  },
  logTime: {
    color: '#64748b',
    fontSize: 11,
    fontFamily: 'monospace',
    marginRight: 6,
  },
  logInfoText: {
    color: '#e2e8f0',
    fontSize: 11,
```

```
    flex: 1,
  },
  logErrorText: {
    color: '#f87171',
    fontSize: 11,
    fontWeight: '600',
    flex: 1,
  },
});
```

เอกสารอ้างอิง

Expo. (2024). *Debugging*. จาก https://docs.expo.dev/versions/v54.0.0/debugging/tools/

Expo. (2024). *EAS Build Overview*. จาก https://docs.expo.dev/versions/v54.0.0/eas-build/introduction/

Expo. (2024). *EAS CLI*. จาก https://docs.expo.dev/versions/v54.0.0/eas/

React Native. (2024). *Debugging*. จาก https://reactnative.dev/docs/debugging

บรรณานุกรม

บรรณานุกรม

Expo. (2024). *Camera*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/camera/

Expo. (2024). *Create an app*. จาก https://docs.expo.dev/versions/v54.0.0/get-started/create-a-project/

Expo. (2024). *Create Routes with Expo Router*. จาก https://docs.expo.dev/versions/v54.0.0/router/create-pages/

Expo. (2024). *Debugging*. จาก https://docs.expo.dev/versions/v54.0.0/debugging/tools/

Expo. (2024). *EAS Build Overview*. จาก https://docs.expo.dev/versions/v54.0.0/eas-build/introduction/

Expo. (2024). *EAS CLI*. จาก https://docs.expo.dev/versions/v54.0.0/eas/

Expo. (2024). *Expo Image*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/image/

Expo. (2024). *Expo Router Introduction*. จาก https://docs.expo.dev/versions/v54.0.0/router/introduction/

Expo. (2024). *FileSystem*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/filesystem/

Expo. (2024). *Icons*. จาก https://docs.expo.dev/versions/v54.0.0/guides/icons/

Expo. (2024). *ImagePicker*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/imagepicker/

Expo. (2024). *Layouts with Expo Router*. จาก https://docs.expo.dev/versions/v54.0.0/router/layouts/

Expo. (2024). *Location*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/location/

Expo. (2024). *SecureStore*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/securestore/

Expo. (2024). *Sensors*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/sensors/

Expo. (2024). *Use custom fonts*. จาก https://docs.expo.dev/versions/v54.0.0/develop/user-interface/fonts/

Expo. (2024). *User Interface and Styling*. จาก https://docs.expo.dev/versions/v54.0.0/develop/user-interface/overview/

Expo. (2024). *Using SQLite*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/sqlite/

Git. (2024). *Git Reference Manual*. จาก https://git-scm.com/doc

JSONPlaceholder. (2024). *JSONPlaceholder - Free Fake REST API*. จาก https://jsonplaceholder.typicode.com/

MDN Web Docs. (2024). *Using the Fetch API*. จาก https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Microsoft. (2024). *Visual Studio Code Documentation*. จาก https://code.visualstudio.com/docs

Node.js. (2024). *Node.js Downloads*. จาก https://nodejs.org/

React Native Async Storage. (2024). *Async Storage Documentation*. จาก https://react-native-async-storage.github.io/async-storage/docs/install/

React Native Maps. (2024). *React Native Maps Installation*. จาก https://github.com/react-native-maps/react-native-maps

React Native. (2024). *Core Components and APIs*. จาก https://reactnative.dev/docs/intro-react-native-components

React Native. (2024). *Debugging*. จาก https://reactnative.dev/docs/debugging

React Native. (2024). *Environment Setup*. จาก https://reactnative.dev/docs/environment-setup

React Native. (2024). *FlatList API Reference*. จาก https://reactnative.dev/docs/flatlist

React Native. (2024). *Handling Text Input*. จาก https://reactnative.dev/docs/handling-text-input

React Native. (2024). *Height and Width*. จาก https://reactnative.dev/docs/height-and-width

React Native. (2024). *Image*. จาก https://reactnative.dev/docs/image

React Native. (2024). *Layout with Flexbox*. จาก https://reactnative.dev/docs/flexbox

React Native. (2024). *Pressable*. จาก https://reactnative.dev/docs/pressable

React Native. (2024). *SectionList API Reference*. จาก https://reactnative.dev/docs/sectionlist

React Native. (2024). *Style*. จาก https://reactnative.dev/docs/style

React Native. (2024). *Text*. จาก https://reactnative.dev/docs/text

React Native. (2024). *TextInput*. จาก https://reactnative.dev/docs/textinput

React Native. (2024). *View*. จาก https://reactnative.dev/docs/view

React. (2024). *State: A Component's Memory*. จาก https://react.dev/learn/state-a-components-memory

React. (2024). *Synchronizing with Effects*. จาก https://react.dev/learn/synchronizing-with-effects

Supabase. (2024). *React Native Supabase Quickstart*. จาก https://supabase.com/docs/guides/getting-started/tutorials/with-react-native

Supabase. (2024). *Supabase Auth with React Native*. จาก https://supabase.com/docs/guides/auth/auth-helpers/react-native

Supabase. (2024). *Supabase Database Quickstart*. จาก https://supabase.com/docs/guides/database/overview

Supabase. (2024). *Supabase JavaScript Auth API Reference*. จาก https://supabase.com/docs/reference/javascript/auth-signup

Supabase. (2024). *Supabase JavaScript Client Database API*. จาก https://supabase.com/docs/reference/javascript/select

Supabase. (2024). *User Management with Supabase*. จาก https://supabase.com/docs/guides/auth

ภาคผนวก

ภาคผนวก

ภาคผนวก ก: สรุปคำสั่งสำคัญของ Expo CLI และ EAS CLI (Expo SDK 54)

ในการพัฒนาแอปพลิเคชันด้วย React Native และ Expo SDK 54 ผู้เรียนสามารถใช้ชุดคำสั่งมาตรฐานผ่าน Terminal หรือ PowerShell สำหรับการสร้างโปรเจค การรันเซิร์ฟเวอร์นักพัฒนา การติดตั้งไลบรารี และการคอมไพล์แอปพลิเคชันขึ้นระบบคลาวด์ ดังต่อไปนี้

1. คำสั่งจัดการโปรเจคและการรันเซิร์ฟเวอร์นักพัฒนา (Expo CLI)

• `npx create-expo-app@latest <project-name>` : สร้างโปรเจค React Native Expo SDK 54 ใหม่จากเทมเพลตมาตรฐาน

• `npm run reset-project` : ล้างหน้าจอตัวอย่างเริ่มต้นและสร้างโฟลเดอร์ `app/` ใหม่ที่ว่างเปล่าสำหรับเริ่มต้นพัฒนาแอปพลิเคชัน

• `npx expo start` : เปิดเซิร์ฟเวอร์ Metro Bundler และแสดง QR Code สำหรับทดสอบผ่านแอปพลิเคชัน Expo Go

• `npx expo start --clear` : เปิดเซิร์ฟเวอร์นักพัฒนาพร้อมล้างหน่วยความจำแคช (Metro Cache) กรณีเกิดปัญหาการโหลดโมดูลไม่สมบูรณ์

• `npx expo install <package-name>` : ติดตั้งไลบรารีโดยตรวจสอบและเลือกเวอร์ชันที่เข้ากันได้กับ Expo SDK 54 โดยอัตโนมัติ

• `npx expo-doctor` : ตรวจสอบสุขภาพของโปรเจค ความเข้ากันได้ของไลบรารี และความถูกต้องของไฟล์ตั้งค่า `app.json`

2. คำสั่งคอมไพล์และเผยแพร่แอปพลิเคชัน (EAS CLI)

• `npm install -g eas-cli` : ติดตั้งเครื่องมือ EAS CLI แบบ Global ลงในเครื่องคอมพิวเตอร์

• `eas login` : เข้าสู่ระบบบัญชีผู้ใช้ Expo เพื่อเชื่อมต่อกับบริการคลาวด์ EAS

• `eas build:configure` : สร้างและตั้งค่าไฟล์ `eas.json` สำหรับกำหนดโปรไฟล์การคอมไพล์ (`development`, `preview`, `production`)

• `eas build --platform android --profile preview` : สั่งคอมไพล์ไฟล์ติดตั้ง `.apk` สำหรับทดสอบบนอุปกรณ์จริง Android

• `eas build --platform all --profile production` : สั่งคอมไพล์ไฟล์สำหรับเผยแพร่จริง (`.aab` สำหรับ Google Play และ `.ipa` สำหรับ App Store)

• `eas submit --platform all` : ส่งไฟล์แอปพลิเคชันที่คอมไพล์เสร็จแล้วขึ้นสู่ Google Play Console และ Apple App Store Connect

ภาคผนวก ข: แนวทางการแก้ไขปัญหาที่พบบ่อย (Troubleshooting Guide)

รวบรวมสาเหตุและแนวทางการแก้ไขข้อผิดพลาดที่ผู้เรียนมักพบบ่อยในระหว่างการฝึกปฏิบัติการพัฒนาแอปพลิเคชันมือถือ

1. สมาร์ทโฟนสแกน QR Code แล้วไม่สามารถเชื่อมต่อกับ Metro Bundler ได้

สาเหตุ: เครื่องคอมพิวเตอร์และสมาร์ทโฟนเชื่อมต่อเครือข่าย Wi-Fi คนละวงกัน หรือระบบ Firewall ของคอมพิวเตอร์บล็อกพอร์ต 8081 **วิธีแก้ไข:** ตรวจสอบให้ทั้งสองอุปกรณ์เชื่อมต่อ Wi-Fi วงเดียวกัน หรือรันคำสั่ง `npx expo start --tunnel` เพื่อสร้างการเชื่อมต่อผ่านเครือข่าย Tunnel

2. ข้อความแจ้งเตือน "Text strings must be rendered within a <Text> component"

สาเหตุ: ใน React Native ข้อความทุกชนิดไม่สามารถวางไว้ภายใต้ `<View>` ได้โดยตรงเหมือนภาษา HTML บนเว็บ **วิธีแก้ไข:** ตรวจสอบโค้ด JSX และครอบข้อความหรือตัวแปรที่แสดงผลด้วยคอมโพเนนต์ `<Text>` เสมอ รวมถึงระวังการเว้นช่องว่างหรือคอมเมนต์ที่อยู่นอกแท็ก

3. รายการ FlatList ไม่แสดงผลหรือมีความสูงเป็นศูนย์เมื่อวางซ้อนในหน้าจอ

สาเหตุ: คอนเทนเนอร์แม่ (`<View>` หรือ `<SafeAreaView>`) ไม่ได้กำหนดขนาดความยืดหยุ่นของพื้นที่หน้าจอ **วิธีแก้ไข:** กำหนดสไตล์ `flex: 1` ให้กับคอนเทนเนอร์หลักของหน้าจอเสมอ และหลีกเลี่ยงการนำ `<FlatList>` ไปวางซ้อนภายใน `<ScrollView>` ที่มีทิศทางการเลื่อนแนวเดียวกัน

4. การเรียกข้อมูลจาก Supabase คืนค่าว่างเปล่า (Empty Array `[]`) ทั้งที่มีข้อมูลในตาราง

สาเหตุ: ตารางในฐานข้อมูล PostgreSQL บน Supabase เปิดใช้งาน Row Level Security (RLS) อยู่ แต่ยังไม่ได้สร้างนโยบาย (Policy) อนุญาตให้อ่านข้อมูล **วิธีแก้ไข:** เข้าไปที่เมนู Authentication > Policies บนแดชบอร์ด Supabase แล้วเพิ่ม Policy สำหรับคำสั่ง **SELECT** ให้ตรงกับระดับสิทธิ์ที่ต้องการ